

BITCOIN

V0.01 ALPHA

ANNOTATED EDITION

Annotations by Claude Opus 4.5

Copyright (c) 2009 Satoshi Nakamoto

To all bitcoiners

ACKNOWLEDGMENTS

This book was made possible by the following sources and resources:

ORIGINAL BITCOIN SOURCE CODE

satoshi.nakamotoinstitute.org/code

Archive of Bitcoin v0.01 source code, preserved by the Satoshi Nakamoto Institute

BITCOIN WHITEPAPER

bitcoin.org/bitcoin.pdf

Satoshi Nakamoto's original paper describing the Bitcoin protocol

BITCOIN WIKI

en.bitcoin.it

Community-maintained documentation on Bitcoin internals

LEARN ME A BITCOIN

learnmeabitcoin.com

Greg Walker's comprehensive Bitcoin technical guides

BITCOIN STACK EXCHANGE

bitcoin.stackexchange.com

Q&A archive of Bitcoin technical discussions

MASTERING BITCOIN

github.com/bitcoinbook/bitcoinbook

Andreas Antonopoulos's open source Bitcoin reference

THE GENESIS BOOK

thegenesisbook.com

Aaron van Wirdum's history of the people and projects that inspired Bitcoin

PROGRAMMING BITCOIN

github.com/jimmy-song/programming-bitcoin

Jimmy Song's hands-on guide to Bitcoin internals (O'Reilly)

Special thanks to Satoshi Nakamoto for creating Bitcoin and releasing the source code to the world.

FUNDAMENTALS

Introduction	15
The Road to Bitcoin	19
Computer Concepts	25
Cryptography Basics	29
C++ Primer	39

THE SOURCE CODE

rc/ (Resources)	53
base58.h	55
bignum.h	63
db.cpp	75
db.h	87
headers.h	99
irc.cpp	103
irc.h	109
key.h	111
license.txt	117
main.cpp	119
main.h	165
makefile	189
makefile.vc	193
market.cpp	197
market.h	203
net.cpp	207
net.h	225
readme.txt	243
script.cpp	247
script.h	265

serialize.h	277
sha.cpp	299
sha.h	309
ui.cpp	315
ui.h	363
ui.rc	371
uibase.cpp	373
uibase.h	407
uint256.h	419
uiproject.fbp	437
util.cpp	497
util.h	507

REFERENCE

What Came After	515
Index	521
Colophon	523

FUNDAMENTALS

The history, cryptography, and computer
science concepts behind Bitcoin

INTRODUCTION

For decades before Bitcoin, cryptographers and computer scientists attempted to create digital money. They all failed. DigiCash went bankrupt. e-gold was shut down by the federal government. Liberty Reserve was seized. Each system had the same fatal flaw: a central point of control that could be pressured, raided, or simply go out of business.

Then, on October 31, 2008, an unknown person using the name Satoshi Nakamoto posted a paper to a cryptography mailing list. The paper described "a purely peer-to-peer version of electronic cash" that required no trusted third party. Two months later, on January 3, 2009, Nakamoto launched the software.

Embedded in the first block was a message: "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks." This headline from a British newspaper served two purposes. First, it proved the block could not have been created before that date. Second, it signaled the motivation behind the project—a response to a financial system that had just brought the global economy to its knees.

This book contains the complete source code of Bitcoin version 0.01, the first public release. These 15,000 lines of C++ represent the solution to a problem that had defeated every previous attempt: how to prevent double-spending in a digital currency without relying on a central authority.

WHY THIS CODE MATTERS

Previous digital currencies required you to trust a company or organization. You had to trust that they wouldn't create money out of thin air, that they wouldn't freeze your account, that they wouldn't disappear with your funds. When that trust was violated—or when governments decided to shut them down—the money vanished.

Bitcoin eliminated the need for trust by replacing it with verification. Every node on the network independently validates every transaction against the same rules encoded in this source code. No single entity controls the network. No company can be shut down to stop it. The rules are enforced by mathematics and the collective agreement of thousands of participants worldwide.

Satoshi Nakamoto's identity remains unknown. But the code is public, and it speaks for itself.

HOW BITCOIN WORKS

The system operates through a few interlocking mechanisms:

- **Transactions** transfer value from one address to another, authorized by digital signatures that prove ownership without revealing private keys.
- **Blocks** bundle transactions together, each block referencing the hash of the previous block, forming a chain.
- **Proof-of-work** requires miners to expend computational resources to add blocks, making it economically impractical to rewrite history.
- **P2P network** connects nodes directly without central servers. Transactions and blocks propagate through gossip—each node relays what it receives to its peers, reaching the entire network in seconds.
- **Consensus** emerges as nodes accept the longest valid chain, creating agreement without coordination.

The key files that implement this system are:

- **main.cpp** — Block validation, mining, and consensus rules
- **script.cpp** — The scripting language for spending conditions
- **net.cpp** — Peer-to-peer networking and message propagation
- **key.h** — Elliptic curve cryptography for digital signatures
- **sha.cpp** — The SHA-256 hash function securing the chain

READING THE CODE

You do not need to be a programmer to read this book. Each file begins with an introduction explaining its purpose, and annotations throughout highlight significant passages. The syntax highlighting makes the structure visible:

ELEMENT	EXAMPLE
Keywords	<code>if</code> <code>int</code> <code>return</code> <code>class</code>
Functions	<code>GetHash()</code> <code>printf()</code>

Numbers	42 0x1F 3.14
Comments	// Satoshi's notes
Operators	+ == && ->

Pay particular attention to the comments—the gray italic text beginning with `//`. These are Satoshi's own explanations, written as the code was developed. They reveal the reasoning behind design decisions that are not always obvious from the code alone.

WHAT YOU WILL FIND

This book is organized to build understanding progressively:

- **The Road to Bitcoin** traces the ideas and failed projects that preceded Bitcoin, from public key cryptography to the cypherpunk movement.
- **Computer Concepts** and **Cryptography Basics** provide the technical foundation needed to understand the code.
- **C++ Primer** explains the programming language and file structure of the project.
- **The Source Code** presents every file from Bitcoin v0.01, annotated with explanations of what the code does and why it matters.

Version 0.01 is remarkably complete. The fundamental architecture—UTXO model, proof-of-work consensus, scripting system, peer-to-peer networking—remains essentially unchanged after fifteen years and trillions of dollars in value secured. Understanding this code means understanding Bitcoin at its foundation.

THE ROAD TO BITCOIN

Bitcoin didn't emerge from a vacuum. It stands on the shoulders of decades of cryptographic research and failed attempts at digital cash. Understanding these predecessors illuminates what made Satoshi's synthesis revolutionary.

THE CYPHERPUNK DREAM

In the early 1990s, a group of cryptographers, hackers, and privacy advocates formed a mailing list dedicated to using cryptography to create a free society. They called themselves the **cypherpunks**. Their manifesto declared: "Privacy is necessary for an open society in the electronic age."

Among their goals: anonymous digital cash—money that couldn't be traced, seized, or censored. For two decades, brilliant minds tried and failed to create this. Their failures became Bitcoin's foundation.

PUBLIC KEY CRYPTOGRAPHY (1976)

In 1976, Whitfield Diffie and Martin Hellman published "New Directions in Cryptography," introducing the concept of **public key cryptography**. Before this, sharing encrypted messages required both parties to have the same secret key—but how do you securely share that key in the first place?

Diffie-Hellman solved this with an elegant mathematical trick: each party has a private key they never share and a public key they broadcast to the world. The math allows two parties to compute a shared secret without ever transmitting it:

Public: Generator point G

Alice (private a):	Bob (private b):
Computes $A = aG$	Computes $B = bG$
Sends A to Bob	Sends B to Alice
Computes $aB = abG$	Computes $bA = baG$

Shared secret: abG (same for both!)

An eavesdropper sees G , A , and B , but cannot compute abG without knowing either a or b . This insight—that you can publish part of your key without compromising your secrets—is the foundation of all cryptocurrency.

The Bitcoin whitepaper cites Diffie-Hellman indirectly through the cryptographic concepts it enabled. Your Bitcoin public key can be shared with the world, yet no one can reverse-engineer your private key. The math that protects Diffie-Hellman key exchange protects your coins.

DAVID CHAUM'S DIGICASH (1989)

Cryptographer David Chaum invented **blind signatures** and founded DigiCash in 1989—the first attempt at cryptographic digital cash. His "eCash" system used clever math to make transactions untraceable while preventing double-spending.

The problem: DigiCash required a central server to validate transactions. When the company went bankrupt in 1998, the money system died with it. **Lesson learned: centralization is a single point of failure.**

HASHCASH (1997)

Adam Back, a British cryptographer, invented **Hashcash** to combat email spam. The idea: require senders to compute a partial hash collision before sending email. Finding such a collision takes CPU work—trivial for one email, prohibitive for millions of spam messages.

```
Hashcash header: 1:20:060408:adam@cypherspace.org::1QTjaYd7niiQA/sc:ePa
```

```
The sender must find a value where SHA1(header) starts with 20 zero bits.  
This requires ~2^20 hash computations—about a second of CPU time.
```

The Bitcoin whitepaper explicitly cites Hashcash in its references. Satoshi adapted the concept: instead of proving work to send email, miners prove work to add blocks. The difficulty is adjusted so the network collectively finds one valid proof every 10 minutes.

B-MONEY (1998)

Wei Dai, a cypherpunk and computer scientist, proposed **b-money**— a theoretical system for anonymous digital cash. His paper describes:

- Money created through proof-of-work
- Transactions broadcast to all participants
- A collective ledger maintained by the network
- Contracts enforced through cryptographic protocols

Sound familiar? Wei Dai's b-money was never implemented, but Satoshi cited it first in the Bitcoin whitepaper's references. In early emails, Satoshi told Wei Dai: "I was very interested to read your b-money page. I'm getting ready to release a paper that expands on your ideas."

BIT GOLD (1998–2005)

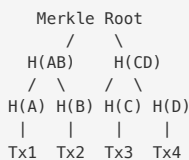
Nick Szabo was the son of a Hungarian refugee who fled after Soviet troops crushed the 1956 revolt. This shaped his distrust of government overreach and drew him to Hayek's writings on monetary economics. After studying computer science and working at NASA's Jet Propulsion Laboratory, he joined the cypherpunks and briefly worked at DigiCash in Amsterdam.

At DigiCash, Szabo saw firsthand how easy it was to manipulate balances in a centralized system. This led to his influential essay "Trusted Third Parties Are Security Holes"—the insight that any system depending on a central party inherits that party's vulnerabilities, whether from hackers, rogue employees, or government pressure.

Szabo designed **Bit Gold** to eliminate trusted third parties entirely. His system used proof-of-work to create tokens, chained hashes together (each valid hash becoming the input for the next), and proposed a distributed "property club" to track ownership. It was never implemented, but its architecture clearly influenced Bitcoin. Szabo later acknowledged: "Bitcoin is an implementation of bit gold."

MERKLE TREES (1979)

Ralph Merkle invented the **Merkle tree**—a data structure that efficiently summarizes large amounts of data into a single hash. His original patent was for digital signatures, but the concept proved far more versatile.



Bitcoin uses Merkle trees to commit to all transactions in a block while enabling "light clients" that can verify transaction inclusion without downloading the entire blockchain. The whitepaper's Section 7 explicitly credits "Protocols for Public Key Cryptosystems" (Merkle, 1980).

TIMESTAMPING (1991)

Stuart Haber and W. Scott Stornetta published "How to Time-Stamp a Digital Document"—a system for proving a document existed at a certain time by publishing hashes in a newspaper. Their subsequent work on hash chains and Merkle trees directly influenced Bitcoin's blockchain structure.

The Bitcoin whitepaper cites three Haber-Stornetta papers. Their idea of chaining hashes together—where each timestamp includes the hash of the previous one—is exactly how Bitcoin blocks link together.

RPOW: REUSABLE PROOF OF WORK (2004)

Hal Finney graduated top of his class from Caltech and spent years developing video games for Intellivision and Atari before becoming a core contributor to PGP. Unlike some cypherpunks, Finney was a pragmatist. He didn't believe cryptography could create an anarchist utopia: "There is no such place as cyberspace. I am in California. Its agents carry physical guns which shoot real bullets."

Still, Finney wanted electronic cash to exist. In 2004, he launched **RPOW** (Reusable Proofs of Work). Users would generate a hashcash-style proof-of-work and exchange it for a token. That token could then be transferred to others, who would redeem it for a fresh token—making the proof of work reusable.

To prevent double-spending without trusting the server operator (himself), Finney ran RPOW on tamper-proof IBM hardware that could cryptographically prove it was running unmodified open-source code. It was clever, but RPOW failed to gain users. The tokens had no reason to hold value—Moore's Law would make them cheaper to produce over time—and without users, there was nothing to buy.

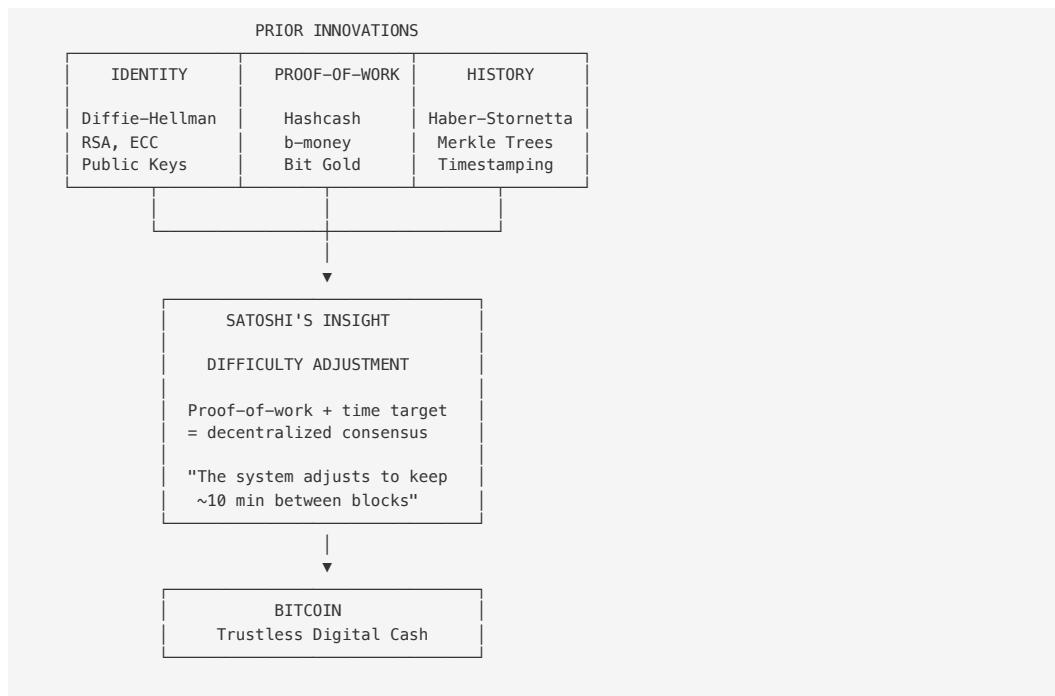
When Satoshi announced Bitcoin on the cryptography mailing list in 2008, Hal Finney was among the first to respond positively. On January 11, 2009, Finney received the first Bitcoin transaction ever: 10 BTC from Satoshi. Finney ran one of the first Bitcoin nodes and contributed code fixes before his death in 2014.

SATOSHI'S SYNTHESIS

Every predecessor solved part of the puzzle:

INNOVATION	CONTRIBUTION
Diffie-Hellman / RSA	Cryptographic keys for identity
DigiCash	Blind signatures (but centralized)
Hashcash	Proof-of-work as rate limiting
b-money	Decentralized ledger concept
Bit Gold	Chained proof-of-work tokens
Merkle trees	Efficient data commitment
Haber-Stornetta	Hash-linked timestamping
RPOW	Transferable proof-of-work

Satoshi's genius was combining these pieces with a novel insight:



Use proof-of-work not just to create money, but to reach **consensus** about which transactions are valid. The longest chain (most cumulative work) is the true history. No trusted party needed—just math and economic incentives.

As Satoshi wrote: "A lot of people automatically dismiss e-currency as a lost cause because of all the companies that failed since the 1990s. I hope it's obvious it was only the centrally controlled nature of those systems that doomed them."

COMPUTER CONCEPTS

Before diving into the code, let's cover some fundamental concepts that appear throughout Bitcoin's implementation.

BITS AND BYTES

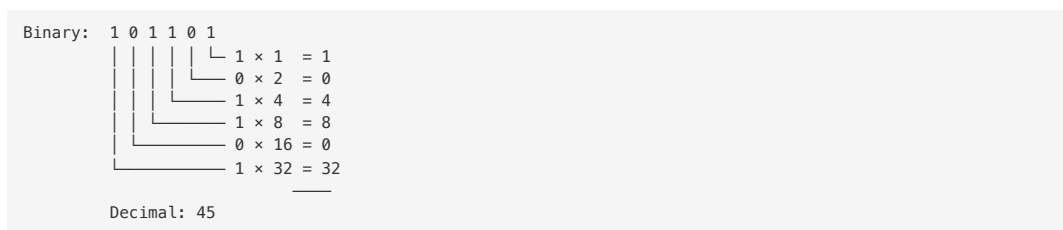
A **bit** is the smallest unit of data—a single 0 or 1. Eight bits make a **byte**, which can represent values from 0 to 255. All computer data, from text to images to Bitcoin transactions, is ultimately stored as bytes.

Common sizes you'll see in Bitcoin:

SIZE	USAGE
32 bits (4 bytes)	Standard integers, timestamps
64 bits (8 bytes)	Large numbers, Bitcoin amounts (in satoshis)
160 bits (20 bytes)	Bitcoin addresses (RIPEMD-160 hash)
256 bits (32 bytes)	SHA-256 hashes, private keys

BINARY (BASE-2)

Computers think in **binary**—base-2, using only 0 and 1. Each position represents a power of 2:



HEXADECIMAL (BASE-16)

Binary is verbose, so programmers use **hexadecimal** (hex)—base-16. It uses digits 0-9 and letters A-F (where A=10, B=11, ... F=15). Each hex digit represents exactly 4 bits:

```
Hex:  0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
Dec:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

Bitcoin hashes are typically shown in hex. A 256-bit hash becomes 64 hex characters:

```
000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
```

This is the hash of Bitcoin's genesis block—the first block ever mined.

LITTLE ENDIAN VS BIG ENDIAN

When storing multi-byte numbers, computers must decide which byte comes first. **Big endian** stores the most significant byte first (like how we write numbers). **Little endian** stores the least significant byte first.

```
Number: 0x12345678 (305,419,896 in decimal)

Big Endian:  [12] [34] [56] [78] (most significant first)
Little Endian: [78] [56] [34] [12] (least significant first)
```

Bitcoin uses little endian for most internal data but big endian for displaying hashes. This can be confusing—the genesis block hash shown above is actually stored in reverse order internally.

MEMORY AND POINTERS

Computer memory is like a vast array of numbered boxes, each holding one byte. The box number is called an **address**. A **pointer** is a variable that stores an address—it "points to" data elsewhere in memory.

In C++ code, you'll see:

- `*ptr` — The value that ptr points to
- `&var` — The address of var
- `ptr->field` — Access a field through a pointer

DATA TYPES

Bitcoin uses several C++ data types:

- `int` — Integer (usually 32 bits)
- `int64` — 64-bit integer (for Bitcoin amounts)
- `unsigned` — Non-negative integers only
- `char` — Single byte, often used for raw data
- `string` — Text or arbitrary byte sequences
- `vector` — Resizable array
- `map` — Key-value dictionary
- `bool` — True or false

CRYPTOGRAPHIC HASHING

A **hash function** takes any input and produces a fixed-size output (the "hash" or "digest"). Bitcoin primarily uses SHA-256, which always produces 256 bits.

Key properties:

- **Deterministic** — Same input always gives same output
- **One-way** — Cannot reverse a hash to find the input
- **Avalanche effect** — Tiny input change completely changes output
- **Collision resistant** — Practically impossible to find two inputs with same hash

```
SHA256("Bitcoin") =
  b4056df6691f8dc72e56302ddad345d65fead3ead9299609a826e2344eb63aa4

SHA256("bitcoin") = (just lowercase 'b')
  6b88c087247aa2f07ee1c5956b8e1a9f4c7f892a70e324f1bb3d161e05ca107b
```

PUBLIC KEY CRYPTOGRAPHY

Bitcoin uses **elliptic curve cryptography** for digital signatures. You have two related keys:

- **Private key** — A secret 256-bit number. Never share this!
- **Public key** — Derived from the private key. Safe to share.

The private key can sign messages; anyone with the public key can verify the signature is authentic. This is how Bitcoin proves ownership—only the holder of the private key can sign transactions spending their coins.

READING C++ SYNTAX

Some patterns you'll encounter:

- `// comment` — Single-line comment (explanatory text)
- `/* comment */` — Multi-line comment
- `if (condition) { ... }` — Conditional execution
- `for (init; condition; increment) { ... }` — Loop
- `class Name { ... };` — Define a data structure
- `return value;` — Exit function with result
- `#include` — Import another file
- `#define` — Create a constant or macro

CRYPTOGRAPHY BASICS

Bitcoin's security rests on mathematical foundations developed over decades of cryptographic research. Understanding these concepts reveals why Bitcoin works and why it's secure.

TWO PILLARS OF BITCOIN CRYPTOGRAPHY

	ECDSA	SHA-256
Purpose	Proves ownership	Creates fingerprints
Used for	Signing transactions	Mining, block hashes, Merkle trees
Property	Asymmetric (public/private keys)	One-way (irreversible)

ECDSA lets you prove you own coins without revealing your private key. **SHA-256** creates a unique 256-bit fingerprint of any data—change one bit and the hash changes completely.

THE DISCRETE LOGARITHM PROBLEM

Many cryptographic systems rely on operations that are easy in one direction but practically impossible to reverse. Consider this:

Easy:	$3^{15} \bmod 17 = 6$	(compute in milliseconds)
Hard:	$3^? \bmod 17 = 6$	(which exponent gives 6?)

For small numbers, you can try all possibilities. But with 256-bit numbers, there are more possibilities than atoms in the universe. This asymmetry—easy forward, impossible backward—is the foundation of public key cryptography.

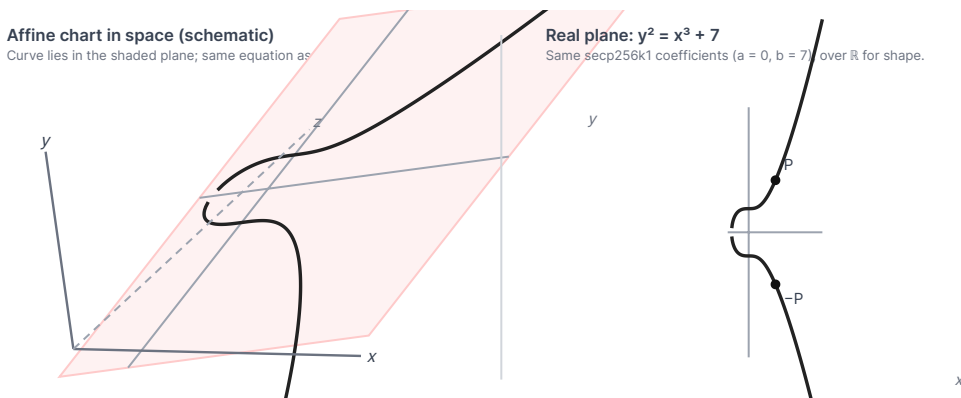
ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

Bitcoin uses a specific type of discrete logarithm problem based on **elliptic curves**. An elliptic curve is defined by an equation like:

$$y^2 = x^3 + ax + b$$

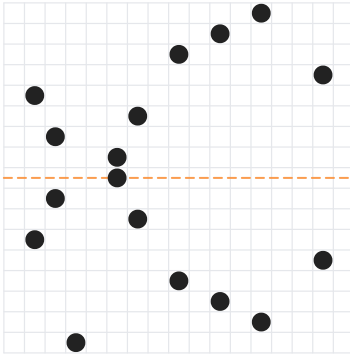
Bitcoin (secp256k1): $y^2 = x^3 + 7$ ($a=0$, $b=7$)

The curve has a distinctive shape—symmetric about the x-axis, with a "bulge" on the left and extending to infinity on the right. Textbooks often show the same Weierstrass equation as the intersection of an algebraic surface with a plane—your mental model can be “the locus lives in a chart embedded in something higher-dimensional,” then projected to the page. For embedded-systems treatments of how modular arithmetic changes the geometry, see for example [How Elliptic Curve Cryptography Works](#) (All About Circuits, 2019).



In real cryptography the coordinates are not arbitrary-precision reals: they live in a **finite field** $\mathbb{F}_p$ (for secp256k1, a huge prime p). Addition still follows chord-and-tangent, but values wrap modulo p . A tiny field makes the pattern visible—each valid (x, y) pair below satisfies $y^2 \equiv x^3 + 7 \pmod{17}$:

F_{17} (schematic grid)



Modular picture (toy field)

Prime $p = 17$. Integer pairs (x, y) with $y^2 \equiv x^3 + 7 \pmod{p}$.

Negation $(x, y) \mapsto (x, -y)$ is reflection across $y = 17/2$ (orange). With modulus, edges identify like a torus—unlike $\mathbb{R}^2$.

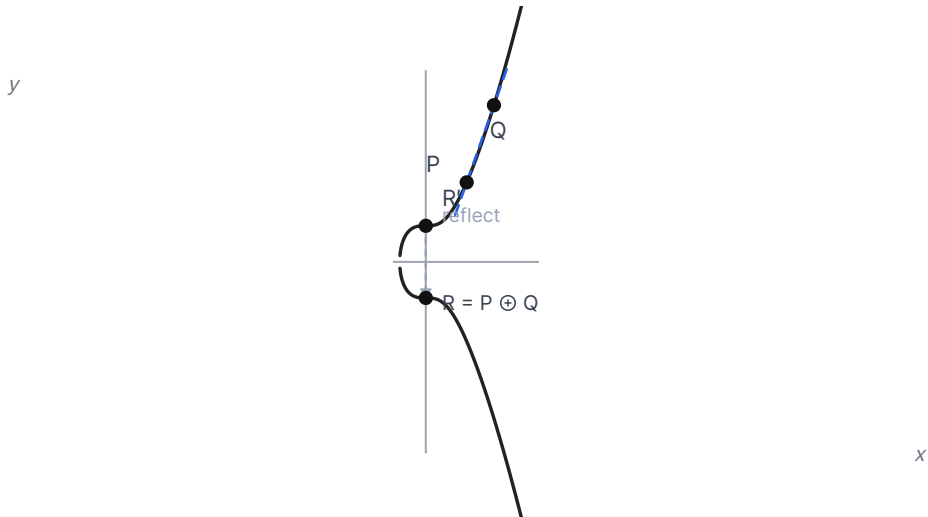
Not secp256k1's field—only illustrates wrapping & symmetry.

For any point $P = (x, y)$ on the curve, there's a mirror point $-P = (x, -y)$. This symmetry is key to the mathematics.

Points on this curve can be "added" together using a geometric rule. To add points P and Q :

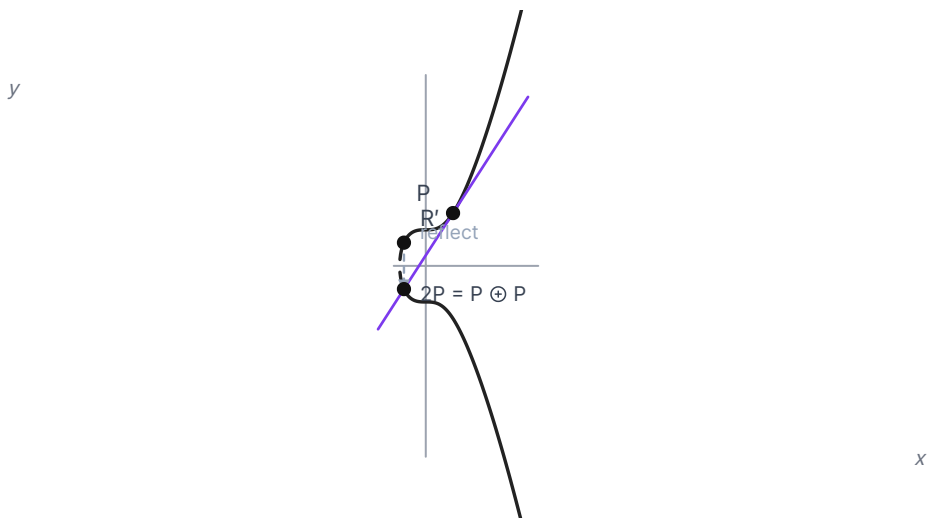
1. Draw a line through P and Q
2. Find where the line intersects the curve (point R')
3. Reflect R' across the x -axis to get $R = P + Q$

Point addition ($P \oplus Q$): the dashed secant meets the curve at P , Q , and a third point R' ; reflecting R' across the x -axis gives $R = P \oplus Q$.



To **double** a point (add it to itself), draw the **tangent line** at P , find where it intersects, and reflect:

Point doubling: $P \oplus P = 2P$. The tangent at P meets the curve again at R' ; reflect across the x -axis to get $2P$.



Using these operations, you can compute kG (adding G to itself k times):

```
G + G = 2G
2G + G = 3G
...
kG = P    (add G to itself k times)
```


Given G and P , finding k is the **elliptic curve discrete logarithm problem**—believed to be computationally infeasible for large k .

Bitcoin uses the **secp256k1** curve, where k is your private key and P is your public key. Anyone can verify that $P = kG$, but nobody can determine k from P alone.

THE GENERATOR POINT G

The generator point G is a specific point on the secp256k1 curve that serves as the starting point for all key derivations. It's a publicly known constant—everyone uses the same G . The secp256k1 generator point coordinates are:

```
G.x = 0x79BE667EF9DCBBAC55A06295CE870B07
      029BFCDB2DCE28D959F2815B16F81798

G.y = 0x483ADA7726A3C4655DA4FBFC0E1108A8
      FD17B448A68554199C47D08FFB10D4B8
```

These coordinates look random but are actually derived from a simple formula, making them "nothing up my sleeve" numbers—verifiably not chosen to hide a backdoor. Satoshi didn't define G in Bitcoin's code; he used OpenSSL's built-in `NID_secp256k1` curve identifier, which includes G and all other curve parameters.

When you generate a Bitcoin key pair:

```
// Your private key: a random 256-bit number
k = 0x1 to 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEB
    BAAEDCE6AF48A03BBFD25E8CD0364140

// Your public key: k times G (point multiplication)
P = k × G

// Computing P from k: ~0.001 seconds
// Computing k from P: ~10^37 years (all computers on Earth)
```

This asymmetry—trivial to compute forward, impossible to reverse—is why your public key (and Bitcoin address) can be safely shared while your private key must remain secret.

A NOTE ON THE SOURCE CODE

You won't find elliptic curve math in Bitcoin's source code. Satoshi delegated **all** cryptography to **OpenSSL**, a battle-tested library. In `key.h`, the entire `CKey` class is a thin wrapper around OpenSSL functions:

```
// From key.h - Satoshi's entire ECC implementation:
EC_KEY* pkey = EC_KEY_new_by_curve_name(NID_secp256k1); // Create key
EC_KEY_generate_key(pkey); // Generate keypair
ECDSA_sign(...); // Sign data
ECDSA_verify(...); // Verify signature
```

The point addition, scalar multiplication, and all the complex curve arithmetic live inside OpenSSL's compiled libraries. This was wise— cryptographic code is notoriously difficult to implement securely, and using a well-audited library avoided potential vulnerabilities.

DIGITAL SIGNATURES (ECDSA)

Bitcoin uses the **Elliptic Curve Digital Signature Algorithm (ECDSA)** to prove ownership. When you spend bitcoins, you create a signature that proves you know the private key without revealing it.

```
Signing (private key k, message m):
1. Generate random number r
2. Compute R = rG (a curve point)
3. Compute s = (hash(m) + R.x × k) / r
4. Signature is (R.x, s)

Verification (public key P, message m, signature):
1. Compute u1 = hash(m) / s
2. Compute u2 = R.x / s
3. Verify that u1×G + u2×P = R
```

The signature proves you know k (private key) because only someone who knows k could produce an s that satisfies the verification equation.

HASH FUNCTIONS IN BITCOIN

Bitcoin uses several hash functions for different purposes:

SHA-256 (SECURE HASH ALGORITHM)

The primary hash function. Takes any input and produces a 256-bit output. Used for:

- Block hashes (proof-of-work mining)
- Transaction IDs
- Merkle tree construction

How SHA-256 Works (Step by Step)

SHA-256 processes input through four main steps:

STEP 1: PADDING

The message is padded to a multiple of 512 bits: append a '1' bit, then '0' bits, then the original length as a 64-bit integer.

STEP 2: INITIALIZE HASH VALUES

Eight 32-bit words are initialized from the square roots of the first 8 primes:

```
H0 = 0x6a09e667  H4 = 0x510e527f
H1 = 0xbb67ae85  H5 = 0x9b05688c
H2 = 0x3c6ef372  H6 = 0x1f83d9ab
H3 = 0xa54ff53a  H7 = 0x5be0cd19
```

STEP 3: PROCESS BLOCKS (64 ROUNDS)

For each 512-bit block, run 64 rounds of compression. Each round:

```
T1 = h + S1(e) + Ch(e,f,g) + K[t] + W[t]
T2 = S0(a) + Maj(a,b,c)

Then rotate: h=g, g=f, f=e, e=d+T1, d=c, c=b, b=a, a=T1+T2
```

The operations mix bits thoroughly:

- **Ch** (Choose): if e then f else g
- **Maj** (Majority): majority vote of a,b,c
- **S0, S1**: rotate and XOR operations
- **K[t]**: 64 constants from cube roots of primes
- **W[t]**: message schedule derived from input

STEP 4: OUTPUT

Concatenate the final H0-H7 values to produce 256 bits (64 hex chars):

```
SHA256("Bitcoin") = b4056df6691f8dc72e56302ddad345d65...
```

The key property: every input bit affects every output bit through 64 rounds of mixing. Bitcoin uses **double SHA-256** (SHA256(SHA256(x))) for extra security.

The key insight: every bit of input affects every bit of output through 64 rounds of mixing. The constants $K[0..63]$ are derived from cube roots of the first 64 primes—nothing up anyone's sleeve.

Bitcoin often uses **double SHA-256** ($\text{SHA256}(\text{SHA256}(x))$) for extra security against length-extension attacks.

THE AVALANCHE EFFECT

A critical property of cryptographic hash functions is the **avalanche effect**: changing even a single character in the input produces a completely different hash. This makes it impossible to predict or reverse-engineer the output:

```
Input: "The Times 03/Jan/2009"
SHA256: 72a0d3c0c831c12b6e6066ce5f6f3d77e6d61c8bbdfd4b7a9a25a11a0c36e65c

Input: "The Times 03/Jan/2008" // Changed 2009 → 2008
SHA256: 9f14e5a317b27c6c5dbf4e6d1a7b8c2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c

Input: "the Times 03/Jan/2009" // Changed T → t
SHA256: 3d2e1f0a9b8c7d6e5f4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8e7f6a5b4c3d2e
```

Notice how changing just one digit (2009→2008) or one letter's case (T→t) produces a hash that shares virtually nothing with the original. This is why Satoshi embedded the Times headline in the genesis block—it proves the block couldn't have been created before that specific date.

RIPEMD-160

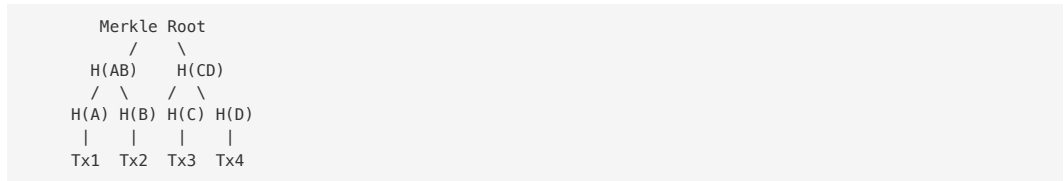
A 160-bit hash function used in combination with SHA-256 to create Bitcoin addresses:

```
Public Key (65 bytes)
  ↓ SHA-256
256-bit hash
  ↓ RIPEMD-160
160-bit hash (20 bytes)
  ↓ Base58Check encoding
Bitcoin Address (e.g., 1A1zP1eP5QGeFi2DMPTfTL5SLmv7DivfNa)
```

Using two different hash functions provides defense in depth—even if one is broken, the other provides protection.

MERKLE TREES

A **Merkle tree** is a data structure that efficiently summarizes many items into a single hash. Each Bitcoin block contains a Merkle root that commits to all transactions in the block.



To prove Tx2 is in the block, you only need H(A), H(CD), and the Merkle root—not all four transactions. This enables "light clients" that verify transactions without downloading the entire blockchain.

PROOF OF WORK

Mining is a cryptographic lottery. Miners must find a number (nonce) such that the block's hash starts with many zeros:

```

Target: Hash must be less than
        000000000000000000000000abc...

Miner tries:
nonce=0: SHA256(block) = 7f3a2b... (too high)
nonce=1: SHA256(block) = 9c4d5e... (too high)
nonce=2: SHA256(block) = 1b8f7a... (too high)
...millions of attempts...
nonce=2347893: SHA256(block) = 0000000000000000abc... (valid!)
  
```

Because SHA-256 is unpredictable, the only way to find a valid nonce is trial and error. The difficulty adjusts so that the network finds a block approximately every 10 minutes, regardless of total computing power.

WHY THIS MATTERS

These cryptographic primitives work together to create Bitcoin's security model:

- **Private keys** prove ownership (discrete log is hard)
- **Signatures** authorize transactions (ECDSA)
- **Hashes** create unique identifiers and commitments
- **Merkle trees** efficiently verify inclusion
- **Proof of work** makes history tamper-proof

No trusted authority is needed—mathematics provides the guarantees.

C++ PRIMER

Bitcoin is written in C++, a powerful but complex language. This primer covers the syntax, patterns, and file organization you'll encounter in the source code.

FILE AND DIRECTORY STRUCTURE

Bitcoin v0.01 follows traditional C++ project conventions. Understanding the file organization helps you navigate the codebase and understand how the pieces fit together.

HEADER FILES (.H) VS SOURCE FILES (.CPP)

C++ splits code into two file types for a crucial reason: **separate compilation**. A large project like Bitcoin would take forever to compile if every change required re-compiling everything.

- **Header files (.h)** — Contain *declarations*: class definitions, function signatures, constants, and type definitions. They describe *what* exists without providing implementation details. Headers are "included" by other files that need to use those declarations.
- **Source files (.cpp)** — Contain *implementations*: the actual code for functions declared in headers. Each .cpp file is compiled independently into an "object file," then all object files are linked together into the final executable.

```
// main.h (header - declarations)
class CBlock {
public:
    int nVersion;
    uint256 GetHash(); // declared but not implemented
};

// main.cpp (source - implementations)
#include "main.h"

uint256 CBlock::GetHash() {
    // actual implementation here
    return Hash(BEGIN(nVersion), END(nNonce));
}
```

This separation means changing a function's implementation (in .cpp) only requires recompiling that one file. Changing a header requires recompiling everything that includes it—which is why headers change less frequently.

THE #INCLUDE DIRECTIVE

The `#include` directive literally copies the contents of another file into the current file before compilation:

```
#include "main.h"      // Local file (in same project)
#include <vector>       // System/library file
#include <openssl/sha.h> // External library
```

Quotes ("") search the local directory first; angle brackets (<>) search system include paths. Bitcoin's `headers.h` is a "precompiled header" that includes all commonly needed files in one place—a pattern that speeds up compilation.

BITCOIN V0.01 FILE ORGANIZATION

FILE	PURPOSE
Core Logic	
main.cpp / main.h	Blockchain, consensus, mining, transactions
script.cpp / script.h	Script interpreter for spending conditions
net.cpp / net.h	P2P networking and message handling
db.cpp / db.h	Berkeley DB persistence layer
Cryptography	
key.h	Elliptic curve key management (secp256k1)
sha.cpp / sha.h	SHA-256 hash implementation
bignum.h	Arbitrary-precision arithmetic (wraps OpenSSL)
uint256.h	256-bit unsigned integer for hashes
base58.h	Base58Check encoding for addresses
Infrastructure	
serialize.h	Binary serialization for network/disk
util.cpp / util.h	Utility functions, logging, argument parsing
irc.cpp / irc.h	IRC-based peer discovery (bootstrap)
headers.h	Precompiled header (includes everything)

User Interface

ui.cpp / ui.h	wxWidgets GUI application code
uibase.cpp / uibase.h	Auto-generated GUI base classes

Build & Config

makefile	Unix/Linux build instructions
makefile.vc	Windows Visual C++ build instructions
ui.rc	Windows resource file (icons, metadata)
uiproject.fbp	wxFormBuilder project (GUI designer)

WHY HEADER-ONLY FILES?

Some Bitcoin files are "header-only" (no corresponding .cpp file): `key.h`, `bignum.h`, `uint256.h`, `base58.h`, `serialize.h`. This pattern is used when:

- The code is mostly **template** code (must be in headers due to C++ rules)
- The code is **small enough** that inline compilation is efficient
- The code **wraps external libraries** (like OpenSSL) with thin adapters

Header-only code is simpler to use (just `#include` it) but increases compile time since it's recompiled in every file that includes it.

THE MAKEFILE

The `makefile` tells the compiler how to build the project. It specifies which files to compile, what compiler flags to use, and what libraries to link:

```
# Compile main.cpp into main.o (object file)
main.o: main.cpp main.h
    g++ -c main.cpp -o main.o

# Link all object files into final executable
bitcoin: main.o net.o db.o script.o ...
    g++ main.o net.o ... -o bitcoin -lssl -ldb
```

Bitcoin v0.01 includes two makefiles: `makefile` for Unix/Linux (using g++) and `makefile.vc` for Windows (using Visual C++). Satoshi initially developed on Windows, so both platforms were supported from day one.

RESOURCE AND PROJECT FILES

- **ui.rc** — Windows resource file. Defines the application icon, version info, and other Windows-specific metadata that gets embedded in the .exe file.
- **uiproject.fbp** — wxFormBuilder project file (XML format). Satoshi used this visual designer to create the GUI, then generated `uibase.cpp` and `uibase.h` from it. The .fbp file is the "source" for the GUI layout.

BASIC SYNTAX

Every C++ statement ends with a semicolon. Blocks of code are grouped with curly braces:

```
int x = 5;           // statement ends with ;
if (x > 0) {         // block starts with {
    DoSomething();
    DoSomethingElse();
}                   // block ends with }
```

Comments explain code without affecting execution:

```
// This is a single-line comment

/* This is a
   multi-line comment */
```

VARIABLES AND DATA TYPES

Variables store data. You must declare their type:

```
int count = 10;           // integer (whole number)
double price = 19.99;     // floating-point (decimal)
char letter = 'A';        // single character
bool isValid = true;      // boolean (true/false)
string name = "Satoshi";  // text string
```

SIGNED VS UNSIGNED

Integers can be **signed** (positive or negative) or **unsigned** (non-negative only).

Unsigned integers can store larger positive values:

```
int balance = -100;       // signed: -2B to +2B
unsigned int amount = 4000000000; // unsigned: 0 to 4B

// Bitcoin uses specific-width types:
int64 nValue;             // 64-bit signed (-9e18 to +9e18)
uint64 nAmount;           // 64-bit unsigned (0 to 18e18)
unsigned char byte;       // 8-bit unsigned (0 to 255)
```

Bitcoin amounts are stored as `int64` in satoshis (1 BTC = 100,000,000 satoshis), allowing for 8 decimal places of precision.

TYPE CASTING

Converting between types is called **casting**:

```
int a = 5;
int b = 2;
double result = (double)a / b; // Cast a to double: 2.5
                                // Without cast: 2 (integer division)

// C++ style casts (more explicit):
int64 n = static_cast<int64>(value);

// Pointer casts (reinterpret raw bytes):
unsigned char* pch = (unsigned char*)&nValue;
```

OPERATORS

```
// Arithmetic
a + b // addition
a - b // subtraction
a * b // multiplication
a / b // division
a % b // modulo (remainder)

// Comparison (return true/false)
a == b // equal
a != b // not equal
a < b // less than
a > b // greater than
a <= b // less than or equal
a >= b // greater than or equal

// Logical
a && b // AND (both true)
a || b // OR (either true)
!a // NOT (invert)

// Bitwise (operate on individual bits)
a & b // AND
a | b // OR
a ^ b // XOR
~a // NOT (complement)
a << n // left shift (multiply by 2^n)
a >> n // right shift (divide by 2^n)

// Assignment
a = b // assign
a += b // add and assign (a = a + b)
a++ // increment (a = a + 1)
a-- // decrement (a = a - 1)
```

CONTROL FLOW

CONDITIONALS

```

if (condition) {
    // executed if condition is true
} else if (other_condition) {
    // executed if other_condition is true
} else {
    // executed if all conditions are false
}

// Ternary operator (compact if-else):
int max = (a > b) ? a : b; // if a>b then a, else b

```

LOOPS

```

// For loop: init; condition; increment
for (int i = 0; i < 10; i++) {
    printf("%d\n", i); // prints 0 through 9
}

// While loop: repeat while condition is true
while (count > 0) {
    count--;
}

// Do-while: execute at least once
do {
    TryConnect();
} while (!connected);

// Loop control:
break; // exit loop immediately
continue; // skip to next iteration

```

SWITCH

```

switch (opcode) {
    case OP_ADD:
        result = a + b;
        break;
    case OP_SUB:
        result = a - b;
        break;
    default:
        error = true;
        break;
}

```

FUNCTIONS

Functions encapsulate reusable code:

```
// Declaration: return_type name(parameters)
int Add(int a, int b) {
    return a + b;
}

// Calling the function:
int sum = Add(3, 5); // sum = 8

// Void functions return nothing:
void PrintMessage(string msg) {
    printf("%s\n", msg.c_str());
}

// Default parameters:
void Connect(string host, int port = 8333) {
    // port defaults to 8333 if not specified
}
```

CLASSES AND STRUCTS

Classes bundle data and functions together:

```
class CTransaction {
public:                                // accessible from outside
    int nVersion;
    vector<CTxIn> vin;                // inputs
    vector<CTxOut> vout;              // outputs

    uint256 GetHash() {               // member function
        return Hash(BEGIN(nVersion), END(nLockTime));
    }

private:                              // only accessible inside class
    int nLockTime;
};

// Creating and using objects:
CTransaction tx;
tx.nVersion = 1;
uint256 hash = tx.GetHash();
```

Structs are like classes but default to public access. Bitcoin uses both interchangeably.

POINTERS AND REFERENCES

Pointers store memory addresses:

```

int value = 42;
int* ptr = &value; // ptr points to value's address
*ptr = 100;        // dereference: now value = 100

// Null pointer (points to nothing):
int* p = NULL;
if (p != NULL) { ... }

// Arrow operator for pointer to struct/class:
CBlock* pblock = new CBlock();
pblock->nVersion = 1; // same as (*pblock).nVersion

```

References are aliases for existing variables:

```

int value = 42;
int& ref = value; // ref IS value (same memory)
ref = 100;        // now value = 100

// Common use: efficient function parameters
void ProcessBlock(CBlock& block) { // no copy made
    block.nNonce++;
}

```

ARRAYS AND VECTORS

```

// Fixed-size array:
int numbers[5] = {1, 2, 3, 4, 5};
numbers[0] = 10; // access by index (0-based)

// Vector (dynamic array from STL):
vector<int> v;
v.push_back(1); // add element
v.push_back(2);
int size = v.size(); // number of elements
int first = v[0]; // access by index

// Iterating:
for (int i = 0; i < v.size(); i++) {
    printf("%d\n", v[i]);
}

// Range-based (modern C++):
for (int n : v) {
    printf("%d\n", n);
}

```

MAPS (DICTIONARIES)

```
// Key-value storage:
map<string, int> balances;
balances["Alice"] = 100;
balances["Bob"] = 50;

// Lookup:
int aliceBalance = balances["Alice"];

// Check if key exists:
if (balances.count("Charlie") > 0) {
    // Charlie exists in map
}

// Iterate over all entries:
for (map<string, int>::iterator it = balances.begin();
     it != balances.end(); ++it) {
    printf("%s: %d\n", it->first.c_str(), it->second);
}
```

THE PREPROCESSOR

Lines starting with `#` are processed before compilation:

```
// Include another file's contents:
#include "main.h"      // local file
#include <vector>       // system file

// Define constants:
#define COIN 100000000 // 1 BTC in satoshis
#define MAX_BLOCK_SIZE 1000000

// Conditional compilation:
#ifdef WIN32
    // Windows-specific code
#else
    // Unix/Linux code
#endif

// Macros (inline code substitution):
#define MIN(a,b) ((a) < (b) ? (a) : (b))
```

COMMON BITCOIN PATTERNS

Patterns you'll see throughout the code:

```

// CRITICAL_BLOCK: Thread-safe access
CRITICAL_BLOCK(cs_main) {
    // Only one thread can execute this at a time
}

// BOOST_FOREACH: Iterate over collections
BOOST_FOREACH(CTransaction& tx, block.vtx) {
    ProcessTransaction(tx);
}

// Serialization: IMPLEMENT_SERIALIZE
IMPLEMENT_SERIALIZE(
    READWRITE(nVersion);
    READWRITE(vin);
    READWRITE(vout);
)

// Error handling:
if (!CheckTransaction())
    return error("Invalid transaction");

// Logging:
printf("ProcessBlock: %s\n", hash.ToString().c_str());

```

MEMORY MANAGEMENT

```

// Allocate on heap (must manually free):
CBlock* pblock = new CBlock();
// ... use pblock ...
delete pblock; // free memory

// Automatic cleanup with smart pointers:
auto_ptr<CBlock> pblock(new CBlock());
// automatically deleted when out of scope

// Stack allocation (automatic cleanup):
CBlock block; // created on stack
// deleted automatically when function returns

```


THE SOURCE CODE

Bitcoin v0.01 — annotated and explained

rc/ (Resources)

11 files ICONS & BITMAPS

THE ORIGINAL BITCOIN LOGO

Before there was the orange "₿" symbol recognized worldwide, there was this— Satoshi's original Bitcoin logo, buried in a Windows resource directory. These icons and bitmaps shipped with v0.1, appearing in the Windows GUI that most early users experienced. The design is notably different from the stylized "B" that would later become synonymous with Bitcoin.

The main `bitcoin.ico` is a Windows multi-resolution icon containing 16×16, 32×32, and 48×48 pixel versions—standard practice for Windows applications of the era. The icon shows a gold coin with "BC" lettering, reflecting the "digital gold" concept Satoshi frequently invoked. This logo appears to be Satoshi's own creation.

The toolbar bitmap files (`.bmp`) were used in the wxWidgets GUI: address book icons for managing contacts, and send arrows for the payment interface. These may have been sourced from standard icon sets common in Windows development at the time. The "mask" files define transparency—a technique from early Windows GUI development before PNG transparency became standard.

In February 2010, a forum user named "Sirius" created the now-iconic orange ₿ logo with the tilted "B" and two vertical strokes. Satoshi approved: "I'm not good with graphics... that looks great." The community adopted it, but these original icons remain as artifacts of Bitcoin's earliest visual identity— assembled by its pseudonymous creator before the world knew what Bitcoin would become.



bitcoin.ico

Main application icon (multi-resolution: 16×16, 32×32, 48×48)



check.ico

Checkmark icon for confirmations



addressbook16.bmp

Address book toolbar icon (16×16)



addressbook16mask.bmp

Address book icon transparency mask



addressbook20.bmp

Address book toolbar icon (20×20)



addressbook20mask.bmp

Address book icon transparency mask



send16.bmp

Send payment toolbar icon (16×16)



send16mask.bmp

Send icon transparency mask



send16masknoshadow.bmp

Send icon mask without shadow



send20.bmp

Send payment toolbar icon (20×20)



send20mask.bmp

Send icon transparency mask

SUMMARY

These resource files represent Satoshi's complete vision: not just the cryptographic protocol, but a usable application with icons, buttons, and visual identity. Whether custom-made or sourced from icon libraries, they show Satoshi was building software for real people, not just publishing academic theory. The gold "BC" coin—Bitcoin's first logo—remains an artifact of the project's bootstrapped origins.

base58.h

202 lines CPP

HUMAN-READABLE MONEY

Every Bitcoin address you've ever seen—those strings starting with "1" or "3"—uses Base58Check encoding defined in this file. Base58 is a binary-to-text encoding designed specifically for human readability: it uses all alphanumeric characters except those easily confused with each other.

The excluded characters are: **0** (zero, confused with O), **O** (confused with 0), **l** (confused with I or 1), and **1** (lowercase L, confused with I or 1). This careful character selection means you can read a Bitcoin address aloud over the phone with minimal ambiguity.

Base58Check adds a crucial feature: a 4-byte checksum appended to the data. If you mistype a single character, the checksum almost certainly won't match, and the address will be rejected as invalid. The probability of a typo accidentally producing a valid address is approximately 1 in 4 billion. This protects users from losing funds to typos.

The encoding process: (1) prepend version byte (0×00 for mainnet addresses), (2) compute SHA256(SHA256(data)) and take first 4 bytes as checksum, (3) append checksum to data, (4) convert to Base58. The version byte is why mainnet addresses start with "1"—0×00 in Base58 is represented as leading "1"s.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5
6 //
7 // Why base-58 instead of standard base-64 encoding?

```

Satoshi's original comment explaining Base58's design rationale. This is one of the best-documented design decisions in v0.1. The four reasons: (1) avoid confusable characters 0/O and l/I, (2) alphanumeric-only is friendlier for account numbers, (3) email clients won't line-break pure alphanumeric strings, (4) double-click selects the entire address. These practical considerations show Satoshi thinking about real-world usability, not just cryptographic elegance.

```

8 // - Don't want 001l characters that look the same in some fonts and
9 //       could be used to create visually identical looking account numbers.
10 // - A string with non-alphanumeric characters is not as easily accepted as an account number.
11 // - E-mail usually won't line-break if there's no punctuation to break at.
12 // - Doubleclicking selects the whole number as one word if it's all alphanumeric.
13 //
14
15
16 static const char* pszBase58 = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";

```

The Base58 alphabet: 123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz. Note the missing characters: no 0 (zero), o (oh), l (eye), or I (ell). This 58-character set maps directly to values 0-57. The character 1 represents value 0, 2 represents 1, etc. This is why addresses starting with many 1s have many leading zero bytes in their decoded form—the genesis coinbase's output had hash160 starting with zeros, giving an address full of 1s.

```

17
18
19 inline string EncodeBase58(const unsigned char* pbegin, const unsigned char* pend)

```

`EncodeBase58()` —converts arbitrary binary data to Base58 string. The algorithm is essentially base conversion: treat input bytes as a big-endian number, repeatedly divide by 58, and map remainders to characters. The use of `CBigNum` handles the arbitrary-precision arithmetic needed for 20+ byte inputs. The string is built in reverse (least significant digit first), then reversed at the end.

```

20 {
21     CAutoBN_CTX pctx;
22     CBigNum bn58 = 58;
23     CBigNum bn0 = 0;
24
25 // Convert big endian data to little endian
26 // Extra zero at the end make sure bignum will interpret as a positive number
27     vector<unsigned char> vchTmp(pend-pbegin+1, 0);

```

Extra zero byte ensures the bignum is interpreted as positive. Without this, a byte sequence starting with 0x80+ would be treated as negative in MPI format. The `reverse_copy()` converts from the input's byte order to bignum's expected format.

```

28 reverse_copy(pbegin, pend, vchTmp.begin());
29
30 // Convert little endian data to bignum
31     CBigNum bn;
32     bn.setvch(vchTmp);
33

```



```

34 // Convert bignum to string
35 string str;
36 str.reserve((pend - pbegin) * 138 / 100 + 1);

```

Space reservation: $(\text{pend} - \text{pbegin}) * 138 / 100 + 1$. Base58 encoding expands data by factor $\log(256)/\log(58) \approx 1.37$. The calculation $138/100 \approx 1.38$ provides a safe upper bound. Pre-allocating avoids repeated reallocations as the string grows.

```

37 CBigNum dv;
38 CBigNum rem;
39 while (bn > bn0)

```

The encoding loop: repeatedly divide by 58, taking remainders. Each remainder (0-57) maps to one Base58 character via the alphabet lookup. The loop continues until the number reaches zero. This produces digits from least to most significant, hence the later reversal.

```

40 {
41 if (!BN_div(&dv, &rem, &bn, &bn58, pctx))
42 throw bignum_error("EncodeBase58 : BN_div failed");
43 bn = dv;
44 unsigned int c = rem.getulong();
45 str += pszBase58[c];
46 }
47
48 // Leading zeroes encoded as base58 zeros

```

Preserving leading zeros—a crucial detail. In Base58, leading 1 characters represent leading zero bytes in the original data. Without this loop, input `0x00 0x00 0xAB` would encode the same as `0xAB`, losing information. By counting leading zero bytes and prepending that many 1s, the encoding becomes invertible. This is essential for addresses: the version byte `0x00` must be preserved.

```

49 for (const unsigned char* p = pbegin; p < pend && *p == 0; p++)
50 str += pszBase58[0];
51
52 // Convert little endian string to big endian
53 reverse(str.begin(), str.end());
54 return str;
55 }
56
57 inline string EncodeBase58(const vector<unsigned char>& vch)
58 {
59 return EncodeBase58(&vch[0], &vch[0] + vch.size());
60 }
61
62 inline bool DecodeBase58(const char* psz, vector<unsigned char>& vchRet)

```

`DecodeBase58()` —converts Base58 string back to binary. The inverse of `EncodeBase58`: treat the string as a base-58 number, convert to binary. Each character is looked up in the alphabet to get its value (0-57), then the running total is multiplied by 58 and the value added. The final result is converted to a byte vector.

```

63 {
64 CAutoBN_CTX pctx;
65 vchRet.clear();
66 CBigNum bn58 = 58;
67 CBigNum bn = 0;
68 CBigNum bnChar;
69 while (isspace(*psz))
70 psz++;

```

```

71
72 // Convert big endian string to bignum
73 for (const char* p = psz; *p; p++)
74 {
75     const char* p1 = strchr(pszBase58, *p);

```

Character validation: `strchr(pszBase58, *p)` returns `NULL` if the character isn't in the Base58 alphabet. Invalid characters (including the excluded `0`, `O`, `I`, `l`) cause decoding to fail rather than silently producing wrong results. This is important for security—a corrupted address should be detected, not misinterpreted.

```

76 if (p1 == NULL)
77 {
78     while (isspace(*p))
79         p++;
80     if (*p != '\0')
81         return false;
82     break;
83 }
84     bnChar.setulong(p1 - pszBase58);
85     if (!BN_mul(&bn, &bn, &bn58, pctx))
86         throw bignum_error("DecodeBase58 : BN_mul failed");
87     bn += bnChar;
88 }
89
90 // Get bignum as little endian data
91     vector<unsigned char> vchTmp = bn.getvch();
92
93 // Trim off sign byte if present
94 if (vchTmp.size() >= 2 && vchTmp.end()[-1] == 0 && vchTmp.end()[-2] >= 0x80)

```

Sign byte trimming: if the high bit of the first data byte is set, MPI format prepends a `0x00` to indicate positive. This extra byte must be removed during decoding to recover the original data. The condition checks for this case: `size ≥ 2`, last byte is `0`, second-to-last has high bit set.

```

95     vchTmp.erase(vchTmp.end()-1);
96
97 // Restore leading zeros
98 int nLeadingZeros = 0;

```

Restoring leading zeros—the inverse of the encoding step. Count leading `1` characters (which represent zero bytes), then prepend that many zero bytes to the output. Combined with the character-by-character conversion, this ensures round-trip correctness: `DecodeBase58(EncodeBase58(data)) == data`.

```

99 for (const char* p = psz; *p == pszBase58[0]; p++)
100     nLeadingZeros++;
101     vchRet.assign(nLeadingZeros + vchTmp.size(), 0);
102
103 // Convert little endian data to big endian
104 reverse_copy(vchTmp.begin(), vchTmp.end(), vchRet.end() - vchTmp.size());
105 return true;
106 }
107
108 inline bool DecodeBase58(const string& str, vector<unsigned char>& vchRet)
109 {
110     return DecodeBase58(str.c_str(), vchRet);
111 }
112
113
114
115

```

```

116
117 inline string EncodeBase58Check(const vector<unsigned char>& vchIn)

```

`EncodeBase58Check()` —adds checksum for error detection. The checksum is the first 4 bytes of SHA256(SHA256(data)). Double-SHA256 was Satoshi's standard construction throughout Bitcoin—it provides extra security margin against length-extension attacks. The 4-byte checksum catches 99.99999998% of random errors ($1 - 1/2^{32}$).

```

118 {
119 // add 4-byte hash check to the end
120     vector<unsigned char> vch(vchIn);
121     uint256 hash = Hash(vch.begin(), vch.end());

```

Double-SHA256 checksum: `Hash(vch.begin(), vch.end())` computes `SHA256(SHA256(vch))`. Only the first 4 bytes are used as checksum—enough for error detection while keeping addresses reasonably short. The checksum is appended to the data, then the combined result is Base58-encoded.

```

122     vch.insert(vch.end(), (unsigned char*)&hash, (unsigned char*)&hash + 4);
123     return EncodeBase58(vch);
124 }
125
126 inline bool DecodeBase58Check(const char* psz, vector<unsigned char>& vchRet)

```

`DecodeBase58Check()` —verifies checksum and strips it. After Base58 decoding, the last 4 bytes are the checksum. The function recomputes the checksum over the data portion and compares. If they match, the checksum is removed and the original data returned. If not, the function returns false—the input was corrupted.

```

127 {
128 if (!DecodeBase58(psz, vchRet))
129     return false;
130 if (vchRet.size() < 4)
131 {
132     vchRet.clear();
133     return false;
134 }
135     uint256 hash = Hash(vchRet.begin(), vchRet.end()-4);
136 if (memcmp(&hash, &vchRet.end()-4, 4) != 0)

```

Checksum verification: `memcmp(&hash, &vchRet.end()-4, 4)`. This constant-time comparison checks if the computed and stored checksums match. A mismatch means the address was corrupted (typo, OCR error, transmission corruption). By returning false rather than the corrupted data, users are protected from sending funds to wrong addresses.

```

137 {
138     vchRet.clear();
139     return false;
140 }
141     vchRet.resize(vchRet.size()-4);
142     return true;
143 }
144
145 inline bool DecodeBase58Check(const string& str, vector<unsigned char>& vchRet)
146 {
147     return DecodeBase58Check(str.c_str(), vchRet);
148 }
149

```

```

150
151
152
153
154
155 static const unsigned char ADDRESSVERSION = 0;

```

ADDRESSVERSION = 0—the version byte for mainnet pay-to-pubkey-hash addresses. This byte is prepended to the hash160 before encoding, causing mainnet addresses to start with '1'. Testnet uses version 111, causing addresses to start with 'm' or 'n'. Version bytes enable the network to evolve—new address types (like P2SH, starting with '3') use different versions.

```

156
157 inline string Hash160ToAddress(uint160 hash160)

```

Hash160ToAddress()—converts a 160-bit hash to a Bitcoin address. This is the final step of address generation: (1) SHA256 the public key, (2) RIPEMD160 the result (giving 'hash160'), (3) prepend version byte, (4) Base58Check encode. The 160-bit hash provides 2^{80} security against collision attacks while keeping addresses manageable length.

```

158 {
159 // add 1-byte version number to the front
160     vector<unsigned char> vch(1, ADDRESSVERSION);
161     vch.insert(vch.end(), UBEGIN(hash160), UEND(hash160));
162     return EncodeBase58Check(vch);
163 }
164
165 inline bool AddressToHash160(const char* psz, uint160& hash160Ret)

```

AddressToHash160()—extracts the hash160 from a Bitcoin address. Used when creating transactions: the recipient's address is decoded to get the hash160, which goes into the scriptPubKey. Version byte validation (line 176: nVersion <= ADDRESSVERSION) rejects addresses from incompatible networks—you can't accidentally send mainnet coins to a testnet address.

```

166 {
167     vector<unsigned char> vch;
168     if (!DecodeBase58Check(psz, vch))
169         return false;
170     if (vch.empty())
171         return false;
172     unsigned char nVersion = vch[0];
173     if (vch.size() != sizeof(hash160Ret) + 1)
174         return false;
175     memcpy(&hash160Ret, &vch[1], sizeof(hash160Ret));
176     return (nVersion <= ADDRESSVERSION);
177 }
178
179 inline bool AddressToHash160(const string& str, uint160& hash160Ret)
180 {
181     return AddressToHash160(str.c_str(), hash160Ret);
182 }
183
184 inline bool IsValidBitcoinAddress(const char* psz)

```

`IsValidBitcoinAddress()` —validates address format. Checks that the string Base58Check decodes successfully and has correct version byte. Note: this only validates format, not that anyone controls the address. An address could be valid format but have no known private key (e.g., 'burn addresses' like 1BitcoinEaterAddressDontSendf59kuE).

```

185 {
186     uint160 hash160;
187     return AddressToHash160(psz, hash160);
188 }
189
190 inline bool IsValidBitcoinAddress(const string& str)
191 {
192     return IsValidBitcoinAddress(str.c_str());
193 }
194
195
196
197
198 inline string PubKeyToAddress(const vector<unsigned char>& vchPubKey)

```

`PubKeyToAddress()` —the complete public key to address pipeline in one line. `Hash160(vchPubKey)` computes `RIPEMD160(SHA256(pubkey))`, then `Hash160ToAddress()` adds version byte and Base58Check encodes. This is how wallet software generates receiving addresses from keys.

```

199 {
200     return Hash160ToAddress(Hash160(vchPubKey));
201 }
202

```

SUMMARY

Base58Check encoding is a masterclass in practical cryptographic design. The checksum catches typos before funds are lost. The careful character selection prevents visual confusion. The version byte enables network evolution. And the encoding is compact enough for human use while being unambiguous enough for computer parsing.

The 4-byte checksum provides strong error detection: the probability of a random typo producing a valid address is 1 in 4.3 billion. For comparison, a credit card number's Luhn checksum only catches 90% of single-digit errors. Bitcoin's double-SHA256 checksum is overkill for error detection alone—but Satoshi consistently chose conservative security margins.

The hash160 construction (RIPEMD160(SHA256(pubkey))) deserves note: using two different hash functions provides defense in depth. If SHA256 were broken, RIPEMD160 still protects addresses. If RIPEMD160 were broken, SHA256 still protects. This belt-and-suspenders approach reflects Satoshi's security-conscious engineering throughout Bitcoin.

bignum.h

499 lines CPP

WHEN 64 BITS AREN'T ENOUGH

Bitcoin deals with astronomically large numbers—256-bit integers that dwarf anything standard CPUs can handle natively. A 64-bit processor maxes out at 18 quintillion; Bitcoin routinely works with numbers that have 77+ digits. This file wraps OpenSSL's BIGNUM library to perform arithmetic on these titans.

Every cryptographic operation in Bitcoin relies on big number math: generating private keys (random 256-bit numbers), deriving public keys (elliptic curve point multiplication), creating signatures (modular arithmetic), verifying signatures (more modular arithmetic), and comparing hashes to difficulty targets. Without this file, none of that would be possible.

OpenSSL's BIGNUM library has been battle-tested for decades in SSL/TLS—the encryption protecting billions of web connections. Satoshi wisely reused this proven code rather than implementing big number arithmetic from scratch. Rolling your own crypto is how security vulnerabilities are born.

The CBigNum class wraps OpenSSL's C interface with C++ conveniences: operator overloading lets you write `a + b * c` instead of calling verbose functions. It also handles memory management automatically—BIGNUM requires manual allocation/deallocation, which the destructor handles safely.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include <stdexcept>
6 #include <vector>
7 #include <openssl/bn.h>
8
9
10
11
12
13 class bignum_error : public std::runtime_error

```

bignum_error—exception class for arithmetic failures. OpenSSL's BIGNUM functions return error codes rather than throwing exceptions. This wrapper converts errors to C++ exceptions, making error handling cleaner. Thrown when operations fail due to memory allocation errors or invalid operations.

```

14 {
15 public:
16 explicit bignum_error(const std::string& str) : std::runtime_error(str) {}
17 };
18
19
20
21 class CAutoBN_CTX

```

CAutoBN_CTX—RAII wrapper for OpenSSL's BN_CTX. A BN_CTX is a 'context' that provides temporary storage for intermediate values during multi-step calculations (like modular exponentiation). Without this, you'd need to allocate/free temporary BIGNUMs manually. The constructor allocates, destructor frees—C++ guarantees cleanup even if exceptions are thrown. This pattern appears throughout security-critical code.

```

22 {
23 protected:
24     BN_CTX* pctx;
25     BN_CTX* operator=(BN_CTX* pnew) { return pctx = pnew; }
26
27 public:
28 CAutoBN_CTX()
29 {
30     pctx = BN_CTX_new();
31     if (pctx == NULL)
32         throw bignum_error("CAutoBN_CTX : BN_CTX_new() returned NULL");
33 }
34
35 ~CAutoBN_CTX()
36 {
37     if (pctx != NULL)
38         BN_CTX_free(pctx);
39 }
40
41 operator BN_CTX*() { return pctx; }
42 BN_CTX& operator*() { return *pctx; }
43 BN_CTX** operator&() { return &pctx; }
44 bool operator!() { return (pctx == NULL); }
45 };
46
47
48
49 class CBigNum : public BIGNUM

```

`CBigNum`—the heart of Bitcoin's big number support. Inherits from OpenSSL's `BIGNUM` struct (unusual but valid C++) to add C++ semantics. The inheritance means `CBigNum` can be passed directly to OpenSSL functions expecting `BIGNUM*`. All the cryptographic heavy lifting—ECDSA signatures, difficulty comparisons, script arithmetic—flows through this class.

```

50 {
51 public:
52 CBigNum()
53 {
54 BN_init(this);

```

`BN_init()`—initializes `BIGNUM` to zero. OpenSSL `BIGNUM`s must be initialized before use; uninitialized memory could cause crashes or security vulnerabilities. This is called in every constructor to ensure valid state.

```

55 }
56
57 CBigNum(const CBigNum& b)
58 {
59 BN_init(this);
60 if (!BN_copy(this, &b))
61 {
62 BN_clear_free(this);
63 throw bignum_error("CBigNum::CBigNum(const CBigNum&) : BN_copy failed");
64 }
65 }
66
67 explicit CBigNum(const std::string& str)
68 {
69 BN_init(this);
70 SetHex(str);
71 }
72
73 CBigNum& operator=(const CBigNum& b)
74 {
75 if (!BN_copy(this, &b))
76 throw bignum_error("CBigNum::operator= : BN_copy failed");
77 return (*this);
78 }
79
80 ~CBigNum()

```

`BN_clear_free()`—secure memory wiping. Unlike regular `free()`, this overwrites memory with zeros before releasing it. Critical for security: freed memory might be reallocated to an attacker-controlled buffer. If private keys weren't cleared, they could leak through memory inspection. This paranoid cleanup appears throughout Bitcoin's key handling code.

```

81 {
82 BN_clear_free(this);
83 }
84
85 CBigNum(char n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }

```

Type conversion constructors—convert native integers to big numbers. The pattern `if (n >= 0) setulong(n); else setint64(n)` handles signed vs unsigned carefully. OpenSSL's `BN_set_word()` only handles unsigned values; negative numbers need the more complex MPI encoding path. These conversions let you write natural code like `CBigNum total = 50 * COIN`.

```

86 CBigNum(short n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
87 CBigNum(int n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
88 CBigNum(long n) { BN_init(this); if (n >= 0) setulong(n); else setint64(n); }
89 CBigNum(int64 n) { BN_init(this); setint64(n); }
90 CBigNum(unsigned char n) { BN_init(this); setulong(n); }
91 CBigNum(unsigned short n) { BN_init(this); setulong(n); }
92 CBigNum(unsigned int n) { BN_init(this); setulong(n); }
93 CBigNum(unsigned long n) { BN_init(this); setulong(n); }
94 CBigNum(uint64 n) { BN_init(this); setuint64(n); }
95 explicit CBigNum(uint256 n) { BN_init(this); setuint256(n); }
96
97 explicit CBigNum(const std::vector<unsigned char>& vch)
98 {
99     BN_init(this);
100     setvch(vch);
101 }
102
103 void setulong(unsigned long n)
104 {
105     if (!BN_set_word(this, n))
106         throw bignum_error("CBigNum conversion from unsigned long : BN_set_word failed");
107 }
108
109 unsigned long getulong() const
110 {
111     return BN_get_word(this);
112 }
113
114 unsigned int getuint() const
115 {
116     return BN_get_word(this);
117 }
118
119 int getint() const
120 {
121     unsigned long n = BN_get_word(this);
122     if (!BN_is_negative(this))
123         return (n > INT_MAX ? INT_MAX : n);
124     else
125         return (n > INT_MAX ? INT_MIN : -(int)n);
126 }
127
128 void setint64(int64 n)

```

`setint64()` —converts 64-bit signed integer to BIGNUM via MPI format. MPI (Multi-Precision Integer) is a big-endian byte format with a 4-byte length prefix. The sign bit is encoded in the high bit of the first data byte. This complexity exists because OpenSSL's native functions don't handle 64-bit signed values directly. The careful handling of leading zeros and sign bits is essential for correct encoding.

```

129 {
130     unsigned char pch[sizeof(n) + 6];
131     unsigned char* p = pch + 4;
132     bool fNegative = false;
133     if (n < (int64)0)
134     {
135         n = -n;
136         fNegative = true;
137     }
138     bool fLeadingZeroes = true;

```

```

139 for (int i = 0; i < 8; i++)
140 {
141     unsigned char c = (n >> 56) & 0xff;
142     n <<= 8;
143     if (fLeadingZeroes)
144     {
145         if (c == 0)
146             continue;
147         if (c & 0x80)
148             *p++ = (fNegative ? 0x80 : 0);
149         else if (fNegative)
150             c |= 0x80;
151         fLeadingZeroes = false;
152     }
153     *p++ = c;
154 }
155 unsigned int nSize = p - (pch + 4);
156 pch[0] = (nSize >> 24) & 0xff;
157 pch[1] = (nSize >> 16) & 0xff;
158 pch[2] = (nSize >> 8) & 0xff;
159 pch[3] = (nSize) & 0xff;
160 BN_mpi2bn(pch, p - pch, this);
161 }
162
163 void setuint64(uint64 n)
164 {
165     unsigned char pch[sizeof(n) + 6];
166     unsigned char* p = pch + 4;
167     bool fLeadingZeroes = true;
168     for (int i = 0; i < 8; i++)
169     {
170         unsigned char c = (n >> 56) & 0xff;
171         n <<= 8;
172         if (fLeadingZeroes)
173         {
174             if (c == 0)
175                 continue;
176             if (c & 0x80)
177                 *p++ = 0;
178             fLeadingZeroes = false;
179         }
180         *p++ = c;
181     }
182     unsigned int nSize = p - (pch + 4);
183     pch[0] = (nSize >> 24) & 0xff;
184     pch[1] = (nSize >> 16) & 0xff;
185     pch[2] = (nSize >> 8) & 0xff;
186     pch[3] = (nSize) & 0xff;
187     BN_mpi2bn(pch, p - pch, this);
188 }
189
190 void setuint256(uint256 n)

```

setuint256() —converts Bitcoin's `uint256` type to `BIGNUM`. This bridges Bitcoin's hash/key representation with OpenSSL's arithmetic. The loop walks backwards through the `uint256`'s bytes (little-endian to big-endian conversion), skipping leading zeros for compactness. Used when checking if a block hash meets the difficulty target: convert hash to `CBigNum`, compare to target `CBigNum`.

```

191 {
192     unsigned char pch[sizeof(n) + 6];
193     unsigned char* p = pch + 4;
194     bool fLeadingZeroes = true;
195     unsigned char* pbegin = (unsigned char*)&n;
196     unsigned char* psrc = pbegin + sizeof(n);
197     while (psrc != pbegin)
198     {
199         unsigned char c = *(--psrc);
200         if (fLeadingZeroes)

```

```

201 {
202 if (c == 0)
203 continue;
204 if (c & 0x80)
205 *p++ = 0;
206         fLeadingZeroes = false;
207 }
208 *p++ = c;
209 }
210 unsigned int nSize = p - (pch + 4);
211     pch[0] = (nSize >> 24) & 0xff;
212     pch[1] = (nSize >> 16) & 0xff;
213     pch[2] = (nSize >> 8) & 0xff;
214     pch[3] = (nSize >> 0) & 0xff;
215 BN_mpi2bn(pch, p - pch, this);
216 }
217
218 uint256 getuint256()
219 {
220 unsigned int nSize = BN_bn2mpi(this, NULL);
221 if (nSize < 4)
222 return 0;
223     std::vector<unsigned char> vch(nSize);
224 BN_bn2mpi(this, &vch[0]);
225 if (vch.size() > 4)
226     vch[4] &= 0x7f;
227     uint256 n = 0;
228 for (int i = 0, j = vch.size()-1; i < sizeof(n) && j >= 4; i++, j--)
229 ((unsigned char*)&n)[i] = vch[j];
230 return n;
231 }
232
233 void setvch(const std::vector<unsigned char>& vch)
234 {
235     std::vector<unsigned char> vch2(vch.size() + 4);
236 unsigned int nSize = vch.size();
237     vch2[0] = (nSize >> 24) & 0xff;
238     vch2[1] = (nSize >> 16) & 0xff;
239     vch2[2] = (nSize >> 8) & 0xff;
240     vch2[3] = (nSize >> 0) & 0xff;
241 reverse_copy(vch.begin(), vch.end(), vch2.begin() + 4);
242 BN_mpi2bn(&vch2[0], vch2.size(), this);
243 }
244
245 std::vector<unsigned char> getvch() const
246 {
247 unsigned int nSize = BN_bn2mpi(this, NULL);
248 if (nSize < 4)
249 return std::vector<unsigned char>();
250     std::vector<unsigned char> vch(nSize);
251 BN_bn2mpi(this, &vch[0]);
252     vch.erase(vch.begin(), vch.begin() + 4);
253 reverse(vch.begin(), vch.end());
254 return vch;
255 }
256
257 CBigNum& SetCompact(unsigned int nCompact)

```

SetCompact() —decodes Bitcoin's compact difficulty format. Block headers store difficulty as a 4-byte 'compact' value (nBits field) rather than a full 256-bit target. Format: first byte is exponent (number of bytes), remaining 3 bytes are mantissa. Example: 0x1d00ffff means a 0x1d (29) byte number with high bytes 00ffff. This encoding saves space in headers while representing any 256-bit target.

```

258 {
259 unsigned int nSize = nCompact >> 24;
260     std::vector<unsigned char> vch(4 + nSize);
261     vch[3] = nSize;
262 if (nSize >= 1) vch[4] = (nCompact >> 16) & 0xff;

```

```

263 if (nSize >= 2) vch[5] = (nCompact >> 8) & 0xff;
264 if (nSize >= 3) vch[6] = (nCompact >> 0) & 0xff;
265 BN_mpi2bn(&vch[0], vch.size(), this);
266 return *this;
267 }
268
269 unsigned int GetCompact() const

```

GetCompact() —encodes BIGNUM to compact format. The inverse of SetCompact(). Mining code uses this to store the current difficulty target in block headers. The format sacrifices precision (only 3 significant bytes) but this is acceptable—difficulty only needs to be approximately correct. The slight loss of precision is bounded and predictable.

```

270 {
271     unsigned int nSize = BN_bn2mpi(this, NULL);
272     std::vector<unsigned char> vch(nSize);
273     nSize -= 4;
274     BN_bn2mpi(this, &vch[0]);
275     unsigned int nCompact = nSize << 24;
276     if (nSize >= 1) nCompact |= (vch[4] << 16);
277     if (nSize >= 2) nCompact |= (vch[5] << 8);
278     if (nSize >= 3) nCompact |= (vch[6] << 0);
279     return nCompact;
280 }
281
282 void SetHex(const std::string& str)

```

SetHex() —parses hexadecimal string to BIGNUM. Block hashes, transaction IDs, and keys are commonly displayed as hex strings. This function handles optional 0x prefix, leading whitespace, and negative signs. The lookup table `phexdigit[256]` provides O(1) hex-to-value conversion. Used extensively in debugging and when loading hardcoded values like the genesis block hash.

```

283 {
284     // skip 0x
285     const char* psz = str.c_str();
286     while (isspace(*psz))
287         psz++;
288     bool fNegative = false;
289     if (*psz == '-')
290     {
291         fNegative = true;
292         psz++;
293     }
294     if (psz[0] == '0' && tolower(psz[1]) == 'x')
295         psz += 2;
296     while (isspace(*psz))
297         psz++;
298
299     // hex string to bignum
300     static char phexdigit[256] = { 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
301                                     0,0,0,0,0,0,0,0,0,0,0,0,0, 0,1,2,3,4,5,6,7,8,9,0,0,0,0,0,0,
302                                     0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
303                                     0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0 };
304     *this = 0;
305     while (isxdigit(*psz))
306     {
307         *this <<= 4;
308         int n = phexdigit[*psz++];
309         *this += n;
310     }
311     if (fNegative)
312         *this = 0 - *this;
313
314     unsigned int GetSerializeSize(int nType=0, int nVersion=VERSION) const

```

Serialization support—enables storing/transmitting `CBigNum` values. Bitcoin's custom serialization framework (see `serialize.h`) uses these methods. `getvch()` converts to byte vector for transmission; `setvch()` reconstructs from bytes. The MPI format ensures consistent encoding across different machines regardless of native integer representation.

```

313 {
314     return ::GetSerializeSize(getvch(), nType, nVersion);
315 }
316
317 template<typename Stream>
318 void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const
319 {
320     ::Serialize(s, getvch(), nType, nVersion);
321 }
322
323 template<typename Stream>
324 void Unserialize(Stream& s, int nType=0, int nVersion=VERSION)
325 {
326     vector<unsigned char> vch;
327     ::Unserialize(s, vch, nType, nVersion);
328     setvch(vch);
329 }
330
331
332 bool operator!() const
333 {
334     return BN_is_zero(this);
335 }
336
337 CBigNum& operator+=(const CBigNum& b)

```

Arithmetic operators—make big numbers feel like native types. `operator+=` calls OpenSSL's `BN_add()`; `operator*=` calls `BN_mul()`; etc. The `CAutoBN_CTX` provides temporary storage needed by multiplication and division. These operators enable readable code like `reward >>= (height / 210000)` for the halving calculation, rather than verbose function calls.

```

338 {
339     if (!BN_add(this, this, &b))
340         throw bignum_error("CBigNum::operator+= : BN_add failed");
341     return *this;
342 }
343
344 CBigNum& operator--=(const CBigNum& b)
345 {
346     *this = *this - b;
347     return *this;
348 }
349
350 CBigNum& operator*=(const CBigNum& b)
351 {
352     CAutoBN_CTX pctx;
353     if (!BN_mul(this, this, &b, pctx))
354         throw bignum_error("CBigNum::operator*= : BN_mul failed");
355     return *this;
356 }
357
358 CBigNum& operator/=(const CBigNum& b)
359 {
360     *this = *this / b;
361     return *this;
362 }
363
364 CBigNum& operator%=(const CBigNum& b)
365 {

```

```

366 *this = *this % b;
367 return *this;
368 }
369
370 CBigNum& operator<=(unsigned int shift)

```

Bit shift operators—essential for Bitcoin's difficulty arithmetic. The halving formula `subsidy >= (height/210000)` uses right-shift to divide by powers of two. Difficulty adjustments use shifts to multiply/divide targets. OpenSSL's `BN_lshift()` and `BN_rshift()` handle arbitrarily large shifts correctly—you can shift a 256-bit number by 200 bits without overflow issues.

```

371 {
372     if (!BN_lshift(this, this, shift))
373         throw bignum_error("CBigNum:operator<= : BN_lshift failed");
374     return *this;
375 }
376
377 CBigNum& operator>=(unsigned int shift)
378 {
379     if (!BN_rshift(this, this, shift))
380         throw bignum_error("CBigNum:operator>= : BN_rshift failed");
381     return *this;
382 }
383
384 CBigNum& operator++()
385 {
386     // prefix operator
387     if (!BN_add(this, this, BN_value_one()))
388         throw bignum_error("CBigNum::operator++ : BN_add failed");
389     return *this;
390 }
391
392 const CBigNum operator++(int)
393 {
394     // postfix operator
395     const CBigNum ret = *this;
396     ++(*this);
397     return ret;
398 }
399
400 CBigNum& operator--()
401 {
402     // prefix operator
403     CBigNum r;
404     if (!BN_sub(&r, this, BN_value_one()))
405         throw bignum_error("CBigNum:operator-- : BN_sub failed");
406     *this = r;
407     return *this;
408 }
409
410 const CBigNum operator--(int)
411 {
412     // postfix operator
413     const CBigNum ret = *this;
414     --(*this);
415     return ret;
416 }
417
418
419 friend inline const CBigNum operator-(const CBigNum& a, const CBigNum& b);
420 friend inline const CBigNum operator/(const CBigNum& a, const CBigNum& b);
421 friend inline const CBigNum operator%(const CBigNum& a, const CBigNum& b);
422 };
423
424
425
426
427 inline const CBigNum operator+(const CBigNum& a, const CBigNum& b)

```

Free-standing arithmetic operators. C++ convention: compound operators (+=) are members, simple operators (+) are free functions built on compound operators. This minimizes code duplication while providing the full expected arithmetic interface. The pattern `CBigNum r; BN_add(&r, &a, &b); return r;` creates a new `BIGNUM` for the result, leaving operands unchanged.

```

428 {
429     CBigNum r;
430     if (!BN_add(&r, &a, &b))
431         throw bignum_error("CBigNum::operator+ : BN_add failed");
432     return r;
433 }
434
435 inline const CBigNum operator-(const CBigNum& a, const CBigNum& b)
436 {
437     CBigNum r;
438     if (!BN_sub(&r, &a, &b))
439         throw bignum_error("CBigNum::operator- : BN_sub failed");
440     return r;
441 }
442
443 inline const CBigNum operator-(const CBigNum& a)
444 {
445     CBigNum r(a);
446     BN_set_negative(&r, !BN_is_negative(&r));
447     return r;
448 }
449
450 inline const CBigNum operator*(const CBigNum& a, const CBigNum& b)
451 {
452     CAutoBN_CTX pctx;
453     CBigNum r;
454     if (!BN_mul(&r, &a, &b, pctx))
455         throw bignum_error("CBigNum::operator* : BN_mul failed");
456     return r;
457 }
458
459 inline const CBigNum operator/(const CBigNum& a, const CBigNum& b)

```

`operator/()` and `operator%()`—division and modulus. Modular arithmetic is the foundation of public-key cryptography. ECDSA signatures involve operations like $(z + r \cdot \text{privateKey}) \cdot k_{\text{inverse}} \bmod n$ where n is the curve order. OpenSSL's `BN_div()` and `BN_mod()` handle arbitrary-precision division with full precision—no rounding errors that would break cryptographic operations.

```

460 {
461     CAutoBN_CTX pctx;
462     CBigNum r;
463     if (!BN_div(&r, NULL, &a, &b, pctx))
464         throw bignum_error("CBigNum::operator/ : BN_div failed");
465     return r;
466 }
467
468 inline const CBigNum operator%(const CBigNum& a, const CBigNum& b)
469 {
470     CAutoBN_CTX pctx;
471     CBigNum r;
472     if (!BN_mod(&r, &a, &b, pctx))
473         throw bignum_error("CBigNum::operator% : BN_div failed");
474     return r;
475 }
476
477 inline const CBigNum operator<<(const CBigNum& a, unsigned int shift)
478 {
479     CBigNum r;
480     if (!BN_lshift(&r, &a, shift))

```



```

481 throw bignum_error("CBigNum:operator<< : BN_lshift failed");
482 return r;
483 }
484
485 inline const CBigNum operator>>(const CBigNum& a, unsigned int shift)
486 {
487     CBigNum r;
488     if (!BN_rshift(&r, &a, shift))
489         throw bignum_error("CBigNum:operator>> : BN_rshift failed");
490     return r;
491 }
492
493 inline bool operator==(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) == 0); }

```

Comparison operators—enable intuitive comparisons like `hash <= target`. This comparison is the core of proof-of-work: a block is valid if its hash, interpreted as a 256-bit number, is less than or equal to the difficulty target. `BN_cmp()` returns -1, 0, or 1 like `strcmp()`. These operators wrap that into natural boolean expressions.

```

494 inline bool operator!=(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) != 0); }
495 inline bool operator<=(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) <= 0); }
496 inline bool operator>=(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) >= 0); }
497 inline bool operator<(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) < 0); }
498 inline bool operator>(const CBigNum& a, const CBigNum& b) { return (BN_cmp(&a, &b) > 0); }
499

```

SUMMARY

Big number arithmetic is the mathematical foundation of Bitcoin's security. Private keys are 256-bit random numbers; public keys are points on an elliptic curve computed via point multiplication (repeated addition of 256-bit coordinates); signatures involve modular inverse and multiplication. All of this requires numbers far larger than CPUs handle natively.

Satoshi's choice to wrap OpenSSL rather than implement custom big number code was wise. Cryptographic arithmetic is notoriously difficult to get right—subtle bugs can leak private keys through timing variations or incorrect carries. OpenSSL's BIGNUM library has been audited extensively and is used by software protecting critical infrastructure worldwide.

The compact difficulty format (`SetCompact/GetCompact`) deserves special note. By encoding 256-bit targets as 4 bytes, block headers stay small (80 bytes total), making lightweight clients practical. A phone can verify proof-of-work by checking just the header, without downloading full blocks. This compression was essential for Bitcoin's scalability vision.

db.cpp

605 lines CPP

MEMORY THAT SURVIVES

Bitcoin needs persistent storage that survives restarts: your wallet (private keys and transaction history), the blockchain index (where blocks are stored on disk), and peer addresses (nodes to reconnect to). Satoshi chose **Berkeley DB** (BDB), a mature embedded key-value database from Sleepycat Software (now Oracle) that had been battle-tested since 1991.

Why BDB? It offered ACID transactions (ensuring wallet operations either complete fully or not at all), required no separate server process, and was well-documented with C++ bindings. Satoshi explained on BitcoinTalk: "It uses a transactional database called Berkeley DB." He later noted optimization possibilities: "Someone should experiment with different Berkeley DB settings and see if there's something that makes the download substantially faster."

BDB served Bitcoin well until v0.8 (2013), when an obscure configuration limit became a consensus bug. BDB has a default limit on "database locks" (concurrent operations). A large but valid block exceeded this limit on some nodes but not others, causing a chain split. The incident—documented as BIP 50—forced an emergency coordinated response where miners intentionally orphaned the longer chain to maintain consensus.

This bug is forensically significant: it proved that consensus rules extend beyond the Bitcoin code to include the database engine, its configuration, the operating system, and even hardware. After the incident, the blockchain index migrated to Google's LevelDB—but the wallet still uses BDB today for backward compatibility with old wallet.dat files.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6
7
8
9
10
11
12 //
13 // CDB
14 //
15
16 static CCriticalSection cs_db;
17 static bool fDbEnvInit = false;
18 DbEnv dbenv(0);
19 static map<string, int> mapFileUseCount;
20
21 class CDBInit

```

CDBInit - initializes BDB environment. Sets up logging, locking, caching. One environment shared by all database files.

```

22 {
23 public:
24 CDBInit()
25 {
26 }
27 ~CDBInit()
28 {
29 if (fDbEnvInit)
30 {
31     dbenv.close(0);
32     fDbEnvInit = false;
33 }
34 }
35 }
36 instance_of_cdbinit;
37
38
39 CDB::CDB(const char* pszFile, const char* pszMode, bool fTxn) : pdb(NULL)
40 {
41 int ret;
42 if (pszFile == NULL)
43 return;
44
45 bool fCreate = strchr(pszMode, 'c');
46 bool fReadOnly = (!strchr(pszMode, '+') && !strchr(pszMode, 'w'));
47 unsigned int nFlags = DB_THREAD;
48 if (fCreate)
49     nFlags |= DB_CREATE;
50 else if (fReadOnly)
51     nFlags |= DB_RDONLY;
52 if (!fReadOnly || fTxn)
53     nFlags |= DB_AUTO_COMMIT;
54
55 CRITICAL_BLOCK(cs_db)
56 {
57 if (!fDbEnvInit)
58 {
59     string strAppDir = GetAppDir();
60     string strLogDir = strAppDir + "\\database";
61     _mkdir(strLogDir.c_str());
62     printf("dbenv.open strAppDir=%s\n", strAppDir.c_str());
63
64     dbenv.set_lg_dir(strLogDir.c_str());
65     dbenv.set_lg_max(10000000);

```

```

66         dbenv.set_lk_max_locks(10000);
67         dbenv.set_lk_max_objects(10000);
68         dbenv.set_errfile(fopen("db.log", "a")); /// debug
69 ///dbenv.log_set_config(DB_LOG_AUTO_REMOVE, 1); /// causes corruption
70         ret = dbenv.open(strAppDir.c_str(),
71                         DB_CREATE |
72                         DB_INIT_LOCK |
73                         DB_INIT_LOG |
74                         DB_INIT_MPOOL |
75                         DB_INIT_TXN |
76                         DB_THREAD |
77                         DB_PRIVATE |
78                         DB_RECOVER,
79         0);
80         if (ret > 0)
81             throw runtime_error(strprintf("CDB() : error %d opening database environment\n", ret));
82         fDbEnvInit = true;
83     }
84
85     strFile = pszFile;
86     ++mapFileUseCount[strFile];
87 }
88
89     pdb = new Db(&dbenv, 0);
90
91     ret = pdb->open(NULL, // Txn pointer
92                  pszFile, // Filename
93                  "main", // Logical db name
94                  DB_BTREE, // Database type
95                  nFlags, // Flags
96         0);
97
98     if (ret > 0)
99     {
100         delete pdb;
101         pdb = NULL;
102         CRITICAL_BLOCK(cs_db)
103             --mapFileUseCount[strFile];
104         strFile = "";
105         throw runtime_error(strprintf("CDB() : can't open database file %s, error %d\n", pszFile, ret));
106     }
107
108     if (fCreate && !Exists(string("version")))
109         WriteVersion(VERSION);
110
111     RandAddSeed();
112 }
113
114 void CDB::Close()

```

CDB::Close - safely closes database. Flushes pending writes, releases locks. Critical for data integrity on shutdown.

```

115 {
116     if (!pdb)
117         return;
118     if (!vTxn.empty())
119         vTxn.front()->abort();
120     vTxn.clear();
121     pdb->close(0);
122     delete pdb;
123     pdb = NULL;
124     dbenv.txn_checkpoint(0, 0, 0);
125
126     CRITICAL_BLOCK(cs_db)
127         --mapFileUseCount[strFile];
128
129     RandAddSeed();
130 }

```

```

131
132 void DBFlush(bool fShutdown)
133 {
134 // Flush log data to the actual data file
135 // on all files that are not in use
136 printf("DBFlush(%s)\n", fShutdown ? "true" : "false");
137 CRITICAL_BLOCK(cs_db)
138 {
139     dbenv.txn_checkpoint(0, 0, 0);
140     map<string, int>::iterator mi = mapFileUseCount.begin();
141     while (mi != mapFileUseCount.end())
142     {
143         string strFile = (*mi).first;
144         int nRefCount = (*mi).second;
145         if (nRefCount == 0)
146         {
147             dbenv.lsn_reset(strFile.c_str(), 0);
148             mapFileUseCount.erase(mi++);
149         }
150         else
151             mi++;
152     }
153     if (fShutdown)
154     {
155         char** listp;
156         if (mapFileUseCount.empty())
157             dbenv.log_archive(&listp, DB_ARCH_REMOVE);
158         dbenv.close(0);
159         fDbEnvInit = false;
160     }
161 }
162 }
163
164
165
166
167
168
169 //
170 // CTxDB
171 //
172
173 bool CTxDB::ReadTxIndex(uint256 hash, CTxIndex& txindex)

```

CTxDB::ReadTxIndex - looks up transaction location. Given txid, returns file position. Essential for validating inputs reference real transactions.

```

174 {
175     assert(!fClient);
176     txindex.SetNull();
177     return Read(make_pair(string("tx"), hash), txindex);
178 }
179
180 bool CTxDB::UpdateTxIndex(uint256 hash, const CTxIndex& txindex)
181 {
182     assert(!fClient);
183     return Write(make_pair(string("tx"), hash), txindex);
184 }
185
186 bool CTxDB::AddTxIndex(const CTransaction& tx, const CDiskTxPos& pos, int nHeight)
187 {
188     assert(!fClient);
189
190 // Add to tx index
191     uint256 hash = tx.GetHash();
192     CTxIndex txindex(pos, tx.vout.size());
193     return Write(make_pair(string("tx"), hash), txindex);
194 }
195

```

```

196 bool CTxDB::EraseTxIndex(const CTransaction& tx)
197 {
198     assert(!fClient);
199     uint256 hash = tx.GetHash();
200
201     return Erase(make_pair(string("tx"), hash));
202 }
203
204 bool CTxDB::ContainsTx(uint256 hash)
205 {
206     assert(!fClient);
207     return Exists(make_pair(string("tx"), hash));
208 }
209
210 bool CTxDB::ReadOwnerTxs(uint160 hash160, int nMinHeight, vector<CTransaction>& vtx)
211 {
212     assert(!fClient);
213     vtx.clear();
214
215     // Get cursor
216     Dbc* pcursor = GetCursor();
217     if (!pcursor)
218         return false;
219
220     unsigned int fFlags = DB_SET_RANGE;
221     loop
222     {
223         // Read next record
224         CDataStream ssKey;
225         if (fFlags == DB_SET_RANGE)
226             ssKey << string("owner") << hash160 << CDiskTxPos(0, 0, 0);
227         CDataStream ssValue;
228         int ret = ReadAtCursor(pcursor, ssKey, ssValue, fFlags);
229         fFlags = DB_NEXT;
230         if (ret == DB_NOTFOUND)
231             break;
232         else if (ret != 0)
233             return false;
234
235         // Unserialize
236         string strType;
237         uint160 hashItem;
238         CDiskTxPos pos;
239         ssKey >> strType >> hashItem >> pos;
240         int nItemHeight;
241         ssValue >> nItemHeight;
242
243         // Read transaction
244         if (strType != "owner" || hashItem != hash160)
245             break;
246         if (nItemHeight >= nMinHeight)
247         {
248             vtx.resize(vtx.size()+1);
249             if (!vtx.back().ReadFromDisk(pos))
250                 return false;
251         }
252     }
253     return true;
254 }
255
256 bool CTxDB::ReadDiskTx(uint256 hash, CTransaction& tx, CTxIndex& txindex)
257 {
258     assert(!fClient);
259     tx.SetNull();
260     if (!ReadTxIndex(hash, txindex))
261         return false;
262     return (tx.ReadFromDisk(txindex.pos));
263 }
264
265 bool CTxDB::ReadDiskTx(uint256 hash, CTransaction& tx)
266 {
267     CTxIndex txindex;

```

```

268 return ReadDiskTx(hash, tx, txindex);
269 }
270
271 bool CTxDB::ReadDiskTx(COutPoint outpoint, CTransaction& tx, CTxIndex& txindex)
272 {
273     return ReadDiskTx(outpoint.hash, tx, txindex);
274 }
275
276 bool CTxDB::ReadDiskTx(COutPoint outpoint, CTransaction& tx)
277 {
278     CTxIndex txindex;
279     return ReadDiskTx(outpoint.hash, tx, txindex);
280 }
281
282 bool CTxDB::WriteBlockIndex(const CDiskBlockIndex& blockindex)

```

WriteBlockIndex - stores block metadata. Height, hash, previous hash, merkle root. The index that makes blockchain traversal fast.

```

283 {
284     return Write(make_pair(string("blockindex"), blockindex.GetBlockHash()), blockindex);
285 }
286
287 bool CTxDB::EraseBlockIndex(uint256 hash)
288 {
289     return Erase(make_pair(string("blockindex"), hash));
290 }
291
292 bool CTxDB::ReadHashBestChain(uint256& hashBestChain)
293 {
294     return Read(string("hashBestChain"), hashBestChain);
295 }
296
297 bool CTxDB::WriteHashBestChain(uint256 hashBestChain)
298 {
299     return Write(string("hashBestChain"), hashBestChain);
300 }
301
302 CBlockIndex* InsertBlockIndex(uint256 hash)
303 {
304     if (hash == 0)
305         return NULL;
306
307     // Return existing
308     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hash);
309     if (mi != mapBlockIndex.end())
310         return (*mi).second;
311
312     // Create new

```

LoadBlockIndex - reconstructs chain state from disk. Called at startup to rebuild in-memory indexes from persisted data.

```

313     CBlockIndex* pindexNew = new CBlockIndex();
314     if (!pindexNew)
315         throw runtime_error("LoadBlockIndex() : new CBlockIndex failed");
316     mi = mapBlockIndex.insert(make_pair(hash, pindexNew)).first;
317     pindexNew->phashBlock = &((*mi).first);
318
319     return pindexNew;
320 }
321
322 bool CTxDB::LoadBlockIndex()
323 {
324     // Get cursor
325     Dbc* pcursor = GetCursor();

```



```

326 if (!pcursor)
327 return false;
328
329 unsigned int fFlags = DB_SET_RANGE;
330     loop
331 {
332 // Read next record
333     CDataStream ssKey;
334 if (fFlags == DB_SET_RANGE)
335     ssKey << make_pair(string("blockindex"), uint256(0));
336     CDataStream ssValue;
337 int ret = ReadAtCursor(pcursor, ssKey, ssValue, fFlags);
338     fFlags = DB_NEXT;
339 if (ret == DB_NOTFOUND)
340 break;
341 else if (ret != 0)
342 return false;
343
344 // Unserialize
345     string strType;
346     ssKey >> strType;
347 if (strType == "blockindex")
348 {
349         CDiskBlockIndex diskindex;
350         ssValue >> diskindex;
351
352 // Construct block index object
353         CBlockIndex* pindexNew = InsertBlockIndex(diskindex.GetBlockHash());
354         pindexNew->pprev         = InsertBlockIndex(diskindex.hashPrev);
355         pindexNew->pnext         = InsertBlockIndex(diskindex.hashNext);
356         pindexNew->nFile         = diskindex.nFile;
357         pindexNew->nBlockPos     = diskindex.nBlockPos;
358         pindexNew->nHeight       = diskindex.nHeight;
359         pindexNew->nVersion      = diskindex.nVersion;
360         pindexNew->hashMerkleRoot = diskindex.hashMerkleRoot;
361         pindexNew->nTime         = diskindex.nTime;
362         pindexNew->nBits         = diskindex.nBits;
363         pindexNew->nNonce        = diskindex.nNonce;
364
365 // Watch for genesis block and best block
366 if (pindexGenesisBlock == NULL && diskindex.GetBlockHash() == hashGenesisBlock)
367     pindexGenesisBlock = pindexNew;
368 }
369 else
370 {
371 break;
372 }
373 }
374
375 if (!ReadHashBestChain(hashBestChain))
376 {
377 if (pindexGenesisBlock == NULL)
378 return true;
379 return error("CTxDB::LoadBlockIndex() : hashBestChain not found\n");
380 }
381
382 if (!mapBlockIndex.count(hashBestChain))
383 return error("CTxDB::LoadBlockIndex() : blockindex for hashBestChain not found\n");
384     pindexBest = mapBlockIndex[hashBestChain];
385     nBestHeight = pindexBest->nHeight;
386 printf("LoadBlockIndex(): hashBestChain=%s height=%d\n", hashBestChain.ToString().substr(0,14).c_str(),
nBestHeight);
387
388 return true;
389 }
390
391
392
393
394
395 //
396 // CAddrDB
    
```

```

397 //
398
399 bool CAddrDB::WriteAddress(const CAddress& addr)
400 {
401     return Write(make_pair(string("addr"), addr.GetKey()), addr);
402 }
403
404 bool CAddrDB::LoadAddresses()
405 {
406     CRITICAL_BLOCK(cs_mapAddresses)
407     {
408         // Load user provided addresses
409         CAutoFile filein = fopen("addr.txt", "rt");
410         if (filein)
411         {
412             try
413             {
414                 char psz[1000];
415                 while (fgets(psz, sizeof(psz), filein))
416                 {
417                     CAddress addr(psz, NODE_NETWORK);
418                     if (addr.ip != 0)
419                         AddAddress(*this, addr);
420                 }
421             }
422             catch (...) { }
423         }
424
425         // Get cursor
426         Dbc* pcursor = GetCursor();
427         if (!pcursor)
428             return false;
429
430         loop
431         {
432             // Read next record
433             CDataStream ssKey;
434             CDataStream ssValue;
435             int ret = ReadAtCursor(pcursor, ssKey, ssValue);
436             if (ret == DB_NOTFOUND)
437                 break;
438             else if (ret != 0)
439                 return false;
440
441             // Unserialize
442             string strType;
443             ssKey >> strType;
444             if (strType == "addr")
445             {
446                 CAddress addr;
447                 ssValue >> addr;
448                 mapAddresses.insert(make_pair(addr.GetKey(), addr));
449             }
450         }
451
452         /// debug print
453         printf("mapAddresses:\n");
454         foreach(const PAIRTYPE(vector<unsigned char>, CAddress)& item, mapAddresses)
455             item.second.print();
456         printf("-----\n");
457     }
458
459     return true;
460 }
461
462 bool LoadAddresses()
463 {
464     return CAddrDB("cr+").LoadAddresses();
465 }
466
467
468

```

```

469
470 //
471 // CReviewDB
472 //
473
474 bool CReviewDB::ReadReviews(uint256 hash, vector<CReview>& vReviews)
475 {
476     vReviews.size(); // msvc workaround, just need to do anything with vReviews
477     return Read(make_pair(string("reviews"), hash), vReviews);
478 }
479
480 bool CReviewDB::WriteReviews(uint256 hash, const vector<CReview>& vReviews)
481 {
482     return Write(make_pair(string("reviews"), hash), vReviews);
483 }
484
485
486
487
488
489
490
491 //
492 // CWalletDB
493 //
494
495 bool CWalletDB::LoadWallet(vector<unsigned char>& vchDefaultKeyRet)
496 {
497     vchDefaultKeyRet.clear();
498
499     //// todo: shouldn't we catch exceptions and try to recover and continue?
500     CRITICAL_BLOCK(cs_mapKeys)
501     CRITICAL_BLOCK(cs_mapWallet)
502     {
503         // Get cursor
504         Dbc* pcursor = GetCursor();
505         if (!pcursor)
506             return false;
507
508         loop
509         {
510             // Read next record
511             CDataStream ssKey;
512             CDataStream ssValue;
513             int ret = ReadAtCursor(pcursor, ssKey, ssValue);
514             if (ret == DB_NOTFOUND)
515                 break;
516             else if (ret != 0)
517                 return false;
518
519             // Unserialize
520             // Taking advantage of the fact that pair serialization
521             // is just the two items serialized one after the other
522             string strType;
523             ssKey >> strType;
524             if (strType == "name")
525             {
526                 string strAddress;
527                 ssKey >> strAddress;
528                 ssValue >> mapAddressBook[strAddress];
529             }
530             else if (strType == "tx")
531             {
532                 uint256 hash;
533                 ssKey >> hash;
534                 CWalletTx& wtx = mapWallet[hash];
535                 ssValue >> wtx;
536
537                 if (wtx.GetHash() != hash)
538                     printf("Error in wallet.dat, hash mismatch\n");
539
540             }
541             //// debug print

```

```

541 //printf("LoadWallet %s\n", wtx.GetHash().ToString().c_str());
542 //printf(" %I2I64d %s %s %s\n",
543 //      wtx.vout[0].nValue,
544 //      DateTimeStr(wtx.nTime).c_str(),
545 //      wtx.hashBlock.ToString().substr(0,14).c_str(),
546 //      wtx.mapValue["message"].c_str());
547 }
548 else if (strType == "key")
549 {
550     vector<unsigned char> vchPubKey;
551     ssKey >> vchPubKey;
552     CPrivKey vchPrivKey;
553     ssValue >> vchPrivKey;
554
555     mapKeys[vchPubKey] = vchPrivKey;
556     mapPubKeys[Hash160(vchPubKey)] = vchPubKey;
557 }
558 else if (strType == "defaultkey")
559 {
560     ssValue >> vchDefaultKeyRet;
561 }
562 else if (strType == "setting") /// or settings or option or options or config?
563 {
564     string strKey;
565     ssKey >> strKey;
566 if (strKey == "fGenerateBitcoins") ssValue >> fGenerateBitcoins;
567 if (strKey == "nTransactionFee") ssValue >> nTransactionFee;
568 if (strKey == "addrIncoming") ssValue >> addrIncoming;
569 }
570 }
571 }
572
573 printf("fGenerateBitcoins = %d\n", fGenerateBitcoins);
574 printf("nTransactionFee = %I64d\n", nTransactionFee);
575 printf("addrIncoming = %s\n", addrIncoming.ToString().c_str());
576
577 return true;
578 }
579
580 bool LoadWallet()
581 {
582     vector<unsigned char> vchDefaultKey;
583 if (!CWalletDB("cr").LoadWallet(vchDefaultKey))
584 return false;
585
586 if (mapKeys.count(vchDefaultKey))
587 {
588 // Set keyUser
589     keyUser.SetPubKey(vchDefaultKey);
590     keyUser.SetPrivKey(mapKeys[vchDefaultKey]);
591 }
592 else
593 {
594 // Create new keyUser and set as default key
595     keyUser.MakeNewKey();
596 if (!AddKey(keyUser))
597 return false;
598 if (!SetAddressBookName(PubKeyToAddress(keyUser.GetPubKey()), "Your Address"))
599 return false;
600 CWalletDB().WriteDefaultKey(keyUser.GetPubKey());
601 }
602
603 return true;
604 }
605

```



SUMMARY

The BDB consensus bug (BIP 50) is forensically significant: Bitcoin v0.8's LevelDB had no lock limits, while BDB nodes with default settings couldn't validate a large (but valid) block. The network temporarily split. Resolution required coordinating miners to intentionally orphan the longer chain.

This incident proved that consensus extends beyond code—it includes the data-base layer, operating system, and even hardware. Satoshi's original BDB choice was pragmatic but introduced hidden consensus constraints.

db.h

421 lines CPP

THE PERSISTENCE LAYER

This header defines the database classes and interfaces used throughout Bitcoin. It declares how wallets, addresses, and blockchain data are structured and stored using Berkeley DB—a mature, embedded key-value database that provides ACID transactions, crash recovery, and concurrent access.

Satoshi chose Berkeley DB for its reliability and simplicity. Unlike client-server databases (MySQL, PostgreSQL), Berkeley DB runs in-process with no network overhead. It's battle-tested in mission-critical systems and provides the transaction guarantees Bitcoin needs: if you crash mid-write, the database recovers to a consistent state on restart.

The database files created here are your Bitcoin node's permanent memory: `wallet.dat` holds your private keys and transaction history, `blkindex.dat` indexes the blockchain for fast lookup, and `addr.dat` remembers peer addresses for reconnection. Losing `wallet.dat` means losing your coins—this file IS your money.

The class hierarchy (CDB as base, specialized subclasses for each file) provides type-safe access patterns. You can't accidentally write a transaction to the address database—the type system prevents it. This defensive design has helped prevent data corruption bugs.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include <db_cxx.h>
6 class CTransaction;
7 class CTxIndex;
8 class CDiskBlockIndex;
9 class CDiskTxPos;
10 class COutPoint;
11 class CUser;
12 class CReview;
13 class CAddress;
14 class CWalletTx;
15
16 extern map<string, string> mapAddressBook;

```

`mapAddressBook`—your address labels, mapping Bitcoin addresses to human-readable names. When you label an address as 'Coinbase', 'Salary', etc., this map stores it. Defined extern here, implemented elsewhere, and persisted via `CWalletDB`.

```

17 extern bool fClient;
18
19
20 extern DbEnv dbenv;

```

`dbenv`—the Berkeley DB environment, shared by all database files. The environment provides: transaction coordination across multiple databases, shared memory pool for caching, logging for crash recovery. All Bitcoin databases open within this single environment, enabling atomic operations that span multiple files.

```

21 extern void DBFlush(bool fShutdown);
22
23
24
25
26 class CDB

```

`CDB`—the base class for all database operations. Wraps a Berkeley DB handle (`Db* pdb`) with C++ conveniences: RAII-style open/close, template methods for type-safe read/write, transaction support. All the specialized database classes (`CTxDB`, `CWalletDB`, etc.) inherit from this, getting consistent error handling and resource management.

```

27 {
28 protected:
29     Db* pdb;
30     string strFile;
31     vector<DbTxn*> vTxn;
32
33 explicit CDB(const char* pszFile, const char* pszMode="r+", bool fTxn=false);

```

Constructor takes filename and mode (`'r'`, `'r+'`, `'w'`). The `fTxn` parameter enables transactional writes—slower but crash-safe. For reading, transactions aren't needed. For critical writes (new keys, spent coins), transactions ensure atomicity: either the entire write succeeds, or nothing changes.

```

34 ~CDB() { Close(); }
35 public:
36 void Close();
37 private:

```



```

38 CDB(const CDB&);
39 void operator=(const CDB&);
40
41 protected:
42 template<typename K, typename T>

```

Read() —generic key-value read. Takes any serializable key type `K` and value type `T`. The key is serialized to bytes, looked up in Berkeley DB, and the value is deserialized back into type `T`. The template magic means you can write `Read(txid, transaction)` with full type safety—if types don't match the stored data, you get a compile error.

```

43 bool Read(const K& key, T& value)
44 {
45     if (!pdb)
46         return false;
47
48     // Key
49     CDataStream ssKey(SER_DISK);
50     ssKey.reserve(1000);
51     ssKey << key;
52     Dbt datKey(&ssKey[0], ssKey.size());
53
54     // Read
55     Dbt datValue;
56     datValue.set_flags(DB_DBT_MALLOC);
57     int ret = pdb->get(GetTxn(), &datKey, &datValue, 0);
58     memset(datKey.get_data(), 0, datKey.get_size());

```

Memory security: `memset(datKey.get_data(), 0, datKey.get_size())`. After reading, the key and value buffers are zeroed. This prevents sensitive data (like private keys) from lingering in freed memory. An attacker with memory access couldn't find keys in deallocated buffers. This paranoid cleanup appears throughout Bitcoin's key-handling code.

```

59 if (datValue.get_data() == NULL)
60     return false;
61
62 // Unserialize value
63 CDataStream ssValue((char*)datValue.get_data(), (char*)datValue.get_data() +
    datValue.get_size(), SER_DISK);
64 ssValue >> value;
65
66 // Clear and free memory
67 memset(datValue.get_data(), 0, datValue.get_size());
68 free(datValue.get_data());
69 return (ret == 0);
70 }
71
72 template<typename K, typename T>

```

Write() —generic key-value write. Serializes key and value, writes to database. The `fOverwrite` parameter controls behavior if key exists: `true` = replace, `false` = fail with `DB_NOOVERWRITE`. The comment 'Clear memory in case it was a private key' explains the final `memset()` —defense in depth against memory snooping.

```

73 bool Write(const K& key, const T& value, bool fOverwrite=true)
74 {
75     if (!pdb)
76         return false;
77
78     // Key
79     CDataStream ssKey(SER_DISK);
80     ssKey.reserve(1000);

```

```

81         ssKey << key;
82         Dbt datKey(&ssKey[0], ssKey.size());
83
84 // Value
85         CDataStream ssValue(SER_DISK);
86         ssValue.reserve(10000);
87         ssValue << value;
88         Dbt datValue(&ssValue[0], ssValue.size());
89
90 // Write
91 int ret = pdb->put(GetTxn(), &datKey, &datValue, (fOverwrite ? 0 : DB_NOOVERWRITE));

```

Berkeley DB flags: `DB_NOOVERWRITE` causes writes to fail if the key exists. Used when adding new keys—you don't want to accidentally overwrite an existing key with a different value. The return code distinguishes 'success' from 'key existed', letting callers handle each case appropriately.

```

92
93 // Clear memory in case it was a private key
94 memset(datKey.get_data(), 0, datKey.get_size());
95 memset(datValue.get_data(), 0, datValue.get_size());
96 return (ret == 0);
97 }
98
99 template<typename K>

```

`Erase()`—removes a key-value pair. Used when spending coins (removing UTXOs) or cleaning up orphan data. Returns `true` even if key didn't exist (`DB_NOTFOUND` is not an error for deletion). The key is zeroed after use for security.

```

100 bool Erase(const K& key)
101 {
102     if (!pdb)
103         return false;
104
105 // Key
106     CDataStream ssKey(SER_DISK);
107     ssKey.reserve(1000);
108     ssKey << key;
109     Dbt datKey(&ssKey[0], ssKey.size());
110
111 // Erase
112 int ret = pdb->del(GetTxn(), &datKey, 0);
113
114 // Clear memory
115 memset(datKey.get_data(), 0, datKey.get_size());
116 return (ret == 0 || ret == DB_NOTFOUND);
117 }
118
119 template<typename K>
120 bool Exists(const K& key)
121 {
122     if (!pdb)
123         return false;
124
125 // Key
126     CDataStream ssKey(SER_DISK);
127     ssKey.reserve(1000);
128     ssKey << key;
129     Dbt datKey(&ssKey[0], ssKey.size());
130
131 // Exists
132 int ret = pdb->exists(GetTxn(), &datKey, 0);
133
134 // Clear memory
135 memset(datKey.get_data(), 0, datKey.get_size());
136 return (ret == 0);

```

```

137 }
138
139 Dbc* GetCursor()

```

GetCursor() —creates a database cursor for iteration. Cursors allow walking through all key-value pairs, essential for operations like 'load all transactions' or 'find all UTXOs for an address'. The cursor maintains position across multiple reads, enabling efficient sequential access.

```

140 {
141     if (!pdb)
142         return NULL;
143     Dbc* pcursor = NULL;
144     int ret = pdb->cursor(NULL, &pcursor, 0);
145     if (ret != 0)
146         return NULL;
147     return pcursor;
148 }
149
150 int ReadAtCursor(Dbc* pcursor, CDataStream& ssKey, CDataStream& ssValue, unsigned int fFlags=DB_NEXT)

```

ReadAtCursor() —reads the next record via cursor. The flags parameter controls behavior: **DB_NEXT** for sequential scan, **DB_SET** for exact key lookup, **DB_SET_RANGE** for 'find first key >= this'. Returns key and value as **CDataStream**s for flexible deserialization. Used during wallet loading to iterate through all stored transactions.

```

151 {
152     // Read at cursor
153     Dbt datKey;
154     if (fFlags == DB_SET || fFlags == DB_SET_RANGE || fFlags == DB_GET_BOTH || fFlags == DB_GET_BOTH_RANGE)
155     {
156         datKey.set_data(&ssKey[0]);
157         datKey.set_size(ssKey.size());
158     }
159     Dbt datValue;
160     if (fFlags == DB_GET_BOTH || fFlags == DB_GET_BOTH_RANGE)
161     {
162         datValue.set_data(&ssValue[0]);
163         datValue.set_size(ssValue.size());
164     }
165     datKey.set_flags(DB_DBT_MALLOC);
166     datValue.set_flags(DB_DBT_MALLOC);
167     int ret = pcursor->get(&datKey, &datValue, fFlags);
168     if (ret != 0)
169         return ret;
170     else if (datKey.get_data() == NULL || datValue.get_data() == NULL)
171         return 99999;
172
173     // Convert to streams
174     ssKey.SetType(SER_DISK);
175     ssKey.clear();
176     ssKey.write((char*)datKey.get_data(), datKey.get_size());
177     ssValue.SetType(SER_DISK);
178     ssValue.clear();
179     ssValue.write((char*)datValue.get_data(), datValue.get_size());
180
181     // Clear and free memory
182     memset(datKey.get_data(), 0, datKey.get_size());
183     memset(datValue.get_data(), 0, datValue.get_size());
184     free(datKey.get_data());
185     free(datValue.get_data());
186     return 0;
187 }
188
189 DbTxn* GetTxn()

```

`GetTxn()` —returns current transaction, if any. Berkeley DB transactions provide atomicity: all writes within a transaction succeed or fail together. The `vTxn` vector supports nested transactions—you can start a sub-transaction within a transaction. Not heavily used in v0.1, but the infrastructure exists.

```

190 {
191     if (!vTxn.empty())
192         return vTxn.back();
193     else
194         return NULL;
195 }
196
197 public:
198 bool TxnBegin()

```

Transaction methods: `TxnBegin()`, `TxnCommit()`, `TxnAbort()`. These wrap Berkeley DB's transaction API. Begin starts a transaction, recording a 'checkpoint'. Commit makes changes permanent. Abort rolls back to the checkpoint. If Bitcoin crashes between Begin and Commit, recovery automatically aborts the incomplete transaction—no corrupted state.

```

199 {
200     if (!pdb)
201         return false;
202     DbTxn* ptxn = NULL;
203     int ret = dbenv.txn_begin(GetTxn(), &ptxn, 0);
204     if (!ptxn || ret != 0)
205         return false;
206     vTxn.push_back(ptxn);
207     return true;
208 }
209
210 bool TxnCommit()
211 {
212     if (!pdb)
213         return false;
214     if (vTxn.empty())
215         return false;
216     int ret = vTxn.back()->commit(0);
217     vTxn.pop_back();
218     return (ret == 0);
219 }
220
221 bool TxnAbort()
222 {
223     if (!pdb)
224         return false;
225     if (vTxn.empty())
226         return false;
227     int ret = vTxn.back()->abort();
228     vTxn.pop_back();
229     return (ret == 0);
230 }
231
232 bool ReadVersion(int& nVersion)

```

`ReadVersion()` / `WriteVersion()` —database schema versioning. As Bitcoin evolves, database formats may change. The version number lets code detect old formats and migrate them. If you open a v0.1 wallet with later software, it knows to upgrade the structure. Forward compatibility planning from day one.

```

233 {
234     nVersion = 0;

```

```

235 return Read(string("version"), nVersion);
236 }
237
238 bool WriteVersion(int nVersion)
239 {
240     return Write(string("version"), nVersion);
241 }
242 };
243
244
245
246
247
248
249
250
251 class CTxDB : public CDB

```

CTxDB—the blockchain index database (`blkindex.dat`). Maps transaction hashes to their location on disk, block hashes to block metadata, and tracks the best chain. This index is what makes blockchain queries fast: rather than scanning every block file, look up the index. If `fClient` is `true` (lightweight mode), this database isn't used.

```

252 {
253 public:
254     CTxDB(const char* pszMode="r+", bool fTxn=false) : CDB(!fClient ? "blkindex.dat" : NULL, pszMode, fTxn)
255     { }
256 private:
257     CTxDB(const CTxDB&);
258     void operator=(const CTxDB&);
259 public:
260     bool ReadTxIndex(uint256 hash, CTxIndex& txindex);

```

CTxDB methods: `ReadTxIndex()`, `UpdateTxIndex()`, `AddTxIndex()`, `EraseTxIndex()`. These maintain the UTXO set—the record of all unspent transaction outputs. When a transaction spends coins, `EraseTxIndex()` removes the spent outputs. When a new transaction is confirmed, `AddTxIndex()` records its outputs as spendable. This is Bitcoin's notion of 'who owns what.'

```

260 bool UpdateTxIndex(uint256 hash, const CTxIndex& txindex);
261 bool AddTxIndex(const CTransaction& tx, const CDiskTxPos& pos, int nHeight);
262 bool EraseTxIndex(const CTransaction& tx);
263 bool ContainsTx(uint256 hash);
264 bool ReadOwnerTxes(uint160 hash160, int nHeight, vector<CTransaction>& vtx);
265 bool ReadDiskTx(uint256 hash, CTransaction& tx, CTxIndex& txindex);
266 bool ReadDiskTx(uint256 hash, CTransaction& tx);
267 bool ReadDiskTx(COutPoint outpoint, CTransaction& tx, CTxIndex& txindex);
268 bool ReadDiskTx(COutPoint outpoint, CTransaction& tx);
269 bool WriteBlockIndex(const CDiskBlockIndex& blockindex);

```

`WriteBlockIndex()` / `EraseBlockIndex()`—maintain the block chain structure. Each `CDiskBlockIndex` records: block hash, previous block hash, height, timestamp, difficulty. This lets the node quickly determine if a block is in the best chain, find a block's ancestors, and identify chain reorganizations.

```

270 bool EraseBlockIndex(uint256 hash);
271 bool ReadHashBestChain(uint256& hashBestChain);

```

`ReadHashBestChain()` —retrieves the current chain tip. On startup, the node reads this to know which chain to continue building on. It's the single most important value in the database: 'what does this node consider the canonical blockchain history?'

```

272 bool WriteHashBestChain(uint256 hashBestChain);
273 bool LoadBlockIndex();
274 };
275
276
277
278
279
280 class CReviewDB : public CDB

```

`CReviewDB` —Satoshi's marketplace feature. Stores user reviews and ratings in `reviews.dat`. This hints at a more ambitious vision where Bitcoin supported reputation systems for commerce. The feature was never fully developed and was removed in later versions, but the infrastructure shows Satoshi thinking about trust in pseudonymous commerce.

```

281 {
282 public:
283 CReviewDB(const char* pszMode="r+", bool fTxn=false) : CDB("reviews.dat", pszMode, fTxn) { }
284 private:
285 CReviewDB(const CReviewDB&);
286 void operator=(const CReviewDB&);
287 public:
288 bool ReadUser(uint256 hash, CUser& user)
289 {
290     return Read(make_pair(string("user"), hash), user);
291 }
292
293 bool WriteUser(uint256 hash, const CUser& user)
294 {
295     return Write(make_pair(string("user"), hash), user);
296 }
297
298 bool ReadReviews(uint256 hash, vector<CReview>& vReviews);
299 bool WriteReviews(uint256 hash, const vector<CReview>& vReviews);
300 };
301
302
303
304
305
306 class CMarketDB : public CDB

```

`CMarketDB` —another marketplace remnant. Would have stored product listings in `market.dat`. Combined with `CReviewDB`, this would have enabled decentralized e-commerce: list products, accumulate reviews, transact in bitcoin. The vision never materialized, but these database stubs remain as archaeological evidence.

```

307 {
308 public:
309 CMarketDB(const char* pszMode="r+", bool fTxn=false) : CDB("market.dat", pszMode, fTxn) { }
310 private:
311 CMarketDB(const CMarketDB&);
312 void operator=(const CMarketDB&);
313 };
314
315
316

```

```

317
318
319 class CAddrDB : public CDB

```

CAddrDB—peer address storage (`addr.dat`). Remembers IP addresses of other Bitcoin nodes. On startup, the node loads these addresses and attempts reconnection. This provides persistence across restarts: you don't need to rediscover the network from scratch each time. Combined with IRC bootstrap and address gossip, ensures network resilience.

```

320 {
321     public:
322     CAddrDB(const char* pszMode="r+", bool fTxn=false) : CDB("addr.dat", pszMode, fTxn) { }
323     private:
324     CAddrDB(const CAddrDB&);
325     void operator=(const CAddrDB&);
326     public:
327     bool WriteAddress(const CAddress& addr);
328     bool LoadAddresses();
329 };
330
331 bool LoadAddresses();
332
333
334
335
336
337 class CWalletDB : public CDB

```

CWalletDB—your money (`wallet.dat`). This is the most security-critical database. Stores: private keys (your ability to spend), transaction history (your record of what you own), address labels (your organizational metadata), and settings. Losing this file with no backup = permanent loss of all your bitcoin. There is no recovery mechanism.

```

338 {
339     public:
340     CWalletDB(const char* pszMode="r+", bool fTxn=false) : CDB("wallet.dat", pszMode, fTxn) { }
341     private:
342     CWalletDB(const CWalletDB&);
343     void operator=(const CWalletDB&);
344     public:
345     bool ReadName(const string& strAddress, string& strName)

```

`ReadName()` / `WriteName()`—address book operations. The key is the address string; the value is your label. `WriteName` also updates the in-memory `mapAddressBook` for fast access. These let you associate human meaning with cryptographic addresses: '1ABC...' becomes 'Mom's birthday gift.'

```

346 {
347     strName = "";
348     return Read(make_pair(string("name"), strAddress), strName);
349 }
350
351 bool WriteName(const string& strAddress, const string& strName)
352 {
353     mapAddressBook[strAddress] = strName;
354     return Write(make_pair(string("name"), strAddress), strName);
355 }
356
357 bool EraseName(const string& strAddress)
358 {
359     mapAddressBook.erase(strAddress);
360     return Erase(make_pair(string("name"), strAddress));
361 }

```

```

362
363 bool ReadTx(uint256 hash, CWalletTx& wtx)

```

`ReadTx()` / `WriteTx()` / `EraseTx()` —transaction storage. Every transaction involving your addresses is stored here: payments received, payments sent, change outputs. The key is the transaction hash (txid); the value is the full `CWalletTx` (transaction plus wallet-specific metadata like 'from address book entry').

```

364 {
365     return Read(make_pair(string("tx"), hash), wtx);
366 }
367
368 bool WriteTx(uint256 hash, const CWalletTx& wtx)
369 {
370     return Write(make_pair(string("tx"), hash), wtx);
371 }
372
373 bool EraseTx(uint256 hash)
374 {
375     return Erase(make_pair(string("tx"), hash));
376 }
377
378 bool ReadKey(const vector<unsigned char>& vchPubKey, CPrivKey& vchPrivKey)

```

`ReadKey()` / `WriteKey()` —private key storage. This is where the actual secrets live. The key is the public key bytes; the value is the encrypted private key (`CPrivKey`). Note `WriteKey()` uses `fOverwrite=false` —you never want to accidentally overwrite an existing key with a different value, as that would destroy the original key.

```

379 {
380     vchPrivKey.clear();
381     return Read(make_pair(string("key"), vchPubKey), vchPrivKey);
382 }
383
384 bool WriteKey(const vector<unsigned char>& vchPubKey, const CPrivKey& vchPrivKey)
385 {
386     return Write(make_pair(string("key"), vchPubKey), vchPrivKey, false);
387 }
388
389 bool ReadDefaultKey(vector<unsigned char>& vchPubKey)

```

`ReadDefaultKey()` / `WriteDefaultKey()` —the 'receiving address' shown in the UI. When someone wants to pay you, you give them this address. It's just one of your keys, designated as the default. You can change it anytime by calling `WriteDefaultKey` with a different key.

```

390 {
391     vchPubKey.clear();
392     return Read(string("defaultkey"), vchPubKey);
393 }
394
395 bool WriteDefaultKey(const vector<unsigned char>& vchPubKey)
396 {
397     return Write(string("defaultkey"), vchPubKey);
398 }
399
400 template<typename T>

```

`ReadSetting()` / `WriteSetting()` —generic settings storage. User preferences (like 'start minimized', 'proxy server address') are stored as key-value pairs with the 'setting' prefix. The template methods accept any serializable type, providing type-safe settings without proliferating `ReadXxxSetting` methods.

```

401 bool ReadSetting(const string& strKey, T& value)
402 {
403     return Read(make_pair(string("setting"), strKey), value);
404 }
405
406 template<typename T>
407 bool WriteSetting(const string& strKey, const T& value)
408 {
409     return Write(make_pair(string("setting"), strKey), value);
410 }
411
412 bool LoadWallet(vector<unsigned char>& vchDefaultKeyRet);

```

`LoadWallet()` —the critical startup function. Reads `wallet.dat`, populating in-memory structures with all keys and transactions. Returns the default receiving address. If this function fails (corrupted database, wrong password for encrypted wallets), the node cannot access its funds. The `vector<unsigned char>` return is the default public key.

```

413 };
414
415 bool LoadWallet();
416
417 inline bool SetAddressBookName(const string& strAddress, const string& strName)

```

`SetAddressBookName()` —convenience wrapper for labeling addresses. Creates a temporary `CWalletDB` just to write one name, then closes it. This pattern appears throughout Bitcoin: open database, do one operation, close. It's simpler than managing long-lived database handles but has some performance overhead.

```

418 {
419     return CWalletDB().WriteName(strAddress, strName);
420 }
421

```

SUMMARY

The database layer is Bitcoin's long-term memory. Without it, every restart would mean losing your transaction history, forgetting peer addresses, and most critically, losing access to your private keys. The careful separation into multiple files (`wallet.dat`, `blkindex.dat`, `addr.dat`) isolates concerns: you can back up just `wallet.dat` to protect your funds, without including the multi-gigabyte blockchain index.

Berkeley DB's transactional guarantees are essential for financial software. If your computer crashes while Bitcoin is updating the UTXO set, you don't want to end up with inconsistent state where coins appear both spent and unspent. The transaction mechanism ensures atomicity: either the entire update succeeds, or the database rolls back to the previous consistent state.

The `wallet.dat` file deserves special attention. Unlike most application data, it's irreplaceable—there's no 'forgot password' recovery, no customer service to restore your account. The private keys stored here ARE your bitcoin. Satoshi's code treats this file with appropriate paranoia: memory wiping, careful writes, transaction protection. Modern wallets add encryption and hardware security modules, but the fundamental truth remains: guard your keys.

headers.h

72 lines CPP

THE GATHERING OF INCLUDES

A convenience header that includes all the commonly needed system headers and Bitcoin headers in one place. Most source files include this rather than listing dozens of individual includes.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #ifdef _MSC_VER
6 #pragma warning(disable:4786)
7 #pragma warning(disable:4804)
8 #pragma warning(disable:4717)
9 #endif
10 #ifdef _WIN32_WINNT
11 #undef _WIN32_WINNT
12 #endif
13 #define _WIN32_WINNT 0x0400
14 #define WIN32_LEAN_AND_MEAN 1
15 #include <wx/wx.h>

```

wxWidgets - the GUI framework used by Bitcoin

```

16 #include <wx/clipbrd.h>
17 #include <wx/snglinst.h>
18 #include <openssl/ecdsa.h>

```

OpenSSL includes - cryptographic functions

```

19 #include <openssl/evp.h>
20 #include <openssl/rand.h>
21 #include <openssl/sha.h>
22 #include <openssl/ripemd.h>
23 #include <windows.h>

```

Windows networking headers for sockets

```

24 #include <winsock2.h>
25 #include <mswsock.h>
26 #include <stdio.h>
27 #include <stdlib.h>
28 #include <io.h>
29 #include <math.h>
30 #include <limits.h>
31 #include <float.h>
32 #include <assert.h>
33 #include <process.h>
34 #include <malloc.h>
35 #include <memory>
36 #define BOUNDSCHECK 1
37 #include <sstream>
38 #include <string>
39 #include <vector>
40 #include <list>
41 #include <deque>
42 #include <map>
43 #include <set>
44 #include <algorithm>
45 #include <numeric>
46 #include <boost/foreach.hpp>
47 #include <boost/lexical_cast.hpp>
48 #include <boost/tuple/tuple.hpp>
49 #include <boost/tuple/tuple_comparison.hpp>
50 #include <boost/tuple/tuple_io.hpp>
51 #include <boost/array.hpp>
52 #pragma hdrstop
53 using namespace std;
54 using namespace boost;
55
56

```

```
57
58 #include "serialize.h"
59 #include "uint256.h"
60 #include "util.h"
61 #include "key.h"
62 #include "bignum.h"
63 #include "base58.h"
64 #include "script.h"
65 #include "db.h"
66 #include "net.h"
67 #include "irc.h"
68 #include "main.h"
69 #include "market.h"
70 #include "uibase.h"
71 #include "ui.h"
72
```

SUMMARY

This was a common C++ pattern in 2009 to simplify compilation and ensure consistent include ordering across the project.

irc.cpp

266 lines CPP

FINDING FRIENDS IN THE DARK

How does a new Bitcoin node find other nodes to connect to? This is the "bootstrap problem"—a chicken-and-egg dilemma that every peer-to-peer network faces. Satoshi's clever solution: use IRC (Internet Relay Chat), a protocol from 1988 that was already running on well-known, stable servers worldwide.

The mechanism is elegant in its simplicity: Bitcoin nodes connect to `irc.lfnet.org`, join channel `#bitcoin`, and encode their IP address in their IRC nickname. By requesting the WHO list, any node can discover peers. No central Bitcoin server needed—just parasitic use of existing infrastructure.

Satoshi on this design (BitcoinTalk, 2010): "We only needed IRC because nobody had a static IP. In the early days there were some steady supporters, but they all had pool-allocated IPs that change every few days. IRC was only intended as a temporary solution." He added: "Bitcoin's built-in `addr` system is the main solution" and "We don't want to overstay our welcome. If we get too big...we'll get off."

This code reveals Satoshi's pragmatism: rather than building custom infrastructure, he leveraged what already existed. IRC servers were free, reliable, and firewall-friendly (port 6667 was commonly allowed). The temporary solution worked perfectly for Bitcoin's first four years until the network was large enough that `addr` propagation alone sufficed for peer discovery.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6
7
8
9
10 #pragma pack(1)
11 struct ircaddr
12 {
13     int ip;
14     short port;
15 };
16
17 string EncodeAddress(const CAddress& addr)
18 {
19     struct ircaddr tmp;
20     tmp.ip = addr.ip;
21     tmp.port = addr.port;
22
23     vector<unsigned char> vch(UBEGIN(tmp), UEND(tmp));
24     return string("u") + EncodeBase58Check(vch);
25 }
26
27 bool DecodeAddress(string str, CAddress& addr)
28 {
29     vector<unsigned char> vch;
30     if (!DecodeBase58Check(str.substr(1), vch))
31         return false;
32
33     struct ircaddr tmp;
34     if (vch.size() != sizeof(tmp))
35         return false;
36     memcpy(&tmp, &vch[0], sizeof(tmp));
37
38     addr = CAddress(tmp.ip, tmp.port);
39     return true;
40 }
41
42
43
44
45
46
47 static bool Send(SOCKET hSocket, const char* pszSend)
48 {
49     if (strstr(pszSend, "PONG") != pszSend)
50         printf("SENDING: %s\n", pszSend);
51     const char* psz = pszSend;
52     const char* pszEnd = psz + strlen(psz);
53     while (psz < pszEnd)
54     {
55         int ret = send(hSocket, psz, pszEnd - psz, 0);
56         if (ret < 0)
57             return false;
58         psz += ret;
59     }
60     return true;
61 }
62
63 bool RecvLine(SOCKET hSocket, string& strLine)
64 {
65     strLine = "";
66     loop
67     {
68         char c;
69         int nBytes = recv(hSocket, &c, 1, 0);
70         if (nBytes > 0)
71         {
72             if (c == '\n')

```



```

73 continue;
74 if (c == '\r')
75 return true;
76     strLine += c;
77 }
78 else if (nBytes <= 0)
79 {
80 if (!strLine.empty())
81 return true;
82 // socket closed
83 printf("IRC socket closed\n");
84 return false;
85 }
86 else
87 {
88 // socket error
89 int nErr = WSAGetLastError();
90 if (nErr != WSAEMSGSIZE && nErr != WSAEINTR && nErr != WSAEINPROGRESS)
91 {
92 printf("IRC recv failed: %d\n", nErr);
93 return false;
94 }
95 }
96 }
97 }
98
99 bool RecvLineIRC(SOCKET hSocket, string& strLine)

```

RecvLineIRC - reads one line from IRC socket. Handles the text-based IRC protocol. Simple but effective for bootstrap purposes.

```

100 {
101     loop
102 {
103 bool fRet = RecvLine(hSocket, strLine);
104 if (fRet)
105 {
106 if (fShutdown)
107 return false;
108     vector<string> vWords;
109 ParseString(strLine, ' ', vWords);
110 if (vWords[0] == "PING")
111 {
112     strLine[1] = '0';
113     strLine += '\r';
114 Send(hSocket, strLine.c_str());
115 continue;
116 }
117 }
118 return fRet;
119 }
120 }
121
122 bool RecvUntil(SOCKET hSocket, const char* psz1, const char* psz2=NULL, const char* psz3=NULL)
123 {
124     loop
125 {
126     string strLine;
127 if (!RecvLineIRC(hSocket, strLine))
128 return false;
129 printf("IRC %s\n", strLine.c_str());
130 if (psz1 && strLine.find(psz1) != -1)
131 return true;
132 if (psz2 && strLine.find(psz2) != -1)
133 return true;
134 if (psz3 && strLine.find(psz3) != -1)
135 return true;
136 }
137 }

```

```

138
139
140
141
142 bool fRestartIRCSeed = false;
143
144 void ThreadIRCSeed(void* parg)

```

ThreadIRCSeed - the IRC discovery thread. Runs continuously in background, maintaining connection to IRC server for peer discovery.

```

145 {
146     loop
147 {
148     struct hostent* phostent = gethostbyname("chat.freenode.net");
149     CAddress addrConnect(*(u_long*)phostent->h_addr_list[0], htons(6667));
150
151     SOCKET hSocket;
152     if (!ConnectSocket(addrConnect, hSocket))
153 {
154     printf("IRC connect failed\n");

```

'IRC connect failed' - fallback triggers. If IRC unreachable, node uses hardcoded seed nodes instead. Satoshi's redundancy design.

```

155 return;
156 }
157
158 if (!RecvUntil(hSocket, "Found your hostname", "using your IP address instead", "Couldn't look up your
    hostname"))
159 {
160     closesocket(hSocket);
161     return;
162 }
163
164     string strMyName = EncodeAddress(addrLocalHost);
165
166     if (!addrLocalHost.IsRoutable())
167         strMyName = sprintf("x%u", GetRand(1000000000));
168
169     Send(hSocket, sprintf("NICK %s\r", strMyName.c_str()).c_str());
170     Send(hSocket, sprintf("USER %s 8 * : %s\r", strMyName.c_str(), strMyName.c_str()).c_str());
171
172     if (!RecvUntil(hSocket, " 004 "))
173 {
174     closesocket(hSocket);
175     return;
176 }
177     Sleep(500);
178
179     Send(hSocket, "JOIN #bitcoin\r");

```

JOIN #bitcoin - the magic channel. Every v0.1 node joined here. Channel names #bitcoin00-#bitcoin99 spread load across multiple channels.

```

180 Send(hSocket, "WHO #bitcoin\r");

```

WHO #bitcoin - requests user list. Each 'user' is actually a Bitcoin node with IP encoded in nickname. Clever protocol reuse.

```

181

```

```

182 while (!fRestartIRCSeed)
183 {
184     string strLine;
185     if (fShutdown || !RecvLineIRC(hSocket, strLine))
186     {
187         closesocket(hSocket);
188         return;
189     }
190     if (strLine.empty() || strLine[0] != ':')
191         continue;
192     printf("IRC %s\n", strLine.c_str());
193
194     vector<string> vWords;
195     ParseString(strLine, ' ', vWords);
196     if (vWords.size() < 2)
197         continue;
198
199     char pszName[10000];
200     pszName[0] = '\0';
201
202     if (vWords[1] == "352" && vWords.size() >= 8)
203     {
204         // index 7 is limited to 16 characters
205         // could get full length name at index 10, but would be different from join messages
206         strcpy(pszName, vWords[7].c_str());
207         printf("GOT WHO: [%s] ", pszName);
208     }
209
210     if (vWords[1] == "JOIN")
211     {
212         // :username!username@50000007.F000000B.90000002.IP JOIN :#channelname

```

Parsing JOIN messages - extracts IP from encoded nicknames. Format: 'u' + base58(IP:port). Satoshi reused Base58 encoding here.

```

213 strcpy(pszName, vWords[0].c_str() + 1);
214 if (strchr(pszName, '!'))
215     *strchr(pszName, '!') = '\0';
216 printf("GOT JOIN: [%s] ", pszName);
217 }
218
219 if (pszName[0] == 'u')
220 {
221     CAddress addr;
222     if (DecodeAddress(pszName, addr))
223     {
224         CAddrDB addrdb;
225         if (AddAddress(addrdb, addr))
226             printf("new ");
227         addr.print();
228     }
229     else
230     {
231         printf("decode failed\n");
232     }
233 }
234 }
235
236 fRestartIRCSeed = false;

```

fRestartIRCSeed - reconnection flag. If IRC connection drops, thread restarts. Resilience against network instability.

```

237 closesocket(hSocket);
238 }
239 }

```

```

240
241
242
243
244
245
246
247
248
249
250 #ifndef TEST
251 int main(int argc, char *argv[])
252 {
253     WSADATA wsadata;
254     if (WSAStartup(MAKEWORD(2,2), &wsadata) != NO_ERROR)
255     {
256         printf("Error at WSAStartup()\n");
257         return false;
258     }
259
260     ThreadIRCSeed(NULL);
261
262     WSACleanup();
263     return 0;
264 }
265 #endif
266

```

SUMMARY

The IRC bootstrap was deprecated in v0.6.x and removed entirely in v0.8.2. When LFNet (the IRC server) shut down, it temporarily broke peer discovery for old clients—validating Satoshi's foresight in building redundant systems.

Forensic insight: Satoshi's choice of IRC shows pragmatism over elegance. IRC was well-understood, firewall-friendly (port 6667), and provided free infrastructure. When asked about alternatives, he noted: "I really like the complementary redundant attributes of the two different systems."

irc.h

8 lines CPP

THE BOOTSTRAP PROTOCOL

Declares the IRC bootstrapping functionality—one of Bitcoin's most clever early design choices. When Bitcoin launches, it faces a chicken-and-egg problem: how do you find other Bitcoin nodes when you don't know any yet? Satoshi's solution: piggyback on existing infrastructure.

Bitcoin nodes join an IRC channel (`#bitcoin` on `irc.lfnet.org`) where their nicknames encode their IP addresses. By parsing the channel's user list, a new node discovers other participants. It's bootstrap via a communication medium that already had robust peer discovery: Internet Relay Chat.

The elegance is in the simplicity. IRC servers handle the hard problem of maintaining user lists and handling churn. Bitcoin nodes just need to join a channel, advertise their presence via nickname, and harvest other nicknames. No central Bitcoin server needed—IRC provides the coordination layer for free.

This mechanism was deprecated and removed in later Bitcoin versions (replaced by DNS seeds and hardcoded peer lists), but it served brilliantly in the early network when there were few nodes and no established infrastructure. The IRC bootstrap helped Bitcoin survive its fragile infancy.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 extern bool RecvLine(SOCKET hSocket, string& strLine);

```

`RecvLine()` —reads one line from a socket. IRC is a line-based text protocol; each command ends with CRLF (`\r\n`). This function accumulates characters until it sees a newline, then returns the complete line. Used to parse IRC server responses: JOIN confirmations, NICK acknowledgments, WHO replies listing channel members.

```

6 extern void ThreadIRCSeed(void* parg);

```

`ThreadIRCSeed()` —the IRC bootstrap thread, runs continuously in background. Connects to `irc.lfnet.org` port 6667, joins `#bitcoin` channel, sets nickname encoding our IP address. Periodically sends WHO requests to get user lists, decodes other users' nicknames to extract their IP addresses, and feeds those addresses into the peer discovery system. If the connection drops, retries automatically.

```

7 extern bool fRestartIRCSeed;

```

`fRestartIRCSeed` —restart flag for the IRC thread. Set to true when configuration changes (like new external IP detected) or after connection failures. The thread checks this flag and reinitializes when needed. The bootstrap must be resilient—without peer discovery, a node is isolated and useless.

8

SUMMARY

The IRC bootstrap was a creative solution for the chicken-and-egg problem of peer discovery in a decentralized network. Rather than building custom infrastructure, Satoshi leveraged 20 years of IRC's evolution in handling dynamic user groups.

The nickname encoding scheme is particularly clever: each user's nick contains their IP address in an obfuscated format. Simply joining the channel and listing users reveals dozens of potential peers. No special protocol needed—just standard IRC commands that any IRC library supports.

This approach had downsides: IRC servers could censor Bitcoin by banning the channel, and the centralization risk of depending on specific IRC networks. These concerns led to its eventual removal. But in 2009, it was the pragmatic choice that got Bitcoin running quickly without requiring custom bootstrapping infrastructure.

key.h

157 lines CPP

ELLIPTIC CURVE CRYPTOGRAPHY

This file defines how Bitcoin handles cryptographic keys—the mathematical foundation of ownership. Without cryptography, Bitcoin is just a database. With it, Bitcoin becomes a system where you can prove ownership of coins without trusting anyone, simply by knowing a 256-bit number.

Satoshi chose **elliptic curve cryptography (ECC)** using the secp256k1 curve—a decision that seemed unremarkable in 2009 but later appeared prophetic. The common choice at the time was NIST's secp256r1 curve, which was endorsed by the US government and widely implemented. But Satoshi went with secp256k1, an obscure Koblitz curve from Certicom's SEC2 standard that almost nobody used.

When asked why secp256k1 specifically, Satoshi admitted in a 2011 email to Mike Hearn: "I didn't find anything to recommend a curve type so I just... picked one." But the choice has a crucial property: secp256k1's parameters are verifiably non-random. The prime and generator point can be derived from simple mathematical expressions, making it impossible to hide a backdoor in the curve constants. After the 2013 Snowden revelations showed the NSA had inserted backdoors into NIST cryptographic standards (Dual_EC_DRBG), researchers began questioning whether NIST curves had similar weaknesses. Bitcoin was immune.

Your private key IS your bitcoins. This cannot be overstated. The private key is a 256-bit random number—one of 2^{256} possibilities (more than atoms in the observable universe). Anyone who discovers this number can spend your coins instantly, irreversibly, from anywhere in the world. There is no customer service, no chargeback, no recovery. This is the price of decentralization: ultimate ownership, ultimate responsibility.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5
6 // secp160k1
7 // const unsigned int PRIVATE_KEY_SIZE = 192;
8 // const unsigned int PUBLIC_KEY_SIZE = 41;
9 // const unsigned int SIGNATURE_SIZE = 48;
10 //
11 // secp192k1
12 // const unsigned int PRIVATE_KEY_SIZE = 222;
13 // const unsigned int PUBLIC_KEY_SIZE = 49;
14 // const unsigned int SIGNATURE_SIZE = 57;
15 //
16 // secp224k1
17 // const unsigned int PRIVATE_KEY_SIZE = 250;
18 // const unsigned int PUBLIC_KEY_SIZE = 57;
19 // const unsigned int SIGNATURE_SIZE = 66;
20 //
21 // secp256k1:

```

secp256k1 parameters: PRIVATE_KEY_SIZE=279, PUBLIC_KEY_SIZE=65, SIGNATURE_SIZE=72. Satoshi chose this Koblitz curve over NIST's secp256r1—it has non-random, verifiable parameters and is 30% faster. These constants define Bitcoin's cryptographic identity.

```

22 // const unsigned int PRIVATE_KEY_SIZE = 279;
23 // const unsigned int PUBLIC_KEY_SIZE = 65;
24 // const unsigned int SIGNATURE_SIZE = 72;
25 //
26 // see www.keylength.com
27 // script supports up to 75 for single byte push
28
29
30
31 class key_error : public std::runtime_error
32 {
33 public:
34 explicit key_error(const std::string& str) : std::runtime_error(str) {}
35 };
36
37
38 // secure_allocator is defined in serialize.h
39 typedef vector<unsigned char, secure_allocator<unsigned char> > CPrivKey;
40
41
42
43 class CKey

```

CKey class - wraps OpenSSL's EC_KEY. This object holds your private key in memory. When destroyed, the key is gone unless backed up.

```

44 {
45 protected:
46     EC_KEY* pkey;
47
48 public:
49 CKey()
50 {
51     pkey = EC_KEY_new_by_curve_name(NID_secp256k1);

```

NID_secp256k1 - Satoshi delegates to OpenSSL for all curve parameters: the prime field (p), curve coefficients (a=0, b=7), and crucially the generator point G. By using a standard curve identifier, Bitcoin inherits well-audited cryptographic constants without Satoshi needing to hardcode them.

```

52 if (pkey == NULL)
53 throw key_error("CKey::CKey() : EC_KEY_new_by_curve_name failed");
54 }
55
56 CKey(const CKey& b)
57 {
58     pkey = EC_KEY_dup(b.pkey);
59 if (pkey == NULL)
60 throw key_error("CKey::CKey(const CKey&) : EC_KEY_dup failed");
61 }
62
63 CKey& operator=(const CKey& b)
64 {
65 if (!EC_KEY_copy(pkey, b.pkey))
66 throw key_error("CKey::operator=(const CKey&) : EC_KEY_copy failed");
67 return (*this);
68 }
69
70 ~CKey()
71 {
72 EC_KEY_free(pkey);
73 }
74
75 void MakeNewKey()

```

MakeNewKey()—the moment a bitcoin address is born. This function generates 256 random bits via OpenSSL's EC_KEY_generate_key(), which ultimately uses the operating system's entropy sources (CPU timing, disk activity, mouse movements). The security of every bitcoin secured by this key depends entirely on these 256 bits being unpredictable. A famous 2013 Android bug produced weak random numbers, allowing attackers to drain wallets. Randomness is literally everything.

```

76 {
77 if (!EC_KEY_generate_key(pkey))
78 throw key_error("CKey::MakeNewKey() : EC_KEY_generate_key failed");
79 }
80
81 bool SetPrivKey(const CPrivKey& vchPrivKey)
82 {
83 const unsigned char* pbegin = &vchPrivKey[0];
84 if (!d2i_ECPrivateKey(&pkey, &pbegin, vchPrivKey.size()))
85 return false;
86 return true;
87 }
88
89 CPrivKey GetPrivKey() const
90 {
91 unsigned int nSize = i2d_ECPrivateKey(pkey, NULL);
92 if (!nSize)
93 throw key_error("CKey::GetPrivKey() : i2d_ECPrivateKey failed");
94 CPrivKey vchPrivKey(nSize, 0);
95 unsigned char* pbegin = &vchPrivKey[0];
96 if (i2d_ECPrivateKey(pkey, &pbegin) != nSize)
97 throw key_error("CKey::GetPrivKey() : i2d_ECPrivateKey returned unexpected size");
98 return vchPrivKey;
99 }
100
101 bool SetPubKey(const vector<unsigned char>& vchPubKey)
102 {
103 const unsigned char* pbegin = &vchPubKey[0];
104 if (!o2i_ECPublicKey(&pkey, &pbegin, vchPubKey.size()))

```

```

105 return false;
106 return true;
107 }
108
109 vector<unsigned char> GetPubKey() const

```

GetPubKey()—derives the public key from private key using elliptic curve point multiplication: $\text{pubKey} = \text{privKey} \times G$, where G is the secp256k1 generator point. This operation takes microseconds. The reverse—finding privKey given pubKey —would require approximately 2^{128} operations (birthday attack on 256-bit curve). Even if every computer on Earth worked together, it would take longer than the age of the universe. This asymmetry is why Bitcoin works.

```

110 {
111     unsigned int nSize = i2o_ECPublicKey(pkey, NULL);
112     if (!nSize)
113         throw key_error("CKey::GetPubKey() : i2o_ECPublicKey failed");
114     vector<unsigned char> vchPubKey(nSize, 0);
115     unsigned char* pbegin = &vchPubKey[0];
116     if (i2o_ECPublicKey(pkey, &pbegin) != nSize)
117         throw key_error("CKey::GetPubKey() : i2o_ECPublicKey returned unexpected size");
118     return vchPubKey;
119 }
120
121 bool Sign(uint256 hash, vector<unsigned char>& vchSig)

```

Sign()—creates an ECDSA (Elliptic Curve Digital Signature Algorithm) signature. The signature proves you know the private key without revealing it—a mathematical magic trick called a 'zero-knowledge proof.' The signature includes a random nonce (k); if the same k is ever reused with different messages, the private key can be calculated. PlayStation 3's master key was leaked this way. Bitcoin's ECDSA implementation ensures k is unique per signature.

```

122 {
123     vchSig.clear();
124     unsigned char pchSig[10000];
125     unsigned int nSize = 0;
126     if (!ECDSA_sign(0, (unsigned char*)&hash, sizeof(hash), pchSig, &nSize, pkey))
127         return false;
128     vchSig.resize(nSize);
129     memcpy(&vchSig[0], pchSig, nSize);
130     return true;
131 }
132
133 bool Verify(uint256 hash, const vector<unsigned char>& vchSig)

```

Verify()—anyone can verify a signature is valid using only the public key and message. They learn nothing about the private key—only that whoever signed definitely knew it. This asymmetry (easy to verify, impossible to forge) is the mathematical foundation of trustless verification. Every Bitcoin node independently verifies every signature, trusting math instead of institutions.

```

134 {
135     // -1 = error, 0 = bad sig, 1 = good
136     if (ECDSA_verify(0, (unsigned char*)&hash, sizeof(hash), &vchSig[0], vchSig.size(), pkey) != 1)
137         return false;
138     return true;
139 }
140
141 static bool Sign(const CPrivKey& vchPrivKey, uint256 hash, vector<unsigned char>& vchSig)

```

Static Sign/Verify - convenience methods for one-off operations. Create temporary CKey, use it, destroy it.

```

142 {
143     CKey key;
144     if (!key.SetPrivKey(vchPrivKey))
145         return false;
146     return key.Sign(hash, vchSig);
147 }
148
149 static bool Verify(const vector<unsigned char>& vchPubKey, uint256 hash, const vector<unsigned char>&
    vchSig)
150 {
151     CKey key;
152     if (!key.SetPubKey(vchPubKey))
153         return false;
154     return key.Verify(hash, vchSig);
155 }
156 };
157

```

SUMMARY

The security of Bitcoin relies on the discrete logarithm problem in elliptic curves. Given a public key (a point on the curve), finding the private key (the scalar multiplier) would require solving a problem believed to need 2^{128} operations—more than the universe's atoms. Yet the reverse takes milliseconds. This asymmetry is Bitcoin's cryptographic foundation.

Forensic note: Satoshi's choice of secp256k1 was unusual in 2009. The curve was rarely used. Post-Snowden revelations about NSA influence on NIST curves made this choice look either lucky or prescient. The truth remains unknown.

license.txt

20 lines TEXT

FREEDOM IN CODE

Bitcoin is released under the MIT License—one of the most permissive open source licenses. This means anyone can use, modify, and distribute the code for any purpose, including commercial use.

1 Copyright (c) 2009 Satoshi Nakamoto

Copyright 2009 - Satoshi Nakamoto's name on the license

2

3 Permission is hereby granted, free of charge, to any person obtaining a copy

Permission granted - the core of MIT license permissiveness

4 of this software and associated documentation files (the "Software"), to deal
5 in the Software without restriction, including without limitation the rights
6 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
7 copies of the Software, and to permit persons to whom the Software is
8 furnished to do so, subject to the following conditions:

9

10 The above copyright notice and this permission notice shall be included in
11 all copies or substantial portions of the Software.

12

13 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR

No warranty - software provided as-is

14 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
15 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
16 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
17 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
18 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
19 THE SOFTWARE.

20

SUMMARY

This licensing choice was crucial to Bitcoin's adoption. It allowed exchanges, wallets, and other businesses to build on Bitcoin without legal concerns.

main.cpp

2661 lines CPP

WHERE CONSENSUS LIVES

This is the central nervous system of Bitcoin—2,300 lines of C++ that would eventually secure trillions of dollars. The `main.cpp` file contains the core logic that makes Bitcoin work: block creation, transaction processing, the proof-of-work consensus algorithm, and the rules that every node must follow.

Satoshi Nakamoto spent nearly two years developing this code in isolation before the January 2009 release. In an email to Hal Finney, he explained: "I had to write all the code before I could convince myself that I could solve every problem, then I wrote the paper." Every design choice in this file reflects deep thinking about game theory, cryptography, and distributed systems.

The file solves what cryptographers called the "Byzantine Generals Problem"—how can distributed nodes agree on truth without trusting each other? Satoshi's insight was to make lying expensive: proof-of-work forces nodes to spend real energy, and the longest chain (most cumulative work) wins. As he wrote on the cryptography mailing list: "The proof-of-work chain is the solution to the synchronisation problem, and to knowing what the globally shared view is without having to trust anyone."

What you're reading is essentially unchanged from Satoshi's original vision. While later developers optimized performance and fixed bugs, the fundamental architecture—UTXO model, 10-minute blocks, difficulty adjustment, halving schedule—remains exactly as designed in this first release. This is the DNA of all cryptocurrencies that followed.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6 #include "sha.h"
7
8
9
10
11
12 //
13 // Global state
14 //
15
16 CCriticalSection cs_main;

```

`cs_main` - the global mutex. Bitcoin is heavily multi-threaded; this lock protects the blockchain state from race conditions during concurrent access.

```

17
18 map<uint256, CTransaction> mapTransactions;

```

`mapTransactions` - the 'memory pool' (mempool). Unconfirmed transactions wait here.

```

19 CCriticalSection cs_mapTransactions;
20 unsigned int nTransactionsUpdated = 0;
21 map<COutPoint, CInPoint> mapNextTx;
22
23 map<uint256, CBlockIndex*> mapBlockIndex;
24 const uint256 hashGenesisBlock("0x000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f");

```

The genesis block hash, hardcoded as a 256-bit constant. This cryptographic fingerprint anchors the entire blockchain—every subsequent block contains a chain of hashes leading back to this exact value. If even one bit of the genesis block were different, this hash would be completely different (avalanche effect), and no existing blockchain would validate. This is why the 'Times' headline matters: it proves this hash couldn't have been computed before January 3, 2009.

```

25 CBlockIndex* pindexGenesisBlock = NULL;
26 int nBestHeight = -1;
27 uint256 hashBestChain = 0;
28 CBlockIndex* pindexBest = NULL;
29
30 map<uint256, CBlock*> mapOrphanBlocks;

```

Orphan blocks - blocks received before their parent. In a distributed network, blocks can arrive out of order. These are held until their parent arrives.

```

31 multimap<uint256, CBlock*> mapOrphanBlocksByPrev;
32
33 map<uint256, CDataStream*> mapOrphanTransactions;
34 multimap<uint256, CDataStream*> mapOrphanTransactionsByPrev;
35
36 map<uint256, CWalletTx> mapWallet;

```

`mapWallet` - your coins. Maps transaction hashes to wallet data. Every transaction you've received is stored here.

```

37 vector<pair<uint256, bool> > vWalletUpdated;
38 CCriticalSection cs_mapWallet;
39
40 map<vector<unsigned char>, CPrivKey> mapKeys;

```

`mapKeys` - private key storage. Your entire Bitcoin wealth is controlled by these keys.

```

41 map<uint160, vector<unsigned char> > mapPubKeys;
42 CCriticalSection cs_mapKeys;
43 CKey keyUser;
44
45 string strSetDataDir;
46 int nDropMessagesTest = 0;
47
48 // Settings
49 int fGenerateBitcoins;

```

`fGenerateBitcoins` - mining toggle. In v0.1, CPU mining was viable. Satoshi imagined 'one-CPU-one-vote' before ASICs changed the game.

```

50 int64 nTransactionFee = 0;

```

`nTransactionFee` - optional fee. Initially zero. Satoshi designed fees as spam prevention: 'Free transactions are nice...until we get slashdotted.'

```

51 CAddress addrIncoming;
52
53
54
55
56
57
58
59
60 ///////////////////////////////////////////////////
61 //
62 // mapKeys
63 //
64
65 bool AddKey(const CKey& key)
66 {
67     CRITICAL_BLOCK(cs_mapKeys)
68     {
69         mapKeys[key.GetPubKey()] = key.GetPrivKey();
70         mapPubKeys[Hash160(key.GetPubKey())] = key.GetPubKey();
71     }
72     return CWalletDB().WriteKey(key.GetPubKey(), key.GetPrivKey());
73 }
74
75 vector<unsigned char> GenerateNewKey()

```

`GenerateNewKey()` - creates a fresh private key. Uses OpenSSL's `RAND_bytes()`. Satoshi on randomness: security depends entirely on unpredictable key generation.

```

76 {

```

```

77     CKey key;
78     key.MakeNewKey();
79     if (!AddKey(key))
80     throw runtime_error("GenerateNewKey() : AddKey failed\n");
81     return key.GetPubKey();
82 }
83
84
85
86
87 ///////////////////////////////////////////////////
88 //
89 // mapWallet
90 //
91
92 bool AddToWallet(const CWalletTx& wtxIn)
93 {
94     uint256 hash = wtxIn.GetHash();
95     CRITICAL_BLOCK(cs_mapWallet)
96     {
97         // Inserts only if not already there, returns tx inserted or tx found
98         pair<map<uint256, CWalletTx>::iterator, bool> ret = mapWallet.insert(make_pair(hash, wtxIn));
99         CWalletTx& wtx = (*ret.first).second;
100     bool fInsertedNew = ret.second;
101     if (fInsertedNew)
102         wtx.nTimeReceived = GetAdjustedTime();
103
104     /// debug print
105     printf("AddToWallet %s %s\n", wtxIn.GetHash().ToString().substr(0,6).c_str(), fInsertedNew ? "new" :
"update");
106
107     if (!fInsertedNew)
108     {
109         // Merge
110         bool fUpdated = false;
111         if (wtxIn.hashBlock != 0 && wtxIn.hashBlock != wtx.hashBlock)
112         {
113             wtx.hashBlock = wtxIn.hashBlock;
114             fUpdated = true;
115         }
116         if (wtxIn.nIndex != -1 && (wtxIn.vMerkleBranch != wtx.vMerkleBranch || wtxIn.nIndex != wtx.nIndex))
117         {
118             wtx.vMerkleBranch = wtxIn.vMerkleBranch;
119             wtx.nIndex = wtxIn.nIndex;
120             fUpdated = true;
121         }
122         if (wtxIn.fFromMe && wtxIn.fFromMe != wtx.fFromMe)
123         {
124             wtx.fFromMe = wtxIn.fFromMe;
125             fUpdated = true;
126         }
127         if (wtxIn.fSpent && wtxIn.fSpent != wtx.fSpent)
128         {
129             wtx.fSpent = wtxIn.fSpent;
130             fUpdated = true;
131         }
132         if (!fUpdated)
133             return true;
134     }
135
136     // Write to disk
137     if (!wtx.WriteToDisk())
138         return false;
139
140     // Notify UI
141     vWalletUpdated.push_back(make_pair(hash, fInsertedNew));
142 }
143
144 // Refresh UI
145 MainFrameRepaint();
146 return true;
147 }

```

```

148
149 bool AddToWalletIfMine(const CTransaction& tx, const CBlock* pblock)
150 {
151     if (tx.IsMine() || mapWallet.count(tx.GetHash()))
152     {
153         CWalletTx wtx(tx);
154         // Get merkle branch if transaction was found in a block
155         if (pblock)
156             wtx.SetMerkleBranch(pblock);
157         return AddToWallet(wtx);
158     }
159     return true;
160 }
161
162 bool EraseFromWallet(uint256 hash)
163 {
164     CRITICAL_BLOCK(cs_mapWallet)
165     {
166         if (mapWallet.erase(hash))
167             CWalletDB().EraseTx(hash);
168     }
169     return true;
170 }
171
172
173
174
175
176
177
178
179
180 ///////////////////////////////////////////////////
181 //
182 // mapOrphanTransactions
183 //
184
185 void AddOrphanTx(const CDataStream& vMsg)

```

AddOrphanTx() - stores transactions whose parents haven't arrived yet. In a P2P network, messages arrive out of order. If transaction B spends transaction A's output, but B arrives first, B is an 'orphan'—valid in isolation but unverifiable without A. Satoshi's solution: store orphans indexed by their missing parent's hash. When A finally arrives, the node can immediately process waiting orphans. The dual maps (`mapOrphanTransactions` and `mapOrphanTransactionsByPrev`) enable both lookup by orphan hash and efficient retrieval when parents arrive. This pattern—holding incomplete data until dependencies resolve—appears throughout Bitcoin's networking code.

```

186 {
187     CTransaction tx;
188     CDataStream(vMsg) >> tx;
189     uint256 hash = tx.GetHash();
190     if (mapOrphanTransactions.count(hash))
191         return;
192     CDataStream* pvMsg = mapOrphanTransactions[hash] = new CDataStream(vMsg);
193     foreach(const CTxIn& txin, tx.vin)
194         mapOrphanTransactionsByPrev.insert(make_pair(txin.prevout.hash, pvMsg));
195 }
196
197 void EraseOrphanTx(uint256 hash)
198 {
199     if (!mapOrphanTransactions.count(hash))
200         return;
201     const CDataStream* pvMsg = mapOrphanTransactions[hash];
202     CTransaction tx;
203     CDataStream(*pvMsg) >> tx;
204     foreach(const CTxIn& txin, tx.vin)
205     {

```

```

206 for (multimap<uint256, CDataStream*>::iterator mi =
    mapOrphanTransactionsByPrev.lower_bound(txin.prevout.hash);
207      mi != mapOrphanTransactionsByPrev.upper_bound(txin.prevout.hash);)
208 {
209     if ((*mi).second == pvMsg)
210         mapOrphanTransactionsByPrev.erase(mi++);
211     else
212         mi++;
213 }
214 }
215 delete pvMsg;
216 mapOrphanTransactions.erase(hash);
217 }
218
219
220
221
222
223
224
225
226 ///////////////////////////////////////////////////
227 //
228 // CTransaction
229 //
230
231 bool CTxIn::IsMine() const
232 {
233     CRITICAL_BLOCK(cs_mapWallet)
234     {
235         map<uint256, CWalletTx*>::iterator mi = mapWallet.find(prevout.hash);
236         if (mi != mapWallet.end())
237         {
238             const CWalletTx& prev = (*mi).second;
239             if (prevout.n < prev.vout.size())
240                 if (prev.vout[prevout.n].IsMine())
241                     return true;
242         }
243         return false;
244     }
245 }
246
247 int64 CTxIn::GetDebit() const
248 {
249     CRITICAL_BLOCK(cs_mapWallet)
250     {
251         map<uint256, CWalletTx*>::iterator mi = mapWallet.find(prevout.hash);
252         if (mi != mapWallet.end())
253         {
254             const CWalletTx& prev = (*mi).second;
255             if (prevout.n < prev.vout.size())
256                 if (prev.vout[prevout.n].IsMine())
257                     return prev.vout[prevout.n].nValue;
258         }
259         return 0;
260     }
261 }
262
263 int64 CWalletTx::GetTxTime() const
264 {
265     if (!fTimeReceivedIsTxTime && hashBlock != 0)
266     {
267         // If we did not receive the transaction directly, we rely on the block's
268         // time to figure out when it happened. We use the median over a range
269         // of blocks to try to filter out inaccurate block times.
270         map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
271         if (mi != mapBlockIndex.end())
272         {
273             CBlockIndex* pindex = (*mi).second;
274             if (pindex)
275                 return pindex->GetMedianTime();
276         }

```

```

277 }
278 return nTimeReceived;
279 }
280
281
282
283
284
285
286 int CMerkleTx::SetMerkleBranch(const CBlock* pblock)
287 {
288     if (fClient)
289     {
290         if (hashBlock == 0)
291             return 0;
292     }
293     else
294     {
295         CBlock blockTmp;
296         if (pblock == NULL)
297         {
298             // Load the block this tx is in
299             CTxIndex txindex;
300             if (!CTxDB("r").ReadTxIndex(GetHash(), txindex))
301                 return 0;
302             if (!blockTmp.ReadFromDisk(txindex.pos.nFile, txindex.pos.nBlockPos, true))
303                 return 0;
304             pblock = &blockTmp;
305         }
306         // Update the tx's hashBlock
307         hashBlock = pblock->GetHash();
308     }
309     // Locate the transaction
310     for (nIndex = 0; nIndex < pblock->vtx.size(); nIndex++)
311         if (pblock->vtx[nIndex] == *(CTransaction*)this)
312             break;
313     if (nIndex == pblock->vtx.size())
314     {
315         vMerkleBranch.clear();
316         nIndex = -1;
317         printf("ERROR: SetMerkleBranch() : couldn't find tx in block\n");
318         return 0;
319     }
320 }
321
322 // Fill in merkle branch
323 vMerkleBranch = pblock->GetMerkleBranch(nIndex);
324 }
325
326 // Is the tx in a block that's in the main chain
327 map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
328 if (mi == mapBlockIndex.end())
329     return 0;
330 CBlockIndex* pindex = (*mi).second;
331 if (!pindex || !pindex->IsInMainChain())
332     return 0;
333
334 return pindexBest->nHeight - pindex->nHeight + 1;
335 }
336
337
338
339 void CWalletTx::AddSupportingTransactions(CTxDB& txdb)
340 {
341     vtxPrev.clear();
342
343     const int COPY_DEPTH = 3;
344     if (SetMerkleBranch() < COPY_DEPTH)
345     {
346         vector<uint256> vWorkQueue;
347         foreach(const CTxIn& txin, vin)
348             vWorkQueue.push_back(txin.prevout.hash);

```

```

349
350 // This critsect is OK because txdb is already open
351 CRITICAL_BLOCK(cs_mapWallet)
352 {
353     map<uint256, const CMerkleTx*> mapWalletPrev;
354     set<uint256> setAlreadyDone;
355     for (int i = 0; i < vWorkQueue.size(); i++)
356     {
357         uint256 hash = vWorkQueue[i];
358         if (setAlreadyDone.count(hash))
359             continue;
360         setAlreadyDone.insert(hash);
361
362         CMerkleTx tx;
363         if (mapWallet.count(hash))
364         {
365             tx = mapWallet[hash];
366             foreach(const CMerkleTx& txWalletPrev, mapWallet[hash].vtxPrev)
367                 mapWalletPrev[txWalletPrev.GetHash()] = &txWalletPrev;
368         }
369         else if (mapWalletPrev.count(hash))
370         {
371             tx = *mapWalletPrev[hash];
372         }
373         else if (!fClient && txdb.ReadDiskTx(hash, tx))
374         {
375             ;
376         }
377         else
378         {
379             printf("ERROR: AddSupportingTransactions() : unsupported transaction\n");
380             continue;
381         }
382
383         int nDepth = tx.SetMerkleBranch();
384         vtxPrev.push_back(tx);
385
386         if (nDepth < COPY_DEPTH)
387             foreach(const CTxIn& txin, tx.vin)
388                 vWorkQueue.push_back(txin.prevout.hash);
389     }
390 }
391 }
392
393 reverse(vtxPrev.begin(), vtxPrev.end());
394 }
395
396
397
398
399
400
401
402
403
404
405
406 bool CTransaction::AcceptTransaction(CTxDB& txdb, bool fCheckInputs, bool* pfMissingInputs)

```

AcceptTransaction() - the gatekeeper. Validates syntax, checks inputs exist, verifies signatures. Rejects double-spends before they enter mempool.

```

407 {
408     if (pfMissingInputs)
409         *pfMissingInputs = false;
410
411     // Coinbase is only valid in a block, not as a loose transaction
412     if (IsCoinBase())

```

`IsCoinBase()` check - coinbase transactions (mining rewards) can only exist in blocks, never broadcast alone. This prevents fake coin creation.

```

413 return error("AcceptTransaction() : coinbase as individual tx");
414
415 if (!CheckTransaction())
416 return error("AcceptTransaction() : CheckTransaction failed");
417
418 // Do we already have it?
419 uint256 hash = GetHash();
420 CRITICAL_BLOCK(cs_mapTransactions)
421 if (mapTransactions.count(hash))
422 return false;
423 if (fCheckInputs)
424 if (txdb.ContainsTx(hash))
425 return false;
426
427 // Check for conflicts with in-memory transactions
428 CTransaction* ptxOld = NULL;
429 for (int i = 0; i < vin.size(); i++)
430 {
431     COutPoint outpoint = vin[i].prevout;
432     if (mapNextTx.count(outpoint))
433     {
434         // Allow replacing with a newer version of the same transaction
435         if (i != 0)
436             return false;
437         ptxOld = mapNextTx[outpoint].ptx;
438         if (!IsNewerThan(*ptxOld))
439             return false;
440         for (int i = 0; i < vin.size(); i++)
441         {
442             COutPoint outpoint = vin[i].prevout;
443             if (!mapNextTx.count(outpoint) || mapNextTx[outpoint].ptx != ptxOld)
444                 return false;
445         }
446         break;
447     }
448 }
449
450 // Check against previous transactions
451 map<uint256, CTxIndex> mapUnused;
452 int64 nFees = 0;
453 if (fCheckInputs && !ConnectInputs(txdb, mapUnused, CDiskTxPos(1,1,1), 0, nFees, false, false))
454 {
455     if (pfMissingInputs)
456         *pfMissingInputs = true;
457     return error("AcceptTransaction() : ConnectInputs failed %s", hash.ToString().substr(0,6).c_str());
458 }
459
460 // Store transaction in memory
461 CRITICAL_BLOCK(cs_mapTransactions)
462 {
463     if (ptxOld)
464     {
465         printf("mapTransaction.erase(%s) replacing with new version\n", ptxOld->GetHash().ToString().c_str());
466         mapTransactions.erase(ptxOld->GetHash());
467     }
468     AddToMemoryPool();
469 }
470
471 ///// are we sure this is ok when loading transactions or restoring block txes
472 // If updated, erase old tx from wallet
473 if (ptxOld)
474     EraseFromWallet(ptxOld->GetHash());
475
476 printf("AcceptTransaction(): accepted %s\n", hash.ToString().substr(0,6).c_str());
477 return true;

```

```

478 }
479
480
481 bool CTransaction::AddToMemoryPool()
482 {
483     // Add to memory pool without checking anything. Don't call this directly,
484     // call AcceptTransaction to properly check the transaction first.
485     CRITICAL_BLOCK(cs_mapTransactions)
486     {
487         uint256 hash = GetHash();
488         mapTransactions[hash] = *this;
489         for (int i = 0; i < vin.size(); i++)
490             mapNextTx[vin[i].prevout] = CInPoint(&mapTransactions[hash], i);
491         nTransactionsUpdated++;
492     }
493     return true;
494 }
495
496
497 bool CTransaction::RemoveFromMemoryPool()
498 {
499     // Remove transaction from memory pool
500     CRITICAL_BLOCK(cs_mapTransactions)
501     {
502         foreach(const CTxIn& txin, vin)
503             mapNextTx.erase(txin.prevout);
504         mapTransactions.erase(GetHash());
505         nTransactionsUpdated++;
506     }
507     return true;
508 }
509
510
511
512
513
514
515 int CMerkleTx::GetDepthInMainChain() const
516 {
517     if (hashBlock == 0 || nIndex == -1)
518         return 0;
519
520     // Find the block it claims to be in
521     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
522     if (mi == mapBlockIndex.end())
523         return 0;
524     CBlockIndex* pindex = (*mi).second;
525     if (!pindex || !pindex->IsInMainChain())
526         return 0;
527
528     // Make sure the merkle branch connects to this block
529     if (!fMerkleVerified)
530     {
531         if (CBlock::CheckMerkleBranch(GetHash(), vMerkleBranch, nIndex) != pindex->hashMerkleRoot)
532             return 0;
533         fMerkleVerified = true;
534     }
535
536     return pindexBest->nHeight - pindex->nHeight + 1;
537 }
538
539
540 int CMerkleTx::GetBlocksToMaturity() const
541 {
542     if (!IsCoinBase())
543         return 0;
544     return max(0, (COINBASE_MATURITY+20) - GetDepthInMainChain());
545 }
546
547
548 bool CMerkleTx::AcceptTransaction(CTxDB& txdb, bool fCheckInputs)
549 {

```



```

550 if (fClient)
551 {
552     if (!IsInMainChain() && !ClientConnectInputs())
553         return false;
554     return CTransaction::AcceptTransaction(txdb, false);
555 }
556 else
557 {
558     return CTransaction::AcceptTransaction(txdb, fCheckInputs);
559 }
560 }
561
562
563
564 bool CWalletTx::AcceptWalletTransaction(CTxDB& txdb, bool fCheckInputs)
565 {
566     CRITICAL_BLOCK(cs_mapTransactions)
567     {
568         foreach(CMerkleTx& tx, vtxPrev)
569         {
570             if (!tx.IsCoinBase())
571             {
572                 uint256 hash = tx.GetHash();
573                 if (!mapTransactions.count(hash) && !txdb.ContainsTx(hash))
574                     tx.AcceptTransaction(txdb, fCheckInputs);
575             }
576         }
577         if (!IsCoinBase())
578             return AcceptTransaction(txdb, fCheckInputs);
579     }
580     return true;
581 }
582
583 void ReacceptWalletTransactions()
584 {
585     // Reaccept any txes of ours that aren't already in a block
586     CTxDB txdb("r");
587     CRITICAL_BLOCK(cs_mapWallet)
588     {
589         foreach(PAIRTYPE(const uint256, CWalletTx)& item, mapWallet)
590         {
591             CWalletTx& wtx = item.second;
592             if (!wtx.IsCoinBase() && !txdb.ContainsTx(wtx.GetHash()))
593                 wtx.AcceptWalletTransaction(txdb, false);
594         }
595     }
596 }
597
598
599 void CWalletTx::RelayWalletTransaction(CTxDB& txdb)
600 {
601     foreach(const CMerkleTx& tx, vtxPrev)
602     {
603         if (!tx.IsCoinBase())
604         {
605             uint256 hash = tx.GetHash();
606             if (!txdb.ContainsTx(hash))
607                 RelayMessage(CInv(MSG_TX, hash), (CTransaction)tx);
608         }
609     }
610     if (!IsCoinBase())
611     {
612         uint256 hash = GetHash();
613         if (!txdb.ContainsTx(hash))
614         {
615             printf("Relaying wtx %s\n", hash.ToString().substr(0,6).c_str());
616             RelayMessage(CInv(MSG_TX, hash), (CTransaction)*this);
617         }
618     }
619 }
620
621 void RelayWalletTransactions()

```

```

622 {
623     static int64 nLastTime;
624     if (GetTime() - nLastTime < 10 * 60)
625         return;
626     nLastTime = GetTime();
627
628     // Rebroadcast any of our txes that aren't in a block yet
629     printf("RelayWalletTransactions()\n");
630     CTxDB txdb("r");
631     CRITICAL_BLOCK(cs_mapWallet)
632     {
633         // Sort them in chronological order
634         multimap<unsigned int, CWalletTx*> mapSorted;
635         foreach(PAIRTYPE(const uint256, CWalletTx)& item, mapWallet)
636         {
637             CWalletTx& wtx = item.second;
638             mapSorted.insert(make_pair(wtx.nTimeReceived, &wtx));
639         }
640         foreach(PAIRTYPE(const unsigned int, CWalletTx*)& item, mapSorted)
641         {
642             CWalletTx& wtx = *item.second;
643             wtx.RelayWalletTransaction(txdb);
644         }
645     }
646 }
647
648
649
650
651
652
653
654
655
656
657 ///////////////////////////////////////////////////
658 //
659 // CBlock and CBlockIndex
660 //
661
662 bool CBlock::ReadFromDisk(const CBlockIndex* pblockindex, bool fReadTransactions)
663 {
664     return ReadFromDisk(pblockindex->nFile, pblockindex->nBlockPos, fReadTransactions);
665 }
666
667 uint256 GetOrphanRoot(const CBlock* pblock)
668 {
669     // Work back to the first block in the orphan chain
670     while (mapOrphanBlocks.count(pblock->hashPrevBlock))
671         pblock = mapOrphanBlocks[pblock->hashPrevBlock];
672     return pblock->GetHash();
673 }
674
675 int64 CBlock::GetBlockValue(int64 nFees) const
676 {
677     int64 nSubsidy = 50 * COIN;

```

50 BTC—the original block reward, equivalent to 5 billion satoshis. Satoshi explained to Mike Hearn in a 2009 email: 'The number of coins and distribution schedule was an educated guess. It was a difficult choice, because once the network is going it's locked in and we're stuck with it.' He chose 50 as a round number that would produce approximately 21 million total coins via geometric series.

```

678
679 // Subsidy is cut in half every 4 years
680     nSubsidy >= (nBestHeight / 210000);

```

The halving formula: $nSubsidy \gg= (nBestHeight / 210000)$. Every 210,000 blocks (approximately 4 years at 10 minutes per block), the reward halves. This creates a geometric series: $50 + 25 + 12.5 + \dots$ converging to 21 million BTC. The 210,000 block interval was calculated as: $4 \text{ years} \times 365 \text{ days} \times 24 \text{ hours} \times 6 \text{ blocks/hour} \approx 210,240$. Satoshi rounded to 210,000. This predetermined, transparent monetary policy—unlike central banks—cannot be changed by any authority.

```

681
682 return nSubsidy + nFees;
683 }
684
685 unsigned int GetNextWorkRequired(const CBlockIndex* pindexLast)

```

`GetNextWorkRequired()`—the difficulty adjustment algorithm, perhaps Satoshi's most elegant engineering. Without this, Bitcoin would be unstable: too many miners would create blocks too fast, too few would halt the network. This function automatically adjusts difficulty to maintain the 10-minute average regardless of how much hash power joins or leaves.

```

686 {
687 const unsigned int nTargetTimespan = 14 * 24 * 60 * 60; // two weeks

```

`nTargetTimespan` = 14 days (1,209,600 seconds). Difficulty recalculates every 2016 blocks. Why 2 weeks? Satoshi explained on BitcoinTalk: long enough to smooth out variance (a lucky streak of fast blocks shouldn't dramatically lower difficulty), yet short enough to adapt to major hash power changes. The formula: $new_difficulty = old_difficulty \times (actual_time / target_time)$. If blocks came 20% too fast, difficulty increases 20%.

```

688 const unsigned int nTargetSpacing = 10 * 60;

```

`nTargetSpacing` = 10 minutes (600 seconds). Why 10 minutes? Satoshi explained the trade-off: 'Shorter intervals would waste bandwidth and get too hard to keep up downloading all the blocks. Longer intervals would make initial sync faster but increase latency.' He also noted that faster blocks cause more 'orphan races' where miners waste work on blocks that get superseded. 10 minutes allows time for blocks to propagate globally before the next is found.

```

689 const unsigned int nInterval = nTargetTimespan / nTargetSpacing;
690
691 // Genesis block
692 if (pindexLast == NULL)
693 return bnProofOfWorkLimit.GetCompact();
694
695 // Only change once per interval
696 if ((pindexLast->nHeight+1) % nInterval != 0)
697 return pindexLast->nBits;
698
699 // Go back by what we want to be 14 days worth of blocks
700 const CBlockIndex* pindexFirst = pindexLast;
701 for (int i = 0; pindexFirst && i < nInterval-1; i++)
702     pindexFirst = pindexFirst->pprev;
703 assert(pindexFirst);
704
705 // Limit adjustment step
706 unsigned int nActualTimespan = pindexLast->nTime - pindexFirst->nTime;
707 printf(" nActualTimespan = %d before bounds\n", nActualTimespan);
708 if (nActualTimespan < nTargetTimespan/4)
709     nActualTimespan = nTargetTimespan/4;
710 if (nActualTimespan > nTargetTimespan*4)

```

```

711         nActualTimespan = nTargetTimespan*4;
712
713 // Retarget
714     CBigNum bnNew;
715     bnNew.SetCompact(pindexLast->nBits);
716     bnNew *= nActualTimespan;
717     bnNew /= nTargetTimespan;
718
719 if (bnNew > bnProofOfWorkLimit)
720     bnNew = bnProofOfWorkLimit;
721
722 /// debug print
723 printf("\n\nGetNextWorkRequired RETARGET *****\n");
724 printf("nTargetTimespan = %d    nActualTimespan = %d\n", nTargetTimespan, nActualTimespan);
725 printf("Before: %08x  %s\n", pindexLast->nBits, CBigNum().SetCompact(pindexLast-
>nBits).getuint256().ToString().c_str());
726 printf("After:  %08x  %s\n", bnNew.GetCompact(), bnNew.getuint256().ToString().c_str());
727
728 return bnNew.GetCompact();
729 }
730
731
732
733
734
735
736
737
738
739 bool CTransaction::DisconnectInputs(CTxDB& txdb)
740 {
741 // Relinquish previous transactions' spent pointers
742 if (!IsCoinBase())
743 {
744 foreach(const CTxIn& txin, vin)
745 {
746         COutPoint prevout = txin.prevout;
747
748 // Get prev txindex from disk
749         CTxIndex txindex;
750 if (!txdb.ReadTxIndex(prevout.hash, txindex))
751 return error("DisconnectInputs() : ReadTxIndex failed");
752
753 if (prevout.n >= txindex.vSpent.size())
754 return error("DisconnectInputs() : prevout.n out of range");
755
756 // Mark outpoint as not spent
757         txindex.vSpent[prevout.n].SetNull();
758
759 // Write back
760         txdb.UpdateTxIndex(prevout.hash, txindex);
761 }
762 }
763
764 // Remove transaction from index
765 if (!txdb.EraseTxIndex(*this))
766 return error("DisconnectInputs() : EraseTxPos failed");
767
768 return true;
769 }
770
771
772 bool CTransaction::ConnectInputs(CTxDB& txdb, map<uint256, CTxIndex>& mapTestPool, CDiskTxPos posThisTx,
int nHeight, int64& nFees, bool fBlock, bool fMiner, int64 nMinFee)

```

ConnectInputs() - the double-spend prevention engine. Verifies each input references an unspent output. This is Bitcoin's core innovation.

```

773 {

```

```

774 // Take over previous transactions' spent pointers
775 if (!IsCoinBase())
776 {
777     int64 nValueIn = 0;
778     for (int i = 0; i < vin.size(); i++)
779     {
780         COutPoint prevout = vin[i].prevout;
781
782         // Read txindex
783         CTxIndex txindex;
784         bool fFound = true;
785         if (fMiner && mapTestPool.count(prevout.hash))
786         {
787             // Get txindex from current proposed changes
788             txindex = mapTestPool[prevout.hash];
789         }
790         else
791         {
792             // Read txindex from txdb
793             fFound = txdb.ReadTxIndex(prevout.hash, txindex);
794         }
795         if (!fFound && (fBlock || fMiner))
796             return fMiner ? false : error("ConnectInputs() : %s prev tx %s index entry not found",
              GetHash().ToString().substr(0,6).c_str(), prevout.hash.ToString().substr(0,6).c_str());
797
798         // Read txPrev
799         CTransaction txPrev;
800         if (!fFound || txindex.pos == CDiskTxPos(1,1,1))
801         {
802             // Get prev tx from single transactions in memory
803             CRITICAL_BLOCK(cs_mapTransactions)
804             {
805                 if (!mapTransactions.count(prevout.hash))
806                     return error("ConnectInputs() : %s mapTransactions prev not found %s",
                        GetHash().ToString().substr(0,6).c_str(), prevout.hash.ToString().substr(0,6).c_str());
807                 txPrev = mapTransactions[prevout.hash];
808             }
809             if (!fFound)
810                 txindex.vSpent.resize(txPrev.vout.size());
811         }
812         else
813         {
814             // Get prev tx from disk
815             if (!txPrev.ReadFromDisk(txindex.pos))
816                 return error("ConnectInputs() : %s ReadFromDisk prev tx %s failed",
                        GetHash().ToString().substr(0,6).c_str(), prevout.hash.ToString().substr(0,6).c_str());
817         }
818
819         if (prevout.n >= txPrev.vout.size() || prevout.n >= txindex.vSpent.size())
820             return error("ConnectInputs() : %s prevout.n out of range %d %d %d",
                        GetHash().ToString().substr(0,6).c_str(), prevout.n, txPrev.vout.size(), txindex.vSpent.size());
821
822         // If prev is coinbase, check that it's matured
823         if (txPrev.IsCoinBase())
824             for (CBlockIndex* pindex = pindexBest; pindex && nBestHeight - pindex->nHeight < COINBASE_MATURITY-1;
              pindex = pindex->pprev)
825                 if (pindex->nBlockPos == txindex.pos.nBlockPos && pindex->nFile == txindex.pos.nFile)
826                     return error("ConnectInputs() : tried to spend coinbase at depth %d", nBestHeight - pindex->nHeight);
827
828         // Verify signature
829         if (!VerifySignature(txPrev, *this, i))
830             return error("ConnectInputs() : %s VerifySignature failed", GetHash().ToString().substr(0,6).c_str());
831
832         // Check for conflicts
833         if (!txindex.vSpent[prevout.n].IsNull())
834             return fMiner ? false : error("ConnectInputs() : %s prev tx already used at %s",
              GetHash().ToString().substr(0,6).c_str(), txindex.vSpent[prevout.n].ToString().c_str());

```

'prev tx already used' - double-spend detected! The transaction tries to spend coins already spent elsewhere. Rejected.

```

835
836 // Mark outpoints as spent
837     txindex.vSpent[prevout.n] = posThisTx;
838
839 // Write back
840 if (fBlock)
841     txdb.UpdateTxIndex(prevout.hash, txindex);
842 else if (fMiner)
843     mapTestPool[prevout.hash] = txindex;
844
845     nValueIn += txPrev.vout[prevout.n].nValue;
846 }
847
848 // Tally transaction fees
849     int64 nTxFee = nValueIn - GetValueOut();
850 if (nTxFee < 0)
851     return error("ConnectInputs() : %s nTxFee < 0", GetHash().ToString().substr(0,6).c_str());
852 if (nTxFee < nMinFee)
853     return false;
854     nFees += nTxFee;
855 }
856
857 if (fBlock)
858 {
859     // Add transaction to disk index
860     if (!txdb.AddTxIndex(*this, posThisTx, nHeight))
861         return error("ConnectInputs() : AddTxPos failed");
862 }
863 else if (fMiner)
864 {
865     // Add transaction to test pool
866     mapTestPool[GetHash()] = CTxIndex(CDiskTxPos(1,1,1), vout.size());
867 }
868
869 return true;
870 }
871
872
873 bool CTransaction::ClientConnectInputs()
874 {
875     if (IsCoinBase())
876         return false;
877
878     // Take over previous transactions' spent pointers
879     CRITICAL_BLOCK(cs_mapTransactions)
880     {
881         int64 nValueIn = 0;
882         for (int i = 0; i < vin.size(); i++)
883         {
884             // Get prev tx from single transactions in memory
885             COutPoint prevout = vin[i].prevout;
886             if (!mapTransactions.count(prevout.hash))
887                 return false;
888             CTransaction& txPrev = mapTransactions[prevout.hash];
889
890             if (prevout.n >= txPrev.vout.size())
891                 return false;
892
893             // Verify signature
894             if (!VerifySignature(txPrev, *this, i))
895                 return error("ConnectInputs() : VerifySignature failed");
896
897             ///// this is redundant with the mapNextTx stuff, not sure which I want to get rid of
898             ///// this has to go away now that posNext is gone
899             // Check for conflicts

```

```

900 // if (!txPrev.vout[prevout.n].posNext.IsNull())
901 //     return error("ConnectInputs() : prev tx already used");
902 //
903 // // Flag outpoints as used
904 // txPrev.vout[prevout.n].posNext = posThisTx;
905
906     nValueIn += txPrev.vout[prevout.n].nValue;
907 }
908 if (GetValueOut() > nValueIn)
909 return false;
910 }
911
912 return true;
913 }
914
915
916
917
918 bool CBlock::DisconnectBlock(CTxDB& txdb, CBlockIndex* pindex)
919 {
920 // Disconnect in reverse order
921 for (int i = vtx.size()-1; i >= 0; i--)
922 if (!vtx[i].DisconnectInputs(txdb))
923 return false;
924
925 // Update block index on disk without changing it in memory.
926 // The memory index structure will be changed after the db commits.
927 if (pindex->pprev)
928 {
929     CDiskBlockIndex blockindexPrev(pindex->pprev);
930     blockindexPrev.hashNext = 0;
931     txdb.WriteBlockIndex(blockindexPrev);
932 }
933
934 return true;
935 }
936
937 bool CBlock::ConnectBlock(CTxDB& txdb, CBlockIndex* pindex)
938 {
939 /// issue here: it doesn't know the version
940 unsigned int nTxPos = pindex->nBlockPos + ::GetSerializeSize(CBlock(), SER_DISK) - 1 +
    GetSizeOfCompactSize(vtx.size());
941
942     map<uint256, CTxIndex> mapUnused;
943     int64 nFees = 0;
944     foreach(CTransaction& tx, vtx)
945     {
946         CDiskTxPos posThisTx(pindex->nFile, pindex->nBlockPos, nTxPos);
947         nTxPos += ::GetSerializeSize(tx, SER_DISK);
948
949         if (!tx.ConnectInputs(txdb, mapUnused, posThisTx, pindex->nHeight, nFees, true, false))
950             return false;
951     }
952
953     if (vtx[0].GetValueOut() > GetBlockValue(nFees))
954         return false;
955
956 // Update block index on disk without changing it in memory.
957 // The memory index structure will be changed after the db commits.
958 if (pindex->pprev)
959 {
960     CDiskBlockIndex blockindexPrev(pindex->pprev);
961     blockindexPrev.hashNext = pindex->GetBlockHash();
962     txdb.WriteBlockIndex(blockindexPrev);
963 }
964
965 // Watch for transactions paying to me
966 foreach(CTransaction& tx, vtx)
967     AddToWalletIfMine(tx, this);
968
969 return true;
970 }

```

```

971
972
973
974 bool Reorganize(CTxDB& txdb, CBlockIndex* pindexNew)

```

Reorganize() - handles chain splits. When a longer valid chain appears, Bitcoin switches to it. 'The majority decision is represented by the longest chain.'

```

975 {
976     printf("*** REORGANIZE ***\n");
977
978     // Find the fork
979     CBlockIndex* pfork = pindexBest;
980     CBlockIndex* plonger = pindexNew;
981     while (pfork != plonger)
982     {
983         if (!(pfork = pfork->pprev))
984             return error("Reorganize() : pfork->pprev is null");
985         while (plonger->nHeight > pfork->nHeight)
986             if (!(plonger = plonger->pprev))
987                 return error("Reorganize() : plonger->pprev is null");
988     }
989
990     // List of what to disconnect
991     vector<CBlockIndex*> vDisconnect;

```

vDisconnect - blocks to undo. During reorg, transactions return to mempool. Users may see confirmations temporarily reverse.

```

992     for (CBlockIndex* pindex = pindexBest; pindex != pfork; pindex = pindex->pprev)
993         vDisconnect.push_back(pindex);
994
995     // List of what to connect
996     vector<CBlockIndex*> vConnect;

```

vConnect - blocks to apply. The new chain's transactions take precedence. This is why '6 confirmations' became the security standard.

```

997     for (CBlockIndex* pindex = pindexNew; pindex != pfork; pindex = pindex->pprev)
998         vConnect.push_back(pindex);
999     reverse(vConnect.begin(), vConnect.end());
1000
1001     // Disconnect shorter branch
1002     vector<CTransaction> vResurrect;
1003     foreach(CBlockIndex* pindex, vDisconnect)
1004     {
1005         CBlock block;
1006         if (!block.ReadFromDisk(pindex->nFile, pindex->nBlockPos, true))
1007             return error("Reorganize() : ReadFromDisk for disconnect failed");
1008         if (!block.DisconnectBlock(txdb, pindex))
1009             return error("Reorganize() : DisconnectBlock failed");
1010
1011         // Queue memory transactions to resurrect
1012         foreach(const CTransaction& tx, block.vtx)
1013             if (!tx.IsCoinBase())
1014                 vResurrect.push_back(tx);
1015     }
1016
1017     // Connect longer branch
1018     vector<CTransaction> vDelete;
1019     for (int i = 0; i < vConnect.size(); i++)
1020     {
1021         CBlockIndex* pindex = vConnect[i];
1022         CBlock block;

```



```

1023 if (!block.ReadFromDisk(pindex->nFile, pindex->nBlockPos, true))
1024 return error("Reorganize() : ReadFromDisk for connect failed");
1025 if (!block.ConnectBlock(txdb, pindex))
1026 {
1027 // Invalid block, delete the rest of this branch
1028     txdb.TxnAbort();
1029 for (int j = i; j < vConnect.size(); j++)
1030 {
1031     CBlockIndex* pindex = vConnect[j];
1032     pindex->EraseBlockFromDisk();
1033     txdb.EraseBlockIndex(pindex->GetBlockHash());
1034     mapBlockIndex.erase(pindex->GetBlockHash());
1035 delete pindex;
1036 }
1037 return error("Reorganize() : ConnectBlock failed");
1038 }
1039
1040 // Queue memory transactions to delete
1041 foreach(const CTransaction& tx, block.vtx)
1042     vDelete.push_back(tx);
1043 }
1044 if (!txdb.WriteHashBestChain(pindexNew->GetBlockHash()))
1045 return error("Reorganize() : WriteHashBestChain failed");
1046
1047 // Commit now because resurrecting could take some time
1048 txdb.TxnCommit();
1049
1050 // Disconnect shorter branch
1051 foreach(CBlockIndex* pindex, vDisconnect)
1052 if (pindex->pprev)
1053     pindex->pprev->pnext = NULL;
1054
1055 // Connect longer branch
1056 foreach(CBlockIndex* pindex, vConnect)
1057 if (pindex->pprev)
1058     pindex->pprev->pnext = pindex;
1059
1060 // Resurrect memory transactions that were in the disconnected branch
1061 foreach(CTransaction& tx, vResurrect)
1062     tx.AcceptTransaction(txdb, false);
1063
1064 // Delete redundant memory transactions that are in the connected branch
1065 foreach(CTransaction& tx, vDelete)
1066     tx.RemoveFromMemoryPool();
1067
1068 return true;
1069 }
1070
1071
1072 bool CBlock::AddToBlockIndex(unsigned int nFile, unsigned int nBlockPos)
1073 {
1074 // Check for duplicate
1075     uint256 hash = GetHash();
1076 if (mapBlockIndex.count(hash))
1077 return error("AddToBlockIndex() : %s already exists", hash.ToString().substr(0,14).c_str());
1078
1079 // Construct new block index object
1080     CBlockIndex* pindexNew = new CBlockIndex(nFile, nBlockPos, *this);
1081 if (!pindexNew)
1082 return error("AddToBlockIndex() : new CBlockIndex failed");
1083     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.insert(make_pair(hash, pindexNew)).first;
1084     pindexNew->phashBlock = &(*mi).first;
1085     map<uint256, CBlockIndex*>::iterator miPrev = mapBlockIndex.find(hashPrevBlock);
1086 if (miPrev != mapBlockIndex.end())
1087 {
1088     pindexNew->pprev = (*miPrev).second;
1089     pindexNew->nHeight = pindexNew->pprev->nHeight + 1;
1090 }
1091
1092     CTxDB txdb;
1093     txdb.TxnBegin();
1094     txdb.WriteBlockIndex(CDiskBlockIndex(pindexNew));

```

```

1095
1096 // New best
1097 if (pindexNew->nHeight > nBestHeight)
1098 {
1099     if (pindexGenesisBlock == NULL && hash == hashGenesisBlock)
1100     {
1101         pindexGenesisBlock = pindexNew;
1102         txdb.WriteHashBestChain(hash);
1103     }
1104     else if (hashPrevBlock == hashBestChain)
1105     {
1106         // Adding to current best branch
1107         if (!ConnectBlock(txdb, pindexNew) || !txdb.WriteHashBestChain(hash))
1108         {
1109             txdb.TxnAbort();
1110             pindexNew->EraseBlockFromDisk();
1111             mapBlockIndex.erase(pindexNew->GetBlockHash());
1112             delete pindexNew;
1113             return error("AddToBlockIndex() : ConnectBlock failed");
1114         }
1115         txdb.TxnCommit();
1116         pindexNew->pprev->pnext = pindexNew;
1117     }
1118     // Delete redundant memory transactions
1119     foreach(CTransaction& tx, vtx)
1120         tx.RemoveFromMemoryPool();
1121 }
1122 else
1123 {
1124     // New best branch
1125     if (!Reorganize(txdb, pindexNew))
1126     {
1127         txdb.TxnAbort();
1128         return error("AddToBlockIndex() : Reorganize failed");
1129     }
1130 }
1131
1132 // New best link
1133 hashBestChain = hash;
1134 pindexBest = pindexNew;
1135 nBestHeight = pindexBest->nHeight;
1136 nTransactionsUpdated++;
1137 printf("AddToBlockIndex: new best=%s height=%d\n", hashBestChain.ToString().substr(0,14).c_str(),
nBestHeight);
1138 }
1139
1140 txdb.TxnCommit();
1141 txdb.Close();
1142
1143 // Relay wallet transactions that haven't gotten in yet
1144 if (pindexNew == pindexBest)
1145     RelayWalletTransactions();
1146
1147 MainFrameRepaint();
1148 return true;
1149 }
1150
1151
1152
1153
1154 bool CBlock::CheckBlock() const

```

`CheckBlock()` - structural validation. Verifies size limits, timestamp sanity, merkle root, and proof-of-work. Rejects malformed blocks.

```

1155 {
1156 // These are checks that are independent of context
1157 // that can be verified before saving an orphan block.
1158

```

```

1159 // Size limits
1160 if (vtx.empty() || vtx.size() > MAX_SIZE || ::GetSerializeSize(*this, SER_DISK) > MAX_SIZE)
1161     return error("CheckBlock() : size limits failed");
1162
1163 // Check timestamp
1164 if (nTime > GetAdjustedTime() + 2 * 60 * 60)
1165     return error("CheckBlock() : block timestamp too far in the future");
1166
1167 // First transaction must be coinbase, the rest must not be
1168 if (vtx.empty() || !vtx[0].IsCoinBase())
1169     return error("CheckBlock() : first tx is not coinbase");
1170 for (int i = 1; i < vtx.size(); i++)
1171     if (vtx[i].IsCoinBase())
1172         return error("CheckBlock() : more than one coinbase");
1173
1174 // Check transactions
1175 foreach(const CTransaction& tx, vtx)
1176     if (!tx.CheckTransaction())
1177         return error("CheckBlock() : CheckTransaction failed");
1178
1179 // Check proof of work matches claimed amount
1180 if (CBigNum().SetCompact(nBits) > bnProofOfWorkLimit)

```

`bnProofOfWorkLimit` check - ensures difficulty isn't impossibly easy. The target hash must be below maximum difficulty (all 1s would be trivial).

```

1181 return error("CheckBlock() : nBits below minimum work");
1182 if (GetHash() > CBigNum().SetCompact(nBits).getuint256())
1183     return error("CheckBlock() : hash doesn't match nBits");
1184
1185 // Check merkle root
1186 if (hashMerkleRoot != BuildMerkleTree())
1187     return error("CheckBlock() : hashMerkleRoot mismatch");
1188
1189 return true;
1190 }
1191
1192 bool CBlock::AcceptBlock()

```

`AcceptBlock()` - adds validated block to chain. Updates indexes, checks difficulty matches expected, writes to disk. Point of no return.

```

1193 {
1194     // Check for duplicate
1195     uint256 hash = GetHash();
1196     if (mapBlockIndex.count(hash))
1197         return error("AcceptBlock() : block already in mapBlockIndex");
1198
1199     // Get prev block index
1200     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashPrevBlock);
1201     if (mi == mapBlockIndex.end())
1202         return error("AcceptBlock() : prev block not found");
1203     CBlockIndex* pindexPrev = (*mi).second;
1204
1205     // Check timestamp against prev
1206     if (nTime <= pindexPrev->GetMedianTimePast())
1207         return error("AcceptBlock() : block's timestamp is too early");
1208
1209     // Check proof of work
1210     if (nBits != GetNextWorkRequired(pindexPrev))
1211         return error("AcceptBlock() : incorrect proof of work");
1212
1213     // Write block to history file
1214     unsigned int nFile;
1215     unsigned int nBlockPos;
1216     if (!WriteToDisk(!fClient, nFile, nBlockPos))

```

```

1217 return error("AcceptBlock() : WriteToDisk failed");
1218 if (!AddToBlockIndex(nFile, nBlockPos))
1219 return error("AcceptBlock() : AddToBlockIndex failed");
1220
1221 if (hashBestChain == hash)
1222 RelayInventory(CInv(MSG_BLOCK, hash));
1223
1224 // // Add atoms to user reviews for coins created
1225 // vector<unsigned char> vchPubKey;
1226 // if (ExtractPubKey(vtx[0].vout[0].scriptPubKey, false, vchPubKey))
1227 // {
1228 //     unsigned short nAtom = GetRand(USHRT_MAX - 100) + 100;
1229 //     vector<unsigned short> vAtoms(1, nAtom);
1230 //     AddAtomsAndPropagate(Hash(vchPubKey.begin(), vchPubKey.end()), vAtoms, true);
1231 // }
1232
1233 return true;
1234 }
1235
1236 bool ProcessBlock(CNode* pfrom, CBlock* pblock)

```

`ProcessBlock()` - main entry point for new blocks. First checks if we've seen it, validates with `CheckBlock()`, then `AcceptBlock()` or orphan it.

```

1237 {
1238     // Check for duplicate
1239     uint256 hash = pblock->GetHash();
1240     if (mapBlockIndex.count(hash))
1241         return error("ProcessBlock() : already have block %d %s", mapBlockIndex[hash]->nHeight,
1242             hash.ToString().substr(0,14).c_str());
1243     if (mapOrphanBlocks.count(hash))
1244         return error("ProcessBlock() : already have block (orphan) %s", hash.ToString().substr(0,14).c_str());
1245     // Preliminary checks
1246     if (!pblock->CheckBlock())
1247     {
1248         delete pblock;
1249         return error("ProcessBlock() : CheckBlock FAILED");
1250     }
1251
1252     // If don't already have its previous block, shunt it off to holding area until we get it
1253     if (!mapBlockIndex.count(pblock->hashPrevBlock))
1254     {
1255         printf("ProcessBlock: ORPHAN BLOCK, prev=%s\n", pblock->hashPrevBlock.ToString().substr(0,14).c_str());
1256         mapOrphanBlocks.insert(make_pair(hash, pblock));
1257         mapOrphanBlocksByPrev.insert(make_pair(pblock->hashPrevBlock, pblock));
1258
1259         // Ask this guy to fill in what we're missing
1260         if (pfrom)
1261             pfrom->PushMessage("getblocks", CBlockLocator(pindexBest), GetOrphanRoot(pblock));
1262         return true;
1263     }
1264
1265     // Store to disk
1266     if (!pblock->AcceptBlock())
1267     {
1268         delete pblock;
1269         return error("ProcessBlock() : AcceptBlock FAILED");
1270     }
1271     delete pblock;
1272
1273     // Recursively process any orphan blocks that depended on this one
1274     vector<uint256> vWorkQueue;
1275     vWorkQueue.push_back(hash);
1276     for (int i = 0; i < vWorkQueue.size(); i++)
1277     {
1278         uint256 hashPrev = vWorkQueue[i];
1279         for (multimap<uint256, CBlock*>::iterator mi = mapOrphanBlocksByPrev.lower_bound(hashPrev);
1280             mi != mapOrphanBlocksByPrev.upper_bound(hashPrev);

```

```

1281 ++mi)
1282 {
1283     CBlock* pblockOrphan = (*mi).second;
1284     if (pblockOrphan->AcceptBlock())
1285         vWorkQueue.push_back(pblockOrphan->GetHash());
1286     mapOrphanBlocks.erase(pblockOrphan->GetHash());
1287     delete pblockOrphan;
1288 }
1289     mapOrphanBlocksByPrev.erase(hashPrev);
1290 }
1291
1292 printf("ProcessBlock: ACCEPTED\n");
1293 return true;
1294 }
1295
1296
1297
1298
1299
1300
1301
1302
1303 template<typename Stream>
1304 bool ScanMessageStart(Stream& s)
1305 {
1306     // Scan ahead to the next pchMessageStart, which should normally be immediately
1307     // at the file pointer. Leaves file pointer at end of pchMessageStart.
1308     s.clear(0);
1309     short prevmask = s.exceptions(0);
1310     const char* p = BEGIN(pchMessageStart);
1311     try
1312     {
1313         loop
1314     {
1315         char c;
1316         s.read(&c, 1);
1317         if (s.fail())
1318         {
1319             s.clear(0);
1320             s.exceptions(prevmask);
1321             return false;
1322         }
1323         if (*p != c)
1324             p = BEGIN(pchMessageStart);
1325         if (*p == c)
1326         {
1327             if (++p == END(pchMessageStart))
1328             {
1329                 s.clear(0);
1330                 s.exceptions(prevmask);
1331                 return true;
1332             }
1333         }
1334     }
1335 }
1336 catch (...)
1337 {
1338     s.clear(0);
1339     s.exceptions(prevmask);
1340     return false;
1341 }
1342 }
1343
1344 string GetAppDir()
1345 {
1346     string strDir;
1347     if (!strSetDataDir.empty())
1348     {
1349         strDir = strSetDataDir;
1350     }
1351     else if (getenv("APPDATA"))
1352     {

```

```

1353         strDir = sprintf("%s\\Bitcoin", getenv("APPDATA"));
1354     }
1355     else if (getenv("USERPROFILE"))
1356     {
1357         string strAppData = sprintf("%s\\Application Data", getenv("USERPROFILE"));
1358         static bool fMkdirDone;
1359         if (!fMkdirDone)
1360         {
1361             fMkdirDone = true;
1362             _mkdir(strAppData.c_str());
1363         }
1364         strDir = sprintf("%s\\Bitcoin", strAppData.c_str());
1365     }
1366     else
1367     {
1368         return ".";
1369     }
1370     static bool fMkdirDone;
1371     if (!fMkdirDone)
1372     {
1373         fMkdirDone = true;
1374         _mkdir(strDir.c_str());
1375     }
1376     return strDir;
1377 }
1378
1379 FILE* OpenBlockFile(unsigned int nFile, unsigned int nBlockPos, const char* pszMode)
1380 {
1381     if (nFile == -1)
1382         return NULL;
1383     FILE* file = fopen(sprintf("%s\\blk%04d.dat", GetAppDir().c_str(), nFile).c_str(), pszMode);
1384     if (!file)
1385         return NULL;
1386     if (nBlockPos != 0 && !strchr(pszMode, 'a') && !strchr(pszMode, 'w'))
1387     {
1388         if (fseek(file, nBlockPos, SEEK_SET) != 0)
1389         {
1390             fclose(file);
1391             return NULL;
1392         }
1393     }
1394     return file;
1395 }
1396
1397 static unsigned int nCurrentBlockFile = 1;
1398
1399 FILE* AppendBlockFile(unsigned int& nFileRet)
1400 {
1401     nFileRet = 0;
1402     loop
1403     {
1404         FILE* file = OpenBlockFile(nCurrentBlockFile, 0, "ab");
1405         if (!file)
1406             return NULL;
1407         if (fseek(file, 0, SEEK_END) != 0)
1408             return NULL;
1409         // FAT32 filesize max 4GB, fseek and ftell max 2GB, so we must stay under 2GB
1410         if (ftell(file) < 0x7F000000 - MAX_SIZE)
1411         {
1412             nFileRet = nCurrentBlockFile;
1413             return file;
1414         }
1415         fclose(file);
1416         nCurrentBlockFile++;
1417     }
1418 }
1419
1420 bool LoadBlockIndex(bool fAllowNew)
1421 {
1422     //
1423     // Load block index
1424     //

```

```

1425     CTxDB txdb("cr");
1426     if (!txdb.LoadBlockIndex())
1427         return false;
1428     txdb.Close();
1429
1430 //
1431 // Init with genesis block
1432 //
1433     if (mapBlockIndex.empty())
1434     {
1435         if (!fAllowNew)
1436             return false;
1437
1438
1439 // Genesis Block:

```

Genesis Block creation. Satoshi hardcoded this first block. The hash was computed offline; this code just reconstructs it for verification.

```

1440 // GetHash()         = 0x000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
1441 // hashMerkleRoot = 0x4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b
1442 // txNew.vin[0].scriptSig      = 486604799 4
1443 // txNew.vout[0].nValue      = 5000000000
1444 // txNew.vout[0].scriptPubKey =
1445 // block.nVersion = 1
1446 // block.nTime    = 1231006505
1447 // block.nBits    = 0x1d00ffff
1448 // block.nNonce   = 2083236893
1449 // CBlock(hash=000000000019d6, ver=1, hashPrevBlock=00000000000000, hashMerkleRoot=4a5e1e,
1450 //         nTime=1231006505, nBits=1d00ffff, nNonce=2083236893, vtx=1)
1451 //     CTransaction(hash=4a5e1e, ver=1, vin.size=1, vout.size=1, nLockTime=0)
1452 //         CTxIn(COutPoint(000000, -1), coinbase
1453 //             04ffff001d0104455468652054696d65732030332f4a616e2f32303039204368616e63656c66f72206f6e206272696e6b206f66
1454 //             207365636f6e64206261696c6f7574206666f722062616e6b73)
1455 //         CTxOut(nValue=50.00000000, scriptPubKey=0x5F1DF16B2B704C8A578D0B)
1456 //         vMerkleTree: 4a5e1e
1457 // Genesis block
1458 char* pszTimestamp = "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks";
1459 CTransaction txNew;
1460 txNew.vin.resize(1);
1461 txNew.vout.resize(1);
1462 txNew.vin[0].scriptSig = CScript() << 486604799 << CBigNum(4) << vector<unsigned char>
1463 ((unsigned char*)pszTimestamp, (unsigned char*)pszTimestamp + strlen(pszTimestamp));
1464 txNew.vout[0].nValue = 50 * COIN;

```

50 * COIN—the first 50 bitcoins ever created, worth exactly \$0 at the time (and now worth millions). Curiously, these specific coins are permanently unspendable due to a quirk: the genesis block's coinbase transaction isn't indexed in the UTXO database. Whether this was intentional or an oversight is unknown. Satoshi never commented on it, and no one has ever been able to ask.

```

1462     txNew.vout[0].scriptPubKey = CScript() <<
1463         CBigNum("0x5F1DF16B2B704C8A578D0BBAF74D385CDE12C11EE50455F3C438EF4C3FBCF649B6DE611FEAE06279A60939E028A8D
1464         65C10B73071A6F16719274855FEB0FD8A6704") << OP_CHECKSIG;
1465     CBlock block;
1466     block.vtx.push_back(txNew);
1467     block.hashPrevBlock = 0;

```

`hashPrevBlock = 0`—the genesis block's defining characteristic. Every other block must reference a valid previous block; this is the only block that points to nothing. The all-zeros hash is Bitcoin's 'Big Bang moment'—the chain has to start somewhere. Validation code explicitly checks for this: if a block claims `hashPrevBlock = 0` but isn't the known genesis block, it's rejected. This single zero anchors the entire blockchain's integrity.

```
1466     block.hashMerkleRoot = block.BuildMerkleTree();
1467     block.nVersion = 1;
1468     block.nTime = 1231006505;
1469     block.nBits = 0x1d00ffff;
```

`nBits = 0x1d00ffff`—the initial difficulty target in compact format. This expands to a 256-bit target where valid hashes must start with approximately 32 zero bits. At this difficulty, Satoshi's circa-2008 computer (likely a modest desktop or laptop) could find valid blocks in minutes to hours. For comparison: today's network runs at difficulty requiring ~80+ leading zero bits, approximately 10^{13} times harder.

```
1470     block.nNonce = 2083236893;
```

`nNonce = 2083236893`—the magic number Satoshi discovered that makes the genesis block hash valid. He had to try approximately 2 billion nonces before finding this one (nonce means 'number used once'). The resulting hash `00000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f` starts with the required zeros, proving work was done. This specific nonce is now immortalized in every Bitcoin client.

```
1471
1472 /// debug print, delete this later
1473 printf("%s\n", block.GetHash().ToString().c_str());
1474 printf("%s\n", block.hashMerkleRoot.ToString().c_str());
1475 printf("%s\n", hashGenesisBlock.ToString().c_str());
1476     txNew.vout[0].scriptPubKey.print();
1477     block.print();
1478     assert(block.hashMerkleRoot ==
1479            uint256("0x4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b"));
1480     assert(block.GetHash() == hashGenesisBlock);
1481
1482 // Start new block file
1483     unsigned int nFile;
1484     unsigned int nBlockPos;
1485     if (!block.WriteToDisk(!fClient, nFile, nBlockPos))
1486         return error("LoadBlockIndex() : writing genesis block to disk failed");
1487     if (!block.AddToBlockIndex(nFile, nBlockPos))
1488         return error("LoadBlockIndex() : genesis block not accepted");
1489 }
1490
1491 return true;
1492 }
1493
1494
1495
1496 void PrintBlockTree()
1497 {
1498     // precompute tree structure
1499     map<CBlockIndex*, vector<CBlockIndex*> > mapNext;
1500     for (map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.begin(); mi != mapBlockIndex.end(); ++mi)
1501     {
1502         CBlockIndex* pindex = (*mi).second;
1503         mapNext[pindex->pprev].push_back(pindex);
1504     } // test
```



```

1505 //while (rand() % 3 == 0)
1506 //    mapNext[pindex->pprev].push_back(pindex);
1507 }
1508
1509     vector<pair<int, CBlockIndex*> > vStack;
1510     vStack.push_back(make_pair(0, pindexGenesisBlock));
1511
1512     int nPrevCol = 0;
1513     while (!vStack.empty())
1514     {
1515         int nCol = vStack.back().first;
1516         CBlockIndex* pindex = vStack.back().second;
1517         vStack.pop_back();
1518
1519         // print split or gap
1520         if (nCol > nPrevCol)
1521         {
1522             for (int i = 0; i < nCol-1; i++)
1523                 printf("| ");
1524             printf("\\\\n");
1525         }
1526         else if (nCol < nPrevCol)
1527         {
1528             for (int i = 0; i < nCol; i++)
1529                 printf("| ");
1530             printf("\\n");
1531         }
1532         nPrevCol = nCol;
1533
1534         // print columns
1535         for (int i = 0; i < nCol; i++)
1536             printf("| ");
1537
1538         // print item
1539         CBlock block;
1540         block.ReadFromDisk(pindex, true);
1541         printf("%d (%u,%u) %s %s tx %d",
1542             pindex->nHeight,
1543             pindex->nFile,
1544             pindex->nBlockPos,
1545             block.GetHash().ToString().substr(0,14).c_str(),
1546             DateTimeStr(block.nTime).c_str(),
1547             block.vtx.size());
1548
1549         CRITICAL_BLOCK(cs_mapWallet)
1550         {
1551             if (mapWallet.count(block.vtx[0].GetHash()))
1552             {
1553                 CWalletTx& wtx = mapWallet[block.vtx[0].GetHash()];
1554                 printf("    mine: %d %d %d", wtx.GetDepthInMainChain(), wtx.GetBlocksToMaturity(), wtx.GetCredit());
1555             }
1556         }
1557         printf("\\n");
1558
1559     }
1560     // put the main timechain first
1561     vector<CBlockIndex*> vNext = mapNext[pindex];
1562     for (int i = 0; i < vNext.size(); i++)
1563     {
1564         if (vNext[i]->pnext)
1565         {
1566             swap(vNext[0], vNext[i]);
1567             break;
1568         }
1569     }
1570
1571     // iterate children
1572     for (int i = 0; i < vNext.size(); i++)
1573         vStack.push_back(make_pair(nCol+i, vNext[i]));
1574 }
1575 }
1576

```

```

1577
1578
1579
1580
1581
1582
1583
1584
1585
1586 ///////////////////////////////////////////////////////////////////
1587 //
1588 // Messages
1589 //
1590
1591
1592 bool AlreadyHave(CTxDB& txdb, const CInv& inv)
1593 {
1594     switch (inv.type)
1595     {
1596     case MSG_TX: return mapTransactions.count(inv.hash) || txdb.ContainsTx(inv.hash);
1597     case MSG_BLOCK: return mapBlockIndex.count(inv.hash) || mapOrphanBlocks.count(inv.hash);
1598     case MSG_REVIEW: return true;
1599     case MSG_PRODUCT: return mapProducts.count(inv.hash);
1600     }
1601     // Don't know what it is, just say we already got one
1602     return true;
1603 }
1604
1605
1606
1607
1608
1609
1610
1611 bool ProcessMessages(CNode* pfrom)
1612 {
1613     CDataStream& vRecv = pfrom->vRecv;
1614     if (vRecv.empty())
1615         return true;
1616     printf("ProcessMessages(%d bytes)\n", vRecv.size());
1617
1618     //
1619     // Message format
1620     // (4) message start
1621     // (12) command
1622     // (4) size
1623     // (x) data
1624     //
1625
1626     loop
1627     {
1628         // Scan for message start
1629         CDataStream::iterator pstart = search(vRecv.begin(), vRecv.end(), BEGIN(pchMessageStart),
            END(pchMessageStart));
1630         if (vRecv.end() - pstart < sizeof(CMessageHeader))
1631         {
1632             if (vRecv.size() > sizeof(CMessageHeader))
1633             {
1634                 printf("\n\nPROCESSMESSAGE MESSAGESTART NOT FOUND\n\n");
1635                 vRecv.erase(vRecv.begin(), vRecv.end() - sizeof(CMessageHeader));
1636             }
1637             break;
1638         }
1639         if (pstart - vRecv.begin() > 0)
1640             printf("\n\nPROCESSMESSAGE SKIPPED %d BYTES\n\n", pstart - vRecv.begin());
1641         vRecv.erase(vRecv.begin(), pstart);
1642
1643         // Read header
1644         CMessageHeader hdr;
1645         vRecv >> hdr;
1646         if (!hdr.IsValid())
1647         {

```

```

1648 printf("\n\nPROCESSMESSAGE: ERRORS IN HEADER %s\n\n", hdr.GetCommand().c_str());
1649 continue;
1650 }
1651     string strCommand = hdr.GetCommand();
1652
1653 // Message size
1654 unsigned int nMessageSize = hdr.nMessageSize;
1655 if (nMessageSize > vRecv.size())
1656 {
1657     // Rewind and wait for rest of message
1658     //// need a mechanism to give up waiting for overlong message size error
1659     printf("MESSAGE-BREAK 2\n");
1660     vRecv.insert(vRecv.begin(), BEGIN(hdr), END(hdr));
1661     Sleep(100);
1662     break;
1663 }
1664
1665 // Copy message to its own buffer
1666     CDataStream vMsg(vRecv.begin(), vRecv.begin() + nMessageSize, vRecv.nType, vRecv.nVersion);
1667     vRecv.ignore(nMessageSize);
1668
1669 // Process message
1670 bool fRet = false;
1671 try
1672 {
1673     CheckForShutdown(2);
1674     CRITICAL_BLOCK(cs_main)
1675     {
1676         fRet = ProcessMessage(pfrom, strCommand, vMsg);
1677     }
1678     CATCH_PRINT_EXCEPTION("ProcessMessage()")
1679     if (!fRet)
1680     printf("ProcessMessage(%s, %d bytes) from %s to %s FAILED\n", strCommand.c_str(), nMessageSize, pfrom->addr.ToString().c_str(), addrLocalHost.ToString().c_str());
1681 }
1682
1683     vRecv.Compact();
1684     return true;
1685 }
1686
1687
1688
1689
1690 bool ProcessMessage(CNode* pfrom, string strCommand, CDataStream& vRecv)
1691 {
1692     static map<unsigned int, vector<unsigned char> > mapReuseKey;
1693     printf("received: %-12s (%d bytes) ", strCommand.c_str(), vRecv.size());
1694     for (int i = 0; i < min(vRecv.size(), (unsigned int)25); i++)
1695         printf("%02x ", vRecv[i] & 0xff);
1696     printf("\n");
1697     if (nDropMessagesTest > 0 && GetRand(nDropMessagesTest) == 0)
1698     {
1699         printf("dropmessages DROPPING RECV MESSAGE\n");
1700         return true;
1701     }
1702
1703
1704
1705     if (strCommand == "version")
1706     {
1707         // Can only do this once
1708         if (pfrom->nVersion != 0)
1709             return false;
1710
1711         int64 nTime;
1712         CAddress addrMe;
1713         vRecv >> pfrom->nVersion >> pfrom->nServices >> nTime >> addrMe;
1714         if (pfrom->nVersion == 0)
1715             return false;
1716
1717         pfrom->vSend.SetVersion(min(pfrom->nVersion, VERSION));
1718         pfrom->vRecv.SetVersion(min(pfrom->nVersion, VERSION));

```

```

1719
1720     pfrom->fClient = !(pfrom->nServices & NODE_NETWORK);
1721     if (pfrom->fClient)
1722     {
1723         pfrom->vSend.nType |= SER_BLOCKHEADERONLY;
1724         pfrom->vRecv.nType |= SER_BLOCKHEADERONLY;
1725     }
1726
1727     AddTimeData(pfrom->addr.ip, nTime);
1728
1729     // Ask the first connected node for block updates
1730     static bool fAskedForBlocks;
1731     if (!fAskedForBlocks && !pfrom->fClient)
1732     {
1733         fAskedForBlocks = true;
1734         pfrom->PushMessage("getblocks", CBlockLocator(pindexBest), uint256(0));
1735     }
1736
1737     printf("version addrMe = %s\n", addrMe.ToString().c_str());
1738 }
1739
1740
1741     else if (pfrom->nVersion == 0)
1742     {
1743         // Must have a version message before anything else
1744         return false;
1745     }
1746
1747
1748     else if (strCommand == "addr")
1749     {
1750         vector<CAddress> vAddr;
1751         vRecv >> vAddr;
1752
1753         // Store the new addresses
1754         CAddrDB addrdb;
1755         foreach(const CAddress& addr, vAddr)
1756         {
1757             if (fShutdown)
1758                 return true;
1759             if (AddAddress(addrdb, addr))
1760             {
1761                 // Put on lists to send to other nodes
1762                 pfrom->setAddrKnown.insert(addr);
1763                 CRITICAL_BLOCK(cs_vNodes)
1764                 foreach(CNode* pnode, vNodes)
1765                     if (!pnode->setAddrKnown.count(addr))
1766                         pnode->vAddrToSend.push_back(addr);
1767             }
1768         }
1769     }
1770
1771
1772     else if (strCommand == "inv")
1773     {
1774         vector<CInv> vInv;
1775         vRecv >> vInv;
1776
1777         CTxDB txdb("r");
1778         foreach(const CInv& inv, vInv)
1779         {
1780             if (fShutdown)
1781                 return true;
1782             pfrom->AddInventoryKnown(inv);
1783
1784             bool fAlreadyHave = AlreadyHave(txdb, inv);
1785             printf(" got inventory: %s %s\n", inv.ToString().c_str(), fAlreadyHave ? "have" : "new");
1786
1787             if (!fAlreadyHave)
1788                 pfrom->AskFor(inv);
1789             else if (inv.type == MSG_BLOCK && mapOrphanBlocks.count(inv.hash))

```

```

1790         pfrom->PushMessage("getblocks", CBlockLocator(pindexBest),
1791             GetOrphanRoot(mapOrphanBlocks[inv.hash]));
1792     }
1793 }
1794
1795 else if (strCommand == "getdata")
1796 {
1797     vector<CInv> vInv;
1798     vRecv >> vInv;
1799
1800     foreach(const CInv& inv, vInv)
1801     {
1802         if (fShutdown)
1803             return true;
1804         printf("received getdata for: %s\n", inv.ToString().c_str());
1805
1806         if (inv.type == MSG_BLOCK)
1807         {
1808             // Send block from disk
1809             map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(inv.hash);
1810             if (mi != mapBlockIndex.end())
1811             {
1812                 //// could optimize this to send header straight from blockindex for client
1813                 CBlock block;
1814                 block.ReadFromDisk((*mi).second, !pfrom->fClient);
1815                 pfrom->PushMessage("block", block);
1816             }
1817         }
1818         else if (inv.IsKnownType())
1819         {
1820             // Send stream from relay memory
1821             CRITICAL_BLOCK(cs_mapRelay)
1822             {
1823                 map<CInv, CDataStream*>::iterator mi = mapRelay.find(inv);
1824                 if (mi != mapRelay.end())
1825                     pfrom->PushMessage(inv.GetCommand(), (*mi).second);
1826             }
1827         }
1828     }
1829 }
1830
1831 else if (strCommand == "getblocks")
1832 {
1833     CBlockLocator locator;
1834     uint256 hashStop;
1835     vRecv >> locator >> hashStop;
1836
1837     // Find the first block the caller has in the main chain
1838     CBlockIndex* pindex = locator.GetBlockIndex();
1839
1840     // Send the rest of the chain
1841     if (pindex)
1842     {
1843         pindex = pindex->pnext;
1844         printf("getblocks %d to %s\n", (pindex ? pindex->nHeight : -1),
1845             hashStop.ToString().substr(0,14).c_str());
1846         for (; pindex; pindex = pindex->pnext)
1847         {
1848             if (pindex->GetBlockHash() == hashStop)
1849             {
1850                 printf(" getblocks stopping at %d %s\n", pindex->nHeight, pindex-
1851                     >GetBlockHash().ToString().substr(0,14).c_str());
1852                 break;
1853             }
1854         }
1855     }
1856
1857     // Bypass setInventoryKnown in case an inventory message got lost
1858     CRITICAL_BLOCK(pfrom->cs_inventory)
1859     {
1860         CInv inv(MSG_BLOCK, pindex->GetBlockHash());
1861         // returns true if wasn't already contained in the set
1862         if (pfrom->setInventoryKnown2.insert(inv).second)
    
```

```

1859 {
1860     pfrom->setInventoryKnown.erase(inv);
1861     pfrom->vInventoryToSend.push_back(inv);
1862 }
1863 }
1864 }
1865 }
1866
1867
1868 else if (strCommand == "tx")
1869 {
1870     vector<uint256> vWorkQueue;
1871     CDataStream vMsg(vRecv);
1872     CTransaction tx;
1873     vRecv >> tx;
1874
1875     CInv inv(MSG_TX, tx.GetHash());
1876     pfrom->AddInventoryKnown(inv);
1877
1878     bool fMissingInputs = false;
1879     if (tx.AcceptTransaction(true, &fMissingInputs))
1880     {
1881         AddToWalletIfMine(tx, NULL);
1882         RelayMessage(inv, vMsg);
1883         mapAlreadyAskedFor.erase(inv);
1884         vWorkQueue.push_back(inv.hash);
1885
1886         // Recursively process any orphan transactions that depended on this one
1887         for (int i = 0; i < vWorkQueue.size(); i++)
1888         {
1889             uint256 hashPrev = vWorkQueue[i];
1890             for (multimap<uint256, CDataStream*>::iterator mi = mapOrphanTransactionsByPrev.lower_bound(hashPrev);
1891                  mi != mapOrphanTransactionsByPrev.upper_bound(hashPrev);
1892                  ++mi)
1893             {
1894                 const CDataStream& vMsg = ((*mi).second);
1895                 CTransaction tx;
1896                 CDataStream(vMsg) >> tx;
1897                 CInv inv(MSG_TX, tx.GetHash());
1898
1899                 if (tx.AcceptTransaction(true))
1900                 {
1901                     printf("  accepted orphan tx %s\n", inv.hash.ToString().substr(0,6).c_str());
1902                     AddToWalletIfMine(tx, NULL);
1903                     RelayMessage(inv, vMsg);
1904                     mapAlreadyAskedFor.erase(inv);
1905                     vWorkQueue.push_back(inv.hash);
1906                 }
1907             }
1908         }
1909
1910         foreach(uint256 hash, vWorkQueue)
1911             EraseOrphanTx(hash);
1912     }
1913     else if (fMissingInputs)
1914     {
1915         printf("storing orphan tx %s\n", inv.hash.ToString().substr(0,6).c_str());
1916         AddOrphanTx(vMsg);
1917     }
1918 }
1919
1920
1921 else if (strCommand == "review")
1922 {
1923     CDataStream vMsg(vRecv);
1924     CReview review;
1925     vRecv >> review;
1926
1927     CInv inv(MSG_REVIEW, review.GetHash());
1928     pfrom->AddInventoryKnown(inv);
1929
1930     if (review.AcceptReview())

```

```

1931 {
1932 // Relay the original message as-is in case it's a higher version than we know how to parse
1933 RelayMessage(inv, vMsg);
1934     mapAlreadyAskedFor.erase(inv);
1935 }
1936 }
1937
1938
1939 else if (strCommand == "block")
1940 {
1941     auto_ptr<CBlock> pblock(new CBlock);
1942     vRecv >> *pblock;
1943
1944     /// debug print
1945     printf("received block:\n"); pblock->print();
1946
1947     CInv inv(MSG_BLOCK, pblock->GetHash());
1948     pfrom->AddInventoryKnown(inv);
1949
1950     if (ProcessBlock(pfrom, pblock.release()))
1951         mapAlreadyAskedFor.erase(inv);
1952 }
1953
1954
1955 else if (strCommand == "getaddr")
1956 {
1957     pfrom->vAddrToSend.clear();
1958     /// need to expand the time range if not enough found
1959     int64 nSince = GetAdjustedTime() - 60 * 60; // in the last hour
1960     CRITICAL_BLOCK(cs_mapAddresses)
1961     {
1962         foreach(const PAIRTYPE(vector<unsigned char>, CAddress)& item, mapAddresses)
1963         {
1964             if (fShutdown)
1965                 return true;
1966             const CAddress& addr = item.second;
1967             if (addr.nTime > nSince)
1968                 pfrom->vAddrToSend.push_back(addr);
1969         }
1970     }
1971 }
1972
1973
1974 else if (strCommand == "checkorder")
1975 {
1976     uint256 hashReply;
1977     CWalletTx order;
1978     vRecv >> hashReply >> order;
1979
1980     /// we have a chance to check the order here
1981
1982     // Keep giving the same key to the same ip until they use it
1983     if (!mapReuseKey.count(pfrom->addr.ip))
1984         mapReuseKey[pfrom->addr.ip] = GenerateNewKey();
1985
1986     // Send back approval of order and pubkey to use
1987     CScript scriptPubKey;
1988     scriptPubKey << mapReuseKey[pfrom->addr.ip] << OP_CHECKSIG;
1989     pfrom->PushMessage("reply", hashReply, (int)0, scriptPubKey);
1990 }
1991
1992
1993 else if (strCommand == "submitorder")
1994 {
1995     uint256 hashReply;
1996     CWalletTx wtxNew;
1997     vRecv >> hashReply >> wtxNew;
1998
1999     // Broadcast
2000     if (!wtxNew.AcceptWalletTransaction())
2001     {
2002         pfrom->PushMessage("reply", hashReply, (int)1);

```

```

2003 return error("submitorder AcceptWalletTransaction() failed, returning error 1");
2004 }
2005     wtxNew.fTimeReceivedIsTxTime = true;
2006 AddToWallet(wtxNew);
2007     wtxNew.RelayWalletTransaction();
2008     mapReuseKey.erase(pfrom->addr.ip);
2009
2010 // Send back confirmation
2011     pfrom->PushMessage("reply", hashReply, (int)0);
2012 }
2013
2014
2015 else if (strCommand == "reply")
2016 {
2017     uint256 hashReply;
2018     vRecv >> hashReply;
2019
2020     CRequestTracker tracker;
2021 CRITICAL_BLOCK(pfrom->cs_mapRequests)
2022 {
2023     map<uint256, CRequestTracker>::iterator mi = pfrom->mapRequests.find(hashReply);
2024 if (mi != pfrom->mapRequests.end())
2025 {
2026         tracker = (*mi).second;
2027         pfrom->mapRequests.erase(mi);
2028     }
2029 }
2030 if (!tracker.IsNull())
2031     tracker.fn(tracker.param1, vRecv);
2032 }
2033
2034
2035 else
2036 {
2037 // Ignore unknown commands for extensibility
2038 printf("ProcessMessage(%s) : Ignored unknown message\n", strCommand.c_str());
2039 }
2040
2041
2042 if (!vRecv.empty())
2043 printf("ProcessMessage(%s) : %d extra bytes\n", strCommand.c_str(), vRecv.size());
2044
2045 return true;
2046 }
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056 bool SendMessages(CNode* pto)
2057 {
2058     CheckForShutdown(2);
2059 CRITICAL_BLOCK(cs_main)
2060 {
2061 // Don't send anything until we get their version message
2062 if (pto->nVersion == 0)
2063     return true;
2064
2065
2066 //
2067 // Message: addr
2068 //
2069     vector<CAddress> vAddrToSend;
2070     vAddrToSend.reserve(pto->vAddrToSend.size());
2071     foreach(const CAddress& addr, pto->vAddrToSend)
2072 if (!pto->setAddrKnown.count(addr))
2073         vAddrToSend.push_back(addr);
2074     pto->vAddrToSend.clear();

```



```

2075 if (!vAddrToSend.empty())
2076     pto->PushMessage("addr", vAddrToSend);
2077
2078
2079 //
2080 // Message: inventory
2081 //
2082     vector<CInv> vInventoryToSend;
2083 CRITICAL_BLOCK(pto->cs_inventory)
2084 {
2085     vInventoryToSend.reserve(pto->vInventoryToSend.size());
2086     foreach(const CInv& inv, pto->vInventoryToSend)
2087     {
2088         // returns true if wasn't already contained in the set
2089         if (pto->setInventoryKnown.insert(inv).second)
2090             vInventoryToSend.push_back(inv);
2091     }
2092     pto->vInventoryToSend.clear();
2093     pto->setInventoryKnown2.clear();
2094 }
2095 if (!vInventoryToSend.empty())
2096     pto->PushMessage("inv", vInventoryToSend);
2097
2098 //
2099 // Message: getdata
2100 //
2101 //
2102     vector<CInv> vAskFor;
2103     int64 nNow = GetTime() * 1000000;
2104     CTxDB txdb("r");
2105     while (!pto->mapAskFor.empty() && (*pto->mapAskFor.begin()).first <= nNow)
2106     {
2107         const CInv& inv = (*pto->mapAskFor.begin()).second;
2108         printf("sending getdata: %s\n", inv.ToString().c_str());
2109         if (!AlreadyHave(txdb, inv))
2110             vAskFor.push_back(inv);
2111         pto->mapAskFor.erase(pto->mapAskFor.begin());
2112     }
2113     if (!vAskFor.empty())
2114         pto->PushMessage("getdata", vAskFor);
2115
2116 }
2117 return true;
2118 }
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133 //////////////////////////////////////
2134 //
2135 // BitcoinMiner
2136 //
2137
2138 int FormatHashBlocks(void* pbuffer, unsigned int len)
2139 {
2140     unsigned char* pdata = (unsigned char*)pbuffer;
2141     unsigned int blocks = 1 + ((len + 8) / 64);
2142     unsigned char* pend = pdata + 64 * blocks;
2143     memset(pdata + len, 0, 64 * blocks - len);
2144     pdata[len] = 0x80;
2145     unsigned int bits = len * 8;
2146     pend[-1] = (bits >> 0) & 0xff;

```

```

2147     pend[-2] = (bits >> 8) & 0xff;
2148     pend[-3] = (bits >> 16) & 0xff;
2149     pend[-4] = (bits >> 24) & 0xff;
2150     return blocks;
2151 }
2152
2153 using CryptoPP::ByteReverse;
2154 static int detectlittleendian = 1;
2155
2156 void BlockSHA256(const void* pin, unsigned int nBlocks, void* pout)
2157 {
2158     unsigned int* pinut = (unsigned int*)pin;
2159     unsigned int* pstate = (unsigned int*)pout;
2160
2161     CryptoPP::SHA256::InitState(pstate);
2162
2163     if (*(char*)&detectlittleendian != 0)
2164     {
2165         for (int n = 0; n < nBlocks; n++)
2166         {
2167             unsigned int pbuf[16];
2168             for (int i = 0; i < 16; i++)
2169                 pbuf[i] = ByteReverse(pinut[n * 16 + i]);
2170             CryptoPP::SHA256::Transform(pstate, pbuf);
2171         }
2172         for (int i = 0; i < 8; i++)
2173             pstate[i] = ByteReverse(pstate[i]);
2174     }
2175     else
2176     {
2177         for (int n = 0; n < nBlocks; n++)
2178             CryptoPP::SHA256::Transform(pstate, pinut + n * 16);
2179     }
2180 }
2181
2182
2183 bool BitcoinMiner()

```

`BitcoinMiner()` - the mining function. Satoshi envisioned 'one-CPU-one-vote' democracy. In 2009, anyone's laptop could mine blocks.

```

2184 {
2185     printf("BitcoinMiner started\n");
2186     SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_LOWEST);

```

`THREAD_PRIORITY_LOWEST` - mining runs in background. Satoshi wanted mining to not interfere with normal computer use.

```

2187
2188     CKey key;
2189     key.MakeNewKey();
2190     CBigNum bnExtraNonce = 0;
2191     while (fGenerateBitcoins)
2192     {
2193         Sleep(50);
2194         CheckForShutdown(3);
2195         while (vNodes.empty())
2196         {
2197             Sleep(1000);
2198             CheckForShutdown(3);
2199         }
2200
2201         unsigned int nTransactionsUpdatedLast = nTransactionsUpdated;
2202         CBlockIndex* pindexPrev = pindexBest;
2203         unsigned int nBits = GetNextWorkRequired(pindexPrev);

```

`GetNextWorkRequired()` - fetches current difficulty target. Miner must find hash below this. Higher difficulty = smaller target = harder.

```

2204
2205
2206 //
2207 // Create coinbase tx
2208 //
2209     CTransaction txNew;
2210     txNew.vin.resize(1);
2211     txNew.vin[0].prevout.SetNull();

```

`prevout.SetNull()` —the signature of money creation. Normal transactions must reference a previous output to spend; coinbase transactions reference nothing because they create coins from thin air.

`SetNull()` sets `hash=0` and `index=-1`, a combination that can never occur naturally (no real transaction has hash 0). This null reference is how nodes identify coinbase transactions. It's the only way new bitcoin enters circulation—every satoshi in existence traces back through a chain of transactions to some block's coinbase with this null `prevout`.

```

2212     txNew.vin[0].scriptSig << nBits << ++bnExtraNonce;

```

`bnExtraNonce` - incremented each block attempt. Combined with `nNonce`, gives 2^{96} possible hashes per second timestamp.

```

2213     txNew.vout.resize(1);
2214     txNew.vout[0].scriptPubKey << key.GetPubKey() << OP_CHECKSIG;
2215
2216
2217 //
2218 // Create new block
2219 //
2220     auto_ptr<CBlock> pblock(new CBlock());
2221     if (!pblock.get())
2222         return false;
2223
2224 // Add our coinbase tx as first transaction
2225     pblock->vtx.push_back(txNew);
2226
2227 // Collect the latest transactions into the block
2228     int64 nFees = 0;
2229     CRITICAL_BLOCK(cs_main)
2230     CRITICAL_BLOCK(cs_mapTransactions)
2231     {
2232         CTxDB txdb("r");
2233         map<uint256, CTxIndex> mapTestPool;
2234         vector<char> vfAlreadyAdded(mapTransactions.size());
2235         bool fFoundSomething = true;
2236         unsigned int nBlockSize = 0;
2237         while (fFoundSomething && nBlockSize < MAX_SIZE/2)
2238         {
2239             fFoundSomething = false;
2240             unsigned int n = 0;
2241             for (map<uint256, CTransaction>::iterator mi = mapTransactions.begin(); mi != mapTransactions.end();
                ++mi, ++n)
2242             {
2243                 if (vfAlreadyAdded[n])
2244                     continue;
2245                 CTransaction& tx = (*mi).second;
2246                 if (tx.IsCoinBase() || !tx.IsFinal())
2247                     continue;
2248

```

```

2249 // Transaction fee requirements, mainly only needed for flood control
2250 // Under 10K (about 80 inputs) is free for first 100 transactions
2251 // Base rate is 0.01 per KB
2252 int64 nMinFee = tx.GetMinFee(pblock->vtx.size() < 100);

```

Satoshi's fee discount for early transactions. The boolean `pblock->vtx.size() < 100` grants a discount to the first 100 transactions in a block: if under 10KB, they're free. After 100 transactions, everyone pays 0.01 BTC/KB. This was flood control—allow normal usage for free, but charge when blocks fill up. Satoshi wrote: 'Free transactions are nice...until we get slashdotted.' The threshold of 100 transactions and 10KB were educated guesses; modern Bitcoin uses more sophisticated fee markets where users bid for block space.

```

2253
2254 map<uint256, CTxIndex> mapTestPoolTmp(mapTestPool);
2255 if (!tx.ConnectInputs(txdb, mapTestPoolTmp, CDiskTxPos(1,1,1), 0, nFees, false, true, nMinFee))
2256 continue;
2257 swap(mapTestPool, mapTestPoolTmp);
2258
2259 pblock->vtx.push_back(tx);
2260 nBlockSize += ::GetSerializeSize(tx, SER_NETWORK);
2261 vfAlreadyAdded[n] = true;
2262 fFoundSomething = true;
2263 }
2264 }
2265 }
2266 pblock->nBits = nBits;
2267 pblock->vtx[0].vout[0].nValue = pblock->GetBlockValue(nFees);

```

`GetBlockValue(nFees)` - calculates reward. Block subsidy plus all transaction fees. Fees incentivize miners after subsidy halves to zero.

```

2268 printf("\n\nRunning BitcoinMiner with %d transactions in block\n", pblock->vtx.size());
2269
2270
2271 //
2272 // Prebuild hash buffer
2273 //
2274 struct unnamed1
2275 {
2276 struct unnamed2
2277 {
2278 int nVersion;
2279 uint256 hashPrevBlock;
2280 uint256 hashMerkleRoot;
2281 unsigned int nTime;
2282 unsigned int nBits;
2283 unsigned int nNonce;
2284 }
2285 block;
2286 unsigned char pchPadding0[64];
2287 uint256 hash1;
2288 unsigned char pchPadding1[64];
2289 }
2290 tmp;
2291
2292 tmp.block.nVersion = pblock->nVersion;
2293 tmp.block.hashPrevBlock = pblock->hashPrevBlock = (pindexPrev ? pindexPrev->GetBlockHash() :
0);
2294 tmp.block.hashMerkleRoot = pblock->hashMerkleRoot = pblock->BuildMerkleTree();
2295 tmp.block.nTime = pblock->nTime = max((pindexPrev ? pindexPrev->GetMedianTimePast()+1 : 0), GetAdjustedTime());
2296 tmp.block.nBits = pblock->nBits = nBits;
2297 tmp.block.nNonce = pblock->nNonce = 1;
2298
2299 unsigned int nBlocks0 = FormatHashBlocks(&tmp.block, sizeof(tmp.block));

```

```

2300 unsigned int nBlocks1 = FormatHashBlocks(&tmp.hash1, sizeof(tmp.hash1));
2301
2302
2303 //
2304 // Search
2305 //
2306 unsigned int nStart = GetTime();
2307 uint256 hashTarget = CBigNum().SetCompact(pblock->nBits).getuint256();

```

hashTarget - the difficulty threshold. Hash must be numerically less than this 256-bit number. Lower target = harder puzzle.

```

2308     uint256 hash;
2309     loop

```

The mining loop. Increment nNonce, double-SHA256 the header, compare to target. Repeat billions of times until valid hash found.

```

2310 {
2311 BlockSHA256(&tmp.block, nBlocks0, &tmp.hash1);

```

BlockSHA256() - optimized SHA-256 for mining. Called millions of times per second. Performance here directly affects mining revenue.

```

2312 BlockSHA256(&tmp.hash1, nBlocks1, &hash);
2313
2314
2315 if (hash <= hashTarget)

```

hash <= hashTarget - FOUND IT! This comparison is why mining works: finding a hash below target is rare and proves work was done.

```

2316 {
2317     pblock->nNonce = tmp.block.nNonce;
2318 assert(hash == pblock->GetHash());
2319
2320 /// debug print
2321 printf("BitcoinMiner:\n");
2322 printf("proof-of-work found \n hash: %s \ntarget: %s\n", hash.GetHex().c_str(),
    hashTarget.GetHex().c_str());

```

'proof-of-work found' - victory message. Satoshi: 'The proof-of-work also solves the problem of determining representation in majority decision making.'

```

2323     pblock->print();
2324
2325 SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_NORMAL);
2326 CRITICAL_BLOCK(cs_main)
2327 {
2328 // Save key
2329 if (!AddKey(key))
2330 return false;
2331     key.MakeNewKey();
2332
2333 // Process this block the same as if we had received it from another node
2334 if (!ProcessBlock(NULL, pblock.release()))
2335 printf("ERROR in BitcoinMiner, ProcessBlock, block not accepted\n");
2336 }

```

```

2337 SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_LOWEST);
2338
2339 Sleep(500);
2340 break;
2341 }
2342
2343 // Update nTime every few seconds
2344 if ((++tmp.block.nNonce & 0x3ffff) == 0)
2345 {
2346     CheckForShutdown(3);
2347     if (tmp.block.nNonce == 0)
2348         break;
2349     if (pindexPrev != pindexBest)
2350         break;
2351     if (nTransactionsUpdated != nTransactionsUpdatedLast && GetTime() - nStart > 60)
2352         break;
2353     if (!fGenerateBitcoins)
2354         break;
2355     tmp.block.nTime = pblock->nTime = max(pindexPrev->GetMedianTimePast()+1,
        GetAdjustedTime());
2356 }
2357 }
2358 }
2359
2360 return true;
2361 }
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380 ///////////////////////////////////////////////////////////////////
2381 //
2382 // Actions
2383 //
2384
2385
2386 int64 GetBalance()
2387 {
2388     int64 nStart, nEnd;
2389     QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
2390
2391     int64 nTotal = 0;
2392     CRITICAL_BLOCK(cs_mapWallet)
2393     {
2394         for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
2395         {
2396             CWalletTx* pcoin = &(*it).second;
2397             if (!pcoin->IsFinal() || pcoin->fSpent)
2398                 continue;
2399             nTotal += pcoin->GetCredit();
2400         }
2401     }
2402
2403     QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
2404     ///printf(" GetBalance() time = %I64d\n", nEnd - nStart);
2405     return nTotal;
2406 }
2407

```

```

2408
2409
2410 bool SelectCoins(int64 nTargetValue, set<CWalletTx*>& setCoinsRet)
2411 {
2412     setCoinsRet.clear();
2413
2414     // List of values less than target
2415     int64 nLowestLarger = _I64_MAX;
2416     CWalletTx* pcoinLowestLarger = NULL;
2417     vector<pair<int64, CWalletTx*> > vValue;
2418     int64 nTotalLower = 0;
2419
2420     CRITICAL_BLOCK(cs_mapWallet)
2421     {
2422         for (map<uint256, CWalletTx*>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
2423         {
2424             CWalletTx* pcoin = &(*it).second;
2425             if (!pcoin->IsFinal() || pcoin->fSpent)
2426                 continue;
2427             int64 n = pcoin->GetCredit();
2428             if (n <= 0)
2429                 continue;
2430             if (n < nTargetValue)
2431             {
2432                 vValue.push_back(make_pair(n, pcoin));
2433                 nTotalLower += n;
2434             }
2435             else if (n == nTargetValue)
2436             {
2437                 setCoinsRet.insert(pcoin);
2438                 return true;
2439             }
2440             else if (n < nLowestLarger)
2441             {
2442                 nLowestLarger = n;
2443                 pcoinLowestLarger = pcoin;
2444             }
2445         }
2446     }
2447
2448     if (nTotalLower < nTargetValue)
2449     {
2450         if (pcoinLowestLarger == NULL)
2451             return false;
2452         setCoinsRet.insert(pcoinLowestLarger);
2453         return true;
2454     }
2455
2456     // Solve subset sum by stochastic approximation
2457     sort(vValue.rbegin(), vValue.rend());
2458     vector<char> vfIncluded;
2459     vector<char> vfBest(vValue.size(), true);
2460     int64 nBest = nTotalLower;
2461
2462     for (int nRep = 0; nRep < 1000 && nBest != nTargetValue; nRep++)
2463     {
2464         vfIncluded.assign(vValue.size(), false);
2465         int64 nTotal = 0;
2466         bool fReachedTarget = false;
2467         for (int nPass = 0; nPass < 2 && !fReachedTarget; nPass++)
2468         {
2469             for (int i = 0; i < vValue.size(); i++)
2470             {
2471                 if (nPass == 0 ? rand() % 2 : !vfIncluded[i])
2472                 {
2473                     nTotal += vValue[i].first;
2474                     vfIncluded[i] = true;
2475                 }
2476                 if (nTotal >= nTargetValue)
2477                 {
2478                     fReachedTarget = true;
2479                     if (nTotal < nBest)
2480                     {
2481                         nBest = nTotal;
2482                         vfBest = vfIncluded;
2483                     }
2484                 }
2485             }
2486         }
2487     }
2488     setCoinsRet.insert(pcoinLowestLarger);
2489     return true;
2490 }

```

```

2480                 nBest = nTotal;
2481                 vfBest = vfIncluded;
2482             }
2483             nTotal -= vValue[i].first;
2484             vfIncluded[i] = false;
2485         }
2486     }
2487 }
2488 }
2489 }
2490
2491 // If the next larger is still closer, return it
2492 if (pcoinLowestLarger && nLowestLarger - nTargetValue <= nBest - nTargetValue)
2493     setCoinsRet.insert(pcoinLowestLarger);
2494 else
2495 {
2496     for (int i = 0; i < vValue.size(); i++)
2497         if (vfBest[i])
2498             setCoinsRet.insert(vValue[i].second);
2499 }
2500 /// debug print
2501 printf("SelectCoins() best subset: ");
2502 for (int i = 0; i < vValue.size(); i++)
2503     if (vfBest[i])
2504         printf("%s ", FormatMoney(vValue[i].first).c_str());
2505 printf("total %s\n", FormatMoney(nBest).c_str());
2506 }
2507
2508 return true;
2509 }
2510
2511
2512
2513
2514 bool CreateTransaction(CScript scriptPubKey, int64 nValue, CWalletTx& wtxNew, int64& nFeeRequiredRet)
2515 {
2516     nFeeRequiredRet = 0;
2517     CRITICAL_BLOCK(cs_main)
2518     {
2519         // txdb must be opened before the mapWallet lock
2520         CTxDB txdb("r");
2521         CRITICAL_BLOCK(cs_mapWallet)
2522         {
2523             int64 nFee = nTransactionFee;
2524             loop
2525             {
2526                 wtxNew.vin.clear();
2527                 wtxNew.vout.clear();
2528                 if (nValue < 0)
2529                     return false;
2530                 int64 nValueOut = nValue;
2531                 nValue += nFee;
2532             }
2533             // Choose coins to use
2534             set<CWalletTx*> setCoins;
2535             if (!SelectCoins(nValue, setCoins))
2536                 return false;
2537             int64 nValueIn = 0;
2538             foreach(CWalletTx* pcoin, setCoins)
2539                 nValueIn += pcoin->GetCredit();
2540
2541             // Fill vout[0] to the payee
2542             wtxNew.vout.push_back(CTxOut(nValueOut, scriptPubKey));
2543
2544             // Fill vout[1] back to self with any change
2545             if (nValueIn > nValue)
2546             {
2547                 // Use the same key as one of the coins
2548                 vector<unsigned char> vchPubKey;
2549                 CTransaction& txFirst = *(&setCoins.begin());
2550                 foreach(const CTxOut& txout, txFirst.vout)
2551                     if (txout.IsMine())

```



```

2552 if (ExtractPubKey(txout.scriptPubKey, true, vchPubKey))
2553 break;
2554 if (vchPubKey.empty())
2555 return false;
2556
2557 // Fill vout[1] to ourself
2558 CScript scriptPubKey;
2559 scriptPubKey << vchPubKey << OP_CHECKSIG;
2560 wtxNew.vout.push_back(CTxOut(nValueIn - nValue, scriptPubKey));
2561 }
2562
2563 // Fill vin
2564 foreach(CWalletTx* pcoin, setCoins)
2565 for (int nOut = 0; nOut < pcoin->vout.size(); nOut++)
2566 if (pcoin->vout[nOut].IsMine())
2567     wtxNew.vin.push_back(CTxIn(pcoin->GetHash(), nOut));
2568
2569 // Sign
2570 int nIn = 0;
2571 foreach(CWalletTx* pcoin, setCoins)
2572 for (int nOut = 0; nOut < pcoin->vout.size(); nOut++)
2573 if (pcoin->vout[nOut].IsMine())
2574     SignSignature(*pcoin, wtxNew, nIn++);
2575
2576 // Check that enough fee is included
2577 if (nFee < wtxNew.GetMinFee(true))
2578 {
2579     nFee = nFeeRequiredRet = wtxNew.GetMinFee(true);
2580 continue;
2581 }
2582
2583 // Fill vtxPrev by copying from previous transactions vtxPrev
2584 wtxNew.AddSupportingTransactions(txdb);
2585 wtxNew.fTimeReceivedIsTxTime = true;
2586
2587 break;
2588 }
2589 }
2590 }
2591 return true;
2592 }
2593
2594 // Call after CreateTransaction unless you want to abort
2595 bool CommitTransactionSpent(const CWalletTx& wtxNew)
2596 {
2597     CRITICAL_BLOCK(cs_main)
2598     CRITICAL_BLOCK(cs_mapWallet)
2599     {
2600         /// todo: make this transactional, never want to add a transaction
2601         /// without marking spent transactions
2602
2603         // Add tx to wallet, because if it has change it's also ours,
2604         // otherwise just for transaction history.
2605         AddToWallet(wtxNew);
2606
2607         // Mark old coins as spent
2608         set<CWalletTx*> setCoins;
2609         foreach(const CTxIn& txin, wtxNew.vin)
2610             setCoins.insert(&mapWallet[txin.prevout.hash]);
2611         foreach(CWalletTx* pcoin, setCoins)
2612         {
2613             pcoin->fSpent = true;
2614             pcoin->WriteToDisk();
2615             vWalletUpdated.push_back(make_pair(pcoin->GetHash(), false));
2616         }
2617     }
2618     MainFrameRepaint();
2619     return true;
2620 }
2621
2622
2623

```

```

2624
2625 bool SendMoney(CScript scriptPubKey, int64 nValue, CWalletTx& wtxNew)
2626 {
2627     CRITICAL_BLOCK(cs_main)
2628     {
2629         int64 nFeeRequired;
2630         if (!CreateTransaction(scriptPubKey, nValue, wtxNew, nFeeRequired))
2631         {
2632             string strError;
2633             if (nValue + nFeeRequired > GetBalance())
2634                 strError = sprintf("Error: This is an oversized transaction that requires a
transaction fee of %s ", FormatMoney(nFeeRequired).c_str());
2635             else
2636                 strError = "Error: Transaction creation failed ";
2637             wxMessageBox(strError, "Sending...");
2638             return error("SendMoney() : %s\n", strError.c_str());
2639         }
2640         if (!CommitTransactionSpent(wtxNew))
2641         {
2642             wxMessageBox("Error finalizing transaction", "Sending...");
2643             return error("SendMoney() : Error finalizing transaction");
2644         }
2645         printf("SendMoney: %s\n", wtxNew.GetHash().ToString().substr(0,6).c_str());
2646         // Broadcast
2647         if (!wtxNew.AcceptTransaction())
2648         {
2649             // This must not fail. The transaction has already been signed and recorded.
2650             throw runtime_error("SendMoney() : wtxNew.AcceptTransaction() failed\n");
2651             wxMessageBox("Error: Transaction not valid", "Sending...");
2652             return error("SendMoney() : Error: Transaction not valid");
2653         }
2654         wtxNew.RelayWalletTransaction();
2655     }
2656     MainFrameRepaint();
2657     return true;
2658 }
2659
2660
2661

```

SUMMARY

This file embodies Satoshi's breakthrough: using proof-of-work not just for rate-limiting (like Adam Back's Hashcash) but as the foundation of distributed consensus. Previous attempts at digital cash—DigiCash, b-money, bit gold—failed because they required trusted parties or couldn't prevent double-spending without a central authority. Satoshi solved both problems simultaneously.

The elegance lies in the alignment of incentives. Miners profit by following rules and extending the longest chain. Attempting to rewrite history requires outspending all honest miners—an attack that becomes exponentially harder with each confirmation. Satoshi calculated that an attacker with 10% of network hashrate has only a 0.0006% chance of catching up after 6 blocks (see the whitepaper's "Calculations" section).

This code has been stress-tested by the most adversarial conditions imaginable: nation-state attackers, billions of dollars at stake, and constant scrutiny from security researchers. That it continues to function—producing a block every 10 minutes without fail since January 3, 2009—is testament to Satoshi's engineering.

Satoshi's final words before disappearing (April 2011): "I've moved on to other things. It's in good hands with Gavin and everyone." This 2,300-line file launched a monetary revolution—a trustless system for transferring value that has operated continuously since January 3, 2009, securing hundreds of billions in value.

main.h

1318 lines CPP

ANATOMY OF A COIN

This header defines the fundamental data structures of Bitcoin: transactions, blocks, and the indexes that make the blockchain queryable. If `main.cpp` is the brain, `main.h` is the anatomy—the skeleton upon which everything hangs. These structures aren't arbitrary; each reflects Satoshi's solutions to specific distributed systems problems that stumped cryptographers for decades.

Satoshi described the core structure in Section 2 of the whitepaper: "We define an electronic coin as a chain of digital signatures. Each owner transfers the coin to the next by digitally signing a hash of the previous transaction." The classes here—`CTransaction`, `CTxIn`, `CTxOut`—implement that vision in C++, turning abstract cryptographic concepts into working code.

The architecture here introduced what's now called the "UTXO model" (Unspent Transaction Output). Unlike bank accounts that track balances, Bitcoin tracks individual coins—each output is like a physical bill that must be spent entirely. This model enables parallel validation, simple proofs, and clear ownership chains. Ethereum later chose a different path (account model), but most cryptocurrencies follow Satoshi's UTXO design.

Every constant in this file encodes economic policy: the 21 million coin limit, the 100-block maturity for mined coins, the fee structure. Unlike fiat currencies where these parameters change at the whim of central banks, Bitcoin's parameters are enforced by code that every participant runs. Change requires global consensus—a feature, not a bug.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 class COutPoint;
6 class CInPoint;
7 class CDiskTxPos;
8 class CCoinBase;
9 class CTxIn;
10 class CTxOut;
11 class CTransaction;
12 class CBlock;
13 class CBlockIndex;
14 class CWalletTx;
15 class CKeyItem;
16
17 static const unsigned int MAX_SIZE = 0x02000000;

```

MAX_SIZE = 32 MB (0×02000000 bytes). This general-purpose limit applies to blocks, transactions, and scripts. Notably, this is NOT the famous '1 MB block size limit'—that was added by Satoshi in July 2010 (commit 172f006), over a year after v0.1. The 1 MB limit was likely a temporary anti-spam measure that became permanent, eventually causing the 'block size wars' and the Bitcoin Cash fork in 2017. In v0.1, blocks could theoretically reach 32 MB, though mining code limited them to 16 MB (MAX_SIZE/2).

```

18 static const int64 COIN = 100000000;

```

COIN = 100,000,000 satoshis. Satoshi chose 8 decimal places for maximum divisibility—more than any fiat currency. Why 100 million? It allows a single bitcoin to be divided into pieces small enough for micropayments even if BTC reaches millions of dollars per coin. The unit was later named 'satoshi' by the community; Satoshi himself never used the term. Internally, all calculations use satoshis (integers) to avoid floating-point precision errors.

```

19 static const int64 CENT = 1000000;

```

CENT = 1,000,000 satoshis (0.01 BTC). This naming reveals Satoshi's framing: Bitcoin as currency, not commodity. He consistently used monetary language—'coins,' 'cents,' 'transactions'—to shape how people thought about the system. The cent unit appeared in fee calculations and the GUI, making Bitcoin feel like familiar money.

```

20 static const int COINBASE_MATURITY = 100;

```

COINBASE_MATURITY = 100 blocks (~17 hours). Freshly mined coins cannot be spent for 100 blocks. Why? If a miner's block gets orphaned in a chain reorganization, the mining reward disappears—it never existed on the winning chain. Without this delay, recipients of those coins would lose money through no fault of their own. 100 blocks provides statistical certainty that reorganizations won't affect mined coins.

```

21
22 static const CBigNum bnProofOfWorkLimit(~uint256(0) >> 32);

```

bnProofOfWorkLimit—the easiest possible difficulty, used for the genesis block. Expressed as `~uint256(0) >> 32`, meaning valid hashes must have at least 32 leading zero bits. This was trivially easy by modern standards: Satoshi's personal computer could find valid blocks in minutes. As more miners joined, difficulty automatically increased. Today's difficulty requires ~80+ leading zeros—approximately 10 trillion times harder than this initial value.

```

23
24
25
26
27
28
29 extern CCriticalSection cs_main;
30 extern map<uint256, CBlockIndex*> mapBlockIndex;
31 extern const uint256 hashGenesisBlock;
32 extern CBlockIndex* pindexGenesisBlock;
33 extern int nBestHeight;

```

nBestHeight - current chain height. This simple integer represents humanity's consensus on Bitcoin's state. Elegant simplicity.

```

34 extern uint256 hashBestChain;
35 extern CBlockIndex* pindexBest;

```

pindexBest - pointer to chain tip. All new blocks must extend from here. The 'longest chain' rule implemented as a single pointer.

```

36 extern unsigned int nTransactionsUpdated;
37 extern string strSetDataDir;
38 extern int nDropMessagesTest;
39
40 // Settings
41 extern int fGenerateBitcoins;

```

fGenerateBitcoins - mining flag. Satoshi's 'one-CPU-one-vote' vision. In 2009, turning this on meant your laptop joined the mining democracy.

```

42 extern int64 nTransactionFee;
43 extern CAddress addrIncoming;
44
45
46
47
48
49
50
51 string GetAppDir();
52 FILE* OpenBlockFile(unsigned int nFile, unsigned int nBlockPos, const char* pszMode="rb");
53 FILE* AppendBlockFile(unsigned int& nFileRet);
54 bool AddKey(const CKey& key);
55 vector<unsigned char> GenerateNewKey();
56 bool AddToWallet(const CWalletTx& wtxIn);
57 void ReacceptWalletTransactions();
58 void RelayWalletTransactions();
59 bool LoadBlockIndex(bool fAllowNew=true);
60 void PrintBlockTree();
61 bool BitcoinMiner();
62 bool ProcessMessages(CNode* pfrom);
63 bool ProcessMessage(CNode* pfrom, string strCommand, CDataStream& vRecv);

```

```

64 bool SendMessages(CNode* pto);
65 int64 GetBalance();
66 bool CreateTransaction(CScript scriptPubKey, int64 nValue, CWalletTx& txNew, int64& nFeeRequiredRet);
67 bool CommitTransactionSpent(const CWalletTx& wtxNew);
68 bool SendMoney(CScript scriptPubKey, int64 nValue, CWalletTx& wtxNew);
69
70
71
72
73
74
75
76
77
78
79
80 class CDiskTxPos
81 {
82 public:
83     unsigned int nFile;
84     unsigned int nBlockPos;
85     unsigned int nTxPos;
86
87     CDiskTxPos()
88     {
89         SetNull();
90     }
91
92     CDiskTxPos(unsigned int nFileIn, unsigned int nBlockPosIn, unsigned int nTxPosIn)
93     {
94         nFile = nFileIn;
95         nBlockPos = nBlockPosIn;
96         nTxPos = nTxPosIn;
97     }
98
99     IMPLEMENT_SERIALIZE( READWRITE(FLATDATA(*this)); )
100 void SetNull() { nFile = -1; nBlockPos = 0; nTxPos = 0; }
101 bool IsNull() const { return (nFile == -1); }
102
103 friend bool operator==(const CDiskTxPos& a, const CDiskTxPos& b)
104 {
105     return (a.nFile == b.nFile &&
106             a.nBlockPos == b.nBlockPos &&
107             a.nTxPos == b.nTxPos);
108 }
109
110 friend bool operator!=(const CDiskTxPos& a, const CDiskTxPos& b)
111 {
112     return !(a == b);
113 }
114
115     string ToString() const
116     {
117         if (IsNull())
118             return sprintf("null");
119         else
120             return sprintf("(nFile=%d, nBlockPos=%d, nTxPos=%d)", nFile, nBlockPos, nTxPos);
121     }
122
123 void print() const
124 {
125     printf("%s", ToString().c_str());
126 }
127 };
128
129
130
131
132 class CInPoint
133 {
134 public:
135     CTransaction* ptx;

```



```

136 unsigned int n;
137
138 CInPoint() { SetNull(); }
139 CInPoint(CTransaction* ptxIn, unsigned int nIn) { ptx = ptxIn; n = nIn; }
140 void SetNull() { ptx = NULL; n = -1; }
141 bool IsNull() const { return (ptx == NULL && n == -1); }
142 };
143
144
145
146
147 class COutPoint

```

COutPoint—a pointer to a specific coin in the UTXO set. Just two fields: a transaction hash and an output index. Together they uniquely identify any output ever created. This simple structure is the foundation of Bitcoin's ownership model: you don't own 'bitcoin in an account,' you own the right to spend specific outputs. When you check your balance, your wallet scans the blockchain for COutPoints that your keys can spend.

```

148 {
149 public:
150     uint256 hash;
151     unsigned int n;
152
153     COutPoint() { SetNull(); }
154     COutPoint(uint256 hashIn, unsigned int nIn) { hash = hashIn; n = nIn; }
155     IMPLEMENT_SERIALIZE( READWRITE(FLATDATA(*this)); )
156     void SetNull() { hash = 0; n = -1; }
157     bool IsNull() const { return (hash == 0 && n == -1); }
158
159     friend bool operator<(const COutPoint& a, const COutPoint& b)
160     {
161         return (a.hash < b.hash || (a.hash == b.hash && a.n < b.n));
162     }
163
164     friend bool operator==(const COutPoint& a, const COutPoint& b)
165     {
166         return (a.hash == b.hash && a.n == b.n);
167     }
168
169     friend bool operator!=(const COutPoint& a, const COutPoint& b)
170     {
171         return !(a == b);
172     }
173
174     string ToString() const
175     {
176         return sprintf("COutPoint(%s, %d)", hash.ToString().substr(0,6).c_str(), n);
177     }
178
179     void print() const
180     {
181         printf("%s\n", ToString().c_str());
182     }
183 };
184
185
186
187
188 //
189 // An input of a transaction. It contains the location of the previous
190 // transaction's output that it claims and a signature that matches the
191 // output's public key.
192 //
193 class CTxIn

```

CTxIn—how coins are consumed. Each input contains: (1) a COutPoint referencing which output to spend, (2) a scriptSig providing proof of ownership (typically a signature), and (3) nSequence for transaction replacement (mostly unused in v0.1). When a transaction is validated, each CTxIn must reference an existing, unspent output, and the scriptSig must satisfy that output's spending conditions. This is the 'chain of digital signatures' from the whitepaper.

```

194 {
195 public:
196     COutPoint prevout;
197     CScript scriptSig;
198     unsigned int nSequence;
199
200 CTxIn()
201 {
202     nSequence = UINT_MAX;
203 }
204
205 explicit CTxIn(COutPoint prevoutIn, CScript scriptSigIn=CScript(), unsigned int nSequenceIn=UINT_MAX)
206 {
207     prevout = prevoutIn;
208     scriptSig = scriptSigIn;
209     nSequence = nSequenceIn;
210 }
211
212 CTxIn(uint256 hashPrevTx, unsigned int nOut, CScript scriptSigIn=CScript(), unsigned int
nSequenceIn=UINT_MAX)
213 {
214     prevout = COutPoint(hashPrevTx, nOut);
215     scriptSig = scriptSigIn;
216     nSequence = nSequenceIn;
217 }
218
219 IMPLEMENT_SERIALIZE
220 (
221 READWRITE(prevout);
222 READWRITE(scriptSig);
223 READWRITE(nSequence);
224 )
225
226 bool IsFinal() const
227 {
228     return (nSequence == UINT_MAX);
229 }
230
231 friend bool operator==(const CTxIn& a, const CTxIn& b)
232 {
233     return (a.prevout == b.prevout &&
234             a.scriptSig == b.scriptSig &&
235             a.nSequence == b.nSequence);
236 }
237
238 friend bool operator!=(const CTxIn& a, const CTxIn& b)
239 {
240     return !(a == b);
241 }
242
243 string ToString() const
244 {
245     string str;
246     str += strprintf("CTxIn(");
247     str += prevout.ToString();
248     if (prevout.IsNull())
249         str += strprintf(", coinbase %s", HexStr(scriptSig.begin(), scriptSig.end(),
false).c_str());
250     else
251         str += strprintf(", scriptSig=%s", scriptSig.ToString().substr(0,24).c_str());
252     if (nSequence != UINT_MAX)

```

```

253         str += sprintf(" nSequence=%u", nSequence);
254         str += " ";
255     return str;
256 }
257
258 void print() const
259 {
260     printf("%s\n", ToString().c_str());
261 }
262
263 bool IsMine() const;
264 int64 GetDebit() const;
265 };
266
267
268
269
270 //
271 // An output of a transaction. It contains the public key that the next input
272 // must be able to sign with to claim it.
273 //
274 class CTxOut

```

CTxOut—how coins are created. Contains just two fields: nValue (amount in satoshis) and scriptPubKey (the spending conditions). The scriptPubKey is a mini program that specifies who can spend this output—typically 'whoever can provide a signature matching this public key'. Once created, a CTxOut exists forever in the blockchain history, but can only be spent once. The sum of all unspent CTxOuts is the UTXO set—the complete state of who owns what.

```

275 {
276     public:
277         int64 nValue;
278         CScript scriptPubKey;
279
280     public:
281     CTxOut()
282     {
283         SetNull();
284     }
285
286     CTxOut(int64 nValueIn, CScript scriptPubKeyIn)
287     {
288         nValue = nValueIn;
289         scriptPubKey = scriptPubKeyIn;
290     }
291
292     IMPLEMENT_SERIALIZE
293     (
294     READWRITE(nValue);
295     READWRITE(scriptPubKey);
296     )
297
298     void SetNull()
299     {
300         nValue = -1;
301         scriptPubKey.clear();
302     }
303
304     bool IsNull()
305     {
306         return (nValue == -1);
307     }
308
309     uint256 GetHash() const
310     {
311         return SerializeHash(*this);
312     }
313

```

```

314 bool IsMine() const
315 {
316     return ::IsMine(scriptPubKey);
317 }
318
319     int64 GetCredit() const
320 {
321     if (IsMine())
322         return nValue;
323     return 0;
324 }
325
326 friend bool operator==(const CTxOut& a, const CTxOut& b)
327 {
328     return (a.nValue == b.nValue &&
329             a.scriptPubKey == b.scriptPubKey);
330 }
331
332 friend bool operator!=(const CTxOut& a, const CTxOut& b)
333 {
334     return !(a == b);
335 }
336
337     string ToString() const
338 {
339     if (scriptPubKey.size() < 6)
340         return "CTxOut(error)";
341     return sprintf("CTxOut(nValue=%I64d.%08I64d, scriptPubKey=%s)", nValue / COIN, nValue % COIN,
342                   scriptPubKey.ToString().substr(0,24).c_str());
343 }
344
345 void print() const
346 {
347     printf("%s\n", ToString().c_str());
348 };
349
350
351
352
353 //
354 // The basic transaction that is broadcasted on the network and contained in
355 // blocks. A transaction can contain multiple inputs and outputs.
356 //
357 class CTransaction

```

CTransaction—the fundamental unit of value transfer. Every bitcoin movement is a CTransaction: a set of inputs (coins being spent) and outputs (coins being created). The transaction is atomic—either all inputs are spent and all outputs created, or nothing happens. This atomicity enables trustless exchanges: Alice can pay Bob in one indivisible operation. Satoshi's whitepaper Section 2: 'We define an electronic coin as a chain of digital signatures.'

```

358 {
359     public:
360     int nVersion;
361     vector<CTxIn> vin;
362     vector<CTxOut> vout;
363     int nLockTime;
364
365     CTransaction()
366     {
367         SetNull();
368     }
369
370     IMPLEMENT_SERIALIZE
371     (
372         READWRITE(this->nVersion);

```

```

374         nVersion = this->nVersion;
375 READWRITE(vin);
376 READWRITE(vout);
377 READWRITE(nLockTime);
378 )
379
380 void SetNull()
381 {
382     nVersion = 1;
383     vin.clear();
384     vout.clear();
385     nLockTime = 0;
386 }
387
388 bool IsNull() const
389 {
390     return (vin.empty() && vout.empty());
391 }
392
393 uint256 GetHash() const
394 {
395     return SerializeHash(*this);
396 }
397
398 bool IsFinal() const
399 {
400     if (nLockTime == 0 || nLockTime < nBestHeight)
401         return true;
402     foreach(const CTxIn& txin, vin)
403         if (!txin.IsFinal())
404             return false;
405     return true;
406 }
407
408 bool IsNewerThan(const CTransaction& old) const
409 {
410     if (vin.size() != old.vin.size())
411         return false;
412     for (int i = 0; i < vin.size(); i++)
413         if (vin[i].prevout != old.vin[i].prevout)
414             return false;
415
416     bool fNewer = false;
417     unsigned int nLowest = UINT_MAX;
418     for (int i = 0; i < vin.size(); i++)
419     {
420         if (vin[i].nSequence != old.vin[i].nSequence)
421         {
422             if (vin[i].nSequence <= nLowest)
423             {
424                 fNewer = false;
425                 nLowest = vin[i].nSequence;
426             }
427             if (old.vin[i].nSequence < nLowest)
428             {
429                 fNewer = true;
430                 nLowest = old.vin[i].nSequence;
431             }
432         }
433     }
434     return fNewer;
435 }
436
437 bool IsCoinBase() const
438 {
439     return (vin.size() == 1 && vin[0].prevout.IsNull());
440 }
441
442 bool CheckTransaction() const
443 {
444     // Basic checks that don't depend on any context
445     if (vin.empty() || vout.empty())

```

```

446 return error("CTransaction::CheckTransaction() : vin or vout empty");
447
448 // Check for negative values
449 foreach(const CTxOut& txout, vout)
450 if (txout.nValue < 0)
451 return error("CTransaction::CheckTransaction() : txout.nValue negative");
452
453 if (IsCoinBase())
454 {
455 if (vin[0].scriptSig.size() < 2 || vin[0].scriptSig.size() > 100)
456 return error("CTransaction::CheckTransaction() : coinbase script size");
457 }
458 else
459 {
460 foreach(const CTxIn& txin, vin)
461 if (txin.prevout.IsNull())
462 return error("CTransaction::CheckTransaction() : prevout is null");
463 }
464
465 return true;
466 }
467
468 bool IsMine() const
469 {
470 foreach(const CTxOut& txout, vout)
471 if (txout.IsMine())
472 return true;
473 return false;
474 }
475
476 int64 GetDebit() const
477 {
478     int64 nDebit = 0;
479     foreach(const CTxIn& txin, vin)
480         nDebit += txin.GetDebit();
481     return nDebit;
482 }
483
484 int64 GetCredit() const
485 {
486     int64 nCredit = 0;
487     foreach(const CTxOut& txout, vout)
488         nCredit += txout.GetCredit();
489     return nCredit;
490 }
491
492 int64 GetValueOut() const
493 {
494     int64 nValueOut = 0;
495     foreach(const CTxOut& txout, vout)
496     {
497         if (txout.nValue < 0)
498             throw runtime_error("CTransaction::GetValueOut() : negative value");
499         nValueOut += txout.nValue;
500     }
501     return nValueOut;
502 }
503
504 int64 GetMinFee(bool fDiscount=false) const

```

GetMinFee()—calculates the minimum required fee. Transaction fees are implicit: total inputs minus total outputs. Since inputs don't store amounts (only references), nodes must look up each referenced output to calculate the fee. This elegant design means fees don't bloat transaction size. In v0.1, minimum fee was 0.01 BTC per 1000 bytes—optional but encouraged. As Satoshi wrote: 'Free transactions are nice if people don't abuse them.'

```

505 {
506     unsigned int nBytes = ::GetSerializeSize(*this, SER_NETWORK);

```

```

507 if (fDiscount && nBytes < 10000)
508     return 0;
509 return (1 + (int64)nBytes / 1000) * CENT;
510 }
511
512
513
514 bool ReadFromDisk(CDiskTxPos pos, FILE** pfileRet=NULL)
515 {
516     CAutoFile filein = OpenBlockFile(pos.nFile, 0, pfileRet ? "rb+" : "rb");
517     if (!filein)
518         return error("CTransaction::ReadFromDisk() : OpenBlockFile failed");
519
520 // Read transaction
521 if (fseek(filein, pos.nTxPos, SEEK_SET) != 0)
522     return error("CTransaction::ReadFromDisk() : fseek failed");
523     filein >> *this;
524
525 // Return file pointer
526 if (pfileRet)
527 {
528     if (fseek(filein, pos.nTxPos, SEEK_SET) != 0)
529         return error("CTransaction::ReadFromDisk() : second fseek failed");
530     *pfileRet = filein.release();
531 }
532 return true;
533 }
534
535
536 friend bool operator==(const CTransaction& a, const CTransaction& b)
537 {
538     return (a.nVersion == b.nVersion &&
539             a.vin == b.vin &&
540             a.vout == b.vout &&
541             a.nLockTime == b.nLockTime);
542 }
543
544 friend bool operator!=(const CTransaction& a, const CTransaction& b)
545 {
546     return !(a == b);
547 }
548
549
550 string ToString() const
551 {
552     string str;
553     str += sprintf("CTransaction(hash=%s, ver=%d, vin.size=%d, vout.size=%d, nLockTime=%d)\n",
554 GetHash().ToString().substr(0,6).c_str(),
555         nVersion,
556         vin.size(),
557         vout.size(),
558         nLockTime);
559     for (int i = 0; i < vin.size(); i++)
560         str += "    " + vin[i].ToString() + "\n";
561     for (int i = 0; i < vout.size(); i++)
562         str += "    " + vout[i].ToString() + "\n";
563     return str;
564 }
565
566 void print() const
567 {
568     printf("%s", ToString().c_str());
569 }
570
571
572
573 bool DisconnectInputs(CTxDB& txdb);
574 bool ConnectInputs(CTxDB& txdb, map<uint256, CTxIndex>& mapTestPool, CDiskTxPos posThisTx, int nHeight,
575     int64& nFees, bool fBlock, bool fMiner, int64 nMinFee=0);
576 bool ClientConnectInputs();
577 bool AcceptTransaction(CTxDB& txdb, bool fCheckInputs=true, bool* pfMissingInputs=NULL);

```

```

578
579 bool AcceptTransaction(bool fCheckInputs=true, bool* pMissingInputs=NULL)
580 {
581     CTxDB txdb("r");
582     return AcceptTransaction(txdb, fCheckInputs, pMissingInputs);
583 }
584
585 protected:
586 bool AddToMemoryPool();
587 public:
588 bool RemoveFromMemoryPool();
589 };
590
591
592
593
594
595 //
596 // A transaction with a merkle branch linking it to the block chain
597 //
598 class CMerkleTx : public CTransaction

```

CMerkleTx - transaction with merkle proof. Whitepaper Section 8: enables SPV clients to verify inclusion without full blockchain.

```

599 {
600 public:
601     uint256 hashBlock;
602     vector<uint256> vMerkleBranch;
603     int nIndex;
604
605     // memory only
606     mutable bool fMerkleVerified;
607
608
609     CMerkleTx()
610     {
611         Init();
612     }
613
614     CMerkleTx(const CTransaction& txIn) : CTransaction(txIn)
615     {
616         Init();
617     }
618
619     void Init()
620     {
621         hashBlock = 0;
622         nIndex = -1;
623         fMerkleVerified = false;
624     }
625
626     int64 GetCredit() const
627     {
628         // Must wait until coinbase is safely deep enough in the chain before valuing it
629         if (IsCoinBase() && GetBlocksToMaturity() > 0)
630             return 0;
631         return CTransaction::GetCredit();
632     }
633
634     IMPLEMENT_SERIALIZE
635     (
636         nSerSize += SerReadWrite(s, *(CTransaction*)this, nType, nVersion, ser_action);
637         nVersion = this->nVersion;
638         READWRITE(hashBlock);
639         READWRITE(vMerkleBranch);
640         READWRITE(nIndex);
641     )
642

```



```

643
644 int SetMerkleBranch(const CBlock* pblock=NULL);
645 int GetDepthInMainChain() const;
646 bool IsInMainChain() const { return GetDepthInMainChain() > 0; }
647 int GetBlocksToMaturity() const;
648 bool AcceptTransaction(CTxDB& txdb, bool fCheckInputs=true);
649 bool AcceptTransaction() { CTxDB txdb("r"); return AcceptTransaction(txdb); }
650 };
651
652
653
654
655 //
656 // A transaction with a bunch of additional info that only the owner cares
657 // about. It includes any unrecorded transactions needed to link it back
658 // to the block chain.
659 //
660 class CWalletTx : public CMerkleTx

```

CWalletTx - your transaction view. Adds wallet metadata (credit, debit, time) to the base transaction. The user-facing layer.

```

661 {
662 public:
663     vector<CMerkleTx> vtxPrev;
664     map<string, string> mapValue;
665     vector<pair<string, string> > vOrderForm;
666     unsigned int fTimeReceivedIsTxTime;
667     unsigned int nTimeReceived; // time received by this node
668     char fFromMe;
669     char fSpent;
670     /// probably need to sign the order info so know it came from payer
671
672     // memory only
673     mutable unsigned int nTimeDisplayed;
674
675
676     CWalletTx()
677     {
678     Init();
679     }
680
681     CWalletTx(const CMerkleTx& txIn) : CMerkleTx(txIn)
682     {
683     Init();
684     }
685
686     CWalletTx(const CTransaction& txIn) : CMerkleTx(txIn)
687     {
688     Init();
689     }
690
691     void Init()
692     {
693         fTimeReceivedIsTxTime = false;
694         nTimeReceived = 0;
695         fFromMe = false;
696         fSpent = false;
697         nTimeDisplayed = 0;
698     }
699
700     IMPLEMENT_SERIALIZE
701 (
702         nSerSize += SerReadWrite(s, *(CMerkleTx*)this, nType, nVersion, ser_action);
703         nVersion = this->nVersion;
704     READWRITE(vtxPrev);
705     READWRITE(mapValue);
706     READWRITE(vOrderForm);
707     READWRITE(fTimeReceivedIsTxTime);

```

```

708 READWRITE(nTimeReceived);
709 READWRITE(fFromMe);
710 READWRITE(fSpent);
711 )
712
713 bool WriteToDisk()
714 {
715     return CWalletDB().WriteTx(GetHash(), *this);
716 }
717
718
719     int64 GetTxTime() const;
720
721 void AddSupportingTransactions(CTxDB& txdb);
722
723 bool AcceptWalletTransaction(CTxDB& txdb, bool fCheckInputs=true);
724 bool AcceptWalletTransaction() { CTxDB txdb("r"); return AcceptWalletTransaction(txdb); }
725
726 void RelayWalletTransaction(CTxDB& txdb);
727 void RelayWalletTransaction() { CTxDB txdb("r"); RelayWalletTransaction(txdb); }
728 };
729
730
731
732
733 //
734 // A txdb record that contains the disk location of a transaction and the
735 // locations of transactions that spend its outputs. vSpent is really only
736 // used as a flag, but having the location is very helpful for debugging.
737 //
738 class CTxIndex
739 {
740 public:
741     CDiskTxPos pos;
742     vector<CDiskTxPos> vSpent;
743
744 CTxIndex()
745 {
746     SetNull();
747 }
748
749 CTxIndex(const CDiskTxPos& posIn, unsigned int nOutputs)
750 {
751     pos = posIn;
752     vSpent.resize(nOutputs);
753 }
754
755     IMPLEMENT_SERIALIZE
756 (
757     if (!(nType & SER_GETHASH))
758         READWRITE(nVersion);
759     READWRITE(pos);
760     READWRITE(vSpent);
761 )
762
763 void SetNull()
764 {
765     pos.SetNull();
766     vSpent.clear();
767 }
768
769 bool IsNull()
770 {
771     return pos.IsNull();
772 }
773
774 friend bool operator==(const CTxIndex& a, const CTxIndex& b)
775 {
776     if (a.pos != b.pos || a.vSpent.size() != b.vSpent.size())
777         return false;
778     for (int i = 0; i < a.vSpent.size(); i++)
779         if (a.vSpent[i] != b.vSpent[i])

```

```

780 return false;
781 return true;
782 }
783
784 friend bool operator!=(const CTxIndex& a, const CTxIndex& b)
785 {
786 return !(a == b);
787 }
788 };
789
790
791
792
793
794 //
795 // Nodes collect new transactions into a block, hash them into a hash tree,
796 // and scan through nonce values to make the block's hash satisfy proof-of-work
797 // requirements. When they solve the proof-of-work, they broadcast the block
798 // to everyone and the block is added to the block chain. The first transaction
799 // in the block is a special one that creates a new coin owned by the creator
800 // of the block.
801 //
802 // Blocks are appended to blk0001.dat files on disk. Their location on disk
803 // is indexed by CBlockIndex objects in memory.
804 //
805 class CBlock

```

CBlock - a block of transactions with proof-of-work. The atomic unit of blockchain consensus. Each ~10 minutes, one is added.

```

806 {
807 public:
808 // header
809 int nVersion;
810 uint256 hashPrevBlock;
811 uint256 hashMerkleRoot;
812 unsigned int nTime;
813 unsigned int nBits;
814 unsigned int nNonce;
815
816 // network and disk
817 vector<CTransaction> vtx;
818
819 // memory only
820 mutable vector<uint256> vMerkleTree;
821
822
823 CBlock()
824 {
825 SetNull();
826 }
827
828 IMPLEMENT_SERIALIZE
829 (
830 READWRITE(this->nVersion);
831 nVersion = this->nVersion;
832 READWRITE(hashPrevBlock);
833 READWRITE(hashMerkleRoot);
834 READWRITE(nTime);
835 READWRITE(nBits);
836 READWRITE(nNonce);
837
838 // ConnectBlock depends on vtx being last so it can calculate offset
839 if (!(nType & (SER_GETHASH|SER_BLOCKHEADERONLY)))
840 READWRITE(vtx);
841 else if (fRead)
842 const_cast<CBlock*>(this)->vtx.clear();
843 )
844

```

```

845 void SetNull()
846 {
847     nVersion = 1;
848     hashPrevBlock = 0;
849     hashMerkleRoot = 0;
850     nTime = 0;
851     nBits = 0;
852     nNonce = 0;
853     vtx.clear();
854     vMerkleTree.clear();
855 }
856
857 bool IsNull() const
858 {
859     return (nBits == 0);
860 }
861
862 uint256 GetHash() const
863 {
864     return Hash(BEGIN(nVersion), END(nNonce));
865 }
866
867
868 uint256 BuildMerkleTree() const
869 {
870     vMerkleTree.clear();
871     foreach(const CTransaction& tx, vtx)
872         vMerkleTree.push_back(tx.GetHash());
873     int j = 0;
874     for (int nSize = vtx.size(); nSize > 1; nSize = (nSize + 1) / 2)
875     {
876         for (int i = 0; i < nSize; i += 2)
877         {
878             int i2 = min(i+1, nSize-1);
879             vMerkleTree.push_back(Hash(BEGIN(vMerkleTree[j+i]), END(vMerkleTree[j+i]),
880 BEGIN(vMerkleTree[j+i2]), END(vMerkleTree[j+i2]))));
881         }
882         j += nSize;
883     }
884     return (vMerkleTree.empty() ? 0 : vMerkleTree.back());
885 }
886
887 vector<uint256> GetMerkleBranch(int nIndex) const
888 {
889     if (vMerkleTree.empty())
890         BuildMerkleTree();
891     vector<uint256> vMerkleBranch;
892     int j = 0;
893     for (int nSize = vtx.size(); nSize > 1; nSize = (nSize + 1) / 2)
894     {
895         int i = min(nIndex^1, nSize-1);
896         vMerkleBranch.push_back(vMerkleTree[j+i]);
897         nIndex >>= 1;
898         j += nSize;
899     }
900     return vMerkleBranch;
901 }
902
903 static uint256 CheckMerkleBranch(uint256 hash, const vector<uint256>& vMerkleBranch, int nIndex)
904 {
905     if (nIndex == -1)
906         return 0;
907     foreach(const uint256& otherside, vMerkleBranch)
908     {
909         if (nIndex & 1)
910             hash = Hash(BEGIN(otherside), END(otherside), BEGIN(hash), END(hash));
911         else
912             hash = Hash(BEGIN(hash), END(hash), BEGIN(otherside), END(otherside));
913         nIndex >>= 1;
914     }
915     return hash;
916 }

```

```

917
918
919 bool WriteToDisk(bool fWriteTransactions, unsigned int& nFileRet, unsigned int& nBlockPosRet)
920 {
921     // Open history file to append
922     CAutoFile fileout = AppendBlockFile(nFileRet);
923     if (!fileout)
924         return error("CBlock::WriteToDisk() : AppendBlockFile failed");
925     if (!fWriteTransactions)
926         fileout.nType |= SER_BLOCKHEADERONLY;
927
928     // Write index header
929     unsigned int nSize = fileout.GetSerializeSize(*this);
930     fileout << FLATDATA(pchMessageStart) << nSize;
931
932     // Write block
933     nBlockPosRet = ftell(fileout);
934     if (nBlockPosRet == -1)
935         return error("CBlock::WriteToDisk() : ftell failed");
936     fileout << *this;
937
938     return true;
939 }
940
941 bool ReadFromDisk(unsigned int nFile, unsigned int nBlockPos, bool fReadTransactions)
942 {
943     SetNull();
944
945     // Open history file to read
946     CAutoFile filein = OpenBlockFile(nFile, nBlockPos, "rb");
947     if (!filein)
948         return error("CBlock::ReadFromDisk() : OpenBlockFile failed");
949     if (!fReadTransactions)
950         filein.nType |= SER_BLOCKHEADERONLY;
951
952     // Read block
953     filein >> *this;
954
955     // Check the header
956     if (CBigNum().SetCompact(nBits) > bnProofOfWorkLimit)
957         return error("CBlock::ReadFromDisk() : nBits errors in block header");
958     if (GetHash() > CBigNum().SetCompact(nBits).getuint256())
959         return error("CBlock::ReadFromDisk() : GetHash() errors in block header");
960
961     return true;
962 }
963
964
965
966 void print() const
967 {
968     printf("CBlock(hash=%s, ver=%d, hashPrevBlock=%s, hashMerkleRoot=%s, nTime=%u, nBits=%08x, nNonce=%u, vtx=%d)\n",
969         GetHash().ToString().substr(0,14).c_str(),
970         nVersion,
971         hashPrevBlock.ToString().substr(0,14).c_str(),
972         hashMerkleRoot.ToString().substr(0,6).c_str(),
973         nTime, nBits, nNonce,
974         vtx.size());
975     for (int i = 0; i < vtx.size(); i++)
976     {
977         printf("  ");
978         vtx[i].print();
979     }
980     printf("  vMerkleTree: ");
981     for (int i = 0; i < vMerkleTree.size(); i++)
982         printf("%s ", vMerkleTree[i].ToString().substr(0,6).c_str());
983     printf("\n");
984 }
985
986
987 int64 GetBlockValue(int64 nFees) const;

```

```

988 bool DisconnectBlock(CTxDB& txdb, CBlockIndex* pindex);
989 bool ConnectBlock(CTxDB& txdb, CBlockIndex* pindex);
990 bool ReadFromDisk(const CBlockIndex* blockindex, bool fReadTransactions);
991 bool AddToBlockIndex(unsigned int nFile, unsigned int nBlockPos);
992 bool CheckBlock() const;
993 bool AcceptBlock();
994 };
995
996
997
998
999
1000
1001 //
1002 // The block chain is a tree shaped structure starting with the
1003 // genesis block at the root, with each block potentially having multiple
1004 // candidates to be the next block. pprev and pnext link a path through the
1005 // main/longest chain. A blockindex may have multiple pprev pointing back
1006 // to it, but pnext will only point forward to the longest branch, or will
1007 // be null if the block is not part of the longest chain.
1008 //
1009 class CBlockIndex

```

CBlockIndex - in-memory block metadata. Height, work, timestamps. Enables fast chain traversal without reading full blocks from disk.

```

1010 {
1011 public:
1012     const uint256* phashBlock;
1013     CBlockIndex* pprev;
1014     CBlockIndex* pnext;
1015     unsigned int nFile;
1016     unsigned int nBlockPos;
1017     int nHeight;
1018
1019     // block header
1020     int nVersion;
1021     uint256 hashMerkleRoot;
1022     unsigned int nTime;
1023     unsigned int nBits;
1024     unsigned int nNonce;
1025
1026
1027     CBlockIndex()
1028     {
1029         phashBlock = NULL;
1030         pprev = NULL;
1031         pnext = NULL;
1032         nFile = 0;
1033         nBlockPos = 0;
1034         nHeight = 0;
1035
1036         nVersion      = 0;
1037         hashMerkleRoot = 0;
1038         nTime         = 0;
1039         nBits         = 0;
1040         nNonce        = 0;
1041     }
1042
1043     CBlockIndex(unsigned int nFileIn, unsigned int nBlockPosIn, CBlock& block)
1044     {
1045         phashBlock = NULL;
1046         pprev = NULL;
1047         pnext = NULL;
1048         nFile = nFileIn;
1049         nBlockPos = nBlockPosIn;
1050         nHeight = 0;
1051
1052         nVersion      = block.nVersion;

```

```

1053         hashMerkleRoot = block.hashMerkleRoot;
1054         nTime           = block.nTime;
1055         nBits           = block.nBits;
1056         nNonce          = block.nNonce;
1057     }
1058
1059     uint256 GetBlockHash() const
1060     {
1061         return *phashBlock;
1062     }
1063
1064     bool IsInMainChain() const
1065     {
1066         return (pnext || this == pindexBest);
1067     }
1068
1069     bool EraseBlockFromDisk()
1070     {
1071         // Open history file
1072         CAutoFile fileout = OpenBlockFile(nFile, nBlockPos, "rb+");
1073         if (!fileout)
1074             return false;
1075
1076         // Overwrite with empty null block
1077         CBlock block;
1078         block.SetNull();
1079         fileout << block;
1080
1081         return true;
1082     }
1083
1084     enum { nMedianTimeSpan=11 };
1085
1086     int64 GetMedianTimePast() const
1087     {
1088         unsigned int pmedian[nMedianTimeSpan];
1089         unsigned int* pbegin = &pmedian[nMedianTimeSpan];
1090         unsigned int* pend = &pmedian[nMedianTimeSpan];
1091
1092         const CBlockIndex* pindex = this;
1093         for (int i = 0; i < nMedianTimeSpan && pindex; i++, pindex = pindex->pprev)
1094             *(&pbegin) = pindex->nTime;
1095
1096         sort(pbegin, pend);
1097         return pbegin[(pend - pbegin)/2];
1098     }
1099
1100     int64 GetMedianTime() const
1101     {
1102         const CBlockIndex* pindex = this;
1103         for (int i = 0; i < nMedianTimeSpan/2; i++)
1104         {
1105             if (!pindex->pnext)
1106                 return nTime;
1107             pindex = pindex->pnext;
1108         }
1109         return pindex->GetMedianTimePast();
1110     }
1111
1112
1113
1114     string ToString() const
1115     {
1116         return strprintf("CBlockIndex(nprev=%08x, pnext=%08x, nFile=%d, nBlockPos=%-6d nHeight=%d, merkle=%s,
1117             hashBlock=%s)",
1118             pprev, pnext, nFile, nBlockPos, nHeight,
1119             hashMerkleRoot.ToString().substr(0,6).c_str(),
1120             GetBlockHash().ToString().substr(0,14).c_str());
1121     }
1122
1123     void print() const
1124     {

```

```

1124 printf("%s\n", ToString().c_str());
1125 }
1126 };
1127
1128
1129
1130 //
1131 // Used to marshal pointers into hashes for db storage.
1132 //
1133 class CDiskBlockIndex : public CBlockIndex
1134 {
1135 public:
1136     uint256 hashPrev;
1137     uint256 hashNext;
1138
1139     CDiskBlockIndex()
1140     {
1141         hashPrev = 0;
1142         hashNext = 0;
1143     }
1144
1145     explicit CDiskBlockIndex(CBlockIndex* pindex) : CBlockIndex(*pindex)
1146     {
1147         hashPrev = (pprev ? pprev->GetBlockHash() : 0);
1148         hashNext = (pnext ? pnext->GetBlockHash() : 0);
1149     }
1150
1151     IMPLEMENT_SERIALIZE
1152     (
1153     if (!(nType & SER_GETHASH))
1154     READWRITE(nVersion);
1155
1156     READWRITE(hashNext);
1157     READWRITE(nFile);
1158     READWRITE(nBlockPos);
1159     READWRITE(nHeight);
1160
1161     // block header
1162     READWRITE(this->nVersion);
1163     READWRITE(hashPrev);
1164     READWRITE(hashMerkleRoot);
1165     READWRITE(nTime);
1166     READWRITE(nBits);
1167     READWRITE(nNonce);
1168     )
1169
1170     uint256 GetBlockHash() const
1171     {
1172         CBlock block;
1173         block.nVersion      = nVersion;
1174         block.hashPrevBlock = hashPrev;
1175         block.hashMerkleRoot = hashMerkleRoot;
1176         block.nTime         = nTime;
1177         block.nBits         = nBits;
1178         block.nNonce        = nNonce;
1179     return block.GetHash();
1180     }
1181
1182
1183     string ToString() const
1184     {
1185         string str = "CDiskBlockIndex(";
1186         str += CBlockIndex::ToString();
1187         str += sprintf("\n                hashBlock=%s, hashPrev=%s, hashNext=%s)",
1188             GetBlockHash().ToString().c_str(),
1189             hashPrev.ToString().substr(0,14).c_str(),
1190             hashNext.ToString().substr(0,14).c_str());
1191     return str;
1192     }
1193
1194     void print() const
1195     {

```



```

1196 printf("%s\n", ToString().c_str());
1197 }
1198 };
1199
1200
1201
1202
1203
1204
1205
1206
1207 //
1208 // Describes a place in the block chain to another node such that if the
1209 // other node doesn't have the same branch, it can find a recent common trunk.
1210 // The further back it is, the further before the fork it may be.
1211 //
1212 class CBlockLocator

```

CBlockLocator – compact chain representation for syncing. Exponentially spaced block hashes let nodes find common ancestor efficiently.

```

1213 {
1214     protected:
1215         vector<uint256> vHave;
1216     public:
1217
1218     CBlockLocator()
1219     {
1220     }
1221
1222     explicit CBlockLocator(const CBlockIndex* pindex)
1223     {
1224         Set(pindex);
1225     }
1226
1227     explicit CBlockLocator(uint256 hashBlock)
1228     {
1229         map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hashBlock);
1230         if (mi != mapBlockIndex.end())
1231             Set((*mi).second);
1232     }
1233
1234     IMPLEMENT_SERIALIZE
1235     (
1236         if (!(nType & SER_GETHASH))
1237             READWRITE(nVersion);
1238         READWRITE(vHave);
1239     )
1240
1241     void Set(const CBlockIndex* pindex)
1242     {
1243         vHave.clear();
1244         int nStep = 1;
1245         while (pindex)
1246         {
1247             vHave.push_back(pindex->GetBlockHash());
1248
1249             // Exponentially larger steps back
1250             for (int i = 0; pindex && i < nStep; i++)
1251                 pindex = pindex->pprev;
1252             if (vHave.size() > 10)
1253                 nStep *= 2;
1254         }
1255         vHave.push_back(hashGenesisBlock);
1256     }
1257
1258     CBlockIndex* GetBlockIndex()
1259     {
1260         // Find the first block the caller has in the main chain

```

```

1261 foreach(const uint256& hash, vHave)
1262 {
1263     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hash);
1264     if (mi != mapBlockIndex.end())
1265     {
1266         CBlockIndex* pindex = (*mi).second;
1267         if (pindex->IsInMainChain())
1268             return pindex;
1269     }
1270 }
1271 return pindexGenesisBlock;
1272 }
1273
1274 uint256 GetBlockHash()
1275 {
1276     // Find the first block the caller has in the main chain
1277     foreach(const uint256& hash, vHave)
1278     {
1279         map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(hash);
1280         if (mi != mapBlockIndex.end())
1281         {
1282             CBlockIndex* pindex = (*mi).second;
1283             if (pindex->IsInMainChain())
1284                 return hash;
1285         }
1286     }
1287     return hashGenesisBlock;
1288 }
1289
1290 int GetHeight()
1291 {
1292     CBlockIndex* pindex = GetBlockIndex();
1293     if (!pindex)
1294         return 0;
1295     return pindex->nHeight;
1296 }
1297 };
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310 extern map<uint256, CTransaction> mapTransactions;
1311 extern map<uint256, CWalletTx> mapWallet;
1312 extern vector<pair<uint256, bool> > vWalletUpdated;
1313 extern CCriticalSection cs_mapWallet;
1314 extern map<vector<unsigned char>, CPrivKey> mapKeys;
1315 extern map<uint160, vector<unsigned char> > mapPubKeys;
1316 extern CCriticalSection cs_mapKeys;
1317 extern CKey keyUser;
1318

```

SUMMARY

These data structures implement what Satoshi called "a purely peer-to-peer version of electronic cash." The design decisions made here have proven remarkably durable—15+ years later, the core structures remain essentially unchanged despite handling trillions of dollars in value.

The UTXO model that Satoshi invented (tracking individual coin outputs rather than account balances) enables several key properties: transactions can be validated in parallel since each input is independent; ownership proofs are simple (just show the chain of signatures); and the entire monetary supply can be audited by summing all unspent outputs. When Ethereum chose the account model instead, they gained programmer convenience but lost these properties.

Notice what's NOT in this header: no 'Account' class, no 'Balance' field, no central ledger. Bitcoin doesn't know your balance—your wallet calculates it by scanning for outputs your keys can spend. This stateless design is why Bitcoin scales to millions of users without centralized databases. Your coins exist as scattered outputs across the blockchain, united only by your private keys.

makefile

84 lines MAKEFILE

ASSEMBLY REQUIRED (UNIX)

The makefile tells the compiler how to build Bitcoin on Unix-like systems. It lists all source files, compiler flags, and library dependencies needed to produce the bitcoin executable.

MAKEFILE

```
1 # Copyright (c) 2009 Satoshi Nakamoto
2 # Distributed under the MIT/X11 software license, see the accompanying
3 # file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5
6 ifneq "$(BUILD)" "debug"
```

Build mode selection - debug or release

```
7 ifneq "$(BUILD)" "release"
8 BUILD=debug
9 endif
10 endif
11 ifeq "$(BUILD)" "debug"
12 D=d
13 # note: gcc 3.x profile doesn't work
14 #DEBUGFLAGS=-O0 -g -pg -D__WXDEBUG__
15 DEBUGFLAGS=-g -D__WXDEBUG__
16 endif
17
18
19
20 INCLUDEPATHS=-I"/boost" -I"/DB/build_unix" -I"/OpenSSL/include" -I"/wxWidgets/lib/vc_lib/mswd" -
I"/wxWidgets/include"
```

Include paths - Boost, Berkeley DB, OpenSSL, wxWidgets

```
21 LIBPATHS=-L"/DB/build_unix" -L"/OpenSSL/out" -L"/wxWidgets/lib/gcc_lib"
22 LIBS= \
```

Library dependencies - all external libraries linked

```
23 -l db_cxx \
24 -l eay32 \
25 -l wxmsw28$(D)_richtext -l wxmsw28$(D)_html -l wxmsw28$(D)_core -l wxbase28$(D) -l wxtiff$(D) -l
wxjpeg$(D) -l wxpng$(D) -l wxzlib$(D) -l wxregex$(D) -l wxexpat$(D) \
26 -l kernel32 -l user32 -l gdi32 -l comdlg32 -l winspool -l winmm -l shell32 -l comctl32 -l ole32 -l
oleaut32 -l uuid -l rpcrt4 -l advapi32 -l ws2_32
27 WXDEFS=-DWIN32 -D__WXMSW__ -D_WINDOWS -DNOPICH
28 CFLAGS=-mthreads -O0 -w -Wno-invalid-offsetof -Wformat $(DEBUGFLAGS) $(WXDEFS) $(INCLUDEPATHS)
29 HEADERS=headers.h util.h main.h serialize.h uint256.h key.h bignum.h script.h db.h base58.h
30
31
32
33 all: bitcoin.exe
34
35
36 headers.h.gch: headers.h $(HEADERS) net.h irc.h market.h uibase.h ui.h
37 g++ -c $(CFLAGS) -o $@ $<
38
39 obj/util.o: util.cpp $(HEADERS)
40 g++ -c $(CFLAGS) -o $@ $<
41
42 obj/script.o: script.cpp $(HEADERS)
43 g++ -c $(CFLAGS) -o $@ $<
44
45 obj/db.o: db.cpp $(HEADERS) market.h
46 g++ -c $(CFLAGS) -o $@ $<
47
48 obj/net.o: net.cpp $(HEADERS) net.h
49 g++ -c $(CFLAGS) -o $@ $<
50
51 obj/main.o: main.cpp $(HEADERS) net.h market.h sha.h
52 g++ -c $(CFLAGS) -o $@ $<
53
```

```

54 obj/market.o: market.cpp          $(HEADERS) market.h
55     g++ -c $(CFLAGS) -o $@ $<
56
57 obj/ui.o: ui.cpp                   $(HEADERS) net.h uibase.h ui.h market.h
58     g++ -c $(CFLAGS) -o $@ $<
59
60 obj/uiibase.o: uibase.cpp          uibase.h
61     g++ -c $(CFLAGS) -o $@ $<
62
63 obj/sha.o: sha.cpp                sha.h
64     g++ -c $(CFLAGS) -O3 -o $@ $<
65
66 obj/irc.o: irc.cpp                $(HEADERS)
67     g++ -c $(CFLAGS) -o $@ $<
68
69 obj/ui_res.o: ui.rc rc/bitcoin.ico rc/check.ico rc/send16.bmp rc/send16mask.bmp
rc/send16masknoshadow.bmp rc/send20.bmp rc/send20mask.bmp rc/addressbook16.bmp rc/addressbook16mask.bmp
rc/addressbook20.bmp rc/addressbook20mask.bmp
70     windres $(WXDEFS) $(INCLUDEPATHS) -o $@ -i $<
71
72
73
74 OBJJS=obj/util.o obj/script.o obj/db.o obj/net.o obj/main.o obj/market.o \
75     obj/ui.o obj/uiibase.o obj/sha.o obj/irc.o obj/ui_res.o
76
77 bitcoin.exe: headers.h.gch $(OBJJS)
78     -kill /f bitcoin.exe
79     g++ $(CFLAGS) -mwindows -WL,--subsystem,windows -o $@ $(LIBPATHS) $(OBJJS) $(LIBS)
80
81 clean:
82     -del /Q obj\*
83     -del /Q headers.h.gch
84

```

SUMMARY

Bitcoin v0.01 depended on wxWidgets (for the GUI), OpenSSL (for cryptography), and Berkeley DB (for data storage).

makefile.vc

78 lines MAKEFILE

ASSEMBLY REQUIRED (WINDOWS)

This makefile builds Bitcoin using Microsoft Visual C++. It's functionally equivalent to the Unix makefile but uses Windows-specific paths and compiler options.

```

1 # Copyright (c) 2009 Satoshi Nakamoto
2 # Distributed under the MIT/X11 software license, see the accompanying
3 # file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5
6 !IF "$(BUILD)" != "debug" && "$(BUILD)" != "release"
7 BUILD=debug
8 !ENDIF
9 !IF "$(BUILD)" == "debug"

```

Debug configuration with symbols and WXDEBUG

```

10 D=d
11 DEBUGFLAGS=/Zi /Od /D__WXDEBUG__
12 !ENDIF
13
14
15
16 INCLUDEPATHS=/I"/boost" /I"/DB/build_windows" /I"/OpenSSL/include" /I"/wxWidgets/lib/vc_lib/mswd"
/I"/wxWidgets/include"

```

Windows include paths for dependencies

```

17 LIBPATHS=/LIBPATH:"/DB/build_windows/$(BUILD)" /LIBPATH:"/OpenSSL/out" /LIBPATH:"/wxWidgets/lib/vc_lib"
18 LIBS= \
19     libdb47s$(D).lib \
20     libeay32.lib \
21     wxmsw28$(D)_richtext.lib wxmsw28$(D)_html.lib wxmsw28$(D)_core.lib wxbase28$(D).lib wxtiff$(D).lib
wxjpeg$(D).lib wxpng$(D).lib wxzlib$(D).lib wxregex$(D).lib wxexpat$(D).lib \
22     kernel32.lib user32.lib gdi32.lib comdlg32.lib winspool.lib winmm.lib shell32.lib comctl32.lib
ole32.lib oleaut32.lib uuid.lib rpcrt4.lib advapi32.lib ws2_32.lib
23 WXDEFS=/DWIN32 /D__WMSW__ /D_WINDOWS /DNOPCH
24 CFLAGS=/c /nologo /Ob0 /MD$(D) /EHsc /GR /Zm300 /YX /Fpobj/headers.pch $(DEBUGFLAGS) $(WXDEFS)
$(INCLUDEPATHS)
25 HEADERS=headers.h util.h main.h serialize.h uint256.h key.h bignum.h script.h db.h base58.h
26
27
28
29 all: bitcoin.exe

```

Build target - produces bitcoin.exe

```

30
31
32 obj\util.obj: util.cpp $(HEADERS)
33     cl $(CFLAGS) /Fo$@ %s
34
35 obj\script.obj: script.cpp $(HEADERS)
36     cl $(CFLAGS) /Fo$@ %s
37
38 obj\db.obj: db.cpp $(HEADERS) market.h
39     cl $(CFLAGS) /Fo$@ %s
40
41 obj\net.obj: net.cpp $(HEADERS) net.h
42     cl $(CFLAGS) /Fo$@ %s
43
44 obj\main.obj: main.cpp $(HEADERS) net.h market.h
45     cl $(CFLAGS) /Fo$@ %s
46
47 obj\market.obj: market.cpp $(HEADERS) market.h
48     cl $(CFLAGS) /Fo$@ %s
49
50 obj\ui.obj: ui.cpp $(HEADERS) net.h uibase.h ui.h market.h
51     cl $(CFLAGS) /Fo$@ %s
52

```

```

53 obj\uibase.obj: uibase.cpp      uibase.h
54     cl $(CFLAGS) /Fo$@ %s
55
56 obj\sha.obj: sha.cpp sha.h
57     cl $(CFLAGS) /O2 /Fo$@ %s
58
59 obj\irc.obj:  irc.cpp           $(HEADERS)
60     cl $(CFLAGS) /Fo$@ %s
61
62 obj\ui.res: ui.rc  rc/bitcoin.ico rc/check.ico rc/send16.bmp rc/send16mask.bmp rc/send16masknoshadow.bmp
63     rc $(INCLUDEPATHS) $(WXDEFS) /Fo$@ %s
64
65
66
67 OBJJS=obj\util.obj obj\script.obj obj\db.obj obj\net.obj obj\main.obj obj\market.obj \
68     obj\ui.obj obj\uibase.obj obj\sha.obj obj\irc.obj obj\ui.res
69
70 bitcoin.exe: $(OBJJS)
71     -kill /f bitcoin.exe & sleep 1
72     link /nologo /DEBUG /SUBSYSTEM:WINDOWS /OUT:$@ $(LIBPATHS) $** $(LIBS)
73
74 clean:
75     -del /Q obj\*
76     -del *.ilk
77     -del *.pdb
78

```

SUMMARY

Satoshi initially developed Bitcoin on Windows, so Windows support was a priority from the very first release.

market.cpp

265 lines CPP

THE ROAD NOT TAKEN

An early experimental feature for a built-in eBay-style marketplace where users could trade goods for bitcoins directly within the application. Satoshi described this as one of his unfinished projects: "I was trying to implement an eBay-style marketplace built into the client" (2011 email to Mike Hearn). The feature was never completed.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6
7
8
9
10
11
12
13
14
15
16 //
17 // Global state variables
18 //
19
20 //// later figure out how these are persisted
21 map<uint256, CProduct> mapMyProducts;

```

mapMyProducts - your products for sale

```

22
23
24
25
26 map<uint256, CProduct> mapProducts;

```

mapProducts - global marketplace listing

```

27 CCriticalSection cs_mapProducts;
28
29 bool AdvertInsert(const CProduct& product)

```

AdvertInsert - adds a product to the marketplace

```

30 {
31     uint256 hash = product.GetHash();
32     bool fNew = false;
33     bool fUpdated = false;
34
35     CRITICAL_BLOCK(cs_mapProducts)
36     {
37         // Insert or find existing product
38         pair<map<uint256, CProduct>::iterator, bool> item = mapProducts.insert(make_pair(hash,
39 product));
40         CProduct* pproduct = &(*(item.first)).second;
41         fNew = item.second;
42         // Update if newer
43         if (product.nSequence > pproduct->nSequence)
44         {
45             *pproduct = product;
46             fUpdated = true;
47         }
48     }
49
50     //if (fNew)
51     //    NotifyProductAdded(hash);
52     //else if (fUpdated)
53     //    NotifyProductUpdated(hash);
54
55     return (fNew || fUpdated);

```

```

56 }
57
58 void AdvertErase(const CProduct& product)
59 {
60     uint256 hash = product.GetHash();
61     CRITICAL_BLOCK(cs_mapProducts)
62         mapProducts.erase(hash);
63     //NotifyProductDeleted(hash);
64 }
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84 template<typename T>
85 unsigned int Union(T& v1, T& v2)
86 {
87     // v1 = v1 union v2
88     // v1 and v2 must be sorted
89     // returns the number of elements added to v1
90
91     //// need to check that this is equivalent, then delete this comment
92     //vector<unsigned short> vUnion(v1.size() + v2.size());
93     //vUnion.erase(set_union(v1.begin(), v1.end(),
94     //                        v2.begin(), v2.end(),
95     //                        vUnion.begin()),
96     //              vUnion.end());
97
98     T vUnion;
99     vUnion.reserve(v1.size() + v2.size());
100    set_union(v1.begin(), v1.end(),
101             v2.begin(), v2.end(),
102             back_inserter(vUnion));
103    unsigned int nAdded = vUnion.size() - v1.size();
104    if (nAdded > 0)
105        v1 = vUnion;
106    return nAdded;
107 }
108
109 void CUser::AddAtom(unsigned short nAtom, bool f0origin)
110 {
111     // Ignore duplicates
112     if (binary_search(vAtomsIn.begin(), vAtomsIn.end(), nAtom) ||
113         find(vAtomsNew.begin(), vAtomsNew.end(), nAtom) != vAtomsNew.end())
114         return;
115
116     //// instead of zero atom, should change to free atom that propagates,
117     //// limited to lower than a certain value like 5 so conflicts quickly
118     // The zero atom never propagates,
119     // new atoms always propagate through the user that created them
120     if (nAtom == 0 || f0origin)
121     {
122         vector<unsigned short> vTmp(1, nAtom);
123         Union(vAtomsIn, vTmp);
124         if (f0origin)
125             vAtomsOut.push_back(nAtom);
126         return;
127     }

```

```

128
129     vAtomsNew.push_back(nAtom);
130
131     if (vAtomsNew.size() >= nFlowthroughRate || vAtomsOut.empty())
132     {
133         // Select atom to flow through to vAtomsOut
134         vAtomsOut.push_back(vAtomsNew[GetRand(vAtomsNew.size())]);
135
136         // Merge vAtomsNew into vAtomsIn
137         sort(vAtomsNew.begin(), vAtomsNew.end());
138         Union(vAtomsIn, vAtomsNew);
139         vAtomsNew.clear();
140     }
141 }
142
143 bool AddAtomsAndPropagate(uint256 hashUserStart, const vector<unsigned short>& vAtoms, bool fOrigin)
144 {
145     CReviewDB reviewdb;
146     map<uint256, vector<unsigned short> > pmapPropagate[2];
147     pmapPropagate[0][hashUserStart] = vAtoms;
148
149     for (int side = 0; !pmapPropagate[side].empty(); side = 1 - side)
150     {
151         map<uint256, vector<unsigned short> >& mapFrom = pmapPropagate[side];
152         map<uint256, vector<unsigned short> >& mapTo = pmapPropagate[1 - side];
153
154         for (map<uint256, vector<unsigned short> >::iterator mi = mapFrom.begin(); mi != mapFrom.end(); ++mi)
155         {
156             const uint256& hashUser = (*mi).first;
157             const vector<unsigned short>& vReceived = (*mi).second;
158
159             ///// this would be a lot easier on the database if it put the new atom at the beginning of the list,
160             ///// so the change would be right next to the vector size.
161
162             // Read user
163             CUser user;
164             reviewdb.ReadUser(hashUser, user);
165             unsigned int nIn = user.vAtomsIn.size();
166             unsigned int nNew = user.vAtomsNew.size();
167             unsigned int nOut = user.vAtomsOut.size();
168
169             // Add atoms received
170             foreach(unsigned short nAtom, vReceived)
171                 user.AddAtom(nAtom, fOrigin);
172             fOrigin = false;
173
174             // Don't bother writing to disk if no changes
175             if (user.vAtomsIn.size() == nIn && user.vAtomsNew.size() == nNew)
176                 continue;
177
178             // Propagate
179             if (user.vAtomsOut.size() > nOut)
180                 foreach(const uint256& hash, user.vLinksOut)
181                     mapTo[hash].insert(mapTo[hash].end(), user.vAtomsOut.begin() + nOut,
182                                         user.vAtomsOut.end());
183
184             // Write back
185             if (!reviewdb.WriteUser(hashUser, user))
186                 return false;
187             mapFrom.clear();
188         }
189         return true;
190     }
191
192
193
194
195
196
197 bool CReview::AcceptReview()
198 {

```



```

199 // Timestamp
200     nTime = GetTime();
201
202 // Check signature
203 if (!CKey::Verify(vchPubKeyFrom, GetSigHash(), vchSig))
204     return false;
205
206     CReviewDB reviewdb;
207
208 // Add review text to recipient
209     vector<CReview> vReviews;
210     reviewdb.ReadReviews(hashTo, vReviews);
211     vReviews.push_back(*this);
212 if (!reviewdb.WriteReviews(hashTo, vReviews))
213     return false;
214
215 // Add link from sender
216     CUser user;
217     uint256 hashFrom = Hash(vchPubKeyFrom.begin(), vchPubKeyFrom.end());
218     reviewdb.ReadUser(hashFrom, user);
219     user.vLinksOut.push_back(hashTo);
220 if (!reviewdb.WriteUser(hashFrom, user))
221     return false;
222
223     reviewdb.Close();
224
225 // Propagate atoms to recipient
226     vector<unsigned short> vZeroAtom(1, 0);
227 if (!AddAtomsAndPropagate(hashTo, user.vAtomsOut.size() ? user.vAtomsOut : vZeroAtom, false))
228     return false;
229
230     return true;
231 }
232
233
234
235
236
237 bool CProduct::CheckSignature()
238 {
239     return (CKey::Verify(vchPubKeyFrom, GetSigHash(), vchSig));
240 }
241
242 bool CProduct::CheckProduct()
243 {
244     if (!CheckSignature())
245         return false;
246
247 // Make sure it's a summary product
248 if (!mapDetails.empty() || !vOrderForm.empty())
249     return false;
250
251 // Look up seller's atom count
252     CReviewDB reviewdb("r");
253     CUser user;
254     reviewdb.ReadUser(GetUserHash(), user);
255     nAtoms = user.GetAtomCount();
256     reviewdb.Close();
257
258 // delme, this is now done by AdvertInsert
259 /// Store to memory
260 //CRITICAL_BLOCK(cs_mapProducts)
261 //     mapProducts[GetHash()] = *this;
262
263     return true;
264 }
265

```



SUMMARY

It shows Satoshi's early vision of Bitcoin as not just a currency but a complete economic system with integrated commerce.

market.h

183 lines CPP

SATOSHI'S UNFINISHED DREAM

Header file for Satoshi's unfinished eBay-style marketplace. Defines data structures for products and reviews that would have powered peer-to-peer commerce directly within the Bitcoin client.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 class CUser;
6 class CReview;

```

Forward declarations for marketplace types

```

7 class CProduct;
8
9 static const unsigned int nFlowthroughRate = 2;
10
11
12
13
14 bool AdvertInsert(const CProduct& product);
15 void AdvertErase(const CProduct& product);
16 bool AddAtomsAndPropagate(uint256 hashUserStart, const vector<unsigned short>& vAtoms, bool fOrigin);
17
18
19
20
21
22
23
24
25 class CUser
26 {
27 public:
28     vector<unsigned short> vAtomsIn;
29     vector<unsigned short> vAtomsNew;
30     vector<unsigned short> vAtomsOut;
31     vector<uint256> vLinksOut;
32
33 CUser()
34 {
35 }
36
37 IMPLEMENT_SERIALIZE
38 (
39 if (!(nType & SER_GETHASH))
40 READWRITE(nVersion);
41 READWRITE(vAtomsIn);
42 READWRITE(vAtomsNew);
43 READWRITE(vAtomsOut);
44 READWRITE(vLinksOut);
45 )
46
47 void SetNull()
48 {
49     vAtomsIn.clear();
50     vAtomsNew.clear();
51     vAtomsOut.clear();
52     vLinksOut.clear();
53 }
54
55 uint256 GetHash() const { return SerializeHash(*this); }
56
57
58 int GetAtomCount() const
59 {
60 return (vAtomsIn.size() + vAtomsNew.size());
61 }
62
63 void AddAtom(unsigned short nAtom, bool fOrigin);
64 };
65
66

```

```

67
68
69
70
71
72 class CReview

```

CReview class - user reviews for marketplace items

```

73 {
74 public:
75 int nVersion;
76     uint256 hashTo;
77     map<string, string> mapValue;
78     vector<unsigned char> vchPubKeyFrom;
79     vector<unsigned char> vchSig;
80
81 // memory only
82 unsigned int nTime;
83 int nAtoms;
84
85
86 CReview()
87 {
88     nVersion = 1;
89     hashTo = 0;
90     nTime = 0;
91     nAtoms = 0;
92 }
93
94 IMPLEMENT_SERIALIZE
95 (
96 READWRITE(this->nVersion);
97     nVersion = this->nVersion;
98 if (!(nType & SER_DISK))
99 READWRITE(hashTo);
100 READWRITE(mapValue);
101 READWRITE(vchPubKeyFrom);
102 if (!(nType & SER_GETHASH))
103 READWRITE(vchSig);
104 )
105
106 uint256 GetHash() const { return SerializeHash(*this); }
107 uint256 GetSigHash() const { return SerializeHash(*this, SER_GETHASH|SER_SKIPSIG); }
108 uint256 GetUserHash() const { return Hash(vchPubKeyFrom.begin(), vchPubKeyFrom.end()); }
109
110
111 bool AcceptReview();
112 };
113
114
115
116
117
118
119
120 class CProduct

```

CProduct class - items listed for sale

```

121 {
122 public:
123 int nVersion;
124     CAddress addr;
125     map<string, string> mapValue;
126     map<string, string> mapDetails;
127     vector<pair<string, string> > vOrderForm;

```

```

128 unsigned int nSequence;
129     vector<unsigned char> vchPubKeyFrom;
130     vector<unsigned char> vchSig;
131
132 // disk only
133 int nAtoms;
134
135 // memory only
136     set<unsigned int> setSources;
137
138 CProduct()
139 {
140     nVersion = 1;
141     nAtoms = 0;
142     nSequence = 0;
143 }
144
145 IMPLEMENT_SERIALIZE
146 (
147     READWRITE(this->nVersion);
148     nVersion = this->nVersion;
149     READWRITE(addr);
150     READWRITE(mapValue);
151     if (!(nType & SER_GETHASH))
152     {
153         READWRITE(mapDetails);
154         READWRITE(v0OrderForm);
155         READWRITE(nSequence);
156     }
157     READWRITE(vchPubKeyFrom);
158     if (!(nType & SER_GETHASH))
159         READWRITE(vchSig);
160     if (nType & SER_DISK)
161         READWRITE(nAtoms);
162 )
163
164     uint256 GetHash() const { return SerializeHash(*this); }
165     uint256 GetSigHash() const { return SerializeHash(*this, SER_GETHASH|SER_SKIPSIG); }
166     uint256 GetUserHash() const { return Hash(vchPubKeyFrom.begin(), vchPubKeyFrom.end()); }
167
168
169 bool CheckSignature();
170 bool CheckProduct();
171 };
172
173
174
175
176
177
178
179
180 extern map<uint256, CProduct> mapProducts;
181 extern CCriticalSection cs_mapProducts;
182 extern map<uint256, CProduct> mapMyProducts;
183

```

SUMMARY

This early marketplace concept predated services like OpenBazaar by years, showing Satoshi's broad vision for Bitcoin.

net.cpp

1021 lines CPP

THE GOSSIP NETWORK

Bitcoin operates as a peer-to-peer network with no central server. This file implements all networking: node discovery, connection management, message routing, and the gossip protocol that propagates transactions and blocks across the globe in seconds. At any moment, thousands of nodes are running this exact code, forming the backbone of the Bitcoin network.

Satoshi's design goal from the whitepaper's opening line: "A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution." This file makes that vision technically possible. There's no Bitcoin Inc. server—just a mesh of equal peers, each running the same rules.

The network is resilient by design. Nodes can join and leave freely. Messages propagate via gossip—each node tells its neighbors, who tell theirs. If half the network disappears tomorrow, the other half continues operating. This architecture has survived government crackdowns, DDoS attacks, and internet outages for over 15 years without a single hour of downtime.

The protocol uses a simple message format: 4-byte magic number (to identify Bitcoin network), 12-byte command name, 4-byte payload size, 4-byte checksum, then the payload. Messages include: "version" (handshake), "addr" (peer addresses), "inv" (announce data), "getdata" (request data), "tx" (transaction), "block" (block data). This simplicity enabled implementations in dozens of languages.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6 #include <winsock2.h>
7
8 void ThreadMessageHandler2(void* parg);
9 void ThreadSocketHandler2(void* parg);
10 void ThreadOpenConnections2(void* parg);
11
12
13
14
15
16
17 //
18 // Global state variables
19 //
20 bool fClient = false;
21 uint64 nLocalServices = (fClient ? 0 : NODE_NETWORK);
22 CAddress addrLocalHost(0, DEFAULT_PORT, nLocalServices);
23 CNode nodeLocalHost(INVALID_SOCKET, CAddress("127.0.0.1", nLocalServices));
24 CNode* pNodeLocalHost = &nodeLocalHost;
25 bool fShutdown = false;
26 array<bool, 10> vfThreadRunning;
27 vector<CNode*> vNodes;

```

vNodes - list of all connected peers. Bitcoin maintains 8-125 connections. Each CNode represents a live TCP connection to another node.

```

28 CCriticalSection cs_vNodes;
29 map<vector<unsigned char>, CAddress> mapAddresses;
30 CCriticalSection cs_mapAddresses;
31 map<CInv, CDataStream> mapRelay;
32 deque<pair<int64, CInv> > vRelayExpiration;
33 CCriticalSection cs_mapRelay;
34 map<CInv, int64> mapAlreadyAskedFor;
35
36
37
38 CAddress addrProxy;
39
40 bool ConnectSocket(const CAddress& addrConnect, SOCKET& hSocketRet)
41 {
42     hSocketRet = INVALID_SOCKET;
43
44     SOCKET hSocket = socket(AF_INET, SOCK_STREAM, 0);
45     if (hSocket == INVALID_SOCKET)
46         return false;
47
48     bool fRoutable = !(addrConnect.GetByte(3) == 10 || (addrConnect.GetByte(3) == 192 &&
49         addrConnect.GetByte(2) == 168));
50     bool fProxy = (addrProxy.ip && fRoutable);
51     struct sockaddr_in sockaddr = (fProxy ? addrProxy.GetSockAddr() : addrConnect.GetSockAddr());
52
53     if (connect(hSocket, (struct sockaddr*)&sockaddr, sizeof(sockaddr)) == SOCKET_ERROR)
54     {
55         closesocket(hSocket);
56         return false;
57     }
58     if (fProxy)
59     {
60         printf("Proxy connecting to %s\n", addrConnect.ToString().c_str());
61         char pszSocks4IP[] = "\4\1\0\0\0\0\0\0user";
62         memcpy(pszSocks4IP + 2, &addrConnect.port, 2);
63         memcpy(pszSocks4IP + 4, &addrConnect.ip, 4);
64         char* pszSocks4 = pszSocks4IP;

```



```

65 int nSize = sizeof(pszSocks4IP);
66
67 int ret = send(hSocket, pszSocks4, nSize, 0);
68 if (ret != nSize)
69 {
70     closesocket(hSocket);
71     return error("Error sending to proxy\n");
72 }
73 char pchRet[8];
74 if (recv(hSocket, pchRet, 8, 0) != 8)
75 {
76     closesocket(hSocket);
77     return error("Error reading proxy response\n");
78 }
79 if (pchRet[1] != 0x5a)
80 {
81     closesocket(hSocket);
82     return error("Proxy returned error %d\n", pchRet[1]);
83 }
84 printf("Proxy connection established %s\n", addrConnect.ToString().c_str());
85 }
86
87     hSocketRet = hSocket;
88     return true;
89 }
90
91
92 bool GetMyExternalIP(unsigned int& ipRet)
93 {
94     CAddress addrConnect("72.233.89.199:80"); // whatismyip.com 198-200
95
96     SOCKET hSocket;
97     if (!ConnectSocket(addrConnect, hSocket))
98     return error("GetMyExternalIP() : connection to %s failed\n", addrConnect.ToString().c_str());
99
100 char* pszGet =

```

GetMyExternalIP - discovers our public IP via external service. Early Bitcoin used whatismyip.com. Needed because NAT hides local IPs.

```

101 "GET /automation/n09230945.asp HTTP/1.1\r\n"
102 "Host: www.whatismyip.com\r\n"
103 "User-Agent: Bitcoin/0.1\r\n"
104 "Connection: close\r\n"
105 "\r\n";
106 send(hSocket, pszGet, strlen(pszGet), 0);
107
108     string strLine;
109     while (RecvLine(hSocket, strLine))
110     {
111         if (strLine.empty())
112         {
113             if (!RecvLine(hSocket, strLine))
114             {
115                 closesocket(hSocket);
116                 return false;
117             }
118             closesocket(hSocket);
119             CAddress addr(strLine.c_str());
120             printf("GetMyExternalIP() received [%s] %s\n", strLine.c_str(), addr.ToString().c_str());
121             if (addr.ip == 0)
122                 return false;
123             ipRet = addr.ip;
124             return true;
125         }
126     }
127     closesocket(hSocket);
128     return error("GetMyExternalIP() : connection closed\n");
129 }

```

```

130
131
132
133
134
135
136 bool AddAddress(CAddrDB& addrdb, const CAddress& addr)
137 {
138     if (!addr.IsRoutable())
139         return false;
140     if (addr.ip == addrLocalHost.ip)
141         return false;
142     CRITICAL_BLOCK(cs_mapAddresses)
143     {
144         map<vector<unsigned char>, CAddress>::iterator it = mapAddresses.find(addr.GetKey());
145         if (it == mapAddresses.end())
146         {
147             // New address
148             mapAddresses.insert(make_pair(addr.GetKey(), addr));
149             addrdb.WriteAddress(addr);
150             return true;
151         }
152         else
153         {
154             CAddress& addrFound = (*it).second;
155             if ((addrFound.nServices | addr.nServices) != addrFound.nServices)
156             {
157                 // Services have been added
158                 addrFound.nServices |= addr.nServices;
159                 addrdb.WriteAddress(addrFound);
160                 return true;
161             }
162         }
163     }
164     return false;
165 }
166
167
168
169
170
171 void AbandonRequests(void (*fn)(void*, CDataStream&), void* param1)
172 {
173     // If the dialog might get closed before the reply comes back,
174     // call this in the destructor so it doesn't get called after it's deleted.
175     CRITICAL_BLOCK(cs_vNodes)
176     {
177         foreach(CNode* pnode, vNodes)
178         {
179             CRITICAL_BLOCK(pnode->cs_mapRequests)
180             {
181                 for (map<uint256, CRequestTracker>::iterator mi = pnode->mapRequests.begin(); mi != pnode->mapRequests.end(); )
182                 {
183                     CRequestTracker& tracker = (*mi).second;
184                     if (tracker.fn == fn && tracker.param1 == param1)
185                         pnode->mapRequests.erase(mi++);
186                     else
187                         mi++;
188                 }
189             }
190         }
191     }
192 }
193
194
195
196
197
198
199
200 //

```

```

201 // Subscription methods for the broadcast and subscription system.
202 // Channel numbers are message numbers, i.e. MSG_TABLE and MSG_PRODUCT.
203 //
204 // The subscription system uses a meet-in-the-middle strategy.
205 // With 100,000 nodes, if senders broadcast to 1000 random nodes and receivers
206 // subscribe to 1000 random nodes, 99.995% (1 - 0.99^1000) of messages will get through.
207 //
208
209 bool AnySubscribed(unsigned int nChannel)
210 {
211     if (pnodeLocalHost->IsSubscribed(nChannel))
212         return true;
213     CRITICAL_BLOCK(cs_vNodes)
214     foreach(CNode* pnode, vNodes)
215         if (pnode->IsSubscribed(nChannel))
216             return true;
217     return false;
218 }
219
220 bool CNode::IsSubscribed(unsigned int nChannel)
221 {
222     if (nChannel >= vfSubscribe.size())
223         return false;
224     return vfSubscribe[nChannel];
225 }
226
227 void CNode::Subscribe(unsigned int nChannel, unsigned int nHops)
228 {
229     if (nChannel >= vfSubscribe.size())
230         return;
231
232     if (!AnySubscribed(nChannel))
233     {
234         // Relay subscribe
235         CRITICAL_BLOCK(cs_vNodes)
236         foreach(CNode* pnode, vNodes)
237             if (pnode != this)
238                 pnode->PushMessage("subscribe", nChannel, nHops);
239     }
240
241     vfSubscribe[nChannel] = true;
242 }
243
244 void CNode::CancelSubscribe(unsigned int nChannel)
245 {
246     if (nChannel >= vfSubscribe.size())
247         return;
248
249     // Prevent from relaying cancel if wasn't subscribed
250     if (!vfSubscribe[nChannel])
251         return;
252     vfSubscribe[nChannel] = false;
253
254     if (!AnySubscribed(nChannel))
255     {
256         // Relay subscription cancel
257         CRITICAL_BLOCK(cs_vNodes)
258         foreach(CNode* pnode, vNodes)
259             if (pnode != this)
260                 pnode->PushMessage("sub-cancel", nChannel);
261     }
262
263     // Clear memory, no longer subscribed
264     if (nChannel == MSG_PRODUCT)
265         CRITICAL_BLOCK(cs_mapProducts)
266             mapProducts.clear();
267 }
268
269
270
271
272

```

```

273
274
275
276
277 CNode* FindNode(unsigned int ip)
278 {
279     CRITICAL_BLOCK(cs_vNodes)
280     {
281         foreach(CNode* pnode, vNodes)
282             if (pnode->addr.ip == ip)
283                 return (pnode);
284     }
285     return NULL;
286 }
287
288 CNode* FindNode(CAddress addr)
289 {
290     CRITICAL_BLOCK(cs_vNodes)
291     {
292         foreach(CNode* pnode, vNodes)
293             if (pnode->addr == addr)
294                 return (pnode);
295     }
296     return NULL;
297 }
298
299 CNode* ConnectNode(CAddress addrConnect, int64 nTimeout)

```

ConnectNode - establishes TCP connection to peer. Performs version handshake, exchanges capabilities. The start of every peer relationship.

```

300 {
301     if (addrConnect.ip == addrLocalHost.ip)
302         return NULL;
303
304     // Look for an existing connection
305     CNode* pnode = FindNode(addrConnect.ip);
306     if (pnode)
307     {
308         if (nTimeout != 0)
309             pnode->AddRef(nTimeout);
310         else
311             pnode->AddRef();
312         return pnode;
313     }
314
315     // debug print
316     printf("trying %s\n", addrConnect.ToString().c_str());
317
318     // Connect
319     SOCKET hSocket;
320     if (ConnectSocket(addrConnect, hSocket))
321     {
322         // debug print
323         printf("connected %s\n", addrConnect.ToString().c_str());
324
325         // Add node
326         CNode* pnode = new CNode(hSocket, addrConnect, false);
327         if (nTimeout != 0)
328             pnode->AddRef(nTimeout);
329         else
330             pnode->AddRef();
331         CRITICAL_BLOCK(cs_vNodes)
332             vNodes.push_back(pnode);
333
334         CRITICAL_BLOCK(cs_mapAddresses)
335             mapAddresses[addrConnect.GetKey()].nLastFailed = 0;
336         return pnode;
337     }

```

```

338 else
339 {
340     CRITICAL_BLOCK(cs_mapAddresses)
341     mapAddresses[addrConnect.GetKey()].nLastFailed = GetTime();
342     return NULL;
343 }
344 }
345
346 void CNode::Disconnect()
347 {
348     printf("disconnecting node %s\n", addr.ToString().c_str());
349
350     closesocket(hSocket);
351
352     // All of a nodes broadcasts and subscriptions are automatically torn down
353     // when it goes down, so a node has to stay up to keep its broadcast going.
354
355     CRITICAL_BLOCK(cs_mapProducts)
356     for (map<uint256, CProduct>::iterator mi = mapProducts.begin(); mi != mapProducts.end(); mi++)
357         AdvertRemoveSource(this, MSG_PRODUCT, 0, (*(mi++)).second);
358
359     // Cancel subscriptions
360     for (unsigned int nChannel = 0; nChannel < vfSubscribe.size(); nChannel++)
361         if (vfSubscribe[nChannel])
362             CancelSubscribe(nChannel);
363 }
364
365
366
367
368
369
370
371
372
373
374
375
376
377 void ThreadSocketHandler(void* parg)
378 {
379     IMPLEMENT_RANDOMIZE_STACK(ThreadSocketHandler(parg));
380
381     loop
382     {
383         vfThreadRunning[0] = true;
384         CheckForShutdown(0);
385         try
386         {
387             ThreadSocketHandler2(parg);
388         }
389         CATCH_PRINT_EXCEPTION("ThreadSocketHandler()")
390         vfThreadRunning[0] = false;
391         Sleep(5000);
392     }
393 }
394
395 void ThreadSocketHandler2(void* parg)
396 {
397     printf("ThreadSocketHandler started\n");
398     SOCKET hListenSocket = *(SOCKET*)parg;
399     list<CNode*> vNodesDisconnected;
400     int nPrevNodeCount = 0;
401
402     loop
403     {
404         //
405         // Disconnect nodes
406         //
407         CRITICAL_BLOCK(cs_vNodes)
408         {
409             // Disconnect duplicate connections

```

```

410         map<unsigned int, CNode*> mapFirst;
411     foreach(CNode* pNode, vNodes)
412     {
413         if (pNode->fDisconnect)
414             continue;
415         unsigned int ip = pNode->addr.ip;
416         if (mapFirst.count(ip) && addrLocalHost.ip < ip)
417         {
418             // In case two nodes connect to each other at once,
419             // the lower ip disconnects its outbound connection
420             CNode* pNodeExtra = mapFirst[ip];
421
422             if (pNodeExtra->GetRefCount() > (pNodeExtra->fNetworkNode ? 1 : 0))
423                 swap(pNodeExtra, pNode);
424
425             if (pNodeExtra->GetRefCount() <= (pNodeExtra->fNetworkNode ? 1 : 0))
426             {
427                 printf("(%d nodes) disconnecting duplicate: %s\n", vNodes.size(), pNodeExtra->addr.ToString().c_str());
428                 if (pNodeExtra->fNetworkNode && !pNode->fNetworkNode)
429                 {
430                     pNode->AddRef();
431                     swap(pNodeExtra->fNetworkNode, pNode->fNetworkNode);
432                     pNodeExtra->Release();
433                 }
434
435                 pNodeExtra->fDisconnect = true;
436             }
437
438             mapFirst[ip] = pNode;
439         }
440         // Disconnect unused nodes
441         vector<CNode*> vNodesCopy = vNodes;
442         foreach(CNode* pNode, vNodesCopy)
443         {
444             if (pNode->ReadyToDisconnect() && pNode->vRecv.empty() && pNode->vSend.empty())
445             {
446                 // remove from vNodes
447                 vNodes.erase(remove(vNodes.begin(), vNodes.end(), pNode), vNodes.end());
448                 pNode->Disconnect();
449             }
450             // hold in disconnected pool until all refs are released
451             pNode->nReleaseTime = max(pNode->nReleaseTime, GetTime() + 5 * 60);
452             if (pNode->fNetworkNode)
453                 pNode->Release();
454             vNodesDisconnected.push_back(pNode);
455         }
456     }
457
458     // Delete disconnected nodes
459     list<CNode*> vNodesDisconnectedCopy = vNodesDisconnected;
460     foreach(CNode* pNode, vNodesDisconnectedCopy)
461     {
462         // wait until threads are done using it
463         if (pNode->GetRefCount() <= 0)
464         {
465             bool fDelete = false;
466             TRY_CRITICAL_BLOCK(pNode->cs_vSend)
467             TRY_CRITICAL_BLOCK(pNode->cs_vRecv)
468             TRY_CRITICAL_BLOCK(pNode->cs_mapRequests)
469             TRY_CRITICAL_BLOCK(pNode->cs_inventory)
470             fDelete = true;
471             if (fDelete)
472             {
473                 vNodesDisconnected.remove(pNode);
474                 delete pNode;
475             }
476         }
477     }
478
479     if (vNodes.size() != nPrevNodeCount)
480     {
481         nPrevNodeCount = vNodes.size();

```

```

482 MainFrameRepaint();
483 }
484
485
486 //
487 // Find which sockets have data to receive
488 //
489 struct timeval timeout;
490     timeout.tv_sec = 0;
491     timeout.tv_usec = 50000; // frequency to poll pnode->vSend
492
493 struct fd_set fdsetRecv;
494 struct fd_set fdsetSend;
495 FD_ZERO(&fdsetRecv);
496 FD_ZERO(&fdsetSend);
497     SOCKET hSocketMax = 0;
498 FD_SET(hListenSocket, &fdsetRecv);
499     hSocketMax = max(hSocketMax, hListenSocket);
500 CRITICAL_BLOCK(cs_vNodes)

```

Socket select loop - the network heartbeat. Uses select() to efficiently wait on multiple sockets. Classic Unix network programming.

```

501 {
502     foreach(CNode* pnode, vNodes)
503     {
504         FD_SET(pnode->hSocket, &fdsetRecv);
505         hSocketMax = max(hSocketMax, pnode->hSocket);
506         TRY_CRITICAL_BLOCK(pnode->cs_vSend)
507         if (!pnode->vSend.empty())
508             FD_SET(pnode->hSocket, &fdsetSend);
509     }
510 }
511
512     vfThreadRunning[0] = false;
513 int nSelect = select(hSocketMax + 1, &fdsetRecv, &fdsetSend, NULL, &timeout);
514     vfThreadRunning[0] = true;
515 CheckForShutdown(0);
516 if (nSelect == SOCKET_ERROR)
517 {
518     int nErr = WSAGetLastError();
519     printf("select failed: %d\n", nErr);
520     for (int i = 0; i <= hSocketMax; i++)
521     {
522         FD_SET(i, &fdsetRecv);
523         FD_SET(i, &fdsetSend);
524     }
525     Sleep(timeout.tv_usec/1000);
526 }
527 RandAddSeed();
528
529 /// debug print
530 //foreach(CNode* pnode, vNodes)
531 //{
532 //     printf("vRecv = %5d ", pnode->vRecv.size());
533 //     printf("vSend = %5d ", pnode->vSend.size());
534 //}
535 //printf("\n");
536
537
538 //
539 // Accept new connections
540 //
541 if (FD_ISSET(hListenSocket, &fdsetRecv))
542 {
543     struct sockaddr_in sockaddr;
544     int len = sizeof(sockaddr);
545     SOCKET hSocket = accept(hListenSocket, (struct sockaddr*)&sockaddr, &len);
546     CAddress addr(sockaddr);

```

```

547 if (hSocket == INVALID_SOCKET)
548 {
549     if (WSAGetLastError() != WSAEWOULDBLOCK)
550         printf("ERROR ThreadSocketHandler accept failed: %d\n", WSAGetLastError());
551 }
552 else
553 {
554     printf("accepted connection from %s\n", addr.ToString().c_str());
555     CNode* pnode = new CNode(hSocket, addr, true);
556     pnode->AddRef();
557     CRITICAL_BLOCK(cs_vNodes)
558         vNodes.push_back(pnode);
559 }
560 }
561
562
563 //
564 // Service each socket
565 //
566     vector<CNode*> vNodesCopy;
567     CRITICAL_BLOCK(cs_vNodes)
568         vNodesCopy = vNodes;
569     foreach(CNode* pnode, vNodesCopy)
570     {
571         CheckForShutdown(0);
572         SOCKET hSocket = pnode->hSocket;
573
574         //
575         // Receive
576         //
577         if (FD_ISSET(hSocket, &fdsetRecv))
578         {
579             TRY_CRITICAL_BLOCK(pnode->cs_vRecv)
580             {
581                 CDataStream& vRecv = pnode->vRecv;
582                 unsigned int nPos = vRecv.size();
583
584                 // typical socket buffer is 8K-64K
585                 const unsigned int nBufSize = 0x10000;
586                 vRecv.resize(nPos + nBufSize);
587                 int nBytes = recv(hSocket, &vRecv[nPos], nBufSize, 0);
588                 vRecv.resize(nPos + max(nBytes, 0));
589                 if (nBytes == 0)
590                 {
591                     // socket closed gracefully
592                     if (!pnode->fDisconnect)
593                         printf("recv: socket closed\n");
594                     pnode->fDisconnect = true;
595                 }
596                 else if (nBytes < 0)
597                 {
598                     // socket error
599                     int nErr = WSAGetLastError();
600                     if (nErr != WSAEWOULDBLOCK && nErr != WSAEMSGSIZE && nErr != WSAEINTR && nErr != WSAEINPROGRESS)
601                     {
602                         if (!pnode->fDisconnect)
603                             printf("recv failed: %d\n", nErr);
604                         pnode->fDisconnect = true;
605                     }
606                 }
607             }
608         }
609
610         //
611         // Send
612         //
613         if (FD_ISSET(hSocket, &fdsetSend))
614         {
615             TRY_CRITICAL_BLOCK(pnode->cs_vSend)
616             {
617                 CDataStream& vSend = pnode->vSend;
618                 if (!vSend.empty())

```



```

619 {
620     int nBytes = send(hSocket, &vSend[0], vSend.size(), 0);
621     if (nBytes > 0)
622     {
623         vSend.erase(vSend.begin(), vSend.begin() + nBytes);
624     }
625     else if (nBytes == 0)
626     {
627         if (pnode->ReadyToDisconnect())
628             pnode->vSend.clear();
629     }
630     else
631     {
632         printf("send error %d\n", nBytes);
633         if (pnode->ReadyToDisconnect())
634             pnode->vSend.clear();
635     }
636 }
637 }
638 }
639 }
640
641
642 Sleep(10);
643 }
644 }
645
646
647
648
649
650
651
652
653
654
655 void ThreadOpenConnections(void* parg)
656 {
657     IMPLEMENT_RANDOMIZE_STACK(ThreadOpenConnections(parg));
658
659     loop
660     {
661         vfThreadRunning[1] = true;
662         CheckForShutdown(1);
663         try
664         {
665             ThreadOpenConnections2(parg);
666         }
667         CATCH_PRINT_EXCEPTION("ThreadOpenConnections()")
668         vfThreadRunning[1] = false;
669         Sleep(5000);
670     }
671 }
672
673 void ThreadOpenConnections2(void* parg)

```

ThreadOpenConnections - actively seeks new peers. Balances between maintaining connections and not overloading the network.

```

674 {
675     printf("ThreadOpenConnections started\n");
676
677     // Initiate network connections
678     const int nMaxConnections = 15;
679     loop
680     {
681         // Wait
682         vfThreadRunning[1] = false;
683         Sleep(500);

```

```

684 while (vNodes.size() >= nMaxConnections || vNodes.size() >= mapAddresses.size())
685 {
686     CheckForShutdown(1);
687     Sleep(2000);
688 }
689     vfThreadRunning[1] = true;
690 CheckForShutdown(1);
691
692
693 // Make a list of unique class C's
694 unsigned char pchIPMask[4] = { 0xff, 0xff, 0xff, 0x00 };
695 unsigned int nIPMask = *(unsigned int*)pchIPMask;
696     vector<unsigned int> vIPC;
697 CRITICAL_BLOCK(cs_mapAddresses)
698 {
699     vIPC.reserve(mapAddresses.size());
700 unsigned int nPrev = 0;
701 foreach(const PAIRTYPE(vector<unsigned char>, CAddress)& item, mapAddresses)
702 {
703     const CAddress& addr = item.second;
704     if (!addr.IsIPv4())
705         continue;
706
707 // Taking advantage of mapAddresses being in sorted order,
708 // with IPs of the same class C grouped together.
709 unsigned int ipC = addr.ip & nIPMask;
710 if (ipC != nPrev)
711     vIPC.push_back(nPrev = ipC);
712 }
713 }
714
715 //
716 // The IP selection process is designed to limit vulnerability to address flooding.
717 // Any class C (a.b.c.?) has an equal chance of being chosen, then an IP is
718 // chosen within the class C. An attacker may be able to allocate many IPs, but
719 // they would normally be concentrated in blocks of class C's. They can hog the
720 // attention within their class C, but not the whole IP address space overall.
721 // A lone node in a class C will get as much attention as someone holding all 255
722 // IPs in another class C.
723 //
724 bool fSuccess = false;
725 int nLimit = vIPC.size();
726 while (!fSuccess && nLimit-- > 0)
727 {
728 // Choose a random class C
729 unsigned int ipC = vIPC[GetRand(vIPC.size())];
730
731 // Organize all addresses in the class C by IP
732 map<unsigned int, vector<CAddress> > mapIP;
733 CRITICAL_BLOCK(cs_mapAddresses)
734 {
735     unsigned int nDelay = ((30 * 60) << vNodes.size());
736     if (nDelay > 8 * 60 * 60)
737         nDelay = 8 * 60 * 60;
738     for (map<vector<unsigned char>, CAddress>::iterator mi = mapAddresses.lower_bound(CAddress(ipC,
739 0).GetKey());
740         mi != mapAddresses.upper_bound(CAddress(ipC | ~nIPMask, 0xffff).GetKey());
741         ++mi)
742     {
743         const CAddress& addr = (*mi).second;
744         unsigned int nRandomizer = (addr.nLastFailed * addr.ip * 7777U) % 20000;
745         if (GetTime() - addr.nLastFailed > nDelay * nRandomizer / 10000)
746             mapIP[addr.ip].push_back(addr);
747     }
748     if (mapIP.empty())
749         break;
750
751 // Choose a random IP in the class C
752     map<unsigned int, vector<CAddress> >::iterator mi = mapIP.begin();
753     advance(mi, GetRand(mapIP.size()));
754

```

```

755 // Once we've chosen an IP, we'll try every given port before moving on
756 foreach(const CAddress& addrConnect, (*mi).second)
757 {
758     if (addrConnect.ip == addrLocalHost.ip || !addrConnect.IsIPv4() || FindNode(addrConnect.ip))
759         continue;
760
761     CNode* pnode = ConnectNode(addrConnect);
762     if (!pnode)
763         continue;
764     pnode->fNetworkNode = true;
765
766     if (addrLocalHost.IsRoutable())
767     {
768         // Advertise our address
769         vector<CAddress> vAddrToSend;
770         vAddrToSend.push_back(addrLocalHost);
771         pnode->PushMessage("addr", vAddrToSend);
772     }
773
774     // Get as many addresses as we can
775     pnode->PushMessage("getaddr");
776
777     // should the one on the receiving end do this too?
778     // Subscribe our local subscription list
779     const unsigned int nHops = 0;
780     for (unsigned int nChannel = 0; nChannel < pnodeLocalHost->vfSubscribe.size(); nChannel++)
781     if (pnodeLocalHost->vfSubscribe[nChannel])
782         pnode->PushMessage("subscribe", nChannel, nHops);
783
784     fSuccess = true;
785     break;
786 }
787 }
788 }
789 }
790
791
792
793
794
795
796
797
798 void ThreadMessageHandler(void* parg)

```

ThreadMessageHandler - processes incoming messages. Blocks, transactions, addresses, pings flow through here. The protocol interpreter.

```

799 {
800     IMPLEMENT_RANDOMIZE_STACK(ThreadMessageHandler(parg));
801
802     loop
803     {
804         vfThreadRunning[2] = true;
805         CheckForShutdown(2);
806         try
807         {
808             ThreadMessageHandler2(parg);
809         }
810         CATCH_PRINT_EXCEPTION("ThreadMessageHandler()")
811         vfThreadRunning[2] = false;
812         Sleep(5000);
813     }
814 }
815
816 void ThreadMessageHandler2(void* parg)
817 {
818     printf("ThreadMessageHandler started\n");
819     SetThreadPriority(GetCurrentThread(), THREAD_PRIORITY_BELOW_NORMAL);

```

```

820     loop
821 {
822 // Poll the connected nodes for messages
823     vector<CNode*> vNodesCopy;
824     CRITICAL_BLOCK(cs_vNodes)
825         vNodesCopy = vNodes;
826     foreach(CNode* pnode, vNodesCopy)
827     {
828         pnode->AddRef();
829     }
830 // Receive messages
831     TRY_CRITICAL_BLOCK(pnode->cs_vRecv)
832     ProcessMessages(pnode);

```

ProcessMessages - deserializes and dispatches messages. Each message type (version, addr, inv, getdata, tx, block) has a handler.

```

833
834 // Send messages
835     TRY_CRITICAL_BLOCK(pnode->cs_vSend)
836     SendMessages(pnode);
837
838     pnode->Release();
839 }
840
841 // Wait and allow messages to bunch up
842     vfThreadRunning[2] = false;
843     Sleep(100);
844     vfThreadRunning[2] = true;
845     CheckForShutdown(2);
846 }
847 }
848
849
850
851
852
853
854
855
856
857 //// todo: start one thread per processor, use getenv("NUMBER_OF_PROCESSORS")
858 void ThreadBitcoinMiner(void* parg)
859 {
860     vfThreadRunning[3] = true;
861     CheckForShutdown(3);
862     try
863     {
864         bool fRet = BitcoinMiner();
865         printf("BitcoinMiner returned %s\n\n", fRet ? "true" : "false");
866     }
867     CATCH_PRINT_EXCEPTION("BitcoinMiner()")
868     vfThreadRunning[3] = false;
869 }
870
871
872
873
874
875
876
877
878
879
880
881 bool StartNode(string& strError)
882 {
883     strError = "";
884

```

```

886 // Sockets startup
887 WSADATA wsadata;
888 int ret = WSAStartup(MAKEWORD(2,2), &wsadata);
889 if (ret != NO_ERROR)
890 {
891     strError = sprintf("Error: TCP/IP socket library failed to start (WSAStartup returned error
892 %d)", ret);
893     printf("%s\n", strError.c_str());
894     return false;
895 }
896 // Get local host ip
897 char pszHostName[255];
898 if (gethostname(pszHostName, 255) == SOCKET_ERROR)
899 {
900     strError = sprintf("Error: Unable to get IP address of this computer (gethostname returned
901 error %d)", WSAGetLastError());
902     printf("%s\n", strError.c_str());
903     return false;
904 }
905 struct hostent* pHostEnt = gethostbyname(pszHostName);
906 if (!pHostEnt)
907 {
908     strError = sprintf("Error: Unable to get IP address of this computer (gethostbyname returned
909 error %d)", WSAGetLastError());
910     printf("%s\n", strError.c_str());
911     return false;
912 }
913 struct sockaddr_in addrLocalHost = {
914     .family = AF_INET,
915     .port = htons(DEFAULT_PORT),
916     .addr = *(long*)(pHostEnt->h_addr_list[0])
917 };
918 printf("addrLocalHost = %s\n", addrLocalHost.ToString().c_str());
919 // Create socket for listening for incoming connections
920 SOCKET hListenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
921 if (hListenSocket == INVALID_SOCKET)
922 {
923     strError = sprintf("Error: Couldn't open socket for incoming connections (socket returned
924 error %d)", WSAGetLastError());
925     printf("%s\n", strError.c_str());
926     return false;
927 }
928 // Set to nonblocking, incoming connections will also inherit this
929 u_long nOne = 1;
930 if (ioctlsocket(hListenSocket, FIONBIO, &nOne) == SOCKET_ERROR)
931 {
932     strError = sprintf("Error: Couldn't set properties on socket for incoming connections
933 (ioctlsocket returned error %d)", WSAGetLastError());
934     printf("%s\n", strError.c_str());
935     return false;
936 }
937 // The sockaddr_in structure specifies the address family,
938 // IP address, and port for the socket that is being bound
939 int nRetryLimit = 15;
940 struct sockaddr_in sockaddr = addrLocalHost.GetSockAddr();
941 if (bind(hListenSocket, (struct sockaddr*)&sockaddr, sizeof(sockaddr)) == SOCKET_ERROR)
942 {
943     int nErr = WSAGetLastError();
944     if (nErr == WSAEADDRINUSE)
945     {
946         strError = sprintf("Error: Unable to bind to port %s on this computer. The program is
947 probably already running.", addrLocalHost.ToString().c_str());
948     }
949     else
950     {
951         strError = sprintf("Error: Unable to bind to port %s on this computer (bind returned error
952 %d)", addrLocalHost.ToString().c_str(), nErr);
953     }
954     printf("%s\n", strError.c_str());
955     return false;
956 }
957 printf("bound to addrLocalHost = %s\n\n", addrLocalHost.ToString().c_str());
958 // Listen for incoming connections

```

```

950 if (listen(hListenSocket, SOMAXCONN) == SOCKET_ERROR)
951 {
952     strError = sprintf("Error: Listening for incoming connections failed (listen returned error
953 %d)", WSAGetLastError());
954 printf("%s\n", strError.c_str());
955 return false;
956 }
957 // Get our external IP address for incoming connections
958 if (addrIncoming.ip)
959     addrLocalHost.ip = addrIncoming.ip;
960
961 if (GetMyExternalIP(addrLocalHost.ip))
962 {
963     addrIncoming = addrLocalHost;
964 CWalletDB().WriteSetting("addrIncoming", addrIncoming);
965 }
966
967 // Get addresses from IRC and advertise ours
968 if (_beginthread(ThreadIRCSeed, 0, NULL) == -1)
969 printf("Error: _beginthread(ThreadIRCSeed) failed\n");
970
971 //
972 // Start threads
973 //
974 if (_beginthread(ThreadSocketHandler, 0, new SOCKET(hListenSocket)) == -1)
975 {
976     strError = "Error: _beginthread(ThreadSocketHandler) failed";
977 printf("%s\n", strError.c_str());
978 return false;
979 }
980
981 if (_beginthread(ThreadOpenConnections, 0, NULL) == -1)

```

Thread startup - Bitcoin spawns multiple networking threads. OpenConnections finds peers, MessageHandler processes data, SocketHandler does I/O.

```

982 {
983     strError = "Error: _beginthread(ThreadOpenConnections) failed";
984 printf("%s\n", strError.c_str());
985 return false;
986 }
987
988 if (_beginthread(ThreadMessageHandler, 0, NULL) == -1)
989 {
990     strError = "Error: _beginthread(ThreadMessageHandler) failed";
991 printf("%s\n", strError.c_str());
992 return false;
993 }
994
995 return true;
996 }
997
998 bool StopNode()
999 {
1000 printf("StopNode()\n");
1001     fShutdown = true;
1002     nTransactionsUpdated++;
1003     while (count(vfThreadRunning.begin(), vfThreadRunning.end(), true))
1004         Sleep(10);
1005     Sleep(50);
1006
1007 // Sockets shutdown
1008 WSACleanup();
1009 return true;
1010 }
1011
1012 void CheckForShutdown(int n)
1013 {

```

```
1014 if (fShutdown)
1015 {
1016 if (n != -1)
1017     vfThreadRunning[n] = false;
1018 _endthread();
1019 }
1020 }
1021
```

SUMMARY

The networking layer embodies Satoshi's trustless philosophy. Nodes don't trust peers—they validate everything independently. A malicious peer can waste your bandwidth but can't steal your coins or corrupt your view of the blockchain.

Message propagation uses "inv" (inventory) announcements: nodes advertise what they have, peers request what they need. This pull-based model prevents spam and lets nodes prioritize important data (blocks over transactions).

net.h

857 lines CPP

PEERS WITHOUT TRUST

This header defines the network message types and node connection structures that form Bitcoin's peer-to-peer protocol. Every message that Bitcoin nodes exchange—blocks, transactions, peer addresses, version handshakes—is defined here. These definitions are the "language" that Bitcoin nodes speak.

The key abstraction is the `CNode` class: a single connection to another Bitcoin node. Each `CNode` maintains send/receive buffers, tracks what data the peer has (to avoid redundant transfers), and handles the message serialization. A typical node maintains 8-125 simultaneous `CNode` connections.

Satoshi designed the protocol with minimal trust: nodes don't believe what peers tell them—they verify independently. A peer claiming to have a valid block must prove it. This skepticism is why Bitcoin survives in adversarial environments where some participants actively try to cheat.

The message types defined here form a request-response pattern: nodes announce what they have ("inv"), peers request what they want ("getdata"), and data flows in response ("tx", "block"). This pull-based model prevents spam—you only download data you explicitly request.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 class CMessageHeader;
6 class CAddress;
7 class CInv;
8 class CRequestTracker;
9 class CNode;
10
11
12
13 static const unsigned short DEFAULT_PORT = htons(8333);

```

`DEFAULT_PORT = 8333`—Bitcoin's well-known port number. Satoshi chose 8333 with no documented rationale—it wasn't assigned by IANA for any other purpose. The `htons()` call converts to network byte order (big-endian). Every Bitcoin node listens on this port; firewalls must allow it for full participation. Some speculate 8333 was chosen because $8+3+3+3 = 17$, but this is probably coincidental.

```

14 static const unsigned int PUBLISH_HOPS = 5;
15 enum
16 {
17     NODE_NETWORK = (1 << 0),

```

`NODE_NETWORK`—service flag indicating a full node. In v0.1, this was the only flag. Later versions added `NODE_BLOOM` (supports bloom filters), `NODE_WITNESS` (supports SegWit), etc. When connecting, nodes exchange service flags to understand each other's capabilities. A node advertising `NODE_NETWORK` commits to serving historical blocks on request.

```

18 };
19
20
21
22
23
24
25 bool ConnectSocket(const CAddress& addrConnect, SOCKET& hSocketRet);
26 bool GetMyExternalIP(unsigned int& ipRet);
27 bool AddAddress(CAddrDB& addrdb, const CAddress& addr);
28 CNode* FindNode(unsigned int ip);
29 CNode* ConnectNode(CAddress addrConnect, int64 nTimeout=0);
30 void AbandonRequests(void (*fn)(void*, CDataStream&), void* param1);
31 bool AnySubscribed(unsigned int nChannel);
32 void ThreadBitcoinMiner(void* parg);
33 bool StartNode(string& strError=REF(string()));
34 bool StopNode();
35 void CheckForShutdown(int n);
36
37
38
39
40
41
42
43
44
45 //
46 // Message header
47 // (4) message start
48 // (12) command
49 // (4) size
50
51 // The message start string is designed to be unlikely to occur in normal data.
52 // The characters are rarely used upper ascii, not valid as UTF-8, and produce

```

```

53 // a large 4-byte int at any alignment.
54 static const char pchMessageStart[4] = { 0xf9, 0xbe, 0xb4, 0xd9 };
    
```

`pchMessageStart` —the 4-byte 'magic number' that begins every Bitcoin message: 0xf9beb4d9 . Satoshi's comment explains: 'designed to be unlikely to occur in normal data.' These bytes are invalid UTF-8, rare in binary files, and form a large 32-bit integer at any alignment. When scanning a data stream, finding this sequence likely indicates a Bitcoin message boundary. Testnet uses different magic bytes to prevent cross-network contamination.

```

55
56 class CMessageHeader
    
```

`CMessageHeader` —the 20-byte header prepended to every network message. Structure: 4-byte magic, 12-byte command (null-padded), 4-byte payload size. Later versions added a 4-byte checksum after the size. The fixed-size command field allows efficient parsing—you can read exactly 20 bytes, validate the header, then read exactly `nMessageSize` more bytes.

```

57 {
58 public:
59 enum { COMMAND_SIZE=12 };
    
```

`COMMAND_SIZE=12` —maximum command name length. Commands like 'version', 'verack', 'inv', 'getdata', 'block', 'tx' easily fit. The padding with zeros allows fixed-size parsing. Longer command names would require protocol changes—but 12 characters has proven sufficient for all Bitcoin message types.

```

60 char pchMessageStart[sizeof(::pchMessageStart)];
61 char pchCommand[COMMAND_SIZE];
62 unsigned int nMessageSize;
63
64 CMessageHeader()
65 {
66 memcpy(pchMessageStart, ::pchMessageStart, sizeof(pchMessageStart));
67 memset(pchCommand, 0, sizeof(pchCommand));
68     pchCommand[1] = 1;
69     nMessageSize = -1;
70 }
71
72 CMessageHeader(const char* pszCommand, unsigned int nMessageSizeIn)
73 {
74 memcpy(pchMessageStart, ::pchMessageStart, sizeof(pchMessageStart));
75 strncpy(pchCommand, pszCommand, COMMAND_SIZE);
76     nMessageSize = nMessageSizeIn;
77 }
78
79 IMPLEMENT_SERIALIZE
80 (
81 READWRITE(FLATDATA(pchMessageStart));
82 READWRITE(FLATDATA(pchCommand));
83 READWRITE(nMessageSize);
84 )
85
86 string GetCommand()
87 {
88 if (pchCommand[COMMAND_SIZE-1] == 0)
89 return string(pchCommand, pchCommand + strlen(pchCommand));
90 else
91 return string(pchCommand, pchCommand + COMMAND_SIZE);
92 }
93
94 bool IsValid()
    
```

`IsValid()` —validates message header before processing payload. Checks: (1) magic bytes match (rejects non-Bitcoin data), (2) command contains only printable ASCII then zeros (rejects malformed commands), (3) message size is reasonable (rejects memory exhaustion attacks). The `0x10000000` (256MB) limit was generous for 2009—modern Bitcoin has stricter limits.

```

95 {
96 // Check start string
97 if (memcmp(pchMessageStart, ::pchMessageStart, sizeof(pchMessageStart)) != 0)
98 return false;
99
100 // Check the command string for errors
101 for (char* p1 = pchCommand; p1 < pchCommand + COMMAND_SIZE; p1++)
102 {
103 if (*p1 == 0)
104 {
105 // Must be all zeros after the first zero
106 for (; p1 < pchCommand + COMMAND_SIZE; p1++)
107 if (*p1 != 0)
108 return false;
109 }
110 else if (*p1 < ' ' || *p1 > 0x7E)
111 return false;
112 }
113
114 // Message size
115 if (nMessageSize > 0x10000000)
116 {
117 printf("CMessageHeader::IsValid() : nMessageSize too large %u\n", nMessageSize);
118 return false;
119 }
120
121 return true;
122 }
123 };
124
125
126
127
128
129
130 static const unsigned char pchIPv4[12] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0xff, 0xff };

```

`pchIPv4` —IPv4-mapped IPv6 prefix. Bitcoin's address format reserves 16 bytes for IP addresses to support IPv6. IPv4 addresses are stored with this 12-byte prefix followed by the 4-byte IPv4 address. This forward-thinking design meant Bitcoin didn't need protocol changes when IPv6 adoption grew.

```

131
132 class CAddress

```

`CAddress` —represents a network peer's address. Contains: services (capabilities bitmap), IP address (16 bytes for IPv6 compatibility), port, timestamp (when last seen), and `nLastFailed` (connection attempt tracking). This structure is shared between nodes during 'addr' messages to propagate peer discovery information. The timestamp helps nodes prefer recently-active peers.

```

133 {
134 public:
135     uint64 nServices;
136     unsigned char pchReserved[12];
137     unsigned int ip;
138     unsigned short port;
139 }

```

```

140 // disk only
141 unsigned int nTime;

```

`nTime` —timestamp when this address was last known to be active. Only serialized to disk (SER_DISK flag), not sent over the network in v0.1. Later versions include timestamps in 'addr' messages to help nodes prioritize fresh addresses. Addresses with old timestamps are less likely to be reachable.

```

142
143 // memory only
144 unsigned int nLastFailed;
145
146 CAddress()
147 {
148     nServices = 0;
149     memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
150     ip = 0;
151     port = DEFAULT_PORT;
152     nTime = GetAdjustedTime();
153     nLastFailed = 0;
154 }
155
156 CAddress(unsigned int ipIn, unsigned short portIn, uint64 nServicesIn=0)
157 {
158     nServices = nServicesIn;
159     memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
160     ip = ipIn;
161     port = portIn;
162     nTime = GetAdjustedTime();
163     nLastFailed = 0;
164 }
165
166 explicit CAddress(const struct sockaddr_in& sockAddr, uint64 nServicesIn=0)
167 {
168     nServices = nServicesIn;
169     memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
170     ip = sockAddr.sin_addr.s_addr;
171     port = sockAddr.sin_port;
172     nTime = GetAdjustedTime();
173     nLastFailed = 0;
174 }
175
176 explicit CAddress(const char* pszIn, uint64 nServicesIn=0)
177 {
178     nServices = nServicesIn;
179     memcpy(pchReserved, pchIPv4, sizeof(pchReserved));
180     ip = 0;
181     port = DEFAULT_PORT;
182     nTime = GetAdjustedTime();
183     nLastFailed = 0;
184
185     char psz[100];
186     if (strlen(pszIn) > ARRAYLEN(psz)-1)
187         return;
188     strcpy(psz, pszIn);
189     unsigned int a, b, c, d, e;
190     if (sscanf(psz, "%u.%u.%u.%u:%u", &a, &b, &c, &d, &e) < 4)
191         return;
192     char* pszPort = strchr(psz, ':');
193     if (pszPort)
194     {
195         *pszPort++ = '\0';
196         port = htons(atoi(pszPort));
197     }
198     ip = inet_addr(psz);
199 }
200
201 IMPLEMENT_SERIALIZE

```

`IMPLEMENT_SERIALIZE`—the serialization specification. This macro generates code for reading/writing `CAddress` to disk or network. The `SER_DISK` check means timestamps are saved locally but not transmitted—a privacy consideration (timestamps could reveal when you learned about addresses).

```

202 (
203 if (nType & SER_DISK)
204 {
205     READWRITE(nVersion);
206     READWRITE(nTime);
207 }
208     READWRITE(nServices);
209     READWRITE(FLATDATA(pchReserved));
210     READWRITE(ip);
211     READWRITE(port);
212 )
213
214 friend inline bool operator==(const CAddress& a, const CAddress& b)
215 {
216     return (memcmp(a.pchReserved, b.pchReserved, sizeof(a.pchReserved)) == 0 &&
217             a.ip == b.ip &&
218             a.port == b.port);
219 }
220
221 friend inline bool operator<(const CAddress& a, const CAddress& b)
222 {
223     int ret = memcmp(a.pchReserved, b.pchReserved, sizeof(a.pchReserved));
224     if (ret < 0)
225         return true;
226     else if (ret == 0)
227     {
228         if (ntohl(a.ip) < ntohl(b.ip))
229             return true;
230         else if (a.ip == b.ip)
231             return ntohs(a.port) < ntohs(b.port);
232     }
233     return false;
234 }
235
236     vector<unsigned char> GetKey() const
237 {
238         CDataStream ss;
239         ss.reserve(18);
240         ss << FLATDATA(pchReserved) << ip << port;
241
242         #if defined(_MSC_VER) && _MSC_VER < 1300
243         return vector<unsigned char>((unsigned char*)&ss.begin()[0], (unsigned char*)&ss.end()[0]);
244         #else
245         return vector<unsigned char>(ss.begin(), ss.end());
246         #endif
247     }
248
249 struct sockaddr_in GetSockAddr() const
250 {
251     struct sockaddr_in sockaddr;
252     sockaddr.sin_family = AF_INET;
253     sockaddr.sin_addr.s_addr = ip;
254     sockaddr.sin_port = port;
255     return sockaddr;
256 }
257
258 bool IsIPv4() const
259 {
260     return (memcmp(pchReserved, pchIPv4, sizeof(pchIPv4)) == 0);
261 }
262
263 bool IsRoutable() const

```

`IsRoutable()` —checks if an IP address is publicly routable. Returns false for private networks (10.x.x.x, 192.168.x.x). Bitcoin nodes shouldn't relay private addresses since they're unreachable from the public internet. This prevents polluting the address database with useless entries.

```

264 {
265     return !(GetByte(3) == 10 || (GetByte(3) == 192 && GetByte(2) == 168));
266 }
267
268 unsigned char GetByte(int n) const
269 {
270     return ((unsigned char*)&ip)[3-n];
271 }
272
273 string ToStringIPPort() const
274 {
275     return sprintf("%u.%u.%u.%u:%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0), ntohs(port));
276 }
277
278 string ToStringIP() const
279 {
280     return sprintf("%u.%u.%u.%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0));
281 }
282
283 string ToString() const
284 {
285     return sprintf("%u.%u.%u.%u:%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0), ntohs(port));
286     //return sprintf("%u.%u.%u.%u", GetByte(3), GetByte(2), GetByte(1), GetByte(0));
287 }
288
289 void print() const
290 {
291     printf("CAddress(%)s\n", ToString().c_str());
292 }
293 };
294
295
296
297
298
299
300
301 enum

```

Message type constants: `MSG_TX`, `MSG_BLOCK`, etc. These identify what kind of inventory item is being advertised or requested. The `inv` message contains a list of (type, hash) pairs; `getdata` requests specific items by (type, hash). `MSG_REVIEW` and `MSG_PRODUCT` hint at Satoshi's marketplace vision that was later removed.

```

302 {
303     MSG_TX = 1,
304     MSG_BLOCK,
305     MSG_REVIEW,
306     MSG_PRODUCT,
307     MSG_TABLE,
308 };
309
310 static const char* ppszTypeName[] =
311 {
312     "ERROR",
313     "tx",
314     "block",
315     "review",
316     "product",
317     "table",

```

```

318 };
319
320 class CInv

```

CInv—an 'inventory item' identifying data by type and hash. When a node receives a new transaction, it announces via 'inv' messages containing CInv entries. Peers check if they already have each item; if not, they request it via 'getdata'. This pattern minimizes redundant data transfer—you only download what you don't have.

```

321 {
322     public:
323     int type;
324     uint256 hash;
325
326     CInv()
327     {
328         type = 0;
329         hash = 0;
330     }
331
332     CInv(int typeIn, const uint256& hashIn)
333     {
334         type = typeIn;
335         hash = hashIn;
336     }
337
338     CInv(const string& strType, const uint256& hashIn)
339     {
340         int i;
341         for (i = 1; i < ARRAYLEN(ppszTypeName); i++)
342         {
343             if (strType == ppszTypeName[i])
344             {
345                 type = i;
346                 break;
347             }
348         }
349         if (i == ARRAYLEN(ppszTypeName))
350             throw std::out_of_range(strprintf("CInv::CInv(string, uint256) : unknown type '%s'", strType.c_str()));
351         hash = hashIn;
352     }
353
354     IMPLEMENT_SERIALIZE
355     (
356     READWRITE(type);
357     READWRITE(hash);
358     )
359
360     friend inline bool operator<(const CInv& a, const CInv& b)
361     {
362         return (a.type < b.type || (a.type == b.type && a.hash < b.hash));
363     }
364
365     bool IsKnownType() const
366     {
367         return (type >= 1 && type < ARRAYLEN(ppszTypeName));
368     }
369
370     const char* GetCommand() const
371     {
372         if (!IsKnownType())
373             throw std::out_of_range(strprintf("CInv::GetCommand() : type=%u unknown type", type));
374         return ppszTypeName[type];
375     }
376
377     string ToString() const
378     {
379         return strprintf("%s %s", GetCommand(), hash.ToString().substr(0,14).c_str());

```



```

380 }
381
382 void print() const
383 {
384     printf("CInv(%s)\n", ToString().c_str());
385 }
386 };
387
388
389
390
391
392 class CRequestTracker

```

`CRequestTracker` —callback mechanism for async requests. Some operations (like requesting market data) need to process the response when it arrives. `CRequestTracker` stores a function pointer and parameter, invoked when the response comes. This predates C++11 lambdas—today you'd use `std::function`.

```

393 {
394     public:
395     void (*fn)(void*, CDataStream&);
396     void* param1;
397
398     explicit CRequestTracker(void (*fnIn)(void*, CDataStream&)=NULL, void* param1In=NULL)
399     {
400         fn = fnIn;
401         param1 = param1In;
402     }
403
404     bool IsNull()
405     {
406         return fn == NULL;
407     }
408 };
409
410
411
412
413
414 extern bool fClient;

```

Global network state variables. `vNodes` is the list of all connected peers. `mapAddresses` stores known peer addresses for reconnection. `mapRelay` caches recently-relayed data to serve to multiple peers without re-fetching. `cs_*` variables are mutexes (`CCriticalSection`) protecting these shared structures from race conditions.

```

415 extern uint64 nLocalServices;
416 extern CAddress addrLocalHost;
417 extern CNode* pnodeLocalHost;
418 extern bool fShutdown;
419 extern array<bool, 10> vfThreadRunning;
420 extern vector<CNode*> vNodes;
421 extern CCriticalSection cs_vNodes;
422 extern map<vector<unsigned char>, CAddress> mapAddresses;
423 extern CCriticalSection cs_mapAddresses;
424 extern map<CInv, CDataStream> mapRelay;
425 extern deque<pair<int64, CInv> > vRelayExpiration;
426 extern CCriticalSection cs_mapRelay;
427 extern map<CInv, int64> mapAlreadyAskedFor;
428 extern CAddress addrProxy;
429
430
431
432

```

433

434 `class CNode`

CNode —the heart of Bitcoin's networking: a single peer connection. Each **CNode** wraps a socket and maintains: send/receive buffers, version information, connection state, inventory tracking (what the peer has/wants), and address gossip state. A running Bitcoin node typically has 8-125 **CNode** instances, each representing one peer relationship.

```

435 {
436 public:
437 // socket
438     uint64 nServices;
439     SOCKET hSocket;
440     CDataStream vSend;

```

Network buffers: **vSend** and **vRecv** are **CDataStream** objects holding outgoing and incoming data. The **CDataStream** class (defined in `serialize.h`) handles Bitcoin's custom serialization format. Critical sections **cs_vSend** and **cs_vRecv** protect these buffers—multiple threads may try to send messages or process received data simultaneously.

```

441     CDataStream vRecv;
442     CCriticalSection cs_vSend;
443     CCriticalSection cs_vRecv;
444     unsigned int nPushPos;
445     CAddress addr;
446     int nVersion;
447     bool fClient;
448     bool fInbound;
449     bool fNetworkNode;
450     bool fDisconnect;
451 protected:
452     int nRefCount;
453 public:
454     int64 nReleaseTime;
455     map<uint256, CRequestTracker> mapRequests;
456     CCriticalSection cs_mapRequests;
457
458 // flood
459     vector<CAddress> vAddrToSend;

```

vAddrToSend —addresses to gossip to this peer. When your node learns a new peer address, it queues it here for eventual transmission. **setAddrKnown** tracks addresses this peer already knows, avoiding redundant announcements. This gossip protocol is how the network discovers new peers without centralized coordination.

```

460     set<CAddress> setAddrKnown;
461
462 // inventory based relay
463     set<CInv> setInventoryKnown;

```

Inventory tracking: **setInventoryKnown** tracks what this peer has (or has been told about). When relaying a transaction, we check this set to avoid re-announcing. **vInventoryToSend** queues items to announce. This tracking prevents bandwidth waste—a transaction propagating through 10,000 nodes shouldn't be sent twice to any peer.

```

464     set<CInv> setInventoryKnown2;
465     vector<CInv> vInventoryToSend;
466     CCriticalSection cs_inventory;

```

467 multimap<int64, CInv> mapAskFor;

`mapAskFor`—items we want to request from this peer, keyed by when to ask. When we receive an 'inv' for something we don't have, we don't request immediately (in case multiple peers announce it—we'd get duplicates). Instead, we schedule the request slightly in the future. If another peer delivers it first, we cancel the request.

```

468
469 // publish and subscription
470     vector<char> vfSubscribe;
471
472
473 CNode(SOCKET hSocketIn, CAddress addrIn, bool fInboundIn=false)
474 {
475     nServices = 0;
476     hSocket = hSocketIn;
477     vSend.SetType(SER_NETWORK);
478     vRecv.SetType(SER_NETWORK);
479     nPushPos = -1;
480     addr = addrIn;
481     nVersion = 0;
482     fClient = false; // set by version message
483     fInbound = fInboundIn;
484     fNetworkNode = false;
485     fDisconnect = false;
486     nRefCount = 0;
487     nReleaseTime = 0;
488     vfSubscribe.assign(256, false);
489
490 // Push a version message
491 /// when NTP implemented, change to just nTime = GetAdjustedTime()
492     int64 nTime = (fInbound ? GetAdjustedTime() : GetTime());
493     PushMessage("version", VERSION, nLocalServices, nTime, addr);

```

Version message on connect: `PushMessage("version", VERSION, nLocalServices, nTime, addr)`. The first message any node sends is `version`, announcing its protocol version, capabilities, and current time. The recipient responds with `verack` (version acknowledge). This handshake establishes the connection and negotiates compatible features.

```

494 }
495
496 ~CNode()
497 {
498     if (hSocket != INVALID_SOCKET)
499         closesocket(hSocket);
500 }
501
502 private:
503     CNode(const CNode&);
504     void operator=(const CNode&);
505 public:
506
507
508 bool ReadyToDisconnect()
509 {
510     return fDisconnect || GetRefCount() <= 0;
511 }
512
513 int GetRefCount()
514 {
515     return max(nRefCount, 0) + (GetTime() < nReleaseTime ? 1 : 0);
516 }
517
518 void AddRef(int64 nTimeout=0)
519 {

```

```

520 if (nTimeout != 0)
521     nReleaseTime = max(nReleaseTime, GetTime() + nTimeout);
522 else
523     nRefCount++;
524 }
525
526 void Release()
527 {
528     nRefCount--;
529 }
530
531
532
533 void AddInventoryKnown(const CInv& inv)

```

`AddInventoryKnown()` and `PushInventory()` —the relay optimization. When we have something to announce, we first check if the peer already knows it. If not, we queue it for announcement. This simple check prevents the quadratic blowup that would occur if every node blindly relayed everything to every peer.

```

534 {
535     CRITICAL_BLOCK(cs_inventory)
536         setInventoryKnown.insert(inv);
537 }
538
539 void PushInventory(const CInv& inv)
540 {
541     CRITICAL_BLOCK(cs_inventory)
542     if (!setInventoryKnown.count(inv))
543         vInventoryToSend.push_back(inv);
544 }
545
546 void AskFor(const CInv& inv)

```

`AskFor()` —schedules a data request with rate limiting. The comment explains: 'We're using `mapAskFor` as a priority queue.' Requests are spaced 2 minutes apart for the same item (retry interval). This prevents hammering peers for data they may not have. The microsecond timestamps ensure FIFO ordering even for requests scheduled at the same second.

```

547 {
548     // We're using mapAskFor as a priority queue,
549     // the key is the earliest time the request can be sent
550     int64& nRequestTime = mapAlreadyAskedFor[inv];
551     printf("askfor %s %I64d\n", inv.ToString().c_str(), nRequestTime);
552
553     // Make sure not to reuse time indexes to keep things in the same order
554     int64 nNow = (GetTime() - 1) * 1000000;
555     static int64 nLastTime;
556     nLastTime = nNow = max(nNow, ++nLastTime);
557
558     // Each retry is 2 minutes after the last
559     nRequestTime = max(nRequestTime + 2 * 60 * 1000000, nNow);
560     mapAskFor.insert(make_pair(nRequestTime, inv));
561 }
562
563
564
565 void BeginMessage(const char* pszCommand)

```

`BeginMessage()` —starts constructing an outbound message. Takes the mutex, records current buffer position, writes the header. `EndMessage()` patches in the actual size and releases the mutex. If anything goes wrong between Begin and End, `AbortMessage()` rolls back the buffer. This transactional pattern prevents malformed messages from being sent.

```

566 {
567     EnterCriticalSection(&cs_vSend);
568     if (nPushPos != -1)
569         AbortMessage();
570         nPushPos = vSend.size();
571         vSend << CMessageHeader(pszCommand, 0);
572     printf("sending: %12s ", pszCommand);
573 }
574
575 void AbortMessage()
576 {
577     if (nPushPos == -1)
578         return;
579         vSend.resize(nPushPos);
580         nPushPos = -1;
581     LeaveCriticalSection(&cs_vSend);
582     printf("(aborted)\n");
583 }
584
585 void EndMessage()
586 {
587     extern int nDropMessagesTest;
588     if (nDropMessagesTest > 0 && GetRand(nDropMessagesTest) == 0)
589     {
590         printf("dropmessages DROPPING SEND MESSAGE\n");
591         AbortMessage();
592         return;
593     }
594
595     if (nPushPos == -1)
596         return;
597
598     // Patch in the size
599     unsigned int nSize = vSend.size() - nPushPos - sizeof(CMessageHeader);
600     memcpy((char*)&vSend[nPushPos] + offsetof(CMessageHeader, nMessageSize), &nSize, sizeof(nSize));
601
602     printf("(%d bytes) ", nSize);
603     //for (int i = nPushPos+sizeof(CMessageHeader); i < min(vSend.size(),
604     //    nPushPos+sizeof(CMessageHeader)+20U); i++)
605     //    printf("%02x ", vSend[i] & 0xff);
606     printf("\n");
607
608     nPushPos = -1;
609     LeaveCriticalSection(&cs_vSend);
610 }
611
612 void EndMessageAbortIfEmpty()
613 {
614     if (nPushPos == -1)
615         return;
616     int nSize = vSend.size() - nPushPos - sizeof(CMessageHeader);
617     if (nSize > 0)
618         EndMessage();
619     else
620         AbortMessage();
621 }
622
623 const char* GetMessageCommand() const
624 {
625     if (nPushPos == -1)
626         return "";
627     return &vSend[nPushPos] + offsetof(CMessageHeader, pchCommand);

```

```

627 }
628
629
630
631
632 void PushMessage(const char* pszCommand)

```

`PushMessage()` —variadic templates for sending messages with any number of arguments. The syntax `vSend << a1 << a2` uses operator overloading to serialize each argument. This elegant design lets you write `PushMessage("block", block)` and have the block automatically serialized according to its type's `IMPLEMENT_SERIALIZE` definition.

```

633 {
634     try
635     {
636         BeginMessage(pszCommand);
637         EndMessage();
638     }
639     catch (...)
640     {
641         AbortMessage();
642         throw;
643     }
644 }
645
646 template<typename T1>
647 void PushMessage(const char* pszCommand, const T1& a1)
648 {
649     try
650     {
651         BeginMessage(pszCommand);
652         vSend << a1;
653         EndMessage();
654     }
655     catch (...)
656     {
657         AbortMessage();
658         throw;
659     }
660 }
661
662 template<typename T1, typename T2>
663 void PushMessage(const char* pszCommand, const T1& a1, const T2& a2)
664 {
665     try
666     {
667         BeginMessage(pszCommand);
668         vSend << a1 << a2;
669         EndMessage();
670     }
671     catch (...)
672     {
673         AbortMessage();
674         throw;
675     }
676 }
677
678 template<typename T1, typename T2, typename T3>
679 void PushMessage(const char* pszCommand, const T1& a1, const T2& a2, const T3& a3)
680 {
681     try
682     {
683         BeginMessage(pszCommand);
684         vSend << a1 << a2 << a3;
685         EndMessage();
686     }
687     catch (...)
688     {

```

```

689 AbortMessage();
690 throw;
691 }
692 }
693
694 template<typename T1, typename T2, typename T3, typename T4>
695 void PushMessage(const char* pszCommand, const T1& a1, const T2& a2, const T3& a3, const T4& a4)
696 {
697     try
698     {
699         BeginMessage(pszCommand);
700         vSend << a1 << a2 << a3 << a4;
701         EndMessage();
702     }
703     catch (...)
704     {
705         AbortMessage();
706         throw;
707     }
708 }
709
710
711 void PushRequest(const char* pszCommand,
712 void (*fn)(void*, CDataStream&), void* param1)
713 {
714     uint256 hashReply;
715     RAND_bytes((unsigned char*)&hashReply, sizeof(hashReply));
716
717     CRITICAL_BLOCK(cs_mapRequests)
718         mapRequests[hashReply] = CRequestTracker(fn, param1);
719
720     PushMessage(pszCommand, hashReply);
721 }
722
723 template<typename T1>
724 void PushRequest(const char* pszCommand, const T1& a1,
725 void (*fn)(void*, CDataStream&), void* param1)
726 {
727     uint256 hashReply;
728     RAND_bytes((unsigned char*)&hashReply, sizeof(hashReply));
729
730     CRITICAL_BLOCK(cs_mapRequests)
731         mapRequests[hashReply] = CRequestTracker(fn, param1);
732
733     PushMessage(pszCommand, hashReply, a1);
734 }
735
736 template<typename T1, typename T2>
737 void PushRequest(const char* pszCommand, const T1& a1, const T2& a2,
738 void (*fn)(void*, CDataStream&), void* param1)
739 {
740     uint256 hashReply;
741     RAND_bytes((unsigned char*)&hashReply, sizeof(hashReply));
742
743     CRITICAL_BLOCK(cs_mapRequests)
744         mapRequests[hashReply] = CRequestTracker(fn, param1);
745
746     PushMessage(pszCommand, hashReply, a1, a2);
747 }
748
749
750
751 bool IsSubscribed(unsigned int nChannel);
752 void Subscribe(unsigned int nChannel, unsigned int nHops=0);
753 void CancelSubscribe(unsigned int nChannel);
754 void Disconnect();
755 };
756
757
758
759
760

```

```

761
762
763
764
765
766 inline void RelayInventory(const CInv& inv)

```

`RelayInventory()` —broadcasts an announcement to all peers. Iterates through `vNodes` calling `PushInventory()` on each. The per-peer filtering (via `setInventoryKnown`) ensures no redundant announcements. This is how new transactions propagate: miner broadcasts to their peers, those peers relay to their peers, exponentially spreading across the network.

```

767 {
768 // Put on lists to offer to the other nodes
769 CRITICAL_BLOCK(cs_vNodes)
770 foreach(CNode* pnode, vNodes)
771     pnode->PushInventory(inv);
772 }
773
774 template<typename T>
775 void RelayMessage(const CInv& inv, const T& a)
776 {
777     CDataStream ss(SER_NETWORK);
778     ss.reserve(10000);
779     ss << a;
780     RelayMessage(inv, ss);
781 }
782
783 template<>

```

`RelayMessage()` —caches and relays data. When relaying transactions or blocks, we might need to serve the same data to multiple peers. Rather than re-serializing each time, `mapRelay` caches the serialized form for 15 minutes. The template specialization handles `CDataStream` efficiently while the generic version accepts any serializable type.

```

784 inline void RelayMessage<>(const CInv& inv, const CDataStream& ss)
785 {
786     CRITICAL_BLOCK(cs_mapRelay)
787     {
788 // Expire old relay messages
789 while (!vRelayExpiration.empty() && vRelayExpiration.front().first < GetTime())
790     {
791         mapRelay.erase(vRelayExpiration.front().second);
792         vRelayExpiration.pop_front();
793     }
794
795 // Save original serialized message so newer versions are preserved
796     mapRelay[inv] = ss;
797     vRelayExpiration.push_back(make_pair(GetTime() + 15 * 60, inv));
798 }
799
800 RelayInventory(inv);
801 }
802
803
804
805
806
807
808
809
810 //
811 // Templates for the publish and subscription system.
812 // The object being published as T& obj needs to have:
813 // a set<unsigned int> setSources member

```



```

814 // specializations of AdvertInsert and AdvertErase
815 // Currently implemented for CTable and CProduct.
816 //
817
818 template<typename T>

```

Publish/subscribe system—Satoshi's marketplace feature. `AdvertStartPublish()` and related functions implement a pub/sub mechanism for `CTable` and `CProduct` objects. Nodes could subscribe to channels and receive product listings. This code was largely unused and removed in later versions, but hints at Satoshi's vision of Bitcoin as a platform for decentralized commerce.

```

819 void AdvertStartPublish(CNode* pfrom, unsigned int nChannel, unsigned int nHops, T& obj)
820 {
821     // Add to sources
822     obj.setSources.insert(pfrom->addr.ip);
823
824     if (!AdvertInsert(obj))
825         return;
826
827     // Relay
828     CRITICAL_BLOCK(cs_vNodes)
829     foreach(CNode* pnode, vNodes)
830     if (pnode != pfrom && (nHops < PUBLISH_HOPS || pnode->IsSubscribed(nChannel)))
831         pnode->PushMessage("publish", nChannel, nHops, obj);
832 }
833
834 template<typename T>
835 void AdvertStopPublish(CNode* pfrom, unsigned int nChannel, unsigned int nHops, T& obj)
836 {
837     uint256 hash = obj.GetHash();
838
839     CRITICAL_BLOCK(cs_vNodes)
840     foreach(CNode* pnode, vNodes)
841     if (pnode != pfrom && (nHops < PUBLISH_HOPS || pnode->IsSubscribed(nChannel)))
842         pnode->PushMessage("pub-cancel", nChannel, nHops, hash);
843
844     AdvertErase(obj);
845 }
846
847 template<typename T>
848 void AdvertRemoveSource(CNode* pfrom, unsigned int nChannel, unsigned int nHops, T& obj)
849 {
850     // Remove a source
851     obj.setSources.erase(pfrom->addr.ip);
852
853     // If no longer supported by any sources, cancel it
854     if (obj.setSources.empty())
855         AdvertStopPublish(pfrom, nChannel, nHops, obj);
856 }
857

```

SUMMARY

The CNode class is Bitcoin's window to the world. Each instance represents a relationship with another participant in the network—a relationship built on mutual verification rather than trust. The careful tracking of what each peer knows, the rate-limited requests, the gossip protocols for addresses and inventory—all work together to create a resilient network where no single point of failure can bring down the system.

Satoshi's protocol design shows deep understanding of distributed systems. The pull-based model (inv/getdata) prevents spam. The peer address gossip enables organic network growth. The version handshake allows protocol evolution. These patterns have proven robust: Bitcoin's peer-to-peer layer has operated continuously since January 2009, surviving attacks, bugs, and massive growth.

The unused marketplace code (MSG_PRODUCT, MSG_REVIEW, pub/sub) reveals a broader vision that Satoshi ultimately didn't pursue. Bitcoin became focused as a currency and settlement layer, with commerce applications built on top rather than integrated into the base protocol. But these remnants show Satoshi thinking beyond simple value transfer.

readme.txt

77 lines TEXT

SATOSHI'S INSTRUCTIONS

The original readme file included with Bitcoin v0.01. This document provides Satoshi's own description of what Bitcoin is and how to use it—a fascinating primary source from the birth of cryptocurrency.

1 BitCoin v0.01 ALPHA**v0.01 ALPHA - the very first public release**

```

2
3 Copyright (c) 2009 Satoshi Nakamoto
4 Distributed under the MIT/X11 software license, see the accompanying
5 file license.txt or http://www.opensource.org/licenses/mit-license.php.
6 This product includes software developed by the OpenSSL Project for use in
7 the OpenSSL Toolkit (http://www.openssl.org/). This product includes
8 cryptographic software written by Eric Young (eay@cryptsoft.com).
9
10

```

11 Compilers Supported

```

12 -----
13 MinGW GCC (v3.4.5)
14 Microsoft Visual C++ 6.0 SP6
15
16

```

17 Dependencies**Dependencies - wxWidgets, OpenSSL, Berkeley DB, Boost**

```

18 -----
19 Libraries you need to obtain separately to build:
20
21          default path    download
22 wxWidgets      \wxWidgets    http://www.wxwidgets.org/downloads/
23 OpenSSL        \OpenSSL      http://www.openssl.org/source/
24 Berkeley DB    \DB          http://www.oracle.com/technology/software/products/berkeley-db/index.html
25 Boost          \Boost       http://www.boost.org/users/download/
26
27 Their licenses:
28 wxWidgets      LGPL 2.1 with very liberal exceptions
29 OpenSSL        Old BSD license with the problematic advertising requirement
30 Berkeley DB    New BSD license with additional requirement that linked software must be free open source
31 Boost          MIT-like license
32
33
34 OpenSSL
35 -----
36 Bitcoin does not use any encryption. If you want to do a no-everything

```

Bitcoin does not use encryption - only digital signatures

```

37 build of OpenSSL to exclude encryption routines, a few patches are required.
38 (OpenSSL v0.9.8h)
39
40 Edit engines\e_gmp.c and put this #ifndef around #include <openssl/rsa.h>
41 #ifndef OPENSSL_NO_RSA
42 #include <openssl/rsa.h>
43 #endif
44
45 Add this to crypto\err\err_all.c before the ERR_load_crypto_strings line:
46 void ERR_load_RSA_strings(void) { }
47
48 Edit ms\mingw32.bat and replace the Configure line's parameters with this
49 no-everything list. You have to put this in the batch file because batch
50 files can't handle more than 9 parameters.
51 perl Configure mingw threads no-rc2 no-rc4 no-rc5 no-idea no-des no-bf no-cast no-aes no-camellia no-
seed no-rsa no-dh
52
53 Also REM out the following line in ms\mingw32.bat. The build fails after it's
54 already finished building libeay32, which is all we care about, but the
55 failure aborts the script before it runs dllwrap to generate libeay32.dll.

```

```

56  REM  if errorlevel 1 goto end
57
58  Build
59  ms\mingw32.bat
60
61  If you want to use it with MSVC, generate the .lib file
62  lib /machine:i386 /def:ms\libey32.def /out:out\libey32.lib
63
64
65  Berkeley DB
66  -----
67  MinGW with MSYS:
68  cd \DB\build_unix
69  sh ../dist/configure --enable-mingw --enable-cxx
70  make
71
72
73  Boost
74  -----
75  You may need Boost version 1.35 to build with MSVC 6.0.  I couldn't get
76  version 1.37 to compile with MSVC 6.0.
77

```

SUMMARY

This readme is a historical artifact documenting how to build Bitcoin in its earliest days, when it ran only on Windows.

script.cpp

1128 lines CPP

THE VIRTUAL MACHINE

This file implements Bitcoin's script interpreter—the virtual machine that determines who can spend coins. Every bitcoin ever transferred—hundreds of billions of dollars in value—passed through this exact code. The interpreter is deceptively simple: a stack of byte arrays, a program counter, and roughly 100 opcodes. Yet this simplicity is its strength.

Satoshi designed Script using "reverse Polish notation" (RPN), a Forth-style syntax where operators come after operands. Instead of "3 + 4", you write "3 4 +". The interpreter maintains a stack; data pushes go onto the stack, operations pop arguments and push results. This approach eliminates parsing complexity—no operator precedence, no parentheses, no ambiguity. The stack state after execution determines validity: if the top element is non-zero (truthy), the script passes.

What makes this remarkable is its generality. Satoshi didn't hardcode "check signature against public key" as the only spending rule. Instead, he created a language that expresses arbitrary conditions. Want multi-signature? `OP_CHECKMULTISIG`. Time locks? `OP_CHECKLOCKTIMEVERIFY` (added later, but the architecture supported it). Escrow? Hash locks? All possible without protocol changes—just different scripts.

The script system is also why Bitcoin survived its early bugs. When critical vulnerabilities were found (like the `OP_CAT` memory explosion in August 2010), Satoshi could disable individual opcodes without breaking the entire system. Scripts using disabled opcodes simply fail; everything else continues working.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6
7 bool CheckSig(vector<unsigned char> vchSig, vector<unsigned char> vchPubKey, CScript scriptCode, const
  CTransaction& txTo, unsigned int nIn, int nHashType);

```

CheckSig - heart of Bitcoin security. Verifies ECDSA signature against transaction hash. If this function has bugs, Bitcoin breaks.

```

8
9
10
11 typedef vector<unsigned char> valtype;
12 static const valtype vchFalse(0);
13 static const valtype vchZero(0);
14 static const valtype vchTrue(1, 1);
15 static const CBigNum bnZero(0);
16 static const CBigNum bnOne(1);
17 static const CBigNum bnFalse(0);
18 static const CBigNum bnTrue(1);
19
20
21 bool CastToBool(const valtype& vch)
22 {
23     return (CBigNum(vch) != bnZero);
24 }
25
26 void MakeSameSize(valtype& vch1, valtype& vch2)
27 {
28     // Lengthen the shorter one
29     if (vch1.size() < vch2.size())
30         vch1.resize(vch2.size(), 0);
31     if (vch2.size() < vch1.size())
32         vch2.resize(vch1.size(), 0);
33 }
34
35
36
37 //
38 // Script is a stack machine (like Forth) that evaluates a predicate
39 // returning a bool indicating valid or not. There are no loops.
40 //
41 #define stacktop(i) (stack.at(stack.size()+(i)))
42 #define altstacktop(i) (altstack.at(altstack.size()+(i)))
43
44 bool EvalScript(const CScript& script, const CTransaction& txTo, unsigned int nIn, int nHashType,

```

EvalScript - the interpreter main loop. Reads opcodes, manipulates stack, checks conditions. Script evaluates to TRUE (non-empty stack, top!=0) or FALSE.

```

45         vector<vector<unsigned char> >*> pvStackRet)
46 {
47     CAutoBN_CTX pctx;
48     CScript::const_iterator pc = script.begin();
49     CScript::const_iterator pend = script.end();
50     CScript::const_iterator pbegincodehash = script.begin();
51     vector<bool> vFExec;
52     vector<valtype> stack;
53     vector<valtype> altstack;
54     if (pvStackRet)
55         pvStackRet->clear();
56
57

```



```

58 while (pc < pend)
59 {
60     bool fExec = !count(vfExec.begin(), vfExec.end(), false);
61
62     //
63     // Read instruction
64     //
65     opcodetype opcode;
66     valtype vchPushValue;
67     if (!script.GetOp(pc, opcode, vchPushValue))
68         return false;
69
70     if (fExec && opcode <= OP_PUSHDATA4)

```

Data push opcodes (0x01-0x4b) push that many bytes onto stack. OP_PUSHDATA1/2/4 handle larger pushes. Most scripts are mostly data pushes.

```

71         stack.push_back(vchPushValue);
72     else if (fExec || (OP_IF <= opcode && opcode <= OP_ENDIF))
73         switch (opcode)
74         {
75             //
76             // Push value
77             //
78             case OP_1NEGATE:

```

OP_1 through OP_16 - push small integers. OP_1NEGATE pushes -1. These enable compact scripts without encoding full numbers.

```

79     case OP_1:
80     case OP_2:
81     case OP_3:
82     case OP_4:
83     case OP_5:
84     case OP_6:
85     case OP_7:
86     case OP_8:
87     case OP_9:
88     case OP_10:
89     case OP_11:
90     case OP_12:
91     case OP_13:
92     case OP_14:
93     case OP_15:
94     case OP_16:
95     {
96         // ( -- value)
97         CBigNum bn((int)opcode - (int)(OP_1 - 1));
98         stack.push_back(bn.getvch());
99     }
100     break;
101
102
103     //
104     // Control
105     //
106     case OP_NOP:
107         break;
108
109     case OP_VER:
110     {
111         CBigNum bn(VERSION);
112         stack.push_back(bn.getvch());
113     }
114     break;
115

```

```
116 case OP_IF:
```

OP_IF/OP_NOTIF - conditional execution. Scripts can have branches! Combined with OP_ELSE/OP_ENDIF, enables complex spending conditions.

```
117 case OP_NOTIF:
118 case OP_VERIFY:
119 case OP_VERNOTIF:
120 {
121 // <expression> if [statements] [else [statements]] endif
122 bool fValue = false;
123 if (fExec)
124 {
125 if (stack.size() < 1)
126 return false;
127 valtype& vch = stacktop(-1);
128 if (opcode == OP_VERIFY || opcode == OP_VERNOTIF)
```

OP_VERIFY/OP_VERNOTIF - immediately fail if reached. Satoshi reserved these for future soft-fork upgrades. Clever forward-compatibility.

```
129             fValue = (CBigNum(VERSION) >= CBigNum(vch));
130 else
131             fValue = CastToBool(vch);
132 if (opcode == OP_NOTIF || opcode == OP_VERNOTIF)
133             fValue = !fValue;
134             stack.pop_back();
135 }
136             vfExec.push_back(fValue);
137 }
138 break;
139
140 case OP_ELSE:
141 {
142 if (vfExec.empty())
143 return false;
144             vfExec.back() = !vfExec.back();
145 }
146 break;
147
148 case OP_ENDIF:
149 {
150 if (vfExec.empty())
151 return false;
152             vfExec.pop_back();
153 }
154 break;
155
156 case OP_VERIFY:
157 {
158 // (true -- ) or
159 // (false -- false) and return
160 if (stack.size() < 1)
161 return false;
162 bool fValue = CastToBool(stacktop(-1));
163 if (fValue)
164             stack.pop_back();
165 else
166             pc = pend;
167 }
168 break;
169
170 case OP_RETURN:
171 {
172             pc = pend;
173 }
```

```

174 break;
175
176
177 //
178 // Stack ops
179 //
180 case OP_TOALTSTACK:
181 {
182     if (stack.size() < 1)
183         return false;
184     altstack.push_back(stacktop(-1));
185     stack.pop_back();
186 }
187 break;
188
189 case OP_FROMALTSTACK:
190 {
191     if (altstack.size() < 1)
192         return false;
193     stack.push_back(altstacktop(-1));
194     altstack.pop_back();
195 }
196 break;
197
198 case OP_2DROP:
199 {
200     // (x1 x2 -- )
201     stack.pop_back();
202     stack.pop_back();
203 }
204 break;
205
206 case OP_2DUP:
207 {
208     // (x1 x2 -- x1 x2 x1 x2)
209     if (stack.size() < 2)
210         return false;
211     valtype vch1 = stacktop(-2);
212     valtype vch2 = stacktop(-1);
213     stack.push_back(vch1);
214     stack.push_back(vch2);
215 }
216 break;
217
218 case OP_3DUP:
219 {
220     // (x1 x2 x3 -- x1 x2 x3 x1 x2 x3)
221     if (stack.size() < 3)
222         return false;
223     valtype vch1 = stacktop(-3);
224     valtype vch2 = stacktop(-2);
225     valtype vch3 = stacktop(-1);
226     stack.push_back(vch1);
227     stack.push_back(vch2);
228     stack.push_back(vch3);
229 }
230 break;
231
232 case OP_2OVER:
233 {
234     // (x1 x2 x3 x4 -- x1 x2 x3 x4 x1 x2)
235     if (stack.size() < 4)
236         return false;
237     valtype vch1 = stacktop(-4);
238     valtype vch2 = stacktop(-3);
239     stack.push_back(vch1);
240     stack.push_back(vch2);
241 }
242 break;
243
244 case OP_2ROT:
245 {

```

```

246 // (x1 x2 x3 x4 x5 x6 -- x3 x4 x5 x6 x1 x2)
247 if (stack.size() < 6)
248 return false;
249     valtype vch1 = stacktop(-6);
250     valtype vch2 = stacktop(-5);
251     stack.erase(stack.end()-6, stack.end()-4);
252     stack.push_back(vch1);
253     stack.push_back(vch2);
254 }
255 break;
256
257 case OP_2SWAP:
258 {
259 // (x1 x2 x3 x4 -- x3 x4 x1 x2)
260 if (stack.size() < 4)
261 return false;
262 swap(stacktop(-4), stacktop(-2));
263 swap(stacktop(-3), stacktop(-1));
264 }
265 break;
266
267 case OP_IFDUP:
268 {
269 // (x - 0 | x x)
270 if (stack.size() < 1)
271 return false;
272     valtype vch = stacktop(-1);
273 if (CastToBool(vch))
274     stack.push_back(vch);
275 }
276 break;
277
278 case OP_DEPTH:
279 {
280 // -- stacksize
281     CBigNum bn(stack.size());
282     stack.push_back(bn.getvch());
283 }
284 break;
285
286 case OP_DROP:
287 {
288 // (x -- )
289 if (stack.size() < 1)
290 return false;
291     stack.pop_back();
292 }
293 break;
294
295 case OP_DUP:
296 {
297 // (x -- x x)
298 if (stack.size() < 1)
299 return false;
300     valtype vch = stacktop(-1);
301     stack.push_back(vch);
302 }
303 break;
304
305 case OP_NIP:
306 {
307 // (x1 x2 -- x2)
308 if (stack.size() < 2)
309 return false;
310     stack.erase(stack.end() - 2);
311 }
312 break;
313
314 case OP_OVER:
315 {
316 // (x1 x2 -- x1 x2 x1)
317 if (stack.size() < 2)

```

```

318 return false;
319         valtype vch = stacktop(-2);
320         stack.push_back(vch);
321     }
322     break;
323
324 case OP_PICK:
325 case OP_ROLL:
326 {
327     // (xn ... x2 x1 x0 n - xn ... x2 x1 x0 xn)
328     // (xn ... x2 x1 x0 n - ... x2 x1 x0 xn)
329     if (stack.size() < 2)
330         return false;
331     int n = CBigNum(stacktop(-1)).getint();
332     stack.pop_back();
333     if (n < 0 || n >= stack.size())
334         return false;
335     valtype vch = stacktop(-n-1);
336     if (opcode == OP_ROLL)
337         stack.erase(stack.end()-n-1);
338     stack.push_back(vch);
339 }
340 break;
341
342 case OP_ROT:
343 {
344     // (x1 x2 x3 -- x2 x3 x1)
345     // x2 x1 x3 after first swap
346     // x2 x3 x1 after second swap
347     if (stack.size() < 3)
348         return false;
349     swap(stacktop(-3), stacktop(-2));
350     swap(stacktop(-2), stacktop(-1));
351 }
352 break;
353
354 case OP_SWAP:
355 {
356     // (x1 x2 -- x2 x1)
357     if (stack.size() < 2)
358         return false;
359     swap(stacktop(-2), stacktop(-1));
360 }
361 break;
362
363 case OP_TUCK:
364 {
365     // (x1 x2 -- x2 x1 x2)
366     if (stack.size() < 2)
367         return false;
368     valtype vch = stacktop(-1);
369     stack.insert(stack.end()-2, vch);
370 }
371 break;
372
373 //
374 //
375 // Splice ops
376 //
377 case OP_CAT:
378 {
379     // (x1 x2 -- out)
380     if (stack.size() < 2)
381         return false;
382     valtype& vch1 = stacktop(-2);
383     valtype& vch2 = stacktop(-1);
384     vch1.insert(vch1.end(), vch2.begin(), vch2.end());
385     stack.pop_back();
386 }
387 break;
388
389 case OP_SUBSTR:

```

```

390 {
391 // (in begin size -- out)
392 if (stack.size() < 3)
393 return false;
394 valtype& vch = stacktop(-3);
395 int nBegin = CBigNum(stacktop(-2)).getint();
396 int nEnd = nBegin + CBigNum(stacktop(-1)).getint();
397 if (nBegin < 0 || nEnd < nBegin)
398 return false;
399 if (nBegin > vch.size())
400 nBegin = vch.size();
401 if (nEnd > vch.size())
402 nEnd = vch.size();
403 vch.erase(vch.begin() + nEnd, vch.end());
404 vch.erase(vch.begin(), vch.begin() + nBegin);
405 stack.pop_back();
406 stack.pop_back();
407 }
408 break;
409
410 case OP_LEFT:
411 case OP_RIGHT:
412 {
413 // (in size -- out)
414 if (stack.size() < 2)
415 return false;
416 valtype& vch = stacktop(-2);
417 int nSize = CBigNum(stacktop(-1)).getint();
418 if (nSize < 0)
419 return false;
420 if (nSize > vch.size())
421 nSize = vch.size();
422 if (opcode == OP_LEFT)
423 vch.erase(vch.begin() + nSize, vch.end());
424 else
425 vch.erase(vch.begin(), vch.end() - nSize);
426 stack.pop_back();
427 }
428 break;
429
430 case OP_SIZE:
431 {
432 // (in -- in size)
433 if (stack.size() < 1)
434 return false;
435 CBigNum bn(stacktop(-1).size());
436 stack.push_back(bn.getvch());
437 }
438 break;
439
440
441 //
442 // Bitwise logic
443 //
444 case OP_INVERT:
445 {
446 // (in - out)
447 if (stack.size() < 1)
448 return false;
449 valtype& vch = stacktop(-1);
450 for (int i = 0; i < vch.size(); i++)
451 vch[i] = ~vch[i];
452 }
453 break;
454
455 case OP_AND:
456 case OP_OR:
457 case OP_XOR:
458 {
459 // (x1 x2 - out)
460 if (stack.size() < 2)
461 return false;

```

```

462             valtype& vch1 = stacktop(-2);
463             valtype& vch2 = stacktop(-1);
464 MakeSameSize(vch1, vch2);
465 if (opcode == OP_AND)
466 {
467     for (int i = 0; i < vch1.size(); i++)
468         vch1[i] &= vch2[i];
469 }
470 else if (opcode == OP_OR)
471 {
472     for (int i = 0; i < vch1.size(); i++)
473         vch1[i] |= vch2[i];
474 }
475 else if (opcode == OP_XOR)
476 {
477     for (int i = 0; i < vch1.size(); i++)
478         vch1[i] ^= vch2[i];
479 }
480         stack.pop_back();
481 }
482 break;
483
484 case OP_EQUAL:
485 case OP_EQUALVERIFY:
486 //case OP_NOTEQUAL: // use OP_NUMNOTEQUAL
487 {
488 // (x1 x2 - bool)
489 if (stack.size() < 2)
490     return false;
491         valtype& vch1 = stacktop(-2);
492         valtype& vch2 = stacktop(-1);
493 bool fEqual = (vch1 == vch2);
494 // OP_NOTEQUAL is disabled because it would be too easy to say
495 // something like n != 1 and have some wiseguy pass in 1 with extra
496 // zero bytes after it (numerically, 0x01 == 0x0001 == 0x000001)
497 //if (opcode == OP_NOTEQUAL)
498 //    fEqual = !fEqual;
499         stack.pop_back();
500         stack.pop_back();
501         stack.push_back(fEqual ? vchTrue : vchFalse);
502 if (opcode == OP_EQUALVERIFY)
503 {
504     if (fEqual)
505         stack.pop_back();
506     else
507         pc = pend;
508 }
509 }
510 break;
511
512
513 //
514 // Numeric
515 //
516 case OP_1ADD:
517 case OP_1SUB:
518 case OP_2MUL:
519 case OP_2DIV:
520 case OP_NEGATE:
521 case OP_ABS:
522 case OP_NOT:
523 case OP_0NOTEQUAL:
524 {
525 // (in -- out)
526 if (stack.size() < 1)
527     return false;
528         CBigNum bn(stacktop(-1));
529 switch (opcode)
530 {
531 case OP_1ADD:         bn += bnOne; break;
532 case OP_1SUB:         bn -= bnOne; break;
533 case OP_2MUL:         bn <<= 1; break;

```

```

534 case OP_2DIV:      bn >>= 1; break;
535 case OP_NEGATE:    bn = -bn; break;
536 case OP_ABS:      if (bn < bnZero) bn = -bn; break;
537 case OP_NOT:      bn = (bn == bnZero); break;
538 case OP_0NOTEQUAL: bn = (bn != bnZero); break;
539 }
540             stack.pop_back();
541             stack.push_back(bn.getvch());
542 }
543 break;
544
545 case OP_ADD:
546 case OP_SUB:
547 case OP_MUL:
548 case OP_DIV:
549 case OP_MOD:
550 case OP_LSHIFT:
551 case OP_RSHIFT:
552 case OP_BOOLAND:
553 case OP_BOOLOR:
554 case OP_NUMEQUAL:
555 case OP_NUMEQUALVERIFY:
556 case OP_NUMNOTEQUAL:
557 case OP_LESSTHAN:
558 case OP_GREATERTHAN:
559 case OP_LESSTHANOREQUAL:
560 case OP_GREATERTHANOREQUAL:
561 case OP_MIN:
562 case OP_MAX:
563 {
564 // (x1 x2 -- out)
565 if (stack.size() < 2)
566 return false;
567             CBigNum bn1(stacktop(-2));
568             CBigNum bn2(stacktop(-1));
569             CBigNum bn;
570 switch (opcode)
571 {
572 case OP_ADD:
573             bn = bn1 + bn2;
574 break;
575
576 case OP_SUB:
577             bn = bn1 - bn2;
578 break;
579
580 case OP_MUL:
581 if (!BN_mul(&bn, &bn1, &bn2, pctx))
582 return false;
583 break;
584
585 case OP_DIV:
586 if (!BN_div(&bn, NULL, &bn1, &bn2, pctx))
587 return false;
588 break;
589
590 case OP_MOD:
591 if (!BN_mod(&bn, &bn1, &bn2, pctx))
592 return false;
593 break;
594
595 case OP_LSHIFT:
596 if (bn2 < bnZero)
597 return false;
598             bn = bn1 << bn2.getulong();
599 break;
600
601 case OP_RSHIFT:
602 if (bn2 < bnZero)
603 return false;
604             bn = bn1 >> bn2.getulong();
605 break;

```



```

606
607 case OP_BOOLAND:          bn = (bn1 != bnZero && bn2 != bnZero); break;
608 case OP_BOOLOR:           bn = (bn1 != bnZero || bn2 != bnZero); break;
609 case OP_NUMEQUAL:         bn = (bn1 == bn2); break;
610 case OP_NUMEQUALVERIFY:   bn = (bn1 == bn2); break;
611 case OP_NUMNOTEQUAL:      bn = (bn1 != bn2); break;
612 case OP_LESSTHAN:         bn = (bn1 < bn2); break;
613 case OP_GREATERTHAN:      bn = (bn1 > bn2); break;
614 case OP_LESSTHANOREQUAL:  bn = (bn1 <= bn2); break;
615 case OP_GREATERTHANOREQUAL: bn = (bn1 >= bn2); break;
616 case OP_MIN:              bn = (bn1 < bn2 ? bn1 : bn2); break;
617 case OP_MAX:              bn = (bn1 > bn2 ? bn1 : bn2); break;
618 }
619         stack.pop_back();
620         stack.pop_back();
621         stack.push_back(bn.getvch());
622
623 if (opcode == OP_NUMEQUALVERIFY)
624 {
625     if (CastToBool(stacktop(-1)))
626         stack.pop_back();
627 else
628     pc = pend;
629 }
630 }
631 break;
632
633 case OP_WITHIN:
634 {
635     // (x min max -- out)
636     if (stack.size() < 3)
637         return false;
638         CBigNum bn1(stacktop(-3));
639         CBigNum bn2(stacktop(-2));
640         CBigNum bn3(stacktop(-1));
641     bool fValue = (bn2 <= bn1 && bn1 < bn3);
642         stack.pop_back();
643         stack.pop_back();
644         stack.pop_back();
645         stack.push_back(fValue ? vchTrue : vchFalse);
646 }
647 break;
648
649
650 //
651 // Crypto
652 //
653 case OP_RIPEMD160:
654 case OP_SHA1:
655 case OP_SHA256:
656 case OP_HASH160:
657 case OP_HASH256:
658 {
659     // (in -- hash)
660     if (stack.size() < 1)
661         return false;
662         valtype& vch = stacktop(-1);
663         valtype vchHash(opcode == OP_RIPEMD160 || opcode == OP_SHA1 || opcode == OP_HASH160 ? 20
: 32);
664     if (opcode == OP_RIPEMD160)
665         RIPEMD160(&vch[0], vch.size(), &vchHash[0]);
666     else if (opcode == OP_SHA1)
667         SHA1(&vch[0], vch.size(), &vchHash[0]);
668     else if (opcode == OP_SHA256)
669         SHA256(&vch[0], vch.size(), &vchHash[0]);
670     else if (opcode == OP_HASH160)
671     {
672         uint160 hash160 = Hash160(vch);
673         memcpy(&vchHash[0], &hash160, sizeof(hash160));
674     }
675     else if (opcode == OP_HASH256)
676     {

```

```

677             uint256 hash = Hash(vch.begin(), vch.end());
678     memcpy(&vchHash[0], &hash, sizeof(hash));
679 }
680         stack.pop_back();
681         stack.push_back(vchHash);
682 }
683 break;
684
685 case OP_CODESEPARATOR:
686 {
687     // Hash starts after the code separator
688         pbegincodehash = pc;
689 }
690 break;
691
692 case OP_CHECKSIG:
693 case OP_CHECKSIGVERIFY:
694 {
695     // (sig pubkey -- bool)
696     if (stack.size() < 2)
697         return false;
698
699         valtype& vchSig = stacktop(-2);
700         valtype& vchPubKey = stacktop(-1);
701
702     ///// debug print
703     //PrintHex(vchSig.begin(), vchSig.end(), "sig: %s\n");
704     //PrintHex(vchPubKey.begin(), vchPubKey.end(), "pubkey: %s\n");
705
706     // Subset of script starting at the most recent codeseparator
707     CScript scriptCode(pbegincodehash, pend);
708
709     // Drop the signature, since there's no way for a signature to sign itself
710     scriptCode.FindAndDelete(CScript(vchSig));
711
712     bool fSuccess = CheckSig(vchSig, vchPubKey, scriptCode, txTo, nIn, nHashType);
713
714         stack.pop_back();
715         stack.pop_back();
716         stack.push_back(fSuccess ? vchTrue : vchFalse);
717     if (opcode == OP_CHECKSIGVERIFY)
718     {
719         if (fSuccess)
720             stack.pop_back();
721     else
722         pc = pend;
723     }
724 }
725 break;
726
727 case OP_CHECKMULTISIG:
728 case OP_CHECKMULTISIGVERIFY:
729 {
730     // ([sig ...] num_of_signatures [pubkey ...] num_of_pubkeys -- bool)
731
732     int i = 1;
733     if (stack.size() < i)
734         return false;
735
736     int nKeysCount = CBigNum(stacktop(-i)).getint();
737     if (nKeysCount < 0)
738         return false;
739     int ikey = ++i;
740     i += nKeysCount;
741     if (stack.size() < i)
742         return false;
743
744     int nSigsCount = CBigNum(stacktop(-i)).getint();
745     if (nSigsCount < 0 || nSigsCount > nKeysCount)
746         return false;
747     int isig = ++i;
748     i += nSigsCount;

```

```

749 if (stack.size() < i)
750 return false;
751
752 // Subset of script starting at the most recent codeseparator
753 CScript scriptCode(pbegincodehash, pend);
754
755 // Drop the signatures, since there's no way for a signature to sign itself
756 for (int i = 0; i < nSigsCount; i++)
757 {
758     valtype& vchSig = stacktop(-isig-i);
759     scriptCode.FindAndDelete(CScript(vchSig));
760 }
761
762 bool fSuccess = true;
763 while (fSuccess && nSigsCount > 0)
764 {
765     valtype& vchSig = stacktop(-isig);
766     valtype& vchPubKey = stacktop(-ikey);
767
768     // Check signature
769     if (CheckSig(vchSig, vchPubKey, scriptCode, txTo, nIn, nHashType))
770     {
771         isig++;
772         nSigsCount--;
773     }
774     ikey++;
775     nKeysCount--;
776
777     // If there are more signatures left than keys left,
778     // then too many signatures have failed
779     if (nSigsCount > nKeysCount)
780         fSuccess = false;
781 }
782
783 while (i-- > 0)
784     stack.pop_back();
785 stack.push_back(fSuccess ? vchTrue : vchFalse);
786
787 if (opcode == OP_CHECKMULTISIGVERIFY)
788 {
789     if (fSuccess)
790         stack.pop_back();
791     else
792         pc = pend;
793 }
794 }
795 break;
796
797 default:
798 return false;
799 }
800 }
801
802
803 if (pvStackRet)
804 *pvStackRet = stack;
805 return (stack.empty() ? false : CastToBool(stack.back()));
806 }
807
808 #undef top
809
810
811
812
813
814
815
816
817
818 uint256 SignatureHash(CScript scriptCode, const CTransaction& txTo, unsigned int nIn, int nHashType)
819 {
820 if (nIn >= txTo.vin.size())

```

```

821 {
822     printf("ERROR: SignatureHash() : nIn=%d out of range\n", nIn);
823     return 1;
824 }
825     CTransaction txTmp(txTo);
826
827 // In case concatenating two scripts ends up with two codeseparators,
828 // or an extra one at the end, this prevents all those possible incompatibilities.
829     scriptCode.FindAndDelete(CScript(OP_CODESEPARATOR));
830
831 // Blank out other inputs' signatures
832 for (int i = 0; i < txTmp.vin.size(); i++)
833     txTmp.vin[i].scriptSig = CScript();
834     txTmp.vin[nIn].scriptSig = scriptCode;
835
836 // Blank out some of the outputs
837 if ((nHashType & 0x1f) == SIGHASH_NONE)
838 {
839     // Wildcard payee
840     txTmp.vout.clear();
841
842     // Let the others update at will
843     for (int i = 0; i < txTmp.vin.size(); i++)
844     if (i != nIn)
845         txTmp.vin[i].nSequence = 0;
846 }
847 else if ((nHashType & 0x1f) == SIGHASH_SINGLE)
848 {
849     // Only lockin the txout payee at same index as txin
850     unsigned int nOut = nIn;
851     if (nOut >= txTmp.vout.size())
852     {
853         printf("ERROR: SignatureHash() : nOut=%d out of range\n", nOut);
854         return 1;
855     }
856     txTmp.vout.resize(nOut+1);
857     for (int i = 0; i < nOut; i++)
858         txTmp.vout[i].SetNull();
859
860     // Let the others update at will
861     for (int i = 0; i < txTmp.vin.size(); i++)
862     if (i != nIn)
863         txTmp.vin[i].nSequence = 0;
864 }
865
866 // Blank out other inputs completely, not recommended for open transactions
867 if (nHashType & SIGHASH_ANYONECANPAY)
868 {
869     txTmp.vin[0] = txTmp.vin[nIn];
870     txTmp.vin.resize(1);
871 }
872
873 // Serialize and hash
874     CDataStream ss(SER_GETHASH);
875     ss.reserve(10000);
876     ss << txTmp << nHashType;
877     return Hash(ss.begin(), ss.end());
878 }
879
880
881 bool CheckSig(vector<unsigned char> vchSig, vector<unsigned char> vchPubKey, CScript scriptCode,
882 const CTransaction& txTo, unsigned int nIn, int nHashType)
883 {
884     CKey key;
885     if (!key.SetPubKey(vchPubKey))
886         return false;
887
888     // Hash type is one byte tacked on to the end of the signature
889     if (vchSig.empty())
890         return false;
891     if (nHashType == 0)
892         nHashType = vchSig.back();

```

```

893 else if (nHashType != vchSig.back())
894 return false;
895     vchSig.pop_back();
896
897 if (key.Verify(SignatureHash(scriptCode, txTo, nIn, nHashType), vchSig))
898 return true;
899
900 return false;
901 }
902
903
904
905
906
907
908
909
910
911
912
913 bool Solver(const CScript& scriptPubKey, vector<pair<opcode_t, val_t>> & vSolutionRet)
914 {
915     // Templates
916     static vector<CScript> vTemplates;
917     if (vTemplates.empty())
918     {
919         // Standard tx, sender provides pubkey, receiver adds signature
920         vTemplates.push_back(CScript() << OP_PUBKEY << OP_CHECKSIG);
921
922         // Short account number tx, sender provides hash of pubkey, receiver provides signature and pubkey
923         vTemplates.push_back(CScript() << OP_DUP << OP_HASH160 << OP_PUBKEYHASH << OP_EQUALVERIFY <<
            OP_CHECKSIG);
924     }
925
926     // Scan templates
927     const CScript& script1 = scriptPubKey;
928     foreach(const CScript& script2, vTemplates)
929     {
930         vSolutionRet.clear();
931         opcode_t opcode1, opcode2;
932         vector<unsigned char> vch1, vch2;
933
934         // Compare
935         CScript::const_iterator pc1 = script1.begin();
936         CScript::const_iterator pc2 = script2.begin();
937         loop
938         {
939             bool f1 = script1.GetOp(pc1, opcode1, vch1);
940             bool f2 = script2.GetOp(pc2, opcode2, vch2);
941             if (!f1 && !f2)
942             {
943                 // Success
944                 reverse(vSolutionRet.begin(), vSolutionRet.end());
945                 return true;
946             }
947             else if (f1 != f2)
948             {
949                 break;
950             }
951             else if (opcode2 == OP_PUBKEY)
952             {
953                 if (vch1.size() <= sizeof(uint256))
954                     break;
955                 vSolutionRet.push_back(make_pair(opcode2, vch1));
956             }
957             else if (opcode2 == OP_PUBKEYHASH)
958             {
959                 if (vch1.size() != sizeof(uint160))
960                     break;
961                 vSolutionRet.push_back(make_pair(opcode2, vch1));
962             }
963             else if (opcode1 != opcode2)

```

```

964 {
965     break;
966 }
967 }
968 }
969
970     vSolutionRet.clear();
971     return false;
972 }
973
974
975 bool Solver(const CScript& scriptPubKey, uint256 hash, int nHashType, CScript& scriptSigRet)
976 {
977     scriptSigRet.clear();
978
979     vector<pair<opcode_t, val_t>> vSolution;
980     if (!Solver(scriptPubKey, vSolution))
981         return false;
982
983     // Compile solution
984     CRITICAL_BLOCK(cs_mapKeys)
985     {
986         foreach(PAIRTYPE(opcode_t, val_t)& item, vSolution)
987         {
988             if (item.first == OP_PUBKEY)
989             {
990                 // Sign
991                 const val_t& vchPubKey = item.second;
992                 if (!mapKeys.count(vchPubKey))
993                     return false;
994                 if (hash != 0)
995                 {
996                     vector<unsigned char> vchSig;
997                     if (!CKey::Sign(mapKeys[vchPubKey], hash, vchSig))
998                         return false;
999                     vchSig.push_back((unsigned char)nHashType);
1000                     scriptSigRet << vchSig;
1001                 }
1002             }
1003             else if (item.first == OP_PUBKEYHASH)
1004             {
1005                 // Sign and give pubkey
1006                 map<uint160, val_t>::iterator mi = mapPubKeys.find(uint160(item.second));
1007                 if (mi == mapPubKeys.end())
1008                     return false;
1009                 const vector<unsigned char>& vchPubKey = (*mi).second;
1010                 if (!mapKeys.count(vchPubKey))
1011                     return false;
1012                 if (hash != 0)
1013                 {
1014                     vector<unsigned char> vchSig;
1015                     if (!CKey::Sign(mapKeys[vchPubKey], hash, vchSig))
1016                         return false;
1017                     vchSig.push_back((unsigned char)nHashType);
1018                     scriptSigRet << vchSig << vchPubKey;
1019                 }
1020             }
1021         }
1022     }
1023
1024     return true;
1025 }
1026
1027
1028 bool IsMine(const CScript& scriptPubKey)
1029 {
1030     CScript scriptSig;
1031     return Solver(scriptPubKey, 0, 0, scriptSig);
1032 }
1033
1034
1035 bool ExtractPubKey(const CScript& scriptPubKey, bool fMineOnly, vector<unsigned char>& vchPubKeyRet)

```

```

1036 {
1037     vchPubKeyRet.clear();
1038
1039     vector<pair<opcodetype, valtype> > vSolution;
1040     if (!Solver(scriptPubKey, vSolution))
1041         return false;
1042
1043     CRITICAL_BLOCK(cs_mapKeys)
1044     {
1045         foreach(PAIRTYPE(opcodetype, valtype)& item, vSolution)
1046         {
1047             valtype vchPubKey;
1048             if (item.first == OP_PUBKEY)
1049             {
1050                 vchPubKey = item.second;
1051             }
1052             else if (item.first == OP_PUBKEYHASH)
1053             {
1054                 map<uint160, valtype>::iterator mi = mapPubKeys.find(uint160(item.second));
1055                 if (mi == mapPubKeys.end())
1056                     continue;
1057                 vchPubKey = (*mi).second;
1058             }
1059             if (!fMineOnly || mapKeys.count(vchPubKey))
1060             {
1061                 vchPubKeyRet = vchPubKey;
1062                 return true;
1063             }
1064         }
1065     }
1066     return false;
1067 }
1068
1069
1070 bool ExtractHash160(const CScript& scriptPubKey, uint160& hash160Ret)
1071 {
1072     hash160Ret = 0;
1073
1074     vector<pair<opcodetype, valtype> > vSolution;
1075     if (!Solver(scriptPubKey, vSolution))
1076         return false;
1077
1078     foreach(PAIRTYPE(opcodetype, valtype)& item, vSolution)
1079     {
1080         if (item.first == OP_PUBKEYHASH)
1081         {
1082             hash160Ret = uint160(item.second);
1083             return true;
1084         }
1085     }
1086     return false;
1087 }
1088
1089
1090 bool SignSignature(const CTransaction& txFrom, CTransaction& txTo, unsigned int nIn, int nHashType,
1091 CScript scriptPrereq)
1092 {
1093     assert(nIn < txTo.vin.size());
1094     CTxIn& txin = txTo.vin[nIn];
1095     assert(txin.prevout.n < txFrom.vout.size());
1096     const CTxOut& txout = txFrom.vout[txin.prevout.n];
1097
1098     // Leave out the signature from the hash, since a signature can't sign itself.
1099     // The checksig op will also drop the signatures from its hash.
1100     uint256 hash = SignatureHash(scriptPrereq + txout.scriptPubKey, txTo, nIn, nHashType);
1101
1102     if (!Solver(txout.scriptPubKey, hash, nHashType, txin.scriptSig))
1103         return false;
1104
1105     txin.scriptSig = scriptPrereq + txin.scriptSig;
1106
1107     // Test solution

```

```

1107 if (scriptPrereq.empty())
1108 if (!EvalScript(txin.scriptSig + CScript(OP_CODESEPARATOR) + txout.scriptPubKey, txTo, nIn))
1109 return false;
1110 return true;
1111 }
1112 }
1113
1114
1115 bool VerifySignature(const CTransaction& txFrom, const CTransaction& txTo, unsigned int nIn, int
nHashType)
1116 {
1117 assert(nIn < txTo.vin.size());
1118 const CTxIn& txin = txTo.vin[nIn];
1119 if (txin.prevout.n >= txFrom.vout.size())
1120 return false;
1121 const CTxOut& txout = txFrom.vout[txin.prevout.n];
1122
1123 if (txin.prevout.hash != txFrom.GetHash())
1124 return false;
1125
1126 return EvalScript(txin.scriptSig + CScript(OP_CODESEPARATOR) + txout.scriptPubKey, txTo, nIn,
nHashType);
1127 }
1128

```

SUMMARY

The standard Pay-to-Public-Key-Hash (P2PKH) script executes as: OP_DUP
OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG

1. OP_DUP: Duplicate the provided public key
2. OP_HASH160: Hash it to get address
3. OP_EQUALVERIFY: Compare to expected address, fail if mismatch
4. OP_CHECKSIG: Verify signature proves ownership of that public key

This elegance—proving ownership in four operations—is why Satoshi's scripting design has survived 15+ years of attacks.

script.h

598 lines CPP

PROGRAMMABLE MONEY

Bitcoin isn't just a ledger—it includes a complete programming language. This header defines that language: Script, a Forth-like stack-based system for expressing spending conditions. When Satoshi created this, he invented what we now call "programmable money" or "smart contracts"—years before Ethereum popularized the term.

Satoshi explained on BitcoinTalk: "The design supports a tremendous variety of possible transaction types that I designed years ago. Escrow transactions, bonded contracts, third party arbitration, multi-party signature, etc. If Bitcoin catches on in a big way, these are things we'll want to explore in the future, but they all had to be designed at the beginning to make sure they would be possible later." This prescient design meant Bitcoin could evolve without hard forks—just creative use of existing opcodes.

Critically, Script is **intentionally NOT Turing-complete**. There are no loops, no function calls, no unbounded recursion. Why? Because anyone can create a script that every full node must execute. If Script allowed infinite loops, a malicious script could hang every node on the network—a trivial denial-of-service attack. Additionally, Turing-complete contracts are difficult to analyze for bugs; Ethereum's DAO hack demonstrated this danger when \$50 million was stolen through an unintended behavior. Satoshi's conservative design traded expressiveness for security.

The opcodes defined here represent Satoshi's careful balance between power and safety. Each one was included for a reason—and many were later disabled when vulnerabilities emerged. The ghost opcodes (OP_CAT, OP_MUL, etc.) remain in the enum as reminders of what not to enable carelessly.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 class CTransaction;
6
7 enum
8 {
9     SIGHASH_ALL = 1,
10    SIGHASH_NONE = 2,
11    SIGHASH_SINGLE = 3,
12    SIGHASH_ANYONECANPAY = 0x80,
13 };
14
15
16
17 enum opcodetype

```

IMPORTANT: In August 2010, Satoshi disabled 15 opcodes including OP_CAT, OP_MUL, OP_DIV due to DoS vulnerabilities. These remain 'ghost opcodes.'

```

18 {
19 // push value
20     OP_0=0,
21     OP_FALSE=OP_0,
22     OP_PUSHDATA1=76,
23     OP_PUSHDATA2,
24     OP_PUSHDATA4,
25     OP_1NEGATE,
26     OP_RESERVED,
27     OP_1,
28     OP_TRUE=OP_1,
29     OP_2,
30     OP_3,
31     OP_4,
32     OP_5,
33     OP_6,
34     OP_7,
35     OP_8,
36     OP_9,
37     OP_10,
38     OP_11,
39     OP_12,
40     OP_13,
41     OP_14,
42     OP_15,
43     OP_16,
44
45 // control
46     OP_NOP,
47     OP_VER,
48     OP_IF,
49     OP_NOTIF,
50     OP_VERIF,
51     OP_VERNOTIF,
52     OP_ELSE,
53     OP_ENDIF,
54     OP_VERIFY,
55     OP_RETURN,

```

OP_RETURN in v0.1 simply jumps to the end of the script—it does NOT fail. No data size limits existed. Later (2014), OP_RETURN was changed to immediately fail, making outputs provably unspendable—a 'clean' way to embed data without bloating the UTXO set. A 40-byte limit (later 80 bytes) was added as policy, not consensus. This limit reflects an unresolved tension in Bitcoin's identity: is it purely a monetary system, or also a data layer? Attempts to remove or raise these limits periodically reignite debate. The question isn't technical but philosophical—what should Bitcoin be? Satoshi left no guidance; he included OP_RETURN without restriction.

```

56
57 // stack ops
58   OP_TOALTSTACK,
59   OP_FROMALTSTACK,
60   OP_2DROP,
61   OP_2DUP,
62   OP_3DUP,
63   OP_2OVER,
64   OP_2ROT,
65   OP_2SWAP,
66   OP_IFDUP,
67   OP_DEPTH,
68   OP_DROP,
69   OP_DUP,

```

OP_DUP - duplicate top stack item. Combined with OP_HASH160 and OP_EQUALVERIFY, forms P2PKH (Pay to Public Key Hash), Bitcoin's standard script.

```

70   OP_NIP,
71   OP_OVER,
72   OP_PICK,
73   OP_ROLL,
74   OP_ROT,
75   OP_SWAP,
76   OP_TUCK,
77
78 // splice ops
79   OP_CAT,
80   OP_SUBSTR,
81   OP_LEFT,
82   OP_RIGHT,
83   OP_SIZE,
84
85 // bit logic
86   OP_INVERT,
87   OP_AND,
88   OP_OR,
89   OP_XOR,
90   OP_EQUAL,
91   OP_EQUALVERIFY,
92   OP_RESERVED1,
93   OP_RESERVED2,
94
95 // numeric
96   OP_1ADD,
97   OP_1SUB,
98   OP_2MUL,
99   OP_2DIV,
100  OP_NEGATE,
101  OP_ABS,
102  OP_NOT,
103  OP_NOTEQUAL,
104
105  OP_ADD,
106  OP_SUB,
107  OP_MUL,

```

```

108     OP_DIV,
109     OP_MOD,
110     OP_LSHIFT,
111     OP_RSHIFT,
112
113     OP_BOOLAND,
114     OP_BOOLOR,
115     OP_NUMEQUAL,
116     OP_NUMEQUALVERIFY,
117     OP_NUMNOTEQUAL,
118     OP_LESSTHAN,
119     OP_GREATERTHAN,
120     OP_LESSTHANOREQUAL,
121     OP_GREATERTHANOREQUAL,
122     OP_MIN,
123     OP_MAX,
124
125     OP_WITHIN,
126
127 // crypto
128     OP_RIPEMD160,
129     OP_SHA1,
130     OP_SHA256,
131     OP_HASH160,

```

OP_HASH160 - RIPEMD160(SHA256(x)). Creates the 20-byte hash used in Bitcoin addresses. Double hashing adds security margin.

```

132     OP_HASH256,

```

OP_HASH256 - SHA256(SHA256(x)). Double SHA256 used for block hashes and txids. Defense against length extension attacks.

```

133     OP_CODESEPARATOR,
134     OP_CHECKSIG,

```

OP_CHECKSIG - the signature verification opcode. This single instruction is why Bitcoin works: proves private key ownership.

```

135     OP_CHECKSIGVERIFY,

```

OP_CHECKSIGVERIFY - CHECKSIG + VERIFY. Fails script immediately if signature invalid. Used in multi-sig scripts.

```

136     OP_CHECKMULTISIG,

```

OP_CHECKMULTISIG - M-of-N multi-signature verification, present from day one. Requires M valid signatures from a set of N public keys. Enables shared custody: a company treasury might require 2-of-3 executives to sign, or an escrow might need 2-of-3 (buyer, seller, arbitrator). Satoshi mentioned 'multi-party signature' in his BitcoinTalk posts as a planned feature. The implementation (script.cpp:727) iterates through signatures and keys, matching each signature to a key in order. A subtle bug existed: an extra unused value was consumed from the stack, fixed years later in BIP 147.

```

137     OP_CHECKMULTISIGVERIFY,
138
139
140 // multi-byte opcodes

```

```

141     OP_SINGLEBYTE_END = 0xF0,
142     OP_DOUBLEBYTE_BEGIN = 0xF000,
143
144 // template matching params
145     OP_PUBKEY,
146     OP_PUBKEYHASH,
147
148
149
150     OP_INVALIDOPCODE = 0xFFFF,
151 };
152
153
154
155
156
157
158
159
160 inline const char* GetOpName(opcodetype opcode)
161 {
162     switch (opcode)
163     {
164 // push value
165     case OP_0:                : return "0";
166     case OP_PUSHDATA1:        : return "OP_PUSHDATA1";
167     case OP_PUSHDATA2:        : return "OP_PUSHDATA2";
168     case OP_PUSHDATA4:        : return "OP_PUSHDATA4";
169     case OP_1NEGATE:          : return "-1";
170     case OP_RESERVED:         : return "OP_RESERVED";
171     case OP_1:                : return "1";
172     case OP_2:                : return "2";
173     case OP_3:                : return "3";
174     case OP_4:                : return "4";
175     case OP_5:                : return "5";
176     case OP_6:                : return "6";
177     case OP_7:                : return "7";
178     case OP_8:                : return "8";
179     case OP_9:                : return "9";
180     case OP_10:               : return "10";
181     case OP_11:               : return "11";
182     case OP_12:               : return "12";
183     case OP_13:               : return "13";
184     case OP_14:               : return "14";
185     case OP_15:               : return "15";
186     case OP_16:               : return "16";
187
188 // control
189     case OP_NOP:              : return "OP_NOP";
190     case OP_VER:              : return "OP_VER";
191     case OP_IF:               : return "OP_IF";
192     case OP_NOTIF:            : return "OP_NOTIF";
193     case OP_VERIF:            : return "OP_VERIF";
194     case OP_VERNOTIF:         : return "OP_VERNOTIF";
195     case OP_ELSE:             : return "OP_ELSE";
196     case OP_ENDIF:            : return "OP_ENDIF";
197     case OP_VERIFY:           : return "OP_VERIFY";
198     case OP_RETURN:           : return "OP_RETURN";
199
200 // stack ops
201     case OP_TOALTSTACK:       : return "OP_TOALTSTACK";
202     case OP_FROMALTSTACK:     : return "OP_FROMALTSTACK";
203     case OP_2DROP:           : return "OP_2DROP";
204     case OP_2DUP:            : return "OP_2DUP";
205     case OP_3DUP:            : return "OP_3DUP";
206     case OP_2OVER:           : return "OP_2OVER";
207     case OP_2ROT:            : return "OP_2ROT";
208     case OP_2SWAP:           : return "OP_2SWAP";
209     case OP_IFDUP:           : return "OP_IFDUP";
210     case OP_DEPTH:           : return "OP_DEPTH";
211     case OP_DROP:            : return "OP_DROP";
212     case OP_DUP:             : return "OP_DUP";

```

```

213 case OP_NIP                : return "OP_NIP";
214 case OP_OVER               : return "OP_OVER";
215 case OP_PICK               : return "OP_PICK";
216 case OP_ROLL               : return "OP_ROLL";
217 case OP_ROT                : return "OP_ROT";
218 case OP_SWAP               : return "OP_SWAP";
219 case OP_TUCK                : return "OP_TUCK";
220
221 // splice ops
222 case OP_CAT                 : return "OP_CAT";
223 case OP_SUBSTR              : return "OP_SUBSTR";
224 case OP_LEFT                : return "OP_LEFT";
225 case OP_RIGHT               : return "OP_RIGHT";
226 case OP_SIZE                : return "OP_SIZE";
227
228 // bit logic
229 case OP_INVERT              : return "OP_INVERT";
230 case OP_AND                 : return "OP_AND";
231 case OP_OR                  : return "OP_OR";
232 case OP_XOR                 : return "OP_XOR";
233 case OP_EQUAL               : return "OP_EQUAL";
234 case OP_EQUALVERIFY         : return "OP_EQUALVERIFY";
235 case OP_RESERVED1           : return "OP_RESERVED1";
236 case OP_RESERVED2           : return "OP_RESERVED2";
237
238 // numeric
239 case OP_1ADD                : return "OP_1ADD";
240 case OP_1SUB                : return "OP_1SUB";
241 case OP_2MUL                : return "OP_2MUL";
242 case OP_2DIV                : return "OP_2DIV";
243 case OP_NEGATE               : return "OP_NEGATE";
244 case OP_ABS                 : return "OP_ABS";
245 case OP_NOT                 : return "OP_NOT";
246 case OP_0NOTEQUAL           : return "OP_0NOTEQUAL";
247 case OP_ADD                 : return "OP_ADD";
248 case OP_SUB                 : return "OP_SUB";
249 case OP_MUL                 : return "OP_MUL";
250 case OP_DIV                 : return "OP_DIV";
251 case OP_MOD                 : return "OP_MOD";
252 case OP_LSHIFT              : return "OP_LSHIFT";
253 case OP_RSHIFT              : return "OP_RSHIFT";
254 case OP_BOOLAND              : return "OP_BOOLAND";
255 case OP_BOOLOR              : return "OP_BOOLOR";
256 case OP_NUMEQUAL            : return "OP_NUMEQUAL";
257 case OP_NUMEQUALVERIFY      : return "OP_NUMEQUALVERIFY";
258 case OP_NUMNOTEQUAL         : return "OP_NUMNOTEQUAL";
259 case OP_LESSTHAN            : return "OP_LESSTHAN";
260 case OP_GREATERTHAN         : return "OP_GREATERTHAN";
261 case OP_LESSTHANOREQUAL     : return "OP_LESSTHANOREQUAL";
262 case OP_GREATERTHANOREQUAL  : return "OP_GREATERTHANOREQUAL";
263 case OP_MIN                  : return "OP_MIN";
264 case OP_MAX                  : return "OP_MAX";
265 case OP_WITHIN               : return "OP_WITHIN";
266
267 // crypto
268 case OP_RIPEMD160           : return "OP_RIPEMD160";
269 case OP_SHA1                 : return "OP_SHA1";
270 case OP_SHA256               : return "OP_SHA256";
271 case OP_HASH160              : return "OP_HASH160";
272 case OP_HASH256              : return "OP_HASH256";
273 case OP_CODESEPARATOR        : return "OP_CODESEPARATOR";
274 case OP_CHECKSIG             : return "OP_CHECKSIG";
275 case OP_CHECKSIGVERIFY       : return "OP_CHECKSIGVERIFY";
276 case OP_CHECKMULTISIG        : return "OP_CHECKMULTISIG";
277 case OP_CHECKMULTISIGVERIFY  : return "OP_CHECKMULTISIGVERIFY";
278
279
280
281 // multi-byte opcodes
282 case OP_SINGLEBYTE_END       : return "OP_SINGLEBYTE_END";
283 case OP_DOUBLEBYTE_BEGIN     : return "OP_DOUBLEBYTE_BEGIN";
284 case OP_PUBKEY                : return "OP_PUBKEY";

```

```

285 case OP_PUBKEYHASH          : return "OP_PUBKEYHASH";
286
287
288
289 case OP_INVALIDOPCODE       : return "OP_INVALIDOPCODE";
290 default:
291 return "UNKNOWN_OPCODE";
292 }
293 };
294
295
296
297
298 inline string ValueString(const vector<unsigned char>& vch)
299 {
300 if (vch.size() <= 4)
301 return sprintf("%d", CBigNum(vch).getint());
302 else
303 return HexNumStr(vch.begin(), vch.end());
304 //return string("(" + HexStr(vch.begin(), vch.end()) + string(")");
305 }
306
307 inline string StackString(const vector<vector<unsigned char> >& vStack)
308 {
309     string str;
310     foreach(const vector<unsigned char>& vch, vStack)
311     {
312         if (!str.empty())
313             str += " ";
314         str += ValueString(vch);
315     }
316     return str;
317 }
318
319
320
321
322
323
324
325
326
327 class CScript : public vector<unsigned char>

```

CScript class - a Bitcoin script as bytes. Inherits from vector. Scripts are data until evaluated.

```

328 {
329 protected:
330     CScript& push_int64(int64 n)
331     {
332         if (n == -1 || (n >= 1 && n <= 16))
333         {
334             push_back(n + (OP_1 - 1));
335         }
336         else
337         {
338             CBigNum bn(n);
339             *this << bn.getvch();
340         }
341         return (*this);
342     }
343
344     CScript& push_uint64(uint64 n)
345     {
346         if (n == -1 || (n >= 1 && n <= 16))
347         {
348             push_back(n + (OP_1 - 1));
349         }
350         else

```

```

351 {
352     CBigNum bn(n);
353     *this << bn.getvch();
354 }
355 return (*this);
356 }
357
358 public:
359 CScript() { }
360 CScript(const CScript& b) : vector<unsigned char>(b.begin(), b.end()) { }
361 CScript(const_iterator pbegin, const_iterator pend) : vector<unsigned char>(pbegin, pend) { }
362 #ifndef _MSC_VER
363 CScript(const unsigned char* pbegin, const unsigned char* pend) : vector<unsigned char>(pbegin, pend) { }
364 #endif
365
366 CScript& operator+=(const CScript& b)
367 {
368     insert(end(), b.begin(), b.end());
369     return *this;
370 }
371
372 friend CScript operator+(const CScript& a, const CScript& b)
373 {
374     CScript ret = a;
375     ret += b;
376     return (ret);
377 }
378
379
380 explicit CScript(char b) { operator<<(b); }
381 explicit CScript(short b) { operator<<(b); }
382 explicit CScript(int b) { operator<<(b); }
383 explicit CScript(long b) { operator<<(b); }
384 explicit CScript(int64 b) { operator<<(b); }
385 explicit CScript(unsigned char b) { operator<<(b); }
386 explicit CScript(unsigned int b) { operator<<(b); }
387 explicit CScript(unsigned short b) { operator<<(b); }
388 explicit CScript(unsigned long b) { operator<<(b); }
389 explicit CScript(uint64 b) { operator<<(b); }
390
391 explicit CScript(opcodetype b) { operator<<(b); }
392 explicit CScript(const uint256& b) { operator<<(b); }
393 explicit CScript(const CBigNum& b) { operator<<(b); }
394 explicit CScript(const vector<unsigned char>& b) { operator<<(b); }
395
396
397 CScript& operator<<(char b) { return (push_int64(b)); }
398 CScript& operator<<(short b) { return (push_int64(b)); }
399 CScript& operator<<(int b) { return (push_int64(b)); }
400 CScript& operator<<(long b) { return (push_int64(b)); }
401 CScript& operator<<(int64 b) { return (push_int64(b)); }
402 CScript& operator<<(unsigned char b) { return (push_uint64(b)); }
403 CScript& operator<<(unsigned int b) { return (push_uint64(b)); }
404 CScript& operator<<(unsigned short b) { return (push_uint64(b)); }
405 CScript& operator<<(unsigned long b) { return (push_uint64(b)); }
406 CScript& operator<<(uint64 b) { return (push_uint64(b)); }
407
408 CScript& operator<<(opcodetype opcode)
409 {
410     if (opcode <= OP_SINGLEBYTE_END)
411     {
412         insert(end(), (unsigned char)opcode);
413     }
414     else
415     {
416         assert(opcode >= OP_DOUBLEBYTE_BEGIN);
417         insert(end(), (unsigned char)(opcode >> 8));
418         insert(end(), (unsigned char)(opcode & 0xFF));
419     }
420     return (*this);
421 }

```



```

422
423     CScript& operator<<(const uint160& b)
424 {
425     insert(end(), sizeof(b));
426     insert(end(), (unsigned char*)&b, (unsigned char*)&b + sizeof(b));
427     return (*this);
428 }
429
430     CScript& operator<<(const uint256& b)
431 {
432     insert(end(), sizeof(b));
433     insert(end(), (unsigned char*)&b, (unsigned char*)&b + sizeof(b));
434     return (*this);
435 }
436
437     CScript& operator<<(const CBigNum& b)
438 {
439     *this << b.getvch();
440     return (*this);
441 }
442
443     CScript& operator<<(const vector<unsigned char>& b)
444 {
445     if (b.size() < OP_PUSHDATA1)
446     {
447         insert(end(), (unsigned char)b.size());
448     }
449     else if (b.size() <= 0xff)
450     {
451         insert(end(), OP_PUSHDATA1);
452         insert(end(), (unsigned char)b.size());
453     }
454     else
455     {
456         insert(end(), OP_PUSHDATA2);
457         unsigned short nSize = b.size();
458         insert(end(), (unsigned char*)&nSize, (unsigned char*)&nSize + sizeof(nSize));
459     }
460     insert(end(), b.begin(), b.end());
461     return (*this);
462 }
463
464     CScript& operator<<(const CScript& b)
465 {
466     // I'm not sure if this should push the script or concatenate scripts.
467     // If there's ever a use for pushing a script onto a script, delete this member fn
468     assert(("warning: pushing a CScript onto a CScript with << is probably not intended, use + to concatenate", false));
469     return (*this);
470 }
471
472
473 bool GetOp(iterator& pc, opcodeType& opcodeRet, vector<unsigned char>& vchRet)
474 {
475     // This is why people hate C++
476     const_iterator pc2 = pc;
477     bool fRet = GetOp(pc2, opcodeRet, vchRet);
478     pc = begin() + (pc2 - begin());
479     return fRet;
480 }
481
482 bool GetOp(const_iterator& pc, opcodeType& opcodeRet, vector<unsigned char>& vchRet) const
483 {
484     opcodeRet = OP_INVALIDOPCODE;
485     vchRet.clear();
486     if (pc >= end())
487         return false;
488
489     // Read instruction
490     unsigned int opcode = *pc++;
491     if (opcode >= OP_SINGLEBYTE_END)
492 {

```

```

493 if (pc + 1 > end())
494 return false;
495     opcode <= 8;
496     opcode |= *pc++;
497 }
498
499 // Immediate operand
500 if (opcode <= OP_PUSHDATA4)
501 {
502     unsigned int nSize = opcode;
503     if (opcode == OP_PUSHDATA1)
504     {
505         if (pc + 1 > end())
506             return false;
507         nSize = *pc++;
508     }
509     else if (opcode == OP_PUSHDATA2)
510     {
511         if (pc + 2 > end())
512             return false;
513         nSize = 0;
514         memcpy(&nSize, &pc[0], 2);
515         pc += 2;
516     }
517     else if (opcode == OP_PUSHDATA4)
518     {
519         if (pc + 4 > end())
520             return false;
521         memcpy(&nSize, &pc[0], 4);
522         pc += 4;
523     }
524     if (pc + nSize > end())
525         return false;
526     vchRet.assign(pc, pc + nSize);
527     pc += nSize;
528 }
529
530     opcodeRet = (opcodeType)opcode;
531 return true;
532 }
533
534 void FindAndDelete(const CScript& b)
535 {
536     iterator pc = begin();
537     opcodeType opcode;
538     vector<unsigned char> vchPushValue;
539     int count = 0;
540 do
541 {
542     while (end() - pc >= b.size() && memcmp(&pc[0], &b[0], b.size()) == 0)
543     {
544         erase(pc, pc + b.size());
545         count++;
546     }
547 }
548 while (GetOp(pc, opcode, vchPushValue));
549 //printf("FindAndDeleted deleted %d items\n", count); /// debug
550 }
551
552
553 void PrintHex() const
554 {
555     printf("CScript(%s)\n", HexStr(begin(), end()).c_str());
556 }
557
558 string ToString() const
559 {
560     string str;
561     opcodeType opcode;
562     vector<unsigned char> vch;
563     const_iterator it = begin();

```

```

565 while (GetOp(it, opcode, vch))
566 {
567     if (!str.empty())
568         str += " ";
569     if (opcode <= OP_PUSHDATA4)
570         str += ValueString(vch);
571     else
572         str += GetOpName(opcode);
573 }
574 return str;
575 }
576
577 void print() const
578 {
579     printf("%s\n", ToString().c_str());
580 }
581 };
582
583
584
585
586
587
588
589
590 bool EvalScript(const CScript& script, const CTransaction& txTo, unsigned int nIn, int nHashType=0,
591               vector<vector<unsigned char> >> pvStackRet=NULL);
592 uint256 SignatureHash(CScript scriptCode, const CTransaction& txTo, unsigned int nIn, int nHashType);
593 bool IsMine(const CScript& scriptPubKey);
594 bool ExtractPubKey(const CScript& scriptPubKey, bool fMineOnly, vector<unsigned char>& vchPubKeyRet);
595 bool ExtractHash160(const CScript& scriptPubKey, uint160& hash160Ret);
596 bool SignSignature(const CTransaction& txFrom, CTransaction& txTo, unsigned int nIn, int
597                   nHashType=SIGHASH_ALL, CScript scriptPrereq=CScript());
598 bool VerifySignature(const CTransaction& txFrom, const CTransaction& txTo, unsigned int nIn, int
599                   nHashType=0);

```

SUMMARY

Script's intentional limitations are features, not bugs. By avoiding Turing-completeness, Satoshi ensured scripts always terminate and have bounded resource usage. This makes validation predictable and prevents the halting problem that plagues other smart contract platforms.

The August 2010 opcode disabling (commit 4bd188c) is forensically significant: Satoshi identified that OP_CAT + OP_DUP could create 1TB stack elements, crashing nodes. His quick response showed active security maintenance.

serialize.h

1152 lines CPP

SPEAKING IN BYTES

For Bitcoin nodes to communicate, they need a common language for encoding data. This file defines Bitcoin's serialization system—the binary format for transactions, blocks, and network messages. At 39KB, it's one of the largest files in v0.1, reflecting the complexity of making C++ objects into portable byte streams.

Satoshi designed a custom format optimized for compactness. Variable-length integers ("CompactSize") save bytes for common small values. The format is deterministic: same data always produces same bytes, critical for hashing. Without determinism, two nodes serializing the same transaction could compute different hashes—catastrophic for consensus.

The ingenious `IMPLEMENT_SERIALIZE` macro is a masterclass in C++ metaprogramming. You write the serialization logic once, and the macro generates three functions: one to calculate size, one to write bytes, one to read bytes. The same code handles both directions, eliminating the bugs that plague separate encode/decode implementations.

This serialization format is now a de facto standard. Every Bitcoin implementation—in C++, Rust, Go, Python, JavaScript—must match it exactly. A single byte difference causes consensus failure. The format hasn't changed since v0.1—testament to Satoshi's foresight in designing for permanence.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include <vector>
6 #include <map>
7 #include <boost/type_traits/is_fundamental.hpp>
8 #if defined(_MSC_VER) || defined(__BORLANDC__)
9 typedef __int64 int64;
10 typedef unsigned __int64 uint64;
11 #else
12 typedef long long int64;
13 typedef unsigned long long uint64;
14 #endif
15 #if defined(_MSC_VER) && _MSC_VER < 1300
16 #define for if (false) ; else for
17 #endif
18 class CScript;
19 class CDataStream;
20 class CAutoFile;
21
22 static const int VERSION = 101;

```

VERSION = 101—the protocol version embedded in serialized data. This allows format evolution: when reading old data, code can handle legacy formats. Version 101 is v0.1.0 (first digit is major, last two are minor). This version is included in network messages and stored data for compatibility checking.

```

23
24
25
26
27
28 //////////////////////////////////////////////////
29 //
30 // Templates for serializing to anything that looks like a stream,
31 // i.e. anything that supports .read(char*, int) and .write(char*, int)
32 //
33
34 enum

```

Serialization context flags. `SER_NETWORK` means data going over the wire—includes all fields. `SER_DISK` means local storage—may include extra metadata. `SER_GETHASH` means computing a hash—excludes signatures (for txid calculation). The modifier `SER_SKIPSIG` omits signatures; `SER_BLOCKHEADERONLY` serializes just the 80-byte header without transactions.

```

35 {
36 // primary actions
37     SER_NETWORK      = (1 << 0),
38     SER_DISK         = (1 << 1),
39     SER_GETHASH      = (1 << 2),
40
41 // modifiers
42     SER_SKIPSIG      = (1 << 16),
43     SER_BLOCKHEADERONLY = (1 << 17),
44 };
45
46 #define IMPLEMENT_SERIALIZE(statements)\

```

IMPLEMENT_SERIALIZE —the heart of Bitcoin's serialization. This macro generates three methods from one block of statements: (1) `GetSerializeSize()` counts bytes without writing, (2) `Serialize()` writes to a stream, (3) `Unserialize()` reads from a stream. The magic is in the boolean flags (`fGetSize`, `fWrite`, `fRead`) and the `READWRITE` macro that checks them.

```

47 unsigned int GetSerializeSize(int nType=0, int nVersion=VERSION) const \
48 {
49     CSerActionGetSerializeSize ser_action; \
50     const bool fGetSize = true; \
51     const bool fWrite = false; \
52     const bool fRead = false; \
53     unsigned int nSerSize = 0; \
54     ser_streamplaceholder s; \

```

`ser_streamplaceholder` —a dummy stream for size calculation. When counting bytes, we don't need an actual output buffer. This placeholder satisfies the template but does nothing. The `nType` and `nVersion` are passed through for context-aware sizing (e.g., different versions might serialize differently).

```

55     s.nType = nType; \
56     s.nVersion = nVersion; \
57 {statements} \
58 return nSerSize; \
59 } \
60 template<typename Stream> \
61 void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const \
62 { \
63     CSerActionSerialize ser_action; \
64     const bool fGetSize = false; \
65     const bool fWrite = true; \
66     const bool fRead = false; \
67     unsigned int nSerSize = 0; \
68 {statements} \
69 } \
70 template<typename Stream> \
71 void Unserialize(Stream& s, int nType=0, int nVersion=VERSION) \
72 { \
73     CSerActionUnserialize ser_action; \
74     const bool fGetSize = false; \
75     const bool fWrite = false; \
76     const bool fRead = true; \
77     unsigned int nSerSize = 0; \
78 {statements} \
79 } \
80 \
81 #define READWRITE(obj) (nSerSize += ::SerReadWrite(s, (obj), nType, nVersion, ser_action))

```

`READWRITE(obj)` —the polymorphic serialization operator. Despite its name, it does three things depending on context: calculates size, writes, or reads. The `::SerReadWrite()` function dispatches based on `ser_action`'s type. This single macro handles integers, strings, vectors, maps, and any custom type with `IMPLEMENT_SERIALIZE`. The elegance: you write `READWRITE(nVersion)`; `READWRITE(vTxIn)`; `READWRITE(vTxOut)`; and it just works for all three operations.

```

82
83
84
85
86
87
88 //

```

```

89 // Basic types
90 //
91 #define WRITEDATA(s, obj)    s.write((char*)&(obj), sizeof(obj))

```

Raw data macros: `WRITEDATA` and `READDATA`. These directly read/write bytes without interpretation. Used for primitive types where the in-memory representation matches the serialized format. The `(char*)&(obj)` cast gets a byte pointer; `sizeof(obj)` determines how many bytes. No endianness conversion—Bitcoin uses native byte order (usually little-endian on modern hardware).

```

92 #define READDATA(s, obj)    s.read((char*)&(obj), sizeof(obj))
93
94 inline unsigned int GetSerializeSize(char a, int, int=0) { return sizeof(a); }

```

Primitive type serialization—overloaded for every basic type. Each `GetSerializeSize()` returns `sizeof(type)`; each `Serialize()` calls `WRITEDATA`; each `Unserialize()` calls `READDATA`. The repetitive overloads ensure type safety: the compiler selects the right version based on argument type. This is verbose but explicit—no surprises about how data is encoded.

```

95 inline unsigned int GetSerializeSize(signed char a, int, int=0) { return sizeof(a); }
96 inline unsigned int GetSerializeSize(unsigned char a, int, int=0) { return sizeof(a); }
97 inline unsigned int GetSerializeSize(signed short a, int, int=0) { return sizeof(a); }
98 inline unsigned int GetSerializeSize(unsigned short a, int, int=0) { return sizeof(a); }
99 inline unsigned int GetSerializeSize(signed int a, int, int=0) { return sizeof(a); }
100 inline unsigned int GetSerializeSize(unsigned int a, int, int=0) { return sizeof(a); }
101 inline unsigned int GetSerializeSize(signed long a, int, int=0) { return sizeof(a); }
102 inline unsigned int GetSerializeSize(unsigned long a, int, int=0) { return sizeof(a); }
103 inline unsigned int GetSerializeSize(int64 a, int, int=0) { return sizeof(a); }
104 inline unsigned int GetSerializeSize(uint64 a, int, int=0) { return sizeof(a); }
105 inline unsigned int GetSerializeSize(float a, int, int=0) { return sizeof(a); }
106 inline unsigned int GetSerializeSize(double a, int, int=0) { return sizeof(a); }
107
108 template<typename Stream> inline void Serialize(Stream& s, char a, int, int=0) { WRITEDATA(s, a); }
109 template<typename Stream> inline void Serialize(Stream& s, signed char a, int, int=0) { WRITEDATA(s, a); }
110 template<typename Stream> inline void Serialize(Stream& s, unsigned char a, int, int=0) { WRITEDATA(s, a); }
111 template<typename Stream> inline void Serialize(Stream& s, signed short a, int, int=0) { WRITEDATA(s, a); }
112 template<typename Stream> inline void Serialize(Stream& s, unsigned short a, int, int=0) { WRITEDATA(s, a); }
113 template<typename Stream> inline void Serialize(Stream& s, signed int a, int, int=0) { WRITEDATA(s, a); }
114 template<typename Stream> inline void Serialize(Stream& s, unsigned int a, int, int=0) { WRITEDATA(s, a); }
115 template<typename Stream> inline void Serialize(Stream& s, signed long a, int, int=0) { WRITEDATA(s, a); }
116 template<typename Stream> inline void Serialize(Stream& s, unsigned long a, int, int=0) { WRITEDATA(s, a); }
117 template<typename Stream> inline void Serialize(Stream& s, int64 a, int, int=0) { WRITEDATA(s, a); }
118 template<typename Stream> inline void Serialize(Stream& s, uint64 a, int, int=0) { WRITEDATA(s, a); }
119 template<typename Stream> inline void Serialize(Stream& s, float a, int, int=0) { WRITEDATA(s, a); }
120 template<typename Stream> inline void Serialize(Stream& s, double a, int, int=0) { WRITEDATA(s, a); }
121
122 template<typename Stream> inline void Unserialize(Stream& s, char& a, int, int=0) { READDATA(s, a); }
123 template<typename Stream> inline void Unserialize(Stream& s, signed char& a, int, int=0) { READDATA(s, a); }
124 template<typename Stream> inline void Unserialize(Stream& s, unsigned char& a, int, int=0) { READDATA(s, a); }
125 template<typename Stream> inline void Unserialize(Stream& s, signed short& a, int, int=0) { READDATA(s, a); }
126 template<typename Stream> inline void Unserialize(Stream& s, unsigned short& a, int, int=0) { READDATA(s, a); }
127 template<typename Stream> inline void Unserialize(Stream& s, signed int& a, int, int=0) { READDATA(s, a); }

```

```

128 template<typename Stream> inline void Unserialize(Stream& s, unsigned int& a, int, int=0) { READDATA(s,
a); }
129 template<typename Stream> inline void Unserialize(Stream& s, signed long& a, int, int=0) { READDATA(s,
a); }
130 template<typename Stream> inline void Unserialize(Stream& s, unsigned long& a, int, int=0) { READDATA(s,
a); }
131 template<typename Stream> inline void Unserialize(Stream& s, int64& a, int, int=0) { READDATA(s, a); }
132 template<typename Stream> inline void Unserialize(Stream& s, uint64& a, int, int=0) { READDATA(s, a); }
133 template<typename Stream> inline void Unserialize(Stream& s, float& a, int, int=0) { READDATA(s, a); }
134 template<typename Stream> inline void Unserialize(Stream& s, double& a, int, int=0) { READDATA(s, a); }
135
136 inline unsigned int GetSerializeSize(bool a, int, int=0) { return sizeof(char); }
    
```

Boolean serialization—stored as a single byte. Note: `true` becomes `1`, `false` becomes `0`. When reading, any non-zero byte becomes `true`. This matches C's truthiness convention but means serialized bools are always `0x00` or `0x01`—never garbage values. Important for deterministic serialization.

```

137 template<typename Stream> inline void Serialize(Stream& s, bool a, int, int=0) { char f=a; WRITEDATA(s,
f); }
138 template<typename Stream> inline void Unserialize(Stream& s, bool& a, int, int=0) { char f; READDATA(s,
f); a=f; }
139
140
141
142
143
144
145 //
    
```

CompactSize format—Satoshi's comment explains the encoding. Most values in Bitcoin are small: transaction input counts, output counts, script lengths. Using 4 bytes for a count that's usually 1-10 wastes space. CompactSize uses 1 byte for values 0-252, growing only when needed. This saves millions of bytes across the blockchain.

```

146 // Compact size
147 // size < 253          -- 1 byte
148 // size <= USHRT_MAX   -- 3 bytes  (253 + 2 bytes)
149 // size <= UINT_MAX    -- 5 bytes  (254 + 4 bytes)
150 // size >  UINT_MAX    -- 9 bytes  (255 + 8 bytes)
151 //
152 inline unsigned int GetSizeOfCompactSize(uint64 nSize)
    
```

`GetSizeOfCompactSize()` —returns how many bytes a CompactSize will occupy. Essential for pre-calculating buffer sizes. The thresholds: <253 uses 1 byte, <65536 uses 3 bytes (marker + 2-byte value), <4 billion uses 5 bytes, larger uses 9 bytes. Most transaction data stays in the 1-byte range.

```

153 {
154     if (nSize < UCHAR_MAX-2) return sizeof(unsigned char);
155     else if (nSize <= USHRT_MAX) return sizeof(unsigned char) + sizeof(unsigned short);
156     else if (nSize <= UINT_MAX) return sizeof(unsigned char) + sizeof(unsigned int);
157     else return sizeof(unsigned char) + sizeof(uint64);
158 }
159
160 template<typename Stream>
161 void WriteCompactSize(Stream& os, uint64 nSize)
    
```

`WriteCompactSize()` —encodes an integer in minimum bytes. The marker bytes 253, 254, 255 signal 'next 2/4/8 bytes contain the actual value.' This self-describing format means you can parse without external length information. Example: the number 500 serializes as 0xFD 0xF4 0x01 (253 marker, then 500 in little-endian).

```

162 {
163 if (nSize < UCHAR_MAX-2)
164 {
165     unsigned char chSize = nSize;
166     WRITEDATA(os, chSize);
167 }
168 else if (nSize <= USHRT_MAX)
169 {
170     unsigned char chSize = UCHAR_MAX-2;
171     unsigned short xSize = nSize;
172     WRITEDATA(os, chSize);
173     WRITEDATA(os, xSize);
174 }
175 else if (nSize <= UINT_MAX)
176 {
177     unsigned char chSize = UCHAR_MAX-1;
178     unsigned int xSize = nSize;
179     WRITEDATA(os, chSize);
180     WRITEDATA(os, xSize);
181 }
182 else
183 {
184     unsigned char chSize = UCHAR_MAX;
185     WRITEDATA(os, chSize);
186     WRITEDATA(os, nSize);
187 }
188 return;
189 }
190
191 template<typename Stream>

```

`ReadCompactSize()` —decodes a CompactSize integer. Reads one byte; if <253, that's the value. If 253 / 254 / 255, reads 2/4/8 more bytes for the actual value. The return type is `uint64`—large enough for any CompactSize. Callers typically cast to appropriate smaller types after validation.

```

192 uint64 ReadCompactSize(Stream& is)
193 {
194     unsigned char chSize;
195     READDATA(is, chSize);
196     if (chSize < UCHAR_MAX-2)
197     {
198         return chSize;
199     }
200     else if (chSize == UCHAR_MAX-2)
201     {
202         unsigned short nSize;
203         READDATA(is, nSize);
204         return nSize;
205     }
206     else if (chSize == UCHAR_MAX-1)
207     {
208         unsigned int nSize;
209         READDATA(is, nSize);
210         return nSize;
211     }
212     else
213     {
214         uint64 nSize;
215         READDATA(is, nSize);

```

```

216 return nSize;
217 }
218 }
219
220
221
222 //
223 // Wrapper for serializing arrays and POD
224 // There's a clever template way to make arrays serialize normally, but MSVC6 doesn't support it
225 //
226 #define FLATDATA(obj) REF(CFlatData((char*)&(obj), (char*)&(obj) + sizeof(obj)))
227 class CFlatData
228 {
229 protected:
230 char* pbegin;
231 char* pend;
232 public:
233 CFlatData(void* pbeginIn, void* pendIn) : pbegin((char*)pbeginIn), pend((char*)pendIn) { }
234 char* begin() { return pbegin; }
235 const char* begin() const { return pbegin; }
236 char* end() { return pend; }
237 const char* end() const { return pend; }
238
239 unsigned int GetSerializeSize(int, int=0) const
240 {
241 return pend - pbegin;
242 }
243
244 template<typename Stream>
245 void Serialize(Stream& s, int, int=0) const
246 {
247     s.write(pbegin, pend - pbegin);
248 }
249
250 template<typename Stream>
251 void Unserialize(Stream& s, int, int=0)
252 {
253     s.read(pbegin, pend - pbegin);
254 }
255 };
256
257
258
259 //
260 // string stored as a fixed length field
261 //
262 template<std::size_t LEN>
263 class CFixedFieldString
264 {
265 protected:
266 const string* pcstr;
267 string* pstr;
268 public:
269 explicit CFixedFieldString(const string& str) : pcstr(&str), pstr(NULL) { }
270 explicit CFixedFieldString(string& str) : pcstr(&str), pstr(&str) { }
271
272 unsigned int GetSerializeSize(int, int=0) const
273 {
274 return LEN;
275 }
276
277 template<typename Stream>
278 void Serialize(Stream& s, int, int=0) const
279 {
280 char pszBuf[LEN];
281 strncpy(pszBuf, pcstr->c_str(), LEN);
282     s.write(pszBuf, LEN);
283 }
284
285 template<typename Stream>
286 void Unserialize(Stream& s, int, int=0)
287 {

```

```

288 if (pstr == NULL)
289 throw std::ios_base::failure("CFixedFieldString::Unserialize : trying to unserialize to const string");
290 char pszBuf[LEN+1];
291     s.read(pszBuf, LEN);
292     pszBuf[LEN] = '\0';
293 *pstr = pszBuf;
294 }
295 };
296
297
298
299
300
301 //
302 // Forward declarations
303 //
304
305 // string
306 template<typename C> unsigned int GetSerializeSize(const basic_string<C>& str, int, int=0);
307 template<typename Stream, typename C> void Serialize(Stream& os, const basic_string<C>& str, int,
    int=0);
308 template<typename Stream, typename C> void Unserialize(Stream& is, basic_string<C>& str, int, int=0);
309
310 // vector
311 template<typename T, typename A> unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int
    nType, int nVersion, const boost::true_type&);
312 template<typename T, typename A> unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int
    nType, int nVersion, const boost::false_type&);
313 template<typename T, typename A> inline unsigned int GetSerializeSize(const std::vector<T, A>& v, int
    nType, int nVersion=VERSION);
314 template<typename Stream, typename T, typename A> void Serialize_impl(Stream& os, const std::vector<T,
    A>& v, int nType, int nVersion, const boost::true_type&);
315 template<typename Stream, typename T, typename A> void Serialize_impl(Stream& os, const std::vector<T,
    A>& v, int nType, int nVersion, const boost::false_type&);
316 template<typename Stream, typename T, typename A> inline void Serialize(Stream& os, const std::vector<T,
    A>& v, int nType, int nVersion=VERSION);
317 template<typename Stream, typename T, typename A> void Unserialize_impl(Stream& is, std::vector<T, A>&
    v, int nType, int nVersion, const boost::true_type&);
318 template<typename Stream, typename T, typename A> void Unserialize_impl(Stream& is, std::vector<T, A>&
    v, int nType, int nVersion, const boost::false_type&);
319 template<typename Stream, typename T, typename A> inline void Unserialize(Stream& is, std::vector<T, A>&
    v, int nType, int nVersion=VERSION);
320
321 // others derived from vector
322 extern inline unsigned int GetSerializeSize(const CScript& v, int nType, int nVersion=VERSION);
323 template<typename Stream> void Serialize(Stream& os, const CScript& v, int nType, int nVersion=VERSION);
324 template<typename Stream> void Unserialize(Stream& is, CScript& v, int nType, int nVersion=VERSION);
325
326 // pair
327 template<typename K, typename T> unsigned int GetSerializeSize(const std::pair<K, T>& item, int nType,
    int nVersion=VERSION);
328 template<typename Stream, typename K, typename T> void Serialize(Stream& os, const std::pair<K, T>&
    item, int nType, int nVersion=VERSION);
329 template<typename Stream, typename K, typename T> void Unserialize(Stream& is, std::pair<K, T>& item,
    int nType, int nVersion=VERSION);
330
331 // map
332 template<typename K, typename T, typename Pred, typename A> unsigned int GetSerializeSize(const
    std::map<K, T, Pred, A>& m, int nType, int nVersion=VERSION);
333 template<typename Stream, typename K, typename Pred, typename A> void Serialize(Stream& os,
    const std::map<K, T, Pred, A>& m, int nType, int nVersion=VERSION);
334 template<typename Stream, typename K, typename T, typename Pred, typename A> void Unserialize(Stream&
    is, std::map<K, T, Pred, A>& m, int nType, int nVersion=VERSION);
335
336 // set
337 template<typename K, typename Pred, typename A> unsigned int GetSerializeSize(const std::set<K, Pred,
    A>& m, int nType, int nVersion=VERSION);
338 template<typename Stream, typename K, typename Pred, typename A> void Serialize(Stream& os, const
    std::set<K, Pred, A>& m, int nType, int nVersion=VERSION);
339 template<typename Stream, typename K, typename Pred, typename A> void Unserialize(Stream& is,
    std::set<K, Pred, A>& m, int nType, int nVersion=VERSION);
340

```

```

341
342
343
344
345 //
346 // If none of the specialized versions above matched, default to calling member function.
347 // "int nType" is changed to "long nType" to keep from getting an ambiguous overload error.
348 // The compiler will only cast int to long if none of the other templates matched.
349 // Thanks to Boost serialization for this idea.
350 //
351 template<typename T>
352 inline unsigned int GetSerializeSize(const T& a, long nType, int nVersion=VERSION)
353 {
354     return a.GetSerializeSize((int)nType, nVersion);
355 }
356
357 template<typename Stream, typename T>
358 inline void Serialize(Stream& os, const T& a, long nType, int nVersion=VERSION)
359 {
360     a.Serialize(os, (int)nType, nVersion);
361 }
362
363 template<typename Stream, typename T>
364 inline void Unserialize(Stream& is, T& a, long nType, int nVersion=VERSION)
365 {
366     a.Unserialize(is, (int)nType, nVersion);
367 }
368
369
370
371
372
373 //
374 // string
375 //
376 template<typename C>
377 unsigned int GetSerializeSize(const basic_string<C>& str, int, int)
378 {
379     return GetSizeOfCompactSize(str.size()) + str.size() * sizeof(str[0]);
380 }
381
382 template<typename Stream, typename C>
383 void Serialize(Stream& os, const basic_string<C>& str, int, int)
384 {
385     WriteCompactSize(os, str.size());
386     if (!str.empty())
387         os.write((char*)&str[0], str.size() * sizeof(str[0]));
388 }
389
390 template<typename Stream, typename C>
391 void Unserialize(Stream& is, basic_string<C>& str, int, int)
392 {
393     unsigned int nSize = ReadCompactSize(is);
394     str.resize(nSize);
395     if (nSize != 0)
396         is.read((char*)&str[0], nSize * sizeof(str[0]));
397 }
398
399
400
401 //
402 // vector
403 //
404 template<typename T, typename A>
405 unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int nType, int nVersion, const
    boost::true_type&)
406 {
407     return (GetSizeOfCompactSize(v.size()) + v.size() * sizeof(T));
408 }
409
410 template<typename T, typename A>

```

```

411 unsigned int GetSerializeSize_impl(const std::vector<T, A>& v, int nType, int nVersion, const
    boost::false_type&)
412 {
413     unsigned int nSize = GetSizeOfCompactSize(v.size());
414     for (typename std::vector<T, A>::const_iterator vi = v.begin(); vi != v.end(); ++vi)
415         nSize += GetSerializeSize(*vi, nType, nVersion);
416     return nSize;
417 }
418
419 template<typename T, typename A>
420 inline unsigned int GetSerializeSize(const std::vector<T, A>& v, int nType, int nVersion)
421 {
422     return GetSerializeSize_impl(v, nType, nVersion, boost::is_fundamental<T>());
423 }
424
425
426 template<typename Stream, typename T, typename A>
427 void Serialize_impl(Stream& os, const std::vector<T, A>& v, int nType, int nVersion, const
    boost::true_type&)
428 {
429     WriteCompactSize(os, v.size());
430     if (!v.empty())
431         os.write((char*)&v[0], v.size() * sizeof(T));
432 }
433
434 template<typename Stream, typename T, typename A>
435 void Serialize_impl(Stream& os, const std::vector<T, A>& v, int nType, int nVersion, const
    boost::false_type&)
436 {
437     WriteCompactSize(os, v.size());
438     for (typename std::vector<T, A>::const_iterator vi = v.begin(); vi != v.end(); ++vi)
439         ::Serialize(os, (*vi), nType, nVersion);
440 }
441
442 template<typename Stream, typename T, typename A>
443 inline void Serialize(Stream& os, const std::vector<T, A>& v, int nType, int nVersion)
444 {
445     Serialize_impl(os, v, nType, nVersion, boost::is_fundamental<T>());
446 }
447
448
449 template<typename Stream, typename T, typename A>
450 void Unserialize_impl(Stream& is, std::vector<T, A>& v, int nType, int nVersion, const
    boost::true_type&)
451 {
452     unsigned int nSize = ReadCompactSize(is);
453     //v.resize(nSize);
454     //is.read((char*)&v[0], nSize * sizeof(T));
455
456     // Limit size per read so bogus size value won't cause out of memory
457     v.clear();
458     unsigned int nSize = ReadCompactSize(is);
459     unsigned int i = 0;
460     while (i < nSize)
461     {
462         unsigned int blk = min(nSize - i, 1 + 4999999 / sizeof(T));
463         v.resize(i + blk);
464         is.read((char*)&v[i], blk * sizeof(T));
465         i += blk;
466     }
467 }
468
469 template<typename Stream, typename T, typename A>
470 void Unserialize_impl(Stream& is, std::vector<T, A>& v, int nType, int nVersion, const
    boost::false_type&)
471 {
472     unsigned int nSize = ReadCompactSize(is);
473     //v.resize(nSize);
474     //for (std::vector<T, A>::iterator vi = v.begin(); vi != v.end(); ++vi)
475     //    Unserialize(is, (*vi), nType, nVersion);
476
477     v.clear();

```

```

478 unsigned int nSize = ReadCompactSize(is);
479 unsigned int i = 0;
480 unsigned int nMid = 0;
481 while (nMid < nSize)
482 {
483     nMid += 5000000 / sizeof(T);
484 if (nMid > nSize)
485     nMid = nSize;
486     v.resize(nMid);
487 for (; i < nMid; i++)
488     Unserialize(is, v[i], nType, nVersion);
489 }
490 }
491
492 template<typename Stream, typename T, typename A>
493 inline void Unserialize(Stream& is, std::vector<T, A>& v, int nType, int nVersion)
494 {
495     Unserialize_impl(is, v, nType, nVersion, boost::is_fundamental<T>());
496 }
497
498
499
500 //
501 // others derived from vector
502 //
503 inline unsigned int GetSerializeSize(const CScript& v, int nType, int nVersion)
504 {
505     return GetSerializeSize((const vector<unsigned char>&)v, nType, nVersion);
506 }
507
508 template<typename Stream>
509 void Serialize(Stream& os, const CScript& v, int nType, int nVersion)
510 {
511     Serialize(os, (const vector<unsigned char>&)v, nType, nVersion);
512 }
513
514 template<typename Stream>
515 void Unserialize(Stream& is, CScript& v, int nType, int nVersion)
516 {
517     Unserialize(is, (vector<unsigned char>&)v, nType, nVersion);
518 }
519
520
521
522 //
523 // pair
524 //
525 template<typename K, typename T>
526 unsigned int GetSerializeSize(const std::pair<K, T>& item, int nType, int nVersion)
527 {
528     return GetSerializeSize(item.first, nType, nVersion) + GetSerializeSize(item.second, nType, nVersion);
529 }
530
531 template<typename Stream, typename K, typename T>
532 void Serialize(Stream& os, const std::pair<K, T>& item, int nType, int nVersion)
533 {
534     Serialize(os, item.first, nType, nVersion);
535     Serialize(os, item.second, nType, nVersion);
536 }
537
538 template<typename Stream, typename K, typename T>
539 void Unserialize(Stream& is, std::pair<K, T>& item, int nType, int nVersion)
540 {
541     Unserialize(is, item.first, nType, nVersion);
542     Unserialize(is, item.second, nType, nVersion);
543 }
544
545
546
547 //
548 // map
549 //

```

```

550 template<typename K, typename T, typename Pred, typename A>
551 unsigned int GetSerializeSize(const std::map<K, T, Pred, A>& m, int nType, int nVersion)
552 {
553     unsigned int nSize = GetSizeOfCompactSize(m.size());
554     for (typename std::map<K, T, Pred, A>::const_iterator mi = m.begin(); mi != m.end(); ++mi)
555         nSize += GetSerializeSize(*mi, nType, nVersion);
556     return nSize;
557 }
558
559 template<typename Stream, typename K, typename T, typename Pred, typename A>
560 void Serialize(Stream& os, const std::map<K, T, Pred, A>& m, int nType, int nVersion)
561 {
562     WriteCompactSize(os, m.size());
563     for (typename std::map<K, T, Pred, A>::const_iterator mi = m.begin(); mi != m.end(); ++mi)
564         Serialize(os, (*mi), nType, nVersion);
565 }
566
567 template<typename Stream, typename K, typename T, typename Pred, typename A>
568 void Unserialize(Stream& is, std::map<K, T, Pred, A>& m, int nType, int nVersion)
569 {
570     m.clear();
571     unsigned int nSize = ReadCompactSize(is);
572     typename std::map<K, T, Pred, A>::iterator mi = m.begin();
573     for (unsigned int i = 0; i < nSize; i++)
574     {
575         pair<K, T> item;
576         Unserialize(is, item, nType, nVersion);
577         mi = m.insert(mi, item);
578     }
579 }
580
581
582
583 //
584 // set
585 //
586 template<typename K, typename Pred, typename A>
587 unsigned int GetSerializeSize(const std::set<K, Pred, A>& m, int nType, int nVersion)
588 {
589     unsigned int nSize = GetSizeOfCompactSize(m.size());
590     for (typename std::set<K, Pred, A>::const_iterator it = m.begin(); it != m.end(); ++it)
591         nSize += GetSerializeSize(*it, nType, nVersion);
592     return nSize;
593 }
594
595 template<typename Stream, typename K, typename Pred, typename A>
596 void Serialize(Stream& os, const std::set<K, Pred, A>& m, int nType, int nVersion)
597 {
598     WriteCompactSize(os, m.size());
599     for (typename std::set<K, Pred, A>::const_iterator it = m.begin(); it != m.end(); ++it)
600         Serialize(os, (*it), nType, nVersion);
601 }
602
603 template<typename Stream, typename K, typename Pred, typename A>
604 void Unserialize(Stream& is, std::set<K, Pred, A>& m, int nType, int nVersion)
605 {
606     m.clear();
607     unsigned int nSize = ReadCompactSize(is);
608     typename std::set<K, Pred, A>::iterator it = m.begin();
609     for (unsigned int i = 0; i < nSize; i++)
610     {
611         K key;
612         Unserialize(is, key, nType, nVersion);
613         it = m.insert(it, key);
614     }
615 }
616
617
618
619 //
620 // Support for IMPLEMENT_SERIALIZE and READWRITE macro
621 //

```



```

622 class CSerActionGetSerializeSize { };
623 class CSerActionSerialize { };
624 class CSerActionUnserialize { };
625
626 template<typename Stream, typename T>
627 inline unsigned int SerReadWrite(Stream& s, const T& obj, int nType, int nVersion,
    CSerActionGetSerializeSize ser_action)
628 {
629     return ::GetSerializeSize(obj, nType, nVersion);
630 }
631
632 template<typename Stream, typename T>
633 inline unsigned int SerReadWrite(Stream& s, const T& obj, int nType, int nVersion, CSerActionSerialize
    ser_action)
634 {
635     ::Serialize(s, obj, nType, nVersion);
636     return 0;
637 }
638
639 template<typename Stream, typename T>
640 inline unsigned int SerReadWrite(Stream& s, T& obj, int nType, int nVersion, CSerActionUnserialize
    ser_action)
641 {
642     ::Unserialize(s, obj, nType, nVersion);
643     return 0;
644 }
645
646 struct ser_streamplaceholder
647 {
648     int nType;
649     int nVersion;
650 };
651
652
653
654
655
656
657
658
659
660 //
661 // Allocator that clears its contents before deletion
662 //
663 template<typename T>
664 struct secure_allocator : public std::allocator<T>
665 {
666     // MSVC8 default copy constructor is broken
667     typedef std::allocator<T> base;
668     typedef typename base::size_type size_type;
669     typedef typename base::difference_type difference_type;
670     typedef typename base::pointer pointer;
671     typedef typename base::const_pointer const_pointer;
672     typedef typename base::reference reference;
673     typedef typename base::const_reference const_reference;
674     typedef typename base::value_type value_type;
675     secure_allocator() throw() {}
676     secure_allocator(const secure_allocator& a) throw() : base(a) {}
677     ~secure_allocator() throw() {}
678     template<typename _Other> struct rebind
679     { typedef secure_allocator<_Other> other; };
680
681     void deallocate(T* p, std::size_t n)
682     {
683         if (p != NULL)
684             memset(p, 0, sizeof(T) * n);
685         allocator<T>::deallocate(p, n);
686     }
687 };
688
689
690

```

```

691 //
692 // Double ended buffer combining vector and stream-like interfaces.
693 // >> and << read and write unformatted data using the above serialization templates.
694 // Fills with data in linear time; some stringstream implementations take N^2 time.
695 //
696 class CDataStream
697 {
698 protected:
699 typedef vector<char, secure_allocator<char> > vector_type;
700     vector_type vch;
701 unsigned int nReadPos;
702 short state;
703 short exceptmask;
704 public:
705 int nType;
706 int nVersion;
707
708 typedef vector_type::allocator_type    allocator_type;
709 typedef vector_type::size_type         size_type;
710 typedef vector_type::difference_type   difference_type;
711 typedef vector_type::reference         reference;
712 typedef vector_type::const_reference   const_reference;
713 typedef vector_type::value_type        value_type;
714 typedef vector_type::iterator          iterator;
715 typedef vector_type::const_iterator    const_iterator;
716 typedef vector_type::reverse_iterator  reverse_iterator;
717
718 explicit CDataStream(int nTypeIn=0, int nVersionIn=VERSION)
719 {
720     Init(nTypeIn, nVersionIn);
721 }
722
723 CDataStream(const_iterator pbegin, const_iterator pend, int nTypeIn=0, int nVersionIn=VERSION) :
724     vch(pbegin, pend)
725 {
726     Init(nTypeIn, nVersionIn);
727 }
728 #if !defined(_MSC_VER) || _MSC_VER >= 1300
729 CDataStream(const char* pbegin, const char* pend, int nTypeIn=0, int nVersionIn=VERSION) : vch(pbegin,
730     pend)
731 {
732     Init(nTypeIn, nVersionIn);
733 }
734 #endif
735 CDataStream(const vector_type& vchIn, int nTypeIn=0, int nVersionIn=VERSION) : vch(vchIn.begin(),
736     vchIn.end())
737 {
738     Init(nTypeIn, nVersionIn);
739 }
740 CDataStream(const vector<char>& vchIn, int nTypeIn=0, int nVersionIn=VERSION) : vch(vchIn.begin(),
741     vchIn.end())
742 {
743     Init(nTypeIn, nVersionIn);
744 }
745 CDataStream(const vector<unsigned char>& vchIn, int nTypeIn=0, int nVersionIn=VERSION) :
746     vch((char*)&vchIn.begin()[0], (char*)&vchIn.end()[0])
747 {
748     Init(nTypeIn, nVersionIn);
749 }
750 void Init(int nTypeIn=0, int nVersionIn=VERSION)
751 {
752     nReadPos = 0;
753     nType = nTypeIn;
754     nVersion = nVersionIn;
755     state = 0;
756     exceptmask = ios::badbit | ios::failbit;
757 }

```

```

758
759     CDataStream& operator+=(const CDataStream& b)
760 {
761     vch.insert(vch.end(), b.begin(), b.end());
762     return *this;
763 }
764
765 friend CDataStream operator+(const CDataStream& a, const CDataStream& b)
766 {
767     CDataStream ret = a;
768     ret += b;
769     return (ret);
770 }
771
772     string str() const
773 {
774     return (string(begin(), end()));
775 }
776
777
778 //
779 // Vector subset
780 //
781     const_iterator begin() const { return vch.begin() + nReadPos; }
782     iterator begin() { return vch.begin() + nReadPos; }
783     const_iterator end() const { return vch.end(); }
784     iterator end() { return vch.end(); }
785     size_type size() const { return vch.size() - nReadPos; }
786     bool empty() const { return vch.size() == nReadPos; }
787     void resize(size_type n, value_type c=0) { vch.resize(n + nReadPos, c); }
788     void reserve(size_type n) { vch.reserve(n + nReadPos); }
789     const_reference operator[](size_type pos) const { return vch[pos + nReadPos]; }
790     reference operator[](size_type pos) { return vch[pos + nReadPos]; }
791     void clear() { vch.clear(); nReadPos = 0; }
792     iterator insert(iterator it, const char& x=char()) { return vch.insert(it, x); }
793     void insert(iterator it, size_type n, const char& x) { vch.insert(it, n, x); }
794
795     void insert(iterator it, const_iterator first, const_iterator last)
796 {
797     if (it == vch.begin() + nReadPos && last - first <= nReadPos)
798     {
799         // special case for inserting at the front when there's room
800         nReadPos -= (last - first);
801         memcpy(&vch[nReadPos], &first[0], last - first);
802     }
803     else
804         vch.insert(it, first, last);
805 }
806
807 #if !defined(_MSC_VER) || _MSC_VER >= 1300
808 void insert(iterator it, const char* first, const char* last)
809 {
810     insert(it, (const_iterator)first, (const_iterator)last);
811 }
812 #endif
813
814     iterator erase(iterator it)
815 {
816     if (it == vch.begin() + nReadPos)
817     {
818         // special case for erasing from the front
819         if (++nReadPos >= vch.size())
820         {
821             // whenever we reach the end, we take the opportunity to clear the buffer
822             nReadPos = 0;
823             return vch.erase(vch.begin(), vch.end());
824         }
825         return vch.begin() + nReadPos;
826     }
827     else
828         return vch.erase(it);
829 }

```

```

830
831     iterator erase(iterator first, iterator last)
832 {
833     if (first == vch.begin() + nReadPos)
834     {
835         // special case for erasing from the front
836         if (last == vch.end())
837         {
838             nReadPos = 0;
839             return vch.erase(vch.begin(), vch.end());
840         }
841         else
842         {
843             nReadPos = (last - vch.begin());
844             return last;
845         }
846     }
847     else
848     return vch.erase(first, last);
849 }
850
851 inline void Compact()
852 {
853     vch.erase(vch.begin(), vch.begin() + nReadPos);
854     nReadPos = 0;
855 }
856
857 bool Rewind(size_type n)
858 {
859     // Rewind by n characters if the buffer hasn't been compacted yet
860     if (n > nReadPos)
861     return false;
862     nReadPos -= n;
863     return true;
864 }
865
866
867 //
868 // Stream subset
869 //
870 void setstate(short bits, const char* psz)
871 {
872     state |= bits;
873     if (state & exceptmask)
874     throw std::ios_base::failure(psz);
875 }
876
877 bool eof() const { return size() == 0; }
878 bool fail() const { return state & (ios::badbit | ios::failbit); }
879 bool good() const { return !eof() && (state == 0); }
880 void clear(short n) { state = n; } // name conflict with vector clear()
881 short exceptions() { return exceptmask; }
882 short exceptions(short mask) { short prev = exceptmask; exceptmask = mask; setstate(0, "CDataStream");
    return prev; }
883     CDataStream* rdbuf() { return this; }
884     int in_avail() { return size(); }
885
886 void SetType(int n) { nType = n; }
887 int GetType() { return nType; }
888 void SetVersion(int n) { nVersion = n; }
889 int GetVersion() { return nVersion; }
890 void ReadVersion() { *this >> nVersion; }
891 void WriteVersion() { *this << nVersion; }
892
893     CDataStream& read(char* pch, int nSize)
894 {
895     // Read from the beginning of the buffer
896     assert(nSize >= 0);
897     unsigned int nReadPosNext = nReadPos + nSize;
898     if (nReadPosNext >= vch.size())
899     {
900     if (nReadPosNext > vch.size())

```

```

901 {
902     setstate(ios::failbit, "CDataStream::read() : end of data");
903     memset(pch, 0, nSize);
904     nSize = vch.size() - nReadPos;
905 }
906 memcpy(pch, &vch[nReadPos], nSize);
907     nReadPos = 0;
908     vch.clear();
909     return (*this);
910 }
911 memcpy(pch, &vch[nReadPos], nSize);
912     nReadPos = nReadPosNext;
913     return (*this);
914 }
915
916     CDataStream& ignore(int nSize)
917 {
918     // Ignore from the beginning of the buffer
919     assert(nSize >= 0);
920     unsigned int nReadPosNext = nReadPos + nSize;
921     if (nReadPosNext >= vch.size())
922     {
923         if (nReadPosNext > vch.size())
924         {
925             setstate(ios::failbit, "CDataStream::ignore() : end of data");
926             nSize = vch.size() - nReadPos;
927         }
928         nReadPos = 0;
929         vch.clear();
930         return (*this);
931     }
932     nReadPos = nReadPosNext;
933     return (*this);
934 }
935
936     CDataStream& write(const char* pch, int nSize)
937 {
938     // Write to the end of the buffer
939     assert(nSize >= 0);
940     vch.insert(vch.end(), pch, pch + nSize);
941     return (*this);
942 }
943
944 template<typename Stream>
945 void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const
946 {
947     // Special case: stream << stream concatenates like stream += stream
948     if (!vch.empty())
949         s.write((char*)&vch[0], vch.size() * sizeof(vch[0]));
950 }
951
952 template<typename T>
953 unsigned int GetSerializeSize(const T& obj)
954 {
955     // Tells the size of the object if serialized to this stream
956     return ::GetSerializeSize(obj, nType, nVersion);
957 }
958
959 template<typename T>
960     CDataStream& operator<<(const T& obj)
961 {
962     // Serialize to this stream
963     ::Serialize(*this, obj, nType, nVersion);
964     return (*this);
965 }
966
967 template<typename T>
968     CDataStream& operator>>(T& obj)
969 {
970     // Unserialize from this stream
971     ::Unserialize(*this, obj, nType, nVersion);
972     return (*this);

```

```

973 }
974 };
975
976 #ifdef TESTCDATASTREAM
977 // VC6sp6
978 // CDataStream:
979 // n=1000      0 seconds
980 // n=2000      0 seconds
981 // n=4000      0 seconds
982 // n=8000      0 seconds
983 // n=16000     0 seconds
984 // n=32000     0 seconds
985 // n=64000     1 seconds
986 // n=128000    1 seconds
987 // n=256000    2 seconds
988 // n=512000    4 seconds
989 // n=1024000   8 seconds
990 // n=2048000  16 seconds
991 // n=4096000  32 seconds
992 // stringstream:
993 // n=1000      1 seconds
994 // n=2000      1 seconds
995 // n=4000      13 seconds
996 // n=8000      87 seconds
997 // n=16000     400 seconds
998 // n=32000     1660 seconds
999 // n=64000     6749 seconds
1000 // n=128000    27241 seconds
1001 // n=256000    109804 seconds
1002 #include <iostream>
1003 int main(int argc, char *argv[])
1004 {
1005     vector<unsigned char> vch(0xcc, 250);
1006     printf("CDataStream:\n");
1007     for (int n = 1000; n <= 4500000; n *= 2)
1008     {
1009         CDataStream ss;
1010         time_t nStart = time(NULL);
1011         for (int i = 0; i < n; i++)
1012             ss.write((char*)&vch[0], vch.size());
1013         printf("n=%-10d %d seconds\n", n, time(NULL) - nStart);
1014     }
1015     printf("stringstream:\n");
1016     for (int n = 1000; n <= 4500000; n *= 2)
1017     {
1018         stringstream ss;
1019         time_t nStart = time(NULL);
1020         for (int i = 0; i < n; i++)
1021             ss.write((char*)&vch[0], vch.size());
1022         printf("n=%-10d %d seconds\n", n, time(NULL) - nStart);
1023     }
1024 }
1025 #endif
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036 //
1037 // Automatic closing wrapper for FILE*
1038 // - Will automatically close the file when it goes out of scope if not null.
1039 // - If you're returning the file pointer, return file.release().
1040 // - If you need to close the file early, use file.fclose() instead of fclose(file).
1041 //
1042 class CAutoFile
1043 {
1044 protected:

```

```

1045     FILE* file;
1046     short state;
1047     short exceptmask;
1048     public:
1049     int nType;
1050     int nVersion;
1051
1052     typedef FILE element_type;
1053
1054     CAutoFile(FILE* filenew=NULL, int nTypeIn=SER_DISK, int nVersionIn=VERSION)
1055     {
1056         file = filenew;
1057         nType = nTypeIn;
1058         nVersion = nVersionIn;
1059         state = 0;
1060         exceptmask = ios::badbit | ios::failbit;
1061     }
1062
1063     ~CAutoFile()
1064     {
1065         fclose();
1066     }
1067
1068     void fclose()
1069     {
1070         if (file != NULL && file != stdin && file != stdout && file != stderr)
1071             ::fclose(file);
1072         file = NULL;
1073     }
1074
1075     FILE* release() { FILE* ret = file; file = NULL; return ret; }
1076     operator FILE*() { return file; }
1077     FILE* operator->() { return file; }
1078     FILE& operator*() { return *file; }
1079     FILE** operator&() { return &file; }
1080     FILE* operator=(FILE* pnew) { return file = pnew; }
1081     bool operator!() { return (file == NULL); }
1082
1083
1084     //
1085     // Stream subset
1086     //
1087     void setstate(short bits, const char* psz)
1088     {
1089         state |= bits;
1090         if (state & exceptmask)
1091             throw std::ios_base::failure(psz);
1092     }
1093
1094     bool fail() const { return state & (ios::badbit | ios::failbit); }
1095     bool good() const { return state == 0; }
1096     void clear(short n = 0) { state = n; }
1097     short exceptions() { return exceptmask; }
1098     short exceptions(short mask) { short prev = exceptmask; exceptmask = mask; setstate(0, "CAutoFile");
1099         return prev; }
1100
1101     void SetType(int n) { nType = n; }
1102     int GetType() { return nType; }
1103     void SetVersion(int n) { nVersion = n; }
1104     int GetVersion() { return nVersion; }
1105     void ReadVersion() { *this >> nVersion; }
1106     void WriteVersion() { *this << nVersion; }
1107
1108     CAutoFile& read(char* pch, int nSize)
1109     {
1110         if (!file)
1111             throw std::ios_base::failure("CAutoFile::read : file handle is NULL");
1112         if (fread(pch, 1, nSize, file) != nSize)
1113             setstate(ios::failbit, feof(file) ? "CAutoFile::read : end of file" : "CAutoFile::read : fread failed");
1114         return (*this);
1115     }

```

```

1116     CAutoFile& write(const char* pch, int nSize)
1117 {
1118     if (!file)
1119         throw std::ios_base::failure("CAutoFile::write : file handle is NULL");
1120     if (fwrite(pch, 1, nSize, file) != nSize)
1121         setstate(ios::failbit, "CAutoFile::write : write failed");
1122     return (*this);
1123 }
1124
1125 template<typename T>
1126 unsigned int GetSerializeSize(const T& obj)
1127 {
1128     // Tells the size of the object if serialized to this stream
1129     return ::GetSerializeSize(obj, nType, nVersion);
1130 }
1131
1132 template<typename T>
1133     CAutoFile& operator<<(const T& obj)
1134 {
1135     // Serialize to this stream
1136     if (!file)
1137         throw std::ios_base::failure("CAutoFile::operator<< : file handle is NULL");
1138     ::Serialize(*this, obj, nType, nVersion);
1139     return (*this);
1140 }
1141
1142 template<typename T>
1143     CAutoFile& operator>>(T& obj)
1144 {
1145     // Unserialize from this stream
1146     if (!file)
1147         throw std::ios_base::failure("CAutoFile::operator>> : file handle is NULL");
1148     ::Unserialize(*this, obj, nType, nVersion);
1149     return (*this);
1150 }
1151 };
1152

```


SUMMARY

The serialization format is consensus-critical. If two nodes serialize differently, they'll compute different transaction/block hashes and fork. Satoshi's `IMPLEMENT_SERIALIZE` macro ensures consistency: one definition, no chance for read/write mismatch. This design pattern has prevented countless potential bugs.

The format's success lies in its simplicity and stability. No schema versioning complexity (version number is inline), no compression (bytes are bytes), no encryption at this layer. Just raw, deterministic encoding. Every alternative cryptocurrency that uses Bitcoin's code inherits this format—it's become the Rosetta Stone of blockchain data.

The CompactSize encoding deserves recognition as elegant engineering. Most blockchain data involves small counts (typically 1-3 inputs/outputs), so optimizing for small values saves significant space. Across millions of transactions, these byte savings compound into gigabytes of reduced blockchain size. Satoshi optimized for the common case while gracefully handling the rare large values.

sha.cpp

555 lines CPP

THE HASH THAT SECURES TRILLIONS

Cryptographic hashing is the foundation of Bitcoin's security. This file implements the SHA-256 hash function that is used throughout Bitcoin for everything from transaction IDs to proof-of-work mining.

A hash function takes any input and produces a fixed-size output (256 bits for SHA-256). The key properties are:

- **Deterministic:** Same input always gives same output
- **One-way:** Cannot reverse the hash to find the input
- **Collision-resistant:** Nearly impossible to find two inputs with the same hash

```

1 // This file is public domain
2 // SHA routines extracted as a standalone file from:
3 // Crypto++: a C++ Class Library of Cryptographic Schemes
4 // Version 5.5.2 (9/24/2007)
5 // http://www.cryptopp.com
6
7 // sha.cpp - modified by Wei Dai from Steve Reid's public domain sha1.c
8
9 // Steve Reid implemented SHA-1. Wei Dai implemented SHA-2.
10 // Both are in the public domain.
11
12 #include <assert.h>
13 #include <memory.h>
14 #include "sha.h"
15
16 namespace CryptoPP
17 {
18
19 // start of Steve Reid's code
20
21 #define blk0(i) (W[i] = data[i])
22 #define blk1(i) (W[i&15] = rotlFixed(W[(i+13)&15]^W[(i+8)&15]^W[(i+2)&15]^W[i&15],1))
23
24 void SHA1::InitState(HashWordType *state)
25 {
26     state[0] = 0x67452301L;
27     state[1] = 0xEFCDA89L;
28     state[2] = 0x98BADCFEL;
29     state[3] = 0x10325476L;
30     state[4] = 0xC3D2E1F0L;
31 }
32
33 #define f1(x,y,z) (z^(x&(y^z)))
34 #define f2(x,y,z) (x^y^z)
35 #define f3(x,y,z) ((x&y)|(z&(x|y)))
36 #define f4(x,y,z) (x^y^z)
37
38 /* (R0+R1), R2, R3, R4 are the different operations used in SHA1 */
39 #define R0(v,w,x,y,z,i) z+=f1(w,x,y)+blk0(i)+0x5A827999+rotrFixed(v,5);w=rotrFixed(w,30);
40 #define R1(v,w,x,y,z,i) z+=f1(w,x,y)+blk1(i)+0x5A827999+rotrFixed(v,5);w=rotrFixed(w,30);
41 #define R2(v,w,x,y,z,i) z+=f2(w,x,y)+blk1(i)+0x6ED9EBA1+rotrFixed(v,5);w=rotrFixed(w,30);
42 #define R3(v,w,x,y,z,i) z+=f3(w,x,y)+blk1(i)+0x8F1BBCDC+rotrFixed(v,5);w=rotrFixed(w,30);
43 #define R4(v,w,x,y,z,i) z+=f4(w,x,y)+blk1(i)+0xCA62C1D6+rotrFixed(v,5);w=rotrFixed(w,30);
44
45 void SHA1::Transform(word32 *state, const word32 *data)
46 {
47     word32 W[16];
48     /* Copy context->state[] to working vars */
49     word32 a = state[0];
50     word32 b = state[1];
51     word32 c = state[2];
52     word32 d = state[3];
53     word32 e = state[4];
54     /* 4 rounds of 20 operations each. Loop unrolled. */
55     R0(a,b,c,d,e, 0); R0(e,a,b,c,d, 1); R0(d,e,a,b,c, 2); R0(c,d,e,a,b, 3);
56     R0(b,c,d,e,a, 4); R0(a,b,c,d,e, 5); R0(e,a,b,c,d, 6); R0(d,e,a,b,c, 7);
57     R0(c,d,e,a,b, 8); R0(b,c,d,e,a, 9); R0(a,b,c,d,e,10); R0(e,a,b,c,d,11);
58     R0(d,e,a,b,c,12); R0(c,d,e,a,b,13); R0(b,c,d,e,a,14); R0(a,b,c,d,e,15);
59     R1(e,a,b,c,d,16); R1(d,e,a,b,c,17); R1(c,d,e,a,b,18); R1(b,c,d,e,a,19);
60     R2(a,b,c,d,e,20); R2(e,a,b,c,d,21); R2(d,e,a,b,c,22); R2(c,d,e,a,b,23);
61     R2(b,c,d,e,a,24); R2(a,b,c,d,e,25); R2(e,a,b,c,d,26); R2(d,e,a,b,c,27);
62     R2(c,d,e,a,b,28); R2(b,c,d,e,a,29); R2(a,b,c,d,e,30); R2(e,a,b,c,d,31);
63     R2(d,e,a,b,c,32); R2(c,d,e,a,b,33); R2(b,c,d,e,a,34); R2(a,b,c,d,e,35);
64     R2(e,a,b,c,d,36); R2(d,e,a,b,c,37); R2(c,d,e,a,b,38); R2(b,c,d,e,a,39);
65     R3(a,b,c,d,e,40); R3(e,a,b,c,d,41); R3(d,e,a,b,c,42); R3(c,d,e,a,b,43);
66     R3(b,c,d,e,a,44); R3(a,b,c,d,e,45); R3(e,a,b,c,d,46); R3(d,e,a,b,c,47);
67     R3(c,d,e,a,b,48); R3(b,c,d,e,a,49); R3(a,b,c,d,e,50); R3(e,a,b,c,d,51);
68     R3(d,e,a,b,c,52); R3(c,d,e,a,b,53); R3(b,c,d,e,a,54); R3(a,b,c,d,e,55);
69     R3(e,a,b,c,d,56); R3(d,e,a,b,c,57); R3(c,d,e,a,b,58); R3(b,c,d,e,a,59);
70     R4(a,b,c,d,e,60); R4(e,a,b,c,d,61); R4(d,e,a,b,c,62); R4(c,d,e,a,b,63);
71     R4(b,c,d,e,a,64); R4(a,b,c,d,e,65); R4(e,a,b,c,d,66); R4(d,e,a,b,c,67);
72     R4(c,d,e,a,b,68); R4(b,c,d,e,a,69); R4(a,b,c,d,e,70); R4(e,a,b,c,d,71);

```

```

73 R4(d,e,a,b,c,72); R4(c,d,e,a,b,73); R4(b,c,d,e,a,74); R4(a,b,c,d,e,75);
74 R4(e,a,b,c,d,76); R4(d,e,a,b,c,77); R4(c,d,e,a,b,78); R4(b,c,d,e,a,79);
75 /* Add the working vars back into context.state[] */
76     state[0] += a;
77     state[1] += b;
78     state[2] += c;
79     state[3] += d;
80     state[4] += e;
81 }
82
83 // end of Steve Reid's code
84
85 // *****
86
87 void SHA224::InitState(HashWordType *state)
88 {
89     static const word32 s[8] = {0xc1059ed8, 0x367cd507, 0x3070dd17, 0xf70e5939, 0xffc00b31, 0x68581511,
90     0x64f98fa7, 0xbefa4fa4};
91     memcpy(state, s, sizeof(s));
92 }
93 void SHA256::InitState(HashWordType *state)

```

SHA256 initialization - sets the initial hash values

```

94 {
95     static const word32 s[8] = {0xa609e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a, 0x510e527f, 0x9b05688c,
96     0x1f83d9ab, 0x5be0cd19};
97     memcpy(state, s, sizeof(s));
98 }
99 static const word32 SHA256_K[64] = {

```

SHA256_K constants - derived from cube roots of first 64 primes

```

100 0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
101 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
102 0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
103 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
104 0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
105 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
106 0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
107 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
108 0x27b70a85, 0x2e1b2138, 0x4d2c6dfe, 0x53380d13,
109 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
110 0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
111 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
112 0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
113 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
114 0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
115 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
116 };
117
118 #define blk2(i) (W[i&15] += s1(W[(i-2)&15]) + W[(i-7)&15] + s0(W[(i-15)&15]))
119
120 #define Ch(x,y,z) (z^(x&(y^z)))
121 #define Maj(x,y,z) ((x&y)|(z&(x|y)))
122
123 #define a(i) T[(0-i)&7]
124 #define b(i) T[(1-i)&7]
125 #define c(i) T[(2-i)&7]
126 #define d(i) T[(3-i)&7]
127 #define e(i) T[(4-i)&7]
128 #define f(i) T[(5-i)&7]
129 #define g(i) T[(6-i)&7]
130 #define h(i) T[(7-i)&7]
131

```

```

132 #define R(i) h(i)+=S1(e(i))+Ch(e(i),f(i),g(i))+SHA256_K[i+j]+(j?blk2(i):blk0(i));\
133 d(i)+=h(i);h(i)+=S0(a(i))+Maj(a(i),b(i),c(i))
134
135 // for SHA256
136 #define S0(x) (rotrFixed(x,2)^rotrFixed(x,13)^rotrFixed(x,22))
137 #define S1(x) (rotrFixed(x,6)^rotrFixed(x,11)^rotrFixed(x,25))
138 #define s0(x) (rotrFixed(x,7)^rotrFixed(x,18)^(x>>3))
139 #define s1(x) (rotrFixed(x,17)^rotrFixed(x,19)^(x>>10))
140
141 void SHA256::Transform(word32 *state, const word32 *data)

```

SHA256 Transform - the core compression function

```

142 {
143     word32 W[16];
144     word32 T[8];
145     /* Copy context->state[] to working vars */
146     memcpy(T, state, sizeof(T));
147     /* 64 operations, partially loop unrolled */
148     for (unsigned int j=0; j<64; j+=16)
149     {
150         R( 0); R( 1); R( 2); R( 3);
151         R( 4); R( 5); R( 6); R( 7);
152         R( 8); R( 9); R(10); R(11);
153         R(12); R(13); R(14); R(15);
154     }
155     /* Add the working vars back into context.state[] */
156     state[0] += a(0);
157     state[1] += b(0);
158     state[2] += c(0);
159     state[3] += d(0);
160     state[4] += e(0);
161     state[5] += f(0);
162     state[6] += g(0);
163     state[7] += h(0);
164 }
165
166 /*
167 // smaller but slower
168 void SHA256_Transform(word32 *state, const word32 *data)

```

SHA256_Transform - external entry point for hashing

```

169 {
170     word32 T[20];
171     word32 W[32];
172     unsigned int i = 0, j = 0;
173     word32 *t = T+8;
174
175     memcpy(t, state, 8*4);
176     word32 e = t[4], a = t[0];
177
178     do
179     {
180         word32 w = data[j];
181         W[j] = w;
182         w += K[j];
183         w += t[7];
184         w += S1(e);
185         w += Ch(e, t[5], t[6]);
186         e = t[3] + w;
187         t[3] = t[3+8] = e;
188         w += S0(t[0]);
189         a = w + Maj(a, t[1], t[2]);
190         t[-1] = t[7] = a;
191         --t;
192         ++j;

```

```

193 if (j%8 == 0)
194     t += 8;
195 } while (j<16);
196
197 do
198 {
199     i = j&0xf;
200     word32 w = s1(W[i+16-2]) + s0(W[i+16-15]) + W[i] + W[i+16-7];
201     W[i+16] = W[i] = w;
202     w += K[j];
203     w += t[7];
204     w += S1(e);
205     w += Ch(e, t[5], t[6]);
206     e = t[3] + w;
207     t[3] = t[3+8] = e;
208     w += S0(t[0]);
209     a = w + Maj(a, t[1], t[2]);
210     t[-1] = t[7] = a;
211
212     w = s1(W[(i+1)+16-2]) + s0(W[(i+1)+16-15]) + W[(i+1)] + W[(i+1)+16-7];
213     W[(i+1)+16] = W[(i+1)] = w;
214     w += K[j+1];
215     w += (t-1)[7];
216     w += S1(e);
217     w += Ch(e, (t-1)[5], (t-1)[6]);
218     e = (t-1)[3] + w;
219     (t-1)[3] = (t-1)[3+8] = e;
220     w += S0((t-1)[0]);
221     a = w + Maj(a, (t-1)[1], (t-1)[2]);
222     (t-1)[-1] = (t-1)[7] = a;
223
224     t-=2;
225     j+=2;
226 if (j%8 == 0)
227     t += 8;
228 } while (j<64);
229
230 state[0] += a;
231 state[1] += t[1];
232 state[2] += t[2];
233 state[3] += t[3];
234 state[4] += e;
235 state[5] += t[5];
236 state[6] += t[6];
237 state[7] += t[7];
238 }
239 */
240
241 #undef S0
242 #undef S1
243 #undef s0
244 #undef s1
245 #undef R
246
247 // *****
248
249 #ifdef WORD64_AVAILABLE
250
251 void SHA384::InitState(HashWordType *state)
252 {
253     static const word64 s[8] = {
254         W64LIT(0xcbbb9d5dc1059ed8), W64LIT(0x629a292a367cd507),
255         W64LIT(0x9159015a3070dd17), W64LIT(0x152fec8f70e5939),
256         W64LIT(0x67332667ffc00b31), W64LIT(0x8eb44a8768581511),
257         W64LIT(0xdb0c2e0d64f98fa7), W64LIT(0x47b5481dbefa4fa4)};
258     memcpy(state, s, sizeof(s));
259 }
260
261 void SHA512::InitState(HashWordType *state)
262 {
263     static const word64 s[8] = {
264         W64LIT(0x6a09e667f3bcc908), W64LIT(0xbb67ae8584caa73b),

```

```

265 W64LIT(0x3c6ef372fe94f82b), W64LIT(0xa54ff53a5f1d36f1),
266 W64LIT(0x510e527fade682d1), W64LIT(0x9b05688c2b3e6c1f),
267 W64LIT(0x1f83d9abfb41bd6b), W64LIT(0x5be0cd19137e2179));
268 memcpy(state, s, sizeof(s));
269 }
270
271 CRYPTOPP_ALIGN_DATA(16) static const word64 SHA512_K[80] CRYPTOPP_SECTION_ALIGN16 = {
272 W64LIT(0x428a2f98d728ae22), W64LIT(0x7137449123ef65cd),
273 W64LIT(0xb5c0fbcfec4d3b2f), W64LIT(0xe9b5dba58189dbbc),
274 W64LIT(0x3956c25bf348b538), W64LIT(0x59f111f1b605d019),
275 W64LIT(0x923f82a4af194f9b), W64LIT(0xab1c5ed5da6d8118),
276 W64LIT(0xd807aa98a3030242), W64LIT(0x12835b0145706fbe),
277 W64LIT(0x243185be4ee4b28c), W64LIT(0x550c7dc3d5ffb4e2),
278 W64LIT(0x72be5d74f27b896f), W64LIT(0x80deb1fe3b1696b1),
279 W64LIT(0x9bdc06a725c71235), W64LIT(0xc19bf174cf692694),
280 W64LIT(0xe49b69c19ef14ad2), W64LIT(0xefbe4786384f25e3),
281 W64LIT(0x0fc19dc68b8cd5b5), W64LIT(0x240ca1cc77ac9c65),
282 W64LIT(0x2de92c6f592b0275), W64LIT(0x4a7484aa6a6a483),
283 W64LIT(0x5cb0a9dcbd41fbd4), W64LIT(0x76f988da831153b5),
284 W64LIT(0x983e5152ee66dfab), W64LIT(0xa831c66d2db43210),
285 W64LIT(0xb00327c898fb213f), W64LIT(0xbf597fc7beef0ee4),
286 W64LIT(0xc6e00bf33da88fc2), W64LIT(0xd5a79147930aa725),
287 W64LIT(0x06ca6351e003826f), W64LIT(0x142929670a0e6e70),
288 W64LIT(0x27b70a8546d22ffc), W64LIT(0x2e1b21385c26c926),
289 W64LIT(0x4d2c6dfc5ac42aed), W64LIT(0x53380d139d95b3df),
290 W64LIT(0x650a73548ba6f3de), W64LIT(0x766a0abb3c77b2a8),
291 W64LIT(0x81c2c92e47edaee6), W64LIT(0x92722c851482353b),
292 W64LIT(0xa2bfe8a14cf10364), W64LIT(0xa81a664bbc423001),
293 W64LIT(0xc24b8b70d0f89791), W64LIT(0xc76c51a30654be30),
294 W64LIT(0xd192e819d6ef5218), W64LIT(0xd69906245565a910),
295 W64LIT(0xf40e35855771202a), W64LIT(0x106aa07032bbd1b8),
296 W64LIT(0x19a4c116b8d2d0c8), W64LIT(0x1e376c085141ab53),
297 W64LIT(0x2748774cdf8eeb99), W64LIT(0x34b0bcb5e19b48a8),
298 W64LIT(0x391c0cb3c5c95a63), W64LIT(0x4ed8aa4ae3418acb),
299 W64LIT(0x5b9cca4f7763e373), W64LIT(0x682e6ff3d6b2b8a3),
300 W64LIT(0x748f82ee5defb2fc), W64LIT(0x78a5636f43172f60),
301 W64LIT(0x84c87814a1f0ab72), W64LIT(0x8cc702081a6439ec),
302 W64LIT(0x90beffffa23631e28), W64LIT(0xa4506cebbe82bde9),
303 W64LIT(0xbef9a3f7b2c67915), W64LIT(0xc67178f2e372532b),
304 W64LIT(0xca273ecee26619c), W64LIT(0xd186b8c721c0c207),
305 W64LIT(0xeada7dd6cde0eb1e), W64LIT(0xf57d4f7fee6ed178),
306 W64LIT(0x06f067aa72176fba), W64LIT(0x0a637dc5a2c898a6),
307 W64LIT(0x113f9804bef90dae), W64LIT(0x1b710b35131c471b),
308 W64LIT(0x28db77f523047d84), W64LIT(0x32caab7b40c72493),
309 W64LIT(0x3c9ebe0a15c9bebc), W64LIT(0x431d67c49c100d4c),
310 W64LIT(0x4cc5d4becb3e42b6), W64LIT(0x597f299cfc657e2a),
311 W64LIT(0x5fcb6fab3ad6faec), W64LIT(0x6c44198c4a475817)
312 };
313
314 #if CRYPTOPP_BOOL_SSE2_ASM_AVAILABLE && CRYPTOPP_BOOL_X86
315 // put assembly version in separate function, otherwise MSVC 2005 SP1 doesn't generate correct code for
// the non-assembly version
316 CRYPTOPP_NAKED static void CRYPTOPP_FASTCALL SHA512_SSE2_Transform(word64 *state, const word64 *data)
317 {
318 #ifdef __GNUC__
319     __asm__ __volatile__
320 (
321     ".intel_syntax noprefix;"
322     AS1( push ebx)
323     AS2( mov ebx, eax)
324     #else
325     AS1( push ebx)
326     AS1( push esi)
327     AS1( push edi)
328     AS2( lea ebx, SHA512_K)
329     #endif
330
331     AS2( mov eax, esp)
332     AS2( and esp, 0xffffffff)
333     AS2( sub esp, 27*16) // 17*16 for expanded data, 20*8 for state
334     AS1( push eax)
335     AS2( xor eax, eax)

```



```

336 AS2(    lea    edi, [esp+4*8] ) // start at middle of state buffer. will decrement pointer each round
        to avoid copying
337 AS2(    lea    esi, [esp+4+20*8] ) // 16-byte alignment, then add 8
338
339 AS2(    movq    mm4, [ecx+0*8] )
340 AS2(    movq    [edi+0*8], mm4 )
341 AS2(    movq    mm0, [ecx+1*8] )
342 AS2(    movq    [edi+1*8], mm0 )
343 AS2(    movq    mm0, [ecx+2*8] )
344 AS2(    movq    [edi+2*8], mm0 )
345 AS2(    movq    mm0, [ecx+3*8] )
346 AS2(    movq    [edi+3*8], mm0 )
347 AS2(    movq    mm5, [ecx+4*8] )
348 AS2(    movq    [edi+4*8], mm5 )
349 AS2(    movq    mm0, [ecx+5*8] )
350 AS2(    movq    [edi+5*8], mm0 )
351 AS2(    movq    mm0, [ecx+6*8] )
352 AS2(    movq    [edi+6*8], mm0 )
353 AS2(    movq    mm0, [ecx+7*8] )
354 AS2(    movq    [edi+7*8], mm0 )
355 ASJ(    jmp, 0, f )
356
357 #define SSE2_S0_S1(r, a, b, c)\
358 AS2(    movq    mm6, r)\
359 AS2(    psrlq    r, a)\
360 AS2(    movq    mm7, r)\
361 AS2(    psllq    mm6, 64-c)\
362 AS2(    pxor    mm7, mm6)\
363 AS2(    psrlq    r, b-a)\
364 AS2(    pxor    mm7, r)\
365 AS2(    psllq    mm6, c-b)\
366 AS2(    pxor    mm7, mm6)\
367 AS2(    psrlq    r, c-b)\
368 AS2(    pxor    r, mm7)\
369 AS2(    psllq    mm6, b-a)\
370 AS2(    pxor    r, mm6)
371
372 #define SSE2_s0(r, a, b, c) \
373 AS2(    movdqa   xmm6, r)\
374 AS2(    psrlq    r, a)\
375 AS2(    movdqa   xmm7, r)\
376 AS2(    psllq    xmm6, 64-c)\
377 AS2(    pxor     xmm7, xmm6)\
378 AS2(    psrlq    r, b-a)\
379 AS2(    pxor     xmm7, r)\
380 AS2(    psrlq    r, c-b)\
381 AS2(    pxor     r, xmm7)\
382 AS2(    psllq    xmm6, c-a)\
383 AS2(    pxor     r, xmm6)
384
385 #define SSE2_s1(r, a, b, c) \
386 AS2(    movdqa   xmm6, r)\
387 AS2(    psrlq    r, a)\
388 AS2(    movdqa   xmm7, r)\
389 AS2(    psllq    xmm6, 64-c)\
390 AS2(    pxor     xmm7, xmm6)\
391 AS2(    psrlq    r, b-a)\
392 AS2(    pxor     xmm7, r)\
393 AS2(    psllq    xmm6, c-b)\
394 AS2(    pxor     xmm7, xmm6)\
395 AS2(    psrlq    r, c-b)\
396 AS2(    pxor     r, xmm7)
397
398 ASL(SHA512_Round)
399 // k + w is in mm0, a is in mm4, e is in mm5
400 AS2(    paddq    mm0, [edi+7*8] ) // h
401 AS2(    movq     mm2, [edi+5*8] ) // f
402 AS2(    movq     mm3, [edi+6*8] ) // g
403 AS2(    pxor     mm2, mm3 )
404 AS2(    pand     mm2, mm5 )
405 SSE2_S0_S1(mm5, 14, 18, 41)
406 AS2(    pxor     mm2, mm3 )
    
```

```

407 AS2(    paddq    mm0, mm2) // h += Ch(e,f,g)
408 AS2(    paddq    mm5, mm0) // h += S1(e)
409 AS2(    movq     mm2, [edi+1*8]) // b
410 AS2(    movq     mm1, mm2)
411 AS2(    por      mm2, mm4)
412 AS2(    pand     mm2, [edi+2*8]) // c
413 AS2(    pand     mm1, mm4)
414 AS2(    por      mm1, mm2)
415 AS2(    paddq    mm1, mm5) // temp = h + Maj(a,b,c)
416 AS2(    paddq    mm5, [edi+3*8]) // e = d + h
417 AS2(    movq     [edi+3*8], mm5)
418 AS2(    movq     [edi+11*8], mm5)
419 SSE2_s0_S1(mm4,28,34,39) // S0(a)
420 AS2(    paddq    mm4, mm1) // a = temp + S0(a)
421 AS2(    movq     [edi-8], mm4)
422 AS2(    movq     [edi+7*8], mm4)
423 AS1(    ret)
424
425 // first 16 rounds
426 ASL(0)
427 AS2(    movq     mm0, [edx+eax*8])
428 AS2(    movq     [esi+eax*8], mm0)
429 AS2(    movq     [esi+eax*8+16*8], mm0)
430 AS2(    paddq    mm0, [ebx+eax*8])
431 ASC(    call,    SHA512_Round)
432 AS1(    inc     eax)
433 AS2(    sub     edi, 8)
434 AS2(    test    eax, 7)
435 ASJ(    jnz,    0, b)
436 AS2(    add     edi, 8*8)
437 AS2(    cmp     eax, 16)
438 ASJ(    jne,    0, b)
439
440 // rest of the rounds
441 AS2(    movdqu   xmm0, [esi+(16-2)*8])
442 ASL(1)
443 // data expansion, W[i-2] already in xmm0
444 AS2(    movdqu   xmm3, [esi])
445 AS2(    paddq    xmm3, [esi+(16-7)*8])
446 AS2(    movdqa   xmm2, [esi+(16-15)*8])
447 SSE2_s1(xmm0, 6, 19, 61)
448 AS2(    paddq    xmm0, xmm3)
449 SSE2_s0(xmm2, 1, 7, 8)
450 AS2(    paddq    xmm0, xmm2)
451 AS2(    movdq2q  mm0, xmm0)
452 AS2(    movhps   xmm1, xmm0)
453 AS2(    paddq    mm0, [ebx+eax*8])
454 AS2(    movlps   [esi], xmm0)
455 AS2(    movlps   [esi+8], xmm1)
456 AS2(    movlps   [esi+8*16], xmm0)
457 AS2(    movlps   [esi+8*17], xmm1)
458 // 2 rounds
459 ASC(    call,    SHA512_Round)
460 AS2(    sub     edi, 8)
461 AS2(    movdq2q  mm0, xmm1)
462 AS2(    paddq    mm0, [ebx+eax*8+8])
463 ASC(    call,    SHA512_Round)
464 // update indices and loop
465 AS2(    add     esi, 16)
466 AS2(    add     eax, 2)
467 AS2(    sub     edi, 8)
468 AS2(    test    eax, 7)
469 ASJ(    jnz,    1, b)
470 // do housekeeping every 8 rounds
471 AS2(    mov     esi, 0xf)
472 AS2(    and     esi, eax)
473 AS2(    lea     esi, [esp+4+20*8+8+esi*8])
474 AS2(    add     edi, 8*8)
475 AS2(    cmp     eax, 80)
476 ASJ(    jne,    1, b)
477
478 #define SSE2_CombineState(i)\

```

```

479 AS2(    movq    mm0, [edi+i*8])\
480 AS2(    paddq    mm0, [ecx+i*8])\
481 AS2(    movq    [ecx+i*8], mm0)
482
483 SSE2_CombineState(0)
484 SSE2_CombineState(1)
485 SSE2_CombineState(2)
486 SSE2_CombineState(3)
487 SSE2_CombineState(4)
488 SSE2_CombineState(5)
489 SSE2_CombineState(6)
490 SSE2_CombineState(7)
491
492 AS1(    pop     esp)
493 AS1(    emms)
494
495 #if defined(__GNUC__)
496 AS1(    pop     ebx)
497 ".att_syntax prefix;"
498 :
499 : "a" (SHA512_K), "c" (state), "d" (data)
500 : "%esi", "%edi", "memory", "cc"
501 );
502 #else
503 AS1(    pop     edi)
504 AS1(    pop     esi)
505 AS1(    pop     ebx)
506 AS1(    ret)
507 #endif
508 }
509 #endif // #if CRYPTOPP_BOOL_SSE2_ASM_AVAILABLE
510
511 void SHA512::Transform(word64 *state, const word64 *data)
512 {
513     #if CRYPTOPP_BOOL_SSE2_ASM_AVAILABLE && CRYPTOPP_BOOL_X86
514     if (HasSSE2())
515     {
516         SHA512_SSE2_Transform(state, data);
517         return;
518     }
519     #endif
520
521     #define S0(x) (rotrFixed(x,28)^rotrFixed(x,34)^rotrFixed(x,39))
522     #define S1(x) (rotrFixed(x,14)^rotrFixed(x,18)^rotrFixed(x,41))
523     #define s0(x) (rotrFixed(x,1)^rotrFixed(x,8)^(x>>7))
524     #define s1(x) (rotrFixed(x,19)^rotrFixed(x,61)^(x>>6))
525
526     #define R(i) h(i)+=S1(e(i))+Ch(e(i),f(i),g(i))+SHA512_K[i+j]+(j?b1k2(i):b1k0(i));\
527 d(i)+=h(i);h(i)+=S0(a(i))+Maj(a(i),b(i),c(i))
528
529     word64 W[16];
530     word64 T[8];
531     /* Copy context->state[] to working vars */
532     memcpy(T, state, sizeof(T));
533     /* 80 operations, partially loop unrolled */
534     for (unsigned int j=0; j<80; j+=16)
535     {
536         R( 0); R( 1); R( 2); R( 3);
537         R( 4); R( 5); R( 6); R( 7);
538         R( 8); R( 9); R(10); R(11);
539         R(12); R(13); R(14); R(15);
540     }
541     /* Add the working vars back into context.state[] */
542     state[0] += a(0);
543     state[1] += b(0);
544     state[2] += c(0);
545     state[3] += d(0);
546     state[4] += e(0);
547     state[5] += f(0);
548     state[6] += g(0);
549     state[7] += h(0);
550 }

```

```
551  
552 #endif  
553  
554 }  
555
```

**SUMMARY**

Every block header is hashed with SHA-256 (actually double SHA-256) to produce the block's identifier. Miners must find a hash below a target value, which is computationally expensive but easy to verify.

sha.h

178 lines CPP

THE ONE-WAY FUNCTION

This file contains the SHA-256 implementation—the cryptographic hash function that secures Bitcoin. SHA-256 (Secure Hash Algorithm, 256-bit) takes any input and produces a 256-bit fingerprint with remarkable properties: nearly impossible to reverse (one-way), tiny changes produce completely different outputs (avalanche effect), and finding two inputs with the same output is computationally infeasible (collision resistance).

Satoshi extracted this code from Crypto++, a well-respected open-source cryptography library by Wei Dai (who also invented b-money, a Bitcoin precursor). The file is public domain, allowing unrestricted use. Rather than depending on the full Crypto++ library, Satoshi extracted just the SHA routines—keeping Bitcoin's dependencies minimal and auditable.

SHA-256 permeates Bitcoin: block hashes (double SHA-256), transaction IDs (double SHA-256), merkle trees (double SHA-256), address derivation (SHA-256 followed by RIPEMD-160), and proof-of-work (finding hashes below target). Mining is essentially a SHA-256 lottery—billions of attempts until one wins.

The performance of SHA-256 directly impacts mining profitability. The `Transform()` function here is called trillions of times per second across the Bitcoin network. Modern miners use custom ASICs that implement these exact operations in silicon, achieving efficiencies impossible in software.

```
1 // This file is public domain
```

Public domain dedication—no license restrictions. Satoshi borrowed from Wei Dai's Crypto++ library. Wei Dai is also the creator of b-money (1998), which Satoshi cited in the Bitcoin whitepaper. The cryptographic community is small; influences interconnect.

```
2 // SHA routines extracted as a standalone file from:
3 // Crypto++: a C++ Class Library of Cryptographic Schemes
4 // Version 5.5.2 (9/24/2007)
5 // http://www.cryptopp.com
6 #ifndef CRYPTOPP_SHA_H
7 #define CRYPTOPP_SHA_H
8 #include <stdlib.h>
9
10 namespace CryptoPP
11 {
12
13 //
14 // Dependencies
15 //
16
17 typedef unsigned char byte;
```

Type definitions for portability. In 2009, C++ lacked standardized fixed-width types (`uint32_t` came with C++11). These typedefs ensure `word32` is exactly 32 bits across compilers. SHA-256 operates on 32-bit words; incorrect sizes would produce wrong hashes—catastrophic for consensus.

```
18 typedef unsigned short word16;
19 typedef unsigned int word32;
20 #if defined(_MSC_VER) || defined(__BORLANDC__)
21 typedef unsigned __int64 word64;
22 #else
23 typedef unsigned long long word64;
24 #endif
25
26 template <class T> inline T rotlFixed(T x, unsigned int y)
```

`rotlFixed()` and `rotrFixed()` —bitwise rotation primitives. SHA-256's compression function heavily uses rotations: `rotr(x,2) XOR rotr(x,13) XOR rotr(x,22)` appears in the Σ_0 function. Many CPUs have dedicated rotation instructions; these templates let the compiler use them when available.

```
27 {
28 assert(y < sizeof(T)*8);
29 return T((x<<y) | (x>>(sizeof(T)*8-y)));
30 }
31
32 template <class T> inline T rotrFixed(T x, unsigned int y)
33 {
34 assert(y < sizeof(T)*8);
35 return T((x>>y) | (x<<(sizeof(T)*8-y)));
36 }
37
38 // ***** endian reversal *****
39
40 #ifdef _MSC_VER
```

CRYPTOPP_FAST_ROTATE —compiler-specific optimizations. Detects whether the target CPU has fast rotation instructions. Intel x86/x64, PowerPC, and modern ARM do; older architectures emulate rotations with shifts and ORs (slower). This macro guides code generation for optimal performance per platform.

```

41 #if _MSC_VER >= 1400
42 #define CRYPTOPP_FAST_ROTATE(x) 1
43 #elif _MSC_VER >= 1300
44 #define CRYPTOPP_FAST_ROTATE(x) ((x) == 32 | (x) == 64)
45 #else
46 #define CRYPTOPP_FAST_ROTATE(x) ((x) == 32)
47 #endif
48 #elif (defined(__MWERKS__) && TARGET_CPU_PPC) || \
49 (defined(__GNUC__) && (defined(_ARCH_PWR2) || defined(_ARCH_PWR) || defined(_ARCH_PPC) ||
   defined(_ARCH_PPC64) || defined(_ARCH_COM)))
50 #define CRYPTOPP_FAST_ROTATE(x) ((x) == 32)
51 #elif defined(__GNUC__) && (CRYPTOPP_B00L_X64 || CRYPTOPP_B00L_X86) // depend on GCC's peephole
   optimization to generate rotate instructions
52 #define CRYPTOPP_FAST_ROTATE(x) 1
53 #else
54 #define CRYPTOPP_FAST_ROTATE(x) 0
55 #endif
56
57 inline byte ByteReverse(byte value)
58 {
59     return value;
60 }
61
62 inline word16 ByteReverse(word16 value)
63 {
64     #ifdef CRYPTOPP_BYTESWAP_AVAILABLE
65         return bswap_16(value);
66     #elif defined(_MSC_VER) && _MSC_VER >= 1300
67         return _byteswap_ushort(value);
68     #else
69         return rotlFixed(value, 8U);
70     #endif
71 }
72
73 inline word32 ByteReverse(word32 value)

```

ByteReverse() —endianness conversion. SHA-256 is defined using big-endian (network byte order), but most modern CPUs are little-endian. This function swaps byte order. The GCC inline assembly `bswap %0` uses the single-instruction byte swap on x86—crucial when called billions of times during mining.

```

74 {
75     #if defined(__GNUC__)
76         __asm__ ("bswap %0" : "=r" (value) : "0" (value));
77         return value;
78     #elif defined(CRYPTOPP_BYTESWAP_AVAILABLE)
79         return bswap_32(value);
80     #elif defined(__MWERKS__) && TARGET_CPU_PPC
81         return (word32)__lbrx(&value, 0);
82     #elif _MSC_VER >= 1400 || (_MSC_VER >= 1300 && !defined(_DLL))
83         return _byteswap_ulong(value);
84     #elif CRYPTOPP_FAST_ROTATE(32)
85         // 5 instructions with rotate instruction, 9 without
86         return (rotrFixed(value, 8U) & 0xff00ff00) | (rotlFixed(value, 8U) & 0x00ff00ff);
87     #else
88         // 6 instructions with rotate instruction, 8 without
89         value = ((value & 0xFF00FF00) >> 8) | ((value & 0x00FF00FF) << 8);
90         return rotlFixed(value, 16U);
91     #endif

```

```

92 }
93
94 #ifdef WORD64_AVAILABLE
95 inline word64 ByteReverse(word64 value)
96 {
97     #if defined(__GNUC__) && defined(__x86_64__)
98         __asm__ ("bswap %0" : "=r" (value) : "0" (value));
99         return value;
100     #elif defined(CRYPTOPP_BYTESWAP_AVAILABLE)
101         return bswap_64(value);
102     #elif defined(_MSC_VER) && _MSC_VER >= 1300
103         return _byteswap_uint64(value);
104     #elif defined(CRYPTOPP_SLOW_WORD64)
105         return (word64(ByteReverse(word32(value))) << 32) | ByteReverse(word32(value>>32));
106     #else
107         value = ((value & W64LIT(0xFF00FF00FF00FF00)) >> 8) | ((value & W64LIT(0x00FF00FF00FF00FF)) << 8);
108         value = ((value & W64LIT(0xFFFF0000FFFF0000)) >> 16) | ((value & W64LIT(0x0000FFFF0000FFFF)) << 16);
109         return rotlFixed(value, 32U);
110     #endif
111 }
112 #endif
113
114
115 //
116 // SHA
117 //
118
119 // http://www.weidai.com/scan-mirror/md.html#SHA-1
120 class SHA1

```

SHA1—160-bit hash, predecessor to SHA-256. Bitcoin uses SHA1 indirectly via RIPEMD160—addresses are RIPEMD160(SHA256(pubkey)). SHA1 is considered cryptographically weak (Google demonstrated a collision in 2017), but Bitcoin doesn't rely on SHA1's collision resistance—only SHA256's.

```

121 {
122 public:
123     typedef word32 HashWordType;
124     static void InitState(word32 *state);
125     static void Transform(word32 *digest, const word32 *data);
126     static const char * StaticAlgorithmName() {return "SHA-1";}
127 };
128
129 typedef SHA1 SHA; // for backwards compatibility
130
131 // implements the SHA-256 standard
132 class SHA256

```

SHA256—the cryptographic heart of Bitcoin. The `Transform()` method processes 512-bit blocks through 64 rounds of mixing, producing 256-bit output. `InitState()` sets the initial hash values (derived from the fractional parts of square roots of the first 8 primes). This is NIST FIPS 180-4's SHA-256, unchanged since standardization in 2001.

```

133 {
134 public:
135     typedef word32 HashWordType;
136     static void InitState(word32 *state);
137     static void Transform(word32 *digest, const word32 *data);

```

`Transform()` —the compression function, called once per 64-byte block of input. Bitcoin mining is essentially: set up a block header (80 bytes), call `Transform` twice (for double-SHA256), check if result is below target, increment nonce, repeat. ASICs implement `Transform` directly in hardware—billions of transforms per second per chip.

```

138 static const char * StaticAlgorithmName() {return "SHA-256";}
139 };
140
141 // implements the SHA-224 standard
142 class SHA224

```

SHA224 —224-bit variant, same `Transform` as SHA256 but different initial values and truncated output. Not used in Bitcoin. Its presence here is incidental (came with the Crypto++ extraction). Demonstrates how SHA-2 family shares implementations.

```

143 {
144 public:
145 typedef word32 HashWordType;
146 static void InitState(word32 *state);
147 static void Transform(word32 *digest, const word32 *data) {SHA256::Transform(digest, data);}
148 static const char * StaticAlgorithmName() {return "SHA-224";}
149 };
150
151 #ifdef WORD64_AVAILABLE
152
153 // implements the SHA-512 standard
154 class SHA512

```

SHA512 —512-bit hash using 64-bit words. Potentially useful for future cryptographic needs (key derivation, HMAC). Not used in Bitcoin's consensus rules, but available in this implementation. SHA-512 is actually faster than SHA-256 on 64-bit CPUs due to better parallelism.

```

155 {
156 public:
157 typedef word64 HashWordType;
158 static void InitState(word64 *state);
159 static void Transform(word64 *digest, const word64 *data);
160 static const char * StaticAlgorithmName() {return "SHA-512";}
161 };
162
163 // implements the SHA-384 standard
164 class SHA384
165 {
166 public:
167 typedef word64 HashWordType;
168 static void InitState(word64 *state);
169 static void Transform(word64 *digest, const word64 *data) {SHA512::Transform(digest, data);}
170 static const char * StaticAlgorithmName() {return "SHA-384";}
171 };
172
173 #endif
174
175 }
176
177 #endif
178

```

SUMMARY

SHA-256's security properties are what make Bitcoin work. The one-way nature means you can't reverse-engineer a private key from a public key hash. The avalanche effect means changing one bit in a block header completely changes the hash—no gradual approach to a valid proof-of-work. Collision resistance means attackers can't create two transactions with the same txid.

The Transform function is the most-executed code in human history. Every second, the Bitcoin network performs approximately 500 exahashes (5×10^{20} double-SHA256 operations). That's Transform() being called roughly 10^{21} times per second globally. No other function comes close. This relentless hashing secures approximately \$1 trillion in value.

Satoshi's choice of double-SHA256 (hashing twice) adds a safety margin. SHA-256's theoretical weakness to length-extension attacks doesn't apply when the result is hashed again. Whether Satoshi knew this or just chose defense-in-depth, the double-hash has proven robust for 15+ years.

ui.cpp

3229 lines CPP

THE FACE OF BITCOIN

The original Bitcoin client had a graphical user interface built with wxWidgets. This file contains the logic behind the GUI—handling button clicks, updating displays, and managing user interactions.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6 #ifdef _MSC_VER
7 #include <crtdbg.h>
8 #endif
9
10
11
12 DEFINE_EVENT_TYPE(wxEVT_CROSSTHREADCALL)
13 DEFINE_EVENT_TYPE(wxEVT_REPLY1)
14 DEFINE_EVENT_TYPE(wxEVT_REPLY2)
15 DEFINE_EVENT_TYPE(wxEVT_REPLY3)
16 DEFINE_EVENT_TYPE(wxEVT_TABLEADDED)
17 DEFINE_EVENT_TYPE(wxEVT_TABLEUPDATED)
18 DEFINE_EVENT_TYPE(wxEVT_TABLEDELETED)
19
20 CMainFrame* pframeMain = NULL;
21 map<string, string> mapAddressBook;
22
23
24 void ThreadRequestProductDetails(void* parg);
25 void ThreadRandSendTest(void* parg);
26 bool fRandSendTest = false;
27 void RandSend();
28
29
30
31
32
33
34 ///////////////////////////////////////////////////////////////////
35 //
36 // Util
37 //
38
39 void HandleCtrlA(wxKeyEvent& event)
40 {
41 // Ctrl-a select all
42 wxTextCtrl* textCtrl = (wxTextCtrl*)event.GetEventObject();
43 if (event.GetModifiers() == wxMOD_CONTROL && event.GetKeyCode() == 'A')
44     textCtrl->SetSelection(-1, -1);
45     event.Skip();
46 }
47
48 bool Is24HourTime()
49 {
50 //char pszHourFormat[256];
51 //pszHourFormat[0] = '\0';
52 //GetLocaleInfo(LOCALE_USER_DEFAULT, LOCALE_ETIME, pszHourFormat, 256);
53 //return (pszHourFormat[0] != '\0');
54 return true;
55 }
56
57 string DateStr(int64 nTime)
58 {
59 return (string)wxDateTime((time_t)nTime).FormatDate();
60 }
61
62 string DateTimeStr(int64 nTime)
63 {
64 wxDateTime datetime((time_t)nTime);
65 if (Is24HourTime())
66 return (string)datetime.Format("%x %H:%M");
67 else
68 return (string)datetime.Format("%x ") + itostr((datetime.GetHour() + 11) % 12 + 1) +
69 (string)datetime.Format(":%M %p");
70 }
71 wxString GetItemText(wxListCtrl* listCtrl, int nIndex, int nColumn)

```

```

72 {
73 // Helper to simplify access to listctrl
74     wxListItem item;
75     item.m_itemId = nIndex;
76     item.m_col = nColumn;
77     item.m_mask = wxLIST_MASK_TEXT;
78     if (!listCtrl->GetItem(item))
79         return "";
80     return item.GetText();
81 }
82
83 int InsertLine(wxListCtrl* listCtrl, const wxString& str0, const wxString& str1)
84 {
85     int nIndex = listCtrl->InsertItem(listCtrl->GetItemCount(), str0);
86     listCtrl->SetItem(nIndex, 1, str1);
87     return nIndex;
88 }
89
90 int InsertLine(wxListCtrl* listCtrl, const wxString& str0, const wxString& str1, const wxString& str2,
91     const wxString& str3, const wxString& str4)
92 {
93     int nIndex = listCtrl->InsertItem(listCtrl->GetItemCount(), str0);
94     listCtrl->SetItem(nIndex, 1, str1);
95     listCtrl->SetItem(nIndex, 2, str2);
96     listCtrl->SetItem(nIndex, 3, str3);
97     listCtrl->SetItem(nIndex, 4, str4);
98     return nIndex;
99 }
100
101 int InsertLine(wxListCtrl* listCtrl, void* pdata, const wxString& str0, const wxString& str1, const
102     wxString& str2, const wxString& str3, const wxString& str4)
103 {
104     int nIndex = listCtrl->InsertItem(listCtrl->GetItemCount(), str0);
105     listCtrl->SetItemPtrData(nIndex, (wxUIntPtr)pdata);
106     listCtrl->SetItem(nIndex, 1, str1);
107     listCtrl->SetItem(nIndex, 2, str2);
108     listCtrl->SetItem(nIndex, 3, str3);
109     listCtrl->SetItem(nIndex, 4, str4);
110     return nIndex;
111 }
112
113 void SetSelection(wxListCtrl* listCtrl, int nIndex)
114 {
115     int nSize = listCtrl->GetItemCount();
116     long nState = (wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED);
117     for (int i = 0; i < nSize; i++)
118         listCtrl->SetItemState(i, (i == nIndex ? nState : 0), nState);
119 }
120
121 int GetSelection(wxListCtrl* listCtrl)
122 {
123     int nSize = listCtrl->GetItemCount();
124     for (int i = 0; i < nSize; i++)
125         if (listCtrl->GetItemState(i, wxLIST_STATE_FOCUSED))
126             return i;
127     return -1;
128 }
129
130 string HtmlEscape(const char* psz, bool fMultiLine=false)
131 {
132     int len = 0;
133     for (const char* p = psz; *p; p++)
134     {
135         if (*p == '<') len += 4;
136         else if (*p == '>') len += 4;
137         else if (*p == '&') len += 5;
138         else if (*p == '"') len += 6;
139         else if (*p == ' ' && p > psz && p[-1] == ' ' && p[1] == ' ') len += 6;
140         else if (*p == '\n' && fMultiLine) len += 5;
141         else
142             len++;
143     }
144 }

```

```

142 }
143     string str;
144     str.reserve(len);
145     for (const char* p = psz; *p; p++)
146     {
147         if (*p == '<') str += "&lt;";
148         else if (*p == '>') str += "&gt;";
149         else if (*p == '&') str += "&amp;";
150         else if (*p == '"') str += "&quot;";
151         else if (*p == ' ' && p > psz && p[-1] == ' ' && p[1] == ' ') str += "&nbsp;";
152         else if (*p == '\n' && fMultiLine) str += "<br>\n";
153         else
154             str += *p;
155     }
156     return str;
157 }
158
159 string HtmlEscape(const string& str, bool fMultiLine=false)
160 {
161     return HtmlEscape(str.c_str(), fMultiLine);
162 }
163
164 void AddToMyProducts(CProduct product)
165 {
166     CProduct& productInsert = mapMyProducts[product.GetHash()];
167     productInsert = product;
168     InsertLine(pframeMain->m_listCtrlProductsSent, &productInsert,
169         product.mapValue["category"],
170         product.mapValue["title"].substr(0, 100),
171         product.mapValue["description"].substr(0, 100),
172         product.mapValue["price"],
173     "");
174 }
175
176
177
178
179
180
181 ///////////////////////////////////////////////////////////////////
182 //
183 // Custom events
184 //
185
186 set<void*> setCallbackAvailable;
187 CCriticalSection cs_setCallbackAvailable;
188
189 void AddCallbackAvailable(void* p)
190 {
191     CRITICAL_BLOCK(cs_setCallbackAvailable)
192         setCallbackAvailable.insert(p);
193 }
194
195 void RemoveCallbackAvailable(void* p)
196 {
197     CRITICAL_BLOCK(cs_setCallbackAvailable)
198         setCallbackAvailable.erase(p);
199 }
200
201 bool IsCallbackAvailable(void* p)
202 {
203     CRITICAL_BLOCK(cs_setCallbackAvailable)
204         return setCallbackAvailable.count(p);
205     return false;
206 }
207
208 template<typename T>
209 void AddPendingCustomEvent(wxEvtHandler* pevthandler, int nEventID, const T pbeginIn, const T pendIn)
210 {
211     if (!pevthandler)
212         return;
213

```

```

214 const char* pbegin = (pendIn != pbeginIn) ? &pbeginIn[0] : NULL;
215 const char* pend = pbegin + (pendIn - pbeginIn) * sizeof(pbeginIn[0]);
216 wxCommandEvent event(nEventID);
217 wxString strData(wxChar(0), (pend - pbegin) / sizeof(wxChar) + 1);
218 memcpy(&strData[0], pbegin, pend - pbegin);
219 event.SetString(strData);
220 event.SetInt(pend - pbegin);
221
222 pevthandler->AddPendingEvent(event);
223 }
224
225 template<class T>
226 void AddPendingCustomEvent(wxEvtHandler* pevthandler, int nEventID, const T& obj)
227 {
228     CDataStream ss;
229     ss << obj;
230 AddPendingCustomEvent(pevthandler, nEventID, ss.begin(), ss.end());
231 }
232
233 void AddPendingReplyEvent1(void* pevthandler, CDataStream& vRecv)
234 {
235     if (IsCallbackAvailable(pevthandler))
236 AddPendingCustomEvent((wxEvtHandler*)pevthandler, wxEVT_REPLY1, vRecv.begin(), vRecv.end());
237 }
238
239 void AddPendingReplyEvent2(void* pevthandler, CDataStream& vRecv)
240 {
241     if (IsCallbackAvailable(pevthandler))
242 AddPendingCustomEvent((wxEvtHandler*)pevthandler, wxEVT_REPLY2, vRecv.begin(), vRecv.end());
243 }
244
245 void AddPendingReplyEvent3(void* pevthandler, CDataStream& vRecv)
246 {
247     if (IsCallbackAvailable(pevthandler))
248 AddPendingCustomEvent((wxEvtHandler*)pevthandler, wxEVT_REPLY3, vRecv.begin(), vRecv.end());
249 }
250
251 CDataStream GetStreamFromEvent(const wxCommandEvent& event)
252 {
253     wxString strData = event.GetString();
254     return CDataStream(strData.begin(), strData.begin() + event.GetInt(), SER_NETWORK);
255 }
256
257
258
259
260
261
262
263 ///////////////////////////////////////////////////
264 //
265 // CMainFrame
266 //
267
268 CMainFrame::CMainFrame(wxWindow* parent) : CMainFrameBase(parent)

```

CMainFrame constructor - initializes the main window

```

269 {
270 Connect(wxEVT_CROSSTHREADCALL, wxCommandEventHandler(CMainFrame::OnCrossThreadCall), NULL, this);
271
272 // Init
273 fRefreshListCtrl = false;
274 fRefreshListCtrlRunning = false;
275 fOnSetFocusAddress = false;
276 pindexBestLast = NULL;
277 m_choiceFilter->SetSelection(0);
278 m_staticTextBalance->SetLabel(FormatMoney(GetBalance()) + " ");
279 m_listCtrl->SetFocus();

```

```

280 SetIcon(wxICON(bitcoin));
281 m_menuOptions->Check(wxID_OPTIONSGENERATEBITCOINS, fGenerateBitcoins);
282
283 // Init toolbar with transparency masked bitmaps
284 m_toolbar->ClearTools();
285
286 //// shouldn't have to do mask separately anymore, bitmap alpha support added in wx 2.8.9,
287 wxBitmap bmpSend(wxT("send20"), wxBITMAP_TYPE_RESOURCE);
288 bmpSend.SetMask(new wxMask(wxBitmap(wxT("send20mask"), wxBITMAP_TYPE_RESOURCE)));
289 m_toolbar->AddTool(wxID_BUTTONSEND, wxT("&Send Coins"), bmpSend, wxNullBitmap, wxITEM_NORMAL,
wxEmptyString, wxEmptyString);
290
291 wxBitmap bmpAddressBook(wxT("addressbook20"), wxBITMAP_TYPE_RESOURCE);
292 bmpAddressBook.SetMask(new wxMask(wxBitmap(wxT("addressbook20mask"), wxBITMAP_TYPE_RESOURCE)));
293 m_toolbar->AddTool(wxID_BUTTONRECEIVE, wxT("&Address Book"), bmpAddressBook, wxNullBitmap,
wxITEM_NORMAL, wxEmptyString, wxEmptyString);
294
295 m_toolbar->Realize();
296
297 // Init column headers
298 int nDateWidth = DateTimeStr(1229413914).size() * 6 + 8;
299 m_listCtrl->InsertColumn(0, "", wxLIST_FORMAT_LEFT, 0);
300 m_listCtrl->InsertColumn(1, "", wxLIST_FORMAT_LEFT, 0);
301 m_listCtrl->InsertColumn(2, "Status", wxLIST_FORMAT_LEFT, 90);
302 m_listCtrl->InsertColumn(3, "Date", wxLIST_FORMAT_LEFT, nDateWidth);
303 m_listCtrl->InsertColumn(4, "Description", wxLIST_FORMAT_LEFT, 409 - nDateWidth);
304 m_listCtrl->InsertColumn(5, "Debit", wxLIST_FORMAT_RIGHT, 79);
305 m_listCtrl->InsertColumn(6, "Credit", wxLIST_FORMAT_RIGHT, 79);
306
307 //m_listCtrlProductsSent->InsertColumn(0, "Category", wxLIST_FORMAT_LEFT, 100);
308 //m_listCtrlProductsSent->InsertColumn(1, "Title", wxLIST_FORMAT_LEFT, 100);
309 //m_listCtrlProductsSent->InsertColumn(2, "Description", wxLIST_FORMAT_LEFT, 100);
310 //m_listCtrlProductsSent->InsertColumn(3, "Price", wxLIST_FORMAT_LEFT, 100);
311 //m_listCtrlProductsSent->InsertColumn(4, "", wxLIST_FORMAT_LEFT, 100);
312
313 //m_listCtrlOrdersSent->InsertColumn(0, "Time", wxLIST_FORMAT_LEFT, 100);
314 //m_listCtrlOrdersSent->InsertColumn(1, "Price", wxLIST_FORMAT_LEFT, 100);
315 //m_listCtrlOrdersSent->InsertColumn(2, "", wxLIST_FORMAT_LEFT, 100);
316 //m_listCtrlOrdersSent->InsertColumn(3, "", wxLIST_FORMAT_LEFT, 100);
317 //m_listCtrlOrdersSent->InsertColumn(4, "", wxLIST_FORMAT_LEFT, 100);
318
319 //m_listCtrlOrdersReceived->InsertColumn(0, "Time", wxLIST_FORMAT_LEFT, 100);
320 //m_listCtrlOrdersReceived->InsertColumn(1, "Price", wxLIST_FORMAT_LEFT, 100);
321 //m_listCtrlOrdersReceived->InsertColumn(2, "Payment Status", wxLIST_FORMAT_LEFT, 100);
322 //m_listCtrlOrdersReceived->InsertColumn(3, "", wxLIST_FORMAT_LEFT, 100);
323 //m_listCtrlOrdersReceived->InsertColumn(4, "", wxLIST_FORMAT_LEFT, 100);
324
325 // Init status bar
326 int pnWidths[3] = { -100, 81, 286 };
327 m_statusBar->SetFieldsCount(3, pnWidths);
328
329 // Fill your address text box
330 vector<unsigned char> vchPubKey;
331 if (CWalletDB("").ReadDefaultKey(vchPubKey))
332 m_textCtrlAddress->SetValue(PubKeyToAddress(vchPubKey));
333
334 // Fill listctrl with wallet transactions
335 RefreshListCtrl();
336 }
337
338 CMainFrame::~CMainFrame()
339 {
340 pframeMain = NULL;
341 }
342
343 void Shutdown(void* parg)
344 {
345 static CCriticalSection cs_Shutdown;
346 CRITICAL_BLOCK(cs_Shutdown)
347 {
348 fShutdown = true;
349 nTransactionsUpdated++;

```



```

350 DBFlush(false);
351 StopNode();
352 DBFlush(true);
353
354 printf("Bitcoin exiting\n");
355 exit(0);
356 }
357 }
358
359 void CMainFrame::OnClose(wxCloseEvent& event)
360 {
361 Destroy();
362 _beginthread(Shutdown, 0, NULL);
363 }
364
365 void CMainFrame::OnMouseEvents(wxMouseEvent& event)
366 {
367 RandAddSeed();
368 RAND_add(&event.m_x, sizeof(event.m_x), 0.25);
369 RAND_add(&event.m_y, sizeof(event.m_y), 0.25);
370 }
371
372 void CMainFrame::OnListColBeginDrag(wxListEvent& event)
373 {
374 // Hidden columns not resizable
375 if (event.GetColumn() <= 1 && !fDebug)
376     event.Veto();
377 }
378
379 void CMainFrame::InsertLine(bool fNew, int nIndex, uint256 hashKey, string strSort, const wxString&
    str2, const wxString& str3, const wxString& str4, const wxString& str5, const wxString& str6)
380 {
381     string str0 = strSort;
382     long nData = *(long*)&hashKey;
383
384     if (fNew)
385     {
386         nIndex = m_listCtrl->InsertItem(0, str0);
387     }
388     else
389     {
390         if (nIndex == -1)
391         {
392             // Find item
393             while ((nIndex = m_listCtrl->FindItem(nIndex, nData)) != -1)
394                 if (GetItemText(m_listCtrl, nIndex, 1) == hashKey.ToString())
395                     break;
396             if (nIndex == -1)
397             {
398                 printf("CMainFrame::InsertLine : Couldn't find item to be updated\n");
399                 return;
400             }
401         }
402
403         // If sort key changed, must delete and reinsert to make it relocate
404         if (GetItemText(m_listCtrl, nIndex, 0) != str0)
405         {
406             m_listCtrl->DeleteItem(nIndex);
407             nIndex = m_listCtrl->InsertItem(0, str0);
408         }
409     }
410
411     m_listCtrl->SetItem(nIndex, 1, hashKey.ToString());
412     m_listCtrl->SetItem(nIndex, 2, str2);
413     m_listCtrl->SetItem(nIndex, 3, str3);
414     m_listCtrl->SetItem(nIndex, 4, str4);
415     m_listCtrl->SetItem(nIndex, 5, str5);
416     m_listCtrl->SetItem(nIndex, 6, str6);
417     m_listCtrl->SetItemData(nIndex, nData);
418 }
419
420 string FormatTxStatus(const CWalletTx& wtx)

```

```

421 {
422 // Status
423 int nDepth = wtx.GetDepthInMainChain();
424 if (!wtx.IsFinal())
425 return sprintf("Open for %d blocks", nBestHeight - wtx.nLockTime);
426 else if (nDepth < 6)
427 return sprintf("%d/unconfirmed", nDepth);
428 else
429 return sprintf("%d blocks", nDepth);
430 }
431
432 void CMainFrame::InsertTransaction(const CWalletTx& wtx, bool fNew, int nIndex)

```

InsertTransaction - displays a transaction in the list

```

433 {
434     int64 nTime = wtx.nTimeDisplayed = wtx.GetTxTime();
435     int64 nCredit = wtx.GetCredit();
436     int64 nDebit = wtx.GetDebit();
437     int64 nNet = nCredit - nDebit;
438     uint256 hash = wtx.GetHash();
439     string strStatus = FormatTxStatus(wtx);
440     map<string, string> mapValue = wtx.mapValue;
441
442 // Find the block the tx is in
443     CBlockIndex* pindex = NULL;
444     map<uint256, CBlockIndex*>::iterator mi = mapBlockIndex.find(wtx.hashBlock);
445     if (mi != mapBlockIndex.end())
446         pindex = (*mi).second;
447
448 // Sort order, unrecorded transactions sort to the top
449     string strSort = sprintf("%010d-%01d-%010u",
450 (pindex ? pindex->nHeight : INT_MAX),
451 (wtx.IsCoinBase() ? 1 : 0),
452     wtx.nTimeReceived);
453
454 // Insert line
455 if (nNet > 0 || wtx.IsCoinBase())
456 {
457 //
458 // Credit
459 //
460         string strDescription;
461
462 if (wtx.IsCoinBase())
463 {
464 // Coinbase
465         strDescription = "Generated";
466 if (nCredit == 0)
467 {
468             int64 nUnmatured = 0;
469 foreach(const CTxOut& txout, wtx.vout)
470             nUnmatured += txout.GetCredit();
471 if (wtx.IsInMainChain())
472             strDescription += sprintf(" (%s matures in %d blocks)",
473 FormatMoney(nUnmatured).c_str(), wtx.GetBlocksToMaturity());
474 else
475             strDescription += " (not accepted)";
476 }
477 else if (!mapValue["from"].empty() || !mapValue["message"].empty())
478 {
479 // Online transaction
480 if (!mapValue["from"].empty())
481             strDescription += "From: " + mapValue["from"];
482 if (!mapValue["message"].empty())
483 {
484 if (!strDescription.empty())
485             strDescription += " - ";

```

```

486         strDescription += mapValue["message"];
487     }
488 }
489 else
490 {
491     // Offline transaction
492     foreach(const CTxOut& txout, wtx.vout)
493     {
494         if (txout.IsMine())
495         {
496             vector<unsigned char> vchPubKey;
497             if (ExtractPubKey(txout.scriptPubKey, true, vchPubKey))
498             {
499                 string strAddress = PubKeyToAddress(vchPubKey);
500                 if (mapAddressBook.count(strAddress))
501                 {
502                     strDescription += "Received with: ";
503                     if (!mapAddressBook[strAddress].empty())
504                         strDescription += mapAddressBook[strAddress] + " ";
505                     strDescription += strAddress;
506                 }
507             }
508             break;
509         }
510     }
511 }
512
513     string strDescription2;
514     foreach(int c, strDescription)
515     if (c >= ' ')
516         strDescription2 += c;
517
518     InsertLine(fNew, nIndex, hash, strSort,
519               strStatus,
520               nTime ? DateTimeStr(nTime) : "",
521               strDescription2,
522               "",
523               FormatMoney(nNet, true));
524 }
525 else
526 {
527     bool fAllFromMe = true;
528     foreach(const CTxIn& txin, wtx.vin)
529         fAllFromMe = fAllFromMe && txin.IsMine();
530
531     bool fAllToMe = true;
532     foreach(const CTxOut& txout, wtx.vout)
533         fAllToMe = fAllToMe && txout.IsMine();
534
535     if (fAllFromMe && fAllToMe)
536     {
537         // Payment to self
538         int64 nValue = wtx.vout[0].nValue;
539         InsertLine(fNew, nIndex, hash, strSort,
540                   strStatus,
541                   nTime ? DateTimeStr(nTime) : "",
542                   "Payment to yourself",
543                   FormatMoney(nNet - nValue, true),
544                   FormatMoney(nValue, true));
545     }
546     else if (fAllFromMe)
547     {
548         //
549         // Debit
550         //
551         int64 nTxFee = nDebit - wtx.GetValueOut();
552         for (int nOut = 0; nOut < wtx.vout.size(); nOut++)
553         {
554             const CTxOut& txout = wtx.vout[nOut];
555             if (txout.IsMine())
556                 continue;
557

```

```

558         string strAddress;
559         if (!mapValue["to"].empty())
560         {
561             // Online transaction
562             strAddress = mapValue["to"];
563         }
564         else
565         {
566             // Offline transaction
567             uint160 hash160;
568             if (ExtractHash160(txout.scriptPubKey, hash160))
569                 strAddress = Hash160ToAddress(hash160);
570         }
571
572         string strDescription = "To: ";
573         if (mapAddressBook.count(strAddress) && !mapAddressBook[strAddress].empty())
574             strDescription += mapAddressBook[strAddress] + " ";
575         strDescription += strAddress;
576         if (!mapValue["message"].empty())
577         {
578             if (!strDescription.empty())
579                 strDescription += " - ";
580             strDescription += mapValue["message"];
581         }
582
583         string strDescription2;
584         foreach(int c, strDescription)
585             if (c >= ' ')
586                 strDescription2 += c;
587
588         int64 nValue = txout.nValue;
589         if (nOut == 0 && nTxFee > 0)
590             nValue += nTxFee;
591
592         InsertLine(fNew, nIndex, hash, sprintf("%s-%d", strSort.c_str(), nOut),
593                 strStatus,
594                 nTime ? DateTimeStr(nTime) : "",
595                 strDescription2,
596         FormatMoney(-nValue, true),
597         "");
598     }
599 }
600 else
601 {
602     //
603     // Mixed debit transaction, can't break down payees
604     //
605     bool fAllMine = true;
606     foreach(const CTxOut& txout, wtx.vout)
607         fAllMine = fAllMine && txout.IsMine();
608     foreach(const CTxIn& txin, wtx.vin)
609         fAllMine = fAllMine && txin.IsMine();
610
611     InsertLine(fNew, nIndex, hash, strSort,
612             strStatus,
613             nTime ? DateTimeStr(nTime) : "",
614             "",
615         FormatMoney(nNet, true),
616         "");
617 }
618 }
619 }
620
621 void CMainFrame::RefreshStatus()

```

RefreshStatus - updates balance and connection status

```

622 {
623     static int nLastTop;

```

```

624 int nTop = m_listCtrl->GetTopItem();
625 if (nTop == nLastTop && pindexBestLast == pindexBest)
626     return;
627
628 TRY_CRITICAL_BLOCK(cs_mapWallet)
629 {
630     int nStart = nTop;
631     int nEnd = min(nStart + 100, m_listCtrl->GetItemCount());
632     if (pindexBestLast == pindexBest)
633     {
634         if (nStart >= nLastTop && nStart < nLastTop + 100)
635             nStart = nLastTop + 100;
636         if (nEnd >= nLastTop && nEnd < nLastTop + 100)
637             nEnd = nLastTop;
638     }
639     nLastTop = nTop;
640     pindexBestLast = pindexBest;
641
642     for (int nIndex = nStart; nIndex < nEnd; nIndex++)
643     {
644         uint256 hash((string)GetItemText(m_listCtrl, nIndex, 1));
645         map<uint256, CWalletTx>::iterator mi = mapWallet.find(hash);
646         if (mi == mapWallet.end())
647         {
648             printf("CMainFrame::RefreshStatus() : tx not found in mapWallet\n");
649             continue;
650         }
651         const CWalletTx& wtx = (*mi).second;
652         if (wtx.IsCoinBase() || wtx.GetTxTime() != wtx.nTimeDisplayed)
653             InsertTransaction(wtx, false, nIndex);
654         else
655             m_listCtrl->SetItem(nIndex, 2, FormatTxStatus(wtx));
656     }
657 }
658
659 void CMainFrame::RefreshListCtrl()
660 {
661     fRefreshListCtrl = true;
662     ::wxWakeUpIdle();
663 }
664
665 void CMainFrame::OnIdle(wxIdleEvent& event)
666 {
667     if (fRefreshListCtrl)
668     {
669         // Collect list of wallet transactions and sort newest first
670         bool fEntered = false;
671         vector<pair<unsigned int, uint256> > vSorted;
672         TRY_CRITICAL_BLOCK(cs_mapWallet)
673         {
674             printf("RefreshListCtrl starting\n");
675             fEntered = true;
676             fRefreshListCtrl = false;
677             vWalletUpdated.clear();
678
679             // Do the newest transactions first
680             vSorted.reserve(mapWallet.size());
681             for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
682             {
683                 const CWalletTx& wtx = (*it).second;
684                 unsigned int nTime = UINT_MAX - wtx.GetTxTime();
685                 vSorted.push_back(make_pair(nTime, (*it).first));
686             }
687             m_listCtrl->DeleteAllItems();
688
689             if (!fEntered)
690                 return;
691             sort(vSorted.begin(), vSorted.end());
692
693             // Fill list control

```

```

696 for (int i = 0; i < vSorted.size();)
697 {
698     if (fShutdown)
699         return;
700     bool fEntered = false;
701     TRY_CRITICAL_BLOCK(cs_mapWallet)
702     {
703         fEntered = true;
704         uint256& hash = vSorted[i++].second;
705         map<uint256, CWalletTx>::iterator mi = mapWallet.find(hash);
706         if (mi != mapWallet.end())
707             InsertTransaction((*mi).second, true);
708     }
709     if (!fEntered || i == 100 || i % 500 == 0)
710         wxYield();
711 }
712
713 printf("RefreshListCtrl done\n");
714 }
715 else
716 {
717     // Check for time updates
718     static int64 nLastTime;
719     if (GetTime() > nLastTime + 30)
720     {
721         TRY_CRITICAL_BLOCK(cs_mapWallet)
722         {
723             nLastTime = GetTime();
724             for (map<uint256, CWalletTx>::iterator it = mapWallet.begin(); it != mapWallet.end(); ++it)
725             {
726                 CWalletTx& wtx = (*it).second;
727                 if (wtx.nTimeDisplayed && wtx.nTimeDisplayed != wtx.GetTxTime())
728                     InsertTransaction(wtx, false);
729             }
730         }
731     }
732 }
733 }
734
735 void CMainFrame::OnPaint(wxPaintEvent& event)
736 {
737     event.Skip();
738 }
739
740 void CMainFrame::OnPaintListCtrl(wxPaintEvent& event)
741 {
742     // Update listctrl contents
743     if (!vWalletUpdated.empty())
744     {
745         TRY_CRITICAL_BLOCK(cs_mapWallet)
746         {
747             pair<uint256, bool> item;
748             foreach(item, vWalletUpdated)
749             {
750                 bool fNew = item.second;
751                 map<uint256, CWalletTx>::iterator mi = mapWallet.find(item.first);
752                 if (mi != mapWallet.end())
753                 {
754                     printf("vWalletUpdated: %s %s\n", (*mi).second.GetHash().ToString().substr(0,6).c_str(), fNew ? "new" :
755                             "");
756                     InsertTransaction((*mi).second, fNew);
757                 }
758             }
759             m_listCtrl->ScrollList(0, INT_MAX);
760             vWalletUpdated.clear();
761         }
762     }
763     // Update status column of visible items only
764     RefreshStatus();
765 }
766 // Update status bar

```

```

767     string strGen = "";
768     if (fGenerateBitcoins)
769         strGen = "    Generating";
770     if (fGenerateBitcoins && vNodes.empty())
771         strGen = "(not connected)";
772     m_statusBar->SetStatusText(strGen, 1);
773
774     string strStatus = sprintf("    %d connections    %d blocks    %d transactions", vNodes.size(),
nBestHeight + 1, m_listCtrl->GetItemCount());
775     m_statusBar->SetStatusText(strStatus, 2);
776
777     // Balance total
778     TRY_CRITICAL_BLOCK(cs_mapWallet)
779         m_staticTextBalance->SetLabel(FormatMoney(GetBalance()) + "    ");
780
781     m_listCtrl->OnPaint(event);
782 }
783
784 void CrossThreadCall(wxCommandEvent& event)
785 {
786     if (pframeMain)
787         pframeMain->GetEventHandler()->AddPendingEvent(event);
788 }
789
790 void CrossThreadCall(int nID, void* pdata)
791 {
792     wxCommandEvent event;
793     event.SetInt(nID);
794     event.SetClientData(pdata);
795     if (pframeMain)
796         pframeMain->GetEventHandler()->AddPendingEvent(event);
797 }
798
799 void CMainFrame::OnCrossThreadCall(wxCommandEvent& event)
800 {
801     void* pdata = event.GetClientData();
802     switch (event.GetInt())
803     {
804     case UICALL_ADDORDER:
805     {
806         break;
807     }
808
809     case UICALL_UPDATEORDER:
810     {
811         break;
812     }
813     }
814 }
815
816 void CMainFrame::OnMenuFileExit(wxCommandEvent& event)
817 {
818     Close(true);
819 }
820
821 void CMainFrame::OnMenuOptionsGenerate(wxCommandEvent& event)
822 {
823     fGenerateBitcoins = event.IsChecked();
824     nTransactionsUpdated++;
825     CWalletDB().WriteSetting("fGenerateBitcoins", fGenerateBitcoins);
826
827     if (fGenerateBitcoins)
828         if (!_beginthread(ThreadBitcoinMiner, 0, NULL) == -1)
829             printf("Error: _beginthread(ThreadBitcoinMiner) failed\n");
830
831     Refresh();
832     wxPaintEvent eventPaint;
833     AddPendingEvent(eventPaint);
834 }
835
836 void CMainFrame::OnMenuOptionsOptions(wxCommandEvent& event)
837 {

```

```

838     COptionsDialog dialog(this);
839     dialog.ShowModal();
840 }
841
842 void CMainFrame::OnMenuHelpAbout(wxCommandEvent& event)
843 {
844     CAboutDialog dialog(this);
845     dialog.ShowModal();
846 }
847
848 void CMainFrame::OnButtonSend(wxCommandEvent& event)
849 {
850     /// debug test
851     if (fRandSendTest)
852     {
853         RandSend();
854         return;
855     }
856
857     // Toolbar: Send
858     CSendDialog dialog(this);
859     dialog.ShowModal();
860 }
861
862 void CMainFrame::OnButtonAddressBook(wxCommandEvent& event)
863 {
864     // Toolbar: Address Book
865     CAddressBookDialog dialogAddr(this, "", false);
866     if (dialogAddr.ShowModal() == 2)
867     {
868         // Send
869         CSendDialog dialogSend(this, dialogAddr.GetAddress());
870         dialogSend.ShowModal();
871     }
872 }
873
874 void CMainFrame::OnSetFocusAddress(wxFocusEvent& event)
875 {
876     // Automatically select-all when entering window
877     m_textCtrlAddress->SetSelection(-1, -1);
878     fOnSetFocusAddress = true;
879     event.Skip();
880 }
881
882 void CMainFrame::OnMouseEventsAddress(wxMouseEvent& event)
883 {
884     if (fOnSetFocusAddress)
885         m_textCtrlAddress->SetSelection(-1, -1);
886     fOnSetFocusAddress = false;
887     event.Skip();
888 }
889
890 void CMainFrame::OnButtonCopy(wxCommandEvent& event)
891 {
892     // Copy address box to clipboard
893     if (wxTheClipboard->Open())
894     {
895         wxTheClipboard->SetData(new wxTextDataObject(m_textCtrlAddress->GetValue()));
896         wxTheClipboard->Close();
897     }
898 }
899
900 void CMainFrame::OnButtonChange(wxCommandEvent& event)
901 {
902     CYourAddressDialog dialog(this, string(m_textCtrlAddress->GetValue()));
903     if (!dialog.ShowModal())
904         return;
905     string strAddress = (string)dialog.GetAddress();
906     if (strAddress != m_textCtrlAddress->GetValue())
907     {
908         uint160 hash160;
909         if (!AddressToHash160(strAddress, hash160))

```



```

910 return;
911 if (!mapPubKeys.count(hash160))
912 return;
913 CWalletDB().WriteDefaultKey(mapPubKeys[hash160]);
914 m_textCtrlAddress->SetValue(strAddress);
915 }
916 }
917
918 void CMainFrame::OnListItemActivatedAllTransactions(wxListEvent& event)
919 {
920     uint256 hash((string).GetItemText(m_listCtrl, event.GetIndex(), 1));
921     CWalletTx wtx;
922     CRITICAL_BLOCK(cs_mapWallet)
923     {
924         map<uint256, CWalletTx>::iterator mi = mapWallet.find(hash);
925         if (mi == mapWallet.end())
926         {
927             printf("CMainFrame::OnListItemActivatedAllTransactions() : tx not found in mapWallet\n");
928             return;
929         }
930         wtx = (*mi).second;
931     }
932     CTxDetailsDialog dialog(this, wtx);
933     dialog.ShowModal();
934     //CTxDetailsDialog* pdialog = new CTxDetailsDialog(this, wtx);
935     //pdialog->Show();
936 }
937
938 void CMainFrame::OnListItemActivatedProductsSent(wxListEvent& event)
939 {
940     CProduct& product = *(CProduct*)event.GetItem().GetData();
941     CEditProductDialog* pdialog = new CEditProductDialog(this);
942     pdialog->SetProduct(product);
943     pdialog->Show();
944 }
945
946 void CMainFrame::OnListItemActivatedOrdersSent(wxListEvent& event)
947 {
948     CWalletTx& order = *(CWalletTx*)event.GetItem().GetData();
949     CViewOrderDialog* pdialog = new CViewOrderDialog(this, order, false);
950     pdialog->Show();
951 }
952
953 void CMainFrame::OnListItemActivatedOrdersReceived(wxListEvent& event)
954 {
955     CWalletTx& order = *(CWalletTx*)event.GetItem().GetData();
956     CViewOrderDialog* pdialog = new CViewOrderDialog(this, order, true);
957     pdialog->Show();
958 }
959
960
961
962
963
964
965
966 //////////////////////////////////////
967 //
968 // CTxDetailsDialog
969 //
970
971 CTxDetailsDialog::CTxDetailsDialog(wxWindow* parent, CWalletTx wtx) : CTxDetailsDialogBase(parent)
972 {
973     string strHTML;
974     strHTML.reserve(4000);
975
976     int64 nTime = wtx.GetTxTime();
977     int64 nCredit = wtx.GetCredit();
978     int64 nDebit = wtx.GetDebit();
979     int64 nNet = nCredit - nDebit;
980
981

```

```

982
983
984     strHTML += "<b>Status:</b> " + FormatTxStatus(wtx) + "<br>";
985     strHTML += "<b>Date:</b> " + (nTime ? DateTimeStr(nTime) : "") + "<br>";
986
987
988 //
989 // From
990 //
991 if (wtx.IsCoinBase())
992 {
993     strHTML += "<b>Source:</b> Generated<br>";
994 }
995 else if (!wtx.mapValue["from"].empty())
996 {
997     // Online transaction
998     if (!wtx.mapValue["from"].empty())
999         strHTML += "<b>From:</b> " + HtmlEscape(wtx.mapValue["from"]) + "<br>";
1000 }
1001 else
1002 {
1003     // Offline transaction
1004     foreach(const CTxOut& txout, wtx.vout)
1005     {
1006         if (txout.IsMine())
1007         {
1008             vector<unsigned char> vchPubKey;
1009             if (ExtractPubKey(txout.scriptPubKey, true, vchPubKey))
1010             {
1011                 string strAddress = PubKeyToAddress(vchPubKey);
1012                 if (mapAddressBook.count(strAddress))
1013                 {
1014                     strHTML += "<b>Received with:</b> ";
1015                     if (!mapAddressBook[strAddress].empty())
1016                         strHTML += mapAddressBook[strAddress] + " ";
1017                     strHTML += HtmlEscape(strAddress);
1018                     strHTML += "<br>";
1019                 }
1020             }
1021             break;
1022         }
1023     }
1024 }
1025
1026
1027 //
1028 // To
1029 //
1030     string strAddress;
1031     if (!wtx.mapValue["to"].empty())
1032     {
1033         // Online transaction
1034         strAddress = wtx.mapValue["to"];
1035         strHTML += "<b>To:</b> ";
1036         if (mapAddressBook.count(strAddress) && !mapAddressBook[strAddress].empty())
1037             strHTML += mapAddressBook[strAddress] + " ";
1038         strHTML += HtmlEscape(strAddress) + "<br>";
1039     }
1040
1041
1042 //
1043 // Amount
1044 //
1045 if (wtx.IsCoinBase() && nCredit == 0)
1046 {
1047     //
1048     // Coinbase
1049     //
1050     int64 nUnmatured = 0;
1051     foreach(const CTxOut& txout, wtx.vout)
1052         nUnmatured += txout.GetCredit();
1053     if (wtx.IsInMainChain())

```

```

1054         strHTML += sprintf("<b>Credit:</b> (%s matures in %d blocks)<br>",
FormatMoney(nUnmatured).c_str(), wtx.GetBlocksToMaturity());
1055     else
1056         strHTML += "<b>Credit:</b> (not accepted)<br>";
1057 }
1058 else if (nNet > 0)
1059 {
1060 //
1061 // Credit
1062 //
1063     strHTML += "<b>Credit:</b> " + FormatMoney(nNet) + "<br>";
1064 }
1065 else
1066 {
1067     bool fAllFromMe = true;
1068     foreach(const CTxIn& txin, wtx.vin)
1069         fAllFromMe = fAllFromMe && txin.IsMine();
1070
1071     bool fAllToMe = true;
1072     foreach(const CTxOut& txout, wtx.vout)
1073         fAllToMe = fAllToMe && txout.IsMine();
1074
1075     if (fAllFromMe)
1076     {
1077 //
1078 // Debit
1079 //
1080     foreach(const CTxOut& txout, wtx.vout)
1081     {
1082         if (txout.IsMine())
1083             continue;
1084
1085         string strAddress;
1086         if (!wtx.mapValue["to"].empty())
1087         {
1088 // Online transaction
1089             strAddress = wtx.mapValue["to"];
1090         }
1091         else
1092         {
1093 // Offline transaction
1094             uint160 hash160;
1095             if (ExtractHash160(txout.scriptPubKey, hash160))
1096                 strAddress = Hash160ToAddress(hash160);
1097         }
1098
1099         strHTML += "<b>Debit:</b> " + FormatMoney(-txout.nValue) + " &nbsp;&nbsp;&nbsp;&nbsp;&nbsp; ";
1100         strHTML += "(to ";
1101         if (mapAddressBook.count(strAddress) && !mapAddressBook[strAddress].empty())
1102             strHTML += mapAddressBook[strAddress] + " ";
1103         strHTML += strAddress;
1104         strHTML += "<br>";
1105     }
1106
1107     if (fAllToMe)
1108     {
1109 // Payment to self
1110         int64 nValue = wtx.vout[0].nValue;
1111         strHTML += "<b>Debit:</b> " + FormatMoney(-nValue) + "<br>";
1112         strHTML += "<b>Credit:</b> " + FormatMoney(nValue) + "<br>";
1113     }
1114
1115     int64 nTxFee = nDebit - wtx.GetValueOut();
1116     if (nTxFee > 0)
1117         strHTML += "<b>Transaction fee:</b> " + FormatMoney(-nTxFee) + "<br>";
1118 }
1119 else
1120 {
1121 //
1122 // Mixed debit transaction
1123 //
1124     foreach(const CTxIn& txin, wtx.vin)

```

```

1125 if (txin.IsMine())
1126     strHTML += "<b>Debit:</b> " + FormatMoney(-txin.GetDebit()) + "<br>";
1127 foreach(const CTxOut& txout, wtx.vout)
1128 if (txout.IsMine())
1129     strHTML += "<b>Credit:</b> " + FormatMoney(txout.GetCredit()) + "<br>";
1130 }
1131 }
1132
1133 strHTML += "<b>Net amount:</b> " + FormatMoney(nNet, true) + "<br>";
1134
1135
1136 //
1137 // Message
1138 //
1139 if (!wtx.mapValue["message"].empty())
1140     strHTML += "<br><b>Message:</b><br>" + HtmlEscape(wtx.mapValue["message"], true) + "<br>";
1141
1142
1143 //
1144 // Debug view
1145 //
1146 if (fDebug)
1147 {
1148     strHTML += "<hr><br>debug print<br><br>";
1149 foreach(const CTxIn& txin, wtx.vin)
1150 if (txin.IsMine())
1151     strHTML += "<b>Debit:</b> " + FormatMoney(-txin.GetDebit()) + "<br>";
1152 foreach(const CTxOut& txout, wtx.vout)
1153 if (txout.IsMine())
1154     strHTML += "<b>Credit:</b> " + FormatMoney(txout.GetCredit()) + "<br>";
1155
1156     strHTML += "<b>Inputs:</b><br>";
1157 CRITICAL_BLOCK(cs_mapWallet)
1158 {
1159     foreach(const CTxIn& txin, wtx.vin)
1160     {
1161         COutPoint prevout = txin.prevout;
1162         map<uint256, CWalletTx>::iterator mi = mapWallet.find(prevout.hash);
1163         if (mi != mapWallet.end())
1164         {
1165             const CWalletTx& prev = (*mi).second;
1166             if (prevout.n < prev.vout.size())
1167             {
1168                 strHTML += HtmlEscape(prev.ToString(), true);
1169                 strHTML += " &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& " + FormatTxStatus(prev) + ", ";
1170                 strHTML = strHTML + "IsMine=" + (prev.vout[prevout.n].IsMine() ? "true" :
1171 "false") + "<br>";
1172             }
1173         }
1174     }
1175
1176     strHTML += "<br><hr><br><b>Transaction:</b><br>";
1177     strHTML += HtmlEscape(wtx.ToString(), true);
1178 }
1179
1180
1181 string(strHTML.begin(), strHTML.end()).swap(strHTML);
1182 m_htmlWin->SetPage(strHTML);
1183 m_buttonOK->SetFocus();
1184 }
1185
1186 void CTxDetailsDialog::OnButtonOK(wxCommandEvent& event)
1187 {
1188     Close();
1189     //Destroy();
1190 }
1191
1192
1193
1194
1195

```

```

1196 //////////////////////////////////////
1197 //
1198 // COptionsDialog
1199 //
1200
1201 COptionsDialog::COptionsDialog(wxWindow* parent) : COptionsDialogBase(parent)
1202 {
1203     m_textCtrlTransactionFee->SetValue(FormatMoney(nTransactionFee));
1204     m_buttonOK->SetFocus();
1205 }
1206
1207 void COptionsDialog::OnKillFocusTransactionFee(wxFocusEvent& event)
1208 {
1209     int64 nTmp = nTransactionFee;
1210     ParseMoney(m_textCtrlTransactionFee->GetValue(), nTmp);
1211     m_textCtrlTransactionFee->SetValue(FormatMoney(nTmp));
1212 }
1213
1214 void COptionsDialog::OnButtonOK(wxCommandEvent& event)
1215 {
1216     // nTransactionFee
1217     int64 nPrevTransactionFee = nTransactionFee;
1218     if (ParseMoney(m_textCtrlTransactionFee->GetValue(), nTransactionFee) && nTransactionFee !=
        nPrevTransactionFee)
1219         CWalletDB().WriteSetting("nTransactionFee", nTransactionFee);
1220
1221     Close();
1222 }
1223
1224 void COptionsDialog::OnButtonCancel(wxCommandEvent& event)
1225 {
1226     Close();
1227 }
1228
1229
1230
1231
1232
1233
1234 //////////////////////////////////////
1235 //
1236 // CAboutDialog
1237 //
1238
1239 CAboutDialog::CAboutDialog(wxWindow* parent) : CAboutDialogBase(parent)
1240 {
1241     m_staticTextVersion->SetLabel(strprintf("version 0.%d.%d Alpha", VERSION/100, VERSION%100));
1242
1243     // Workaround until upgrade to wxWidgets supporting UTF-8
1244     wxString str = m_staticTextMain->GetLabel();
1245     if (str.Find('\0') != wxNOT_FOUND)
1246         str.Remove(str.Find('\0'), 1);
1247     m_staticTextMain->SetLabel(str);
1248 }
1249
1250 void CAboutDialog::OnButtonOK(wxCommandEvent& event)
1251 {
1252     Close();
1253 }
1254
1255
1256
1257
1258
1259
1260 //////////////////////////////////////
1261 //
1262 // CSendDialog
1263 //
1264
1265 CSendDialog::CSendDialog(wxWindow* parent, const wxString& strAddress) : CSendDialogBase(parent)
1266 {

```

```

1267 // Init
1268     m_textCtrlAddress->SetValue(strAddress);
1269     m_choiceTransferType->SetSelection(0);
1270     m_bitmapCheckMark->Show(false);
1271 //// todo: should add a display of your balance for convenience
1272
1273 // Set Icon
1274     wxBitmap bmpSend(wxT("send16"), wxBITMAP_TYPE_RESOURCE);
1275     bmpSend.SetMask(new wxMask(wxBitmap(wxT("send16masknoshadow"), wxBITMAP_TYPE_RESOURCE)));
1276     wxIcon iconSend;
1277     iconSend.CopyFromBitmap(bmpSend);
1278     SetIcon(iconSend);
1279
1280     wxCommandEvent event;
1281     OnTextAddress(event);
1282
1283 // Fixup the tab order
1284     m_buttonPaste->MoveAfterInTabOrder(m_buttonCancel);
1285     m_buttonAddress->MoveAfterInTabOrder(m_buttonPaste);
1286     this->Layout();
1287 }
1288
1289 void CSendDialog::OnTextAddress(wxCommandEvent& event)
1290 {
1291     // Check mark
1292     bool fBitcoinAddress = IsValidBitcoinAddress(m_textCtrlAddress->GetValue());
1293     m_bitmapCheckMark->Show(fBitcoinAddress);
1294
1295     // Grey out message if bitcoin address
1296     bool fEnable = !fBitcoinAddress;
1297     m_staticTextFrom->Enable(fEnable);
1298     m_textCtrlFrom->Enable(fEnable);
1299     m_staticTextMessage->Enable(fEnable);
1300     m_textCtrlMessage->Enable(fEnable);
1301     m_textCtrlMessage->SetBackgroundColour(wxSystemSettings::GetColour(fEnable ? wxSYS_COLOUR_WINDOW :
1302     wxSYS_COLOUR_BTNFACE));
1303 }
1304
1305 void CSendDialog::OnKillFocusAmount(wxFocusEvent& event)
1306 {
1307     // Reformat the amount
1308     if (m_textCtrlAmount->GetValue().Trim().empty())
1309         return;
1310     int64 nTmp;
1311     if (ParseMoney(m_textCtrlAmount->GetValue(), nTmp))
1312         m_textCtrlAmount->SetValue(FormatMoney(nTmp));
1313 }
1314
1315 void CSendDialog::OnButtonAddressBook(wxCommandEvent& event)
1316 {
1317     // Open address book
1318     CAddressBookDialog dialog(this, m_textCtrlAddress->GetValue(), true);
1319     if (dialog.ShowModal())
1320         m_textCtrlAddress->SetValue(dialog.GetAddress());
1321 }
1322
1323 void CSendDialog::OnButtonPaste(wxCommandEvent& event)
1324 {
1325     // Copy clipboard to address box
1326     if (wxTheClipboard->Open())
1327     {
1328         wxTextDataObject data;
1329         wxTheClipboard->GetData(data);
1330         m_textCtrlAddress->SetValue(data.GetText());
1331     }
1332     wxTheClipboard->Close();
1333 }
1334
1335 void CSendDialog::OnButtonSend(wxCommandEvent& event)

```

```

1338 {
1339     CWalletTx wtx;
1340     string strAddress = (string)m_textCtrlAddress->GetValue();
1341
1342     // Parse amount
1343     int64 nValue = 0;
1344     if (!ParseMoney(m_textCtrlAmount->GetValue(), nValue) || nValue <= 0)
1345     {
1346         wxMessageBox("Error in amount ");
1347         return;
1348     }
1349     if (nValue > GetBalance())
1350     {
1351         wxMessageBox("Amount exceeds your balance ");
1352         return;
1353     }
1354     if (nValue + nTransactionFee > GetBalance())
1355     {
1356         wxMessageBox(string("Total exceeds your balance when the ") + FormatMoney(nTransactionFee) + "
transaction fee is included ");
1357         return;
1358     }
1359
1360     // Parse bitcoin address
1361     uint160 hash160;
1362     bool fBitcoinAddress = AddressToHash160(strAddress, hash160);
1363
1364     if (fBitcoinAddress)
1365     {
1366         // Send to bitcoin address
1367         CScript scriptPubKey;
1368         scriptPubKey << OP_DUP << OP_HASH160 << hash160 << OP_EQUALVERIFY << OP_CHECKSIG;
1369
1370         if (!SendMoney(scriptPubKey, nValue, wtx))
1371             return;
1372
1373         wxMessageBox("Payment sent ", "Sending...");
1374     }
1375     else
1376     {
1377         // Parse IP address
1378         CAddress addr(strAddress.c_str());
1379         if (addr.ip == 0)
1380         {
1381             wxMessageBox("Invalid address ");
1382             return;
1383         }
1384
1385         // Message
1386         wtx.mapValue["to"] = strAddress;
1387         wtx.mapValue["from"] = m_textCtrlFrom->GetValue();
1388         wtx.mapValue["message"] = m_textCtrlMessage->GetValue();
1389
1390         // Send to IP address
1391         CSendingDialog* pdialog = new CSendingDialog(this, addr, nValue, wtx);
1392         if (!pdialog->ShowModal())
1393             return;
1394     }
1395
1396     if (!mapAddressBook.count(strAddress))
1397         SetAddressBookName(strAddress, "");
1398
1399     EndModal(true);
1400 }
1401
1402 void CSendDialog::OnButtonCancel(wxCommandEvent& event)
1403 {
1404     // Cancel
1405     EndModal(false);
1406 }
1407
1408

```

```

1409
1410
1411
1412
1413 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
1414 //
1415 // CSendingDialog
1416 //
1417
1418 CSendingDialog::CSendingDialog(wxWindow* parent, const CAddress& addrIn, int64 nPriceIn, const
    CWalletTx& wtxIn) : CSendingDialogBase(NULL) // we have to give null so parent can't destroy us
1419 {
1420     addr = addrIn;
1421     nPrice = nPriceIn;
1422     wtx = wtxIn;
1423     start = wxDateTime::UNow();
1424     strStatus = "";
1425     fCanCancel = true;
1426     fAbort = false;
1427     fSuccess = false;
1428     fUIDone = false;
1429     fWorkDone = false;
1430
1431     SetTitle(strprintf("Sending %s to %s...", FormatMoney(nPrice).c_str(), wtx.mapValue["to"].c_str()));
1432     m_textCtrlStatus->SetValue("");
1433
1434     _beginthread(SendingDialogStartTransfer, 0, this);
1435 }
1436
1437 CSendingDialog::~CSendingDialog()
1438 {
1439     printf("~CSendingDialog()\n");
1440 }
1441
1442 void CSendingDialog::Close()
1443 {
1444     // Last one out turn out the lights.
1445     // fWorkDone signals that work side is done and UI thread should call destroy.
1446     // fUIDone signals that UI window has closed and work thread should call destroy.
1447     // This allows the window to disappear and end modality when cancelled
1448     // without making the user wait for ConnectNode to return. The dialog object
1449     // hangs around in the background until the work thread exits.
1450     if (IsModal())
1451         EndModal(fSuccess);
1452     else
1453         Show(false);
1454     if (fWorkDone)
1455         Destroy();
1456     else
1457         fUIDone = true;
1458 }
1459
1460 void CSendingDialog::OnClose(wxCloseEvent& event)
1461 {
1462     if (!event.CanVeto() || fWorkDone || fAbort || !fCanCancel)
1463     {
1464         Close();
1465     }
1466     else
1467     {
1468         event.Veto();
1469         wxCommandEvent cmdevent;
1470         OnButtonCancel(cmdevent);
1471     }
1472 }
1473
1474 void CSendingDialog::OnButtonOK(wxCommandEvent& event)
1475 {
1476     if (fWorkDone)
1477         Close();
1478 }
1479

```



```

1480 void CSendingDialog::OnButtonCancel(wxCommandEvent& event)
1481 {
1482     if (fCanCancel)
1483         fAbort = true;
1484 }
1485
1486 void CSendingDialog::OnPaint(wxPaintEvent& event)
1487 {
1488     if (strStatus.size() > 130)
1489         m_textCtrlStatus->SetValue(string("\n") + strStatus);
1490     else
1491         m_textCtrlStatus->SetValue(string("\n\n") + strStatus);
1492     m_staticTextSending->SetFocus();
1493     if (!fCanCancel)
1494         m_buttonCancel->Enable(false);
1495     if (fWorkDone)
1496     {
1497         m_buttonOK->Enable(true);
1498         m_buttonOK->SetFocus();
1499         m_buttonCancel->Enable(false);
1500     }
1501     if (fAbort && fCanCancel && IsShown())
1502     {
1503         strStatus = "CANCELLED";
1504         m_buttonOK->Enable(true);
1505         m_buttonOK->SetFocus();
1506         m_buttonCancel->Enable(false);
1507         m_buttonCancel->SetLabel("Cancelled");
1508     }
1509     Close();
1510     wxMessageBox("Transfer cancelled ", "Sending...", wxOK, this);
1511     event.Skip();
1512
1513     /// debug test
1514     if (fRandSendTest && fWorkDone && fSuccess)
1515     {
1516         Close();
1517         Sleep(1000);
1518         RandSend();
1519     }
1520 }
1521
1522
1523 ///
1524 /// Everything from here on is not in the UI thread and must only communicate
1525 /// with the rest of the dialog through variables and calling repaint.
1526 ///
1527
1528 void CSendingDialog::Repaint()
1529 {
1530     Refresh();
1531     wxPaintEvent event;
1532     AddPendingEvent(event);
1533 }
1534
1535 bool CSendingDialog::Status()
1536 {
1537     if (fUIDone)
1538     {
1539         Destroy();
1540         return false;
1541     }
1542     if (fAbort && fCanCancel)
1543     {
1544         strStatus = "CANCELLED";
1545         Repaint();
1546         fWorkDone = true;
1547         return false;
1548     }
1549     return true;
1550 }
1551

```

```

1552 bool CSendingDialog::Status(const string& str)
1553 {
1554     if (!Status())
1555         return false;
1556     strStatus = str;
1557     Repaint();
1558     return true;
1559 }
1560
1561 bool CSendingDialog::Error(const string& str)
1562 {
1563     fCanCancel = false;
1564     fWorkDone = true;
1565     Status(string("Error: ") + str);
1566     return false;
1567 }
1568
1569 void CSendingDialog::StartTransfer(void* parg)
1570 {
1571     ((CSendingDialog*)parg)->StartTransfer();
1572 }
1573
1574 void CSendingDialog::StartTransfer()
1575 {
1576     // Make sure we have enough money
1577     if (nPrice + nTransactionFee > GetBalance())
1578     {
1579         Error("You don't have enough money");
1580         return;
1581     }
1582
1583     // We may have connected already for product details
1584     if (!Status("Connecting..."))
1585         return;
1586     CNode* pnode = ConnectNode(addr, 5 * 60);
1587     if (!pnode)
1588     {
1589         Error("Unable to connect");
1590         return;
1591     }
1592
1593     // Send order to seller, with response going to OnReply2 via event handler
1594     if (!Status("Requesting public key..."))
1595         return;
1596     pnode->PushRequest("checkorder", wtx, SendingDialogOnReply2, this);
1597 }
1598
1599 void CSendingDialog::OnReply2(void* parg, CDataStream& vRecv)
1600 {
1601     ((CSendingDialog*)parg)->OnReply2(vRecv);
1602 }
1603
1604 void CSendingDialog::OnReply2(CDataStream& vRecv)
1605 {
1606     if (!Status("Received public key..."))
1607         return;
1608
1609     CScript scriptPubKey;
1610     int nRet;
1611     try
1612     {
1613         vRecv >> nRet;
1614         if (nRet > 0)
1615         {
1616             string strMessage;
1617             vRecv >> strMessage;
1618             Error("Transfer was not accepted");
1619             //// todo: enlarge the window and enable a hidden white box to put seller's message
1620             return;
1621         }
1622         vRecv >> scriptPubKey;
1623     }

```

```

1624 catch (...)
1625 {
1626     //// what do we want to do about this?
1627     Error("Invalid response received");
1628     return;
1629 }
1630
1631 // Should already be connected
1632     CNode* pnode = ConnectNode(addr, 5 * 60);
1633     if (!pnode)
1634     {
1635         Error("Lost connection");
1636         return;
1637     }
1638
1639 // Pause to give the user a chance to cancel
1640     while (wxDateTime::UNow() < start + wxTimeSpan(0, 0, 0, 2 * 1000))
1641     {
1642         Sleep(200);
1643         if (!Status())
1644             return;
1645     }
1646
1647     CRITICAL_BLOCK(cs_main)
1648     {
1649         // Pay
1650         if (!Status("Creating transaction..."))
1651             return;
1652         if (nPrice + nTransactionFee > GetBalance())
1653         {
1654             Error("You don't have enough money");
1655             return;
1656         }
1657         int64 nFeeRequired;
1658         if (!CreateTransaction(scriptPubKey, nPrice, wtx, nFeeRequired))
1659         {
1660             if (nPrice + nFeeRequired > GetBalance())
1661                 Error(sprintf("This is an oversized transaction that requires a transaction fee of %s",
1662                     FormatMoney(nFeeRequired).c_str()));
1662             else
1663                 Error("Transaction creation failed");
1664             return;
1665         }
1666
1667         // Last chance to cancel
1668         Sleep(50);
1669         if (!Status())
1670             return;
1671         fCanCancel = false;
1672         if (fAbort)
1673         {
1674             fCanCancel = true;
1675             if (!Status())
1676                 return;
1677             fCanCancel = false;
1678         }
1679         if (!Status("Sending payment..."))
1680             return;
1681
1682         // Commit
1683         if (!CommitTransactionSpent(wtx))
1684         {
1685             Error("Error finalizing payment");
1686             return;
1687         }
1688
1689         // Send payment tx to seller, with response going to OnReply3 via event handler
1690         pnode->PushRequest("submitorder", wtx, SendingDialogOnReply3, this);
1691
1692         // Accept and broadcast transaction
1693         if (!wtx.AcceptTransaction())
    
```

```

1694 printf("ERROR: CSendingDialog : wtxNew.AcceptTransaction() %s failed\n",
      wtx.GetHash().ToString().c_str());
1695     wtx.RelayWalletTransaction();
1696
1697 Status("Waiting for confirmation...");
1698 MainFrameRepaint();
1699 }
1700 }
1701
1702 void CSendingDialog::OnReply3(void* parg, CDataStream& vRecv)
1703 {
1704     ((CSendingDialog*)parg)->OnReply3(vRecv);
1705 }
1706
1707 void CSendingDialog::OnReply3(CDataStream& vRecv)
1708 {
1709     int nRet;
1710     try
1711     {
1712         vRecv >> nRet;
1713         if (nRet > 0)
1714         {
1715             Error("The payment was sent, but the recipient was unable to verify it.\n"
1716                 "The transaction is recorded and will credit to the recipient if it is valid,\n"
1717                 "but without comment information.");
1718             return;
1719         }
1720     }
1721     catch (...)
1722     {
1723         /// what do we want to do about this?
1724         Error("Payment was sent, but an invalid response was received");
1725         return;
1726     }
1727
1728     fSuccess = true;
1729     fWorkDone = true;
1730     Status("Payment completed");
1731 }
1732
1733
1734
1735
1736
1737
1738 //////////////////////////////////////////////////
1739 //
1740 // CYourAddressDialog
1741 //
1742
1743 CYourAddressDialog::CYourAddressDialog(wxWindow* parent, const string& strInitSelected) :
1744     CYourAddressDialogBase(parent)
1745 {
1746     // Init column headers
1747     m_listCtrl->InsertColumn(0, "Name", wxLIST_FORMAT_LEFT, 200);
1748     m_listCtrl->InsertColumn(1, "Bitcoin Address", wxLIST_FORMAT_LEFT, 350);
1749     m_listCtrl->SetFocus();
1750
1751     // Fill listctrl with address book data
1752     CRITICAL_BLOCK(cs_mapKeys)
1753     {
1754         foreach(const PAIRTYPE(string, string)& item, mapAddressBook)
1755         {
1756             string strAddress = item.first;
1757             string strName = item.second;
1758             uint160 hash160;
1759             bool fMine = (AddressToHash160(strAddress, hash160) && mapPubKeys.count(hash160));
1760             if (fMine)
1761             {
1762                 int nIndex = InsertLine(m_listCtrl, strName, strAddress);
1763                 if (strAddress == strInitSelected)

```

```

1763             m_listCtrl->SetItemState(nIndex, wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED,
wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED);
1764 }
1765 }
1766 }
1767 }
1768
1769 wxString CYourAddressDialog::GetAddress()
1770 {
1771     int nIndex = GetSelection(m_listCtrl);
1772     if (nIndex == -1)
1773         return "";
1774     return GetItemText(m_listCtrl, nIndex, 1);
1775 }
1776
1777 void CYourAddressDialog::OnListEndLabelEdit(wxListEvent& event)
1778 {
1779     // Update address book with edited name
1780     if (event.IsEditCancelled())
1781         return;
1782     string strAddress = (string)GetItemText(m_listCtrl, event.GetIndex(), 1);
1783     SetAddressBookName(strAddress, string(event.GetText()));
1784     pframeMain->RefreshListCtrl();
1785 }
1786
1787 void CYourAddressDialog::OnListItemSelected(wxListEvent& event)
1788 {
1789 }
1790
1791 void CYourAddressDialog::OnListItemActivated(wxListEvent& event)
1792 {
1793     // Doubleclick returns selection
1794     EndModal(true);
1795 }
1796
1797 void CYourAddressDialog::OnButtonRename(wxCommandEvent& event)
1798 {
1799     // Ask new name
1800     int nIndex = GetSelection(m_listCtrl);
1801     if (nIndex == -1)
1802         return;
1803     string strName = (string)m_listCtrl->GetItemText(nIndex);
1804     string strAddress = (string)GetItemText(m_listCtrl, nIndex, 1);
1805     CGetTextFromUserDialog dialog(this, "Rename Bitcoin Address", "New Name", strName);
1806     if (!dialog.ShowModal())
1807         return;
1808     strName = dialog.GetValue();
1809
1810     // Change name
1811     SetAddressBookName(strAddress, strName);
1812     m_listCtrl->SetItemText(nIndex, strName);
1813     pframeMain->RefreshListCtrl();
1814 }
1815
1816 void CYourAddressDialog::OnButtonNew(wxCommandEvent& event)
1817 {
1818     // Ask name
1819     CGetTextFromUserDialog dialog(this, "New Bitcoin Address", "Name", "");
1820     if (!dialog.ShowModal())
1821         return;
1822     string strName = dialog.GetValue();
1823
1824     // Generate new key
1825     string strAddress = PubKeyToAddress(GenerateNewKey());
1826     SetAddressBookName(strAddress, strName);
1827
1828     // Add to list and select it
1829     int nIndex = InsertLine(m_listCtrl, strName, strAddress);
1830     SetSelection(m_listCtrl, nIndex);
1831     m_listCtrl->SetFocus();
1832 }
1833

```

```

1834 void CYourAddressDialog::OnButtonCopy(wxCommandEvent& event)
1835 {
1836     // Copy address box to clipboard
1837     if (wxTheClipboard->Open())
1838     {
1839         wxTheClipboard->SetData(new wxTextDataObject(GetAddress()));
1840         wxTheClipboard->Close();
1841     }
1842 }
1843
1844 void CYourAddressDialog::OnButtonOK(wxCommandEvent& event)
1845 {
1846     // OK
1847     EndModal(true);
1848 }
1849
1850 void CYourAddressDialog::OnButtonCancel(wxCommandEvent& event)
1851 {
1852     // Cancel
1853     EndModal(false);
1854 }
1855
1856 void CYourAddressDialog::OnClose(wxCloseEvent& event)
1857 {
1858     // Close
1859     EndModal(false);
1860 }
1861
1862
1863
1864
1865
1866
1867 //////////////////////////////////////
1868 //
1869 // CAddressBookDialog
1870 //
1871
1872 CAddressBookDialog::CAddressBookDialog(wxWindow* parent, const wxString& strInitSelected, bool
fSendingIn) : CAddressBookDialogBase(parent)
1873 {
1874     fSending = fSendingIn;
1875     if (!fSending)
1876         m_buttonCancel->Show(false);
1877
1878     // Init column headers
1879     m_listCtrl->InsertColumn(0, "Name", wxLIST_FORMAT_LEFT, 200);
1880     m_listCtrl->InsertColumn(1, "Address", wxLIST_FORMAT_LEFT, 350);
1881     m_listCtrl->SetFocus();
1882
1883     // Set Icon
1884     wxBitmap bmpAddressBook(wxT("addressbook16"), wxBITMAP_TYPE_RESOURCE);
1885     bmpAddressBook.SetMask(new wxMask(wxBitmap(wxT("addressbook16mask"), wxBITMAP_TYPE_RESOURCE)));
1886     wxIcon iconAddressBook;
1887     iconAddressBook.CopyFromBitmap(bmpAddressBook);
1888     SetIcon(iconAddressBook);
1889
1890     // Fill listctrl with address book data
1891     CRITICAL_BLOCK(cs_mapKeys)
1892     {
1893         foreach(const PAIRTYPE(string, string)& item, mapAddressBook)
1894         {
1895             string strAddress = item.first;
1896             string strName = item.second;
1897             uint160 hash160;
1898             bool fMine = (AddressToHash160(strAddress, hash160) && mapPubKeys.count(hash160));
1899             if (!fMine)
1900             {
1901                 int nIndex = InsertLine(m_listCtrl, strName, strAddress);
1902                 if (strAddress == strInitSelected)
1903                     m_listCtrl->SetItemState(nIndex, wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED,
wxLIST_STATE_SELECTED|wxLIST_STATE_FOCUSED);

```

```

1904 }
1905 }
1906 }
1907 }
1908
1909 wxString CAddressBookDialog::GetAddress()
1910 {
1911     int nIndex = GetSelection(m_listCtrl);
1912     if (nIndex == -1)
1913         return "";
1914     return GetItemText(m_listCtrl, nIndex, 1);
1915 }
1916
1917 void CAddressBookDialog::OnListEndLabelEdit(wxListEvent& event)
1918 {
1919     // Update address book with edited name
1920     if (event.IsEditCancelled())
1921         return;
1922     string strAddress = (string)GetItemText(m_listCtrl, event.GetIndex(), 1);
1923     SetAddressBookName(strAddress, string(event.GetText()));
1924     pframeMain->RefreshListCtrl();
1925 }
1926
1927 void CAddressBookDialog::OnListItemSelected(wxListEvent& event)
1928 {
1929 }
1930
1931 void CAddressBookDialog::OnListItemActivated(wxListEvent& event)
1932 {
1933     if (fSending)
1934     {
1935         // Doubleclick returns selection
1936         EndModal(GetAddress() != "" ? 2 : 0);
1937     }
1938     else
1939     {
1940         // Doubleclick edits item
1941         wxCommandEvent event;
1942         OnButtonEdit(event);
1943     }
1944 }
1945
1946 void CAddressBookDialog::OnButtonEdit(wxCommandEvent& event)
1947 {
1948     // Ask new name
1949     int nIndex = GetSelection(m_listCtrl);
1950     if (nIndex == -1)
1951         return;
1952     string strName = (string)m_listCtrl->GetItemText(nIndex);
1953     string strAddress = (string)GetItemText(m_listCtrl, nIndex, 1);
1954     string strAddressOrg = strAddress;
1955     CGetTextFromUserDialog dialog(this, "Edit Address", "Name", strName, "Address", strAddress);
1956     if (!dialog.ShowModal())
1957         return;
1958     strName = dialog.GetValue1();
1959     strAddress = dialog.GetValue2();
1960
1961     // Change name
1962     if (strAddress != strAddressOrg)
1963         CWalletDB().EraseName(strAddressOrg);
1964     SetAddressBookName(strAddress, strName);
1965     m_listCtrl->SetItem(nIndex, 1, strAddress);
1966     m_listCtrl->SetItemText(nIndex, strName);
1967     pframeMain->RefreshListCtrl();
1968 }
1969
1970 void CAddressBookDialog::OnButtonNew(wxCommandEvent& event)
1971 {
1972     // Ask name
1973     CGetTextFromUserDialog dialog(this, "New Address", "Name", "", "Address", "");
1974     if (!dialog.ShowModal())
1975         return;

```

```

1976     string strName = dialog.GetValue1();
1977     string strAddress = dialog.GetValue2();
1978
1979 // Add to list and select it
1980 SetAddressBookName(strAddress, strName);
1981 int nIndex = InsertLine(m_listCtrl, strName, strAddress);
1982 SetSelection(m_listCtrl, nIndex);
1983     m_listCtrl->SetFocus();
1984 }
1985
1986 void CAddressBookDialog::OnButtonDelete(wxCommandEvent& event)
1987 {
1988     for (int nIndex = m_listCtrl->GetItemCount()-1; nIndex >= 0; nIndex--)
1989     {
1990         if (m_listCtrl->GetItemState(nIndex, wxLIST_STATE_SELECTED))
1991         {
1992             string strAddress = (string)GetItemText(m_listCtrl, nIndex, 1);
1993             CWalletDB().EraseName(strAddress);
1994             m_listCtrl->DeleteItem(nIndex);
1995         }
1996     }
1997     pframeMain->RefreshListCtrl();
1998 }
1999
2000 void CAddressBookDialog::OnButtonCopy(wxCommandEvent& event)
2001 {
2002 // Copy address box to clipboard
2003 if (wxTheClipboard->Open())
2004 {
2005     wxTheClipboard->SetData(new wxTextDataObject(GetAddress()));
2006     wxTheClipboard->Close();
2007 }
2008 }
2009
2010 void CAddressBookDialog::OnButtonOK(wxCommandEvent& event)
2011 {
2012 // OK
2013 EndModal(GetAddress() != "" ? 1 : 0);
2014 }
2015
2016 void CAddressBookDialog::OnButtonCancel(wxCommandEvent& event)
2017 {
2018 // Cancel
2019 EndModal(0);
2020 }
2021
2022 void CAddressBookDialog::OnClose(wxCloseEvent& event)
2023 {
2024 // Close
2025 EndModal(0);
2026 }
2027
2028
2029
2030
2031
2032
2033 ///////////////////////////////////////////////////////////////////
2034 //
2035 // CProductsDialog
2036 //
2037
2038 bool CompareIntStringPairBestFirst(const pair<int, string>& item1, const pair<int, string>& item2)
2039 {
2040     return (item1.first > item2.first);
2041 }
2042
2043 CProductsDialog::CProductsDialog(wxWindow* parent) : CProductsDialogBase(parent)
2044 {
2045 // Init column headers
2046     m_listCtrl->InsertColumn(0, "Title", wxLIST_FORMAT_LEFT, 200);
2047     m_listCtrl->InsertColumn(1, "Price", wxLIST_FORMAT_LEFT, 80);

```



```

2048     m_listCtrl->InsertColumn(2, "Seller", wxLIST_FORMAT_LEFT, 80);
2049     m_listCtrl->InsertColumn(3, "Stars", wxLIST_FORMAT_LEFT, 50);
2050     m_listCtrl->InsertColumn(4, "Power", wxLIST_FORMAT_LEFT, 50);
2051
2052     // Tally top categories
2053     map<string, int> mapTopCategories;
2054     CRITICAL_BLOCK(cs_mapProducts)
2055     for (map<uint256, CProduct>::iterator mi = mapProducts.begin(); mi != mapProducts.end(); ++mi)
2056         mapTopCategories[(mi).second.mapValue["category"]]++;
2057
2058     // Sort top categories
2059     vector<pair<int, string> > vTopCategories;
2060     for (map<string, int>::iterator mi = mapTopCategories.begin(); mi != mapTopCategories.end(); ++mi)
2061         vTopCategories.push_back(make_pair((mi).second, (mi).first));
2062     sort(vTopCategories.begin(), vTopCategories.end(), CompareIntStringPairBestFirst);
2063
2064     // Fill categories combo box
2065     int nLimit = 250;
2066     for (vector<pair<int, string> >::iterator it = vTopCategories.begin(); it != vTopCategories.end() &&
nLimit-- > 0; ++it)
2067         m_comboBoxCategory->Append((it).second);
2068
2069     // Fill window with initial search
2070     //wxCommandEvent event;
2071     //OnButtonSearch(event);
2072 }
2073
2074 void CProductsDialog::OnCombobox(wxCommandEvent& event)
2075 {
2076     OnButtonSearch(event);
2077 }
2078
2079 bool CompareProductsBestFirst(const CProduct* p1, const CProduct* p2)
2080 {
2081     return (p1->nAtoms > p2->nAtoms);
2082 }
2083
2084 void CProductsDialog::OnButtonSearch(wxCommandEvent& event)
2085 {
2086     string strCategory = (string)m_comboBoxCategory->GetValue();
2087     string strSearch = (string)m_textCtrlSearch->GetValue();
2088
2089     // Search products
2090     vector<CProduct*> vProductsFound;
2091     CRITICAL_BLOCK(cs_mapProducts)
2092     {
2093         for (map<uint256, CProduct>::iterator mi = mapProducts.begin(); mi != mapProducts.end(); ++mi)
2094         {
2095             CProduct& product = (mi).second;
2096             if (product.mapValue["category"].find(strCategory) != -1)
2097             {
2098                 if (product.mapValue["title"].find(strSearch) != -1 ||
2099                     product.mapValue["description"].find(strSearch) != -1 ||
2100                     product.mapValue["seller"].find(strSearch) != -1)
2101                 {
2102                     vProductsFound.push_back(&product);
2103                 }
2104             }
2105         }
2106     }
2107
2108     // Sort
2109     sort(vProductsFound.begin(), vProductsFound.end(), CompareProductsBestFirst);
2110
2111     // Display
2112     foreach(CProduct* pproduct, vProductsFound)
2113     {
2114         InsertLine(m_listCtrl,
2115             pproduct->mapValue["title"],
2116             pproduct->mapValue["price"],
2117             pproduct->mapValue["seller"],
2118             pproduct->mapValue["stars"],

```

```

2119 itostr(pproduct->nAtoms));
2120 }
2121 }
2122
2123 void CProductsDialog::OnListItemActivated(wxListEvent& event)
2124 {
2125     // DoubleClick opens product
2126     CViewProductDialog* pdialog = new CViewProductDialog(this, m_vProduct[event.GetIndex()]);
2127     pdialog->Show();
2128 }
2129
2130
2131
2132
2133
2134
2135
2136 //////////////////////////////////////
2137 //
2138 // CEditProductDialog
2139 //
2140
2141 CEditProductDialog::CEditProductDialog(wxWindow* parent) : CEditProductDialogBase(parent)
2142 {
2143     m_textCtrlLabel[0] = m_textCtrlLabel0;
2144     m_textCtrlLabel[1] = m_textCtrlLabel1;
2145     m_textCtrlLabel[2] = m_textCtrlLabel2;
2146     m_textCtrlLabel[3] = m_textCtrlLabel3;
2147     m_textCtrlLabel[4] = m_textCtrlLabel4;
2148     m_textCtrlLabel[5] = m_textCtrlLabel5;
2149     m_textCtrlLabel[6] = m_textCtrlLabel6;
2150     m_textCtrlLabel[7] = m_textCtrlLabel7;
2151     m_textCtrlLabel[8] = m_textCtrlLabel8;
2152     m_textCtrlLabel[9] = m_textCtrlLabel9;
2153     m_textCtrlLabel[10] = m_textCtrlLabel10;
2154     m_textCtrlLabel[11] = m_textCtrlLabel11;
2155     m_textCtrlLabel[12] = m_textCtrlLabel12;
2156     m_textCtrlLabel[13] = m_textCtrlLabel13;
2157     m_textCtrlLabel[14] = m_textCtrlLabel14;
2158     m_textCtrlLabel[15] = m_textCtrlLabel15;
2159     m_textCtrlLabel[16] = m_textCtrlLabel16;
2160     m_textCtrlLabel[17] = m_textCtrlLabel17;
2161     m_textCtrlLabel[18] = m_textCtrlLabel18;
2162     m_textCtrlLabel[19] = m_textCtrlLabel19;
2163
2164     m_textCtrlField[0] = m_textCtrlField0;
2165     m_textCtrlField[1] = m_textCtrlField1;
2166     m_textCtrlField[2] = m_textCtrlField2;
2167     m_textCtrlField[3] = m_textCtrlField3;
2168     m_textCtrlField[4] = m_textCtrlField4;
2169     m_textCtrlField[5] = m_textCtrlField5;
2170     m_textCtrlField[6] = m_textCtrlField6;
2171     m_textCtrlField[7] = m_textCtrlField7;
2172     m_textCtrlField[8] = m_textCtrlField8;
2173     m_textCtrlField[9] = m_textCtrlField9;
2174     m_textCtrlField[10] = m_textCtrlField10;
2175     m_textCtrlField[11] = m_textCtrlField11;
2176     m_textCtrlField[12] = m_textCtrlField12;
2177     m_textCtrlField[13] = m_textCtrlField13;
2178     m_textCtrlField[14] = m_textCtrlField14;
2179     m_textCtrlField[15] = m_textCtrlField15;
2180     m_textCtrlField[16] = m_textCtrlField16;
2181     m_textCtrlField[17] = m_textCtrlField17;
2182     m_textCtrlField[18] = m_textCtrlField18;
2183     m_textCtrlField[19] = m_textCtrlField19;
2184
2185     m_buttonDel[0] = m_buttonDel0;
2186     m_buttonDel[1] = m_buttonDel1;
2187     m_buttonDel[2] = m_buttonDel2;
2188     m_buttonDel[3] = m_buttonDel3;
2189     m_buttonDel[4] = m_buttonDel4;
2190     m_buttonDel[5] = m_buttonDel5;

```

```

2191     m_buttonDel[6] = m_buttonDel6;
2192     m_buttonDel[7] = m_buttonDel7;
2193     m_buttonDel[8] = m_buttonDel8;
2194     m_buttonDel[9] = m_buttonDel9;
2195     m_buttonDel[10] = m_buttonDel10;
2196     m_buttonDel[11] = m_buttonDel11;
2197     m_buttonDel[12] = m_buttonDel12;
2198     m_buttonDel[13] = m_buttonDel13;
2199     m_buttonDel[14] = m_buttonDel14;
2200     m_buttonDel[15] = m_buttonDel15;
2201     m_buttonDel[16] = m_buttonDel16;
2202     m_buttonDel[17] = m_buttonDel17;
2203     m_buttonDel[18] = m_buttonDel18;
2204     m_buttonDel[19] = m_buttonDel19;
2205
2206     for (int i = 1; i < FIELDS_MAX; i++)
2207         ShowLine(i, false);
2208
2209     LayoutAll();
2210 }
2211
2212 void CEditProductDialog::LayoutAll()
2213 {
2214     m_scrolledWindow->Layout();
2215     m_scrolledWindow->GetSizer()->Fit(m_scrolledWindow);
2216     this->Layout();
2217 }
2218
2219 void CEditProductDialog::ShowLine(int i, bool fShow)
2220 {
2221     m_textCtrlLabel[i]->Show(fShow);
2222     m_textCtrlField[i]->Show(fShow);
2223     m_buttonDel[i]->Show(fShow);
2224 }
2225
2226 void CEditProductDialog::OnButtonDel0(wxCommandEvent& event) { OnButtonDel(event, 0); }
2227 void CEditProductDialog::OnButtonDel1(wxCommandEvent& event) { OnButtonDel(event, 1); }
2228 void CEditProductDialog::OnButtonDel2(wxCommandEvent& event) { OnButtonDel(event, 2); }
2229 void CEditProductDialog::OnButtonDel3(wxCommandEvent& event) { OnButtonDel(event, 3); }
2230 void CEditProductDialog::OnButtonDel4(wxCommandEvent& event) { OnButtonDel(event, 4); }
2231 void CEditProductDialog::OnButtonDel5(wxCommandEvent& event) { OnButtonDel(event, 5); }
2232 void CEditProductDialog::OnButtonDel6(wxCommandEvent& event) { OnButtonDel(event, 6); }
2233 void CEditProductDialog::OnButtonDel7(wxCommandEvent& event) { OnButtonDel(event, 7); }
2234 void CEditProductDialog::OnButtonDel8(wxCommandEvent& event) { OnButtonDel(event, 8); }
2235 void CEditProductDialog::OnButtonDel9(wxCommandEvent& event) { OnButtonDel(event, 9); }
2236 void CEditProductDialog::OnButtonDel10(wxCommandEvent& event) { OnButtonDel(event, 10); }
2237 void CEditProductDialog::OnButtonDel11(wxCommandEvent& event) { OnButtonDel(event, 11); }
2238 void CEditProductDialog::OnButtonDel12(wxCommandEvent& event) { OnButtonDel(event, 12); }
2239 void CEditProductDialog::OnButtonDel13(wxCommandEvent& event) { OnButtonDel(event, 13); }
2240 void CEditProductDialog::OnButtonDel14(wxCommandEvent& event) { OnButtonDel(event, 14); }
2241 void CEditProductDialog::OnButtonDel15(wxCommandEvent& event) { OnButtonDel(event, 15); }
2242 void CEditProductDialog::OnButtonDel16(wxCommandEvent& event) { OnButtonDel(event, 16); }
2243 void CEditProductDialog::OnButtonDel17(wxCommandEvent& event) { OnButtonDel(event, 17); }
2244 void CEditProductDialog::OnButtonDel18(wxCommandEvent& event) { OnButtonDel(event, 18); }
2245 void CEditProductDialog::OnButtonDel19(wxCommandEvent& event) { OnButtonDel(event, 19); }
2246
2247 void CEditProductDialog::OnButtonDel(wxCommandEvent& event, int n)
2248 {
2249     Freeze();
2250     int x, y;
2251     m_scrolledWindow->GetViewStart(&x, &y);
2252     int i;
2253     for (i = n; i < FIELDS_MAX-1; i++)
2254     {
2255         m_textCtrlLabel[i]->SetValue(m_textCtrlLabel[i+1]->GetValue());
2256         m_textCtrlField[i]->SetValue(m_textCtrlField[i+1]->GetValue());
2257         if (!m_buttonDel[i+1]->IsShown())
2258             break;
2259     }
2260     m_textCtrlLabel[i]->SetValue("");
2261     m_textCtrlField[i]->SetValue("");
2262     ShowLine(i, false);

```

```

2263     m_buttonAddField->Enable(true);
2264     LayoutAll();
2265     m_scrolledWindow->Scroll(0, y);
2266     Thaw();
2267 }
2268
2269 void CEditProductDialog::OnButtonAddField(wxCommandEvent& event)
2270 {
2271     for (int i = 0; i < FIELDS_MAX; i++)
2272     {
2273         if (!m_buttonDel[i]->IsShown())
2274         {
2275             Freeze();
2276             ShowLine(i, true);
2277             if (i == FIELDS_MAX-1)
2278                 m_buttonAddField->Enable(false);
2279             LayoutAll();
2280             m_scrolledWindow->Scroll(0, 99999);
2281             Thaw();
2282             break;
2283         }
2284     }
2285 }
2286
2287 void CEditProductDialog::OnButtonSend(wxCommandEvent& event)
2288 {
2289     CProduct product;
2290     GetProduct(product);
2291
2292     // Sign the detailed product
2293     product.vchPubKeyFrom = keyUser.GetPubKey();
2294     if (!keyUser.Sign(product.GetSigHash(), product.vchSig))
2295     {
2296         wxMessageBox("Error digitally signing the product ");
2297         return;
2298     }
2299
2300     // Save detailed product
2301     AddToMyProducts(product);
2302
2303     // Strip down to summary product
2304     product.mapDetails.clear();
2305     product.vOrderForm.clear();
2306
2307     // Sign the summary product
2308     if (!keyUser.Sign(product.GetSigHash(), product.vchSig))
2309     {
2310         wxMessageBox("Error digitally signing the product ");
2311         return;
2312     }
2313
2314     // Verify
2315     if (!product.CheckProduct())
2316     {
2317         wxMessageBox("Errors found in product ");
2318         return;
2319     }
2320
2321     // Broadcast
2322     AdvertStartPublish(pnodeLocalHost, MSG_PRODUCT, 0, product);
2323
2324     Destroy();
2325 }
2326
2327 void CEditProductDialog::OnButtonPreview(wxCommandEvent& event)
2328 {
2329     CProduct product;
2330     GetProduct(product);
2331     CViewProductDialog* pdialog = new CViewProductDialog(this, product);
2332     pdialog->Show();
2333 }
2334

```

```

2335 void CEditProductDialog::OnButtonCancel(wxCommandEvent& event)
2336 {
2337     Destroy();
2338 }
2339
2340 void CEditProductDialog::SetProduct(const CProduct& productIn)
2341 {
2342     CProduct product = productIn;
2343
2344     m_comboBoxCategory->SetValue(product.mapValue["category"]);
2345     m_textCtrlTitle->SetValue(product.mapValue["title"]);
2346     m_textCtrlPrice->SetValue(product.mapValue["price"]);
2347     m_textCtrlDescription->SetValue(product.mapValue["description"]);
2348     m_textCtrlInstructions->SetValue(product.mapValue["instructions"]);
2349
2350     for (int i = 0; i < FIELDS_MAX; i++)
2351     {
2352         bool fUsed = i < product.vOrderForm.size();
2353         m_buttonDel[i]->Show(fUsed);
2354         m_textCtrlLabel[i]->Show(fUsed);
2355         m_textCtrlField[i]->Show(fUsed);
2356         if (!fUsed)
2357             continue;
2358
2359         m_textCtrlLabel[i]->SetValue(product.vOrderForm[i].first);
2360         string strControl = product.vOrderForm[i].second;
2361         if (strControl.substr(0, 5) == "text=")
2362             m_textCtrlField[i]->SetValue("");
2363         else if (strControl.substr(0, 7) == "choice=")
2364             m_textCtrlField[i]->SetValue(strControl.substr(7));
2365         else
2366             m_textCtrlField[i]->SetValue(strControl);
2367     }
2368 }
2369
2370 void CEditProductDialog::GetProduct(CProduct& product)
2371 {
2372     // map<string, string> mapValue;
2373     // vector<pair<string, string> > vOrderForm;
2374
2375     product.mapValue["category"] = m_comboBoxCategory->GetValue().Trim();
2376     product.mapValue["title"] = m_textCtrlTitle->GetValue().Trim();
2377     product.mapValue["price"] = m_textCtrlPrice->GetValue().Trim();
2378     product.mapValue["description"] = m_textCtrlDescription->GetValue().Trim();
2379     product.mapValue["instructions"] = m_textCtrlInstructions->GetValue().Trim();
2380
2381     for (int i = 0; i < FIELDS_MAX; i++)
2382     {
2383         if (m_buttonDel[i]->IsShown())
2384         {
2385             string strLabel = (string)m_textCtrlLabel[i]->GetValue().Trim();
2386             string strControl = (string)m_textCtrlField[i]->GetValue();
2387             if (strControl.empty())
2388                 strControl = "text=";
2389             else
2390                 strControl = "choice=" + strControl;
2391             product.vOrderForm.push_back(make_pair(strLabel, strControl));
2392         }
2393     }
2394 }
2395
2396
2397
2398
2399
2400
2401
2402 ///////////////////////////////////////////////////
2403 //
2404 // CViewProductDialog
2405 //
2406

```

```

2407 CViewProductDialog::CViewProductDialog(wxWindow* parent, const CProduct& productIn) :
    CViewProductDialogBase(parent)
2408 {
2409     Connect(wxEVT_REPLY1, wxCommandEventHandler(CViewProductDialog::OnReply1), NULL, this);
2410     AddCallbackAvailable(GetEventHandler());
2411
2412     // Fill display with product summary while waiting for details
2413     product = productIn;
2414     UpdateProductDisplay(false);
2415
2416     m_buttonBack->Enable(false);
2417     m_buttonNext->Enable(!product.v0OrderForm.empty());
2418     m_htmlWinReviews->Show(true);
2419     m_scrolledWindow->Show(false);
2420     this->Layout();
2421
2422     // Request details from seller
2423     _beginthread(ThreadRequestProductDetails, 0, new pair<CProduct, wxEvtHandler*>(product,
        GetEventHandler()));
2424 }
2425
2426 CViewProductDialog::~CViewProductDialog()
2427 {
2428     RemoveCallbackAvailable(GetEventHandler());
2429 }
2430
2431 void ThreadRequestProductDetails(void* parg)
2432 {
2433     // Extract parameters
2434     pair<CProduct, wxEvtHandler*> pitem = (pair<CProduct, wxEvtHandler*>*)parg;
2435     CProduct product = pitem->first;
2436     wxEvtHandler* pevthandler = pitem->second;
2437     delete pitem;
2438
2439     // Connect to seller
2440     CNode* pnode = ConnectNode(product.addr, 5 * 60);
2441     if (!pnode)
2442     {
2443         CDataStream ssEmpty;
2444         AddPendingReplyEvent1(pevthandler, ssEmpty);
2445         return;
2446     }
2447
2448     // Request detailed product, with response going to OnReply1 via dialog's event handler
2449     pnode->PushRequest("getdetails", product.GetHash(), AddPendingReplyEvent1, (void*)pevthandler);
2450 }
2451
2452 void CViewProductDialog::OnReply1(wxCommandEvent& event)
2453 {
2454     CDataStream ss = GetStreamFromEvent(event);
2455     if (ss.empty())
2456     {
2457         product.mapValue["description"] = "-- CAN'T CONNECT TO SELLER --\n";
2458         UpdateProductDisplay(true);
2459         return;
2460     }
2461
2462     int nRet;
2463     CProduct product2;
2464     try
2465     {
2466         ss >> nRet;
2467         if (nRet > 0)
2468             throw false;
2469         ss >> product2;
2470         if (product2.GetHash() != product.GetHash())
2471             throw false;
2472         if (!product2.CheckSignature())
2473             throw false;
2474     }
2475     catch (...)
2476     {

```

```

2477 product.mapValue["description"] = "--- INVALID RESPONSE ---\n";
2478 UpdateProductDisplay(true);
2479 return;
2480 }
2481
2482     product = product2;
2483     UpdateProductDisplay(true);
2484 }
2485
2486 bool CompareReviewsBestFirst(const CReview* p1, const CReview* p2)
2487 {
2488     return (p1->nAtoms > p2->nAtoms);
2489 }
2490
2491 void CViewProductDialog::UpdateProductDisplay(bool fDetails)
2492 {
2493     // Product and reviews
2494     string strHTML;
2495     strHTML.reserve(4000);
2496     strHTML += "<html>\n"
2497 "<head>\n"
2498 "<meta http-equiv=\"content-type\" content=\"text/html; charset=UTF-8\">\n"
2499 "</head>\n"
2500 "<body>\n";
2501     strHTML += "<b>Category:</b>" + HtmlEscape(product.mapValue["category"]) + "<br>\n";
2502     strHTML += "<b>Title:</b>" + HtmlEscape(product.mapValue["title"]) + "<br>\n";
2503     strHTML += "<b>Price:</b>" + HtmlEscape(product.mapValue["price"]) + "<br>\n";
2504
2505     if (!fDetails)
2506         strHTML += "<b>Loading details...</b><br>\n<br>\n";
2507     else
2508         strHTML += HtmlEscape(product.mapValue["description"], true) + "<br>\n<br>\n";
2509
2510     strHTML += "<b>Reviews:</b><br>\n<br>\n";
2511
2512     if (!product.vchPubKeyFrom.empty())
2513     {
2514         CReviewDB reviewdb("r");
2515
2516         // Get reviews
2517         vector<CReview> vReviews;
2518         reviewdb.ReadReviews(product.GetUserHash(), vReviews);
2519
2520         // Get reviewer's number of atoms
2521         vector<CReview*> vSortedReviews;
2522         vSortedReviews.reserve(vReviews.size());
2523         for (vector<CReview>::reverse_iterator it = vReviews.rbegin(); it != vReviews.rend(); ++it)
2524         {
2525             CReview& review = *it;
2526             CUser user;
2527             reviewdb.ReadUser(review.GetUserHash(), user);
2528             review.nAtoms = user.GetAtomCount();
2529             vSortedReviews.push_back(&review);
2530         }
2531
2532         reviewdb.Close();
2533
2534         // Sort
2535         stable_sort(vSortedReviews.begin(), vSortedReviews.end(), CompareReviewsBestFirst);
2536
2537         // Format reviews
2538         foreach(CReview* preview, vSortedReviews)
2539         {
2540             CReview& review = *preview;
2541             int nStars = atoi(review.mapValue["stars"].c_str());
2542             if (nStars < 1 || nStars > 5)
2543                 continue;
2544
2545             strHTML += "<b>" + itostr(nStars) + (nStars == 1 ? " star" : " stars") + "</b>\n";
2546             strHTML += "   &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& ";
2547             strHTML += DateStr(atoi64(review.mapValue["date"])) + "<br>\n";
2548             strHTML += HtmlEscape(review.mapValue["review"], true);

```

```

2549         strHTML += "<br>\n<br>\n";
2550     }
2551 }
2552
2553     strHTML += "</body>\n</html>\n";
2554
2555 // Shrink capacity to fit
2556 string(strHTML.begin(), strHTML.end()).swap(strHTML);
2557
2558     m_htmlWinReviews->SetPage(strHTML);
2559
2560 // need to find some other indicator to use so can allow empty order form
2561 if (product.vOrderForm.empty())
2562     return;
2563
2564 // Order form
2565     m_staticTextInstructions->SetLabel(product.mapValue["instructions"]);
2566     for (int i = 0; i < FIELDS_MAX; i++)
2567     {
2568         m_staticTextLabel[i] = NULL;
2569         m_textCtrlField[i] = NULL;
2570         m_choiceField[i] = NULL;
2571     }
2572
2573 // Construct flexgridSizer
2574     wxBoxSizer* bSizer21 = (wxBoxSizer*)m_scrolledWindow->GetSizer();
2575     wxFlexGridSizer* fgSizer;
2576     fgSizer = new wxFlexGridSizer(0, 2, 0, 0);
2577     fgSizer->AddGrowCol(1);
2578     fgSizer->SetFlexibleDirection(wxBOTH);
2579     fgSizer->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
2580
2581 // Construct order form fields
2582     wxWindow* windowLast = NULL;
2583     for (int i = 0; i < product.vOrderForm.size(); i++)
2584     {
2585         string strLabel = product.vOrderForm[i].first;
2586         string strControl = product.vOrderForm[i].second;
2587
2588         if (strLabel.size() < 20)
2589             strLabel.insert(strLabel.begin(), 20 - strLabel.size(), ' ');
2590
2591         m_staticTextLabel[i] = new wxStaticText(m_scrolledWindow, wxID_ANY, strLabel, wxDefaultPosition,
wxDefaultSize, wxALIGN_RIGHT);
2592         m_staticTextLabel[i]->Wrap(-1);
2593         fgSizer->Add(m_staticTextLabel[i], 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxALL, 5);
2594
2595         if (strControl.substr(0, 5) == "text=")
2596         {
2597             m_textCtrlField[i] = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString,
wxDefaultPosition, wxDefaultSize, 0);
2598             fgSizer->Add(m_textCtrlField[i], 1, wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND, 5);
2599             windowLast = m_textCtrlField[i];
2600         }
2601         else if (strControl.substr(0, 7) == "choice=")
2602         {
2603             vector<string> vChoices;
2604             ParseString(strControl.substr(7), ',', vChoices);
2605
2606             wxArrayString arraystring;
2607             foreach(const string& str, vChoices)
2608                 arraystring.Add(str);
2609
2610             m_choiceField[i] = new wxChoice(m_scrolledWindow, wxID_ANY, wxDefaultPosition,
wxDefaultSize, arraystring, 0);
2611             fgSizer->Add(m_choiceField[i], 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
2612             windowLast = m_choiceField[i];
2613         }
2614         else
2615         {
2616             m_textCtrlField[i] = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString,
wxDefaultPosition, wxDefaultSize, 0);

```



```

2617         fgSizer->Add(m_textCtrlField[i], 1, wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND, 5);
2618         m_staticTextLabel[i]->Show(false);
2619         m_textCtrlField[i]->Show(false);
2620     }
2621 }
2622
2623 // Insert after instructions and before submit/cancel buttons
2624 bSizer21->Insert(2, fgSizer, 0, wxEXPAND|wxRIGHT|wxLEFT, 5);
2625 m_scrolledWindow->Layout();
2626 bSizer21->Fit(m_scrolledWindow);
2627 this->Layout();
2628
2629 // Fixup the tab order
2630 m_buttonSubmitForm->MoveAfterInTabOrder(windowLast);
2631 m_buttonCancelForm->MoveAfterInTabOrder(m_buttonSubmitForm);
2632 //m_buttonBack->MoveAfterInTabOrder(m_buttonCancelForm);
2633 //m_buttonNext->MoveAfterInTabOrder(m_buttonBack);
2634 //m_buttonCancel->MoveAfterInTabOrder(m_buttonNext);
2635 this->Layout();
2636 }
2637
2638 void CViewProductDialog::GetOrder(CWalletTx& wtx)
2639 {
2640     wtx.SetNull();
2641     for (int i = 0; i < product.vOrderForm.size(); i++)
2642     {
2643         string strValue;
2644         if (m_textCtrlField[i])
2645             strValue = m_textCtrlField[i]->GetValue().Trim();
2646         else
2647             strValue = m_choiceField[i]->GetStringSelection();
2648         wtx.vOrderForm.push_back(make_pair(m_staticTextLabel[i]->GetLabel(), strValue));
2649     }
2650 }
2651
2652 void CViewProductDialog::OnButtonSubmitForm(wxCommandEvent& event)
2653 {
2654     m_buttonSubmitForm->Enable(false);
2655     m_buttonCancelForm->Enable(false);
2656
2657     CWalletTx wtx;
2658     GetOrder(wtx);
2659
2660     CSendingDialog* pdialog = new CSendingDialog(this, product.addr, atoi64(product.mapValue["price"]),
2661     wtx);
2662     if (!pdialog->ShowModal())
2663     {
2664         m_buttonSubmitForm->Enable(true);
2665         m_buttonCancelForm->Enable(true);
2666         return;
2667     }
2668
2669 void CViewProductDialog::OnButtonCancelForm(wxCommandEvent& event)
2670 {
2671     Destroy();
2672 }
2673
2674 void CViewProductDialog::OnButtonBack(wxCommandEvent& event)
2675 {
2676     Freeze();
2677     m_htmlWinReviews->Show(true);
2678     m_scrolledWindow->Show(false);
2679     m_buttonBack->Enable(false);
2680     m_buttonNext->Enable(!product.vOrderForm.empty());
2681     this->Layout();
2682     Thaw();
2683 }
2684
2685 void CViewProductDialog::OnButtonNext(wxCommandEvent& event)
2686 {
2687     if (!product.vOrderForm.empty())

```

```

2688 {
2689     Freeze();
2690     m_htmlWinReviews->Show(false);
2691     m_scrolledWindow->Show(true);
2692     m_buttonBack->Enable(true);
2693     m_buttonNext->Enable(false);
2694     this->Layout();
2695     Thaw();
2696 }
2697 }
2698
2699 void CViewProductDialog::OnButtonCancel(wxCommandEvent& event)
2700 {
2701     Destroy();
2702 }
2703
2704
2705
2706
2707
2708
2709
2710 ///////////////////////////////////////////////////////////////////
2711 //
2712 // CViewOrderDialog
2713 //
2714
2715 CViewOrderDialog::CViewOrderDialog(wxWindow* parent, CWalletTx order, bool fReceived) :
CViewOrderDialogBase(parent)
2716 {
2717     int64 nPrice = (fReceived ? order.GetCredit() : order.GetDebit());
2718
2719     string strHTML;
2720     strHTML.reserve(4000);
2721     strHTML += "<html>\n"
2722 "<head>\n"
2723 "<meta http-equiv=\"content-type\" content=\"text/html; charset=UTF-8\">\n"
2724 "</head>\n"
2725 "<body>\n";
2726     strHTML += "<b>Time:</b> " + HtmlEscape(DateTimeStr(order.nTimeReceived)) + "<br>\n";
2727     strHTML += "<b>Price:</b> " + HtmlEscape(FormatMoney(nPrice)) + "<br>\n";
2728     strHTML += "<b>Status:</b> " + HtmlEscape(FormatTxStatus(order)) + "<br>\n";
2729
2730     strHTML += "<table>\n";
2731     for (int i = 0; i < order.vOrderForm.size(); i++)
2732     {
2733         strHTML += " <tr><td><b> " + HtmlEscape(order.vOrderForm[i].first) + ";</b></td>";
2734         strHTML += "<td> " + HtmlEscape(order.vOrderForm[i].second) + "</td></tr>\n";
2735     }
2736     strHTML += "</table>\n";
2737
2738     strHTML += "</body>\n</html>\n";
2739
2740 // Shrink capacity to fit
2741 // (strings are ref counted, so it may live on in SetPage)
2742 string(strHTML.begin(), strHTML.end()).swap(strHTML);
2743
2744     m_htmlWin->SetPage(strHTML);
2745 }
2746
2747 void CViewOrderDialog::OnButtonOK(wxCommandEvent& event)
2748 {
2749     Destroy();
2750 }
2751
2752
2753
2754
2755
2756
2757
2758 ///////////////////////////////////////////////////////////////////

```

```

2759 //
2760 // CEditReviewDialog
2761 //
2762
2763 CEditReviewDialog::CEditReviewDialog(wxWindow* parent) : CEditReviewDialogBase(parent)
2764 {
2765 }
2766
2767 void CEditReviewDialog::OnButtonSubmit(wxCommandEvent& event)
2768 {
2769     if (m_choiceStars->GetSelection() == -1)
2770     {
2771         wxMessageBox("Please select a rating ");
2772         return;
2773     }
2774
2775     CReview review;
2776     GetReview(review);
2777
2778     // Sign the review
2779     review.vchPubKeyFrom = keyUser.GetPubKey();
2780     if (!keyUser.Sign(review.GetSigHash(), review.vchSig))
2781     {
2782         wxMessageBox("Unable to digitally sign the review ");
2783         return;
2784     }
2785
2786     // Broadcast
2787     if (!review.AcceptReview())
2788     {
2789         wxMessageBox("Save failed ");
2790         return;
2791     }
2792     RelayMessage(CInv(MSG_REVIEW, review.GetHash()), review);
2793
2794     Destroy();
2795 }
2796
2797 void CEditReviewDialog::OnButtonCancel(wxCommandEvent& event)
2798 {
2799     Destroy();
2800 }
2801
2802 void CEditReviewDialog::GetReview(CReview& review)
2803 {
2804     review.mapValue["time"] = i64tostr(GetAdjustedTime());
2805     review.mapValue["stars"] = itostr(m_choiceStars->GetSelection()+1);
2806     review.mapValue["review"] = m_textCtrlReview->GetValue();
2807 }
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
2821 //
2822 // CMyApp
2823 //
2824
2825 // Define a new application
2826 class CMyApp: public wxApp
2827 {
2828 public:
2829     CMyApp(){};
2830     ~CMyApp(){};

```



```

2903 if (!isalnum(strMutexName[i]))
2904     strMutexName[i] = '.';
2905     wxSingleInstanceChecker* psingleinstancechecker = new wxSingleInstanceChecker(strMutexName);
2906 if (psingleinstancechecker->IsAnotherRunning())
2907 {
2908     printf("Existing instance found\n");
2909     unsigned int nStart = GetTime();
2910     loop
2911 {
2912     // Show the previous instance and exit
2913     HWND hwndPrev = FindWindow("wxWindowClassNR", "Bitcoin");
2914 if (hwndPrev)
2915 {
2916 if (IsIconic(hwndPrev))
2917 ShowWindow(hwndPrev, SW_RESTORE);
2918 SetForegroundWindow(hwndPrev);
2919 return false;
2920 }
2921
2922 if (GetTime() > nStart + 60)
2923 return false;
2924
2925 // Resume this instance if the other exits
2926 delete psingleinstancechecker;
2927 Sleep(1000);
2928     psingleinstancechecker = new wxSingleInstanceChecker(strMutexName);
2929 if (!psingleinstancechecker->IsAnotherRunning())
2930 break;
2931 }
2932 }
2933
2934 //
2935 // Parameters
2936 //
2937     wxImage::AddHandler(new wxPNGHandler);
2938     map<string, string> mapArgs = ParseParameters(argc, argv);
2939
2940 if (mapArgs.count("/datadir"))
2941     strSetDataDir = mapArgs["/datadir"];
2942
2943 if (mapArgs.count("/proxy"))
2944     addrProxy = CAddress(mapArgs["/proxy"].c_str());
2945
2946 if (mapArgs.count("/debug"))
2947     fDebug = true;
2948
2949 if (mapArgs.count("/dropmessages"))
2950 {
2951     nDropMessagesTest = atoi(mapArgs["/dropmessages"]);
2952 if (nDropMessagesTest == 0)
2953     nDropMessagesTest = 20;
2954 }
2955
2956 if (mapArgs.count("/loadblockindextest"))
2957 {
2958     CTxDB txdb("r");
2959     txdb.LoadBlockIndex();
2960 PrintBlockTree();
2961 ExitProcess(0);
2962 }
2963
2964 //
2965 // Load data files
2966 //
2967     string strErrors;
2968     int64 nStart, nEnd;
2969
2970 printf("Loading addresses...\n");
2971 QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
2972 if (!LoadAddresses())
2973     strErrors += "Error loading addr.dat\n";
2974 QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);

```

```

2975 printf(" addresses   %0I64d\n", nEnd - nStart);
2976
2977 printf("Loading block index...\n");
2978 QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
2979 if (!LoadBlockIndex())
2980     strErrors += "Error loading blkindex.dat      \n";
2981 QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
2982 printf(" block index %0I64d\n", nEnd - nStart);
2983
2984 printf("Loading wallet...\n");
2985 QueryPerformanceCounter((LARGE_INTEGER*)&nStart);
2986 if (!LoadWallet())
2987     strErrors += "Error loading wallet.dat      \n";
2988 QueryPerformanceCounter((LARGE_INTEGER*)&nEnd);
2989 printf(" wallet       %0I64d\n", nEnd - nStart);
2990
2991 printf("Done loading\n");
2992
2993 //// debug print
2994 printf("mapBlockIndex.size() = %d\n",   mapBlockIndex.size());
2995 printf("nBestHeight = %d\n",           nBestHeight);
2996 printf("mapKeys.size() = %d\n",         mapKeys.size());
2997 printf("mapPubKeys.size() = %d\n",      mapPubKeys.size());
2998 printf("mapWallet.size() = %d\n",       mapWallet.size());
2999 printf("mapAddressBook.size() = %d\n",   mapAddressBook.size());
3000
3001 if (!strErrors.empty())
3002 {
3003     wxMessageBox(strErrors);
3004     OnExit();
3005     return false;
3006 }
3007
3008 // Add wallet transactions that aren't already in a block to mapTransactions
3009 ReacceptWalletTransactions();
3010
3011 //
3012 // Parameters
3013 //
3014 if (mapArgs.count("/printblockindex") || mapArgs.count("/printblocktree"))
3015 {
3016     PrintBlockTree();
3017     OnExit();
3018     return false;
3019 }
3020
3021 if (mapArgs.count("/gen"))
3022 {
3023     if (mapArgs["/gen"].empty())
3024         fGenerateBitcoins = true;
3025     else
3026         fGenerateBitcoins = atoi(mapArgs["/gen"].c_str());
3027 }
3028
3029 //
3030 // Create the main frame window
3031 //
3032 {
3033     pframeMain = new CMainFrame(NULL);
3034     pframeMain->Show();
3035
3036 if (!StartNode(strErrors))
3037     wxMessageBox(strErrors);
3038
3039 if (fGenerateBitcoins)
3040 if (_beginthread(ThreadBitcoinMiner, 0, NULL) == -1)
3041     printf("Error: _beginthread(ThreadBitcoinMiner) failed\n");
3042
3043 //
3044 // Tests
3045 //
3046 if (argc >= 2 && strcmp(argv[1], "/send") == 0)

```

```

3047 {
3048     int64 nValue = 1;
3049     if (argc >= 3)
3050         ParseMoney(argv[2], nValue);
3051
3052     string strAddress;
3053     if (argc >= 4)
3054         strAddress = argv[3];
3055     CAddress addr(strAddress.c_str());
3056
3057     CWalletTx wtx;
3058     wtx.mapValue["to"] = strAddress;
3059     wtx.mapValue["from"] = addrLocalHost.ToString();
3060     wtx.mapValue["message"] = "command line send";
3061
3062     // Send to IP address
3063     CSendingDialog* pdialog = new CSendingDialog(pframeMain, addr, nValue, wtx);
3064     if (!pdialog->ShowModal())
3065         return false;
3066 }
3067
3068 if (mapArgs.count("/randsendtest"))
3069 {
3070     if (!mapArgs["/randsendtest"].empty())
3071         _beginthread(ThreadRandSendTest, 0, new string(mapArgs["/randsendtest"]));
3072     else
3073         fRandSendTest = true;
3074     fDebug = true;
3075 }
3076 }
3077
3078 return true;
3079 }
3080
3081 int CMyApp::OnExit()
3082 {
3083     Shutdown(NULL);
3084     return wxApp::OnExit();
3085 }
3086
3087 bool CMyApp::OnExceptionInMainLoop()
3088 {
3089     try
3090     {
3091         throw;
3092     }
3093     catch (std::exception& e)
3094     {
3095         PrintException(&e, "CMyApp::OnExceptionInMainLoop()");
3096         wxLogWarning(_T("Exception %s %s"), typeid(e).name(), e.what());
3097         Sleep(1000);
3098         throw;
3099     }
3100     catch (...)
3101     {
3102         PrintException(NULL, "CMyApp::OnExceptionInMainLoop()");
3103         wxLogWarning(_T("Unknown exception"));
3104         Sleep(1000);
3105         throw;
3106     }
3107
3108     return true;
3109 }
3110
3111 void CMyApp::OnUnhandledException()
3112 {
3113     // this shows how we may let some exception propagate uncaught
3114     try
3115     {
3116         throw;
3117     }
3118     catch (std::exception& e)

```

```

3119 {
3120     PrintException(&e, "CMyApp::OnUnhandledException()");
3121     wxLogWarning(_T("Exception %s %s"), typeid(e).name(), e.what());
3122     Sleep(1000);
3123     throw;
3124 }
3125 catch (...)
3126 {
3127     PrintException(NULL, "CMyApp::OnUnhandledException()");
3128     wxLogWarning(_T("Unknown exception"));
3129     Sleep(1000);
3130     throw;
3131 }
3132 }
3133
3134 void CMyApp::OnFatalException()
3135 {
3136     wxMessageBox("Program has crashed and will terminate. ", "", wxOK | wxICON_ERROR);
3137 }
3138
3139
3140
3141 void MainFrameRepaint()
3142 {
3143     if (pframeMain)
3144     {
3145         printf("MainFrameRepaint()\n");
3146         wxPaintEvent event;
3147         pframeMain->Refresh();
3148         pframeMain->AddPendingEvent(event);
3149     }
3150 }
3151
3152
3153
3154
3155
3156
3157
3158 // randsendtest to bitcoin address
3159 void ThreadRandSendTest(void* parg)
3160 {
3161     string strAddress = *(string*)parg;
3162     uint160 hash160;
3163     if (!AddressToHash160(strAddress, hash160))
3164     {
3165         wxMessageBox(strprintf("ThreadRandSendTest: Bitcoin address '%s' not valid ", strAddress.c_str()));
3166         return;
3167     }
3168
3169     loop
3170     {
3171         Sleep((GetRand(30) * 1000 + 100));
3172
3173         // Message
3174         CWalletTx wtx;
3175         wtx.mapValue["to"] = strAddress;
3176         wtx.mapValue["from"] = addrLocalHost.ToString();
3177         static int nRep;
3178         wtx.mapValue["message"] = strprintf("randsendtest %d\n", ++nRep);
3179
3180         // Value
3181         int64 nValue = (GetRand(9) + 1) * 100 * CENT;
3182         if (GetBalance() < nValue)
3183         {
3184             wxMessageBox("Out of money ");
3185             return;
3186         }
3187         nValue += (nRep % 100) * CENT;
3188
3189         // Send to bitcoin address
3190         CScript scriptPubKey;

```



```

3191         scriptPubKey << OP_DUP << OP_HASH160 << hash160 << OP_EQUALVERIFY << OP_CHECKSIG;
3192
3193     if (!SendMoney(scriptPubKey, nValue, wtx))
3194     return;
3195 }
3196 }
3197
3198
3199 // randsendtest to any connected node
3200 void RandSend()
3201 {
3202     CWalletTx wtx;
3203
3204     while (vNodes.empty())
3205     Sleep(1000);
3206     CAddress addr;
3207     CRITICAL_BLOCK(cs_vNodes)
3208     {
3209         addr = vNodes[GetRand(vNodes.size())]->addr;
3210     }
3211     // Message
3212     wtx.mapValue["to"] = addr.ToString();
3213     wtx.mapValue["from"] = addrLocalHost.ToString();
3214     static int nRep;
3215     wtx.mapValue["message"] = sprintf("randsendtest %d\n", ++nRep);
3216
3217     // Value
3218     int64 nValue = (GetRand(999) + 1) * CENT;
3219     if (GetBalance() < nValue)
3220     {
3221         wxMessageBox("Out of money ");
3222         return;
3223     }
3224     // Send to IP address
3225     CSendingDialog* pdialog = new CSendingDialog(pframeMain, addr, nValue, wtx);
3226     if (!pdialog->Show())
3227     wxMessageBox("ShowModal Failed ");
3228 }
3229

```

SUMMARY

Every dialog box, every status update, every user action flows through this code. It's where the abstract blockchain meets the human user.

ui.h

418 lines CPP

WINDOWS TO THE BLOCKCHAIN

Declares the GUI classes and helper functions. The main window, transaction dialogs, and address book are all defined here.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5
6
7
8 DECLARE_EVENT_TYPE(wxEVT_CROSSTHREADCALL, -1)
9 DECLARE_EVENT_TYPE(wxEVT_REPLY1, -1)
10 DECLARE_EVENT_TYPE(wxEVT_REPLY2, -1)

```

DECLARE_EVENT_TYPE - custom events for thread communication

```

11 DECLARE_EVENT_TYPE(wxEVT_REPLY3, -1)
12 DECLARE_EVENT_TYPE(wxEVT_TABLEADDED, -1)
13 DECLARE_EVENT_TYPE(wxEVT_TABLEUPDATED, -1)
14 DECLARE_EVENT_TYPE(wxEVT_TABLEDELETED, -1)
15
16 enum
17 {
18     UICALL_ADDORDER = 1,
19     UICALL_UPDATEORDER,
20 };
21
22
23
24 extern void HandleCtrlA(wxKeyEvent& event);
25 extern string DateTimeStr(int64 nTime);
26 extern string FormatTxStatus(const CWalletTx& wtx);
27 extern void CrossThreadCall(int nID, void* pdata);
28 extern void MainFrameRepaint();
29
30
31
32
33
34
35 class CMainFrame : public CMainFrameBase
36 {
37 protected:
38 // Event handlers

```

CMainFrame class - the main Bitcoin window

```

39 void OnClose(wxCloseEvent& event);
40 void OnMouseEvents(wxMouseEvent& event);
41 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
42 void OnIdle(wxIdleEvent& event);
43 void OnPaint(wxPaintEvent& event);
44 void OnPaintListCtrl(wxPaintEvent& event);
45 void OnMenuFileExit(wxCommandEvent& event);
46 void OnMenuOptionsGenerate(wxCommandEvent& event);
47 void OnMenuOptionsOptions(wxCommandEvent& event);
48 void OnMenuHelpAbout(wxCommandEvent& event);
49 void OnButtonSend(wxCommandEvent& event);
50 void OnButtonAddressBook(wxCommandEvent& event);
51 void OnSetFocusAddress(wxFocusEvent& event);
52 void OnMouseEventsAddress(wxMouseEvent& event);
53 void OnButtonCopy(wxCommandEvent& event);
54 void OnButtonChange(wxCommandEvent& event);
55 void OnListColBeginDrag(wxListEvent& event);
56 void OnListItemActivatedAllTransactions(wxListEvent& event);
57 void OnListItemActivatedProductsSent(wxListEvent& event);
58 void OnListItemActivatedOrdersSent(wxListEvent& event);
59 void OnListItemActivatedOrdersReceived(wxListEvent& event);
60
61 public:

```

```

62 /** Constructor */
63 CMainFrame(wxWindow* parent);
64 ~CMainFrame();
65
66 // Custom
67 bool fRefreshListCtrl;
68 bool fRefreshListCtrlRunning;
69 bool fOnSetFocusAddress;
70     CBlockIndex* pindexBestLast;
71     set<uint256> setUnmaturedDisplayed;
72
73 void OnCrossThreadCall(wxCommandEvent& event);
74 void InsertLine(bool fNew, int nIndex, uint256 hashKey, string strSort, const wxString& str1, const
    wxString& str2, const wxString& str3, const wxString& str4, const wxString& str5);
75 void InsertTransaction(const CWalletTx& wtx, bool fNew, int nIndex=-1);
76 void RefreshListCtrl();
77 void RefreshStatus();
78 };
79
80
81
82
83 class CTxDetailsDialog : public CTxDetailsDialogBase
84 {
85 protected:
86 // Event handlers
87 void OnButtonOK(wxCommandEvent& event);
88
89 public:
90 /** Constructor */
91 CTxDetailsDialog(wxWindow* parent, CWalletTx wtx);
92
93 // State
94     CWalletTx wtx;
95 };
96
97
98
99 class COptionsDialog : public COptionsDialogBase
100 {
101 protected:
102 // Event handlers
103 void OnKillFocusTransactionFee(wxFocusEvent& event);
104 void OnButtonOK(wxCommandEvent& event);
105 void OnButtonCancel(wxCommandEvent& event);
106
107 public:
108 /** Constructor */
109 COptionsDialog(wxWindow* parent);
110 };
111
112
113
114 class CAboutDialog : public CAboutDialogBase
115 {
116 protected:
117 // Event handlers
118 void OnButtonOK(wxCommandEvent& event);
119
120 public:
121 /** Constructor */
122 CAboutDialog(wxWindow* parent);
123 };
124
125
126
127 class CSendDialog : public CSendDialogBase
128 {
129 protected:
130 // Event handlers
131 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
132 void OnTextAddress(wxCommandEvent& event);

```

```

133 void OnKillFocusAmount(wxFocusEvent& event);
134 void OnButtonAddressBook(wxCommandEvent& event);
135 void OnButtonPaste(wxCommandEvent& event);
136 void OnButtonSend(wxCommandEvent& event);
137 void OnButtonCancel(wxCommandEvent& event);
138
139 public:
140 /** Constructor */
141 CSendDialog(wxWindow* parent, const wxString& strAddress="");
142 };
143
144
145
146 class CSendingDialog : public CSendingDialogBase
147 {
148 public:
149 // Event handlers
150 void OnClose(wxCloseEvent& event);
151 void OnButtonOK(wxCommandEvent& event);
152 void OnButtonCancel(wxCommandEvent& event);
153 void OnPaint(wxPaintEvent& event);
154
155 public:
156 /** Constructor */
157 CSendingDialog(wxWindow* parent, const CAddress& addrIn, int64 nPriceIn, const CWalletTx& wtxIn);
158 ~CSendingDialog();
159
160 // State
161 CAddress addr;
162 int64 nPrice;
163 CWalletTx wtx;
164 wxDateTime start;
165 string strStatus;
166 bool fCanCancel;
167 bool fAbort;

```

Dialog classes - send coins, options, address book

```

168 bool fSuccess;
169 bool fUIDone;
170 bool fWorkDone;
171
172 void Close();
173 void Repaint();
174 bool Status();
175 bool Status(const string& str);
176 bool Error(const string& str);
177 void StartTransfer();
178 void OnReply2(CDataStream& vRecv);
179 void OnReply3(CDataStream& vRecv);
180 };
181
182 void SendingDialogStartTransfer(void* parg);
183 void SendingDialogOnReply2(void* parg, CDataStream& vRecv);
184 void SendingDialogOnReply3(void* parg, CDataStream& vRecv);
185
186
187
188 class CYourAddressDialog : public CYourAddressDialogBase
189 {
190 protected:
191 // Event handlers
192 void OnListEndLabelEdit(wxListEvent& event);
193 void OnListItemSelected(wxListEvent& event);
194 void OnListItemActivated(wxListEvent& event);
195 void OnButtonRename(wxCommandEvent& event);
196 void OnButtonNew(wxCommandEvent& event);
197 void OnButtonCopy(wxCommandEvent& event);
198 void OnButtonOK(wxCommandEvent& event);

```

```

199 void OnButtonCancel(wxCommandEvent& event);
200 void OnClose(wxCloseEvent& event);
201
202 public:
203 /** Constructor */
204 CYourAddressDialog(wxWindow* parent);
205 CYourAddressDialog(wxWindow* parent, const string& strInitSelected);
206
207 // Custom
208 wxString GetAddress();
209 };
210
211
212
213 class CAddressBookDialog : public CAddressBookDialogBase
214 {
215 protected:
216 // Event handlers
217 void OnListEndLabelEdit(wxListEvent& event);
218 void OnListItemSelected(wxListEvent& event);
219 void OnListItemActivated(wxListEvent& event);
220 void OnButtonEdit(wxCommandEvent& event);
221 void OnButtonDelete(wxCommandEvent& event);
222 void OnButtonNew(wxCommandEvent& event);
223 void OnButtonCopy(wxCommandEvent& event);
224 void OnButtonOK(wxCommandEvent& event);
225 void OnButtonCancel(wxCommandEvent& event);
226 void OnClose(wxCloseEvent& event);
227
228 public:
229 /** Constructor */
230 CAddressBookDialog(wxWindow* parent, const wxString& strInitSelected, bool fSendingIn);
231
232 // Custom
233 bool fSending;
234 wxString GetAddress();
235 };
236
237
238
239 class CProductsDialog : public CProductsDialogBase
240 {
241 protected:
242 // Event handlers
243 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
244 void OnCombobox(wxCommandEvent& event);
245 void OnButtonSearch(wxCommandEvent& event);
246 void OnListItemActivated(wxListEvent& event);
247
248 public:
249 /** Constructor */
250 CProductsDialog(wxWindow* parent);
251
252 // Custom
253 vector<CProduct> m_vProduct;
254 };
255
256
257
258 class CEditProductDialog : public CEditProductDialogBase
259 {
260 protected:
261 // Event handlers
262 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
263 void OnButtonDel0(wxCommandEvent& event);
264 void OnButtonDel1(wxCommandEvent& event);
265 void OnButtonDel2(wxCommandEvent& event);
266 void OnButtonDel3(wxCommandEvent& event);
267 void OnButtonDel4(wxCommandEvent& event);
268 void OnButtonDel5(wxCommandEvent& event);
269 void OnButtonDel6(wxCommandEvent& event);
270 void OnButtonDel7(wxCommandEvent& event);

```

```

271 void OnButtonDel8(wxCommandEvent& event);
272 void OnButtonDel9(wxCommandEvent& event);
273 void OnButtonDel10(wxCommandEvent& event);
274 void OnButtonDel11(wxCommandEvent& event);
275 void OnButtonDel12(wxCommandEvent& event);
276 void OnButtonDel13(wxCommandEvent& event);
277 void OnButtonDel14(wxCommandEvent& event);
278 void OnButtonDel15(wxCommandEvent& event);
279 void OnButtonDel16(wxCommandEvent& event);
280 void OnButtonDel17(wxCommandEvent& event);
281 void OnButtonDel18(wxCommandEvent& event);
282 void OnButtonDel19(wxCommandEvent& event);
283 void OnButtonAddField(wxCommandEvent& event);
284 void OnButtonSend(wxCommandEvent& event);
285 void OnButtonPreview(wxCommandEvent& event);
286 void OnButtonCancel(wxCommandEvent& event);
287
288 public:
289 /** Constructor */
290 CEditProductDialog(wxWindow* parent);
291
292 // Custom
293 enum { FIELDS_MAX = 20 };
294     wxTextCtrl* m_textCtrlLabel[FIELDS_MAX];
295     wxTextCtrl* m_textCtrlField[FIELDS_MAX];
296     wxButton* m_buttonDel[FIELDS_MAX];
297
298 void LayoutAll();
299 void ShowLine(int i, bool fShow=true);
300 void OnButtonDel(wxCommandEvent& event, int n);
301 void SetProduct(const CProduct& productIn);
302 void GetProduct(CProduct& product);
303
304 };
305
306
307
308 class CViewProductDialog : public CViewProductDialogBase
309 {
310 protected:
311 // Event handlers
312 void OnButtonSubmitForm(wxCommandEvent& event);
313 void OnButtonCancelForm(wxCommandEvent& event);
314 void OnButtonBack(wxCommandEvent& event);
315 void OnButtonNext(wxCommandEvent& event);
316 void OnButtonCancel(wxCommandEvent& event);
317
318 public:
319 /** Constructor */
320 CViewProductDialog(wxWindow* parent, const CProduct& productIn);
321 ~CViewProductDialog();
322
323 // Custom
324     CProduct product;
325     enum { FIELDS_MAX = 20 };
326     wxStaticText* m_staticTextLabel[FIELDS_MAX];
327     wxTextCtrl* m_textCtrlField[FIELDS_MAX];
328     wxChoice* m_choiceField[FIELDS_MAX];
329
330 void GetOrder(CWalletTx& order);
331 void UpdateProductDisplay(bool fDetails);
332 void OnReply1(wxCommandEvent& event);
333 };
334
335
336
337 class CViewOrderDialog : public CViewOrderDialogBase
338 {
339 protected:
340 // Event handlers
341 void OnButtonOK(wxCommandEvent& event);
342

```

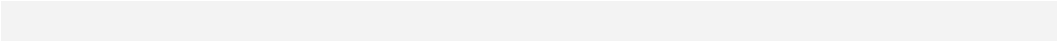


```

343 public:
344 /** Constructor */
345 CViewOrderDialog(wxWindow* parent, CWalletTx order, bool fReceived);
346
347 // Custom
348 bool fReceived;
349 };
350
351
352
353 class CEditReviewDialog : public CEditReviewDialogBase
354 {
355 protected:
356 // Event handlers
357 void OnKeyDown(wxKeyEvent& event) { HandleCtrlA(event); }
358 void OnButtonSubmit(wxCommandEvent& event);
359 void OnButtonCancel(wxCommandEvent& event);
360
361 public:
362 /** Constructor */
363 CEditReviewDialog(wxWindow* parent);
364
365 // Custom
366 void GetReview(CReview& review);
367 };
368
369
370
371 class CGetTextFromUserDialog : public CGetTextFromUserDialogBase
372 {
373 protected:
374 // Event handlers
375 void OnButtonOK(wxCommandEvent& event) { EndModal(true); }
376 void OnButtonCancel(wxCommandEvent& event) { EndModal(false); }
377 void OnClose(wxCloseEvent& event) { EndModal(false); }
378
379 void OnKeyDown(wxKeyEvent& event)
380 {
381 if (event.GetKeyCode() == '\r' || event.GetKeyCode() == WXK_NUMPAD_ENTER)
382 EndModal(true);
383 else
384 HandleCtrlA(event);
385 }
386
387 public:
388 /** Constructor */
389 CGetTextFromUserDialog(wxWindow* parent,
390 const string& strCaption,
391 const string& strMessage1,
392 const string& strValue1="",
393 const string& strMessage2="",
394 const string& strValue2="") : CGetTextFromUserDialogBase(parent, wxID_ANY, strCaption)
395 {
396 m_staticTextMessage1->SetLabel(strMessage1);
397 m_textCtrl1->SetValue(strValue1);
398 if (!strMessage2.empty())
399 {
400 m_staticTextMessage2->Show(true);
401 m_staticTextMessage2->SetLabel(strMessage2);
402 m_textCtrl2->Show(true);
403 m_textCtrl2->SetValue(strValue2);
404 SetSize(wxDefaultCoord, 180);
405 }
406 }
407
408 // Custom
409 string GetValue() { return (string)m_textCtrl1->GetValue(); }
410 string GetValue1() { return (string)m_textCtrl1->GetValue(); }
411 string GetValue2() { return (string)m_textCtrl2->GetValue(); }
412 };
413
414

```

415
416
417
418



SUMMARY

The GUI made Bitcoin accessible to non-technical users, critical for early adoption of the network.

ui.rc

15 lines TEXT

THE BITCOIN ICON

A Windows resource file that defines the application icon and other Windows-specific resources. This is what gave the original Bitcoin client its distinctive icon in the Windows taskbar.

```
1 bitcoin          ICON    "rc/bitcoin.ico"
```

bitcoin.ico - the original Bitcoin application icon

```
2
3 #include "wx/msw/wx.rc"
4
5 check            ICON    "rc/check.ico"
```

check.ico - checkmark icon for confirmed transactions

```
6 send16           BITMAP  "rc/send16.bmp"
```

send16 - toolbar icons for send functionality

```
7 send16mask       BITMAP  "rc/send16mask.bmp"
8 send16masknoshadow BITMAP "rc/send16masknoshadow.bmp"
9 send20           BITMAP  "rc/send20.bmp"
10 send20mask      BITMAP  "rc/send20mask.bmp"
11 addressbook16    BITMAP  "rc/addressbook16.bmp"
12 addressbook16mask BITMAP "rc/addressbook16mask.bmp"
13 addressbook20    BITMAP  "rc/addressbook20.bmp"
14 addressbook20mask BITMAP "rc/addressbook20mask.bmp"
15
```

SUMMARY

These resources defined the visual identity of the first Bitcoin client that users would see on their Windows desktops.

uibase.cpp

1807 lines CPP

THE SKELETON OF THE INTERFACE

Auto-generated code from wxFormBuilder that defines the base GUI classes. This code creates the actual windows, buttons, and text fields that make up the Bitcoin interface.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 ///////////////////////////////////////////////////////////////////
6 // C++ code generated with wxFormBuilder (version Apr 16 2008)
7 // http://www.wxformbuilder.org/
8 //
9 // PLEASE DO "NOT" EDIT THIS FILE!
10 ///////////////////////////////////////////////////////////////////

```

CMainFrameBase - the main window layout

```

11
12 #include "uibase.h"
13
14 ///////////////////////////////////////////////////////////////////
15
16 CMainFrameBase::CMainFrameBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint&
17 pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
18 {
19     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
20     this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_BTNFACE));
21
22     m_menubar = new wxMenuBar();
23     m_menubar->SetBackgroundColour(wxColour(240, 240, 240));
24
25     m_menuFile = new wxMenu();
26     wxMenuItem* m_menuFileExit;
27     m_menuFileExit = new wxMenuItem(m_menuFile, wxID_ANY, wxString(wxT("E&xit")), wxEmptyString,
28 wxITEM_NORMAL);
29     m_menuFile->Append(m_menuFileExit);
30
31     m_menubar->Append(m_menuFile, wxT("&File"));
32
33     m_menuOptions = new wxMenu();
34     wxMenuItem* m_menuOptionsGenerateBitcoins;
35     m_menuOptionsGenerateBitcoins = new wxMenuItem(m_menuOptions, wxID_OPTIONSGENERATEBITCOINS,
36 wxString(wxT("&Generate Coins")), wxEmptyString, wxITEM_CHECK);
37     m_menuOptions->Append(m_menuOptionsGenerateBitcoins);
38
39     wxMenuItem* m_menuOptionsOptions;
40     m_menuOptionsOptions = new wxMenuItem(m_menuOptions, wxID_ANY, wxString(wxT("&Options...")),
41 wxEmptyString, wxITEM_NORMAL);
42     m_menuOptions->Append(m_menuOptionsOptions);
43
44     m_menubar->Append(m_menuOptions, wxT("&Options"));
45
46     m_menuHelp = new wxMenu();
47     wxMenuItem* m_menuHelpAbout;
48     m_menuHelpAbout = new wxMenuItem(m_menuHelp, wxID_ANY, wxString(wxT("&About...")), wxEmptyString,
49 wxITEM_NORMAL);
50     m_menuHelp->Append(m_menuHelpAbout);
51
52     m_menubar->Append(m_menuHelp, wxT("&Help"));
53
54     this->SetMenuBar(m_menubar);
55
56     m_toolBar = this->CreateToolBar(wxTB_FLAT|wxTB_HORZ_TEXT, wxID_ANY);
57     m_toolBar->SetToolBitmapSize(wxSize(20,20));
58     m_toolBar->SetToolSeparation(1);
59     m_toolBar->SetFont(wxFont(wxNORMAL_FONT->GetPointSize(), 70, 90, 90, false, wxEmptyString));
60
61     m_toolBar->AddTool(wxID_BUTTONSEND, wxT("&Send Coins"), wxBitmap(wxT("send20")),
62 wxBITMAP_TYPE_RESOURCE), wxNullBitmap, wxITEM_NORMAL, wxEmptyString, wxEmptyString);
63     m_toolBar->AddTool(wxID_BUTTONRECEIVE, wxT("&Address Book"), wxBitmap(wxT("addressbook20")),
64 wxBITMAP_TYPE_RESOURCE), wxNullBitmap, wxITEM_NORMAL, wxEmptyString, wxEmptyString);
65     m_toolBar->Realize();
66

```

```

60     m_statusBar = this->CreateStatusBar(1, wxST_SIZEGRIP, wxID_ANY);
61     m_statusBar->SetBackgroundColour(wxColour(240, 240, 240));
62
63     wxBoxSizer* bSizer2;
64     bSizer2 = new wxBoxSizer(wxVERTICAL);
65
66
67     bSizer2->Add(0, 2, 0, wxEXPAND, 5);
68
69     wxBoxSizer* bSizer85;
70     bSizer85 = new wxBoxSizer(wxHORIZONTAL);
71
72     m_staticText32 = new wxStaticText(this, wxID_ANY, wxT("Your Bitcoin Address:"), wxDefaultPosition,
    wxDefaultSize, 0);
73     m_staticText32->Wrap(-1);
74     bSizer85->Add(m_staticText32, 0, wxALIGN_CENTER_VERTICAL|wxLEFT, 5);
75
76     m_textCtrlAddress = new wxTextCtrl(this, wxID_TEXTCTRLADDRESS, wxEmptyString, wxDefaultPosition,
    wxSize(250,-1), wxTE_READONLY);
77     m_textCtrlAddress->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
78
79     bSizer85->Add(m_textCtrlAddress, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
80
81     m_buttonCopy = new wxButton(this, wxID_BUTTONCOPY, wxT("&Copy to Clipboard"), wxDefaultPosition,
    wxDefaultSize, wxBU_EXACTFIT);
82     bSizer85->Add(m_buttonCopy, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT, 5);
83
84     m_button91 = new wxButton(this, wxID_BUTTONCHANGE, wxT("C&hange..."), wxDefaultPosition,
    wxDefaultSize, 0);
85     bSizer85->Add(m_button91, 0, wxRIGHT, 5);
86
87
88     bSizer85->Add(0, 0, 0, wxEXPAND, 5);
89
90     bSizer2->Add(bSizer85, 0, wxEXPAND|wxRIGHT|wxLEFT, 5);
91
92     wxBoxSizer* bSizer3;
93     bSizer3 = new wxBoxSizer(wxHORIZONTAL);
94
95     m_panel14 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
96     wxBoxSizer* bSizer66;
97     bSizer66 = new wxBoxSizer(wxHORIZONTAL);
98
99     m_staticText41 = new wxStaticText(m_panel14, wxID_ANY, wxT("Balance:"), wxDefaultPosition,
    wxSize(-1,15), 0);
100    m_staticText41->Wrap(-1);

```

Transaction list and status bar setup

```

101    bSizer66->Add(m_staticText41, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT, 5);
102
103    m_staticTextBalance = new wxStaticText(m_panel14, wxID_ANY, wxEmptyString, wxDefaultPosition,
    wxSize(120,15), wxALIGN_RIGHT|wxST_NO_AUTORESIZE);
104    m_staticTextBalance->Wrap(-1);
105    m_staticTextBalance->SetFont(wxFont(8, 70, 90, 90, false, wxEmptyString));
106    m_staticTextBalance->SetBackgroundColour(wxColour(240, 240, 240));
107
108    bSizer66->Add(m_staticTextBalance, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
109
110    m_panel14->SetSizer(bSizer66);
111    m_panel14->Layout();
112    bSizer66->Fit(m_panel14);
113    bSizer3->Add(m_panel14, 1, wxEXPAND|wxALIGN_BOTTOM|wxALL, 5);
114
115
116    bSizer3->Add(0, 0, 0, wxEXPAND, 5);
117
118    wxString m_choiceFilterChoices[] = { wxT(" All"), wxT(" Sent"), wxT(" Received"), wxT(" In
    Progress") };

```

```

119 int m_choiceFilterNChoices = sizeof(m_choiceFilterChoices) / sizeof(wxString);
120 m_choiceFilter = new wxChoice(this, wxID_ANY, wxDefaultPosition, wxSize(110,-1),
    m_choiceFilterNChoices, m_choiceFilterChoices, 0);
121 m_choiceFilter->SetSelection(0);
122 m_choiceFilter->Hide();
123
124 bSizer3->Add(m_choiceFilter, 0, wxALIGN_BOTTOM|wxTOP|wxRIGHT|wxLEFT, 5);
125
126 bSizer2->Add(bSizer3, 0, wxEXPAND, 5);
127
128 m_notebook = new wxNotebook(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, 0);
129 m_panel7 = new wxPanel(m_notebook, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
130 wxBoxSizer* bSizer157;
131 bSizer157 = new wxBoxSizer(wxVERTICAL);
132
133 m_listCtrl = new wxListCtrl(m_panel7, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_DESCENDING|wxALWAYS_SHOW_SB);
134 bSizer157->Add(m_listCtrl, 1, wxEXPAND|wxALL, 5);
135
136 m_panel7->SetSizer(bSizer157);
137 m_panel7->Layout();
138 bSizer157->Fit(m_panel7);
139 m_notebook->AddPage(m_panel7, wxT("All Transactions"), false);
140
141 bSizer2->Add(m_notebook, 1, wxEXPAND, 5);
142
143 wxBoxSizer* bSizer_TabsForFutureUse;
144 bSizer_TabsForFutureUse = new wxBoxSizer(wxVERTICAL);
145
146 m_panel9 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
147 m_panel9->Hide();
148
149 wxBoxSizer* bSizer159;
150 bSizer159 = new wxBoxSizer(wxVERTICAL);
151
152 m_listCtrlEscrows = new wxListCtrl(m_panel9, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT);
153 bSizer159->Add(m_listCtrlEscrows, 1, wxALL|wxEXPAND, 5);
154
155 m_panel9->SetSizer(bSizer159);
156 m_panel9->Layout();
157 bSizer159->Fit(m_panel9);
158 bSizer_TabsForFutureUse->Add(m_panel9, 1, wxEXPAND | wxALL, 5);
159
160 m_panel8 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
161 m_panel8->Hide();
162
163 wxBoxSizer* bSizer158;
164 bSizer158 = new wxBoxSizer(wxVERTICAL);
165
166 m_listCtrlOrdersSent = new wxListCtrl(m_panel8, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT);
167 bSizer158->Add(m_listCtrlOrdersSent, 1, wxALL|wxEXPAND, 5);
168
169 m_panel8->SetSizer(bSizer158);
170 m_panel8->Layout();
171 bSizer158->Fit(m_panel8);
172 bSizer_TabsForFutureUse->Add(m_panel8, 1, wxEXPAND | wxALL, 5);
173
174 m_panel10 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
175 m_panel10->Hide();
176
177 wxBoxSizer* bSizer160;
178 bSizer160 = new wxBoxSizer(wxVERTICAL);
179
180 m_listCtrlProductsSent = new wxListCtrl(m_panel10, wxID_ANY, wxDefaultPosition, wxDefaultSize,
    wxLC_NO_SORT_HEADER|wxLC_REPORT);
181 bSizer160->Add(m_listCtrlProductsSent, 1, wxALL|wxEXPAND, 5);
182
183 m_panel10->SetSizer(bSizer160);
184 m_panel10->Layout();
185 bSizer160->Fit(m_panel10);

```



```

186     bSizer_TabsForFutureUse->Add(m_panel10, 1, wxEXPAND | wxALL, 5);
187
188     m_panel11 = new wxPanel(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxTAB_TRAVERSAL);
189     m_panel11->Hide();
190
191     wxBoxSizer* bSizer161;
192     bSizer161 = new wxBoxSizer(wxVERTICAL);
193
194     m_listCtrlOrdersReceived = new wxListCtrl(m_panel11, wxID_ANY, wxDefaultPosition, wxDefaultSize,
        wxLC_NO_SORT_HEADER|wxLC_REPORT);
195     bSizer161->Add(m_listCtrlOrdersReceived, 1, wxALL|wxEXPAND, 5);
196
197     m_panel11->SetSizer(bSizer161);
198     m_panel11->Layout();
199     bSizer161->Fit(m_panel11);
200     bSizer_TabsForFutureUse->Add(m_panel11, 1, wxEXPAND | wxALL, 5);

```

Dialog base classes for send/receive

```

201
202     bSizer2->Add(bSizer_TabsForFutureUse, 1, wxEXPAND, 5);
203
204     this->SetSizer(bSizer2);
205     this->Layout();
206
207 // Connect Events
208 this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CMainFrameBase::OnClose));
209 this->Connect(wxEVT_IDLE, wxIdleEventHandler(CMainFrameBase::OnIdle));
210 this->Connect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
211 this->Connect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
212 this->Connect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
213 this->Connect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
214 this->Connect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
215 this->Connect(wxEVT_RIGHT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
216 this->Connect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
217 this->Connect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
218 this->Connect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
219 this->Connect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
220 this->Connect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
221 this->Connect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
222 this->Connect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
223 this->Connect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaint));
224 this->Connect(m_menuFileExit->GetId(), wxEVT_COMMAND_MENU_SELECTED,
        wxCommandEventHandler(CMainFrameBase::OnMenuFileExit));
225 this->Connect(m_menuOptionsGenerateBitcoins->GetId(), wxEVT_COMMAND_MENU_SELECTED,
        wxCommandEventHandler(CMainFrameBase::OnMenuOptionsGenerate));
226 this->Connect(m_menuOptionsOptions->GetId(), wxEVT_COMMAND_MENU_SELECTED,
        wxCommandEventHandler(CMainFrameBase::OnMenuOptionsOptions));
227 this->Connect(m_menuHelpAbout->GetId(), wxEVT_COMMAND_MENU_SELECTED,
        wxCommandEventHandler(CMainFrameBase::OnMenuHelpAbout));
228 this->Connect(wxID_BUTTONSEND, wxEVT_COMMAND_TOOL_CLICKED,
        wxCommandEventHandler(CMainFrameBase::OnButtonSend));
229 this->Connect(wxID_BUTTONRECEIVE, wxEVT_COMMAND_TOOL_CLICKED,
        wxCommandEventHandler(CMainFrameBase::OnButtonAddressBook));
230 m_textCtrlAddress->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CMainFrameBase::OnKeyDown), NULL,
        this);
231 m_textCtrlAddress->Connect(wxEVT_LEFT_DOWN,
        wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
232 m_textCtrlAddress->Connect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress),
        NULL, this);
233 m_textCtrlAddress->Connect(wxEVT_MIDDLE_DOWN,
        wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
234 m_textCtrlAddress->Connect(wxEVT_MIDDLE_UP,
        wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
235 m_textCtrlAddress->Connect(wxEVT_RIGHT_DOWN,
        wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
236 m_textCtrlAddress->Connect(wxEVT_RIGHT_UP,
        wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);

```

```

237     m_textCtrlAddress->Connect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress),
    NULL, this);
238     m_textCtrlAddress->Connect(wxEVT_LEFT_DCLICK,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
239     m_textCtrlAddress->Connect(wxEVT_MIDDLE_DCLICK,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
240     m_textCtrlAddress->Connect(wxEVT_RIGHT_DCLICK,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
241     m_textCtrlAddress->Connect(wxEVT_LEAVE_WINDOW,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
242     m_textCtrlAddress->Connect(wxEVT_ENTER_WINDOW,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
243     m_textCtrlAddress->Connect(wxEVT_MOUSEWHEEL,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
244     m_textCtrlAddress->Connect(wxEVT_SET_FOCUS, wxFocusEventHandler(CMainFrameBase::OnSetFocusAddress),
    NULL, this);
245     m_buttonCopy->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CMainFrameBase::OnButtonCopy), NULL, this);
246     m_button91->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CMainFrameBase::OnButtonChange), NULL, this);
247     m_listCtrl->Connect(wxEVT_COMMAND_LIST_COL_BEGIN_DRAG,
    wxListEventHandler(CMainFrameBase::OnListColBeginDrag), NULL, this);
248     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
    wxListEventHandler(CMainFrameBase::OnListItemActivatedAllTransactions), NULL, this);
249     m_listCtrl->Connect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaintListCtrl), NULL, this);
250     m_listCtrlOrdersSent->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
    wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersSent), NULL, this);
251     m_listCtrlProductsSent->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
    wxListEventHandler(CMainFrameBase::OnListItemActivatedProductsSent), NULL, this);
252     m_listCtrlOrdersReceived->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
    wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersReceived), NULL, this);
253 }
254
255 CMainFrameBase::~CMainFrameBase()
256 {
257     // Disconnect Events
258     this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CMainFrameBase::OnClose));
259     this->Disconnect(wxEVT_IDLE, wxIdleEventHandler(CMainFrameBase::OnIdle));
260     this->Disconnect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
261     this->Disconnect(wxEVT_LEFT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
262     this->Disconnect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
263     this->Disconnect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
264     this->Disconnect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
265     this->Disconnect(wxEVT_RIGHT_UP, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
266     this->Disconnect(wxEVT_MOTION, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
267     this->Disconnect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
268     this->Disconnect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
269     this->Disconnect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
270     this->Disconnect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
271     this->Disconnect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
272     this->Disconnect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CMainFrameBase::OnMouseEvents));
273     this->Disconnect(wxEVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaint));
274     this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED,
    wxCommandEventHandler(CMainFrameBase::OnMenuFileExit));
275     this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED,
    wxCommandEventHandler(CMainFrameBase::OnMenuOptionsGenerate));
276     this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED,
    wxCommandEventHandler(CMainFrameBase::OnMenuOptionsOptions));
277     this->Disconnect(wxID_ANY, wxEVT_COMMAND_MENU_SELECTED,
    wxCommandEventHandler(CMainFrameBase::OnMenuHelpAbout));
278     this->Disconnect(wxID_BUTTONSEND, wxEVT_COMMAND_TOOL_CLICKED,
    wxCommandEventHandler(CMainFrameBase::OnButtonSend));
279     this->Disconnect(wxID_BUTTONRECEIVE, wxEVT_COMMAND_TOOL_CLICKED,
    wxCommandEventHandler(CMainFrameBase::OnButtonAddressBook));
280     m_textCtrlAddress->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CMainFrameBase::OnKeyDown), NULL,
    this);
281     m_textCtrlAddress->Disconnect(wxEVT_LEFT_DOWN,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
282     m_textCtrlAddress->Disconnect(wxEVT_LEFT_UP,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
283     m_textCtrlAddress->Disconnect(wxEVT_MIDDLE_DOWN,
    wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);

```

```

284     m_textCtrlAddress->Disconnect(wx.EVT_MIDDLE_UP,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
285     m_textCtrlAddress->Disconnect(wx.EVT_RIGHT_DOWN,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
286     m_textCtrlAddress->Disconnect(wx.EVT_RIGHT_UP,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
287     m_textCtrlAddress->Disconnect(wx.EVT_MOTION,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
288     m_textCtrlAddress->Disconnect(wx.EVT_LEFT_DCLICK,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
289     m_textCtrlAddress->Disconnect(wx.EVT_MIDDLE_DCLICK,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
290     m_textCtrlAddress->Disconnect(wx.EVT_RIGHT_DCLICK,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
291     m_textCtrlAddress->Disconnect(wx.EVT_LEAVE_WINDOW,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
292     m_textCtrlAddress->Disconnect(wx.EVT_ENTER_WINDOW,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
293     m_textCtrlAddress->Disconnect(wx.EVT_MOUSEWHEEL,
wxMouseEventHandler(CMainFrameBase::OnMouseEventsAddress), NULL, this);
294     m_textCtrlAddress->Disconnect(wx.EVT_SET_FOCUS,
wxFocusEventHandler(CMainFrameBase::OnSetFocusAddress), NULL, this);
295     m_buttonCopy->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CMainFrameBase::OnButtonCopy), NULL, this);
296     m_button91->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CMainFrameBase::OnButtonChange), NULL, this);
297     m_listCtrl->Disconnect(wx.EVT_COMMAND_LIST_COL_BEGIN_DRAG,
wxListEventHandler(CMainFrameBase::OnListColBeginDrag), NULL, this);
298     m_listCtrl->Disconnect(wx.EVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedAllTransactions), NULL, this);
299     m_listCtrl->Disconnect(wx.EVT_PAINT, wxPaintEventHandler(CMainFrameBase::OnPaintListCtrl), NULL,
this);
300     m_listCtrlOrdersSent->Disconnect(wx.EVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersSent), NULL, this);
301     m_listCtrlProductsSent->Disconnect(wx.EVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedProductsSent), NULL, this);
302     m_listCtrlOrdersReceived->Disconnect(wx.EVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CMainFrameBase::OnListItemActivatedOrdersReceived), NULL, this);
303 }
304
305 CTxDetailsDialogBase:CTxDetailsDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
306 {
307     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
308
309     wxBoxSizer* bSizer64;
310     bSizer64 = new wxBoxSizer(wx.VERTICAL);
311
312     wxBoxSizer* bSizer66;
313     bSizer66 = new wxBoxSizer(wx.VERTICAL);
314
315     m_htmlWin = new wxHtmlWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxHW_SCROLLBAR_AUTO);
316     bSizer66->Add(m_htmlWin, 1, wxALL|wxEXPAND, 5);
317
318     bSizer64->Add(bSizer66, 1, wxEXPAND, 5);
319
320     wxBoxSizer* bSizer65;
321     bSizer65 = new wxBoxSizer(wx.VERTICAL);
322
323     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(85,25), 0);
324     bSizer65->Add(m_buttonOK, 0, wxALL, 5);
325
326     bSizer64->Add(bSizer65, 0, wxALIGN_RIGHT, 5);
327
328     this->SetSizer(bSizer64);
329     this->Layout();
330
331 // Connect Events
332     m_buttonOK->Connect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CTxDetailsDialogBase::OnButtonOK), NULL, this);
333 }
334

```

```

335 CTxDetailsDialogBase::~CTxDetailsDialogBase()
336 {
337     // Disconnect Events
338     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CTxDetailsDialogBase::OnButtonOK), NULL, this);
339 }
340
341 COptionsDialogBase::COptionsDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
342 {
343     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
344
345     wxBoxSizer* bSizer55;
346     bSizer55 = new wxBoxSizer(wxVERTICAL);
347
348     wxBoxSizer* bSizer57;
349     bSizer57 = new wxBoxSizer(wxVERTICAL);
350
351     bSizer57->Add(0, 20, 0, wxEXPAND, 5);
352
353     m_staticText32 = new wxStaticText(this, wxID_ANY, wxT("Optional transaction fee you give to the
nodes that process your transactions."), wxDefaultPosition, wxDefaultSize, 0);
354     m_staticText32->Wrap(-1);
355     bSizer57->Add(m_staticText32, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
356
357     wxBoxSizer* bSizer56;
358     bSizer56 = new wxBoxSizer(wxHORIZONTAL);
359
360     m_staticText31 = new wxStaticText(this, wxID_ANY, wxT("Transaction fee:"), wxDefaultPosition,
wxDefaultSize, 0);
361     m_staticText31->Wrap(-1);
362     bSizer56->Add(m_staticText31, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT, 5);
363
364     m_textCtrlTransactionFee = new wxTextCtrl(this, wxID_TRANSACTIONFEE, wxEmptyString,
wxDefaultPosition, wxSize(70,-1), 0);
365     bSizer56->Add(m_textCtrlTransactionFee, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
366
367     bSizer57->Add(bSizer56, 0, wxEXPAND, 5);
368
369     bSizer55->Add(bSizer57, 1, wxEXPAND|wxLEFT, 5);
370
371     wxBoxSizer* bSizer58;
372     bSizer58 = new wxBoxSizer(wxHORIZONTAL);
373
374     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(85,25), 0);
375     bSizer58->Add(m_buttonOK, 0, wxALL, 5);
376
377     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1),
0);
378     m_buttonCancel->SetMinSize(wxSize(85,25));
379
380     bSizer58->Add(m_buttonCancel, 0, wxALL, 5);
381
382     bSizer55->Add(bSizer58, 0, wxALIGN_RIGHT, 5);
383
384     this->SetSizer(bSizer55);
385     this->Layout();
386
387     // Connect Events
388     m_textCtrlTransactionFee->Connect(wxEVT_KILL_FOCUS,
wxFocusEventHandler(COptionsDialogBase::OnKillFocusTransactionFee), NULL, this);
389     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(COptionsDialogBase::OnButtonOK), NULL, this);
390     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(COptionsDialogBase::OnButtonCancel), NULL, this);
391 }
392
393 COptionsDialogBase::~COptionsDialogBase()
394 {
395     // Disconnect Events

```

```

397     m_textCtrlTransactionFee->Disconnect(wx.EVT_KILL_FOCUS,
wxFocusEventHandler(COptionsDialogBase::OnKillFocusTransactionFee), NULL, this);
398     m_buttonOK->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(COptionsDialogBase::OnButtonOK), NULL, this);
399     m_buttonCancel->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(COptionsDialogBase::OnButtonCancel), NULL, this);
400 }
401
402 CAboutDialogBase::CAboutDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
403 {
404     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
405
406     wxBoxSizer* bSizer60;
407     bSizer60 = new wxBoxSizer(wxVERTICAL);
408
409     wxBoxSizer* bSizer62;
410     bSizer62 = new wxBoxSizer(wxHORIZONTAL);
411
412     bSizer62->Add(60, 0, 0, wxEXPAND, 5);
413
414     wxBoxSizer* bSizer63;
415     bSizer63 = new wxBoxSizer(wxVERTICAL);
416
417     bSizer63->Add(0, 50, 0, wxEXPAND, 5);
418
419     wxBoxSizer* bSizer64;
420     bSizer64 = new wxBoxSizer(wxHORIZONTAL);
421
422     m_staticText40 = new wxStaticText(this, wxID_ANY, wxT("Bitcoin "), wxDefaultPosition, wxDefaultSize,
0);
423
424     m_staticText40->Wrap(-1);
425     m_staticText40->SetFont(wxFont(10, 74, 90, 92, false, wxT("Tahoma")));
426
427     bSizer64->Add(m_staticText40, 0, wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxLEFT, 5);
428
429     m_staticTextVersion = new wxStaticText(this, wxID_ANY, wxT("version"), wxDefaultPosition,
wxDefaultSize, 0);
430
431     m_staticTextVersion->Wrap(-1);
432     m_staticTextVersion->SetFont(wxFont(10, 74, 90, 90, false, wxT("Tahoma")));
433
434     bSizer64->Add(m_staticTextVersion, 0, wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxRIGHT, 5);
435
436     bSizer63->Add(bSizer64, 0, wxEXPAND, 5);
437
438     bSizer63->Add(0, 4, 0, wxEXPAND, 5);
439
440     m_staticTextMain = new wxStaticText(this, wxID_ANY, wxT("Copyright © 2009 Satoshi Nakamoto.\n\nThis
is experimental software. Do not rely on it for actual financial transactions.\n\nDistributed under the
MIT/X11 software license, see the accompanying file license.txt or
http://www.opensource.org/licenses/mit-license.php.\n\nThis product includes software developed by the
OpenSSL Project for use in the OpenSSL Toolkit (http://www.openssl.org/) and cryptographic software
written by Eric Young (eay@cryptsoft.com)."), wxDefaultPosition, wxDefaultSize, 0);
441
442     m_staticTextMain->Wrap(400);
443     bSizer63->Add(m_staticTextMain, 0, wxALL, 5);
444
445     bSizer63->Add(0, 0, 1, wxEXPAND, 5);
446
447     bSizer62->Add(bSizer63, 1, wxEXPAND, 5);
448
449     bSizer60->Add(bSizer62, 1, wxEXPAND, 5);
450
451     wxBoxSizer* bSizer61;
452     bSizer61 = new wxBoxSizer(wxHORIZONTAL);
453
454     bSizer61->Add(0, 0, 1, wxEXPAND, 5);
455
456     bSizer61->Add(0, 0, 1, wxEXPAND, 5);
457

```

```

458     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(85,25), 0);
459     bSizer61->Add(m_buttonOK, 0, wxALL, 5);
460
461     bSizer60->Add(bSizer61, 0, wxALIGN_RIGHT|wxEXPAND, 5);
462
463     this->SetSizer(bSizer60);
464     this->Layout();
465
466     // Connect Events
467     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
468         wxCommandEventHandler(CAboutDialogBase::OnButtonOK), NULL, this);
469 }
470 CAboutDialogBase::~CAboutDialogBase()
471 {
472     // Disconnect Events
473     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
474         wxCommandEventHandler(CAboutDialogBase::OnButtonOK), NULL, this);
475 }
476 CSendDialogBase::CSendDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const wxPoint&
477     pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
478 {
479     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
480
481     wxBoxSizer* bSizer21;
482     bSizer21 = new wxBoxSizer(wxVERTICAL);
483
484     bSizer21->Add(0, 5, 0, wxEXPAND, 5);
485
486     wxFlexGridSizer* fgSizer1;
487     fgSizer1 = new wxFlexGridSizer(3, 2, 0, 0);
488     fgSizer1->AddGrowableCol(1);
489     fgSizer1->SetFlexibleDirection(wxBOTH);
490     fgSizer1->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
491
492
493     fgSizer1->Add(0, 0, 0, wxEXPAND, 5);
494
495     m_staticText14 = new wxStaticText(this, wxID_ANY, wxT("Enter the recipient's IP address (e.g.
123.45.6.7) for online transfer with comments and confirmation, \nor bitcoin address (e.g.
1NS17ia9jJgTHD1VXjvLCEnZuQ3rJED9L) if recipient is not online."), wxDefaultPosition, wxDefaultSize, 0);
496     m_staticText14->Wrap(-1);
497     fgSizer1->Add(m_staticText14, 0, wxTOP|wxRIGHT|wxLEFT, 5);
498
499     wxBoxSizer* bSizer47;
500     bSizer47 = new wxBoxSizer(wxHORIZONTAL);
501
502     bSizer47->SetMinSize(wxSize(70,-1));
503
504     bSizer47->Add(0, 0, 1, wxEXPAND, 5);
505
506     m_bitmapCheckMark = new wxStaticBitmap(this, wxID_ANY, wxICON(check), wxDefaultPosition,
507         wxSize(16,16), 0);
508     bSizer47->Add(m_bitmapCheckMark, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
509
510     m_staticText36 = new wxStaticText(this, wxID_ANY, wxT("Pay &To:"), wxDefaultPosition, wxSize(-1,-1),
511         wxALIGN_RIGHT);
512     m_staticText36->Wrap(-1);
513     bSizer47->Add(m_staticText36, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT, 5);
514
515     fgSizer1->Add(bSizer47, 1, wxEXPAND|wxLEFT, 5);
516
517     wxBoxSizer* bSizer19;
518     bSizer19 = new wxBoxSizer(wxHORIZONTAL);
519
520     m_textCtrlAddress = new wxTextCtrl(this, wxID_TEXTCTRLPAYTO, wxEmptyString, wxDefaultPosition,
521         wxDefaultSize, 0);
522     bSizer19->Add(m_textCtrlAddress, 1, wxALIGN_CENTER_VERTICAL|wxALL, 5);

```

```

521     m_buttonPaste = new wxButton(this, wxID_BUTTONPASTE, wxT("&Paste"), wxDefaultPosition,
wxSize(-1,-1), wxBU_EXACTFIT);
522     bSizer19->Add(m_buttonPaste, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT, 5);
523
524     m_buttonAddress = new wxButton(this, wxID_BUTTONADDRESSBOOK, wxT(" Address &Book..."),
wxDefaultPosition, wxDefaultSize, 0);
525     bSizer19->Add(m_buttonAddress, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT, 5);
526
527     fgSizer1->Add(bSizer19, 1, wxEXPAND|wxRIGHT, 5);
528
529     m_staticText19 = new wxStaticText(this, wxID_ANY, wxT("&Amount:"), wxDefaultPosition, wxSize(-1,-1),
wxALIGN_RIGHT);
530     m_staticText19->Wrap(-1);
531     fgSizer1->Add(m_staticText19, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT|wxALIGN_RIGHT, 5);
532
533     m_textCtrlAmount = new wxTextCtrl(this, wxID_TEXTCTRLAMOUNT, wxEmptyString, wxDefaultPosition,
wxSize(145,-1), 0);
534     m_textCtrlAmount->SetMaxLength(20);
535     m_textCtrlAmount->SetFont(wxFont(wxNORMAL_FONT->GetPointSize(), 70, 90, 90, false, wxEmptyString));
536
537     fgSizer1->Add(m_textCtrlAmount, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
538
539     m_staticText20 = new wxStaticText(this, wxID_ANY, wxT("T&ransfer:"), wxDefaultPosition,
wxSize(-1,-1), wxALIGN_RIGHT);
540     m_staticText20->Wrap(-1);
541     fgSizer1->Add(m_staticText20, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
542
543     wxString m_choiceTransferTypeChoices[] = { wxT(" Standard") };
544     int m_choiceTransferTypeNChoices = sizeof(m_choiceTransferTypeChoices) / sizeof(wxString);
545     m_choiceTransferType = new wxChoice(this, wxID_CHOICETRANSFERTYPE, wxDefaultPosition, wxDefaultSize,
m_choiceTransferTypeNChoices, m_choiceTransferTypeChoices, 0);
546     m_choiceTransferType->SetSelection(0);
547     fgSizer1->Add(m_choiceTransferType, 0, wxALIGN_CENTER_VERTICAL|wxALL, 5);
548
549     bSizer21->Add(fgSizer1, 0, wxEXPAND|wxLEFT, 5);
550
551     wxBoxSizer* bSizer672;
552     bSizer672 = new wxBoxSizer(wxHORIZONTAL);
553
554     wxBoxSizer* bSizer681;
555     bSizer681 = new wxBoxSizer(wxVERTICAL);
556
557     m_staticTextFrom = new wxStaticText(this, wxID_ANY, wxT("&From:"), wxDefaultPosition, wxDefaultSize,
0);
558     m_staticTextFrom->Wrap(-1);
559     bSizer681->Add(m_staticTextFrom, 0, wxBOTTOM|wxLEFT, 5);
560
561     m_textCtrlFrom = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxSize(-1,-1), 0);
562     bSizer681->Add(m_textCtrlFrom, 0, wxLEFT|wxEXPAND, 5);
563
564     bSizer672->Add(bSizer681, 1, wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT, 5);
565
566     bSizer21->Add(bSizer672, 0, wxEXPAND, 5);
567
568     wxBoxSizer* bSizer67;
569     bSizer67 = new wxBoxSizer(wxHORIZONTAL);
570
571     wxBoxSizer* bSizer68;
572     bSizer68 = new wxBoxSizer(wxVERTICAL);
573
574     m_staticTextMessage = new wxStaticText(this, wxID_ANY, wxT("&Message:"), wxDefaultPosition,
wxDefaultSize, 0);
575     m_staticTextMessage->Wrap(-1);
576     bSizer68->Add(m_staticTextMessage, 0, wxTOP|wxBOTTOM|wxLEFT, 5);
577
578     m_textCtrlMessage = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
wxTE_MULTILINE);
579     bSizer68->Add(m_textCtrlMessage, 1, wxEXPAND|wxLEFT, 5);
580
581     bSizer67->Add(bSizer68, 1, wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT, 5);
582
583     bSizer21->Add(bSizer67, 1, wxEXPAND, 5);

```

```

584
585     wxBoxSizer* bSizer23;
586     bSizer23 = new wxBoxSizer(wxHORIZONTAL);
587
588
589     bSizer23->Add(0, 0, 1, wxEXPAND, 5);
590
591     m_buttonSend = new wxButton(this, wxID_BUTTONSEND, wxT("&Send"), wxDefaultPosition, wxSize(-1,-1),
592 0);
593     m_buttonSend->SetFont(wxFont(wxNORMAL_FONT->GetPointSize(), 70, 90, 90, false, wxEmptyString));
594     m_buttonSend->SetMinSize(wxSize(85,25));
595
596     bSizer23->Add(m_buttonSend, 0, wxALL, 5);
597
598     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1),
599 0);
600     m_buttonCancel->SetMinSize(wxSize(85,25));
601
602     bSizer23->Add(m_buttonCancel, 0, wxALL, 5);
603
604     bSizer21->Add(bSizer23, 0, wxEXPAND, 5);
605
606     this->SetSizer(bSizer21);
607     this->Layout();
608
609 // Connect Events
610 m_textCtrlAddress->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
611 this);
612 m_textCtrlAddress->Connect(wxEVT_COMMAND_TEXT_UPDATED,
613 wxCommandEventHandler(CSendDialogBase::OnTextAddress), NULL, this);
614 m_buttonPaste->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
615 wxCommandEventHandler(CSendDialogBase::OnButtonPaste), NULL, this);
616 m_buttonAddress->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
617 wxCommandEventHandler(CSendDialogBase::OnButtonAddressBook), NULL, this);
618 m_textCtrlAmount->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
619 this);
620 m_textCtrlAmount->Connect(wxEVT_KILL_FOCUS, wxFocusEventHandler(CSendDialogBase::OnKillFocusAmount),
621 NULL, this);
622 m_textCtrlFrom->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL, this);
623 m_textCtrlMessage->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
624 this);
625 m_buttonSend->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
626 wxCommandEventHandler(CSendDialogBase::OnButtonSend), NULL, this);
627 m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
628 wxCommandEventHandler(CSendDialogBase::OnButtonCancel), NULL, this);
629 }
630
631 CSendDialogBase::~CSendDialogBase()
632 {
633 // Disconnect Events
634 m_textCtrlAddress->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
635 this);
636 m_textCtrlAddress->Disconnect(wxEVT_COMMAND_TEXT_UPDATED,
637 wxCommandEventHandler(CSendDialogBase::OnTextAddress), NULL, this);
638 m_buttonPaste->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
639 wxCommandEventHandler(CSendDialogBase::OnButtonPaste), NULL, this);
640 m_buttonAddress->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
641 wxCommandEventHandler(CSendDialogBase::OnButtonAddressBook), NULL, this);
642 m_textCtrlAmount->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
643 this);
644 m_textCtrlAmount->Disconnect(wxEVT_KILL_FOCUS,
645 wxFocusEventHandler(CSendDialogBase::OnKillFocusAmount), NULL, this);
646 m_textCtrlFrom->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
647 this);
648 m_textCtrlMessage->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CSendDialogBase::OnKeyDown), NULL,
649 this);
650 m_buttonSend->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
651 wxCommandEventHandler(CSendDialogBase::OnButtonSend), NULL, this);
652 m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
653 wxCommandEventHandler(CSendDialogBase::OnButtonCancel), NULL, this);
654 }

```



```

635 CSendingDialogBase::CSendingDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
    wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
636 {
637     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
638
639     wxBoxSizer* bSizer68;
640     bSizer68 = new wxBoxSizer(wxVERTICAL);
641
642     m_staticTextSending = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition,
        wxSize(-1,14), 0);
643     m_staticTextSending->Wrap(-1);
644     bSizer68->Add(m_staticTextSending, 0, wxALIGN_CENTER_VERTICAL|wxEXPAND|wxTOP|wxRIGHT|wxLEFT, 8);
645
646     m_textCtrlStatus = new wxTextCtrl(this, wxID_ANY, wxT("\n\nConnecting..."), wxDefaultPosition,
        wxDefaultSize, wxTE_CENTRE|wxTE_MULTILINE|wxTE_NO_VSCROLL|wxTE_READONLY|wxNO_BORDER);
647     m_textCtrlStatus->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_BTNFACE));
648
649     bSizer68->Add(m_textCtrlStatus, 1, wxEXPAND|wxRIGHT|wxLEFT, 10);
650
651     wxBoxSizer* bSizer69;
652     bSizer69 = new wxBoxSizer(wxHORIZONTAL);
653
654     bSizer69->Add(0, 0, 1, wxEXPAND, 5);
655
656     m_buttonOK = new wxButton(this, wxID_ANY, wxT("OK"), wxDefaultPosition, wxDefaultSize, 0);
657     m_buttonOK->Enable(false);
658     m_buttonOK->SetMinSize(wxSize(85,25));
659
660     bSizer69->Add(m_buttonOK, 0, wxALL, 5);
661
662     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1),
        0);
663     m_buttonCancel->SetMinSize(wxSize(85,25));
664
665     bSizer69->Add(m_buttonCancel, 0, wxALL, 5);
666
667     bSizer68->Add(bSizer69, 0, wxEXPAND, 5);
668
669     this->SetSizer(bSizer68);
670     this->Layout();
671
672 // Connect Events
673 this->Connect(wx.EVT_CLOSE_WINDOW, wxCloseEventHandler(CSendingDialogBase::OnClose));
674 this->Connect(wx.EVT_PAINT, wxPaintEventHandler(CSendingDialogBase::OnPaint));
675 m_buttonOK->Connect(wx.EVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CSendingDialogBase::OnButtonOK), NULL, this);
676 m_buttonCancel->Connect(wx.EVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CSendingDialogBase::OnButtonCancel), NULL, this);
677 }
678
679 CSendingDialogBase::~CSendingDialogBase()
680 {
681 // Disconnect Events
682 this->Disconnect(wx.EVT_CLOSE_WINDOW, wxCloseEventHandler(CSendingDialogBase::OnClose));
683 this->Disconnect(wx.EVT_PAINT, wxPaintEventHandler(CSendingDialogBase::OnPaint));
684 m_buttonOK->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CSendingDialogBase::OnButtonOK), NULL, this);
685 m_buttonCancel->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
    wxCommandEventHandler(CSendingDialogBase::OnButtonCancel), NULL, this);
686 }
687
688 CYourAddressDialogBase::CYourAddressDialogBase(wxWindow* parent, wxWindowID id, const wxString& title,
    const wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
689 {
690     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
691
692     wxBoxSizer* bSizer68;
693     bSizer68 = new wxBoxSizer(wxVERTICAL);
694
695     bSizer68->Add(0, 5, 0, wxEXPAND, 5);
696
697

```

```

698
699     m_staticText45 = new wxStaticText(this, wxID_ANY, wxT("These are your Bitcoin addresses for
receiving payments.\nYou may want to give a different one to each sender so you can keep track of who is
paying you."), wxDefaultPosition, wxDefaultSize, 0);
700     m_staticText45->Wrap(600);
701     bSizer68->Add(m_staticText45, 0, wxALL, 5);
702
703     m_listCtrl = new wxListCtrl(this, wxID_LISTCTRL, wxDefaultPosition, wxDefaultSize,
wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING);
704     bSizer68->Add(m_listCtrl, 1, wxALL|wxEXPAND, 5);
705
706     wxBoxSizer* bSizer69;
707     bSizer69 = new wxBoxSizer(wxHORIZONTAL);
708
709
710     bSizer69->Add(0, 0, 1, wxEXPAND, 5);
711
712     m_buttonRename = new wxButton(this, wxID_BUTTONRENAME, wxT("&Rename..."), wxDefaultPosition,
wxDefaultSize, 0);
713     m_buttonRename->SetMinSize(wxSize(85,25));
714
715     bSizer69->Add(m_buttonRename, 0, wxALL, 5);
716
717     m_buttonNew = new wxButton(this, wxID_BUTTONNEW, wxT("&New Address..."), wxDefaultPosition,
wxSize(-1,-1), 0);
718     m_buttonNew->SetMinSize(wxSize(110,25));
719
720     bSizer69->Add(m_buttonNew, 0, wxALL, 5);
721
722     m_buttonCopy = new wxButton(this, wxID_BUTTONCOPY, wxT("&Copy to Clipboard"), wxDefaultPosition,
wxSize(-1,-1), 0);
723     m_buttonCopy->SetMinSize(wxSize(120,25));
724
725     bSizer69->Add(m_buttonCopy, 0, wxALL, 5);
726
727     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxDefaultSize, 0);
728     m_buttonOK->SetMinSize(wxSize(85,25));
729
730     bSizer69->Add(m_buttonOK, 0, wxALL, 5);
731
732     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1),
0);
733     m_buttonCancel->SetMinSize(wxSize(85,25));
734
735     bSizer69->Add(m_buttonCancel, 0, wxALL, 5);
736
737     bSizer68->Add(bSizer69, 0, wxEXPAND, 5);
738
739     this->SetSizer(bSizer68);
740     this->Layout();
741
742     // Connect Events
743     this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CYourAddressDialogBase::OnClose));
744     m_listCtrl->Connect(wxEVT_COMMAND_LIST_END_LABEL_EDIT,
wxListEventHandler(CYourAddressDialogBase::OnListEndLabelEdit), NULL, this);
745     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CYourAddressDialogBase::OnListItemActivated), NULL, this);
746     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_SELECTED,
wxListEventHandler(CYourAddressDialogBase::OnListItemSelected), NULL, this);
747     m_buttonRename->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonRename), NULL, this);
748     m_buttonNew->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonNew), NULL, this);
749     m_buttonCopy->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonCopy), NULL, this);
750     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonOK), NULL, this);
751     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonCancel), NULL, this);
752 }
753
754 CYourAddressDialogBase::~CYourAddressDialogBase()

```

```

755 {
756 // Disconnect Events
757 this->Disconnect(wx.EVT_CLOSE_WINDOW, wxCloseEventHandler(CYourAddressDialogBase::OnClose));
758 m_listCtrl->Disconnect(wx.EVT_COMMAND_LIST_END_LABEL_EDIT,
wxListEventHandler(CYourAddressDialogBase::OnListEndLabelEdit), NULL, this);
759 m_listCtrl->Disconnect(wx.EVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CYourAddressDialogBase::OnListItemActivated), NULL, this);
760 m_listCtrl->Disconnect(wx.EVT_COMMAND_LIST_ITEM_SELECTED,
wxListEventHandler(CYourAddressDialogBase::OnListItemSelected), NULL, this);
761 m_buttonRename->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonRename), NULL, this);
762 m_buttonNew->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonNew), NULL, this);
763 m_buttonCopy->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonCopy), NULL, this);
764 m_buttonOK->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonOK), NULL, this);
765 m_buttonCancel->Disconnect(wx.EVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CYourAddressDialogBase::OnButtonCancel), NULL, this);
766 }
767
768 CAddressBookDialogBase::CAddressBookDialogBase(wxWindow* parent, wxWindowID id, const wxString& title,
const wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
769 {
770 this->SetSizeHints(wxDefaultSize, wxDefaultSize);
771
772 wxBoxSizer* bSizer68;
773 bSizer68 = new wxBoxSizer(wxVERTICAL);
774
775
776 bSizer68->Add(0, 5, 0, wxEXPAND, 5);
777
778 m_staticText55 = new wxStaticText(this, wxID_ANY, wxT("Bitcoin Address"), wxDefaultPosition,
wxDefaultSize, 0);
779 m_staticText55->Wrap(-1);
780 m_staticText55->Hide();
781
782 bSizer68->Add(m_staticText55, 0, wxTOP|wxRIGHT|wxLEFT, 5);
783
784 m_listCtrl = new wxListCtrl(this, wxID_LISTCTRL, wxDefaultPosition, wxDefaultSize,
wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING);
785 bSizer68->Add(m_listCtrl, 1, wxALL|wxEXPAND, 5);
786
787 wxBoxSizer* bSizer69;
788 bSizer69 = new wxBoxSizer(wxHORIZONTAL);
789
790
791 bSizer69->Add(0, 0, 1, wxEXPAND, 5);
792
793 m_buttonEdit = new wxButton(this, wxID_BUTTONEDIT, wxT("&Edit.."), wxDefaultPosition,
wxDefaultSize, 0);
794 m_buttonEdit->SetMinSize(wxSize(85,25));
795
796 bSizer69->Add(m_buttonEdit, 0, wxALL, 5);
797
798 m_buttonNew = new wxButton(this, wxID_BUTTONNEW, wxT("&New Address.."), wxDefaultPosition,
wxDefaultSize, 0);
799 m_buttonNew->SetMinSize(wxSize(110,25));
800
801 bSizer69->Add(m_buttonNew, 0, wxALL, 5);
802
803 m_buttonDelete = new wxButton(this, wxID_BUTTONDELETE, wxT("&Delete"), wxDefaultPosition,
wxDefaultSize, 0);
804 m_buttonDelete->SetMinSize(wxSize(85,25));
805
806 bSizer69->Add(m_buttonDelete, 0, wxALL, 5);
807
808 m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(-1,-1), 0);
809 m_buttonOK->SetMinSize(wxSize(85,25));
810
811 bSizer69->Add(m_buttonOK, 0, wxALL, 5);
812

```

```

813     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxSize(-1,-1),
814     0);
815     m_buttonCancel->SetMinSize(wxSize(85,25));
816     bSizer69->Add(m_buttonCancel, 0, wxALL, 5);
817
818     bSizer68->Add(bSizer69, 0, wxEXPAND, 5);
819
820     this->SetSizer(bSizer68);
821     this->Layout();
822
823     // Connect Events
824     this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CAddressBookDialogBase::OnClose));
825     m_listCtrl->Connect(wxEVT_COMMAND_LIST_END_LABEL_EDIT,
826     wxListEventHandler(CAddressBookDialogBase::OnListEndLabelEdit), NULL, this);
827     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
828     wxListEventHandler(CAddressBookDialogBase::OnListItemActivated), NULL, this);
829     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_SELECTED,
830     wxListEventHandler(CAddressBookDialogBase::OnListItemSelected), NULL, this);
831     m_buttonEdit->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
832     wxCommandEventHandler(CAddressBookDialogBase::OnButtonEdit), NULL, this);
833     m_buttonNew->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
834     wxCommandEventHandler(CAddressBookDialogBase::OnButtonNew), NULL, this);
835     m_buttonDelete->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
836     wxCommandEventHandler(CAddressBookDialogBase::OnButtonDelete), NULL, this);
837     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
838     wxCommandEventHandler(CAddressBookDialogBase::OnButtonOK), NULL, this);
839     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
840     wxCommandEventHandler(CAddressBookDialogBase::OnButtonCancel), NULL, this);
841 }
842
843 CAddressBookDialogBase::~CAddressBookDialogBase()
844 {
845     // Disconnect Events
846     this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CAddressBookDialogBase::OnClose));
847     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_END_LABEL_EDIT,
848     wxListEventHandler(CAddressBookDialogBase::OnListEndLabelEdit), NULL, this);
849     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
850     wxListEventHandler(CAddressBookDialogBase::OnListItemActivated), NULL, this);
851     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_SELECTED,
852     wxListEventHandler(CAddressBookDialogBase::OnListItemSelected), NULL, this);
853     m_buttonEdit->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
854     wxCommandEventHandler(CAddressBookDialogBase::OnButtonEdit), NULL, this);
855     m_buttonNew->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
856     wxCommandEventHandler(CAddressBookDialogBase::OnButtonNew), NULL, this);
857     m_buttonDelete->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
858     wxCommandEventHandler(CAddressBookDialogBase::OnButtonDelete), NULL, this);
859     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
860     wxCommandEventHandler(CAddressBookDialogBase::OnButtonOK), NULL, this);
861     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
862     wxCommandEventHandler(CAddressBookDialogBase::OnButtonCancel), NULL, this);
863 }
864
865 CProductsDialogBase::CProductsDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
866 wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size, style)
867 {
868     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
869
870     wxBoxSizer* bSizer22;
871     bSizer22 = new wxBoxSizer(wxVERTICAL);
872
873     wxBoxSizer* bSizer23;
874     bSizer23 = new wxBoxSizer(wxHORIZONTAL);
875
876     m_comboBoxCategory = new wxComboBox(this, wxID_ANY, wxT("(Any Category)"), wxDefaultPosition,
877     wxSize(150,-1), 0, NULL, 0);
878     m_comboBoxCategory->Append(wxT("(Any Category)"));
879     bSizer23->Add(m_comboBoxCategory, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
880
881     m_textCtrlSearch = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
882     0);
883     bSizer23->Add(m_textCtrlSearch, 1, wxALL|wxALIGN_CENTER_VERTICAL, 5);

```

```

865
866     m_buttonSearch = new wxButton(this, wxID_ANY, wxT("&Search"), wxDefaultPosition, wxDefaultSize, 0);
867     bSizer23->Add(m_buttonSearch, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
868
869     bSizer22->Add(bSizer23, 0, wxEXPAND|wxTOP|wxBOTTOM|wxRIGHT, 5);
870
871     m_listCtrl = new wxListCtrl(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
wxLC_NO_SORT_HEADER|wxLC_REPORT);
872     bSizer22->Add(m_listCtrl, 1, wxALL|wxEXPAND, 5);
873
874     this->SetSizer(bSizer22);
875     this->Layout();
876
877 // Connect Events
878     m_comboBoxCategory->Connect(wxEVT_COMMAND_COMBOBOX_SELECTED,
wxCommandEventHandler(CProductsDialogBase::OnCombobox), NULL, this);
879     m_textCtrlSearch->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CProductsDialogBase::OnKeyDown), NULL,
this);
880     m_buttonSearch->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CProductsDialogBase::OnButtonSearch), NULL, this);
881     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CProductsDialogBase::OnListItemActivated), NULL, this);
882 }
883
884 CProductsDialogBase::~CProductsDialogBase()
885 {
886 // Disconnect Events
887     m_comboBoxCategory->Disconnect(wxEVT_COMMAND_COMBOBOX_SELECTED,
wxCommandEventHandler(CProductsDialogBase::OnCombobox), NULL, this);
888     m_textCtrlSearch->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CProductsDialogBase::OnKeyDown),
NULL, this);
889     m_buttonSearch->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CProductsDialogBase::OnButtonSearch), NULL, this);
890     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CProductsDialogBase::OnListItemActivated), NULL, this);
891 }
892
893 CEditProductDialogBase::CEditProductDialogBase(wxWindow* parent, wxWindowID id, const wxString& title,
const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
894 {
895     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
896     this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
897
898     wxBoxSizer* bSizer20;
899     bSizer20 = new wxBoxSizer(wxVERTICAL);
900
901     m_scrolledWindow = new wxScrolledWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL);
902     m_scrolledWindow->SetScrollRate(5, 5);
903     m_scrolledWindow->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_WINDOW));
904
905     wxBoxSizer* bSizer21;
906     bSizer21 = new wxBoxSizer(wxVERTICAL);
907
908     wxFlexGridSizer* fgSizer8;
909     fgSizer8 = new wxFlexGridSizer(0, 2, 0, 0);
910     fgSizer8->AddGrowableCol(1);
911     fgSizer8->SetFlexibleDirection(wxBOTH);
912     fgSizer8->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
913
914     m_staticText106 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Category"), wxDefaultPosition,
wxDefaultSize, wxALIGN_RIGHT);
915     m_staticText106->Wrap(-1);
916     fgSizer8->Add(m_staticText106, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
917
918     m_comboBoxCategory = new wxComboBox(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0, NULL, 0);
919     m_comboBoxCategory->SetMinSize(wxSize(180,-1));
920
921     fgSizer8->Add(m_comboBoxCategory, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
922

```

```

923     m_staticText108 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Title"), wxDefaultPosition,
wxDefaultSize, wxALIGN_RIGHT);
924     m_staticText108->Wrap(-1);
925     fgSizer8->Add(m_staticText108, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
926
927     m_textCtrlTitle = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
928     fgSizer8->Add(m_textCtrlTitle, 1, wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND, 5);
929
930     m_staticText107 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Price"), wxDefaultPosition,
wxDefaultSize, wxALIGN_RIGHT);
931     m_staticText107->Wrap(-1);
932     fgSizer8->Add(m_staticText107, 0, wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxLEFT, 5);
933
934     m_textCtrlPrice = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
935     m_textCtrlPrice->SetMinSize(wxSize(105,-1));
936
937     fgSizer8->Add(m_textCtrlPrice, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
938
939     bSizer21->Add(fgSizer8, 0, wxEXPAND|wxTOP|wxRIGHT|wxLEFT, 5);
940
941     m_staticText22 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Page 1: Description"),
wxDefaultPosition, wxDefaultSize, 0);
942     m_staticText22->Wrap(-1);
943     bSizer21->Add(m_staticText22, 0, wxTOP|wxRIGHT|wxLEFT, 5);
944
945     m_textCtrlDescription = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, wxTE_MULTILINE);
946     m_textCtrlDescription->SetMinSize(wxSize(-1,170));
947
948     bSizer21->Add(m_textCtrlDescription, 0, wxALL|wxEXPAND, 5);
949
950     m_staticText23 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Page 2: Order Form"),
wxDefaultPosition, wxDefaultSize, 0);
951     m_staticText23->Wrap(-1);
952     bSizer21->Add(m_staticText23, 0, wxTOP|wxRIGHT|wxLEFT, 5);
953
954     m_textCtrlInstructions = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString,
wxDefaultPosition, wxDefaultSize, wxTE_MULTILINE);
955     m_textCtrlInstructions->SetMinSize(wxSize(-1,120));
956
957     bSizer21->Add(m_textCtrlInstructions, 0, wxEXPAND|wxALL, 5);
958
959     fgSizer5 = new wxFlexGridSizer(0, 3, 0, 0);
960     fgSizer5->AddGrowableCol(1);
961     fgSizer5->SetFlexibleDirection(wxBOTH);
962     fgSizer5->SetNonFlexibleGrowMode(wxFLEX_GROWMODE_SPECIFIED);
963
964     m_staticText24 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Label"), wxDefaultPosition,
wxDefaultSize, 0);
965     m_staticText24->Wrap(-1);
966     fgSizer5->Add(m_staticText24, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT, 5);
967
968     m_staticText25 = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Comma separated list of choices,
or leave blank for text field"), wxDefaultPosition, wxDefaultSize, 0);
969     m_staticText25->Wrap(-1);
970     fgSizer5->Add(m_staticText25, 0, wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT, 5);
971
972
973     fgSizer5->Add(0, 0, 1, wxEXPAND, 5);
974
975     m_textCtrlLabel0 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
976     m_textCtrlLabel0->SetMinSize(wxSize(150,-1));
977
978     fgSizer5->Add(m_textCtrlLabel0, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
979
980     m_textCtrlField0 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
981     fgSizer5->Add(m_textCtrlField0, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
982

```

```

983     m_buttonDel0 = new wxButton(m_scrolledWindow, wxID_DEL0, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
984     fgSizer5->Add(m_buttonDel0, 0, wxRIGHT|wxLEFT|wxALIGN_CENTER_VERTICAL, 5);
985
986     m_textCtrlLabel1 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
987     m_textCtrlLabel1->SetMinSize(wxSize(150,-1));
988
989     fgSizer5->Add(m_textCtrlLabel1, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
990
991     m_textCtrlField1 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
992     fgSizer5->Add(m_textCtrlField1, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
993
994     m_buttonDel1 = new wxButton(m_scrolledWindow, wxID_DEL1, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
995     fgSizer5->Add(m_buttonDel1, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
996
997     m_textCtrlLabel2 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
998     m_textCtrlLabel2->SetMinSize(wxSize(150,-1));
999
1000    fgSizer5->Add(m_textCtrlLabel2, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1001
1002    m_textCtrlField2 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1003    fgSizer5->Add(m_textCtrlField2, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1004
1005    m_buttonDel2 = new wxButton(m_scrolledWindow, wxID_DEL2, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1006    fgSizer5->Add(m_buttonDel2, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1007
1008    m_textCtrlLabel3 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1009    m_textCtrlLabel3->SetMinSize(wxSize(150,-1));
1010
1011    fgSizer5->Add(m_textCtrlLabel3, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1012
1013    m_textCtrlField3 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1014    fgSizer5->Add(m_textCtrlField3, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1015
1016    m_buttonDel3 = new wxButton(m_scrolledWindow, wxID_DEL3, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1017    fgSizer5->Add(m_buttonDel3, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1018
1019    m_textCtrlLabel4 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1020    m_textCtrlLabel4->SetMinSize(wxSize(150,-1));
1021
1022    fgSizer5->Add(m_textCtrlLabel4, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1023
1024    m_textCtrlField4 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1025    fgSizer5->Add(m_textCtrlField4, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1026
1027    m_buttonDel4 = new wxButton(m_scrolledWindow, wxID_DEL4, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1028    fgSizer5->Add(m_buttonDel4, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1029
1030    m_textCtrlLabel5 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1031    m_textCtrlLabel5->SetMinSize(wxSize(150,-1));
1032
1033    fgSizer5->Add(m_textCtrlLabel5, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1034
1035    m_textCtrlField5 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1036    fgSizer5->Add(m_textCtrlField5, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1037
1038    m_buttonDel5 = new wxButton(m_scrolledWindow, wxID_DEL5, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);

```

```

1039     fgSizer5->Add(m_buttonDel5, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1040
1041     m_textCtrlLabel6 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1042     m_textCtrlLabel6->SetMinSize(wxSize(150,-1));
1043
1044     fgSizer5->Add(m_textCtrlLabel6, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1045
1046     m_textCtrlField6 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1047     fgSizer5->Add(m_textCtrlField6, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1048
1049     m_buttonDel6 = new wxButton(m_scrolledWindow, wxID_DEL6, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1050     fgSizer5->Add(m_buttonDel6, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1051
1052     m_textCtrlLabel7 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1053     m_textCtrlLabel7->SetMinSize(wxSize(150,-1));
1054
1055     fgSizer5->Add(m_textCtrlLabel7, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1056
1057     m_textCtrlField7 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1058     fgSizer5->Add(m_textCtrlField7, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1059
1060     m_buttonDel7 = new wxButton(m_scrolledWindow, wxID_DEL7, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1061     fgSizer5->Add(m_buttonDel7, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1062
1063     m_textCtrlLabel8 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1064     m_textCtrlLabel8->SetMinSize(wxSize(150,-1));
1065
1066     fgSizer5->Add(m_textCtrlLabel8, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1067
1068     m_textCtrlField8 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1069     fgSizer5->Add(m_textCtrlField8, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1070
1071     m_buttonDel8 = new wxButton(m_scrolledWindow, wxID_DEL8, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1072     fgSizer5->Add(m_buttonDel8, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1073
1074     m_textCtrlLabel9 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1075     m_textCtrlLabel9->SetMinSize(wxSize(150,-1));
1076
1077     fgSizer5->Add(m_textCtrlLabel9, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1078
1079     m_textCtrlField9 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1080     fgSizer5->Add(m_textCtrlField9, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1081
1082     m_buttonDel9 = new wxButton(m_scrolledWindow, wxID_DEL9, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1083     fgSizer5->Add(m_buttonDel9, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1084
1085     m_textCtrlLabel10 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1086     m_textCtrlLabel10->SetMinSize(wxSize(150,-1));
1087
1088     fgSizer5->Add(m_textCtrlLabel10, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1089
1090     m_textCtrlField10 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1091     fgSizer5->Add(m_textCtrlField10, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1092
1093     m_buttonDel10 = new wxButton(m_scrolledWindow, wxID_DEL10, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1094     fgSizer5->Add(m_buttonDel10, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1095

```



```

1096     m_textCtrlLabel11 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1097     m_textCtrlLabel11->SetMinSize(wxSize(150,-1));
1098
1099     fgSizer5->Add(m_textCtrlLabel11, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1100
1101     m_textCtrlField11 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1102     fgSizer5->Add(m_textCtrlField11, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1103
1104     m_buttonDel11 = new wxButton(m_scrolledWindow, wxID_DEL11, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1105     fgSizer5->Add(m_buttonDel11, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1106
1107     m_textCtrlLabel12 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1108     m_textCtrlLabel12->SetMinSize(wxSize(150,-1));
1109
1110     fgSizer5->Add(m_textCtrlLabel12, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1111
1112     m_textCtrlField12 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1113     fgSizer5->Add(m_textCtrlField12, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1114
1115     m_buttonDel12 = new wxButton(m_scrolledWindow, wxID_DEL12, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1116     fgSizer5->Add(m_buttonDel12, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1117
1118     m_textCtrlLabel13 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1119     m_textCtrlLabel13->SetMinSize(wxSize(150,-1));
1120
1121     fgSizer5->Add(m_textCtrlLabel13, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1122
1123     m_textCtrlField13 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1124     fgSizer5->Add(m_textCtrlField13, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1125
1126     m_buttonDel13 = new wxButton(m_scrolledWindow, wxID_DEL13, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1127     fgSizer5->Add(m_buttonDel13, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1128
1129     m_textCtrlLabel14 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1130     m_textCtrlLabel14->SetMinSize(wxSize(150,-1));
1131
1132     fgSizer5->Add(m_textCtrlLabel14, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1133
1134     m_textCtrlField14 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1135     fgSizer5->Add(m_textCtrlField14, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1136
1137     m_buttonDel14 = new wxButton(m_scrolledWindow, wxID_DEL14, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1138     fgSizer5->Add(m_buttonDel14, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1139
1140     m_textCtrlLabel15 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1141     m_textCtrlLabel15->SetMinSize(wxSize(150,-1));
1142
1143     fgSizer5->Add(m_textCtrlLabel15, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1144
1145     m_textCtrlField15 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1146     fgSizer5->Add(m_textCtrlField15, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1147
1148     m_buttonDel15 = new wxButton(m_scrolledWindow, wxID_DEL15, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1149     fgSizer5->Add(m_buttonDel15, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1150
1151     m_textCtrlLabel16 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);

```

```

1152     m_textCtrlLabel16->SetMinSize(wxSize(150,-1));
1153
1154     fgSizer5->Add(m_textCtrlLabel16, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1155
1156     m_textCtrlField16 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1157     fgSizer5->Add(m_textCtrlField16, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1158
1159     m_buttonDel16 = new wxButton(m_scrolledWindow, wxID_DEL16, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1160     fgSizer5->Add(m_buttonDel16, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1161
1162     m_textCtrlLabel17 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1163     m_textCtrlLabel17->SetMinSize(wxSize(150,-1));
1164
1165     fgSizer5->Add(m_textCtrlLabel17, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1166
1167     m_textCtrlField17 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1168     fgSizer5->Add(m_textCtrlField17, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1169
1170     m_buttonDel17 = new wxButton(m_scrolledWindow, wxID_DEL17, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1171     fgSizer5->Add(m_buttonDel17, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1172
1173     m_textCtrlLabel18 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1174     m_textCtrlLabel18->SetMinSize(wxSize(150,-1));
1175
1176     fgSizer5->Add(m_textCtrlLabel18, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1177
1178     m_textCtrlField18 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1179     fgSizer5->Add(m_textCtrlField18, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1180
1181     m_buttonDel18 = new wxButton(m_scrolledWindow, wxID_DEL18, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1182     fgSizer5->Add(m_buttonDel18, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1183
1184     m_textCtrlLabel19 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1185     m_textCtrlLabel19->SetMinSize(wxSize(150,-1));
1186
1187     fgSizer5->Add(m_textCtrlLabel19, 0, wxALL|wxALIGN_CENTER_VERTICAL, 5);
1188
1189     m_textCtrlField19 = new wxTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxSize(-1,-1), 0);
1190     fgSizer5->Add(m_textCtrlField19, 0, wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL, 5);
1191
1192     m_buttonDel19 = new wxButton(m_scrolledWindow, wxID_DEL19, wxT("Delete"), wxDefaultPosition,
wxSize(60,20), 0);
1193     fgSizer5->Add(m_buttonDel19, 0, wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT, 5);
1194
1195     bSizer21->Add(fgSizer5, 0, wxEXPAND, 5);
1196
1197     wxBoxSizer* bSizer25;
1198     bSizer25 = new wxBoxSizer(wxHORIZONTAL);
1199
1200     m_buttonAddField = new wxButton(m_scrolledWindow, wxID_ANY, wxT("&Add Field"), wxDefaultPosition,
wxDefaultSize, 0);
1201     bSizer25->Add(m_buttonAddField, 0, wxALL, 5);
1202
1203     bSizer21->Add(bSizer25, 0, wxALIGN_CENTER_HORIZONTAL, 5);
1204
1205     m_scrolledWindow->SetSizer(bSizer21);
1206     m_scrolledWindow->Layout();
1207     bSizer21->Fit(m_scrolledWindow);
1208     bSizer20->Add(m_scrolledWindow, 1, wxEXPAND|wxALL, 5);
1209
1210     wxBoxSizer* bSizer26;
1211     bSizer26 = new wxBoxSizer(wxHORIZONTAL);

```

```

1212
1213     m_buttonOK = new wxButton(this, wxID_BUTTONSEND, wxT("&Send"), wxDefaultPosition, wxDefaultSize, 0);
1214     m_buttonOK->SetMinSize(wxSize(85,25));
1215
1216     bSizer26->Add(m_buttonOK, 0, wxALL, 5);
1217
1218     m_buttonPreview = new wxButton(this, wxID_BUTTONPREVIEW, wxT("&Preview"), wxDefaultPosition,
wxDefaultSize, 0);
1219     m_buttonPreview->SetMinSize(wxSize(85,25));
1220
1221     bSizer26->Add(m_buttonPreview, 0, wxALL, 5);
1222
1223     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize,
0);
1224     m_buttonCancel->SetMinSize(wxSize(85,25));
1225
1226     bSizer26->Add(m_buttonCancel, 0, wxALL, 5);
1227
1228     bSizer20->Add(bSizer26, 0, wxALIGN_RIGHT, 5);
1229
1230     this->SetSizer(bSizer20);
1231     this->Layout();
1232
1233     // Connect Events
1234     m_textCtrlTitle->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL,
this);
1235     m_textCtrlPrice->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL,
this);
1236     m_textCtrlDescription->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1237     m_textCtrlInstructions->Connect(wxEVT_KEY_DOWN,
wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1238     m_textCtrlLabel0->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1239     m_textCtrlField0->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1240     m_buttonDel0->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel0), NULL, this);
1241     m_textCtrlLabel1->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1242     m_textCtrlField1->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1243     m_buttonDel1->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel1), NULL, this);
1244     m_textCtrlLabel2->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1245     m_textCtrlField2->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1246     m_buttonDel2->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel2), NULL, this);
1247     m_textCtrlLabel3->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1248     m_textCtrlField3->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1249     m_buttonDel3->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel3), NULL, this);
1250     m_textCtrlLabel4->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1251     m_textCtrlField4->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1252     m_buttonDel4->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel4), NULL, this);
1253     m_textCtrlLabel5->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1254     m_textCtrlField5->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1255     m_buttonDel5->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel5), NULL, this);
1256     m_textCtrlLabel6->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1257     m_textCtrlField6->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);

```

[illegible]

```

1294     m_buttonDel18->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel18), NULL, this);
1295     m_textCtrlLabel19->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1296     m_textCtrlField19->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1297     m_buttonDel19->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel19), NULL, this);
1298     m_buttonAddField->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonAddField), NULL, this);
1299     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonSend), NULL, this);
1300     m_buttonPreview->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonPreview), NULL, this);
1301     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonCancel), NULL, this);
1302 }
1303
1304 CEditProductDialogBase::~CEditProductDialogBase()
1305 {
1306     // Disconnect Events
1307     m_textCtrlTitle->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1308     m_textCtrlPrice->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1309     m_textCtrlDescription->Disconnect(wxEVT_KEY_DOWN,
wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1310     m_textCtrlInstructions->Disconnect(wxEVT_KEY_DOWN,
wxKeyEventHandler(CEditProductDialogBase::OnKeyDown), NULL, this);
1311     m_textCtrlLabel0->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1312     m_textCtrlField0->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1313     m_buttonDel0->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel0), NULL, this);
1314     m_textCtrlLabel1->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1315     m_textCtrlField1->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1316     m_buttonDel1->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel1), NULL, this);
1317     m_textCtrlLabel2->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1318     m_textCtrlField2->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1319     m_buttonDel2->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel2), NULL, this);
1320     m_textCtrlLabel3->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1321     m_textCtrlField3->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1322     m_buttonDel3->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel3), NULL, this);
1323     m_textCtrlLabel4->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1324     m_textCtrlField4->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1325     m_buttonDel4->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel4), NULL, this);
1326     m_textCtrlLabel5->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1327     m_textCtrlField5->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1328     m_buttonDel5->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel5), NULL, this);
1329     m_textCtrlLabel6->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1330     m_textCtrlField6->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
NULL, this);
1331     m_buttonDel6->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CEditProductDialogBase::OnButtonDel6), NULL, this);

```

```

1332     m_textCtrlLabel7->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1333     NULL, this);
1334     m_textCtrlField7->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1335     NULL, this);
1336     m_buttonDel7->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1337     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel7), NULL, this);
1338     m_textCtrlLabel8->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1339     NULL, this);
1340     m_textCtrlField8->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1341     NULL, this);
1342     m_buttonDel8->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1343     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel8), NULL, this);
1344     m_textCtrlLabel9->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1345     NULL, this);
1346     m_textCtrlField9->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1347     NULL, this);
1348     m_buttonDel9->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1349     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel9), NULL, this);
1350     m_textCtrlLabel10->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1351     NULL, this);
1352     m_textCtrlField10->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1353     NULL, this);
1354     m_buttonDel10->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1355     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel10), NULL, this);
1356     m_textCtrlLabel11->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1357     NULL, this);
1358     m_textCtrlField11->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1359     NULL, this);
1360     m_buttonDel11->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1361     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel11), NULL, this);
1362     m_textCtrlLabel12->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1363     NULL, this);
1364     m_textCtrlField12->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1365     NULL, this);
1366     m_buttonDel12->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1367     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel12), NULL, this);
1368     m_textCtrlLabel13->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1369     NULL, this);
1370     m_textCtrlField13->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1371     NULL, this);
1372     m_buttonDel13->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1373     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel13), NULL, this);
1374     m_textCtrlLabel14->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1375     NULL, this);
1376     m_textCtrlField14->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1377     NULL, this);
1378     m_buttonDel14->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1379     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel14), NULL, this);
1380     m_textCtrlLabel15->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1381     NULL, this);
1382     m_textCtrlField15->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1383     NULL, this);
1384     m_buttonDel15->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1385     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel15), NULL, this);
1386     m_textCtrlLabel16->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1387     NULL, this);
1388     m_textCtrlField16->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1389     NULL, this);
1390     m_buttonDel16->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1391     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel16), NULL, this);
1392     m_textCtrlLabel17->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1393     NULL, this);
1394     m_textCtrlField17->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1395     NULL, this);
1396     m_buttonDel17->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1397     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel17), NULL, this);
1398     m_textCtrlLabel18->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1399     NULL, this);
1400     m_textCtrlField18->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditProductDialogBase::OnKeyDown),
1401     NULL, this);
1402     m_buttonDel18->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1403     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel18), NULL, this);

```

```

1368     m_textCtrlLabel19->Disconnect(wxEVT_KEY_DOWN, wxEventHandler(CEditProductDialogBase::OnKeyDown),
1369     NULL, this);
1370     m_textCtrlField19->Disconnect(wxEVT_KEY_DOWN, wxEventHandler(CEditProductDialogBase::OnKeyDown),
1371     NULL, this);
1372     m_buttonDel19->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1373     wxCommandEventHandler(CEditProductDialogBase::OnButtonDel19), NULL, this);
1374     m_buttonAddField->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1375     wxCommandEventHandler(CEditProductDialogBase::OnButtonAddField), NULL, this);
1376     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1377     wxCommandEventHandler(CEditProductDialogBase::OnButtonSend), NULL, this);
1378     m_buttonPreview->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1379     wxCommandEventHandler(CEditProductDialogBase::OnButtonPreview), NULL, this);
1380     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1381     wxCommandEventHandler(CEditProductDialogBase::OnButtonCancel), NULL, this);
1382 }
1383
1384 CViewProductDialogBase::CViewProductDialogBase(wxWindow* parent, wxWindowID id, const wxString& title,
1385     const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1386 {
1387     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1388     this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
1389
1390     wxBoxSizer* bSizer20;
1391     bSizer20 = new wxBoxSizer(wxVERTICAL);
1392
1393     wxBoxSizer* bSizer116;
1394     bSizer116 = new wxBoxSizer(wxHORIZONTAL);
1395
1396     m_htmlWinReviews = new wxHtmlWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
1397     wxHW_SCROLLBAR_AUTO);
1398     m_htmlWinReviews->Hide();
1399
1400     bSizer116->Add(m_htmlWinReviews, 1, wxALL|wxEXPAND, 5);
1401
1402     m_scrolledWindow = new wxScrolledWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
1403     wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL);
1404     m_scrolledWindow->SetScrollRate(5, 5);
1405     m_scrolledWindow->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_WINDOW));
1406
1407     wxBoxSizer* bSizer21;
1408     bSizer21 = new wxBoxSizer(wxVERTICAL);
1409
1410     m_richTextHeading = new wxRichTextCtrl(m_scrolledWindow, wxID_ANY, wxEmptyString, wxDefaultPosition,
1411     wxSize(-1,50), wxTE_READONLY|wxNO_BORDER);
1412     bSizer21->Add(m_richTextHeading, 0, wxEXPAND, 5);
1413
1414     m_staticTextInstructions = new wxStaticText(m_scrolledWindow, wxID_ANY, wxT("Order Form instructions
1415     here\nmultiple lines\n1\n2\n3\n4\n5\n6"), wxDefaultPosition, wxDefaultSize, 0);
1416     m_staticTextInstructions->Wrap(-1);
1417     bSizer21->Add(m_staticTextInstructions, 0, wxALL|wxEXPAND, 5);
1418
1419     wxBoxSizer* bSizer25;
1420     bSizer25 = new wxBoxSizer(wxHORIZONTAL);
1421
1422     m_buttonSubmitForm = new wxButton(m_scrolledWindow, wxID_BUTTONSAMPLE, wxT("&Submit"),
1423     wxDefaultPosition, wxDefaultSize, 0);
1424     bSizer25->Add(m_buttonSubmitForm, 0, wxALL, 5);
1425
1426     m_buttonCancelForm = new wxButton(m_scrolledWindow, wxID_CANCEL2, wxT("Cancel"), wxDefaultPosition,
1427     wxDefaultSize, 0);
1428     bSizer25->Add(m_buttonCancelForm, 0, wxALL, 5);
1429
1430     bSizer21->Add(bSizer25, 0, wxALIGN_CENTER_HORIZONTAL, 5);
1431
1432     m_scrolledWindow->SetSizer(bSizer21);
1433     m_scrolledWindow->Layout();
1434     bSizer21->Fit(m_scrolledWindow);
1435     bSizer116->Add(m_scrolledWindow, 1, wxEXPAND|wxALL, 5);
1436
1437     bSizer20->Add(bSizer116, 1, wxEXPAND, 5);
1438
1439     wxBoxSizer* bSizer26;

```



```

1426     bSizer26 = new wxBoxSizer(wxHORIZONTAL);
1427
1428     m_buttonBack = new wxButton(this, wxID_BUTTONBACK, wxT("< &Back "), wxDefaultPosition,
wxDefaultSize, 0);
1429     m_buttonBack->Enable(false);
1430     m_buttonBack->SetMinSize(wxSize(85,25));
1431
1432     bSizer26->Add(m_buttonBack, 0, wxALL, 5);
1433
1434     m_buttonNext = new wxButton(this, wxID_BUTTONNEXT, wxT(" &Next >"), wxDefaultPosition,
wxDefaultSize, 0);
1435     m_buttonNext->SetMinSize(wxSize(85,25));
1436
1437     bSizer26->Add(m_buttonNext, 0, wxALL, 5);
1438
1439     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize,
0);
1440     m_buttonCancel->SetMinSize(wxSize(85,25));
1441
1442     bSizer26->Add(m_buttonCancel, 0, wxALL, 5);
1443
1444     bSizer20->Add(bSizer26, 0, wxALIGN_RIGHT, 5);
1445
1446     this->SetSizer(bSizer20);
1447     this->Layout();
1448
1449     // Connect Events
1450     m_buttonSubmitForm->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonSubmitForm), NULL, this);
1451     m_buttonCancelForm->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonCancelForm), NULL, this);
1452     m_buttonBack->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonBack), NULL, this);
1453     m_buttonNext->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonNext), NULL, this);
1454     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonCancel), NULL, this);
1455 }
1456
1457 CViewProductDialogBase::~CViewProductDialogBase()
1458 {
1459     // Disconnect Events
1460     m_buttonSubmitForm->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonSubmitForm), NULL, this);
1461     m_buttonCancelForm->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonCancelForm), NULL, this);
1462     m_buttonBack->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonBack), NULL, this);
1463     m_buttonNext->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonNext), NULL, this);
1464     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CViewProductDialogBase::OnButtonCancel), NULL, this);
1465 }
1466
1467 CViewOrderDialogBase::CViewOrderDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1468 {
1469     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1470     this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
1471
1472     wxBoxSizer* bSizer20;
1473     bSizer20 = new wxBoxSizer(wxVERTICAL);
1474
1475     wxBoxSizer* bSizer116;
1476     bSizer116 = new wxBoxSizer(wxHORIZONTAL);
1477
1478     m_htmlWin = new wxHtmlWindow(this, wxID_ANY, wxDefaultPosition, wxDefaultSize, wxHW_SCROLLBAR_AUTO);
1479     bSizer116->Add(m_htmlWin, 1, wxALL|wxEXPAND, 5);
1480
1481     bSizer20->Add(bSizer116, 1, wxEXPAND, 5);
1482
1483     wxBoxSizer* bSizer26;

```



```

1484     bSizer26 = new wxBoxSizer(wxHORIZONTAL);
1485
1486     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxDefaultSize, 0);
1487     m_buttonOK->SetMinSize(wxSize(85,25));
1488
1489     bSizer26->Add(m_buttonOK, 0, wxALL, 5);
1490
1491     bSizer20->Add(bSizer26, 0, wxALIGN_RIGHT, 5);
1492
1493     this->SetSizer(bSizer20);
1494     this->Layout();
1495
1496     // Connect Events
1497     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1498         wxCommandEventHandler(CViewOrderDialogBase::OnButtonOK), NULL, this);
1499 }
1500 CViewOrderDialogBase::~CViewOrderDialogBase()
1501 {
1502     // Disconnect Events
1503     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1504         wxCommandEventHandler(CViewOrderDialogBase::OnButtonOK), NULL, this);
1505 }
1506 CEditReviewDialogBase::CEditReviewDialogBase(wxWindow* parent, wxWindowID id, const wxString& title,
1507     const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1508 {
1509     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1510     this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_MENU));
1511
1512     wxBoxSizer* bSizer112;
1513     bSizer112 = new wxBoxSizer(wxVERTICAL);
1514
1515     bSizer112->Add(0, 3, 0, 0, 5);
1516
1517     m_staticTextSeller = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition,
1518         wxDefaultSize, 0);
1519     m_staticTextSeller->Wrap(-1);
1520     bSizer112->Add(m_staticTextSeller, 0, wxALL|wxEXPAND, 5);
1521
1522     bSizer112->Add(0, 3, 0, 0, 5);
1523
1524     m_staticText110 = new wxStaticText(this, wxID_ANY, wxT("Rating"), wxDefaultPosition, wxDefaultSize,
1525         0);
1526     m_staticText110->Wrap(-1);
1527     bSizer112->Add(m_staticText110, 0, wxTOP|wxRIGHT|wxLEFT, 5);
1528
1529     wxString m_choiceStarsChoices[] = { wxT(" 1 star"), wxT(" 2 stars"), wxT(" 3 stars"), wxT(" 4
1530         stars"), wxT(" 5 stars") };
1531     int m_choiceStarsNChoices = sizeof(m_choiceStarsChoices) / sizeof(wxString);
1532     m_choiceStars = new wxChoice(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
1533         m_choiceStarsNChoices, m_choiceStarsChoices, 0);
1534     m_choiceStars->SetSelection(0);
1535     bSizer112->Add(m_choiceStars, 0, wxALL, 5);
1536
1537     m_staticText43 = new wxStaticText(this, wxID_ANY, wxT("Review"), wxDefaultPosition, wxDefaultSize,
1538         0);
1539     m_staticText43->Wrap(-1);
1540     bSizer112->Add(m_staticText43, 0, wxTOP|wxRIGHT|wxLEFT, 5);
1541
1542     m_textCtrlReview = new wxTextCtrl(this, wxID_ANY, wxEmptyString, wxDefaultPosition, wxDefaultSize,
1543         wxTE_MULTILINE);
1544     bSizer112->Add(m_textCtrlReview, 1, wxALL|wxEXPAND, 5);
1545
1546     wxBoxSizer* bSizer113;
1547     bSizer113 = new wxBoxSizer(wxHORIZONTAL);
1548
1549     m_buttonSubmit = new wxButton(this, wxID_SUBMIT, wxT("&Submit"), wxDefaultPosition, wxDefaultSize,
1550         0);
1551     m_buttonSubmit->SetMinSize(wxSize(85,25));

```

```

1546
1547     bSizer113->Add(m_buttonSubmit, 0, wxALL, 5);
1548
1549     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize,
1550     0);
1551     m_buttonCancel->SetMinSize(wxSize(85,25));
1552
1553     bSizer113->Add(m_buttonCancel, 0, wxALL, 5);
1554
1555     bSizer112->Add(bSizer113, 0, wxALIGN_RIGHT, 5);
1556
1557     this->SetSizer(bSizer112);
1558     this->Layout();
1559
1560     // Connect Events
1561     m_textCtrlReview->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditReviewDialogBase::OnKeyDown), NULL,
1562     this);
1563     m_buttonSubmit->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1564     wxCommandEventHandler(CEditReviewDialogBase::OnButtonSubmit), NULL, this);
1565     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1566     wxCommandEventHandler(CEditReviewDialogBase::OnButtonCancel), NULL, this);
1567 }
1568
1569 CEditReviewDialogBase::~CEditReviewDialogBase()
1570 {
1571     // Disconnect Events
1572     m_textCtrlReview->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CEditReviewDialogBase::OnKeyDown),
1573     NULL, this);
1574     m_buttonSubmit->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1575     wxCommandEventHandler(CEditReviewDialogBase::OnButtonSubmit), NULL, this);
1576     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1577     wxCommandEventHandler(CEditReviewDialogBase::OnButtonCancel), NULL, this);
1578 }
1579
1580 CPokerLobbyDialogBase::CPokerLobbyDialogBase(wxWindow* parent, wxWindowID id, const wxString& title,
1581 const wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1582 {
1583     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1584     this->SetBackgroundColour(wxSystemSettings::GetColour(wxSYS_COLOUR_BTNFACE));
1585
1586     wxBoxSizer* bSizer156;
1587     bSizer156 = new wxBoxSizer(wxHORIZONTAL);
1588
1589     m_treeCtrl = new wxTreeCtrl(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
1590 wxTR_HAS_BUTTONS|wxTR_HIDE_ROOT|wxTR_LINES_AT_ROOT);
1591     m_treeCtrl->SetMinSize(wxSize(130,-1));
1592
1593     bSizer156->Add(m_treeCtrl, 0, wxEXPAND|wxTOP|wxBOTTOM|wxLEFT, 5);
1594
1595     wxBoxSizer* bSizer172;
1596     bSizer172 = new wxBoxSizer(wxVERTICAL);
1597
1598     m_listCtrl = new wxListCtrl(this, wxID_ANY, wxDefaultPosition, wxDefaultSize,
1599 wxLC_NO_SORT_HEADER|wxLC_REPORT);
1600     bSizer172->Add(m_listCtrl, 1, wxEXPAND|wxALL, 5);
1601
1602     m_buttonNewTable = new wxButton(this, wxID_OPENNEWTABLE, wxT("&Open New Table"), wxDefaultPosition,
1603     wxDefaultSize, 0);
1604     bSizer172->Add(m_buttonNewTable, 0, wxALL, 5);
1605
1606     bSizer156->Add(bSizer172, 1, wxEXPAND, 5);
1607
1608     this->SetSizer(bSizer156);
1609     this->Layout();
1610
1611     // Connect Events
1612     m_treeCtrl->Connect(wxEVT_COMMAND_TREE_SEL_CHANGED,
1613     wxTreeEventHandler(CPokerLobbyDialogBase::OnTreeSelChanged), NULL, this);
1614     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
1615     wxListEventHandler(CPokerLobbyDialogBase::OnListItemActivated), NULL, this);
1616     m_listCtrl->Connect(wxEVT_COMMAND_LIST_ITEM_SELECTED,
1617     wxListEventHandler(CPokerLobbyDialogBase::OnListItemSelected), NULL, this);

```

```

1604     m_buttonNewTable->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CPokerLobbyDialogBase::OnButtonNewTable), NULL, this);
1605 }
1606
1607 CPokerLobbyDialogBase::~CPokerLobbyDialogBase()
1608 {
1609     // Disconnect Events
1610     m_treeCtrl->Disconnect(wxEVT_COMMAND_TREE_SEL_CHANGED,
wxTreeEventHandler(CPokerLobbyDialogBase::OnTreeSelChanged), NULL, this);
1611     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_ACTIVATED,
wxListEventHandler(CPokerLobbyDialogBase::OnListItemActivated), NULL, this);
1612     m_listCtrl->Disconnect(wxEVT_COMMAND_LIST_ITEM_SELECTED,
wxListEventHandler(CPokerLobbyDialogBase::OnListItemSelected), NULL, this);
1613     m_buttonNewTable->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CPokerLobbyDialogBase::OnButtonNewTable), NULL, this);
1614 }
1615
1616 CPokerDialogBase::CPokerDialogBase(wxWindow* parent, wxWindowID id, const wxString& title, const
wxPoint& pos, const wxSize& size, long style) : wxFrame(parent, id, title, pos, size, style)
1617 {
1618     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1619
1620     wxBoxSizer* bSizer174;
1621     bSizer174 = new wxBoxSizer(wxVERTICAL);
1622
1623     m_checkSitOut = new wxCheckBox(this, wxID_ANY, wxT("Deal Me Out"), wxDefaultPosition, wxDefaultSize,
0);
1624
1625     bSizer174->Add(m_checkSitOut, 0, wxALL, 5);
1626
1627     m_buttonDealHand = new wxButton(this, wxID_DEALHAND, wxT("&Deal Hand"), wxDefaultPosition,
wxSize(150,25), 0);
1628     bSizer174->Add(m_buttonDealHand, 0, wxALL, 5);
1629
1630     m_buttonFold = new wxButton(this, wxID_FOLD, wxT("&Fold"), wxDefaultPosition, wxSize(80,25), 0);
1631     bSizer174->Add(m_buttonFold, 0, wxALL, 5);
1632
1633     m_buttonCall = new wxButton(this, wxID_CALL, wxT("&Call"), wxDefaultPosition, wxSize(80,25), 0);
1634     bSizer174->Add(m_buttonCall, 0, wxALL, 5);
1635
1636     m_buttonRaise = new wxButton(this, wxID_RAISE, wxT("&Raise"), wxDefaultPosition, wxSize(80,25), 0);
1637     bSizer174->Add(m_buttonRaise, 0, wxALL, 5);
1638
1639     m_buttonLeaveTable = new wxButton(this, wxID_LEAVE_TABLE, wxT("&Leave Table"), wxDefaultPosition,
wxSize(90,25), 0);
1640     bSizer174->Add(m_buttonLeaveTable, 0, wxALL, 5);
1641
1642     m_textDitchPlayer = new wxTextCtrl(this, wxID_DITCHPLAYER, wxEmptyString, wxDefaultPosition,
wxSize(45,-1), wxTE_PROCESS_ENTER);
1643     bSizer174->Add(m_textDitchPlayer, 0, wxALL, 5);
1644
1645     m_checkPreFold = new wxCheckBox(this, wxID_ANY, wxT("FOLD"), wxDefaultPosition, wxSize(100,-1), 0);
1646
1647     bSizer174->Add(m_checkPreFold, 0, wxALL, 5);
1648
1649     m_checkPreCall = new wxCheckBox(this, wxID_ANY, wxT("CALL"), wxDefaultPosition, wxSize(100,-1), 0);
1650
1651     bSizer174->Add(m_checkPreCall, 0, wxALL, 5);
1652
1653     m_checkPreCallAny = new wxCheckBox(this, wxID_ANY, wxT("CALL ANY"), wxDefaultPosition,
wxSize(100,-1), 0);
1654
1655     bSizer174->Add(m_checkPreCallAny, 0, wxALL, 5);
1656
1657     m_checkPreRaise = new wxCheckBox(this, wxID_ANY, wxT("RAISE"), wxDefaultPosition, wxSize(100,-1),
0);
1658
1659     bSizer174->Add(m_checkPreRaise, 0, wxALL, 5);
1660
1661     m_checkPreRaiseAny = new wxCheckBox(this, wxID_ANY, wxT("RAISE ANY"), wxDefaultPosition,
wxSize(100,-1), 0);
1662

```

```

1663     bSizer174->Add(m_checkPreRaiseAny, 0, wxALL, 5);
1664
1665     this->SetSizer(bSizer174);
1666     this->Layout();
1667     m_statusBar = this->CreateStatusBar(1, wxST_SIZEGRIP, wxID_ANY);
1668
1669     // Connect Events
1670     this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CPokerDialogBase::OnClose));
1671     this->Connect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1672     this->Connect(wxEVT_LEFT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1673     this->Connect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1674     this->Connect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1675     this->Connect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1676     this->Connect(wxEVT_RIGHT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1677     this->Connect(wxEVT_MOTION, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1678     this->Connect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1679     this->Connect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1680     this->Connect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1681     this->Connect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1682     this->Connect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1683     this->Connect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1684     this->Connect(wxEVT_PAINT, wxPaintEventHandler(CPokerDialogBase::OnPaint));
1685     this->Connect(wxEVT_SIZE, wxSizeEventHandler(CPokerDialogBase::OnSize));
1686     m_checkSitOut->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1687         wxCommandEventHandler(CPokerDialogBase::OnCheckSitOut), NULL, this);
1687     m_buttonDealHand->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1688         wxCommandEventHandler(CPokerDialogBase::OnButtonDealHand), NULL, this);
1688     m_buttonFold->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1689         wxCommandEventHandler(CPokerDialogBase::OnButtonFold), NULL, this);
1689     m_buttonCall->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1690         wxCommandEventHandler(CPokerDialogBase::OnButtonCall), NULL, this);
1690     m_buttonRaise->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1691         wxCommandEventHandler(CPokerDialogBase::OnButtonRaise), NULL, this);
1691     m_buttonLeaveTable->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1692         wxCommandEventHandler(CPokerDialogBase::OnButtonLeaveTable), NULL, this);
1692     m_textDitchPlayer->Connect(wxEVT_COMMAND_TEXT_ENTER,
1693         wxCommandEventHandler(CPokerDialogBase::OnDitchPlayer), NULL, this);
1693     m_checkPreFold->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1694         wxCommandEventHandler(CPokerDialogBase::OnCheckPreFold), NULL, this);
1694     m_checkPreCall->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1695         wxCommandEventHandler(CPokerDialogBase::OnCheckPreCall), NULL, this);
1695     m_checkPreCallAny->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1696         wxCommandEventHandler(CPokerDialogBase::OnCheckPreCallAny), NULL, this);
1696     m_checkPreRaise->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1697         wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaise), NULL, this);
1697     m_checkPreRaiseAny->Connect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1698         wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaiseAny), NULL, this);
1698 }
1699
1700 CPokerDialogBase::~CPokerDialogBase()
1701 {
1702     // Disconnect Events
1703     this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CPokerDialogBase::OnClose));
1704     this->Disconnect(wxEVT_LEFT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1705     this->Disconnect(wxEVT_LEFT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1706     this->Disconnect(wxEVT_MIDDLE_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1707     this->Disconnect(wxEVT_MIDDLE_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1708     this->Disconnect(wxEVT_RIGHT_DOWN, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1709     this->Disconnect(wxEVT_RIGHT_UP, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1710     this->Disconnect(wxEVT_MOTION, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1711     this->Disconnect(wxEVT_LEFT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1712     this->Disconnect(wxEVT_MIDDLE_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1713     this->Disconnect(wxEVT_RIGHT_DCLICK, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1714     this->Disconnect(wxEVT_LEAVE_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1715     this->Disconnect(wxEVT_ENTER_WINDOW, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1716     this->Disconnect(wxEVT_MOUSEWHEEL, wxMouseEventHandler(CPokerDialogBase::OnMouseEvents));
1717     this->Disconnect(wxEVT_PAINT, wxPaintEventHandler(CPokerDialogBase::OnPaint));
1718     this->Disconnect(wxEVT_SIZE, wxSizeEventHandler(CPokerDialogBase::OnSize));
1719     m_checkSitOut->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED,
1720         wxCommandEventHandler(CPokerDialogBase::OnCheckSitOut), NULL, this);
1720     m_buttonDealHand->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1721         wxCommandEventHandler(CPokerDialogBase::OnButtonDealHand), NULL, this);

```

```

1721     m_buttonFold->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnButtonFold), NULL, this);
1722     m_buttonCall->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnButtonCall), NULL, this);
1723     m_buttonRaise->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnButtonRaise), NULL, this);
1724     m_buttonLeaveTable->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnButtonLeaveTable), NULL, this);
1725     m_textDitchPlayer->Disconnect(wxEVT_COMMAND_TEXT_ENTER,
wxCommandEventHandler(CPokerDialogBase::OnDitchPlayer), NULL, this);
1726     m_checkPreFold->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnCheckPreFold), NULL, this);
1727     m_checkPreCall->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnCheckPreCall), NULL, this);
1728     m_checkPreCallAny->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnCheckPreCallAny), NULL, this);
1729     m_checkPreRaise->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaise), NULL, this);
1730     m_checkPreRaiseAny->Disconnect(wxEVT_COMMAND_CHECKBOX_CLICKED,
wxCommandEventHandler(CPokerDialogBase::OnCheckPreRaiseAny), NULL, this);
1731 }
1732
1733 CGetTextFromUserDialogBase::CGetTextFromUserDialogBase(wxWindow* parent, wxWindowID id, const wxString&
title, const wxPoint& pos, const wxSize& size, long style) : wxDialog(parent, id, title, pos, size,
style)
1734 {
1735     this->SetSizeHints(wxDefaultSize, wxDefaultSize);
1736
1737     wxBoxSizer* bSizer79;
1738     bSizer79 = new wxBoxSizer(wxVERTICAL);
1739
1740     wxBoxSizer* bSizer81;
1741     bSizer81 = new wxBoxSizer(wxVERTICAL);
1742
1743     bSizer81->Add(0, 0, 1, wxEXPAND, 5);
1744
1745     m_staticTextMessage1 = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1746     m_staticTextMessage1->Wrap(-1);
1747     bSizer81->Add(m_staticTextMessage1, 0, wxTOP|wxRIGHT|wxLEFT, 5);
1748
1749     m_textCtrl1 = new wxTextCtrl(this, wxID_TEXTCTRL, wxEmptyString, wxDefaultPosition, wxDefaultSize,
wxTE_PROCESS_ENTER);
1750     bSizer81->Add(m_textCtrl1, 0, wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL, 5);
1751
1752     m_staticTextMessage2 = new wxStaticText(this, wxID_ANY, wxEmptyString, wxDefaultPosition,
wxDefaultSize, 0);
1753     m_staticTextMessage2->Wrap(-1);
1754     m_staticTextMessage2->Hide();
1755
1756     bSizer81->Add(m_staticTextMessage2, 0, wxTOP|wxRIGHT|wxLEFT, 5);
1757
1758     m_textCtrl2 = new wxTextCtrl(this, wxID_TEXTCTRL, wxEmptyString, wxDefaultPosition, wxDefaultSize,
wxTE_PROCESS_ENTER);
1759     m_textCtrl2->Hide();
1760
1761     bSizer81->Add(m_textCtrl2, 0, wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL, 5);
1762
1763     bSizer81->Add(0, 0, 1, wxEXPAND, 5);
1764
1765     bSizer79->Add(bSizer81, 1, wxEXPAND|wxALL, 10);
1766
1767     wxBoxSizer* bSizer80;
1768     bSizer80 = new wxBoxSizer(wxHORIZONTAL);
1769
1770     bSizer80->Add(0, 0, 1, wxEXPAND, 5);
1771
1772     m_buttonOK = new wxButton(this, wxID_OK, wxT("OK"), wxDefaultPosition, wxSize(-1,-1), 0);
1773     m_buttonOK->SetMinSize(wxSize(85,25));

```

```

1777
1778     bSizer80->Add(m_buttonOK, 0, wxALL, 5);
1779
1780     m_buttonCancel = new wxButton(this, wxID_CANCEL, wxT("Cancel"), wxDefaultPosition, wxDefaultSize,
1781     0);
1782     m_buttonCancel->SetMinSize(wxSize(85,25));
1783     bSizer80->Add(m_buttonCancel, 0, wxALL, 5);
1784
1785     bSizer79->Add(bSizer80, 0, wxEXPAND, 5);
1786
1787     this->SetSizer(bSizer79);
1788     this->Layout();
1789
1790     // Connect Events
1791     this->Connect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CGetTextFromUserDialogBase::OnClose));
1792     m_textCtrl1->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown), NULL,
1793     this);
1794     m_textCtrl2->Connect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown), NULL,
1795     this);
1796     m_buttonOK->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1797     wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonOK), NULL, this);
1798     m_buttonCancel->Connect(wxEVT_COMMAND_BUTTON_CLICKED,
1799     wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonCancel), NULL, this);
1800 }
1801
1802 CGetTextFromUserDialogBase::~CGetTextFromUserDialogBase()
1803 {
1804     // Disconnect Events
1805     this->Disconnect(wxEVT_CLOSE_WINDOW, wxCloseEventHandler(CGetTextFromUserDialogBase::OnClose));
1806     m_textCtrl1->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown),
1807     NULL, this);
1808     m_textCtrl2->Disconnect(wxEVT_KEY_DOWN, wxKeyEventHandler(CGetTextFromUserDialogBase::OnKeyDown),
1809     NULL, this);
1810     m_buttonOK->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1811     wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonOK), NULL, this);
1812     m_buttonCancel->Disconnect(wxEVT_COMMAND_BUTTON_CLICKED,
1813     wxCommandEventHandler(CGetTextFromUserDialogBase::OnButtonCancel), NULL, this);
1814 }
1815

```

SUMMARY

Satoshi used wxFormBuilder to visually design the interface, which then generated this code automatically.

uibase.h

721 lines CPP

GUI BLUEPRINT

Header file for the auto-generated UI base classes. Declares the window classes and their components that are implemented in uibase.cpp.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 ///////////////////////////////////////////////////////////////////
6 // C++ code generated with wxFormBuilder (version Apr 16 2008)
7 // http://www.wxformbuilder.org/
8 //
9 // PLEASE DO "NOT" EDIT THIS FILE!
10 ///////////////////////////////////////////////////////////////////
11
12 #ifndef __uibase__
13 #define __uibase__
14
15 #include <wx/string.h>

```

wxWidgets includes for GUI components

```

16 #include <wx/bitmap.h>
17 #include <wx/image.h>
18 #include <wx/icon.h>
19 #include <wx/menu.h>
20 #include <wx/gdicmn.h>
21 #include <wx/font.h>
22 #include <wx/colour.h>
23 #include <wx/settings.h>
24 #include <wx/toolbar.h>
25 #include <wx/statusbr.h>
26 #include <wx/stattext.h>
27 #include <wx/textctrl.h>
28 #include <wx/button.h>
29 #include <wx/sizer.h>
30 #include <wx/panel.h>

```

CMainFrameBase class declaration

```

31 #include <wx/choice.h>
32 #include <wx/listctrl.h>
33 #include <wx/notebook.h>
34 #include <wx/frame.h>
35 #include <wx/html/htmlwin.h>
36 #include <wx/dialog.h>
37 #include <wx/statbmp.h>
38 #include <wx/combobox.h>
39 #include <wx/scrolwin.h>
40 #include <wx/richtext/richtextctrl.h>
41 #include <wx/treectrl.h>
42 #include <wx/checkbox.h>
43
44 ///////////////////////////////////////////////////////////////////
45
46 #define wxID_MAINFRAME 1000
47 #define wxID_OPTIONSGENERATEBITCOINS 1001
48 #define wxID_BUTTONSEND 1002
49 #define wxID_BUTTONRECEIVE 1003
50 #define wxID_TEXTCTRLADDRESS 1004
51 #define wxID_BUTTONCOPY 1005
52 #define wxID_BUTTONCHANGE 1006
53 #define wxID_TRANSACTIONFEE 1007
54 #define wxID_TEXTCTRLPAYTO 1008
55 #define wxID_BUTTONPASTE 1009
56 #define wxID_BUTTONADDRESSBOOK 1010
57 #define wxID_TEXTCTRLAMOUNT 1011
58 #define wxID_CHOICETRANSFERTYPE 1012
59 #define wxID_LISTCTRL 1013
60 #define wxID_BUTTONRENAME 1014
61 #define wxID_BUTTONNEW 1015

```



```

62 #define wxID_BUTTONEDIT 1016
63 #define wxID_BUTTONDELETE 1017
64 #define wxID_DEL0 1018
65 #define wxID_DEL1 1019
66 #define wxID_DEL2 1020
67 #define wxID_DEL3 1021
68 #define wxID_DEL4 1022
69 #define wxID_DEL5 1023
70 #define wxID_DEL6 1024
71 #define wxID_DEL7 1025
72 #define wxID_DEL8 1026
73 #define wxID_DEL9 1027
74 #define wxID_DEL10 1028
75 #define wxID_DEL11 1029
76 #define wxID_DEL12 1030
77 #define wxID_DEL13 1031
78 #define wxID_DEL14 1032
79 #define wxID_DEL15 1033
80 #define wxID_DEL16 1034
81 #define wxID_DEL17 1035
82 #define wxID_DEL18 1036
83 #define wxID_DEL19 1037
84 #define wxID_BUTTONPREVIEW 1038
85 #define wxID_BUTTONSAMPLE 1039
86 #define wxID_CANCEL2 1040
87 #define wxID_BUTTONBACK 1041
88 #define wxID_BUTTONNEXT 1042
89 #define wxID_SUBMIT 1043
90 #define wxID_OPENNEWTABLE 1044
91 #define wxID_DEALHAND 1045
92 #define wxID_FOLD 1046
93 #define wxID_CALL 1047
94 #define wxID_RAISE 1048
95 #define wxID_LEAVE_TABLE 1049
96 #define wxID_DITCHPLAYER 1050
97 #define wxID_TEXTCTRL 1051
98
99 ///////////////////////////////////////////////////////////////////
100 /// Class CMainFrameBase

```

Dialog class declarations

```

101 ///////////////////////////////////////////////////////////////////
102 class CMainFrameBase : public wxFrame
103 {
104 private:
105
106 protected:
107     wxMenuBar* m_menubar;
108     wxMenu* m_menuFile;
109     wxMenu* m_menuHelp;
110     wxToolBar* m_toolBar;
111     wxStatusBar* m_statusBar;
112
113     wxStaticText* m_staticText32;
114     wxTextCtrl* m_textCtrlAddress;
115     wxButton* m_buttonCopy;
116     wxButton* m_button91;
117
118     wxPanel* m_panel14;
119     wxStaticText* m_staticText41;
120     wxStaticText* m_staticTextBalance;
121
122     wxChoice* m_choiceFilter;
123     wxNotebook* m_notebook;
124     wxPanel* m_panel7;
125     wxPanel* m_panel9;
126     wxPanel* m_panel8;
127     wxPanel* m_panel10;

```

```

128     wxPanel* m_panel11;
129
130 // Virtual event handlers, override them in your derived class
131 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
132 virtual void OnIdle(wxIdleEvent& event){ event.Skip(); }
133 virtual void OnMouseEvents(wxMouseEvent& event){ event.Skip(); }
134 virtual void OnPaint(wxPaintEvent& event){ event.Skip(); }
135 virtual void OnMenuFileExit(wxCommandEvent& event){ event.Skip(); }
136 virtual void OnMenuOptionsGenerate(wxCommandEvent& event){ event.Skip(); }
137 virtual void OnMenuOptionsOptions(wxCommandEvent& event){ event.Skip(); }
138 virtual void OnMenuHelpAbout(wxCommandEvent& event){ event.Skip(); }
139 virtual void OnButtonSend(wxCommandEvent& event){ event.Skip(); }
140 virtual void OnButtonAddressBook(wxCommandEvent& event){ event.Skip(); }
141 virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
142 virtual void OnMouseEventAddress(wxMouseEvent& event){ event.Skip(); }
143 virtual void OnSetFocusAddress(wxFocusEvent& event){ event.Skip(); }
144 virtual void OnButtonCopy(wxCommandEvent& event){ event.Skip(); }
145 virtual void OnButtonChange(wxCommandEvent& event){ event.Skip(); }
146 virtual void OnListColBeginDrag(wxListEvent& event){ event.Skip(); }
147 virtual void OnListItemActivatedAllTransactions(wxListEvent& event){ event.Skip(); }
148 virtual void OnPaintListCtrl(wxPaintEvent& event){ event.Skip(); }
149 virtual void OnListItemActivatedOrdersSent(wxListEvent& event){ event.Skip(); }
150 virtual void OnListItemActivatedProductsSent(wxListEvent& event){ event.Skip(); }
151 virtual void OnListItemActivatedOrdersReceived(wxListEvent& event){ event.Skip(); }
152
153
154 public:
155     wxMenu* m_menuOptions;
156     wxListCtrl* m_listCtrl;
157     wxListCtrl* m_listCtrlEscrows;
158     wxListCtrl* m_listCtrlOrdersSent;
159     wxListCtrl* m_listCtrlProductsSent;
160     wxListCtrl* m_listCtrlOrdersReceived;
161 CMainFrameBase(wxWindow* parent, wxWindowID id = wxID_MAINFRAME, const wxString& title = wxT("Bitcoin"),
const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(705,484), long style =
wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
162 ~CMainFrameBase();
163
164 };
165
166 ///////////////////////////////////////////////////////////////////
167 /// Class CTxDetailsDialogBase
168 ///////////////////////////////////////////////////////////////////
169 class CTxDetailsDialogBase : public wxDialog
170 {
171 private:
172
173 protected:
174     wxHtmlWindow* m_htmlWin;
175     wxButton* m_buttonOK;
176
177 // Virtual event handlers, override them in your derived class
178 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
179
180
181 public:
182 CTxDetailsDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title =
wxT("Transaction Details"), const wxPoint& pos = wxDefaultPosition, const wxSize& size =
wxSize(620,450), long style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
183 ~CTxDetailsDialogBase();
184
185 };
186
187 ///////////////////////////////////////////////////////////////////
188 /// Class COptionsDialogBase
189 ///////////////////////////////////////////////////////////////////
190 class COptionsDialogBase : public wxDialog
191 {
192 private:
193
194 protected:
195

```

```

196     wxStaticText* m_staticText32;
197     wxStaticText* m_staticText31;
198     wxTextCtrl* m_textCtrlTransactionFee;
199     wxButton* m_buttonOK;
200     wxButton* m_buttonCancel;
201
202 // Virtual event handlers, override them in your derived class
203 virtual void OnKillFocusTransactionFee(wxFocusEvent& event){ event.Skip(); }
204 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
205 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
206
207
208 public:
209 COptionsDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Options"),
    const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(500,261), long style =
    wxDEFAULT_DIALOG_STYLE);
210 ~COptionsDialogBase();
211
212 };
213
214 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
215 /// Class CAboutDialogBase
216 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
217 class CAboutDialogBase : public wxDialog
218 {
219 private:
220
221 protected:
222
223
224     wxStaticText* m_staticText40;
225
226     wxStaticText* m_staticTextMain;
227
228
229     wxButton* m_buttonOK;
230
231 // Virtual event handlers, override them in your derived class
232 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
233
234
235 public:
236     wxStaticText* m_staticTextVersion;
237 CAboutDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("About
    Bitcoin"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(507,298), long style =
    wxDEFAULT_DIALOG_STYLE);
238 ~CAboutDialogBase();
239
240 };
241
242 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
243 /// Class CSendDialogBase
244 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
245 class CSendDialogBase : public wxDialog
246 {
247 private:
248
249 protected:
250
251
252     wxStaticText* m_staticText14;
253
254     wxStaticBitmap* m_bitmapCheckMark;
255     wxStaticText* m_staticText36;
256     wxTextCtrl* m_textCtrlAddress;
257     wxButton* m_buttonPaste;
258     wxButton* m_buttonAddress;
259     wxStaticText* m_staticText19;
260     wxTextCtrl* m_textCtrlAmount;
261     wxStaticText* m_staticText20;
262     wxChoice* m_choiceTransferType;
263     wxStaticText* m_staticTextFrom;

```

```

264     wxTextCtrl* m_textCtrlFrom;
265     wxStaticText* m_staticTextMessage;
266     wxTextCtrl* m_textCtrlMessage;
267
268     wxButton* m_buttonSend;
269     wxButton* m_buttonCancel;
270
271 // Virtual event handlers, override them in your derived class
272 virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
273 virtual void OnTextAddress(wxCommandEvent& event){ event.Skip(); }
274 virtual void OnButtonPaste(wxCommandEvent& event){ event.Skip(); }
275 virtual void OnButtonAddressBook(wxCommandEvent& event){ event.Skip(); }
276 virtual void OnKillFocusAmount(wxFocusEvent& event){ event.Skip(); }
277 virtual void OnButtonSend(wxCommandEvent& event){ event.Skip(); }
278 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
279
280
281 public:
282 CSendDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Send Coins"),
283     const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(675,338), long style =
284     wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
285 ~CSendDialogBase();
286
287 };
288
289 //////////////////////////////////////
290 /// Class CSendingDialogBase
291 //////////////////////////////////////
292 class CSendingDialogBase : public wxDialog
293 {
294 private:
295
296 protected:
297     wxStaticText* m_staticTextSending;
298     wxTextCtrl* m_textCtrlStatus;
299
300     wxButton* m_buttonOK;
301     wxButton* m_buttonCancel;
302
303 // Virtual event handlers, override them in your derived class
304 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
305 virtual void OnPaint(wxPaintEvent& event){ event.Skip(); }
306 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
307 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
308
309 public:
310 CSendingDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title =
311     wxT("Sending..."), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(442,151), long
312     style = wxDEFAULT_DIALOG_STYLE);
313 ~CSendingDialogBase();
314
315 };
316
317 //////////////////////////////////////
318 /// Class CYourAddressDialogBase
319 //////////////////////////////////////
320 class CYourAddressDialogBase : public wxDialog
321 {
322 private:
323
324 protected:
325     wxStaticText* m_staticText45;
326     wxListCtrl* m_listCtrl;
327
328     wxButton* m_buttonRename;
329     wxButton* m_buttonNew;
330     wxButton* m_buttonCopy;
331     wxButton* m_buttonOK;
332     wxButton* m_buttonCancel;
333

```

```

332 // Virtual event handlers, override them in your derived class
333 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
334 virtual void OnListEndLabelEdit(wxListEvent& event){ event.Skip(); }
335 virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
336 virtual void OnListItemSelected(wxListEvent& event){ event.Skip(); }
337 virtual void OnButtonRename(wxCommandEvent& event){ event.Skip(); }
338 virtual void OnButtonNew(wxCommandEvent& event){ event.Skip(); }
339 virtual void OnButtonCopy(wxCommandEvent& event){ event.Skip(); }
340 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
341 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
342
343
344 public:
345 CYourAddressDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Your
Bitcoin Address"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(610,390), long
style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
346 ~CYourAddressDialogBase();
347
348 };
349
350 ///////////////////////////////////////////////////////////////////
351 /// Class CAddressBookDialogBase
352 ///////////////////////////////////////////////////////////////////
353 class CAddressBookDialogBase : public wxDialog
354 {
355 private:
356 protected:
357
358     wxStaticText* m_staticText55;
359     wxListCtrl* m_listCtrl;
360
361     wxButton* m_buttonEdit;
362     wxButton* m_buttonNew;
363     wxButton* m_buttonDelete;
364     wxButton* m_buttonOK;
365
366
367 // Virtual event handlers, override them in your derived class
368 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
369 virtual void OnListEndLabelEdit(wxListEvent& event){ event.Skip(); }
370 virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
371 virtual void OnListItemSelected(wxListEvent& event){ event.Skip(); }
372 virtual void OnButtonEdit(wxCommandEvent& event){ event.Skip(); }
373 virtual void OnButtonNew(wxCommandEvent& event){ event.Skip(); }
374 virtual void OnButtonDelete(wxCommandEvent& event){ event.Skip(); }
375 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
376 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
377
378
379 public:
380     wxButton* m_buttonCancel;
381 CAddressBookDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Address
Book"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(610,390), long style =
wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
382 ~CAddressBookDialogBase();
383
384 };
385
386 ///////////////////////////////////////////////////////////////////
387 /// Class CProductsDialogBase
388 ///////////////////////////////////////////////////////////////////
389 class CProductsDialogBase : public wxDialog
390 {
391 private:
392 protected:
393
394     wxComboBox* m_comboBoxCategory;
395     wxTextCtrl* m_textCtrlSearch;
396     wxButton* m_buttonSearch;
397     wxListCtrl* m_listCtrl;
398
399 // Virtual event handlers, override them in your derived class

```

```

400 virtual void OnCombobox(wxCommandEvent& event){ event.Skip(); }
401 virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
402 virtual void OnButtonSearch(wxCommandEvent& event){ event.Skip(); }
403 virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
404
405
406 public:
407 CProductsDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title =
    wxT("Marketplace"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(708,535), long
    style = wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER);
408 ~CProductsDialogBase();
409
410 };
411
412 //////////////////////////////////////
413 /// Class CEditProductDialogBase
414 //////////////////////////////////////
415 class CEditProductDialogBase : public wxFrame
416 {
417 private:
418
419 protected:
420     wxScrolledWindow* m_scrolledWindow;
421     wxStaticText* m_staticText106;
422     wxComboBox* m_comboBoxCategory;
423     wxStaticText* m_staticText108;
424     wxTextCtrl* m_textCtrlTitle;
425     wxStaticText* m_staticText107;
426     wxTextCtrl* m_textCtrlPrice;
427     wxStaticText* m_staticText22;
428     wxTextCtrl* m_textCtrlDescription;
429     wxStaticText* m_staticText23;
430     wxTextCtrl* m_textCtrlInstructions;
431     wxStaticText* m_staticText24;
432     wxStaticText* m_staticText25;
433
434     wxTextCtrl* m_textCtrlLabel0;
435     wxTextCtrl* m_textCtrlField0;
436     wxButton* m_buttonDel0;
437     wxTextCtrl* m_textCtrlLabel1;
438     wxTextCtrl* m_textCtrlField1;
439     wxButton* m_buttonDel1;
440     wxTextCtrl* m_textCtrlLabel2;
441     wxTextCtrl* m_textCtrlField2;
442     wxButton* m_buttonDel2;
443     wxTextCtrl* m_textCtrlLabel3;
444     wxTextCtrl* m_textCtrlField3;
445     wxButton* m_buttonDel3;
446     wxTextCtrl* m_textCtrlLabel4;
447     wxTextCtrl* m_textCtrlField4;
448     wxButton* m_buttonDel4;
449     wxTextCtrl* m_textCtrlLabel5;
450     wxTextCtrl* m_textCtrlField5;
451     wxButton* m_buttonDel5;
452     wxTextCtrl* m_textCtrlLabel6;
453     wxTextCtrl* m_textCtrlField6;
454     wxButton* m_buttonDel6;
455     wxTextCtrl* m_textCtrlLabel7;
456     wxTextCtrl* m_textCtrlField7;
457     wxButton* m_buttonDel7;
458     wxTextCtrl* m_textCtrlLabel8;
459     wxTextCtrl* m_textCtrlField8;
460     wxButton* m_buttonDel8;
461     wxTextCtrl* m_textCtrlLabel9;
462     wxTextCtrl* m_textCtrlField9;
463     wxButton* m_buttonDel9;
464     wxTextCtrl* m_textCtrlLabel10;
465     wxTextCtrl* m_textCtrlField10;
466     wxButton* m_buttonDel10;
467     wxTextCtrl* m_textCtrlLabel11;
468     wxTextCtrl* m_textCtrlField11;
469     wxButton* m_buttonDel11;

```

```

470     wxTextCtrl* m_textCtrlLabel12;
471     wxTextCtrl* m_textCtrlField12;
472     wxButton* m_buttonDel12;
473     wxTextCtrl* m_textCtrlLabel13;
474     wxTextCtrl* m_textCtrlField13;
475     wxButton* m_buttonDel13;
476     wxTextCtrl* m_textCtrlLabel14;
477     wxTextCtrl* m_textCtrlField14;
478     wxButton* m_buttonDel14;
479     wxTextCtrl* m_textCtrlLabel15;
480     wxTextCtrl* m_textCtrlField15;
481     wxButton* m_buttonDel15;
482     wxTextCtrl* m_textCtrlLabel16;
483     wxTextCtrl* m_textCtrlField16;
484     wxButton* m_buttonDel16;
485     wxTextCtrl* m_textCtrlLabel17;
486     wxTextCtrl* m_textCtrlField17;
487     wxButton* m_buttonDel17;
488     wxTextCtrl* m_textCtrlLabel18;
489     wxTextCtrl* m_textCtrlField18;
490     wxButton* m_buttonDel18;
491     wxTextCtrl* m_textCtrlLabel19;
492     wxTextCtrl* m_textCtrlField19;
493     wxButton* m_buttonDel19;
494     wxButton* m_buttonAddField;
495     wxButton* m_buttonOK;
496     wxButton* m_buttonPreview;
497     wxButton* m_buttonCancel;
498
499     // Virtual event handlers, override them in your derived class
500     virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
501     virtual void OnButtonDel0(wxCommandEvent& event){ event.Skip(); }
502     virtual void OnButtonDel1(wxCommandEvent& event){ event.Skip(); }
503     virtual void OnButtonDel2(wxCommandEvent& event){ event.Skip(); }
504     virtual void OnButtonDel3(wxCommandEvent& event){ event.Skip(); }
505     virtual void OnButtonDel4(wxCommandEvent& event){ event.Skip(); }
506     virtual void OnButtonDel5(wxCommandEvent& event){ event.Skip(); }
507     virtual void OnButtonDel6(wxCommandEvent& event){ event.Skip(); }
508     virtual void OnButtonDel7(wxCommandEvent& event){ event.Skip(); }
509     virtual void OnButtonDel8(wxCommandEvent& event){ event.Skip(); }
510     virtual void OnButtonDel9(wxCommandEvent& event){ event.Skip(); }
511     virtual void OnButtonDel10(wxCommandEvent& event){ event.Skip(); }
512     virtual void OnButtonDel11(wxCommandEvent& event){ event.Skip(); }
513     virtual void OnButtonDel12(wxCommandEvent& event){ event.Skip(); }
514     virtual void OnButtonDel13(wxCommandEvent& event){ event.Skip(); }
515     virtual void OnButtonDel14(wxCommandEvent& event){ event.Skip(); }
516     virtual void OnButtonDel15(wxCommandEvent& event){ event.Skip(); }
517     virtual void OnButtonDel16(wxCommandEvent& event){ event.Skip(); }
518     virtual void OnButtonDel17(wxCommandEvent& event){ event.Skip(); }
519     virtual void OnButtonDel18(wxCommandEvent& event){ event.Skip(); }
520     virtual void OnButtonDel19(wxCommandEvent& event){ event.Skip(); }
521     virtual void OnButtonAddField(wxCommandEvent& event){ event.Skip(); }
522     virtual void OnButtonSend(wxCommandEvent& event){ event.Skip(); }
523     virtual void OnButtonPreview(wxCommandEvent& event){ event.Skip(); }
524     virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
525
526
527     public:
528         wxFlexGridSizer* fgSizer5;
529         CEditProductDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Edit
Product"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(660,640), long style =
wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
530         ~CEditProductDialogBase();
531
532     };
533
534     //////////////////////////////////////
535     /// Class CViewProductDialogBase
536     //////////////////////////////////////
537     class CViewProductDialogBase : public wxFrame
538     {
539     private:

```

```

540
541 protected:
542     wxHtmlWindow* m_htmlWinReviews;
543     wxScrolledWindow* m_scrolledWindow;
544     wxRichTextCtrl* m_richTextHeading;
545     wxStaticText* m_staticTextInstructions;
546     wxButton* m_buttonSubmitForm;
547     wxButton* m_buttonCancelForm;
548     wxButton* m_buttonBack;
549     wxButton* m_buttonNext;
550     wxButton* m_buttonCancel;
551
552 // Virtual event handlers, override them in your derived class
553 virtual void OnButtonSubmitForm(wxCommandEvent& event){ event.Skip(); }
554 virtual void OnButtonCancelForm(wxCommandEvent& event){ event.Skip(); }
555 virtual void OnButtonBack(wxCommandEvent& event){ event.Skip(); }
556 virtual void OnButtonNext(wxCommandEvent& event){ event.Skip(); }
557 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
558
559
560 public:
561 CViewProductDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Order
Form"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(630,520), long style =
wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
562 ~CViewProductDialogBase();
563
564 };
565
566 //////////////////////////////////////////////////
567 // Class CViewOrderDialogBase
568 //////////////////////////////////////////////////
569 class CViewOrderDialogBase : public wxFrame
570 {
571 private:
572
573 protected:
574     wxHtmlWindow* m_htmlWin;
575     wxButton* m_buttonOK;
576
577 // Virtual event handlers, override them in your derived class
578 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
579
580
581 public:
582 CViewOrderDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("View
Order"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(630,520), long style =
wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
583 ~CViewOrderDialogBase();
584
585 };
586
587 //////////////////////////////////////////////////
588 // Class CEditReviewDialogBase
589 //////////////////////////////////////////////////
590 class CEditReviewDialogBase : public wxFrame
591 {
592 private:
593
594 protected:
595
596     wxStaticText* m_staticTextSeller;
597
598     wxStaticText* m_staticText110;
599     wxChoice* m_choiceStars;
600     wxStaticText* m_staticText43;
601     wxTextCtrl* m_textCtrlReview;
602     wxButton* m_buttonSubmit;
603     wxButton* m_buttonCancel;
604
605 // Virtual event handlers, override them in your derived class
606 virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
607 virtual void OnButtonSubmit(wxCommandEvent& event){ event.Skip(); }

```



```

608 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
609
610
611 public:
612 CEditReviewDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Enter
Review"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(630,440), long style =
wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER|wxTAB_TRAVERSAL);
613 ~CEditReviewDialogBase();
614
615 };
616
617 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
618 // Class CPokerLobbyDialogBase
619 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
620 class CPokerLobbyDialogBase : public wxFrame
621 {
622 private:
623
624 protected:
625     wxTreeCtrl* m_treeCtrl;
626     wxListCtrl* m_listCtrl;
627     wxButton* m_buttonNewTable;
628
629 // Virtual event handlers, override them in your derived class
630 virtual void OnTreeSelChanged(wxTreeEvent& event){ event.Skip(); }
631 virtual void OnListItemActivated(wxListEvent& event){ event.Skip(); }
632 virtual void OnListItemSelected(wxListEvent& event){ event.Skip(); }
633 virtual void OnButtonNewTable(wxCommandEvent& event){ event.Skip(); }
634
635
636 public:
637 CPokerLobbyDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Poker
Lobby"), const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(586,457), long style =
wxDEFAULT_FRAME_STYLE|wxTAB_TRAVERSAL);
638 ~CPokerLobbyDialogBase();
639
640 };
641
642 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
643 // Class CPokerDialogBase
644 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
645 class CPokerDialogBase : public wxFrame
646 {
647 private:
648
649 protected:
650     wxButton* m_buttonDealHand;
651     wxButton* m_buttonFold;
652     wxButton* m_buttonCall;
653     wxButton* m_buttonRaise;
654     wxButton* m_buttonLeaveTable;
655     wxTextCtrl* m_textDitchPlayer;
656
657 // Virtual event handlers, override them in your derived class
658 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
659 virtual void OnMouseEvents(wxMouseEvent& event){ event.Skip(); }
660 virtual void OnPaint(wxPaintEvent& event){ event.Skip(); }
661 virtual void OnSize(wxSizeEvent& event){ event.Skip(); }
662 virtual void OnCheckSitOut(wxCommandEvent& event){ event.Skip(); }
663 virtual void OnButtonDealHand(wxCommandEvent& event){ event.Skip(); }
664 virtual void OnButtonFold(wxCommandEvent& event){ event.Skip(); }
665 virtual void OnButtonCall(wxCommandEvent& event){ event.Skip(); }
666 virtual void OnButtonRaise(wxCommandEvent& event){ event.Skip(); }
667 virtual void OnButtonLeaveTable(wxCommandEvent& event){ event.Skip(); }
668 virtual void OnDitchPlayer(wxCommandEvent& event){ event.Skip(); }
669 virtual void OnCheckPreFold(wxCommandEvent& event){ event.Skip(); }
670 virtual void OnCheckPreCall(wxCommandEvent& event){ event.Skip(); }
671 virtual void OnCheckPreCallAny(wxCommandEvent& event){ event.Skip(); }
672 virtual void OnCheckPreRaise(wxCommandEvent& event){ event.Skip(); }
673 virtual void OnCheckPreRaiseAny(wxCommandEvent& event){ event.Skip(); }
674
675

```

```

676 public:
677     wxCheckBox* m_checkSitOut;
678     wxCheckBox* m_checkPreFold;
679     wxCheckBox* m_checkPreCall;
680     wxCheckBox* m_checkPreCallAny;
681     wxCheckBox* m_checkPreRaise;
682     wxCheckBox* m_checkPreRaiseAny;
683     wxStatusBar* m_statusBar;
684 CPokerDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title = wxT("Poker"), const
wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(806,550), long style =
wxDEFAULT_FRAME_STYLE|wxFRAME_NO_TASKBAR|wxFULL_REPAINT_ON_RESIZE|wxTAB_TRAVERSAL);
685 ~CPokerDialogBase();
686
687 };
688
689 //////////////////////////////////////////////////
690 // Class CGetTextFromUserDialogBase
691 //////////////////////////////////////////////////
692 class CGetTextFromUserDialogBase : public wxDialog
693 {
694 private:
695
696 protected:
697
698     wxStaticText* m_staticTextMessage1;
699     wxTextCtrl* m_textCtrl1;
700     wxStaticText* m_staticTextMessage2;
701     wxTextCtrl* m_textCtrl2;
702
703
704     wxButton* m_buttonOK;
705     wxButton* m_buttonCancel;
706
707 // Virtual event handlers, override them in your derived class
708 virtual void OnClose(wxCloseEvent& event){ event.Skip(); }
709 virtual void OnKeyDown(wxKeyEvent& event){ event.Skip(); }
710 virtual void OnButtonOK(wxCommandEvent& event){ event.Skip(); }
711 virtual void OnButtonCancel(wxCommandEvent& event){ event.Skip(); }
712
713
714 public:
715 CGetTextFromUserDialogBase(wxWindow* parent, wxWindowID id = wxID_ANY, const wxString& title =
wxEmptyString, const wxPoint& pos = wxDefaultPosition, const wxSize& size = wxSize(403,138), long style
= wxDEFAULT_DIALOG_STYLE);
716 ~CGetTextFromUserDialogBase();
717
718 };
719
720 #endif //__uibase__
721

```

SUMMARY

These declarations allow other code to interact with the GUI without knowing the implementation details.

uint256.h

751 lines CPP

NUMBERS BEYOND IMAGINATION

Standard computers work with 32-bit or 64-bit integers, but Bitcoin needs 256-bit numbers. This file implements `uint256`—a custom 256-bit unsigned integer type that can represent numbers up to $2^{256} - 1$. That's approximately 10^{77} , a number so vast it exceeds the estimated atoms in the observable universe by many orders of magnitude.

Every SHA-256 hash is a `uint256`. Every transaction ID is a `uint256`. Every block hash is a `uint256`. Every private key is conceptually a `uint256` (though stored differently). This type is the atomic unit of Bitcoin's cryptographic identity—the "DNA" that makes every transaction unique and every block identifiable.

The implementation stores 256 bits as an array of eight 32-bit words (or four 64-bit words on 64-bit systems). Arithmetic operations—addition, subtraction, comparison, bit shifting—are implemented by operating across these words with proper carry/borrow handling. It's essentially reimplementing grade-school arithmetic for numbers too big for the processor to handle natively.

Also defined here is `uint160`, a 160-bit integer used for Bitcoin addresses. The address derivation process (`RIPEMD160(SHA256(pubkey))`) produces 160 bits. The shorter length was a deliberate trade-off: addresses are 27-34 characters instead of ~64, making them more human-manageable while still providing 2^{80} security against collision attacks (via the birthday paradox).

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include <limits.h>
6 #include <string>
7 #if defined(_MSC_VER) || defined(__BORLANDC__)
8 typedef __int64 int64;

```

Portable 64-bit integer definitions. In 2009, C++ didn't have standardized 64-bit types (`int64_t` came with C++11). This code detects the compiler (MSVC vs GCC) and uses the appropriate type. Modern Bitcoin uses `<cstdint>`, but this worked across Windows and Linux compilers of the era.

```

9 typedef unsigned __int64 uint64;
10 #else
11 typedef long long int64;
12 typedef unsigned long long uint64;
13 #endif
14 #if defined(_MSC_VER) && _MSC_VER < 1300
15 #define for if (false) ; else for

```

MSVC 6.0 compatibility hack: `#define for if(false);else for`. Ancient Visual C++ had a bug where variables declared in `for`-loop initializers leaked into the enclosing scope. This macro forces standard behavior. Satoshi supported very old compilers—Bitcoin needed to run on whatever machines people had.

```

16 #endif
17
18
19 inline int Testuint256AdHoc(vector<string> vArg);
20
21
22
23 // We have to keep a separate base class without constructors

```

Satoshi's comment explains a C++ subtlety: 'We have to keep a separate base class without constructors so the compiler will let us use it in a union.' C++ unions can't contain members with non-trivial constructors (pre-C++11). By keeping `base_uint` constructor-free, it can be used in unions elsewhere in the codebase. The derived `uint160`/`uint256` classes add constructors.

```

24 // so the compiler will let us use it in a union
25 template<unsigned int BITS>
26 class base_uint

```

`base_uint` —template foundation for large integers. The template parameter `BITS` specifies the size (160 or 256). `WIDTH` calculates how many 32-bit words are needed: $256/32 = 8$ words. The `pn[]` array stores these words in little-endian order (least significant word first). This simplifies carry propagation in arithmetic.

```

27 {
28 protected:
29 enum { WIDTH=BITS/32 };
30 unsigned int pn[WIDTH];
31 public:
32
33 bool operator!() const

```

operator!() —tests if the value is zero. Iterates through all words; if any is non-zero, returns `false`. Used in boolean contexts: `if (!hash)` checks for the all-zeros hash, which indicates 'no parent' in the genesis block's `prevhash` field.

```

34 {
35     for (int i = 0; i < WIDTH; i++)
36         if (pn[i] != 0)
37             return false;
38     return true;
39 }
40
41 const base_uint operator~() const
    
```

operator~() —bitwise NOT (complement). Flips every bit. Used in difficulty calculations: the compact difficulty format encodes targets, and some calculations need inverted values. Each word is independently complemented.

```

42 {
43     base_uint ret;
44     for (int i = 0; i < WIDTH; i++)
45         ret.pn[i] = ~pn[i];
46     return ret;
47 }
48
49 const base_uint operator-() const
    
```

operator-() —unary negation (two's complement). Computes the additive inverse: `-x` such that `x + (-x) = 0`. Implemented as bitwise NOT plus one (`~x + 1`), the standard two's complement formula. The increment handles the carry across all 8 words correctly.

```

50 {
51     base_uint ret;
52     for (int i = 0; i < WIDTH; i++)
53         ret.pn[i] = ~pn[i];
54     ret++;
55     return ret;
56 }
57
58
59 base_uint& operator=(uint64 b)
    
```

operator=(uint64) —assigns a 64-bit value. The low two words get the value (split into low and high 32 bits); remaining words are zeroed. This is the most common initialization: `uint256 hash = 0` creates an all-zeros hash.

```

60 {
61     pn[0] = (unsigned int)b;
62     pn[1] = (unsigned int)(b >> 32);
63     for (int i = 2; i < WIDTH; i++)
64         pn[i] = 0;
65     return *this;
66 }
67
68 base_uint& operator^=(const base_uint& b)
69 {
70     for (int i = 0; i < WIDTH; i++)
71         pn[i] ^= b.pn[i];
72     return *this;
73 }
    
```

```

74
75     base_uint& operator&=(const base_uint& b)
76 {
77     for (int i = 0; i < WIDTH; i++)
78         pn[i] &= b.pn[i];
79     return *this;
80 }
81
82     base_uint& operator|=(const base_uint& b)
83 {
84     for (int i = 0; i < WIDTH; i++)
85         pn[i] |= b.pn[i];
86     return *this;
87 }
88
89     base_uint& operator^=(uint64 b)
90 {
91     pn[0] ^= (unsigned int)b;
92     pn[1] ^= (unsigned int)(b >> 32);
93     return *this;
94 }
95
96     base_uint& operator&=(uint64 b)
97 {
98     pn[0] &= (unsigned int)b;
99     pn[1] &= (unsigned int)(b >> 32);
100    return *this;
101 }
102
103     base_uint& operator|=(uint64 b)
104 {
105     pn[0] |= (unsigned int)b;
106     pn[1] |= (unsigned int)(b >> 32);
107     return *this;
108 }
109
110     base_uint& operator<<=(unsigned int shift)

```

operator<<=() —left bit shift. Shifts the entire 256-bit value left by `shift` positions. The implementation handles cross-word shifts: if shifting by 40 bits, some bits move from word 0 to word 1, others from word 1 to word 2. The calculation `k = shift / 32` gives whole-word shift; `shift % 32` gives within-word shift. Critical for difficulty adjustments and halving calculations.

```

111 {
112     base_uint a(*this);
113     for (int i = 0; i < WIDTH; i++)
114         pn[i] = 0;
115     int k = shift / 32;
116     shift = shift % 32;
117     for (int i = 0; i < WIDTH; i++)
118     {
119         if (i+k+1 < WIDTH && shift != 0)
120             pn[i+k+1] |= (a.pn[i] >> (32-shift));
121         if (i+k < WIDTH)
122             pn[i+k] |= (a.pn[i] << shift);
123     }
124     return *this;
125 }
126
127     base_uint& operator>>=(unsigned int shift)

```

`operator>>=()` —right bit shift. The mirror of left shift, used in the halving formula: `nSubsidy >>= (nBestHeight / 210000)` divides the block reward by 2 for each halving era. Each right shift by 1 is equivalent to integer division by 2.

```

128 {
129     base_uint a(*this);
130     for (int i = 0; i < WIDTH; i++)
131         pn[i] = 0;
132     int k = shift / 32;
133     shift = shift % 32;
134     for (int i = 0; i < WIDTH; i++)
135     {
136         if (i-k-1 >= 0 && shift != 0)
137             pn[i-k-1] |= (a.pn[i] << (32-shift));
138         if (i-k >= 0)
139             pn[i-k] |= (a.pn[i] >> shift);
140     }
141     return *this;
142 }
143
144 base_uint& operator+=(const base_uint& b)

```

`operator+=()` —addition with carry propagation. This is grade-school addition scaled to 256 bits. For each word pair, add plus incoming carry, store low 32 bits as result, carry the high bits to the next word. The `uint64` temporary naturally captures the 33-bit intermediate result. Used in address derivation and difficulty calculations.

```

145 {
146     uint64 carry = 0;
147     for (int i = 0; i < WIDTH; i++)
148     {
149         uint64 n = carry + pn[i] + b.pn[i];
150         pn[i] = n & 0xffffffff;
151         carry = n >> 32;
152     }
153     return *this;
154 }
155
156 base_uint& operator--=(const base_uint& b)
157 {
158     *this += -b;
159     return *this;
160 }
161
162 base_uint& operator+=(uint64 b64)
163 {
164     base_uint b;
165     b = b64;
166     *this += b;
167     return *this;
168 }
169
170 base_uint& operator--=(uint64 b64)
171 {
172     base_uint b;
173     b = b64;
174     *this += -b;
175     return *this;
176 }
177
178
179 base_uint& operator++()

```

`operator++()` —prefix increment. The elegant implementation: increment word 0; if it wrapped to 0 (overflow), increment word 1; repeat. The `while (++pn[i] == 0)` chain handles the carry propagation. Used when iterating through nonces during mining: `++block.nNonce`.

```

180 {
181 // prefix operator
182 int i = 0;
183 while (++pn[i] == 0 && i < WIDTH-1)
184     i++;
185 return *this;
186 }
187
188 const base_uint operator++(int)
189 {
190 // postfix operator
191 const base_uint ret = *this;
192 ++(*this);
193 return ret;
194 }
195
196 base_uint& operator--()
197 {
198 // prefix operator
199 int i = 0;
200 while (--pn[i] == -1 && i < WIDTH-1)
201     i++;
202 return *this;
203 }
204
205 const base_uint operator--(int)
206 {
207 // postfix operator
208 const base_uint ret = *this;
209 --(*this);
210 return ret;
211 }
212
213
214 friend inline bool operator<(const base_uint& a, const base_uint& b)

```

`operator<()` —less-than comparison. Compares from most significant word down. This is the core of proof-of-work validation: `hash <= target` checks if a block hash is 'small enough' to meet difficulty. The first non-equal word determines the result. If all words are equal, returns `false` (not strictly less than).

```

215 {
216 for (int i = base_uint::WIDTH-1; i >= 0; i--)
217 {
218 if (a.pn[i] < b.pn[i])
219 return true;
220 else if (a.pn[i] > b.pn[i])
221 return false;
222 }
223 return false;
224 }
225
226 friend inline bool operator<=(const base_uint& a, const base_uint& b)
227 {
228 for (int i = base_uint::WIDTH-1; i >= 0; i--)
229 {
230 if (a.pn[i] < b.pn[i])
231 return true;
232 else if (a.pn[i] > b.pn[i])
233 return false;

```



```

234 }
235 return true;
236 }
237
238 friend inline bool operator>(const base_uint& a, const base_uint& b)
239 {
240     for (int i = base_uint::WIDTH-1; i >= 0; i--)
241     {
242         if (a.pn[i] > b.pn[i])
243             return true;
244         else if (a.pn[i] < b.pn[i])
245             return false;
246     }
247     return false;
248 }
249
250 friend inline bool operator>=(const base_uint& a, const base_uint& b)
251 {
252     for (int i = base_uint::WIDTH-1; i >= 0; i--)
253     {
254         if (a.pn[i] > b.pn[i])
255             return true;
256         else if (a.pn[i] < b.pn[i])
257             return false;
258     }
259     return true;
260 }
261
262 friend inline bool operator==(const base_uint& a, const base_uint& b)

```

`operator==()` —equality comparison. Compares all words; if any differ, returns `false`. Used to check for specific hashes: `if (hash == hashGenesisBlock)` identifies the genesis block. Note this is a bitwise comparison—there's no concept of 'approximately equal' for cryptographic hashes.

```

263 {
264     for (int i = 0; i < base_uint::WIDTH; i++)
265         if (a.pn[i] != b.pn[i])
266             return false;
267     return true;
268 }
269
270 friend inline bool operator==(const base_uint& a, uint64 b)
271 {
272     if (a.pn[0] != (unsigned int)b)
273         return false;
274     if (a.pn[1] != (unsigned int)(b >> 32))
275         return false;
276     for (int i = 2; i < base_uint::WIDTH; i++)
277         if (a.pn[i] != 0)
278             return false;
279     return true;
280 }
281
282 friend inline bool operator!=(const base_uint& a, const base_uint& b)
283 {
284     return (!(a == b));
285 }
286
287 friend inline bool operator!=(const base_uint& a, uint64 b)
288 {
289     return (!(a == b));
290 }
291
292
293
294     std::string GetHex() const

```

`GetHex()` —converts to hexadecimal string. Transaction IDs, block hashes, and addresses are typically displayed as hex. The loop iterates backward through bytes (big-endian display convention) and formats each as two hex digits. The 64-character result is how you see txids in block explorers.

```

295 {
296   char psz[sizeof(pn)*2 + 1];
297   for (int i = 0; i < sizeof(pn); i++)
298     sprintf(psz + i*2, "%02x", ((unsigned char*)pn)[sizeof(pn) - i - 1]);
299   return string(psz, psz + sizeof(pn)*2);
300 }
301
302 void SetHex(const std::string& str)

```

`SetHex()` —parses hexadecimal string. The inverse of `GetHex()` : takes a string like `'000000000019d6...'` and populates the `uint256`. Handles optional `0x` prefix and leading whitespace. The `phexdigit[]` lookup table provides fast hex-to-value conversion. Used when loading hardcoded hashes (like genesis block) from source code.

```

303 {
304   for (int i = 0; i < WIDTH; i++)
305     pn[i] = 0;
306
307   // skip 0x
308   const char* psz = str.c_str();
309   while (isspace(*psz))
310     psz++;
311   if (psz[0] == '0' && tolower(psz[1]) == 'x')
312     psz += 2;
313   while (isspace(*psz))
314     psz++;
315
316   // hex string to uint
317   static char phexdigit[256] = { 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,1,2,3,4,5,6,7,8,9,0,0,0,0,0,0,
0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
0,0xa,0xb,0xc,0xd,0xe,0xf,0,0,0,0,0,0,0,0,0,0 };
318   const char* pbegin = psz;
319   while (phexdigit[*psz] || *psz == '0')
320     psz++;
321   psz--;
322   unsigned char* p1 = (unsigned char*)pn;
323   unsigned char* pend = p1 + WIDTH * 4;
324   while (psz >= pbegin && p1 < pend)
325   {
326     *p1 = phexdigit[(unsigned char)*psz--];
327     if (psz >= pbegin)
328     {
329       *p1 |= (phexdigit[(unsigned char)*psz--] << 4);
330       p1++;
331     }
332   }
333 }
334
335   std::string ToString() const
336 {
337   return (GetHex());
338 }
339
340 unsigned char* begin()
341 {
342   return (unsigned char*)&pn[0];
343 }
344
345 unsigned char* end()

```

```

346 {
347     return (unsigned char*)&pn[WIDTH];
348 }
349
350 unsigned int size()
351 {
352     return sizeof(pn);
353 }
354
355
356 unsigned int GetSerializeSize(int nType=0, int nVersion=VERSION) const

```

Serialization methods: `GetSerializeSize()`, `Serialize()`, `Unserialize()`. These integrate with Bitcoin's serialization framework (`serialize.h`). A `uint256` serializes as exactly 32 bytes (256 bits), raw binary, little-endian. This is how hashes are stored on disk and transmitted over the network—no formatting overhead.

```

357 {
358     return sizeof(pn);
359 }
360
361 template<typename Stream>
362 void Serialize(Stream& s, int nType=0, int nVersion=VERSION) const
363 {
364     s.write((char*)pn, sizeof(pn));
365 }
366
367 template<typename Stream>
368 void Unserialize(Stream& s, int nType=0, int nVersion=VERSION)
369 {
370     s.read((char*)pn, sizeof(pn));
371 }
372
373
374 friend class uint160;
375 friend class uint256;
376 friend inline int Testuint256AdHoc(vector<string> vArg);
377 };
378
379 typedef base_uint<160> base_uint160;

```

Typedefs: `base_uint160` and `base_uint256`. These instantiate the template at 160 and 256 bits. The actual `uint160` / `uint256` classes inherit from these, adding constructors. This two-level hierarchy satisfies C++ union requirements (base class has no constructors) while providing convenient construction syntax.

```

380 typedef base_uint<256> base_uint256;
381
382
383
384 //

```

Satoshi's comment on the duplicate code: 'uint160 and uint256 could be implemented as templates, but to keep compile errors and debugging cleaner, they're copy and pasted.' Pragmatic engineering over theoretical purity. Debugging template instantiation errors was even harder in 2009 compilers.

```

385 // uint160 and uint256 could be implemented as templates, but to keep
386 // compile errors and debugging cleaner, they're copy and pasted.
387 // It's safe to search and replace 160 with 256 and vice versa.
388 //
389
390

```

```

391
392 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
393 //
394 // uint160
395 //
396
397 class uint160 : public base_uint160

```

uint160 —160-bit integer for address hashes. Bitcoin addresses are RIPEMD160(SHA256(pubkey)), producing 160 bits. Why not 256? Shorter addresses are more usable (27-34 characters vs ~64). The security is still 2^{80} against birthday attacks—far beyond brute-forceable. Every 'hash160' in Bitcoin's script system is this type.

```

398 {
399 public:
400 typedef base_uint160 basetype;
401
402 uint160()
403 {
404 }
405
406 uint160(const basetype& b)
407 {
408     for (int i = 0; i < WIDTH; i++)
409         pn[i] = b.pn[i];
410 }
411
412 uint160& operator=(const basetype& b)
413 {
414     for (int i = 0; i < WIDTH; i++)
415         pn[i] = b.pn[i];
416     return *this;
417 }
418
419 uint160(uint64 b)
420 {
421     pn[0] = (unsigned int)b;
422     pn[1] = (unsigned int)(b >> 32);
423     for (int i = 2; i < WIDTH; i++)
424         pn[i] = 0;
425 }
426
427 uint160& operator=(uint64 b)
428 {
429     pn[0] = (unsigned int)b;
430     pn[1] = (unsigned int)(b >> 32);
431     for (int i = 2; i < WIDTH; i++)
432         pn[i] = 0;
433     return *this;
434 }
435
436 explicit uint160(const std::string& str)
437 {
438     SetHex(str);
439 }
440
441 explicit uint160(const std::vector<unsigned char>& vch)
442 {
443     if (vch.size() == sizeof(pn))
444         memcpy(pn, &vch[0], sizeof(pn));
445     else
446         *this = 0;
447 }
448 };
449
450 inline bool operator==(const uint160& a, uint64 b) { return (base_uint160)a == b; }
451 inline bool operator!=(const uint160& a, uint64 b) { return (base_uint160)a != b; }

```

```

452 inline const uint160 operator<<(const base_uint160& a, unsigned int shift) { return uint160(a) <<=
    shift; }
453 inline const uint160 operator>>(const base_uint160& a, unsigned int shift) { return uint160(a) >>=
    shift; }
454 inline const uint160 operator<<(const uint160& a, unsigned int shift) { return uint160(a) <<= shift; }
455 inline const uint160 operator>>(const uint160& a, unsigned int shift) { return uint160(a) >>= shift; }
456
457 inline const uint160 operator^(const base_uint160& a, const base_uint160& b) { return uint160(a) ^ b; }
458 inline const uint160 operator&(const base_uint160& a, const base_uint160& b) { return uint160(a) & b; }
459 inline const uint160 operator|(const base_uint160& a, const base_uint160& b) { return uint160(a) | b; }
460 inline const uint160 operator+(const base_uint160& a, const base_uint160& b) { return uint160(a) + b; }
461 inline const uint160 operator-(const base_uint160& a, const base_uint160& b) { return uint160(a) - b; }
462
463 inline bool operator<(const base_uint160& a, const uint160& b) { return (base_uint160)a <
    (base_uint160)b; }
464 inline bool operator<=(const base_uint160& a, const uint160& b) { return (base_uint160)a <=
    (base_uint160)b; }
465 inline bool operator>(const base_uint160& a, const uint160& b) { return (base_uint160)a >
    (base_uint160)b; }
466 inline bool operator>=(const base_uint160& a, const uint160& b) { return (base_uint160)a >=
    (base_uint160)b; }
467 inline bool operator==(const base_uint160& a, const uint160& b) { return (base_uint160)a ==
    (base_uint160)b; }
468 inline bool operator!=(const base_uint160& a, const uint160& b) { return (base_uint160)a !=
    (base_uint160)b; }
469 inline const uint160 operator^(const base_uint160& a, const uint160& b) { return (base_uint160)a ^
    (base_uint160)b; }
470 inline const uint160 operator&(const base_uint160& a, const uint160& b) { return (base_uint160)a &
    (base_uint160)b; }
471 inline const uint160 operator|(const base_uint160& a, const uint160& b) { return (base_uint160)a |
    (base_uint160)b; }
472 inline const uint160 operator+(const base_uint160& a, const uint160& b) { return (base_uint160)a +
    (base_uint160)b; }
473 inline const uint160 operator-(const base_uint160& a, const uint160& b) { return (base_uint160)a -
    (base_uint160)b; }
474
475 inline bool operator<(const uint160& a, const base_uint160& b) { return (base_uint160)a <
    (base_uint160)b; }
476 inline bool operator<=(const uint160& a, const base_uint160& b) { return (base_uint160)a <=
    (base_uint160)b; }
477 inline bool operator>(const uint160& a, const base_uint160& b) { return (base_uint160)a >
    (base_uint160)b; }
478 inline bool operator>=(const uint160& a, const base_uint160& b) { return (base_uint160)a >=
    (base_uint160)b; }
479 inline bool operator==(const uint160& a, const base_uint160& b) { return (base_uint160)a ==
    (base_uint160)b; }
480 inline bool operator!=(const uint160& a, const base_uint160& b) { return (base_uint160)a !=
    (base_uint160)b; }
481 inline const uint160 operator^(const uint160& a, const base_uint160& b) { return (base_uint160)a ^
    (base_uint160)b; }
482 inline const uint160 operator&(const uint160& a, const base_uint160& b) { return (base_uint160)a &
    (base_uint160)b; }
483 inline const uint160 operator|(const uint160& a, const base_uint160& b) { return (base_uint160)a |
    (base_uint160)b; }
484 inline const uint160 operator+(const uint160& a, const base_uint160& b) { return (base_uint160)a +
    (base_uint160)b; }
485 inline const uint160 operator-(const uint160& a, const base_uint160& b) { return (base_uint160)a -
    (base_uint160)b; }
486
487 inline bool operator<(const uint160& a, const uint160& b) { return (base_uint160)a < (base_uint160)b; }
488 inline bool operator<=(const uint160& a, const uint160& b) { return (base_uint160)a <= (base_uint160)b;
    }
489 inline bool operator>(const uint160& a, const uint160& b) { return (base_uint160)a > (base_uint160)b; }
490 inline bool operator>=(const uint160& a, const uint160& b) { return (base_uint160)a >= (base_uint160)b;
    }
491 inline bool operator==(const uint160& a, const uint160& b) { return (base_uint160)a == (base_uint160)b;
    }
492 inline bool operator!=(const uint160& a, const uint160& b) { return (base_uint160)a != (base_uint160)b;
    }
493 inline const uint160 operator^(const uint160& a, const uint160& b) { return (base_uint160)a ^
    (base_uint160)b; }

```

```

494 inline const uint160 operator&(const uint160& a, const uint160& b) { return (base_uint160)a &
    (base_uint160)b; }
495 inline const uint160 operator|(const uint160& a, const uint160& b) { return (base_uint160)a |
    (base_uint160)b; }
496 inline const uint160 operator+(const uint160& a, const uint160& b) { return (base_uint160)a +
    (base_uint160)b; }
497 inline const uint160 operator-(const uint160& a, const uint160& b) { return (base_uint160)a -
    (base_uint160)b; }
498
499
500
501
502
503
504 ///////////////////////////////////////////////////
505 //
506 // uint256
507 //
508
509 class uint256 : public base_uint256

```

uint256—256-bit integer for hashes. The workhorse of Bitcoin: block hashes, transaction hashes, merkle roots, difficulty targets, private keys (conceptually). Every SHA-256 result is a uint256. The choice of 256 bits matches modern cryptographic standards and provides a massive security margin.

```

510 {
511 public:
512 typedef base_uint256 basetype;
513
514 uint256()
515 {
516 }
517
518 uint256(const basetype& b)
519 {
520 for (int i = 0; i < WIDTH; i++)
521     pn[i] = b.pn[i];
522 }
523
524 uint256& operator=(const basetype& b)
525 {
526 for (int i = 0; i < WIDTH; i++)
527     pn[i] = b.pn[i];
528 return *this;
529 }
530
531 uint256(uint64 b)
532 {
533     pn[0] = (unsigned int)b;
534     pn[1] = (unsigned int)(b >> 32);
535 for (int i = 2; i < WIDTH; i++)
536     pn[i] = 0;
537 }
538
539 uint256& operator=(uint64 b)
540 {
541     pn[0] = (unsigned int)b;
542     pn[1] = (unsigned int)(b >> 32);
543 for (int i = 2; i < WIDTH; i++)
544     pn[i] = 0;
545 return *this;
546 }
547
548 explicit uint256(const std::string& str)
549 {
550 SetHex(str);
551 }
552
553 explicit uint256(const std::vector<unsigned char>& vch)

```

```

554 {
555     if (vch.size() == sizeof(pn))
556         memcpy(pn, &vch[0], sizeof(pn));
557     else
558         *this = 0;
559 }
560 };
561
562 inline bool operator==(const uint256& a, uint64 b) { return (base_uint256)a == b; }

```

Free-standing operators for `uint256`. C++ requires these for mixed expressions like `hash + 1` where the `uint256` is on the left and a native integer on the right. Each is implemented in terms of the compound assignment operators, keeping code DRY while providing intuitive arithmetic syntax.

```

563 inline bool operator!=(const uint256& a, uint64 b) { return (base_uint256)a != b; }
564 inline const uint256 operator<<(const base_uint256& a, unsigned int shift) { return uint256(a) <<=
    shift; }
565 inline const uint256 operator>>(const base_uint256& a, unsigned int shift) { return uint256(a) >>=
    shift; }
566 inline const uint256 operator<<(const uint256& a, unsigned int shift) { return uint256(a) <<= shift; }
567 inline const uint256 operator>>(const uint256& a, unsigned int shift) { return uint256(a) >>= shift; }
568
569 inline const uint256 operator^(const base_uint256& a, const base_uint256& b) { return uint256(a) ^= b; }
570 inline const uint256 operator&(const base_uint256& a, const base_uint256& b) { return uint256(a) &= b; }
571 inline const uint256 operator|(const base_uint256& a, const base_uint256& b) { return uint256(a) |= b; }
572 inline const uint256 operator+(const base_uint256& a, const base_uint256& b) { return uint256(a) += b; }
573 inline const uint256 operator-(const base_uint256& a, const base_uint256& b) { return uint256(a) -= b; }
574
575 inline bool operator<(const base_uint256& a, const uint256& b) { return (base_uint256)a <
    (base_uint256)b; }
576 inline bool operator<=(const base_uint256& a, const uint256& b) { return (base_uint256)a <=
    (base_uint256)b; }
577 inline bool operator>(const base_uint256& a, const uint256& b) { return (base_uint256)a >
    (base_uint256)b; }
578 inline bool operator>=(const base_uint256& a, const uint256& b) { return (base_uint256)a >=
    (base_uint256)b; }
579 inline bool operator==(const base_uint256& a, const uint256& b) { return (base_uint256)a ==
    (base_uint256)b; }
580 inline bool operator!=(const base_uint256& a, const uint256& b) { return (base_uint256)a !=
    (base_uint256)b; }
581 inline const uint256 operator^(const base_uint256& a, const uint256& b) { return (base_uint256)a ^
    (base_uint256)b; }
582 inline const uint256 operator&(const base_uint256& a, const uint256& b) { return (base_uint256)a &
    (base_uint256)b; }
583 inline const uint256 operator|(const base_uint256& a, const uint256& b) { return (base_uint256)a |
    (base_uint256)b; }
584 inline const uint256 operator+(const base_uint256& a, const uint256& b) { return (base_uint256)a +
    (base_uint256)b; }
585 inline const uint256 operator-(const base_uint256& a, const uint256& b) { return (base_uint256)a -
    (base_uint256)b; }
586
587 inline bool operator<(const uint256& a, const base_uint256& b) { return (base_uint256)a <
    (base_uint256)b; }
588 inline bool operator<=(const uint256& a, const base_uint256& b) { return (base_uint256)a <=
    (base_uint256)b; }
589 inline bool operator>(const uint256& a, const base_uint256& b) { return (base_uint256)a >
    (base_uint256)b; }
590 inline bool operator>=(const uint256& a, const base_uint256& b) { return (base_uint256)a >=
    (base_uint256)b; }
591 inline bool operator==(const uint256& a, const base_uint256& b) { return (base_uint256)a ==
    (base_uint256)b; }
592 inline bool operator!=(const uint256& a, const base_uint256& b) { return (base_uint256)a !=
    (base_uint256)b; }
593 inline const uint256 operator^(const uint256& a, const base_uint256& b) { return (base_uint256)a ^
    (base_uint256)b; }
594 inline const uint256 operator&(const uint256& a, const base_uint256& b) { return (base_uint256)a &
    (base_uint256)b; }
595 inline const uint256 operator|(const uint256& a, const base_uint256& b) { return (base_uint256)a |
    (base_uint256)b; }

```

```

596 inline const uint256 operator+(const uint256& a, const base_uint256& b) { return (base_uint256)a +
    (base_uint256)b; }
597 inline const uint256 operator-(const uint256& a, const base_uint256& b) { return (base_uint256)a -
    (base_uint256)b; }
598
599 inline bool operator<(const uint256& a, const uint256& b) { return (base_uint256)a < (base_uint256)b; }
600 inline bool operator<=(const uint256& a, const uint256& b) { return (base_uint256)a <= (base_uint256)b;
    }
601 inline bool operator>(const uint256& a, const uint256& b) { return (base_uint256)a > (base_uint256)b; }
602 inline bool operator>=(const uint256& a, const uint256& b) { return (base_uint256)a >= (base_uint256)b;
    }
603 inline bool operator==(const uint256& a, const uint256& b) { return (base_uint256)a == (base_uint256)b;
    }
604 inline bool operator!=(const uint256& a, const uint256& b) { return (base_uint256)a != (base_uint256)b;
    }
605 inline const uint256 operator^(const uint256& a, const uint256& b) { return (base_uint256)a ^
    (base_uint256)b; }
606 inline const uint256 operator&(const uint256& a, const uint256& b) { return (base_uint256)a &
    (base_uint256)b; }
607 inline const uint256 operator|(const uint256& a, const uint256& b) { return (base_uint256)a |
    (base_uint256)b; }
608 inline const uint256 operator+(const uint256& a, const uint256& b) { return (base_uint256)a +
    (base_uint256)b; }
609 inline const uint256 operator-(const uint256& a, const uint256& b) { return (base_uint256)a -
    (base_uint256)b; }
610
611
612
613
614
615
616
617
618
619
620
621
622 inline int Testuint256AdHoc(vector<string> vArg)

```

`Testuint256AdHoc()` —Satoshi's test/debugging code. This function exercises the `uint256` implementation with various operations: increment, decrement, shifts, hex parsing. It prints results for manual verification. The code was left in the release—evidence that Satoshi tested thoroughly before shipping. Modern Bitcoin has proper unit tests, but this served its purpose in 2009.

```

623 {
624     uint256 g(0);
625
626
627     printf("%s\n", g.ToString().c_str());
628     g--; printf("g--\n");
629     printf("%s\n", g.ToString().c_str());
630     g--; printf("g--\n");
631     printf("%s\n", g.ToString().c_str());
632     g++; printf("g++\n");
633     printf("%s\n", g.ToString().c_str());
634     g++; printf("g++\n");
635     printf("%s\n", g.ToString().c_str());
636     g++; printf("g++\n");
637     printf("%s\n", g.ToString().c_str());
638     g++; printf("g++\n");
639     printf("%s\n", g.ToString().c_str());
640
641
642
643     uint256 a(7);
644     printf("a=7\n");
645     printf("%s\n", a.ToString().c_str());
646

```



```

647     uint256 b;
648     printf("b undefined\n");
649     printf("%s\n", b.ToString().c_str());
650     int c = 3;
651
652     a = c;
653     a.pn[3] = 15;
654     printf("%s\n", a.ToString().c_str());
655     uint256 k(c);
656
657     a = 5;
658     a.pn[3] = 15;
659     printf("%s\n", a.ToString().c_str());
660     b = 1;
661     b <=<= 52;
662
663     a |= b;
664
665     a ^= 0x500;
666
667     printf("a %s\n", a.ToString().c_str());
668
669     a = a | b | (uint256)0x1000;
670
671
672     printf("a %s\n", a.ToString().c_str());
673     printf("b %s\n", b.ToString().c_str());
674
675     a = 0xfffffffffe;
676     a.pn[4] = 9;
677
678     printf("%s\n", a.ToString().c_str());
679     a++;
680     printf("%s\n", a.ToString().c_str());
681     a++;
682     printf("%s\n", a.ToString().c_str());
683     a++;
684     printf("%s\n", a.ToString().c_str());
685     a++;
686     printf("%s\n", a.ToString().c_str());
687
688     a--;
689     printf("%s\n", a.ToString().c_str());
690     a--;
691     printf("%s\n", a.ToString().c_str());
692     a--;
693     printf("%s\n", a.ToString().c_str());
694     uint256 d = a--;
695     printf("%s\n", d.ToString().c_str());
696     printf("%s\n", a.ToString().c_str());
697     a--;
698     printf("%s\n", a.ToString().c_str());
699     a--;
700     printf("%s\n", a.ToString().c_str());
701
702     d = a;
703
704     printf("%s\n", d.ToString().c_str());
705     for (int i = uint256::WIDTH-1; i >= 0; i--) printf("%08x", d.pn[i]); printf("\n");
706
707     uint256 neg = d;
708     neg = ~neg;
709     printf("%s\n", neg.ToString().c_str());
710
711
712     uint256 e = uint256("0xABCDEF123abcdef12345678909832180000011111111");
713     printf("\n");
714     printf("%s\n", e.ToString().c_str());
715
716
717     printf("\n");
718     uint256 x1 = uint256("0xABCDEF123abcdef12345678909832180000011111111");
    
```

```

719     uint256 x2;
720     printf("%s\n", x1.ToString().c_str());
721     for (int i = 0; i < 270; i += 4)
722     {
723         x2 = x1 << i;
724         printf("%s\n", x2.ToString().c_str());
725     }
726
727     printf("\n");
728     printf("%s\n", x1.ToString().c_str());
729     for (int i = 0; i < 270; i += 4)
730     {
731         x2 = x1;
732         x2 >>= i;
733         printf("%s\n", x2.ToString().c_str());
734     }
735
736
737     for (int i = 0; i < 100; i++)
738     {
739         uint256 k = (~uint256(0) >> i);
740         printf("%s\n", k.ToString().c_str());
741     }
742
743     for (int i = 0; i < 100; i++)
744     {
745         uint256 k = (~uint256(0) << i);
746         printf("%s\n", k.ToString().c_str());
747     }
748
749     return (0);
750 }
751

```

SUMMARY

The `uint256` type is Bitcoin's cryptographic primitive. It represents numbers in a space so vast that randomly generated values will never collide—the probability of two random `uint256`s being equal is about 1 in 10^{77} , far less than the chance of any conceivable event. This is why Bitcoin can use hashes as unique identifiers without a central registry: cryptographic randomness guarantees uniqueness.

The implementation is straightforward but careful. Grade-school arithmetic scaled to 256 bits, with proper carry handling and efficient word-at-a-time comparisons. The code prioritizes correctness over micro-optimization—these operations aren't in the critical path (mining uses specialized SHA256 code). What matters is that the arithmetic is provably correct, because wrong results could cause consensus failures or security vulnerabilities.

The `uint160` for addresses represents an interesting design decision. Satoshi could have used full 256-bit hashes for addresses (as some later cryptocurrencies did), but 160 bits provides a better user experience with shorter addresses while maintaining enormous security margins. The birthday attack bound of 2^{80} is still astronomically secure—no one can find colliding addresses.

uiproject.fbp

11826 lines XML

THE INTERFACE SATOSHI BUILT

This is the blueprint for Bitcoin's original graphical interface—not code, but a visual design file created with **wxFormBuilder**, a drag-and-drop GUI designer for wxWidgets applications. Satoshi didn't just write a protocol; he designed an application that regular people could use.

The XML you see here defines every button, menu, text field, and dialog that made up the Bitcoin v0.1 experience on Windows. wxFormBuilder would read this file and generate the corresponding C++ code (`uibase.cpp` and `uibase.h`), which Satoshi then extended with Bitcoin-specific logic in `ui.cpp`.

What's remarkable is how *complete* the interface was from day one: a main window with balance display, transaction history, address book, send/receive dialogs, and even a "Generate Coins" menu option for CPU mining. This wasn't a command-line tool for cryptographers—it was software designed for a future where ordinary people would send digital cash to each other.

The design decisions embedded here reveal Satoshi's thinking about usability: the prominent display of "Your Bitcoin Address," the simple Send Coins flow, the transaction fee as an optional setting. Some features hint at plans that never fully materialized—like sending coins directly to IP addresses for "online transfer with comments and confirmation."

```

1<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2<wxFormBuilder_Project>
3<FileVersion major="1" minor="9" />
4<object class="Project" expanded="1">
5<property name="class_decoration"></property>
6<property name="code_generation">C++</property>
7<property name="disconnect_events">1</property>
8<property name="encoding">UTF-8</property>
9<property name="event_generation">connect</property>
10<property name="file_subbase"></property>
11<property name="first_id">1000</property>
12<property name="help_provider">none</property>
13<property name="internationalize">0</property>
14<property name="name"></property>
15<property name="namespace"></property>
16<property name="path"></property>
17<property name="precompiled_header"></property>
18<property name="relative_path">1</property>
19<property name="use_enum">0</property>
20<property name="use_microsoft_bom">0</property>
21<object class="Frame" expanded="0">
22<property name="bg">wxSYS_COLOUR_BTNFACE</property>
23<property name="center"></property>
24<property name="context_help"></property>
25<property name="enabled">1</property>
26<property name="extra_style"></property>
27<property name="fg"></property>
28<property name="font"></property>
29<property name="hidden">0</property>
30<property name="id">wxID_MAINFRAME</property>
31<property name="maximum_size"></property>
32<property name="minimum_size"></property>
33<property name="name">CMainFrameBase</property>
34<property name="pos"></property>
35<property name="size">705,484</property>
36<property
  name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
37<property name="subclass"></property>
38<property name="title">Bitcoin</property>
39<property name="tooltip"></property>
40<property name="window_extra_style"></property>
41<property name="window_name"></property>
42<property name="window_style">wxTAB_TRAVERSAL</property>
43<property name="xrc_skip_size">1</property>
44<event name="OnActivate"></event>
45<event name="OnActivateApp"></event>
46<event name="OnChar"></event>
47<event name="OnClose">OnClose</event>
48<event name="OnEnterWindow"></event>
49<event name="OnEraseBackground"></event>
50<event name="OnHibernate"></event>
51<event name="OnIconize"></event>
52<event name="OnIdle">OnIdle</event>
53<event name="OnKeyDown"></event>
54<event name="OnKeyUp"></event>
55<event name="OnKillFocus"></event>
56<event name="OnLeaveWindow"></event>
57<event name="OnLeftDClick"></event>
58<event name="OnLeftDown"></event>
59<event name="OnLeftUp"></event>
60<event name="OnMiddleDClick"></event>
61<event name="OnMiddleDown"></event>
62<event name="OnMiddleUp"></event>
63<event name="OnMotion"></event>
64<event name="OnMouseEvents">OnMouseEvents</event>
65<event name="OnMouseWheel"></event>
66<event name="OnPaint">OnPaint</event>
67<event name="OnRightDClick"></event>
68<event name="OnRightDown"></event>
69<event name="OnRightUp"></event>
70<event name="OnSetFocus"></event>
71<event name="OnSize"></event>
72<event name="OnUpdateUI"></event>
73<object class="wxMenuBar" expanded="1">
74<property name="bg">240,240,240</property>
75<property name="context_help"></property>
76<property name="enabled">1</property>
77<property name="fg"></property>
78<property name="font"></property>
79<property name="hidden">0</property>
80<property name="id">wxID_ANY</property>
81<property name="label">MyMenuBar</property>
82<property name="maximum_size"></property>
83<property name="minimum_size"></property>
84<property name="name">m_menuBar</property>
85<property name="permission">protected</property>
86<property name="pos"></property>
87<property name="size"></property>
88<property name="style"></property>
89<property name="subclass"></property>
90<property name="tooltip"></property>
91<property name="window_extra_style"></property>
92<property name="window_name"></property>
93<property name="window_style"></property>
94<event name="OnChar"></event>
95<event name="OnEnterWindow"></event>
96<event name="OnEraseBackground"></event>
97<event name="OnKeyDown"></event>
98<event name="OnKeyUp"></event>
99<event name="OnKillFocus"></event>
100<event name="OnLeaveWindow"></event>
101<event name="OnLeftDClick"></event>
102<event name="OnLeftDown"></event>
103<event name="OnLeftUp"></event>
104<event name="OnMiddleDClick"></event>
105<event name="OnMiddleDown"></event>
106<event name="OnMiddleUp"></event>
107<event name="OnMotion"></event>
108<event name="OnMouseEvents"></event>
109<event name="OnMouseWheel"></event>
110<event name="OnPaint"></event>
111<event name="OnRightDClick"></event>
112<event name="OnRightDown"></event>
113<event name="OnRightUp"></event>
114<event name="OnSetFocus"></event>
115<event name="OnSize"></event>
116<event name="OnUpdateUI"></event>
117<object class="wxMenu" expanded="1">
118<property name="label">&File</property>
119<property name="name">m_menuFile</property>
120<property name="permission">protected</property>
121<object class="wxMenuItem" expanded="1">
122<property name="bitmap"></property>
123<property name="checked">0</property>
124<property name="enabled">1</property>
125<property name="help"></property>
126<property name="id">wxID_ANY</property>
127<property name="kind">wxITEM_NORMAL</property>
128<property name="label">E&xample</property>
129<property name="name">m_menuFileExit</property>
130<property name="permission">none</property>
131<property name="shortcut"></property>
132<property name="unchecked_bitmap"></property>
133<event name="OnMenuSelection">OnMenuFileExit</event>
134<event name="OnUpdateUI"></event>
135</object>
136</object>
137<object class="wxMenu" expanded="1">
138<property name="label">&Options</property>
139<property name="name">m_menuOptions</property>
140<property name="permission">public</property>
141<object class="wxMenuItem" expanded="1">
142<property name="bitmap"></property>
143<property name="checked">0</property>
144<property name="enabled">1</property>
145<property name="help"></property>
146<property name="id">wxID_OPTIONSGENERATEBITCOINS</property>
147<property name="kind">wxITEM_CHECK</property>
148<property name="label">&Generate Coins</property>
149<property name="name">m_menuOptionsGenerateBitcoins</property>
150<property name="permission">none</property>
151<property name="shortcut"></property>
152<property name="unchecked_bitmap"></property>
153<event name="OnMenuSelection">OnMenuOptionsGenerate</event>
154<event name="OnUpdateUI"></event>
155</object>
156<object class="wxMenuItem" expanded="1">
157<property name="bitmap"></property>
158<property name="checked">0</property>
159<property name="enabled">1</property>
160<property name="help"></property>
161<property name="id">wxID_ANY</property>
162<property name="kind">wxITEM_NORMAL</property>
163<property name="label">&Options...</property>
164<property name="name">m_menuOptionsOptions</property>
165<property name="permission">none</property>
166<property name="shortcut"></property>
167<property name="unchecked_bitmap"></property>
168<event name="OnMenuSelection">OnMenuOptionsOptions</event>
169<event name="OnUpdateUI"></event>
170</object>
171</object>
172<object class="wxMenu" expanded="1">
173<property name="label">&Help</property>
174<property name="name">m_menuHelp</property>
175<property name="permission">protected</property>
176<object class="wxMenuItem" expanded="1">
177<property name="bitmap"></property>
178<property name="checked">0</property>
179<property name="enabled">1</property>
180<property name="help"></property>
181<property name="id">wxID_ANY</property>
182<property name="kind">wxITEM_NORMAL</property>
183<property name="label">&About...</property>
184<property name="name">m_menuHelpAbout</property>
185<property name="permission">none</property>
186<property name="shortcut"></property>
187<property name="unchecked_bitmap"></property>
188<event name="OnMenuSelection">OnMenuHelpAbout</event>
189<event name="OnUpdateUI"></event>
190</object>
191</object>
192</object>
193<object class="wxToolBar" expanded="0">
194<property name="bg"></property>
195<property name="bitmapsize">20,20</property>
196<property name="context_help"></property>
197<property name="enabled">1</property>
198<property name="fg"></property>
199<property name="font">90,90,-1,70,0</property>
200<property name="hidden">0</property>
201<property name="id">wxID_ANY</property>
202<property name="margins"></property>
203<property name="maximum_size"></property>
204<property name="minimum_size"></property>
205<property name="name">m_toolBar</property>

```

```

206<property name="packing">1</property>
207<property name="permission">protected</property>
208<property name="pos"></property>
209<property name="separation">1</property>
210<property name="size">-1,</property>
211<property name="style">wxTB_FLAT|wxTB_HORZ_TEXT</property>
212<property name="subclass"></property>
213<property name="tooltip"></property>
214<property name="window_extra_style"></property>
215<property name="window_name"></property>
216<property name="window_style"></property>
217<event name="OnChar"></event>
218<event name="OnEnterWindow"></event>
219<event name="OnEraseBackground"></event>
220<event name="OnKeyDown"></event>
221<event name="OnKeyUp"></event>
222<event name="OnKillFocus"></event>
223<event name="OnLeaveWindow"></event>
224<event name="OnLeftDClick"></event>
225<event name="OnLeftDown"></event>
226<event name="OnLeftUp"></event>
227<event name="OnMiddleDClick"></event>
228<event name="OnMiddleDown"></event>
229<event name="OnMiddleUp"></event>
230<event name="OnMotion"></event>
231<event name="OnMouseEvents"></event>
232<event name="OnMouseWheel"></event>
233<event name="OnPaint"></event>
234<event name="OnRightDClick"></event>
235<event name="OnRightDown"></event>
236<event name="OnRightUp"></event>
237<event name="OnSetFocus"></event>
238<event name="OnSize"></event>
239<event name="OnUpdateUI"></event>
240<object class="tool" expanded="1">
241<property name="bitmap">send20; Load From Resource</property>
242<property name="id">wxID_BUTTONSEND</property>
243<property name="kind">wxITEM_NORMAL</property>
244<property name="label">&send; Send Coins</property>
245<property name="name">m_tool</property>
246<property name="statusbar"></property>
247<property name="tooltip"></property>
248<event name="OnMenuSelection"></event>
249<event name="OnToolClicked">OnButtonSend</event>
250<event name="OnToolEnter"></event>
251<event name="OnToolRClicked"></event>
252<event name="OnUpdateUI"></event>
253</object>
254<object class="tool" expanded="1">
255<property name="bitmap">addressbook20; Load From Resource</property>
256<property name="id">wxID_BUTTONRECEIVE</property>
257<property name="kind">wxITEM_NORMAL</property>
258<property name="label">&address; Address Book</property>
259<property name="name">m_tool2</property>
260<property name="statusbar"></property>
261<property name="tooltip"></property>
262<event name="OnMenuSelection"></event>
263<event name="OnToolClicked">OnButtonAddressBook</event>
264<event name="OnToolEnter"></event>
265<event name="OnToolRClicked"></event>
266<event name="OnUpdateUI"></event>
267</object>
268</object>
269<object class="wxStatusBar" expanded="1">
270<property name="bg">240,240,240</property>
271<property name="context_help"></property>
272<property name="enabled">1</property>
273<property name="fg"></property>
274<property name="fields">1</property>
275<property name="font"></property>
276<property name="hidden">0</property>
277<property name="id">wxID_ANY</property>
278<property name="maximum_size"></property>
279<property name="minimum_size"></property>
280<property name="name">m_statusBar</property>
281<property name="permission">protected</property>
282<property name="pos"></property>
283<property name="size"></property>
284<property name="style">wxST_SIZEGRIP</property>
285<property name="subclass"></property>
286<property name="tooltip"></property>
287<property name="window_extra_style"></property>
288<property name="window_name"></property>
289<property name="window_style"></property>
290<event name="OnChar"></event>
291<event name="OnEnterWindow"></event>
292<event name="OnEraseBackground"></event>
293<event name="OnKeyDown"></event>
294<event name="OnKeyUp"></event>
295<event name="OnKillFocus"></event>
296<event name="OnLeaveWindow"></event>
297<event name="OnLeftDClick"></event>
298<event name="OnLeftDown"></event>
299<event name="OnLeftUp"></event>
300<event name="OnMiddleDClick"></event>
301<event name="OnMiddleDown"></event>
302<event name="OnMiddleUp"></event>
303<event name="OnMotion"></event>
304<event name="OnMouseEvents"></event>
305<event name="OnMouseWheel"></event>
306<event name="OnPaint"></event>
307<event name="OnRightDClick"></event>
308<event name="OnRightDown"></event>
309<event name="OnRightUp"></event>
310<event name="OnSetFocus"></event>
311<event name="OnSize"></event>
312<event name="OnUpdateUI"></event>
313</object>
314<object class="wxBoxSizer" expanded="0">
315<property name="minimum_size"></property>
316<property name="name">bSizer2</property>
317<property name="orient">wxVERTICAL</property>
318<property name="permission">none</property>
319<object class="sizeritem" expanded="1">
320<property name="border">5</property>
321<property name="flag">wxEXPAND</property>
322<property name="proportion">0</property>
323<object class="spacer" expanded="1">
324<property name="height">2</property>
325<property name="permission">protected</property>
326<property name="width">0</property>
327</object>
328</object>
329<object class="sizeritem" expanded="1">
330<property name="border">5</property>
331<property name="flag">wxEXPAND|wxRIGHT|wxLEFT</property>
332<property name="proportion">0</property>
333<object class="wxBoxSizer" expanded="1">
334<property name="minimum_size"></property>
335<property name="name">bSizer8</property>
336<property name="orient">wxHORIZONTAL</property>
337<property name="permission">none</property>
338<object class="sizeritem" expanded="1">
339<property name="border">5</property>
340<property name="flag">wxALIGN_CENTER_VERTICAL|wxLEFT</property>
341<property name="proportion">0</property>
342<object class="wxStaticText" expanded="1">
343<property name="bg"></property>
344<property name="context_help"></property>
345<property name="enabled">1</property>
346<property name="fg"></property>
347<property name="font"></property>
348<property name="hidden">0</property>
349<property name="id">wxID_ANY</property>
350<property name="label">Your Bitcoin Address:</property>
351<property name="maximum_size"></property>
352<property name="minimum_size"></property>
353<property name="name">m_staticText32</property>
354<property name="permission">protected</property>
355<property name="pos"></property>
356<property name="size"></property>
357<property name="style"></property>
358<property name="subclass"></property>
359<property name="tooltip"></property>
360<property name="window_extra_style"></property>
361<property name="window_name"></property>
362<property name="window_style"></property>
363<property name="wrap">1</property>
364<event name="OnChar"></event>
365<event name="OnEnterWindow"></event>
366<event name="OnEraseBackground"></event>
367<event name="OnKeyDown"></event>
368<event name="OnKeyUp"></event>
369<event name="OnKillFocus"></event>
370<event name="OnLeaveWindow"></event>
371<event name="OnLeftDClick"></event>
372<event name="OnLeftDown"></event>
373<event name="OnLeftUp"></event>
374<event name="OnMiddleDClick"></event>
375<event name="OnMiddleDown"></event>
376<event name="OnMiddleUp"></event>
377<event name="OnMotion"></event>
378<event name="OnMouseEvents"></event>
379<event name="OnMouseWheel"></event>
380<event name="OnPaint"></event>
381<event name="OnRightDClick"></event>
382<event name="OnRightDown"></event>
383<event name="OnRightUp"></event>
384<event name="OnSetFocus"></event>
385<event name="OnSize"></event>
386<event name="OnUpdateUI"></event>
387</object>
388</object>
389<object class="sizeritem" expanded="1">
390<property name="border">5</property>
391<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
392<property name="proportion">0</property>
393<object class="wxTextCtrl" expanded="1">
394<property name="bg">wxSYS_COLOUR_MENU</property>
395<property name="context_help"></property>
396<property name="enabled">1</property>
397<property name="fg"></property>
398<property name="font"></property>
399<property name="hidden">0</property>
400<property name="id">wxID_TEXTCTRLADDRESS</property>
401<property name="maximum_size">-1,-1</property>
402<property name="maxlength">0</property>
403<property name="minimum_size">-1,-1</property>
404<property name="name">m_textCtrlAddress</property>
405<property name="permission">protected</property>
406<property name="pos"></property>
407<property name="size">250,-1</property>
408<property name="style">wxTE_READONLY</property>
409<property name="subclass"></property>
410<property name="tooltip"></property>

```

```

411<property name="value"></property>
412<property name="window_extra_style"></property>
413<property name="window_name"></property>
414<property name="window_style"></property>
415<event name="OnChar"></event>
416<event name="OnEnterWindow"></event>
417<event name="OnEraseBackground"></event>
418<event name="OnKeyDown"><OnKeyDown/></event>
419<event name="OnKeyUp"></event>
420<event name="OnKillFocus"></event>
421<event name="OnLeaveWindow"></event>
422<event name="OnLeftDClick"></event>
423<event name="OnLeftDown"></event>
424<event name="OnLeftUp"></event>
425<event name="OnMiddleClick"></event>
426<event name="OnMiddleDown"></event>
427<event name="OnMiddleUp"></event>
428<event name="OnMotion"></event>
429<event name="OnMouseEvents"><OnMouseEventsAddress/></event>
430<event name="OnMouseWheel"></event>
431<event name="OnPaint"></event>
432<event name="OnRightDClick"></event>
433<event name="OnRightDown"></event>
434<event name="OnRightUp"></event>
435<event name="OnSetFocus"><OnSetFocusAddress/></event>
436<event name="OnSize"></event>
437<event name="OnText"></event>
438<event name="OnTextEnter"></event>
439<event name="OnTextMaxLen"></event>
440<event name="OnTextURL"></event>
441<event name="OnUpdateUI"></event>
442</object>
443</object>
444<object class="sizeritem" expanded="1">
445<property name="border">5</property>
446<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT</property>
447<property name="proportion">0</property>
448<object class="wxButton" expanded="1">
449<property name="bg"></property>
450<property name="context_help"></property>
451<property name="default">0</property>
452<property name="enabled">1</property>
453<property name="fg"></property>
454<property name="font"></property>
455<property name="hidden">0</property>
456<property name="id">wxID_BUTTONCOPY</property>
457<property name="label">&Copy to Clipboard</property>
458<property name="maximum_size"></property>
459<property name="minimum_size"></property>
460<property name="name">m_buttonCopy</property>
461<property name="permission">protected</property>
462<property name="pos"></property>
463<property name="size"></property>
464<property name="style">wxBU_EXACTFIT</property>
465<property name="subclass"></property>
466<property name="tooltip"></property>
467<property name="window_extra_style"></property>
468<property name="window_name"></property>
469<property name="window_style"></property>
470<event name="OnButtonClick"><OnButtonCopy/></event>
471<event name="OnChar"></event>
472<event name="OnEnterWindow"></event>
473<event name="OnEraseBackground"></event>
474<event name="OnKeyDown"></event>
475<event name="OnKeyUp"></event>
476<event name="OnKillFocus"></event>
477<event name="OnLeaveWindow"></event>
478<event name="OnLeftDClick"></event>
479<event name="OnLeftDown"></event>
480<event name="OnLeftUp"></event>
481<event name="OnMiddleClick"></event>
482<event name="OnMiddleDown"></event>
483<event name="OnMiddleUp"></event>
484<event name="OnMotion"></event>
485<event name="OnMouseEvents"></event>
486<event name="OnMouseWheel"></event>
487<event name="OnPaint"></event>
488<event name="OnRightDClick"></event>
489<event name="OnRightDown"></event>
490<event name="OnRightUp"></event>
491<event name="OnSetFocus"></event>
492<event name="OnSize"></event>
493<event name="OnUpdateUI"></event>
494</object>
495</object>
496<object class="sizeritem" expanded="1">
497<property name="border">5</property>
498<property name="flag">wxRIGHT</property>
499<property name="proportion">0</property>
500<object class="wxButton" expanded="1">
501<property name="bg"></property>
502<property name="context_help"></property>
503<property name="default">0</property>
504<property name="enabled">1</property>
505<property name="fg"></property>
506<property name="font"></property>
507<property name="hidden">0</property>
508<property name="id">wxID_BUTTONCHANGE</property>
509<property name="label">C&hange...</property>
510<property name="maximum_size"></property>
511<property name="minimum_size"></property>
512<property name="name">m_button91</property>
513<property name="permission">protected</property>
514<property name="pos"></property>
515<property name="size"></property>
516<property name="style"></property>
517<property name="subclass"></property>
518<property name="tooltip"></property>
519<property name="window_extra_style"></property>
520<property name="window_name"></property>
521<property name="window_style"></property>
522<event name="OnButtonClick"><OnButtonChange/></event>
523<event name="OnChar"></event>
524<event name="OnEnterWindow"></event>
525<event name="OnEraseBackground"></event>
526<event name="OnKeyDown"></event>
527<event name="OnKeyUp"></event>
528<event name="OnKillFocus"></event>
529<event name="OnLeaveWindow"></event>
530<event name="OnLeftDClick"></event>
531<event name="OnLeftDown"></event>
532<event name="OnLeftUp"></event>
533<event name="OnMiddleClick"></event>
534<event name="OnMiddleDown"></event>
535<event name="OnMiddleUp"></event>
536<event name="OnMotion"></event>
537<event name="OnMouseEvents"></event>
538<event name="OnMouseWheel"></event>
539<event name="OnPaint"></event>
540<event name="OnRightDClick"></event>
541<event name="OnRightDown"></event>
542<event name="OnRightUp"></event>
543<event name="OnSetFocus"></event>
544<event name="OnSize"></event>
545<event name="OnUpdateUI"></event>
546</object>
547</object>
548<object class="sizeritem" expanded="1">
549<property name="border">5</property>
550<property name="flag">wxEXPAND</property>
551<property name="proportion">0</property>
552<object class="spacer" expanded="1">
553<property name="height">0</property>
554<property name="permission">protected</property>
555<property name="width">0</property>
556</object>
557</object>
558</object>
559</object>
560<object class="sizeritem" expanded="1">
561<property name="border">5</property>
562<property name="flag">wxEXPAND</property>
563<property name="proportion">0</property>
564<object class="wxBoxSizer" expanded="1">
565<property name="minimum_size"></property>
566<property name="name">Sizer3</property>
567<property name="orient">wxHORIZONTAL</property>
568<property name="permission">none</property>
569<object class="sizeritem" expanded="1">
570<property name="border">5</property>
571<property name="flag">wxEXPAND|wxALIGN_BOTTOM|wxALL</property>
572<property name="proportion">1</property>
573<object class="wxPanel" expanded="1">
574<property name="bg"></property>
575<property name="context_help"></property>
576<property name="enabled">1</property>
577<property name="fg"></property>
578<property name="font"></property>
579<property name="hidden">0</property>
580<property name="id">wxID_ANY</property>
581<property name="maximum_size"></property>
582<property name="minimum_size"></property>
583<property name="name">m_panel14</property>
584<property name="permission">protected</property>
585<property name="pos"></property>
586<property name="size"></property>
587<property name="subclass"></property>
588<property name="tooltip"></property>
589<property name="window_extra_style"></property>
590<property name="window_name"></property>
591<property name="window_style">wxTAB_TRAVERSAL</property>
592<event name="OnChar"></event>
593<event name="OnEnterWindow"></event>
594<event name="OnEraseBackground"></event>
595<event name="OnKeyDown"></event>
596<event name="OnKeyUp"></event>
597<event name="OnKillFocus"></event>
598<event name="OnLeaveWindow"></event>
599<event name="OnLeftDClick"></event>
600<event name="OnLeftDown"></event>
601<event name="OnLeftUp"></event>
602<event name="OnMiddleClick"></event>
603<event name="OnMiddleDown"></event>
604<event name="OnMiddleUp"></event>
605<event name="OnMotion"></event>
606<event name="OnMouseEvents"></event>
607<event name="OnMouseWheel"></event>
608<event name="OnPaint"></event>
609<event name="OnRightDClick"></event>
610<event name="OnRightDown"></event>
611<event name="OnRightUp"></event>
612<event name="OnSetFocus"></event>
613<event name="OnSize"></event>
614<event name="OnUpdateUI"></event>
615<object class="wxBoxSizer" expanded="1">
616<property name="minimum_size"></property>

```



```

617<property name="name">bSizer66</property>
618<property name="orient">wxHORIZONTAL</property>
619<property name="permission">none</property>
620<object class="sizeritem" expanded="1">
621<property name="border">5</property>
622<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT</property>
623<property name="proportion">0</property>
624<object class="wxStaticText" expanded="1">
625<property name="bg"></property>
626<property name="context_help"></property>
627<property name="enabled">1</property>
628<property name="fg"></property>
629<property name="font"></property>
630<property name="hidden">0</property>
631<property name="id">wxID_ANY</property>
632<property name="label">Balance</property>
633<property name="maximum_size"></property>
634<property name="minimum_size"></property>
635<property name="name">m_staticText41</property>
636<property name="permission">protected</property>
637<property name="pos"></property>
638<property name="size">1,15</property>
639<property name="style"></property>
640<property name="subclass"></property>
641<property name="tooltip"></property>
642<property name="window_extra_style"></property>
643<property name="window_name"></property>
644<property name="window_style"></property>
645<property name="wrap">1</property>
646<event name="OnChar"></event>
647<event name="OnEnterWindow"></event>
648<event name="OnEraseBackground"></event>
649<event name="OnKeyDown"></event>
650<event name="OnKeyUp"></event>
651<event name="OnKillFocus"></event>
652<event name="OnLeaveWindow"></event>
653<event name="OnLeftClick"></event>
654<event name="OnLeftDown"></event>
655<event name="OnLeftUp"></event>
656<event name="OnMiddleClick"></event>
657<event name="OnMiddleDown"></event>
658<event name="OnMiddleUp"></event>
659<event name="OnMotion"></event>
660<event name="OnMouseEvents"></event>
661<event name="OnMouseWheel"></event>
662<event name="OnPaint"></event>
663<event name="OnRightClick"></event>
664<event name="OnRightDown"></event>
665<event name="OnRightUp"></event>
666<event name="OnSetFocus"></event>
667<event name="OnSize"></event>
668<event name="OnUpdateUI"></event>
669</object>
670</object>
671<object class="sizeritem" expanded="1">
672<property name="border">5</property>
673<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
674<property name="proportion">0</property>
675<object class="wxStaticText" expanded="1">
676<property name="bg">240,240,240</property>
677<property name="context_help"></property>
678<property name="enabled">1</property>
679<property name="fg"></property>
680<property name="font">90,90,8,70,0</property>
681<property name="hidden">0</property>
682<property name="id">wxID_ANY</property>
683<property name="label"></property>
684<property name="maximum_size"></property>
685<property name="minimum_size"></property>
686<property name="name">m_staticTextBalance</property>
687<property name="permission">protected</property>
688<property name="pos"></property>
689<property name="size">120,15</property>
690<property name="style">wxALIGN_RIGHT|wxST_NO_AUTORESIZE</property>
691<property name="subclass"></property>
692<property name="tooltip"></property>
693<property name="window_extra_style"></property>
694<property name="window_name"></property>
695<property name="window_style"></property>
696<property name="wrap">1</property>
697<event name="OnChar"></event>
698<event name="OnEnterWindow"></event>
699<event name="OnEraseBackground"></event>
700<event name="OnKeyDown"></event>
701<event name="OnKeyUp"></event>
702<event name="OnKillFocus"></event>
703<event name="OnLeaveWindow"></event>
704<event name="OnLeftClick"></event>
705<event name="OnLeftDown"></event>
706<event name="OnLeftUp"></event>
707<event name="OnMiddleClick"></event>
708<event name="OnMiddleDown"></event>
709<event name="OnMiddleUp"></event>
710<event name="OnMotion"></event>
711<event name="OnMouseEvents"></event>
712<event name="OnMouseWheel"></event>
713<event name="OnPaint"></event>
714<event name="OnRightClick"></event>
715<event name="OnRightDown"></event>
716<event name="OnRightUp"></event>
717<event name="OnSetFocus"></event>
718<event name="OnSize"></event>
719<event name="OnUpdateUI"></event>
720</object>
721</object>
722</object>
723</object>
724</object>
725<object class="sizeritem" expanded="1">
726<property name="border">5</property>
727<property name="flag">wxEXPAND</property>
728<property name="proportion">0</property>
729<object class="spacer" expanded="1">
730<property name="height">0</property>
731<property name="permission">protected</property>
732<property name="width">0</property>
733</object>
734</object>
735<object class="sizeritem" expanded="1">
736<property name="border">5</property>
737<property name="flag">wxALIGN_BOTTOM|wxTOP|wxRIGHT|wxLEFT</property>
738<property name="proportion">0</property>
739<object class="wxChoice" expanded="1">
740<property name="bg"></property>
741<property name="choices">Received6quot; &quot; In Progress6quot; &quot;
  Received6quot; &quot; Sent6quot; &quot;
742<property name="context_help"></property>
743<property name="enabled">1</property>
744<property name="fg"></property>
745<property name="font"></property>
746<property name="hidden">1</property>
747<property name="id">wxID_ANY</property>
748<property name="maximum_size"></property>
749<property name="minimum_size"></property>
750<property name="name">m_choiceFilter</property>
751<property name="permission">protected</property>
752<property name="pos"></property>
753<property name="selection">0</property>
754<property name="size">110,1</property>
755<property name="subclass"></property>
756<property name="tooltip"></property>
757<property name="window_extra_style"></property>
758<property name="window_name"></property>
759<property name="window_style"></property>
760<event name="OnChar"></event>
761<event name="OnChoice"></event>
762<event name="OnEnterWindow"></event>
763<event name="OnEraseBackground"></event>
764<event name="OnKeyDown"></event>
765<event name="OnKeyUp"></event>
766<event name="OnKillFocus"></event>
767<event name="OnLeaveWindow"></event>
768<event name="OnLeftClick"></event>
769<event name="OnLeftDown"></event>
770<event name="OnLeftUp"></event>
771<event name="OnMiddleClick"></event>
772<event name="OnMiddleDown"></event>
773<event name="OnMiddleUp"></event>
774<event name="OnMotion"></event>
775<event name="OnMouseEvents"></event>
776<event name="OnMouseWheel"></event>
777<event name="OnPaint"></event>
778<event name="OnRightClick"></event>
779<event name="OnRightDown"></event>
780<event name="OnRightUp"></event>
781<event name="OnSetFocus"></event>
782<event name="OnSize"></event>
783<event name="OnUpdateUI"></event>
784</object>
785</object>
786</object>
787</object>
788<object class="sizeritem" expanded="1">
789<property name="border">5</property>
790<property name="flag">wxEXPAND</property>
791<property name="proportion">1</property>
792<object class="wxNotebook" expanded="1">
793<property name="bg"></property>
794<property name="bmpsize"></property>
795<property name="context_help"></property>
796<property name="enabled">1</property>
797<property name="fg"></property>
798<property name="font"></property>
799<property name="hidden">0</property>
800<property name="id">wxID_ANY</property>
801<property name="maximum_size"></property>
802<property name="minimum_size"></property>
803<property name="name">m_notebook</property>
804<property name="permission">protected</property>
805<property name="pos"></property>
806<property name="size"></property>
807<property name="style"></property>
808<property name="subclass"></property>
809<property name="tooltip"></property>
810<property name="window_extra_style"></property>
811<property name="window_name"></property>
812<property name="window_style"></property>
813<event name="OnChar"></event>
814<event name="OnEnterWindow"></event>
815<event name="OnEraseBackground"></event>
816<event name="OnKeyDown"></event>
817<event name="OnKeyUp"></event>
818<event name="OnKillFocus"></event>
819<event name="OnLeaveWindow"></event>
820<event name="OnLeftClick"></event>

```

```

821<event name="OnLeftDown"></event>
822<event name="OnLeftUp"></event>
823<event name="OnMiddleClick"></event>
824<event name="OnMiddleDown"></event>
825<event name="OnMiddleUp"></event>
826<event name="OnMotion"></event>
827<event name="OnMouseEvents"></event>
828<event name="OnMouseWheel"></event>
829<event name="OnNotebookPageChanging"></event>
830<event name="OnNotebookPageChanging"></event>
831<event name="OnPaint"></event>
832<event name="OnRightClick"></event>
833<event name="OnRightDown"></event>
834<event name="OnRightUp"></event>
835<event name="OnSetFocus"></event>
836<event name="OnSize"></event>
837<event name="OnUpdateUI"></event>
838<object class="notebookpage" expanded="1">
839<property name="bitmap"></property>
840<property name="label">All Transactions</property>
841<property name="select">0</property>
842<object class="wxPanel" expanded="1">
843<property name="bg"></property>
844<property name="context_help"></property>
845<property name="enabled">1</property>
846<property name="fg"></property>
847<property name="font"></property>
848<property name="hidden">0</property>
849<property name="id">wxID_ANY</property>
850<property name="maximum_size"></property>
851<property name="minimum_size"></property>
852<property name="name">m_panel7</property>
853<property name="permission">protected</property>
854<property name="pos"></property>
855<property name="size"></property>
856<property name="subclass"></property>
857<property name="tooltip"></property>
858<property name="window_extra_style"></property>
859<property name="window_name"></property>
860<property name="window_style">wxTAB_TRAVERSAL</property>
861<event name="OnChar"></event>
862<event name="OnEnterWindow"></event>
863<event name="OnEraseBackground"></event>
864<event name="OnKeyDown"></event>
865<event name="OnKeyUp"></event>
866<event name="OnKillFocus"></event>
867<event name="OnLeaveWindow"></event>
868<event name="OnLeftClick"></event>
869<event name="OnLeftDown"></event>
870<event name="OnLeftUp"></event>
871<event name="OnMiddleClick"></event>
872<event name="OnMiddleDown"></event>
873<event name="OnMiddleUp"></event>
874<event name="OnMotion"></event>
875<event name="OnMouseEvents"></event>
876<event name="OnMouseWheel"></event>
877<event name="OnPaint"></event>
878<event name="OnRightClick"></event>
879<event name="OnRightDown"></event>
880<event name="OnRightUp"></event>
881<event name="OnSetFocus"></event>
882<event name="OnSize"></event>
883<event name="OnUpdateUI"></event>
884<object class="wxBoxSizer" expanded="1">
885<property name="minimum_size"></property>
886<property name="name">bSizer157</property>
887<property name="orient">wxVERTICAL</property>
888<property name="permission">none</property>
889<object class="sizeritem" expanded="1">
890<property name="border">5</property>
891<property name="flag">wxEXPAND|wxALL</property>
892<property name="proportion">1</property>
893<object class="wxListCtrl" expanded="1">
894<property name="bg"></property>
895<property name="context_help"></property>
896<property name="enabled">1</property>
897<property name="fg"></property>
898<property name="font"></property>
899<property name="hidden">0</property>
900<property name="id">wxID_ANY</property>
901<property name="maximum_size"></property>
902<property name="minimum_size"></property>
903<property name="name">m_listCtrl</property>
904<property name="permission">public</property>
905<property name="pos"></property>
906<property name="size"></property>
907<property
name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_DESCENDING</p
roperty>
908<property name="subclass"></property>
909<property name="tooltip"></property>
910<property name="window_extra_style"></property>
911<property name="window_name"></property>
912<property name="window_style">wxALWAYS_SHOW_SB</property>
913<event name="OnChar"></event>
914<event name="OnEnterWindow"></event>
915<event name="OnEraseBackground"></event>
916<event name="OnKeyDown"></event>
917<event name="OnKeyUp"></event>
918<event name="OnKillFocus"></event>
919<event name="OnLeaveWindow"></event>
920<event name="OnLeftClick"></event>
921<event name="OnLeftDown"></event>
922<event name="OnLeftUp"></event>
923<event name="OnListBeginDrag"></event>
924<event name="OnListBeginLabelEdit"></event>
925<event name="OnListBeginDrag"></event>
926<event name="OnListCacheHint"></event>
927<event name="OnListColBeginDrag"></event>
928<event name="OnListColClick"></event>
929<event name="OnListColDragging"></event>
930<event name="OnListColEndDrag"></event>
931<event name="OnListColRightClick"></event>
932<event name="OnListDeleteAllItems"></event>
933<event name="OnListDeleteItem"></event>
934<event name="OnListEndLabelEdit"></event>
935<event name="OnListInsertItem"></event>
936<event
name="OnListItemActivated">OnListItemActivatedAllTransactions</event>
937<event name="OnListItemDeselected"></event>
938<event name="OnListItemFocused"></event>
939<event name="OnListItemMiddleClick"></event>
940<event name="OnListItemRightClick"></event>
941<event name="OnListItemSelected"></event>
942<event name="OnListKeyDown"></event>
943<event name="OnMiddleClick"></event>
944<event name="OnMiddleDown"></event>
945<event name="OnMiddleUp"></event>
946<event name="OnMotion"></event>
947<event name="OnMouseEvents"></event>
948<event name="OnMouseWheel"></event>
949<event name="OnPaint">OnPaintListCtrl</event>
950<event name="OnRightClick"></event>
951<event name="OnRightDown"></event>
952<event name="OnRightUp"></event>
953<event name="OnSetFocus"></event>
954<event name="OnSize"></event>
955<event name="OnUpdateUI"></event>
956</object>
957</object>
958</object>
959</object>
960</object>
961</object>
962</object>
963<object class="sizeritem" expanded="1">
964<property name="border">5</property>
965<property name="flag">wxEXPAND</property>
966<property name="proportion">1</property>
967<object class="wxBoxSizer" expanded="1">
968<property name="minimum_size"></property>
969<property name="name">bSizer_TabsForFutureUse</property>
970<property name="orient">wxVERTICAL</property>
971<property name="permission">none</property>
972<object class="sizeritem" expanded="1">
973<property name="border">5</property>
974<property name="flag">wxEXPAND | wxALL</property>
975<property name="proportion">1</property>
976<object class="wxPanel" expanded="1">
977<property name="bg"></property>
978<property name="context_help"></property>
979<property name="enabled">1</property>
980<property name="fg"></property>
981<property name="font"></property>
982<property name="hidden">1</property>
983<property name="id">wxID_ANY</property>
984<property name="maximum_size"></property>
985<property name="minimum_size"></property>
986<property name="name">m_panel6</property>
987<property name="permission">protected</property>
988<property name="pos"></property>
989<property name="size"></property>
990<property name="subclass"></property>
991<property name="tooltip"></property>
992<property name="window_extra_style"></property>
993<property name="window_name"></property>
994<property name="window_style">wxTAB_TRAVERSAL</property>
995<event name="OnChar"></event>
996<event name="OnEnterWindow"></event>
997<event name="OnEraseBackground"></event>
998<event name="OnKeyDown"></event>
999<event name="OnKeyUp"></event>
1000<event name="OnKillFocus"></event>
1001<event name="OnLeaveWindow"></event>
1002<event name="OnLeftClick"></event>
1003<event name="OnLeftDown"></event>
1004<event name="OnLeftUp"></event>
1005<event name="OnMiddleClick"></event>
1006<event name="OnMiddleDown"></event>
1007<event name="OnMiddleUp"></event>
1008<event name="OnMotion"></event>
1009<event name="OnMouseEvents"></event>
1010<event name="OnMouseWheel"></event>
1011<event name="OnPaint"></event>
1012<event name="OnRightClick"></event>
1013<event name="OnRightDown"></event>
1014<event name="OnRightUp"></event>
1015<event name="OnSetFocus"></event>
1016<event name="OnSize"></event>
1017<event name="OnUpdateUI"></event>
1018<object class="wxBoxSizer" expanded="1">
1019<property name="minimum_size"></property>
1020<property name="name">bSizer159</property>
1021<property name="orient">wxVERTICAL</property>
1022<property name="permission">none</property>

```

```

1023<object class="sizeritem" expanded="1">
1024<property name="border">5</property>
1025<property name="flag">wxALL|wxEXPAND</property>
1026<property name="proportion">1</property>
1027<object class="wxListCtrl" expanded="1">
1028<property name="bg"></property>
1029<property name="context_help"></property>
1030<property name="enabled">1</property>
1031<property name="fg"></property>
1032<property name="font"></property>
1033<property name="hidden">0</property>
1034<property name="id">wxID_ANY</property>
1035<property name="maximum_size"></property>
1036<property name="minimum_size"></property>
1037<property name="m_listCtrlScrows</property>
1038<property name="permission">public</property>
1039<property name="pos"></property>
1040<property name="size"></property>
1041<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
1042<property name="subclass"></property>
1043<property name="tooltip"></property>
1044<property name="window_extra_style"></property>
1045<property name="window_name"></property>
1046<property name="window_style"></property>
1047<event name="OnChar"></event>
1048<event name="OnEnterWindow"></event>
1049<event name="OnEraseBackground"></event>
1050<event name="OnKeyDown"></event>
1051<event name="OnKeyUp"></event>
1052<event name="OnKillFocus"></event>
1053<event name="OnLeaveWindow"></event>
1054<event name="OnLeftClick"></event>
1055<event name="OnLeftDown"></event>
1056<event name="OnLeftUp"></event>
1057<event name="OnListBeginDrag"></event>
1058<event name="OnListBeginLabelEdit"></event>
1059<event name="OnListBeginRDrag"></event>
1060<event name="OnListCacheHint"></event>
1061<event name="OnListColBeginDrag"></event>
1062<event name="OnListColClick"></event>
1063<event name="OnListColDragging"></event>
1064<event name="OnListColEndDrag"></event>
1065<event name="OnListColRightClick"></event>
1066<event name="OnListDeleteAllItems"></event>
1067<event name="OnListDeleteItem"></event>
1068<event name="OnListEndLabelEdit"></event>
1069<event name="OnListInsertItem"></event>
1070<event name="OnListItemActivated"></event>
1071<event name="OnListItemDeselected"></event>
1072<event name="OnListItemFocused"></event>
1073<event name="OnListItemMiddleClick"></event>
1074<event name="OnListItemRightClick"></event>
1075<event name="OnListItemSelected"></event>
1076<event name="OnListKeyDown"></event>
1077<event name="OnMiddleClick"></event>
1078<event name="OnMiddleDown"></event>
1079<event name="OnMiddleUp"></event>
1080<event name="OnMotion"></event>
1081<event name="OnMouseEvents"></event>
1082<event name="OnMouseWheel"></event>
1083<event name="OnPaint"></event>
1084<event name="OnRightClick"></event>
1085<event name="OnRightDown"></event>
1086<event name="OnRightUp"></event>
1087<event name="OnSetFocus"></event>
1088<event name="OnSize"></event>
1089<event name="OnUpdateUI"></event>
1090</object>
1091</object>
1092</object>
1093</object>
1094</object>
1095<object class="sizeritem" expanded="1">
1096<property name="border">5</property>
1097<property name="flag">wxEXPAND | wxALL</property>
1098<property name="proportion">1</property>
1099<object class="wxPanel" expanded="1">
1100<property name="bg"></property>
1101<property name="context_help"></property>
1102<property name="enabled">1</property>
1103<property name="fg"></property>
1104<property name="font"></property>
1105<property name="hidden">1</property>
1106<property name="id">wxID_ANY</property>
1107<property name="maximum_size"></property>
1108<property name="minimum_size"></property>
1109<property name="name">m_panel8</property>
1110<property name="permission">protected</property>
1111<property name="pos"></property>
1112<property name="size"></property>
1113<property name="subclass"></property>
1114<property name="tooltip"></property>
1115<property name="window_extra_style"></property>
1116<property name="window_name"></property>
1117<property name="window_style">wxTAB_TRAVERSAL</property>
1118<event name="OnChar"></event>
1119<event name="OnEnterWindow"></event>
1120<event name="OnEraseBackground"></event>
1121<event name="OnKeyDown"></event>
1122<event name="OnKeyUp"></event>
1123<event name="OnKillFocus"></event>
1124<event name="OnLeaveWindow"></event>
1125<event name="OnLeftClick"></event>
1126<event name="OnLeftDown"></event>
1127<event name="OnLeftUp"></event>
1128<event name="OnMiddleClick"></event>
1129<event name="OnMiddleDown"></event>
1130<event name="OnMiddleUp"></event>
1131<event name="OnMotion"></event>
1132<event name="OnMouseEvents"></event>
1133<event name="OnMouseWheel"></event>
1134<event name="OnPaint"></event>
1135<event name="OnRightClick"></event>
1136<event name="OnRightDown"></event>
1137<event name="OnRightUp"></event>
1138<event name="OnSetFocus"></event>
1139<event name="OnSize"></event>
1140<event name="OnUpdateUI"></event>
1141<object class="wxBoxSizer" expanded="1">
1142<property name="minimum_size"></property>
1143<property name="name">bSizer158</property>
1144<property name="orient">wxVERTICAL</property>
1145<property name="permission">none</property>
1146<object class="sizeritem" expanded="1">
1147<property name="border">5</property>
1148<property name="flag">wxALL|wxEXPAND</property>
1149<property name="proportion">1</property>
1150<object class="wxListCtrl" expanded="1">
1151<property name="bg"></property>
1152<property name="context_help"></property>
1153<property name="enabled">1</property>
1154<property name="fg"></property>
1155<property name="font"></property>
1156<property name="hidden">0</property>
1157<property name="id">wxID_ANY</property>
1158<property name="maximum_size"></property>
1159<property name="minimum_size"></property>
1160<property name="name">m_listCtrlOrdersSent</property>
1161<property name="permission">public</property>
1162<property name="pos"></property>
1163<property name="size"></property>
1164<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
1165<property name="subclass"></property>
1166<property name="tooltip"></property>
1167<property name="window_extra_style"></property>
1168<property name="window_name"></property>
1169<property name="window_style"></property>
1170<event name="OnChar"></event>
1171<event name="OnEnterWindow"></event>
1172<event name="OnEraseBackground"></event>
1173<event name="OnKeyDown"></event>
1174<event name="OnKeyUp"></event>
1175<event name="OnKillFocus"></event>
1176<event name="OnLeaveWindow"></event>
1177<event name="OnLeftClick"></event>
1178<event name="OnLeftDown"></event>
1179<event name="OnLeftUp"></event>
1180<event name="OnListBeginDrag"></event>
1181<event name="OnListBeginLabelEdit"></event>
1182<event name="OnListBeginRDrag"></event>
1183<event name="OnListCacheHint"></event>
1184<event name="OnListColBeginDrag"></event>
1185<event name="OnListColClick"></event>
1186<event name="OnListColDragging"></event>
1187<event name="OnListColEndDrag"></event>
1188<event name="OnListColRightClick"></event>
1189<event name="OnListDeleteAllItems"></event>
1190<event name="OnListDeleteItem"></event>
1191<event name="OnListEndLabelEdit"></event>
1192<event name="OnListInsertItem"></event>
1193<event name="OnListItemActivated"></event>
1194<event name="OnListItemDeselected"></event>
1195<event name="OnListItemFocused"></event>
1196<event name="OnListItemMiddleClick"></event>
1197<event name="OnListItemRightClick"></event>
1198<event name="OnListItemSelected"></event>
1199<event name="OnListKeyDown"></event>
1200<event name="OnMiddleClick"></event>
1201<event name="OnMiddleDown"></event>
1202<event name="OnMiddleUp"></event>
1203<event name="OnMotion"></event>
1204<event name="OnMouseEvents"></event>
1205<event name="OnMouseWheel"></event>
1206<event name="OnPaint"></event>
1207<event name="OnRightClick"></event>
1208<event name="OnRightDown"></event>
1209<event name="OnRightUp"></event>
1210<event name="OnSetFocus"></event>
1211<event name="OnSize"></event>
1212<event name="OnUpdateUI"></event>
1213</object>
1214</object>
1215</object>
1216</object>
1217</object>
1218<object class="sizeritem" expanded="1">
1219<property name="border">5</property>
1220<property name="flag">wxEXPAND | wxALL</property>
1221<property name="proportion">1</property>
1222<object class="wxPanel" expanded="1">
1223<property name="bg"></property>
1224<property name="context_help"></property>
1225<property name="enabled">1</property>
1226<property name="fg"></property>
1227<property name="font"></property>

```

```

1228<property name="hidden">1</property>
1229<property name="id">wxID_ANY</property>
1230<property name="maximum_size"></property>
1231<property name="minimum_size"></property>
1232<property name="name">m_panel16</property>
1233<property name="permission">protected</property>
1234<property name="pos"></property>
1235<property name="size"></property>
1236<property name="subclass"></property>
1237<property name="tooltip"></property>
1238<property name="window_extra_style"></property>
1239<property name="window_name"></property>
1240<property name="window_style">wxTAB_TRAVERSAL</property>
1241<event name="OnChar"></event>
1242<event name="OnEnterWindow"></event>
1243<event name="OnEraseBackground"></event>
1244<event name="OnKeyDown"></event>
1245<event name="OnKeyUp"></event>
1246<event name="OnKillFocus"></event>
1247<event name="OnLeaveWindow"></event>
1248<event name="OnLeftDClick"></event>
1249<event name="OnLeftDown"></event>
1250<event name="OnLeftUp"></event>
1251<event name="OnMiddleDClick"></event>
1252<event name="OnMiddleDown"></event>
1253<event name="OnMiddleUp"></event>
1254<event name="OnMotion"></event>
1255<event name="OnMouseEvents"></event>
1256<event name="OnMouseWheel"></event>
1257<event name="OnPaint"></event>
1258<event name="OnRightDClick"></event>
1259<event name="OnRightDown"></event>
1260<event name="OnRightUp"></event>
1261<event name="OnSetFocus"></event>
1262<event name="OnSize"></event>
1263<event name="OnUpdateUI"></event>
1264<object class="wxBoxSizer" expanded="1">
1265<property name="minimum_size"></property>
1266<property name="name">bsizer16</property>
1267<property name="orient">wxVERTICAL</property>
1268<property name="permission">none</property>
1269<object class="sizeritem" expanded="1">
1270<property name="border">5</property>
1271<property name="flag">wxALL|wxEXPAND</property>
1272<property name="proportion">1</property>
1273<object class="wxListCtrl" expanded="1">
1274<property name="bg"></property>
1275<property name="context_help"></property>
1276<property name="enabled">1</property>
1277<property name="fg"></property>
1278<property name="font"></property>
1279<property name="hidden">0</property>
1280<property name="id">wxID_ANY</property>
1281<property name="maximum_size"></property>
1282<property name="minimum_size"></property>
1283<property name="name">m_listCtrlProductsSent</property>
1284<property name="permission">public</property>
1285<property name="pos"></property>
1286<property name="size"></property>
1287<property name="style">wxC_NO_SORT_HEADER|wxLC_REPORT</property>
1288<property name="subclass"></property>
1289<property name="tooltip"></property>
1290<property name="window_extra_style"></property>
1291<property name="window_name"></property>
1292<property name="window_style"></property>
1293<event name="OnChar"></event>
1294<event name="OnEnterWindow"></event>
1295<event name="OnEraseBackground"></event>
1296<event name="OnKeyDown"></event>
1297<event name="OnKeyUp"></event>
1298<event name="OnKillFocus"></event>
1299<event name="OnLeaveWindow"></event>
1300<event name="OnLeftDClick"></event>
1301<event name="OnLeftDown"></event>
1302<event name="OnLeftUp"></event>
1303<event name="OnListBeginDrag"></event>
1304<event name="OnListBeginLabelEdit"></event>
1305<event name="OnListBeginRDrag"></event>
1306<event name="OnListCacheHint"></event>
1307<event name="OnListColBeginDrag"></event>
1308<event name="OnListColClick"></event>
1309<event name="OnListColDragging"></event>
1310<event name="OnListColEndDrag"></event>
1311<event name="OnListColRightClick"></event>
1312<event name="OnListDeleteAllItems"></event>
1313<event name="OnListDeleteItem"></event>
1314<event name="OnListEndLabelEdit"></event>
1315<event name="OnListInsertItem"></event>
1316<event
    name="OnListItemActivated">OnListItemActivatedProductsSent</event>
1317<event name="OnListItemDeselected"></event>
1318<event name="OnListItemFocused"></event>
1319<event name="OnListItemMiddleClick"></event>
1320<event name="OnListItemRightClick"></event>
1321<event name="OnListItemSelected"></event>
1322<event name="OnListKeyDown"></event>
1323<event name="OnMiddleDClick"></event>
1324<event name="OnMiddleDown"></event>
1325<event name="OnMiddleUp"></event>
1326<event name="OnMotion"></event>
1327<event name="OnMouseEvents"></event>
1328<event name="OnMouseWheel"></event>
1329<event name="OnPaint"></event>
1330<event name="OnRightDClick"></event>
1331<event name="OnRightDown"></event>
1332<event name="OnRightUp"></event>
1333<event name="OnSetFocus"></event>
1334<event name="OnSize"></event>
1335<event name="OnUpdateUI"></event>
1336</object>
1337</object>
1338</object>
1339</object>
1340</object>
1341<object class="sizeritem" expanded="1">
1342<property name="border">5</property>
1343<property name="flag">wxEXPAND|wxALL</property>
1344<property name="proportion">1</property>
1345<object class="wxPanel" expanded="1">
1346<property name="bg"></property>
1347<property name="context_help"></property>
1348<property name="enabled">1</property>
1349<property name="fg"></property>
1350<property name="font"></property>
1351<property name="hidden">1</property>
1352<property name="id">wxID_ANY</property>
1353<property name="maximum_size"></property>
1354<property name="minimum_size"></property>
1355<property name="name">m_panel11</property>
1356<property name="permission">protected</property>
1357<property name="pos"></property>
1358<property name="size"></property>
1359<property name="subclass"></property>
1360<property name="tooltip"></property>
1361<property name="window_extra_style"></property>
1362<property name="window_name"></property>
1363<property name="window_style">wxTAB_TRAVERSAL</property>
1364<event name="OnChar"></event>
1365<event name="OnEnterWindow"></event>
1366<event name="OnEraseBackground"></event>
1367<event name="OnKeyDown"></event>
1368<event name="OnKeyUp"></event>
1369<event name="OnKillFocus"></event>
1370<event name="OnLeaveWindow"></event>
1371<event name="OnLeftDClick"></event>
1372<event name="OnLeftDown"></event>
1373<event name="OnLeftUp"></event>
1374<event name="OnMiddleDClick"></event>
1375<event name="OnMiddleDown"></event>
1376<event name="OnMiddleUp"></event>
1377<event name="OnMotion"></event>
1378<event name="OnMouseEvents"></event>
1379<event name="OnMouseWheel"></event>
1380<event name="OnPaint"></event>
1381<event name="OnRightDClick"></event>
1382<event name="OnRightDown"></event>
1383<event name="OnRightUp"></event>
1384<event name="OnSetFocus"></event>
1385<event name="OnSize"></event>
1386<event name="OnUpdateUI"></event>
1387<object class="wxBoxSizer" expanded="1">
1388<property name="minimum_size"></property>
1389<property name="name">bsizer161</property>
1390<property name="orient">wxVERTICAL</property>
1391<property name="permission">none</property>
1392<object class="sizeritem" expanded="1">
1393<property name="border">5</property>
1394<property name="flag">wxALL|wxEXPAND</property>
1395<property name="proportion">1</property>
1396<object class="wxListCtrl" expanded="1">
1397<property name="bg"></property>
1398<property name="context_help"></property>
1399<property name="enabled">1</property>
1400<property name="fg"></property>
1401<property name="font"></property>
1402<property name="hidden">0</property>
1403<property name="id">wxID_ANY</property>
1404<property name="maximum_size"></property>
1405<property name="minimum_size"></property>
1406<property name="name">m_listCtrlOrdersReceived</property>
1407<property name="permission">public</property>
1408<property name="pos"></property>
1409<property name="size"></property>
1410<property name="style">wxC_NO_SORT_HEADER|wxLC_REPORT</property>
1411<property name="subclass"></property>
1412<property name="tooltip"></property>
1413<property name="window_extra_style"></property>
1414<property name="window_name"></property>
1415<property name="window_style"></property>
1416<event name="OnChar"></event>
1417<event name="OnEnterWindow"></event>
1418<event name="OnEraseBackground"></event>
1419<event name="OnKeyDown"></event>
1420<event name="OnKeyUp"></event>
1421<event name="OnKillFocus"></event>
1422<event name="OnLeaveWindow"></event>
1423<event name="OnLeftDClick"></event>
1424<event name="OnLeftDown"></event>
1425<event name="OnLeftUp"></event>
1426<event name="OnListBeginDrag"></event>
1427<event name="OnListBeginLabelEdit"></event>
1428<event name="OnListBeginRDrag"></event>
1429<event name="OnListCacheHint"></event>
1430<event name="OnListColBeginDrag"></event>
1431<event name="OnListColClick"></event>
1432<event name="OnListColDragging"></event>

```

```

1433<event name="OnListColEndDrag"></event>
1434<event name="OnListColRightClick"></event>
1435<event name="OnListDeleteAllItems"></event>
1436<event name="OnListDeleteItem"></event>
1437<event name="OnListEndLabelEdit"></event>
1438<event name="OnListInsertItem"></event>
1439<event
  name="OnListItemActivated">OnListItemActivatedOrdersReceived</event>
1440<event name="OnListItemDeselected"></event>
1441<event name="OnListItemDeselected"></event>
1442<event name="OnListItemMiddleClick"></event>
1443<event name="OnListItemRightClick"></event>
1444<event name="OnListItemSelected"></event>
1445<event name="OnListKeyDown"></event>
1446<event name="OnMiddleClick"></event>
1447<event name="OnMiddleDown"></event>
1448<event name="OnMiddleUp"></event>
1449<event name="OnMotion"></event>
1450<event name="OnMouseEvents"></event>
1451<event name="OnMouseWheel"></event>
1452<event name="OnPaint"></event>
1453<event name="OnRightDClick"></event>
1454<event name="OnRightDown"></event>
1455<event name="OnRightUp"></event>
1456<event name="OnSetFocus"></event>
1457<event name="OnSize"></event>
1458<event name="OnUpdateUI"></event>
1459</object>
1460</object>
1461</object>
1462</object>
1463</object>
1464</object>
1465</object>
1466</object>
1467</object>
1468<object class="Dialog" expanded="0">
1469<property name="bg"></property>
1470<property name="center"></property>
1471<property name="context_help"></property>
1472<property name="enabled">1</property>
1473<property name="extra_style"></property>
1474<property name="fg"></property>
1475<property name="font"></property>
1476<property name="hidden">0</property>
1477<property name="id">wxID_ANY</property>
1478<property name="maximum_size"></property>
1479<property name="minimum_size"></property>
1480<property name="name">CTxDetailsDialogBase</property>
1481<property name="pos"></property>
1482<property name="size">620,450</property>
1483<property
  name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
1484<property name="subclass"></property>
1485<property name="title">Transaction Details</property>
1486<property name="tooltip"></property>
1487<property name="window_extra_style"></property>
1488<property name="window_name"></property>
1489<property name="window_style"></property>
1490<event name="OnActivate"></event>
1491<event name="OnActivateApp"></event>
1492<event name="OnChar"></event>
1493<event name="OnClose"></event>
1494<event name="OnEnterWindow"></event>
1495<event name="OnEraseBackground"></event>
1496<event name="OnHibernate"></event>
1497<event name="OnIconize"></event>
1498<event name="OnIdle"></event>
1499<event name="OnInitDialog"></event>
1500<event name="OnKeyDown"></event>
1501<event name="OnKeyUp"></event>
1502<event name="OnKillFocus"></event>
1503<event name="OnLeaveWindow"></event>
1504<event name="OnLeftDClick"></event>
1505<event name="OnLeftDown"></event>
1506<event name="OnLeftUp"></event>
1507<event name="OnMiddleDClick"></event>
1508<event name="OnMiddleDown"></event>
1509<event name="OnMiddleUp"></event>
1510<event name="OnMotion"></event>
1511<event name="OnMouseEvents"></event>
1512<event name="OnMouseWheel"></event>
1513<event name="OnPaint"></event>
1514<event name="OnRightDClick"></event>
1515<event name="OnRightDown"></event>
1516<event name="OnRightUp"></event>
1517<event name="OnSetFocus"></event>
1518<event name="OnSize"></event>
1519<event name="OnUpdateUI"></event>
1520<object class="wxBoxSizer" expanded="1">
1521<property name="minimum_size"></property>
1522<property name="name">sbSizer64</property>
1523<property name="orient">wxVERTICAL</property>
1524<property name="permission">none</property>
1525<object class="sizeritem" expanded="1">
1526<property name="border">5</property>
1527<property name="flag">wxEXPAND</property>
1528<property name="proportion">1</property>
1529<object class="wxBoxSizer" expanded="1">
1530<property name="minimum_size"></property>
1531<property name="name">sbSizer66</property>
1532<property name="orient">wxVERTICAL</property>
1533<property name="permission">none</property>
1534<object class="sizeritem" expanded="1">
1535<property name="border">5</property>
1536<property name="flag">wxALL|wxEXPAND</property>
1537<property name="proportion">1</property>
1538<object class="wxHtmlWindow" expanded="1">
1539<property name="bg"></property>
1540<property name="context_help"></property>
1541<property name="enabled">1</property>
1542<property name="fg"></property>
1543<property name="font"></property>
1544<property name="hidden">0</property>
1545<property name="id">wxID_ANY</property>
1546<property name="maximum_size"></property>
1547<property name="minimum_size"></property>
1548<property name="name">m_htmlWin</property>
1549<property name="permission">protected</property>
1550<property name="pos"></property>
1551<property name="size"></property>
1552<property name="style">wxHW_SCROLLBAR_AUTO</property>
1553<property name="subclass"></property>
1554<property name="tooltip"></property>
1555<property name="window_extra_style"></property>
1556<property name="window_name"></property>
1557<property name="window_style"></property>
1558<event name="OnChar"></event>
1559<event name="OnEnterWindow"></event>
1560<event name="OnEraseBackground"></event>
1561<event name="OnHtmlCellClicked"></event>
1562<event name="OnHtmlCellHover"></event>
1563<event name="OnHtmlLinkClicked"></event>
1564<event name="OnKeyDown"></event>
1565<event name="OnKeyUp"></event>
1566<event name="OnKillFocus"></event>
1567<event name="OnLeaveWindow"></event>
1568<event name="OnLeftDClick"></event>
1569<event name="OnLeftDown"></event>
1570<event name="OnLeftUp"></event>
1571<event name="OnMiddleDClick"></event>
1572<event name="OnMiddleDown"></event>
1573<event name="OnMiddleUp"></event>
1574<event name="OnMotion"></event>
1575<event name="OnMouseEvents"></event>
1576<event name="OnMouseWheel"></event>
1577<event name="OnPaint"></event>
1578<event name="OnRightDClick"></event>
1579<event name="OnRightDown"></event>
1580<event name="OnRightUp"></event>
1581<event name="OnSetFocus"></event>
1582<event name="OnSize"></event>
1583<event name="OnUpdateUI"></event>
1584</object>
1585</object>
1586</object>
1587</object>
1588<object class="sizeritem" expanded="1">
1589<property name="border">5</property>
1590<property name="flag">wxALIGN_RIGHT</property>
1591<property name="proportion">0</property>
1592<object class="wxBoxSizer" expanded="1">
1593<property name="minimum_size"></property>
1594<property name="name">sbSizer65</property>
1595<property name="orient">wxVERTICAL</property>
1596<property name="permission">none</property>
1597<object class="sizeritem" expanded="1">
1598<property name="border">5</property>
1599<property name="flag">wxALL</property>
1600<property name="proportion">0</property>
1601<object class="wxButton" expanded="1">
1602<property name="bg"></property>
1603<property name="context_help"></property>
1604<property name="default">0</property>
1605<property name="enabled">1</property>
1606<property name="fg"></property>
1607<property name="font"></property>
1608<property name="hidden">0</property>
1609<property name="id">wxID_OK</property>
1610<property name="label">OK</property>
1611<property name="maximum_size"></property>
1612<property name="minimum_size"></property>
1613<property name="name">m_buttonOK</property>
1614<property name="permission">protected</property>
1615<property name="pos"></property>
1616<property name="size">85,25</property>
1617<property name="style"></property>
1618<property name="subclass"></property>
1619<property name="tooltip"></property>
1620<property name="window_extra_style"></property>
1621<property name="window_name"></property>
1622<property name="window_style"></property>
1623<event name="OnButtonClicked">OnButtonOK</event>
1624<event name="OnChar"></event>
1625<event name="OnEnterWindow"></event>
1626<event name="OnEraseBackground"></event>
1627<event name="OnKeyDown"></event>
1628<event name="OnKeyUp"></event>
1629<event name="OnKillFocus"></event>
1630<event name="OnLeaveWindow"></event>
1631<event name="OnLeftDClick"></event>
1632<event name="OnLeftDown"></event>
1633<event name="OnLeftUp"></event>
1634<event name="OnMiddleDClick"></event>
1635<event name="OnMiddleDown"></event>
1636<event name="OnMiddleUp"></event>

```



```

1637<event name="OnMotion"></event>
1638<event name="OnMouseEvents"></event>
1639<event name="OnMouseWheel"></event>
1640<event name="OnPaint"></event>
1641<event name="OnRightDClick"></event>
1642<event name="OnRightDown"></event>
1643<event name="OnRightUp"></event>
1644<event name="OnSetFocus"></event>
1645<event name="OnSize"></event>
1646<event name="OnUpdateUI"></event>
1647</object>
1648</object>
1649</object>
1650</object>
1651</object>
1652</object>
1653<object class="Dialog" expanded="0">
1654<property name="bg"></property>
1655<property name="center"></property>
1656<property name="context_help"></property>
1657<property name="enabled">1</property>
1658<property name="extra_style"></property>
1659<property name="fg"></property>
1660<property name="font"></property>
1661<property name="hidden">0</property>
1662<property name="id">wxID_ANY</property>
1663<property name="maximum_size"></property>
1664<property name="minimum_size"></property>
1665<property name="name">OptionsDialogBase</property>
1666<property name="pos"></property>
1667<property name="size">500, 263</property>
1668<property name="style">wxDEFAULT_DIALOG_STYLE</property>
1669<property name="subclass"></property>
1670<property name="title">Options</property>
1671<property name="tooltip"></property>
1672<property name="window_extra_style"></property>
1673<property name="window_name"></property>
1674<property name="window_style"></property>
1675<event name="OnActivate"></event>
1676<event name="OnActivateApp"></event>
1677<event name="OnChar"></event>
1678<event name="OnClose"></event>
1679<event name="OnEnterWindow"></event>
1680<event name="OnEraseBackground"></event>
1681<event name="OnHibernate"></event>
1682<event name="OnIconize"></event>
1683<event name="OnIdle"></event>
1684<event name="OnInitDialog"></event>
1685<event name="OnKeyDown"></event>
1686<event name="OnKeyUp"></event>
1687<event name="OnKillFocus"></event>
1688<event name="OnLeaveWindow"></event>
1689<event name="OnLeftDClick"></event>
1690<event name="OnLeftDown"></event>
1691<event name="OnLeftUp"></event>
1692<event name="OnMiddleDClick"></event>
1693<event name="OnMiddleDown"></event>
1694<event name="OnMiddleUp"></event>
1695<event name="OnMotion"></event>
1696<event name="OnMouseEvents"></event>
1697<event name="OnMouseWheel"></event>
1698<event name="OnPaint"></event>
1699<event name="OnRightDClick"></event>
1700<event name="OnRightDown"></event>
1701<event name="OnRightUp"></event>
1702<event name="OnSetFocus"></event>
1703<event name="OnSize"></event>
1704<event name="OnUpdateUI"></event>
1705<object class="wxBoxSizer" expanded="1">
1706<property name="minimum_size"></property>
1707<property name="name">bSizer55</property>
1708<property name="orient">wxVERTICAL</property>
1709<property name="permission">none</property>
1710<object class="sizeritem" expanded="1">
1711<property name="border">5</property>
1712<property name="flag">wxEXPAND|wxLEFT</property>
1713<property name="proportion">1</property>
1714<object class="wxBoxSizer" expanded="1">
1715<property name="minimum_size"></property>
1716<property name="name">bSizer57</property>
1717<property name="orient">wxVERTICAL</property>
1718<property name="permission">none</property>
1719<object class="sizeritem" expanded="1">
1720<property name="border">5</property>
1721<property name="flag">wxEXPAND</property>
1722<property name="proportion">0</property>
1723<object class="spacer" expanded="1">
1724<property name="height">20</property>
1725<property name="permission">protected</property>
1726<property name="width">0</property>
1727</object>
1728</object>
1729<object class="sizeritem" expanded="1">
1730<property name="border">5</property>
1731<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
1732<property name="proportion">0</property>
1733<object class="wxStaticText" expanded="1">
1734<property name="bg"></property>
1735<property name="context_help"></property>
1736<property name="enabled">1</property>
1737<property name="fg"></property>
1738<property name="font"></property>
1739<property name="hidden">0</property>
1740<property name="id">wxID_ANY</property>
1741<property name="label">Optional transaction fee you give to the
nodes that process your transactions.</property>
1742<property name="maximum_size"></property>
1743<property name="minimum_size"></property>
1744<property name="name">m_staticText32</property>
1745<property name="permission">protected</property>
1746<property name="pos"></property>
1747<property name="size"></property>
1748<property name="style"></property>
1749<property name="subclass"></property>
1750<property name="tooltip"></property>
1751<property name="window_extra_style"></property>
1752<property name="window_name"></property>
1753<property name="window_style"></property>
1754<property name="wrap">1</property>
1755<event name="OnChar"></event>
1756<event name="OnEnterWindow"></event>
1757<event name="OnEraseBackground"></event>
1758<event name="OnKeyDown"></event>
1759<event name="OnKeyUp"></event>
1760<event name="OnKillFocus"></event>
1761<event name="OnLeaveWindow"></event>
1762<event name="OnLeftDClick"></event>
1763<event name="OnLeftDown"></event>
1764<event name="OnLeftUp"></event>
1765<event name="OnMiddleDClick"></event>
1766<event name="OnMiddleDown"></event>
1767<event name="OnMiddleUp"></event>
1768<event name="OnMotion"></event>
1769<event name="OnMouseEvents"></event>
1770<event name="OnMouseWheel"></event>
1771<event name="OnPaint"></event>
1772<event name="OnRightDClick"></event>
1773<event name="OnRightDown"></event>
1774<event name="OnRightUp"></event>
1775<event name="OnSetFocus"></event>
1776<event name="OnSize"></event>
1777<event name="OnUpdateUI"></event>
1778</object>
1779</object>
1780<object class="sizeritem" expanded="1">
1781<property name="border">5</property>
1782<property name="flag">wxEXPAND</property>
1783<property name="proportion">0</property>
1784<object class="wxBoxSizer" expanded="1">
1785<property name="minimum_size"></property>
1786<property name="name">bSizer56</property>
1787<property name="orient">wxHORIZONTAL</property>
1788<property name="permission">none</property>
1789<object class="sizeritem" expanded="1">
1790<property name="border">5</property>
1791<property
name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT</property>
1792<property name="proportion">0</property>
1793<object class="wxStaticText" expanded="1">
1794<property name="bg"></property>
1795<property name="context_help"></property>
1796<property name="enabled">1</property>
1797<property name="fg"></property>
1798<property name="font"></property>
1799<property name="hidden">0</property>
1800<property name="id">wxID_ANY</property>
1801<property name="label">Transaction fee</property>
1802<property name="maximum_size"></property>
1803<property name="minimum_size"></property>
1804<property name="name">m_staticText31</property>
1805<property name="permission">protected</property>
1806<property name="pos"></property>
1807<property name="size"></property>
1808<property name="style"></property>
1809<property name="subclass"></property>
1810<property name="tooltip"></property>
1811<property name="window_extra_style"></property>
1812<property name="window_name"></property>
1813<property name="window_style"></property>
1814<property name="wrap">1</property>
1815<event name="OnChar"></event>
1816<event name="OnEnterWindow"></event>
1817<event name="OnEraseBackground"></event>
1818<event name="OnKeyDown"></event>
1819<event name="OnKeyUp"></event>
1820<event name="OnKillFocus"></event>
1821<event name="OnLeaveWindow"></event>
1822<event name="OnLeftDClick"></event>
1823<event name="OnLeftDown"></event>
1824<event name="OnLeftUp"></event>
1825<event name="OnMiddleDClick"></event>
1826<event name="OnMiddleDown"></event>
1827<event name="OnMiddleUp"></event>
1828<event name="OnMotion"></event>
1829<event name="OnMouseEvents"></event>
1830<event name="OnMouseWheel"></event>
1831<event name="OnPaint"></event>
1832<event name="OnRightDClick"></event>
1833<event name="OnRightDown"></event>
1834<event name="OnRightUp"></event>
1835<event name="OnSetFocus"></event>
1836<event name="OnSize"></event>
1837<event name="OnUpdateUI"></event>
1838</object>
1839</object>
1840<object class="sizeritem" expanded="1">

```

```

1841<property name="border">5</property>
1842<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
1843<property name="proportion">0</property>
1844<object class="wxTextCtrl" expanded="1">
1845<property name="bg"></property>
1846<property name="context_help"></property>
1847<property name="enabled">1</property>
1848<property name="fg"></property>
1849<property name="font"></property>
1850<property name="hidden">0</property>
1851<property name="id">wxID_TRANSACTIONFEE</property>
1852<property name="maximum_size"></property>
1853<property name="maxlength">0</property>
1854<property name="minimum_size"></property>
1855<property name="name">m_textCtrlTransactionFee</property>
1856<property name="permission">protected</property>
1857<property name="pos"></property>
1858<property name="size">70,-1</property>
1859<property name="style"></property>
1860<property name="subclass"></property>
1861<property name="tooltip"></property>
1862<property name="value"></property>
1863<property name="window_extra_style"></property>
1864<property name="window_name"></property>
1865<property name="window_style"></property>
1866<event name="OnChar"></event>
1867<event name="OnEnterWindow"></event>
1868<event name="OnEraseBackground"></event>
1869<event name="OnKeyDown"></event>
1870<event name="OnKeyUp"></event>
1871<event name="OnKillFocus">OnKillFocusTransactionFee</event>
1872<event name="OnLeaveWindow"></event>
1873<event name="OnLeftClick"></event>
1874<event name="OnLeftDown"></event>
1875<event name="OnLeftUp"></event>
1876<event name="OnMiddleClick"></event>
1877<event name="OnMiddleDown"></event>
1878<event name="OnMiddleUp"></event>
1879<event name="OnMotion"></event>
1880<event name="OnMouseEvents"></event>
1881<event name="OnMouseWheel"></event>
1882<event name="OnPaint"></event>
1883<event name="OnRightClick"></event>
1884<event name="OnRightDown"></event>
1885<event name="OnRightUp"></event>
1886<event name="OnSetFocus"></event>
1887<event name="OnSize"></event>
1888<event name="OnText"></event>
1889<event name="OnTextEnter"></event>
1890<event name="OnTextMaxLen"></event>
1891<event name="OnTextURL"></event>
1892<event name="OnUpdateUI"></event>
1893</object>
1894</object>
1895</object>
1896</object>
1897</object>
1898</object>
1899<object class="sizeritem" expanded="1">
1900<property name="border">5</property>
1901<property name="flag">wxALIGN_RIGHT</property>
1902<property name="proportion">0</property>
1903<object class="wxBoxSizer" expanded="1">
1904<property name="minimum_size"></property>
1905<property name="name">s5Sizer</property>
1906<property name="orient">wxHORIZONTAL</property>
1907<property name="permission">none</property>
1908<object class="sizeritem" expanded="1">
1909<property name="border">5</property>
1910<property name="flag">wxALL</property>
1911<property name="proportion">0</property>
1912<object class="wxButton" expanded="1">
1913<property name="bg"></property>
1914<property name="context_help"></property>
1915<property name="default">0</property>
1916<property name="enabled">1</property>
1917<property name="fg"></property>
1918<property name="font"></property>
1919<property name="hidden">0</property>
1920<property name="id">wxID_OK</property>
1921<property name="label">OK</property>
1922<property name="maximum_size"></property>
1923<property name="minimum_size"></property>
1924<property name="name">m_buttonOK</property>
1925<property name="permission">protected</property>
1926<property name="pos"></property>
1927<property name="size">85,25</property>
1928<property name="style"></property>
1929<property name="subclass"></property>
1930<property name="tooltip"></property>
1931<property name="window_extra_style"></property>
1932<property name="window_name"></property>
1933<property name="window_style"></property>
1934<event name="OnButtonClick">OnButtonOK</event>
1935<event name="OnChar"></event>
1936<event name="OnEnterWindow"></event>
1937<event name="OnEraseBackground"></event>
1938<event name="OnKeyDown"></event>
1939<event name="OnKeyUp"></event>
1940<event name="OnKillFocus"></event>
1941<event name="OnLeaveWindow"></event>
1942<event name="OnLeftClick"></event>
1943<event name="OnLeftDown"></event>
1944<event name="OnLeftUp"></event>
1945<event name="OnMiddleClick"></event>
1946<event name="OnMiddleDown"></event>
1947<event name="OnMiddleUp"></event>
1948<event name="OnMotion"></event>
1949<event name="OnMouseEvents"></event>
1950<event name="OnMouseWheel"></event>
1951<event name="OnPaint"></event>
1952<event name="OnRightClick"></event>
1953<event name="OnRightDown"></event>
1954<event name="OnRightUp"></event>
1955<event name="OnSetFocus"></event>
1956<event name="OnSize"></event>
1957<event name="OnUpdateUI"></event>
1958</object>
1959</object>
1960<object class="sizeritem" expanded="1">
1961<property name="border">5</property>
1962<property name="flag">wxALL</property>
1963<property name="proportion">0</property>
1964<object class="wxButton" expanded="1">
1965<property name="bg"></property>
1966<property name="context_help"></property>
1967<property name="default">0</property>
1968<property name="enabled">1</property>
1969<property name="fg"></property>
1970<property name="font"></property>
1971<property name="hidden">0</property>
1972<property name="id">wxID_CANCEL</property>
1973<property name="label">Cancel</property>
1974<property name="maximum_size"></property>
1975<property name="minimum_size">85,25</property>
1976<property name="name">m_buttonCancel</property>
1977<property name="permission">protected</property>
1978<property name="pos"></property>
1979<property name="size">1,-1</property>
1980<property name="style"></property>
1981<property name="subclass"></property>
1982<property name="tooltip"></property>
1983<property name="window_extra_style"></property>
1984<property name="window_name"></property>
1985<property name="window_style"></property>
1986<event name="OnButtonClick">OnButtonCancel</event>
1987<event name="OnChar"></event>
1988<event name="OnEnterWindow"></event>
1989<event name="OnEraseBackground"></event>
1990<event name="OnKeyDown"></event>
1991<event name="OnKeyUp"></event>
1992<event name="OnKillFocus"></event>
1993<event name="OnLeaveWindow"></event>
1994<event name="OnLeftClick"></event>
1995<event name="OnLeftDown"></event>
1996<event name="OnLeftUp"></event>
1997<event name="OnMiddleClick"></event>
1998<event name="OnMiddleDown"></event>
1999<event name="OnMiddleUp"></event>
2000<event name="OnMotion"></event>
2001<event name="OnMouseEvents"></event>
2002<event name="OnMouseWheel"></event>
2003<event name="OnPaint"></event>
2004<event name="OnRightClick"></event>
2005<event name="OnRightDown"></event>
2006<event name="OnRightUp"></event>
2007<event name="OnSetFocus"></event>
2008<event name="OnSize"></event>
2009<event name="OnUpdateUI"></event>
2010</object>
2011</object>
2012</object>
2013</object>
2014</object>
2015</object>
2016<object class="Dialog" expanded="0">
2017<property name="bg"></property>
2018<property name="center"></property>
2019<property name="context_help"></property>
2020<property name="enabled">1</property>
2021<property name="extra_style"></property>
2022<property name="fg"></property>
2023<property name="font"></property>
2024<property name="hidden">0</property>
2025<property name="id">wxID_ANY</property>
2026<property name="maximum_size"></property>
2027<property name="minimum_size"></property>
2028<property name="name">AboutDialogBase</property>
2029<property name="pos"></property>
2030<property name="size">507,298</property>
2031<property name="style">wxDEFAULT_DIALOG_STYLE</property>
2032<property name="subclass"></property>
2033<property name="title">About Bitcoin</property>
2034<property name="tooltip"></property>
2035<property name="window_extra_style"></property>
2036<property name="window_name"></property>
2037<property name="window_style"></property>
2038<event name="OnActivate"></event>
2039<event name="OnActivateApp"></event>
2040<event name="OnChar"></event>
2041<event name="OnClose"></event>
2042<event name="OnEnterWindow"></event>
2043<event name="OnEraseBackground"></event>
2044<event name="OnHibernate"></event>
2045<event name="OnIconize"></event>
2046<event name="OnIdle"></event>

```

```

2047<event name="OnInitDialog"></event>
2048<event name="OnKeyDown"></event>
2049<event name="OnKeyUp"></event>
2050<event name="OnKillFocus"></event>
2051<event name="OnLeaveWindow"></event>
2052<event name="OnLeftDClick"></event>
2053<event name="OnLeftDown"></event>
2054<event name="OnLeftUp"></event>
2055<event name="OnMiddleClick"></event>
2056<event name="OnMiddleDown"></event>
2057<event name="OnMiddleUp"></event>
2058<event name="OnMotion"></event>
2059<event name="OnMouseEvents"></event>
2060<event name="OnMouseWheel"></event>
2061<event name="OnPaint"></event>
2062<event name="OnRightDClick"></event>
2063<event name="OnRightDown"></event>
2064<event name="OnRightUp"></event>
2065<event name="OnSetFocus"></event>
2066<event name="OnSize"></event>
2067<event name="OnUpdateUI"></event>
2068<object class="wxBoxSizer" expanded="0">
2069  <property name="minimum_size"></property>
2070  <property name="name">bSizer60</property>
2071  <property name="orient">wxVERTICAL</property>
2072  <property name="permission">none</property>
2073</object class="sizeritem" expanded="1">
2074<property name="border">5</property>
2075<property name="flag">wxEXPAND</property>
2076<property name="proportion">1</property>
2077<object class="wxBoxSizer" expanded="1">
2078  <property name="minimum_size"></property>
2079  <property name="name">bSizer62</property>
2080  <property name="orient">wxHORIZONTAL</property>
2081  <property name="permission">none</property>
2082<object class="sizeritem" expanded="1">
2083  <property name="border">5</property>
2084  <property name="flag">wxEXPAND</property>
2085  <property name="proportion">0</property>
2086<object class="spacer" expanded="1">
2087  <property name="height">0</property>
2088<property name="permission">protected</property>
2089<property name="width">60</property>
2090</object>
2091</object>
2092<object class="sizeritem" expanded="1">
2093  <property name="border">5</property>
2094  <property name="flag">wxEXPAND</property>
2095  <property name="proportion">1</property>
2096<object class="wxBoxSizer" expanded="1">
2097  <property name="minimum_size"></property>
2098  <property name="name">bSizer63</property>
2099  <property name="orient">wxVERTICAL</property>
2100  <property name="permission">none</property>
2101<object class="sizeritem" expanded="1">
2102  <property name="border">5</property>
2103  <property name="flag">wxEXPAND</property>
2104  <property name="proportion">0</property>
2105<object class="spacer" expanded="1">
2106  <property name="height">50</property>
2107  <property name="permission">protected</property>
2108<property name="width">0</property>
2109</object>
2110</object>
2111<object class="sizeritem" expanded="1">
2112  <property name="border">5</property>
2113  <property name="flag">wxEXPAND</property>
2114  <property name="proportion">0</property>
2115<object class="wxBoxSizer" expanded="1">
2116  <property name="minimum_size"></property>
2117  <property name="name">bSizer64</property>
2118  <property name="orient">wxHORIZONTAL</property>
2119  <property name="permission">none</property>
2120<object class="sizeritem" expanded="1">
2121  <property name="border">5</property>
2122<property
    name="flag">wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxLEFT</property>
2123<property name="proportion">0</property>
2124<object class="wxStaticText" expanded="1">
2125  <property name="bg"></property>
2126  <property name="context_help"></property>
2127  <property name="enabled">1</property>
2128  <property name="fg"></property>
2129  <property name="font">Tahoma, 90, 92, 10, 74, 0</property>
2130  <property name="hidden">0</property>
2131  <property name="id">wxID_ANY</property>
2132  <property name="label">Bitcoin </property>
2133  <property name="maximum_size"></property>
2134  <property name="minimum_size"></property>
2135  <property name="name">m_staticText40</property>
2136  <property name="permission">protected</property>
2137  <property name="pos"></property>
2138  <property name="size"></property>
2139  <property name="style"></property>
2140  <property name="subclass"></property>
2141  <property name="tooltip"></property>
2142  <property name="window_extra_style"></property>
2143  <property name="window_name"></property>
2144  <property name="window_style"></property>
2145  <property name="wrap">1</property>
2146<event name="OnChar"></event>
2147<event name="OnEnterWindow"></event>
2148<event name="OnEraseBackground"></event>
2149<event name="OnKeyDown"></event>
2150<event name="OnKeyUp"></event>
2151<event name="OnKillFocus"></event>
2152<event name="OnLeaveWindow"></event>
2153<event name="OnLeftDClick"></event>
2154<event name="OnLeftDown"></event>
2155<event name="OnLeftUp"></event>
2156<event name="OnMiddleDClick"></event>
2157<event name="OnMiddleDown"></event>
2158<event name="OnMiddleUp"></event>
2159<event name="OnMotion"></event>
2160<event name="OnMouseEvents"></event>
2161<event name="OnMouseWheel"></event>
2162<event name="OnPaint"></event>
2163<event name="OnRightDClick"></event>
2164<event name="OnRightDown"></event>
2165<event name="OnRightUp"></event>
2166<event name="OnSetFocus"></event>
2167<event name="OnSize"></event>
2168<event name="OnUpdateUI"></event>
2169</object>
2170</object>
2171<object class="sizeritem" expanded="1">
2172<property name="border">5</property>
2173<property
    name="flag">wxALIGN_BOTTOM|wxTOP|wxBOTTOM|wxRIGHT</property>
2174<property name="proportion">0</property>
2175<object class="wxStaticText" expanded="1">
2176  <property name="bg"></property>
2177  <property name="context_help"></property>
2178  <property name="enabled">1</property>
2179  <property name="fg"></property>
2180  <property name="font">Tahoma, 90, 90, 10, 74, 0</property>
2181  <property name="hidden">0</property>
2182  <property name="id">wxID_ANY</property>
2183  <property name="label">version</property>
2184  <property name="maximum_size"></property>
2185  <property name="minimum_size"></property>
2186  <property name="name">m_staticTextVersion</property>
2187  <property name="permission">public</property>
2188  <property name="pos"></property>
2189  <property name="size"></property>
2190  <property name="style"></property>
2191  <property name="subclass"></property>
2192  <property name="tooltip"></property>
2193  <property name="window_extra_style"></property>
2194  <property name="window_name"></property>
2195  <property name="window_style"></property>
2196  <property name="wrap">1</property>
2197<event name="OnChar"></event>
2198<event name="OnEnterWindow"></event>
2199<event name="OnEraseBackground"></event>
2200<event name="OnKeyDown"></event>
2201<event name="OnKeyUp"></event>
2202<event name="OnKillFocus"></event>
2203<event name="OnLeaveWindow"></event>
2204<event name="OnLeftDClick"></event>
2205<event name="OnLeftDown"></event>
2206<event name="OnLeftUp"></event>
2207<event name="OnMiddleDClick"></event>
2208<event name="OnMiddleDown"></event>
2209<event name="OnMiddleUp"></event>
2210<event name="OnMotion"></event>
2211<event name="OnMouseEvents"></event>
2212<event name="OnMouseWheel"></event>
2213<event name="OnPaint"></event>
2214<event name="OnRightDClick"></event>
2215<event name="OnRightDown"></event>
2216<event name="OnRightUp"></event>
2217<event name="OnSetFocus"></event>
2218<event name="OnSize"></event>
2219<event name="OnUpdateUI"></event>
2220</object>
2221</object>
2222</object>
2223</object>
2224<object class="sizeritem" expanded="1">
2225<property name="border">5</property>
2226<property name="flag">wxEXPAND</property>
2227<property name="proportion">0</property>
2228<object class="spacer" expanded="1">
2229  <property name="height">4</property>
2230<property name="permission">protected</property>
2231<property name="width">0</property>
2232</object>
2233</object>
2234<object class="sizeritem" expanded="1">
2235<property name="border">5</property>
2236<property name="flag">wxALL</property>
2237<property name="proportion">0</property>
2238<object class="wxStaticText" expanded="1">
2239  <property name="bg"></property>
2240  <property name="context_help"></property>
2241  <property name="enabled">1</property>
2242  <property name="fg"></property>
2243  <property name="font"></property>
2244  <property name="hidden">0</property>
2245  <property name="id">wxID_ANY</property>

```



```

2246<property name="label">Copyright © 2009 Satoshi
Nakamoto.&#x0A;&#x0A;This is experimental software. Do not rely on
it for actual financial transactions.&#x0A;&#x0A;Distributed under
the MIT/X11 software license, see the accompanying file license.txt
or http://www.opensource.org/licenses/mit-
license.php.&#x0A;&#x0A;This product includes software developed by
the OpenSSL Project for use in the OpenSSL Toolkit
(http://www.openssl.org/) and cryptographic software written by Eric
Young (eay@cryptsoft.com).</property>
2247<property name="maximum_size"></property>
2248<property name="minimum_size"></property>
2249<property name="name">m_staticTextMain</property>
2250<property name="permission">protected</property>
2251<property name="pos"></property>
2252<property name="size"></property>
2253<property name="style"></property>
2254<property name="subclass"></property>
2255<property name="tooltip"></property>
2256<property name="window_extra_style"></property>
2257<property name="window_name"></property>
2258<property name="window_style"></property>
2259<property name="wrap">400</property>
2260<event name="OnChar"></event>
2261<event name="OnEnterWindow"></event>
2262<event name="OnEraseBackground"></event>
2263<event name="OnKeyDown"></event>
2264<event name="OnKeyUp"></event>
2265<event name="OnKillFocus"></event>
2266<event name="OnLeaveWindow"></event>
2267<event name="OnLeftDClick"></event>
2268<event name="OnLeftDown"></event>
2269<event name="OnLeftUp"></event>
2270<event name="OnMiddleDClick"></event>
2271<event name="OnMiddleDown"></event>
2272<event name="OnMiddleUp"></event>
2273<event name="OnMotion"></event>
2274<event name="OnMouseEvents"></event>
2275<event name="OnMouseWheel"></event>
2276<event name="OnPaint"></event>
2277<event name="OnRightDClick"></event>
2278<event name="OnRightDown"></event>
2279<event name="OnRightUp"></event>
2280<event name="OnSetFocus"></event>
2281<event name="OnSize"></event>
2282<event name="OnUpdateUI"></event>
2283</object>
2284</object>
2285<object class="sizeritem" expanded="1">
2286<property name="border">5</property>
2287<property name="flag">wxEXPAND</property>
2288<property name="proportion">1</property>
2289<object class="spacer" expanded="1">
2290<property name="height">0</property>
2291<property name="permission">protected</property>
2292<property name="width">0</property>
2293</object>
2294</object>
2295</object>
2296</object>
2297</object>
2298</object>
2299<object class="sizeritem" expanded="1">
2300<property name="border">5</property>
2301<property name="flag">wxALIGN_RIGHT|wxEXPAND</property>
2302<property name="proportion">9</property>
2303<object class="wxBoxSizer" expanded="1">
2304<property name="minimum_size"></property>
2305<property name="name">bSizer61</property>
2306<property name="orient">wxHORIZONTAL</property>
2307<property name="permission">none</property>
2308<object class="sizeritem" expanded="1">
2309<property name="border">5</property>
2310<property name="flag">wxEXPAND</property>
2311<property name="proportion">1</property>
2312<object class="spacer" expanded="1">
2313<property name="height">0</property>
2314<property name="permission">protected</property>
2315<property name="width">0</property>
2316</object>
2317</object>
2318<object class="sizeritem" expanded="1">
2319<property name="border">5</property>
2320<property name="flag">wxALL</property>
2321<property name="proportion">0</property>
2322<object class="wxButton" expanded="1">
2323<property name="bg"></property>
2324<property name="context_help"></property>
2325<property name="default">0</property>
2326<property name="enabled">1</property>
2327<property name="fg"></property>
2328<property name="font"></property>
2329<property name="hidden">0</property>
2330<property name="id">wxID_OK</property>
2331<property name="label">OK</property>
2332<property name="maximum_size"></property>
2333<property name="minimum_size"></property>
2334<property name="name">m_buttonOK</property>
2335<property name="permission">protected</property>
2336<property name="pos"></property>
2337<property name="size">85,25</property>
2338<property name="style"></property>
2339<property name="subclass"></property>
2340<property name="tooltip"></property>
2341<property name="window_extra_style"></property>
2342<property name="window_name"></property>
2343<property name="window_style"></property>
2344<event name="OnButtonDClick">OnButtonOK</event>
2345<event name="OnChar"></event>
2346<event name="OnEnterWindow"></event>
2347<event name="OnEraseBackground"></event>
2348<event name="OnKeyDown"></event>
2349<event name="OnKeyUp"></event>
2350<event name="OnKillFocus"></event>
2351<event name="OnLeaveWindow"></event>
2352<event name="OnLeftDClick"></event>
2353<event name="OnLeftDown"></event>
2354<event name="OnLeftUp"></event>
2355<event name="OnMiddleDClick"></event>
2356<event name="OnMiddleDown"></event>
2357<event name="OnMiddleUp"></event>
2358<event name="OnMotion"></event>
2359<event name="OnMouseEvents"></event>
2360<event name="OnMouseWheel"></event>
2361<event name="OnPaint"></event>
2362<event name="OnRightDClick"></event>
2363<event name="OnRightDown"></event>
2364<event name="OnRightUp"></event>
2365<event name="OnSetFocus"></event>
2366<event name="OnSize"></event>
2367<event name="OnUpdateUI"></event>
2368</object>
2369</object>
2370</object>
2371</object>
2372</object>
2373</object>
2374<object class="Dialog" expanded="0">
2375<property name="bg"></property>
2376<property name="center"></property>
2377<property name="context_help"></property>
2378<property name="enabled">1</property>
2379<property name="extra_style"></property>
2380<property name="fg"></property>
2381<property name="font"></property>
2382<property name="hidden">0</property>
2383<property name="id">wxID_ANY</property>
2384<property name="maximum_size"></property>
2385<property name="minimum_size"></property>
2386<property name="name">CsendDialogBase</property>
2387<property name="pos"></property>
2388<property name="size">675,338</property>
2389<property
name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
2390<property name="subclass"></property>
2391<property name="title">Send Coins</property>
2392<property name="tooltip"></property>
2393<property name="window_extra_style"></property>
2394<property name="window_name"></property>
2395<property name="window_style"></property>
2396<event name="OnActivate"></event>
2397<event name="OnActivateApp"></event>
2398<event name="OnChar"></event>
2399<event name="OnClose"></event>
2400<event name="OnEnterWindow"></event>
2401<event name="OnEraseBackground"></event>
2402<event name="OnHibernate"></event>
2403<event name="OnIconize"></event>
2404<event name="OnIdle"></event>
2405<event name="OnInitDialog"></event>
2406<event name="OnKeyDown"></event>
2407<event name="OnKeyUp"></event>
2408<event name="OnKillFocus"></event>
2409<event name="OnLeaveWindow"></event>
2410<event name="OnLeftDClick"></event>
2411<event name="OnLeftDown"></event>
2412<event name="OnLeftUp"></event>
2413<event name="OnMiddleDClick"></event>
2414<event name="OnMiddleDown"></event>
2415<event name="OnMiddleUp"></event>
2416<event name="OnMotion"></event>
2417<event name="OnMouseEvents"></event>
2418<event name="OnMouseWheel"></event>
2419<event name="OnPaint"></event>
2420<event name="OnRightDClick"></event>
2421<event name="OnRightDown"></event>
2422<event name="OnRightUp"></event>
2423<event name="OnSetFocus"></event>
2424<event name="OnSize"></event>
2425<event name="OnUpdateUI"></event>
2426<object class="wxBoxSizer" expanded="1">
2427<property name="minimum_size"></property>
2428<property name="name">bSizer21</property>
2429<property name="orient">wxVERTICAL</property>
2430<property name="permission">none</property>
2431<object class="sizeritem" expanded="1">
2432<property name="border">5</property>
2433<property name="flag">wxEXPAND</property>
2434<property name="proportion">0</property>
2435<object class="spacer" expanded="1">
2436<property name="height">5</property>
2437<property name="permission">protected</property>
2438<property name="width">0</property>
2439</object>
2440</object>
2441<object class="sizeritem" expanded="1">
2442<property name="border">5</property>

```

```

2443<property name="flag">wxEXPAND|wxLEFT</property>
2444<property name="proportion">0</property>
2445<object class="wxFlexGridSizer" expanded="0">
2446<property name="cols">2</property>
2447<property name="flexible_direction">wxBOTH</property>
2448<property name="growablecols">1</property>
2449<property name="growablerows"></property>
2450<property name="hgap">0</property>
2451<property name="minimum_size"></property>
2452<property name="name">fgSizer1</property>
2453<property
  name="non_flexible_grow_mode">wxFLEX_GROWMODE_SPECIFIED</property>
2454<property name="permission">none</property>
2455<property name="rows">3</property>
2456<property name="vgap">0</property>
2457<object class="sizeritem" expanded="1">
2458<property name="border">5</property>
2459<property name="flag">wxEXPAND</property>
2460<property name="proportion">0</property>
2461<object class="spacer" expanded="1">
2462<property name="height">0</property>
2463<property name="permission">protected</property>
2464<property name="width">0</property>
2465</object>
2466</object>
2467<object class="sizeritem" expanded="1">
2468<property name="border">5</property>
2469<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
2470<property name="proportion">0</property>
2471<object class="wxStaticText" expanded="1">
2472<property name="bg"></property>
2473<property name="context_help"></property>
2474<property name="enabled">1</property>
2475<property name="fg"></property>
2476<property name="font"></property>
2477<property name="hidden">0</property>
2478<property name="id">wxID_ANY</property>
2479<property name="label">Enter the recipient's IP address (e.g.
  123.45.67) for online transfer with comments and confirmation,
  &#xA0;or bitcoin address (e.g. 1NS17iag9jJgTHD1VXjvLCEnZuQ3rJED9L)
  if recipient is not online.</property>
2480<property name="maximum_size"></property>
2481<property name="minimum_size"></property>
2482<property name="name">m_staticText14</property>
2483<property name="permission">protected</property>
2484<property name="pos"></property>
2485<property name="size"></property>
2486<property name="style"></property>
2487<property name="subclass"></property>
2488<property name="tooltip"></property>
2489<property name="window_extra_style"></property>
2490<property name="window_name"></property>
2491<property name="window_style">1</property>
2492<property name="wrap">1</property>
2493<event name="OnChar"></event>
2494<event name="OnEnterWindow"></event>
2495<event name="OnEraseBackground"></event>
2496<event name="OnKeyDown"></event>
2497<event name="OnKeyUp"></event>
2498<event name="OnKillFocus"></event>
2499<event name="OnLeaveWindow"></event>
2500<event name="OnLeftDClick"></event>
2501<event name="OnLeftDown"></event>
2502<event name="OnLeftUp"></event>
2503<event name="OnMiddleDClick"></event>
2504<event name="OnMiddleDown"></event>
2505<event name="OnMiddleUp"></event>
2506<event name="OnMotion"></event>
2507<event name="OnMouseEvents"></event>
2508<event name="OnMouseWheel"></event>
2509<event name="OnPaint"></event>
2510<event name="OnRightDClick"></event>
2511<event name="OnRightDown"></event>
2512<event name="OnRightUp"></event>
2513<event name="OnSetFocus"></event>
2514<event name="OnSize"></event>
2515<event name="OnUpdateUI"></event>
2516</object>
2517</object>
2518<object class="sizeritem" expanded="1">
2519<property name="border">5</property>
2520<property name="flag">wxEXPAND|wxLEFT</property>
2521<property name="proportion">1</property>
2522<object class="wxBoxSizer" expanded="1">
2523<property name="minimum_size">70,1</property>
2524<property name="name">bsizer47</property>
2525<property name="orient">wxHORIZONTAL</property>
2526<property name="permission">none</property>
2527<object class="sizeritem" expanded="1">
2528<property name="border">5</property>
2529<property name="flag">wxEXPAND</property>
2530<property name="proportion">1</property>
2531<object class="spacer" expanded="1">
2532<property name="height">0</property>
2533<property name="permission">protected</property>
2534<property name="width">0</property>
2535</object>
2536</object>
2537<object class="sizeritem" expanded="1">
2538<property name="border">5</property>
2539<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
2540<property name="proportion">0</property>
2541<object class="wxStaticBitmap" expanded="1">
2542<property name="bg"></property>
2543<property name="bitmap">check; Load From Icon Resource [-1; -1]
  </property>
2544<property name="context_help"></property>
2545<property name="enabled">1</property>
2546<property name="fg"></property>
2547<property name="font"></property>
2548<property name="hidden">0</property>
2549<property name="id">wxID_ANY</property>
2550<property name="maximum_size"></property>
2551<property name="minimum_size"></property>
2552<property name="name">m_bitmapCheckBox</property>
2553<property name="permission">protected</property>
2554<property name="pos"></property>
2555<property name="size">16,16</property>
2556<property name="subclass"></property>
2557<property name="tooltip"></property>
2558<property name="window_extra_style"></property>
2559<property name="window_name"></property>
2560<property name="window_style"></property>
2561<event name="OnChar"></event>
2562<event name="OnEnterWindow"></event>
2563<event name="OnEraseBackground"></event>
2564<event name="OnKeyDown"></event>
2565<event name="OnKeyUp"></event>
2566<event name="OnKillFocus"></event>
2567<event name="OnLeaveWindow"></event>
2568<event name="OnLeftDClick"></event>
2569<event name="OnLeftDown"></event>
2570<event name="OnLeftUp"></event>
2571<event name="OnMiddleDClick"></event>
2572<event name="OnMiddleDown"></event>
2573<event name="OnMiddleUp"></event>
2574<event name="OnMotion"></event>
2575<event name="OnMouseEvents"></event>
2576<event name="OnMouseWheel"></event>
2577<event name="OnPaint"></event>
2578<event name="OnRightDClick"></event>
2579<event name="OnRightDown"></event>
2580<event name="OnRightUp"></event>
2581<event name="OnSetFocus"></event>
2582<event name="OnSize"></event>
2583<event name="OnUpdateUI"></event>
2584</object>
2585</object>
2586<object class="sizeritem" expanded="1">
2587<property name="border">5</property>
2588<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT</property>
2589<property name="proportion">0</property>
2590<object class="wxStaticText" expanded="1">
2591<property name="bg"></property>
2592<property name="context_help"></property>
2593<property name="enabled">1</property>
2594<property name="fg"></property>
2595<property name="font"></property>
2596<property name="hidden">0</property>
2597<property name="id">wxID_ANY</property>
2598<property name="label">Pay &#xA0;To</property>
2599<property name="maximum_size"></property>
2600<property name="minimum_size"></property>
2601<property name="name">m_staticText36</property>
2602<property name="permission">protected</property>
2603<property name="pos"></property>
2604<property name="size">1,1</property>
2605<property name="style">wxALIGN_RIGHT</property>
2606<property name="subclass"></property>
2607<property name="tooltip"></property>
2608<property name="window_extra_style"></property>
2609<property name="window_name"></property>
2610<property name="window_style"></property>
2611<property name="wrap">1</property>
2612<event name="OnChar"></event>
2613<event name="OnEnterWindow"></event>
2614<event name="OnEraseBackground"></event>
2615<event name="OnKeyDown"></event>
2616<event name="OnKeyUp"></event>
2617<event name="OnKillFocus"></event>
2618<event name="OnLeaveWindow"></event>
2619<event name="OnLeftDClick"></event>
2620<event name="OnLeftDown"></event>
2621<event name="OnLeftUp"></event>
2622<event name="OnMiddleDClick"></event>
2623<event name="OnMiddleDown"></event>
2624<event name="OnMiddleUp"></event>
2625<event name="OnMotion"></event>
2626<event name="OnMouseEvents"></event>
2627<event name="OnMouseWheel"></event>
2628<event name="OnPaint"></event>
2629<event name="OnRightDClick"></event>
2630<event name="OnRightDown"></event>
2631<event name="OnRightUp"></event>
2632<event name="OnSetFocus"></event>
2633<event name="OnSize"></event>
2634<event name="OnUpdateUI"></event>
2635</object>
2636</object>
2637</object>
2638</object>
2639<object class="sizeritem" expanded="1">
2640<property name="border">5</property>
2641<property name="flag">wxEXPAND|wxRIGHT</property>
2642<property name="proportion">1</property>

```

```

2643<object class="wxBoxSizer" expanded="1">
2644<property name="minimum_size"></property>
2645<property name="name">sbSizer19</property>
2646<property name="orient">wxHORIZONTAL</property>
2647<property name="permission">none</property>
2648<object class="sizeritem" expanded="1">
2649<property name="border">5</property>
2650<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
2651<property name="proportion">1</property>
2652<object class="wxTextCtrl" expanded="1">
2653<property name="bg"></property>
2654<property name="context_help"></property>
2655<property name="enabled">1</property>
2656<property name="fg"></property>
2657<property name="font"></property>
2658<property name="hidden">0</property>
2659<property name="id">wxID_TEXTCTRLPAYTO</property>
2660<property name="maximum_size"></property>
2661<property name="maxlength">0</property>
2662<property name="minimum_size"></property>
2663<property name="name">m_textCtrlAddress</property>
2664<property name="permission">protected</property>
2665<property name="pos"></property>
2666<property name="size"></property>
2667<property name="style"></property>
2668<property name="subclass"></property>
2669<property name="tooltip"></property>
2670<property name="value"></property>
2671<property name="window_extra_style"></property>
2672<property name="window_name"></property>
2673<property name="window_style"></property>
2674<event name="OnChar"></event>
2675<event name="OnEnterWindow"></event>
2676<event name="OnEraseBackground"></event>
2677<event name="OnKeyDown">OnKeyDown</event>
2678<event name="OnKeyUp"></event>
2679<event name="OnKillFocus"></event>
2680<event name="OnLeaveWindow"></event>
2681<event name="OnLeftClick"></event>
2682<event name="OnLeftDown"></event>
2683<event name="OnLeftUp"></event>
2684<event name="OnMiddleClick"></event>
2685<event name="OnMiddleDown"></event>
2686<event name="OnMiddleUp"></event>
2687<event name="OnMotion"></event>
2688<event name="OnMouseEvents"></event>
2689<event name="OnMouseWheel"></event>
2690<event name="OnPaint"></event>
2691<event name="OnRightClick"></event>
2692<event name="OnRightDown"></event>
2693<event name="OnRightUp"></event>
2694<event name="OnSetFocus"></event>
2695<event name="OnSize"></event>
2696<event name="OnText">OnTextAddress</event>
2697<event name="OnTextEnter"></event>
2698<event name="OnTextMaxLen"></event>
2699<event name="OnTextURL"></event>
2700<event name="OnUpdateUI"></event>
2701</object>
2702</object>
2703<object class="sizeritem" expanded="1">
2704<property name="border">5</property>
2705<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT</property>
2706<property name="proportion">0</property>
2707<object class="wxButton" expanded="1">
2708<property name="bg"></property>
2709<property name="context_help"></property>
2710<property name="default">0</property>
2711<property name="enabled">1</property>
2712<property name="fg"></property>
2713<property name="font"></property>
2714<property name="hidden">0</property>
2715<property name="id">wxID_BUTTONPASTE</property>
2716<property name="label">&Paste</property>
2717<property name="maximum_size"></property>
2718<property name="minimum_size"></property>
2719<property name="name">m_buttonPaste</property>
2720<property name="permission">protected</property>
2721<property name="pos"></property>
2722<property name="size">1, -1</property>
2723<property name="style">wxBU_EXACTFIT</property>
2724<property name="subclass"></property>
2725<property name="tooltip"></property>
2726<property name="window_extra_style"></property>
2727<property name="window_name"></property>
2728<property name="window_style"></property>
2729<event name="OnButtonClick">OnButtonPaste</event>
2730<event name="OnChar"></event>
2731<event name="OnEnterWindow"></event>
2732<event name="OnEraseBackground"></event>
2733<event name="OnKeyDown"></event>
2734<event name="OnKeyUp"></event>
2735<event name="OnKillFocus"></event>
2736<event name="OnLeaveWindow"></event>
2737<event name="OnLeftClick"></event>
2738<event name="OnLeftDown"></event>
2739<event name="OnLeftUp"></event>
2740<event name="OnMiddleClick"></event>
2741<event name="OnMiddleDown"></event>
2742<event name="OnMiddleUp"></event>
2743<event name="OnMotion"></event>
2744<event name="OnMouseEvents"></event>
2745<event name="OnMouseWheel"></event>
2746<event name="OnPaint"></event>
2747<event name="OnRightClick"></event>
2748<event name="OnRightDown"></event>
2749<event name="OnRightUp"></event>
2750<event name="OnSetFocus"></event>
2751<event name="OnSize"></event>
2752<event name="OnUpdateUI"></event>
2753</object>
2754</object>
2755<object class="sizeritem" expanded="1">
2756<property name="border">5</property>
2757<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT</property>
2758<property name="proportion">0</property>
2759<object class="wxButton" expanded="1">
2760<property name="bg"></property>
2761<property name="context_help"></property>
2762<property name="default">0</property>
2763<property name="enabled">1</property>
2764<property name="fg"></property>
2765<property name="font"></property>
2766<property name="hidden">0</property>
2767<property name="id">wxID_BUTTONONADDRESSBOOK</property>
2768<property name="label">Address &Book...</property>
2769<property name="maximum_size"></property>
2770<property name="minimum_size"></property>
2771<property name="name">m_buttonAddress</property>
2772<property name="permission">protected</property>
2773<property name="pos"></property>
2774<property name="size"></property>
2775<property name="style"></property>
2776<property name="subclass"></property>
2777<property name="tooltip"></property>
2778<property name="window_extra_style"></property>
2779<property name="window_name"></property>
2780<property name="window_style"></property>
2781<event name="OnButtonClick">OnButtonAddressBook</event>
2782<event name="OnChar"></event>
2783<event name="OnEnterWindow"></event>
2784<event name="OnEraseBackground"></event>
2785<event name="OnKeyDown"></event>
2786<event name="OnKeyUp"></event>
2787<event name="OnKillFocus"></event>
2788<event name="OnLeaveWindow"></event>
2789<event name="OnLeftClick"></event>
2790<event name="OnLeftDown"></event>
2791<event name="OnLeftUp"></event>
2792<event name="OnMiddleClick"></event>
2793<event name="OnMiddleDown"></event>
2794<event name="OnMiddleUp"></event>
2795<event name="OnMotion"></event>
2796<event name="OnMouseEvents"></event>
2797<event name="OnMouseWheel"></event>
2798<event name="OnPaint"></event>
2799<event name="OnRightClick"></event>
2800<event name="OnRightDown"></event>
2801<event name="OnRightUp"></event>
2802<event name="OnSetFocus"></event>
2803<event name="OnSize"></event>
2804<event name="OnUpdateUI"></event>
2805</object>
2806</object>
2807</object>
2808</object>
2809<object class="sizeritem" expanded="1">
2810<property name="border">5</property>
2811<property name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxBOTTOM|wxLEFT|wxALIGN_RIGHT</property>
2812<property name="proportion">0</property>
2813<object class="wxStaticText" expanded="1">
2814<property name="bg"></property>
2815<property name="context_help"></property>
2816<property name="enabled">1</property>
2817<property name="fg"></property>
2818<property name="font"></property>
2819<property name="hidden">0</property>
2820<property name="id">wxID_ANY</property>
2821<property name="label">&Amount</property>
2822<property name="maximum_size"></property>
2823<property name="minimum_size"></property>
2824<property name="name">m_staticText19</property>
2825<property name="permission">protected</property>
2826<property name="pos"></property>
2827<property name="size">1, -1</property>
2828<property name="style">wxALIGN_RIGHT</property>
2829<property name="subclass"></property>
2830<property name="tooltip"></property>
2831<property name="window_extra_style"></property>
2832<property name="window_name"></property>
2833<property name="window_style"></property>
2834<property name="wrap">1</property>
2835<event name="OnChar"></event>
2836<event name="OnEnterWindow"></event>
2837<event name="OnEraseBackground"></event>
2838<event name="OnKeyDown"></event>
2839<event name="OnKeyUp"></event>
2840<event name="OnKillFocus"></event>
2841<event name="OnLeaveWindow"></event>
2842<event name="OnLeftClick"></event>
2843<event name="OnLeftDown"></event>
2844<event name="OnLeftUp"></event>
2845<event name="OnMiddleClick"></event>
2846<event name="OnMiddleDown"></event>

```

```

2847<event name="OnMiddleUp"></event>
2848<event name="OnMotion"></event>
2849<event name="OnMouseEvents"></event>
2850<event name="OnMouseWheel"></event>
2851<event name="OnPaint"></event>
2852<event name="OnRightDClick"></event>
2853<event name="OnRightDown"></event>
2854<event name="OnRightUp"></event>
2855<event name="OnSetFocus"></event>
2856<event name="OnSize"></event>
2857<event name="OnUpdateUI"></event>
2858</object>
2859</object>
2860<object class="sizeritem" expanded="1">
2861<property name="border">5</property>
2862<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
2863<property name="proportion">0</property>
2864<object class="wxTextCtrl" expanded="1">
2865<property name="bg"></property>
2866<property name="context_help"></property>
2867<property name="enabled">1</property>
2868<property name="fg"></property>
2869<property name="font">90,90,-1,70,0</property>
2870<property name="hidden">0</property>
2871<property name="id">wxID_TEXTCTRLAMOUNT</property>
2872<property name="maximum_size"></property>
2873<property name="maxlength">20</property>
2874<property name="minimum_size"></property>
2875<property name="name">m_textCtrlAmount</property>
2876<property name="permission">protected</property>
2877<property name="pos"></property>
2878<property name="size">145,-1</property>
2879<property name="style"></property>
2880<property name="subclass"></property>
2881<property name="tooltip"></property>
2882<property name="value"></property>
2883<property name="window_extra_style"></property>
2884<property name="window_name"></property>
2885<property name="window_style"></property>
2886<event name="OnChar"></event>
2887<event name="OnEnterWindow"></event>
2888<event name="OnEraseBackground"></event>
2889<event name="OnKeyDown">OnKeyDown</event>
2890<event name="OnKeyUp"></event>
2891<event name="OnKillFocus">OnKillFocusAmount</event>
2892<event name="OnLeaveWindow"></event>
2893<event name="OnLeftDClick"></event>
2894<event name="OnLeftDown"></event>
2895<event name="OnLeftUp"></event>
2896<event name="OnMiddleDClick"></event>
2897<event name="OnMiddleDown"></event>
2898<event name="OnMiddleUp"></event>
2899<event name="OnMotion"></event>
2900<event name="OnMouseEvents"></event>
2901<event name="OnMouseWheel"></event>
2902<event name="OnPaint"></event>
2903<event name="OnRightDClick"></event>
2904<event name="OnRightDown"></event>
2905<event name="OnRightUp"></event>
2906<event name="OnSetFocus"></event>
2907<event name="OnSize"></event>
2908<event name="OnText"></event>
2909<event name="OnTextEnter"></event>
2910<event name="OnTextMaxLen"></event>
2911<event name="OnTextURL"></event>
2912<event name="OnUpdateUI"></event>
2913</object>
2914</object>
2915<object class="sizeritem" expanded="1">
2916<property name="border">5</property>
2917<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxL
    EFT</property>
2918<property name="proportion">0</property>
2919<object class="wxStaticText" expanded="1">
2920<property name="bg"></property>
2921<property name="context_help"></property>
2922<property name="enabled">1</property>
2923<property name="fg"></property>
2924<property name="font"></property>
2925<property name="hidden">0</property>
2926<property name="id">wxID_ANY</property>
2927<property name="label">T&amp;P;ransfer:</property>
2928<property name="maximum_size"></property>
2929<property name="minimum_size"></property>
2930<property name="name">m_staticText20</property>
2931<property name="permission">protected</property>
2932<property name="pos"></property>
2933<property name="size">1,-1</property>
2934<property name="style">wxALIGN_RIGHT</property>
2935<property name="subclass"></property>
2936<property name="tooltip"></property>
2937<property name="window_extra_style"></property>
2938<property name="window_name"></property>
2939<property name="window_style"></property>
2940<property name="wrap">1</property>
2941<event name="OnChar"></event>
2942<event name="OnEnterWindow"></event>
2943<event name="OnEraseBackground"></event>
2944<event name="OnKeyDown"></event>
2945<event name="OnKeyUp"></event>
2946<event name="OnKillFocus"></event>
2947<event name="OnLeaveWindow"></event>
2948<event name="OnLeftDClick"></event>
2949<event name="OnLeftDown"></event>
2950<event name="OnLeftUp"></event>
2951<event name="OnMiddleDClick"></event>
2952<event name="OnMiddleDown"></event>
2953<event name="OnMiddleUp"></event>
2954<event name="OnMotion"></event>
2955<event name="OnMouseEvents"></event>
2956<event name="OnMouseWheel"></event>
2957<event name="OnPaint"></event>
2958<event name="OnRightDClick"></event>
2959<event name="OnRightDown"></event>
2960<event name="OnRightUp"></event>
2961<event name="OnSetFocus"></event>
2962<event name="OnSize"></event>
2963<event name="OnUpdateUI"></event>
2964</object>
2965</object>
2966<object class="sizeritem" expanded="1">
2967<property name="border">5</property>
2968<property name="flag">wxALIGN_CENTER_VERTICAL|wxALL</property>
2969<property name="proportion">0</property>
2970<object class="wxChoice" expanded="1">
2971<property name="bg"></property>
2972<property name="choices">Standard</property>
2973<property name="context_help"></property>
2974<property name="enabled">1</property>
2975<property name="fg"></property>
2976<property name="font"></property>
2977<property name="hidden">0</property>
2978<property name="id">wxID_CHOICETRANSFERTYPE</property>
2979<property name="maximum_size"></property>
2980<property name="minimum_size"></property>
2981<property name="name">m_choiceTransferTypes</property>
2982<property name="permission">protected</property>
2983<property name="pos"></property>
2984<property name="selection">0</property>
2985<property name="size"></property>
2986<property name="subclass"></property>
2987<property name="tooltip"></property>
2988<property name="window_extra_style"></property>
2989<property name="window_name"></property>
2990<property name="window_style"></property>
2991<event name="OnChar"></event>
2992<event name="OnChoice"></event>
2993<event name="OnEnterWindow"></event>
2994<event name="OnEraseBackground"></event>
2995<event name="OnKeyDown"></event>
2996<event name="OnKeyUp"></event>
2997<event name="OnKillFocus"></event>
2998<event name="OnLeaveWindow"></event>
2999<event name="OnLeftDClick"></event>
3000<event name="OnLeftDown"></event>
3001<event name="OnLeftUp"></event>
3002<event name="OnMiddleDClick"></event>
3003<event name="OnMiddleDown"></event>
3004<event name="OnMiddleUp"></event>
3005<event name="OnMotion"></event>
3006<event name="OnMouseEvents"></event>
3007<event name="OnMouseWheel"></event>
3008<event name="OnPaint"></event>
3009<event name="OnRightDClick"></event>
3010<event name="OnRightDown"></event>
3011<event name="OnRightUp"></event>
3012<event name="OnSetFocus"></event>
3013<event name="OnSize"></event>
3014<event name="OnUpdateUI"></event>
3015</object>
3016</object>
3017</object>
3018</object>
3019<object class="sizeritem" expanded="1">
3020<property name="border">5</property>
3021<property name="flag">wxEXPAND</property>
3022<property name="proportion">0</property>
3023<object class="wxBoxSizer" expanded="1">
3024<property name="minimum_size"></property>
3025<property name="name">bSizer672</property>
3026<property name="orient">wxHORIZONTAL</property>
3027<property name="permission">none</property>
3028<object class="sizeritem" expanded="1">
3029<property name="border">5</property>
3030<property name="flag">wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT</property>
3031<property name="proportion">1</property>
3032<object class="wxBoxSizer" expanded="1">
3033<property name="minimum_size"></property>
3034<property name="name">bSizer681</property>
3035<property name="orient">wxVERTICAL</property>
3036<property name="permission">none</property>
3037<object class="sizeritem" expanded="1">
3038<property name="border">5</property>
3039<property name="flag">wxBOTTOM|wxLEFT</property>
3040<property name="proportion">0</property>
3041<object class="wxStaticText" expanded="1">
3042<property name="bg"></property>
3043<property name="context_help"></property>
3044<property name="enabled">1</property>
3045<property name="fg"></property>
3046<property name="font"></property>
3047<property name="hidden">0</property>
3048<property name="id">wxID_ANY</property>
3049<property name="label">T&amp;P;ransfer:</property>
3050<property name="maximum_size"></property>

```

```

3051<property name="minimum_size"></property>
3052<property name="name">m_staticTextFrom</property>
3053<property name="permission">protected</property>
3054<property name="pos"></property>
3055<property name="size"></property>
3056<property name="style"></property>
3057<property name="subclass"></property>
3058<property name="tooltip"></property>
3059<property name="window_extra_style"></property>
3060<property name="window_name"></property>
3061<property name="window_style"></property>
3062<property name="wrap">1</property>
3063<event name="OnChar"></event>
3064<event name="OnEnterWindow"></event>
3065<event name="OnEraseBackground"></event>
3066<event name="OnKeyDown"></event>
3067<event name="OnKeyUp"></event>
3068<event name="OnKillFocus"></event>
3069<event name="OnLeaveWindow"></event>
3070<event name="OnLeftDClick"></event>
3071<event name="OnLeftDown"></event>
3072<event name="OnLeftUp"></event>
3073<event name="OnMiddleDClick"></event>
3074<event name="OnMiddleDown"></event>
3075<event name="OnMiddleUp"></event>
3076<event name="OnMotion"></event>
3077<event name="OnMouseEvents"></event>
3078<event name="OnMouseWheel"></event>
3079<event name="OnPaint"></event>
3080<event name="OnRightDClick"></event>
3081<event name="OnRightDown"></event>
3082<event name="OnRightUp"></event>
3083<event name="OnSetFocus"></event>
3084<event name="OnSize"></event>
3085<event name="OnUpdateUI"></event>
3086</object>
3087</object>
3088<object class="sizeritem" expanded="1">
3089<property name="border">5</property>
3090<property name="flag">wxLEFT|wxEXPAND</property>
3091<property name="proportion">0</property>
3092<object class="wxTextCtrl" expanded="1">
3093<property name="bg"></property>
3094<property name="context_help"></property>
3095<property name="enabled">1</property>
3096<property name="fg"></property>
3097<property name="font"></property>
3098<property name="hidden">0</property>
3099<property name="id">wxID_ANY</property>
3100<property name="maximum_size"></property>
3101<property name="maxlength">0</property>
3102<property name="minimum_size"></property>
3103<property name="name">m_textCtrlFrom</property>
3104<property name="permission">protected</property>
3105<property name="pos"></property>
3106<property name="size">1, -1</property>
3107<property name="style"></property>
3108<property name="subclass"></property>
3109<property name="tooltip"></property>
3110<property name="value"></property>
3111<property name="window_extra_style"></property>
3112<property name="window_name"></property>
3113<property name="window_style"></property>
3114<event name="OnChar"></event>
3115<event name="OnEnterWindow"></event>
3116<event name="OnEraseBackground"></event>
3117<event name="OnKeyDown">OnKeyDown</event>
3118<event name="OnKeyUp"></event>
3119<event name="OnKillFocus"></event>
3120<event name="OnLeaveWindow"></event>
3121<event name="OnLeftDClick"></event>
3122<event name="OnLeftDown"></event>
3123<event name="OnLeftUp"></event>
3124<event name="OnMiddleDClick"></event>
3125<event name="OnMiddleDown"></event>
3126<event name="OnMiddleUp"></event>
3127<event name="OnMotion"></event>
3128<event name="OnMouseEvents"></event>
3129<event name="OnMouseWheel"></event>
3130<event name="OnPaint"></event>
3131<event name="OnRightDClick"></event>
3132<event name="OnRightDown"></event>
3133<event name="OnRightUp"></event>
3134<event name="OnSetFocus"></event>
3135<event name="OnSize"></event>
3136<event name="OnText"></event>
3137<event name="OnTextEnter"></event>
3138<event name="OnTextMaxLen"></event>
3139<event name="OnTextURL"></event>
3140<event name="OnUpdateUI"></event>
3141</object>
3142</object>
3143</object>
3144</object>
3145</object>
3146</object>
3147<object class="sizeritem" expanded="1">
3148<property name="border">5</property>
3149<property name="flag">wxEXPAND</property>
3150<property name="proportion">1</property>
3151<object class="wxBoxSizer" expanded="1">
3152<property name="minimum_size"></property>
3153<property name="name">bSizer67</property>
3154<property name="orient">wxHORIZONTAL</property>
3155<property name="permission">none</property>
3156<object class="sizeritem" expanded="1">
3157<property name="border">5</property>
3158<property name="flag">wxEXPAND|wxBOTTOM|wxRIGHT|wxLEFT</property>
3159<property name="proportion">1</property>
3160<object class="wxBoxSizer" expanded="1">
3161<property name="minimum_size"></property>
3162<property name="name">bSizer68</property>
3163<property name="orient">wxVERTICAL</property>
3164<property name="permission">none</property>
3165<object class="sizeritem" expanded="1">
3166<property name="border">5</property>
3167<property name="flag">wxTOP|wxBOTTOM|wxLEFT</property>
3168<property name="proportion">0</property>
3169<object class="wxStaticText" expanded="1">
3170<property name="bg"></property>
3171<property name="context_help"></property>
3172<property name="enabled">1</property>
3173<property name="fg"></property>
3174<property name="font"></property>
3175<property name="hidden">0</property>
3176<property name="id">wxID_ANY</property>
3177<property name="label">6amp;Message</property>
3178<property name="maximum_size"></property>
3179<property name="minimum_size"></property>
3180<property name="name">m_staticTextMessage</property>
3181<property name="permission">protected</property>
3182<property name="pos"></property>
3183<property name="size"></property>
3184<property name="style"></property>
3185<property name="subclass"></property>
3186<property name="tooltip"></property>
3187<property name="window_extra_style"></property>
3188<property name="window_name"></property>
3189<property name="window_style"></property>
3190<property name="wrap">1</property>
3191<event name="OnChar"></event>
3192<event name="OnEnterWindow"></event>
3193<event name="OnEraseBackground"></event>
3194<event name="OnKeyDown"></event>
3195<event name="OnKeyUp"></event>
3196<event name="OnKillFocus"></event>
3197<event name="OnLeaveWindow"></event>
3198<event name="OnLeftDClick"></event>
3199<event name="OnLeftDown"></event>
3200<event name="OnLeftUp"></event>
3201<event name="OnMiddleDClick"></event>
3202<event name="OnMiddleDown"></event>
3203<event name="OnMiddleUp"></event>
3204<event name="OnMotion"></event>
3205<event name="OnMouseEvents"></event>
3206<event name="OnMouseWheel"></event>
3207<event name="OnPaint"></event>
3208<event name="OnRightDClick"></event>
3209<event name="OnRightDown"></event>
3210<event name="OnRightUp"></event>
3211<event name="OnSetFocus"></event>
3212<event name="OnSize"></event>
3213<event name="OnUpdateUI"></event>
3214</object>
3215</object>
3216<object class="sizeritem" expanded="1">
3217<property name="border">5</property>
3218<property name="flag">wxEXPAND|wxLEFT</property>
3219<property name="proportion">1</property>
3220<object class="wxTextCtrl" expanded="1">
3221<property name="bg"></property>
3222<property name="context_help"></property>
3223<property name="enabled">1</property>
3224<property name="fg"></property>
3225<property name="font"></property>
3226<property name="hidden">0</property>
3227<property name="id">wxID_ANY</property>
3228<property name="maximum_size"></property>
3229<property name="maxlength">0</property>
3230<property name="minimum_size"></property>
3231<property name="name">m_textCtrlMessage</property>
3232<property name="permission">protected</property>
3233<property name="pos"></property>
3234<property name="size"></property>
3235<property name="style">wxTE_MULTILINE</property>
3236<property name="subclass"></property>
3237<property name="tooltip"></property>
3238<property name="value"></property>
3239<property name="window_extra_style"></property>
3240<property name="window_name"></property>
3241<property name="window_style"></property>
3242<event name="OnChar"></event>
3243<event name="OnEnterWindow"></event>
3244<event name="OnEraseBackground"></event>
3245<event name="OnKeyDown">OnKeyDown</event>
3246<event name="OnKeyUp"></event>
3247<event name="OnKillFocus"></event>
3248<event name="OnLeaveWindow"></event>
3249<event name="OnLeftDClick"></event>
3250<event name="OnLeftDown"></event>
3251<event name="OnLeftUp"></event>
3252<event name="OnMiddleDClick"></event>
3253<event name="OnMiddleDown"></event>
3254<event name="OnMiddleUp"></event>
3255<event name="OnMotion"></event>
3256<event name="OnMouseEvents"></event>

```



```

3257<event name="OnMouseWheel"></event>
3258<event name="OnPaint"></event>
3259<event name="OnRightDClick"></event>
3260<event name="OnRightDown"></event>
3261<event name="OnRightUp"></event>
3262<event name="OnSetFocus"></event>
3263<event name="OnSize"></event>
3264<event name="OnText"></event>
3265<event name="OnTextEnter"></event>
3266<event name="OnTextMaxLen"></event>
3267<event name="OnTextURL"></event>
3268<event name="OnUpdateUI"></event>
3269</object>
3270</object>
3271</object>
3272</object>
3273</object>
3274</object>
3275<object class="sizeritem" expanded="1">
3276<property name="border">5</property>
3277<property name="flag">wxEXPAND</property>
3278<property name="proportion">0</property>
3279<object class="wxBoxSizer" expanded="1">
3280<property name="minimum_size"></property>
3281<property name="name">bSizer23</property>
3282<property name="orient">wxHORIZONTAL</property>
3283<property name="permission">none</property>
3284<object class="sizeritem" expanded="1">
3285<property name="border">5</property>
3286<property name="flag">wxEXPAND</property>
3287<property name="proportion">1</property>
3288<object class="spacer" expanded="1">
3289<property name="height">0</property>
3290<property name="permission">protected</property>
3291<property name="width">0</property>
3292</object>
3293</object>
3294<object class="sizeritem" expanded="1">
3295<property name="border">5</property>
3296<property name="flag">wxALL</property>
3297<property name="proportion">0</property>
3298<object class="wxButton" expanded="1">
3299<property name="bg"></property>
3300<property name="context_help"></property>
3301<property name="default">0</property>
3302<property name="enabled">1</property>
3303<property name="fg"></property>
3304<property name="font">90,90,-1,70,0</property>
3305<property name="hidden">0</property>
3306<property name="id">wxID_BUTTONSEND</property>
3307<property name="label">&amp;Send</property>
3308<property name="maximum_size"></property>
3309<property name="minimum_size">85,25</property>
3310<property name="name">m_buttonSend</property>
3311<property name="permission">protected</property>
3312<property name="pos"></property>
3313<property name="size">1,-1</property>
3314<property name="style"></property>
3315<property name="subclass"></property>
3316<property name="tooltip"></property>
3317<property name="window_extra_style"></property>
3318<property name="window_name"></property>
3319<property name="window_style"></property>
3320<event name="OnButtonClick">OnButtonSend</event>
3321<event name="OnChar"></event>
3322<event name="OnEnterWindow"></event>
3323<event name="OnEraseBackground"></event>
3324<event name="OnKeyDown"></event>
3325<event name="OnKeyUp"></event>
3326<event name="OnKillFocus"></event>
3327<event name="OnLeaveWindow"></event>
3328<event name="OnLeftDClick"></event>
3329<event name="OnLeftDown"></event>
3330<event name="OnLeftUp"></event>
3331<event name="OnMiddleDClick"></event>
3332<event name="OnMiddleDown"></event>
3333<event name="OnMiddleUp"></event>
3334<event name="OnMotion"></event>
3335<event name="OnMouseEvents"></event>
3336<event name="OnMouseWheel"></event>
3337<event name="OnPaint"></event>
3338<event name="OnRightDClick"></event>
3339<event name="OnRightDown"></event>
3340<event name="OnRightUp"></event>
3341<event name="OnSetFocus"></event>
3342<event name="OnSize"></event>
3343<event name="OnUpdateUI"></event>
3344</object>
3345</object>
3346<object class="sizeritem" expanded="1">
3347<property name="border">5</property>
3348<property name="flag">wxALL</property>
3349<property name="proportion">0</property>
3350<object class="wxButton" expanded="1">
3351<property name="bg"></property>
3352<property name="context_help"></property>
3353<property name="default">0</property>
3354<property name="enabled">1</property>
3355<property name="fg"></property>
3356<property name="font"></property>
3357<property name="hidden">0</property>
3358<property name="id">wxID_CANCEL</property>
3359<property name="label">Cancel</property>
3360<property name="maximum_size"></property>
3361<property name="minimum_size">85,25</property>
3362<property name="name">m_buttonCancel</property>
3363<property name="permission">protected</property>
3364<property name="pos"></property>
3365<property name="size">1,-1</property>
3366<property name="style"></property>
3367<property name="subclass"></property>
3368<property name="tooltip"></property>
3369<property name="window_extra_style"></property>
3370<property name="window_name"></property>
3371<property name="window_style"></property>
3372<event name="OnButtonClick">OnButtonCancel</event>
3373<event name="OnChar"></event>
3374<event name="OnEnterWindow"></event>
3375<event name="OnEraseBackground"></event>
3376<event name="OnKeyDown"></event>
3377<event name="OnKeyUp"></event>
3378<event name="OnKillFocus"></event>
3379<event name="OnLeaveWindow"></event>
3380<event name="OnLeftDClick"></event>
3381<event name="OnLeftDown"></event>
3382<event name="OnLeftUp"></event>
3383<event name="OnMiddleDClick"></event>
3384<event name="OnMiddleDown"></event>
3385<event name="OnMiddleUp"></event>
3386<event name="OnMotion"></event>
3387<event name="OnMouseEvents"></event>
3388<event name="OnMouseWheel"></event>
3389<event name="OnPaint"></event>
3390<event name="OnRightDClick"></event>
3391<event name="OnRightDown"></event>
3392<event name="OnRightUp"></event>
3393<event name="OnSetFocus"></event>
3394<event name="OnSize"></event>
3395<event name="OnUpdateUI"></event>
3396</object>
3397</object>
3398</object>
3399</object>
3400</object>
3401</object>
3402<object class="Dialog" expanded="1">
3403<property name="bg"></property>
3404<property name="center"></property>
3405<property name="context_help"></property>
3406<property name="enabled">1</property>
3407<property name="extra_style"></property>
3408<property name="fg"></property>
3409<property name="font"></property>
3410<property name="hidden">0</property>
3411<property name="id">wxID_ANY</property>
3412<property name="maximum_size"></property>
3413<property name="minimum_size"></property>
3414<property name="name">CSENDINGDialogBase</property>
3415<property name="pos"></property>
3416<property name="size">442,151</property>
3417<property name="style">wxDEFAULT_DIALOG_STYLE</property>
3418<property name="subclass"></property>
3419<property name="title">Sending...</property>
3420<property name="tooltip"></property>
3421<property name="window_extra_style"></property>
3422<property name="window_name"></property>
3423<property name="window_style"></property>
3424<event name="OnActivate"></event>
3425<event name="OnActivateApp"></event>
3426<event name="OnChar"></event>
3427<event name="OnClose">OnClose</event>
3428<event name="OnEnterWindow"></event>
3429<event name="OnEraseBackground"></event>
3430<event name="OnHibernate"></event>
3431<event name="OnIconize"></event>
3432<event name="OnIdle"></event>
3433<event name="OnInitDialog"></event>
3434<event name="OnKeyDown"></event>
3435<event name="OnKeyUp"></event>
3436<event name="OnKillFocus"></event>
3437<event name="OnLeaveWindow"></event>
3438<event name="OnLeftDClick"></event>
3439<event name="OnLeftDown"></event>
3440<event name="OnLeftUp"></event>
3441<event name="OnMiddleDClick"></event>
3442<event name="OnMiddleDown"></event>
3443<event name="OnMiddleUp"></event>
3444<event name="OnMotion"></event>
3445<event name="OnMouseEvents"></event>
3446<event name="OnMouseWheel"></event>
3447<event name="OnPaint">OnPaint</event>
3448<event name="OnRightDClick"></event>
3449<event name="OnRightDown"></event>
3450<event name="OnRightUp"></event>
3451<event name="OnSetFocus"></event>
3452<event name="OnSize"></event>
3453<event name="OnUpdateUI"></event>
3454<object class="wxBoxSizer" expanded="1">
3455<property name="minimum_size"></property>
3456<property name="name">bSizer68</property>
3457<property name="orient">wxVERTICAL</property>
3458<property name="permission">none</property>
3459<object class="sizeritem" expanded="1">
3460<property name="border">8</property>

```

```

3461<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxEXPAND|wxTOP|wxRIGHT|wxLEFT</p
  property>
3462<property name="proportion">0</property>
3463<object class="wxStaticText" expanded="1">
3464<property name="bg"></property>
3465<property name="context_help">0</property>
3466<property name="enabled">1</property>
3467<property name="fg"></property>
3468<property name="font"></property>
3469<property name="hidden">0</property>
3470<property name="id">wxID_ANY</property>
3471<property name="label"></property>
3472<property name="maximum_size"></property>
3473<property name="minimum_size"></property>
3474<property name="name">m_staticTextSending</property>
3475<property name="permission">protected</property>
3476<property name="pos"></property>
3477<property name="size">1,14</property>
3478<property name="style"></property>
3479<property name="subclass"></property>
3480<property name="tooltip"></property>
3481<property name="window_extra_style"></property>
3482<property name="window_name"></property>
3483<property name="window_style"></property>
3484<property name="wrap">1</property>
3485<event name="OnChar"></event>
3486<event name="OnEnterWindow"></event>
3487<event name="OnEraseBackground"></event>
3488<event name="OnKeyDown"></event>
3489<event name="OnKeyUp"></event>
3490<event name="OnKillFocus"></event>
3491<event name="OnLeaveWindow"></event>
3492<event name="OnLeftDClick"></event>
3493<event name="OnLeftDown"></event>
3494<event name="OnLeftUp"></event>
3495<event name="OnMiddleDClick"></event>
3496<event name="OnMiddleDown"></event>
3497<event name="OnMiddleUp"></event>
3498<event name="OnMotion"></event>
3499<event name="OnMouseEvents"></event>
3500<event name="OnMouseWheel"></event>
3501<event name="OnPaint"></event>
3502<event name="OnRightDClick"></event>
3503<event name="OnRightDown"></event>
3504<event name="OnRightUp"></event>
3505<event name="OnSetFocus"></event>
3506<event name="OnSize"></event>
3507<event name="OnUpdateUI"></event>
3508</object>
3509</object>
3510<object class="sizeritem" expanded="1">
3511<property name="border">10</property>
3512<property name="flag">wxEXPAND|wxRIGHT|wxLEFT</property>
3513<property name="proportion">1</property>
3514<object class="wxTextCtrl" expanded="1">
3515<property name="bg">wxSYS_COLOUR_BTNFACE</property>
3516<property name="context_help"></property>
3517<property name="enabled">1</property>
3518<property name="fg"></property>
3519<property name="font"></property>
3520<property name="hidden">0</property>
3521<property name="id">wxID_ANY</property>
3522<property name="maximum_size"></property>
3523<property name="maxLength">0</property>
3524<property name="minimum_size"></property>
3525<property name="name">m_textCtrlStatus</property>
3526<property name="permission">protected</property>
3527<property name="pos"></property>
3528<property name="size"></property>
3529<property
  name="style">wxTE_CENTRE|wxTE_MULTILINE|wxTE_NO_VSCROLL|wxTE_READONLY<
  Y</property>
3530<property name="subclass"></property>
3531<property name="tooltip"></property>
3532<property name="value">&#x0A;&#x0A;Connecting...</property>
3533<property name="window_extra_style"></property>
3534<property name="window_name"></property>
3535<property name="window_style">wxNO_BORDER</property>
3536<event name="OnChar"></event>
3537<event name="OnEnterWindow"></event>
3538<event name="OnEraseBackground"></event>
3539<event name="OnKeyDown"></event>
3540<event name="OnKeyUp"></event>
3541<event name="OnKillFocus"></event>
3542<event name="OnLeaveWindow"></event>
3543<event name="OnLeftDClick"></event>
3544<event name="OnLeftDown"></event>
3545<event name="OnLeftUp"></event>
3546<event name="OnMiddleDClick"></event>
3547<event name="OnMiddleDown"></event>
3548<event name="OnMiddleUp"></event>
3549<event name="OnMotion"></event>
3550<event name="OnMouseEvents"></event>
3551<event name="OnMouseWheel"></event>
3552<event name="OnPaint"></event>
3553<event name="OnRightDClick"></event>
3554<event name="OnRightDown"></event>
3555<event name="OnRightUp"></event>
3556<event name="OnSetFocus"></event>
3557<event name="OnSize"></event>
3558<event name="OnText"></event>
3559<event name="OnTextEnter"></event>
3560<event name="OnTextMaxLen"></event>
3561<event name="OnTextURL"></event>
3562<event name="OnUpdateUI"></event>
3563</object>
3564</object>
3565<object class="sizeritem" expanded="1">
3566<property name="border">5</property>
3567<property name="flag">wxEXPAND</property>
3568<property name="proportion">0</property>
3569<object class="wxBoxSizer" expanded="1">
3570<property name="minimum_size"></property>
3571<property name="name">bSizer69</property>
3572<property name="orient">wxHORIZONTAL</property>
3573<property name="permission">none</property>
3574<object class="sizeritem" expanded="1">
3575<property name="border">5</property>
3576<property name="flag">wxEXPAND</property>
3577<property name="proportion">1</property>
3578<object class="spacer" expanded="1">
3579<property name="height">0</property>
3580<property name="permission">protected</property>
3581<property name="width">0</property>
3582</object>
3583</object>
3584<object class="sizeritem" expanded="1">
3585<property name="border">5</property>
3586<property name="flag">wxALL</property>
3587<property name="proportion">0</property>
3588<object class="wxButton" expanded="1">
3589<property name="bg"></property>
3590<property name="context_help"></property>
3591<property name="default">0</property>
3592<property name="enabled">0</property>
3593<property name="fg"></property>
3594<property name="font"></property>
3595<property name="hidden">0</property>
3596<property name="id">wxID_ANY</property>
3597<property name="label">OK</property>
3598<property name="maximum_size"></property>
3599<property name="minimum_size">85,25</property>
3600<property name="name">m_buttonOK</property>
3601<property name="permission">protected</property>
3602<property name="pos"></property>
3603<property name="size"></property>
3604<property name="style"></property>
3605<property name="subclass"></property>
3606<property name="tooltip"></property>
3607<property name="window_extra_style"></property>
3608<property name="window_name"></property>
3609<property name="window_style"></property>
3610<event name="OnButtonClick">OnButtonOK</event>
3611<event name="OnChar"></event>
3612<event name="OnEnterWindow"></event>
3613<event name="OnEraseBackground"></event>
3614<event name="OnKeyDown"></event>
3615<event name="OnKeyUp"></event>
3616<event name="OnKillFocus"></event>
3617<event name="OnLeaveWindow"></event>
3618<event name="OnLeftDClick"></event>
3619<event name="OnLeftDown"></event>
3620<event name="OnLeftUp"></event>
3621<event name="OnMiddleDClick"></event>
3622<event name="OnMiddleDown"></event>
3623<event name="OnMiddleUp"></event>
3624<event name="OnMotion"></event>
3625<event name="OnMouseEvents"></event>
3626<event name="OnMouseWheel"></event>
3627<event name="OnPaint"></event>
3628<event name="OnRightDClick"></event>
3629<event name="OnRightDown"></event>
3630<event name="OnRightUp"></event>
3631<event name="OnSetFocus"></event>
3632<event name="OnSize"></event>
3633<event name="OnUpdateUI"></event>
3634</object>
3635</object>
3636<object class="sizeritem" expanded="1">
3637<property name="border">5</property>
3638<property name="flag">wxALL</property>
3639<property name="proportion">0</property>
3640<object class="wxButton" expanded="1">
3641<property name="bg"></property>
3642<property name="context_help"></property>
3643<property name="default">0</property>
3644<property name="enabled">1</property>
3645<property name="fg"></property>
3646<property name="font"></property>
3647<property name="hidden">0</property>
3648<property name="id">wxID_CANCEL</property>
3649<property name="label">Cancel</property>
3650<property name="maximum_size"></property>
3651<property name="minimum_size">85,25</property>
3652<property name="name">m_buttonCancel</property>
3653<property name="permission">protected</property>
3654<property name="pos"></property>
3655<property name="size">1,1</property>
3656<property name="style"></property>
3657<property name="subclass"></property>
3658<property name="tooltip"></property>
3659<property name="window_extra_style"></property>
3660<property name="window_name"></property>
3661<property name="window_style"></property>
3662<event name="OnButtonClick">OnButtonCancel</event>

```

```

3663<event name="OnChar"></event>
3664<event name="OnEnterWindow"></event>
3665<event name="OnEraseBackground"></event>
3666<event name="OnKeyDown"></event>
3667<event name="OnKeyUp"></event>
3668<event name="OnKillFocus"></event>
3669<event name="OnLeaveWindow"></event>
3670<event name="OnLeftDClick"></event>
3671<event name="OnLeftDown"></event>
3672<event name="OnLeftUp"></event>
3673<event name="OnMiddleClick"></event>
3674<event name="OnMiddleDown"></event>
3675<event name="OnMiddleUp"></event>
3676<event name="OnMotion"></event>
3677<event name="OnMouseEvents"></event>
3678<event name="OnMouseWheel"></event>
3679<event name="OnPaint"></event>
3680<event name="OnRightDClick"></event>
3681<event name="OnRightDown"></event>
3682<event name="OnRightUp"></event>
3683<event name="OnSetFocus"></event>
3684<event name="OnSize"></event>
3685<event name="OnUpdateUI"></event>
3686</object>
3687</object>
3688</object>
3689</object>
3690</object>
3691</object>
3692<object class="Dialog" expanded="0">
3693<property name="bg"></property>
3694<property name="center"></property>
3695<property name="context_help"></property>
3696<property name="enabled">1</property>
3697<property name="extra_style"></property>
3698<property name="fg"></property>
3699<property name="font"></property>
3700<property name="hidden">0</property>
3701<property name="id">wxID_ANY</property>
3702<property name="maximum_size"></property>
3703<property name="minimum_size"></property>
3704<property name="name">CYouAddressDialogBase</property>
3705<property name="pos"></property>
3706<property name="size">610, 390</property>
3707<property
  name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
3708<property name="subclass"></property>
3709<property name="title">Your Bitcoin Address</property>
3710<property name="tooltip"></property>
3711<property name="window_extra_style"></property>
3712<property name="window_name"></property>
3713<property name="window_style"></property>
3714<event name="OnActivate"></event>
3715<event name="OnActivateApp"></event>
3716<event name="OnChar"></event>
3717<event name="OnClose">OnClose</event>
3718<event name="OnEnterWindow"></event>
3719<event name="OnEraseBackground"></event>
3720<event name="OnHibernated"></event>
3721<event name="OnIconize"></event>
3722<event name="OnIdle"></event>
3723<event name="OnInitDialog"></event>
3724<event name="OnKeyDown"></event>
3725<event name="OnKeyUp"></event>
3726<event name="OnKillFocus"></event>
3727<event name="OnLeaveWindow"></event>
3728<event name="OnLeftDClick"></event>
3729<event name="OnLeftDown"></event>
3730<event name="OnLeftUp"></event>
3731<event name="OnMiddleClick"></event>
3732<event name="OnMiddleDown"></event>
3733<event name="OnMiddleUp"></event>
3734<event name="OnMotion"></event>
3735<event name="OnMouseEvents"></event>
3736<event name="OnMouseWheel"></event>
3737<event name="OnPaint"></event>
3738<event name="OnRightDClick"></event>
3739<event name="OnRightDown"></event>
3740<event name="OnRightUp"></event>
3741<event name="OnSetFocus"></event>
3742<event name="OnSize"></event>
3743<event name="OnUpdateUI"></event>
3744<object class="wxBoxSizer" expanded="1">
3745<property name="minimum_size"></property>
3746<property name="name">sbSizer68</property>
3747<property name="orient">wxVERTICAL</property>
3748<property name="permission">none</property>
3749<object class="sizeritem" expanded="1">
3750<property name="border">5</property>
3751<property name="flag">wxEXPAND</property>
3752<property name="proportion">0</property>
3753<object class="spacer" expanded="1">
3754<property name="height">5</property>
3755<property name="permission">protected</property>
3756<property name="width">0</property>
3757</object>
3758</object>
3759<object class="sizeritem" expanded="1">
3760<property name="border">5</property>
3761<property name="flag">wxALL</property>
3762<property name="proportion">0</property>
3763<object class="wxStaticText" expanded="1">
3764<property name="bg"></property>
3765<property name="context_help"></property>
3766<property name="enabled">1</property>
3767<property name="fg"></property>
3768<property name="font"></property>
3769<property name="hidden">0</property>
3770<property name="id">wxID_ANY</property>
3771<property name="label">These are your Bitcoin addresses for
  receiving payments.&#xA;You may want to give a different one to
  each sender so you can keep track of who is paying you.</property>
3772<property name="maximum_size"></property>
3773<property name="minimum_size"></property>
3774<property name="name">m_staticText45</property>
3775<property name="permission">protected</property>
3776<property name="pos"></property>
3777<property name="size"></property>
3778<property name="style"></property>
3779<property name="subclass"></property>
3780<property name="tooltip"></property>
3781<property name="window_extra_style"></property>
3782<property name="window_name"></property>
3783<property name="window_style"></property>
3784<property name="wrap">600</property>
3785<event name="OnChar"></event>
3786<event name="OnEnterWindow"></event>
3787<event name="OnEraseBackground"></event>
3788<event name="OnKeyDown"></event>
3789<event name="OnKeyUp"></event>
3790<event name="OnKillFocus"></event>
3791<event name="OnLeaveWindow"></event>
3792<event name="OnLeftDClick"></event>
3793<event name="OnLeftDown"></event>
3794<event name="OnLeftUp"></event>
3795<event name="OnMiddleClick"></event>
3796<event name="OnMiddleDown"></event>
3797<event name="OnMiddleUp"></event>
3798<event name="OnMotion"></event>
3799<event name="OnMouseEvents"></event>
3800<event name="OnMouseWheel"></event>
3801<event name="OnPaint"></event>
3802<event name="OnRightDClick"></event>
3803<event name="OnRightDown"></event>
3804<event name="OnRightUp"></event>
3805<event name="OnSetFocus"></event>
3806<event name="OnSize"></event>
3807<event name="OnUpdateUI"></event>
3808</object>
3809</object>
3810<object class="sizeritem" expanded="1">
3811<property name="border">5</property>
3812<property name="flag">wxALL|wxEXPAND</property>
3813<property name="proportion">1</property>
3814<object class="wxListCtrl" expanded="1">
3815<property name="bg"></property>
3816<property name="context_help"></property>
3817<property name="enabled">1</property>
3818<property name="fg"></property>
3819<property name="font"></property>
3820<property name="hidden">0</property>
3821<property name="id">wxID_LISTCTRL</property>
3822<property name="maximum_size"></property>
3823<property name="minimum_size"></property>
3824<property name="name">m_listCtrl</property>
3825<property name="permission">protected</property>
3826<property name="pos"></property>
3827<property name="size"></property>
3828<property
  name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING</pr
  operty>
3829<property name="subclass"></property>
3830<property name="tooltip"></property>
3831<property name="window_extra_style"></property>
3832<property name="window_name"></property>
3833<property name="window_style"></property>
3834<event name="OnChar"></event>
3835<event name="OnEnterWindow"></event>
3836<event name="OnEraseBackground"></event>
3837<event name="OnKeyDown"></event>
3838<event name="OnKeyUp"></event>
3839<event name="OnKillFocus"></event>
3840<event name="OnLeaveWindow"></event>
3841<event name="OnLeftDClick"></event>
3842<event name="OnLeftDown"></event>
3843<event name="OnLeftUp"></event>
3844<event name="OnListBeginDrag"></event>
3845<event name="OnListBeginLabelEdit"></event>
3846<event name="OnListBeginRDrag"></event>
3847<event name="OnListCacheHint"></event>
3848<event name="OnListColBeginDrag"></event>
3849<event name="OnListColClick"></event>
3850<event name="OnListColDragging"></event>
3851<event name="OnListColEndDrag"></event>
3852<event name="OnListColRightClick"></event>
3853<event name="OnListDeleteAllItems"></event>
3854<event name="OnListDeleteItem"></event>
3855<event name="OnListEndLabelEdit">OnListEndLabelEdit</event>
3856<event name="OnListInsertItem"></event>
3857<event name="OnListItemActivated">OnListItemActivated</event>
3858<event name="OnListItemDeselected"></event>
3859<event name="OnListItemFocused"></event>
3860<event name="OnListItemMiddleClick"></event>
3861<event name="OnListItemRightClick"></event>
3862<event name="OnListItemSelected">OnListItemSelected</event>
3863<event name="OnListKeyDown"></event>

```



```

3864<event name="OnMiddleClick"></event>
3865<event name="OnMiddleDown"></event>
3866<event name="OnMiddleUp"></event>
3867<event name="OnMotion"></event>
3868<event name="OnMouseEvents"></event>
3869<event name="OnMouseWheel"></event>
3870<event name="OnPaint"></event>
3871<event name="OnRightClick"></event>
3872<event name="OnRightDown"></event>
3873<event name="OnRightUp"></event>
3874<event name="OnSetFocus"></event>
3875<event name="OnSize"></event>
3876<event name="OnUpdateUI"></event>
3877</object>
3878</object>
3879<object class="sizeritem" expanded="1">
3880<property name="border">5</property>
3881<property name="flag">wxEXPAND</property>
3882<property name="proportion">0</property>
3883<object class="wxBoxSizer" expanded="1">
3884<property name="minimum_size"></property>
3885<property name="name">bSizer69</property>
3886<property name="orient">wxHORIZONTAL</property>
3887<property name="permission">none</property>
3888<object class="sizeritem" expanded="1">
3889<property name="border">5</property>
3890<property name="flag">wxEXPAND</property>
3891<property name="proportion">1</property>
3892<object class="spacer" expanded="1">
3893<property name="height">0</property>
3894<property name="permission">protected</property>
3895<property name="width">0</property>
3896</object>
3897</object>
3898<object class="sizeritem" expanded="1">
3899<property name="border">5</property>
3900<property name="flag">wxALL</property>
3901<property name="proportion">0</property>
3902<object class="wxButton" expanded="1">
3903<property name="bg"></property>
3904<property name="context_help"></property>
3905<property name="default">0</property>
3906<property name="enabled">1</property>
3907<property name="fg"></property>
3908<property name="font"></property>
3909<property name="hidden">0</property>
3910<property name="id">wxID_BUTTONRENAME</property>
3911<property name="label">&Rename...</property>
3912<property name="maximum_size"></property>
3913<property name="minimum_size">85,25</property>
3914<property name="name">m_buttonRename</property>
3915<property name="permission">protected</property>
3916<property name="pos"></property>
3917<property name="size"></property>
3918<property name="style"></property>
3919<property name="subclass"></property>
3920<property name="tooltip"></property>
3921<property name="window_extra_style"></property>
3922<property name="window_name"></property>
3923<property name="window_style"></property>
3924<event name="OnButtonClick">OnButtonClick</event>
3925<event name="OnChar"></event>
3926<event name="OnEnterWindow"></event>
3927<event name="OnEraseBackground"></event>
3928<event name="OnKeyDown"></event>
3929<event name="OnKeyUp"></event>
3930<event name="OnKillFocus"></event>
3931<event name="OnLeaveWindow"></event>
3932<event name="OnLeftClick"></event>
3933<event name="OnLeftDown"></event>
3934<event name="OnLeftUp"></event>
3935<event name="OnMiddleClick"></event>
3936<event name="OnMiddleDown"></event>
3937<event name="OnMiddleUp"></event>
3938<event name="OnMotion"></event>
3939<event name="OnMouseEvents"></event>
3940<event name="OnMouseWheel"></event>
3941<event name="OnPaint"></event>
3942<event name="OnRightClick"></event>
3943<event name="OnRightDown"></event>
3944<event name="OnRightUp"></event>
3945<event name="OnSetFocus"></event>
3946<event name="OnSize"></event>
3947<event name="OnUpdateUI"></event>
3948</object>
3949</object>
3950<object class="sizeritem" expanded="1">
3951<property name="border">5</property>
3952<property name="flag">wxALL</property>
3953<property name="proportion">0</property>
3954<object class="wxButton" expanded="1">
3955<property name="bg"></property>
3956<property name="context_help"></property>
3957<property name="default">0</property>
3958<property name="enabled">1</property>
3959<property name="fg"></property>
3960<property name="font"></property>
3961<property name="hidden">0</property>
3962<property name="id">wxID_BUTTONNEW</property>
3963<property name="label">&New Address...</property>
3964<property name="maximum_size"></property>
3965<property name="minimum_size">110,25</property>
3966<property name="name">m_buttonNew</property>
3967<property name="permission">protected</property>
3968<property name="pos"></property>
3969<property name="size">1,1</property>
3970<property name="style"></property>
3971<property name="subclass"></property>
3972<property name="tooltip"></property>
3973<property name="window_extra_style"></property>
3974<property name="window_name"></property>
3975<property name="window_style"></property>
3976<event name="OnButtonClick">OnButtonClick</event>
3977<event name="OnChar"></event>
3978<event name="OnEnterWindow"></event>
3979<event name="OnEraseBackground"></event>
3980<event name="OnKeyDown"></event>
3981<event name="OnKeyUp"></event>
3982<event name="OnKillFocus"></event>
3983<event name="OnLeaveWindow"></event>
3984<event name="OnLeftClick"></event>
3985<event name="OnLeftDown"></event>
3986<event name="OnLeftUp"></event>
3987<event name="OnMiddleClick"></event>
3988<event name="OnMiddleDown"></event>
3989<event name="OnMiddleUp"></event>
3990<event name="OnMotion"></event>
3991<event name="OnMouseEvents"></event>
3992<event name="OnMouseWheel"></event>
3993<event name="OnPaint"></event>
3994<event name="OnRightClick"></event>
3995<event name="OnRightDown"></event>
3996<event name="OnRightUp"></event>
3997<event name="OnSetFocus"></event>
3998<event name="OnSize"></event>
3999<event name="OnUpdateUI"></event>
4000</object>
4001</object>
4002<object class="sizeritem" expanded="1">
4003<property name="border">5</property>
4004<property name="flag">wxALL</property>
4005<property name="proportion">0</property>
4006<object class="wxButton" expanded="1">
4007<property name="bg"></property>
4008<property name="context_help"></property>
4009<property name="default">0</property>
4010<property name="enabled">1</property>
4011<property name="fg"></property>
4012<property name="font"></property>
4013<property name="hidden">0</property>
4014<property name="id">wxID_BUTTONCOPY</property>
4015<property name="label">&Copy to Clipboard</property>
4016<property name="maximum_size"></property>
4017<property name="minimum_size">120,25</property>
4018<property name="name">m_buttonCopy</property>
4019<property name="permission">protected</property>
4020<property name="pos"></property>
4021<property name="size">1,1</property>
4022<property name="style"></property>
4023<property name="subclass"></property>
4024<property name="tooltip"></property>
4025<property name="window_extra_style"></property>
4026<property name="window_name"></property>
4027<property name="window_style"></property>
4028<event name="OnButtonClick">OnButtonClick</event>
4029<event name="OnChar"></event>
4030<event name="OnEnterWindow"></event>
4031<event name="OnEraseBackground"></event>
4032<event name="OnKeyDown"></event>
4033<event name="OnKeyUp"></event>
4034<event name="OnKillFocus"></event>
4035<event name="OnLeaveWindow"></event>
4036<event name="OnLeftClick"></event>
4037<event name="OnLeftDown"></event>
4038<event name="OnLeftUp"></event>
4039<event name="OnMiddleClick"></event>
4040<event name="OnMiddleDown"></event>
4041<event name="OnMiddleUp"></event>
4042<event name="OnMotion"></event>
4043<event name="OnMouseEvents"></event>
4044<event name="OnMouseWheel"></event>
4045<event name="OnPaint"></event>
4046<event name="OnRightClick"></event>
4047<event name="OnRightDown"></event>
4048<event name="OnRightUp"></event>
4049<event name="OnSetFocus"></event>
4050<event name="OnSize"></event>
4051<event name="OnUpdateUI"></event>
4052</object>
4053</object>
4054<object class="sizeritem" expanded="1">
4055<property name="border">5</property>
4056<property name="flag">wxALL</property>
4057<property name="proportion">0</property>
4058<object class="wxButton" expanded="1">
4059<property name="bg"></property>
4060<property name="context_help"></property>
4061<property name="default">0</property>
4062<property name="enabled">1</property>
4063<property name="fg"></property>
4064<property name="font"></property>
4065<property name="hidden">0</property>
4066<property name="id">wxID_OK</property>
4067<property name="label">OK</property>
4068<property name="maximum_size"></property>
4069<property name="minimum_size">85,25</property>

```

```

4070<property name="name">m_buttonOK</property>
4071<property name="permission">protected</property>
4072<property name="pos"></property>
4073<property name="size"></property>
4074<property name="style"></property>
4075<property name="subclass"></property>
4076<property name="tooltip"></property>
4077<property name="window_extra_style"></property>
4078<property name="window_name"></property>
4079<property name="window_style"></property>
4080<event name="OnButtonClick">OnButtonOK</event>
4081<event name="OnChar"></event>
4082<event name="OnEnterWindow"></event>
4083<event name="OnEraseBackground"></event>
4084<event name="OnKeyDown"></event>
4085<event name="OnKeyUp"></event>
4086<event name="OnKillFocus"></event>
4087<event name="OnLeaveWindow"></event>
4088<event name="OnLeftDClick"></event>
4089<event name="OnLeftDown"></event>
4090<event name="OnLeftUp"></event>
4091<event name="OnMiddleDClick"></event>
4092<event name="OnMiddleDown"></event>
4093<event name="OnMiddleUp"></event>
4094<event name="OnMotion"></event>
4095<event name="OnMouseEvents"></event>
4096<event name="OnMouseWheel"></event>
4097<event name="OnPaint"></event>
4098<event name="OnRightDClick"></event>
4099<event name="OnRightDown"></event>
4100<event name="OnRightUp"></event>
4101<event name="OnSetFocus"></event>
4102<event name="OnSize"></event>
4103<event name="OnUpdateUI"></event>
4104</object>
4105</object>
4106<object class="sizeritem" expanded="1">
4107<property name="border">5</property>
4108<property name="flag">wxALL</property>
4109<property name="proportion">0</property>
4110<object class="wxButton" expanded="1">
4111<property name="bg"></property>
4112<property name="context_help"></property>
4113<property name="default">0</property>
4114<property name="enabled">1</property>
4115<property name="fg"></property>
4116<property name="font"></property>
4117<property name="hidden">0</property>
4118<property name="id">wxID_CANCEL</property>
4119<property name="label">Cancel</property>
4120<property name="maximum_size"></property>
4121<property name="minimum_size">85,25</property>
4122<property name="name">m_buttonCancel</property>
4123<property name="permission">protected</property>
4124<property name="pos"></property>
4125<property name="size">1,-1</property>
4126<property name="style"></property>
4127<property name="subclass"></property>
4128<property name="tooltip"></property>
4129<property name="window_extra_style"></property>
4130<property name="window_name"></property>
4131<property name="window_style"></property>
4132<event name="OnButtonClick">OnButtonCancel</event>
4133<event name="OnChar"></event>
4134<event name="OnEnterWindow"></event>
4135<event name="OnEraseBackground"></event>
4136<event name="OnKeyDown"></event>
4137<event name="OnKeyUp"></event>
4138<event name="OnKillFocus"></event>
4139<event name="OnLeaveWindow"></event>
4140<event name="OnLeftDClick"></event>
4141<event name="OnLeftDown"></event>
4142<event name="OnLeftUp"></event>
4143<event name="OnMiddleDClick"></event>
4144<event name="OnMiddleDown"></event>
4145<event name="OnMiddleUp"></event>
4146<event name="OnMotion"></event>
4147<event name="OnMouseEvents"></event>
4148<event name="OnMouseWheel"></event>
4149<event name="OnPaint"></event>
4150<event name="OnRightDClick"></event>
4151<event name="OnRightDown"></event>
4152<event name="OnRightUp"></event>
4153<event name="OnSetFocus"></event>
4154<event name="OnSize"></event>
4155<event name="OnUpdateUI"></event>
4156</object>
4157</object>
4158</object>
4159</object>
4160</object>
4161</object>
4162<object class="Dialog" expanded="0">
4163<property name="bg"></property>
4164<property name="center"></property>
4165<property name="context_help"></property>
4166<property name="enabled">1</property>
4167<property name="extra_style"></property>
4168<property name="fg"></property>
4169<property name="font"></property>
4170<property name="hidden">0</property>
4171<property name="id">wxID_ANY</property>
4172<property name="maximum_size"></property>
4173<property name="minimum_size"></property>
4174<property name="name">AddressBookDialogBase</property>
4175<property name="pos"></property>
4176<property name="size">610,390</property>
4177<property
    name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
4178<property name="subclass"></property>
4179<property name="title">Address Book</property>
4180<property name="tooltip"></property>
4181<property name="window_extra_style"></property>
4182<property name="window_name"></property>
4183<property name="window_style"></property>
4184<event name="OnActivate"></event>
4185<event name="OnActivateApp"></event>
4186<event name="OnChar"></event>
4187<event name="OnClose">OnClose</event>
4188<event name="OnEnterWindow"></event>
4189<event name="OnEraseBackground"></event>
4190<event name="OnHibernate"></event>
4191<event name="OnIconize"></event>
4192<event name="OnIdle"></event>
4193<event name="OnInitDialog"></event>
4194<event name="OnKeyDown"></event>
4195<event name="OnKeyUp"></event>
4196<event name="OnKillFocus"></event>
4197<event name="OnLeaveWindow"></event>
4198<event name="OnLeftDClick"></event>
4199<event name="OnLeftDown"></event>
4200<event name="OnLeftUp"></event>
4201<event name="OnMiddleDClick"></event>
4202<event name="OnMiddleDown"></event>
4203<event name="OnMiddleUp"></event>
4204<event name="OnMotion"></event>
4205<event name="OnMouseEvents"></event>
4206<event name="OnMouseWheel"></event>
4207<event name="OnPaint"></event>
4208<event name="OnRightDClick"></event>
4209<event name="OnRightDown"></event>
4210<event name="OnRightUp"></event>
4211<event name="OnSetFocus"></event>
4212<event name="OnSize"></event>
4213<event name="OnUpdateUI"></event>
4214<object class="wxBoxSizer" expanded="1">
4215<property name="minimum_size"></property>
4216<property name="name">Sizer68</property>
4217<property name="orient">wxVERTICAL</property>
4218<property name="permission">none</property>
4219<object class="sizeritem" expanded="1">
4220<property name="border">5</property>
4221<property name="flag">wxEXPAND</property>
4222<property name="proportion">0</property>
4223<object class="spacer" expanded="1">
4224<property name="height">5</property>
4225<property name="permission">protected</property>
4226<property name="width">0</property>
4227</object>
4228</object>
4229<object class="sizeritem" expanded="1">
4230<property name="border">5</property>
4231<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
4232<property name="proportion">0</property>
4233<object class="wxStaticText" expanded="1">
4234<property name="bg"></property>
4235<property name="context_help"></property>
4236<property name="enabled">1</property>
4237<property name="fg"></property>
4238<property name="font"></property>
4239<property name="hidden">1</property>
4240<property name="id">wxID_ANY</property>
4241<property name="label">Bitcoin Address</property>
4242<property name="maximum_size"></property>
4243<property name="minimum_size"></property>
4244<property name="name">m_staticText55</property>
4245<property name="permission">protected</property>
4246<property name="pos"></property>
4247<property name="size"></property>
4248<property name="style"></property>
4249<property name="subclass"></property>
4250<property name="tooltip"></property>
4251<property name="window_extra_style"></property>
4252<property name="window_name"></property>
4253<property name="window_style"></property>
4254<property name="wrap">1</property>
4255<event name="OnChar"></event>
4256<event name="OnEnterWindow"></event>
4257<event name="OnEraseBackground"></event>
4258<event name="OnKeyDown"></event>
4259<event name="OnKeyUp"></event>
4260<event name="OnKillFocus"></event>
4261<event name="OnLeaveWindow"></event>
4262<event name="OnLeftDClick"></event>
4263<event name="OnLeftDown"></event>
4264<event name="OnLeftUp"></event>
4265<event name="OnMiddleDClick"></event>
4266<event name="OnMiddleDown"></event>
4267<event name="OnMiddleUp"></event>
4268<event name="OnMotion"></event>
4269<event name="OnMouseEvents"></event>
4270<event name="OnMouseWheel"></event>
4271<event name="OnPaint"></event>
4272<event name="OnRightDClick"></event>
4273<event name="OnRightDown"></event>
4274<event name="OnRightUp"></event>

```

```

4275<event name="OnSetFocus"></event>
4276<event name="OnSize"></event>
4277<event name="OnUpdateUI"></event>
4278</object>
4279</object>
4280<object class="sizeritem" expanded="1">
4281<property name="border">5</property>
4282<property name="flag">wxALL|wxEXPAND</property>
4283<property name="proportion">1</property>
4284<object class="wxListCtrl" expanded="1">
4285<property name="bg"></property>
4286<property name="context_help"></property>
4287<property name="enabled">1</property>
4288<property name="fg"></property>
4289<property name="font"></property>
4290<property name="hidden">0</property>
4291<property name="id">wxID_LISTCTRL</property>
4292<property name="maximum_size"></property>
4293<property name="minimum_size"></property>
4294<property name="name">m_listCtrl</property>
4295<property name="permission">protected</property>
4296<property name="pos"></property>
4297<property name="size"></property>
4298<property
  name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT|wxLC_SORT_ASCENDING</pr
  operty>
4299<property name="subclass"></property>
4300<property name="tooltip"></property>
4301<property name="window_extra_style"></property>
4302<property name="window_name"></property>
4303<property name="window_style"></property>
4304<event name="OnChar"></event>
4305<event name="OnEnterWindow"></event>
4306<event name="OnEraseBackground"></event>
4307<event name="OnKeyDown"></event>
4308<event name="OnKeyUp"></event>
4309<event name="OnKillFocus"></event>
4310<event name="OnLeaveWindow"></event>
4311<event name="OnLeftClick"></event>
4312<event name="OnLeftDown"></event>
4313<event name="OnLeftUp"></event>
4314<event name="OnListBeginDrag"></event>
4315<event name="OnListBeginLabelEdit"></event>
4316<event name="OnListBeginRDrag"></event>
4317<event name="OnListCacheHint"></event>
4318<event name="OnListColBeginDrag"></event>
4319<event name="OnListColClick"></event>
4320<event name="OnListColDragging"></event>
4321<event name="OnListColEndDrag"></event>
4322<event name="OnListColRightClick"></event>
4323<event name="OnListDeleteAllItems"></event>
4324<event name="OnListDeleteItem"></event>
4325<event name="OnListEndLabelEdit">OnListEndLabelEdit</event>
4326<event name="OnListInsertItem"></event>
4327<event name="OnListItemActivated">OnListItemActivated</event>
4328<event name="OnListItemDeselected"></event>
4329<event name="OnListItemFocused"></event>
4330<event name="OnListItemMiddleClick"></event>
4331<event name="OnListItemRightClick"></event>
4332<event name="OnListItemSelected">OnListItemSelected</event>
4333<event name="OnListKeyDown"></event>
4334<event name="OnMiddleClick"></event>
4335<event name="OnMiddleDown"></event>
4336<event name="OnMiddleUp"></event>
4337<event name="OnMotion"></event>
4338<event name="OnMouseEvents"></event>
4339<event name="OnMouseWheel"></event>
4340<event name="OnPaint"></event>
4341<event name="OnRightClick"></event>
4342<event name="OnRightDown"></event>
4343<event name="OnRightUp"></event>
4344<event name="OnSetFocus"></event>
4345<event name="OnSize"></event>
4346<event name="OnUpdateUI"></event>
4347</object>
4348</object>
4349<object class="sizeritem" expanded="1">
4350<property name="border">5</property>
4351<property name="flag">wxEXPAND</property>
4352<property name="proportion">0</property>
4353<object class="wxBoxSizer" expanded="1">
4354<property name="minimum_size"></property>
4355<property name="name">sbSizer69</property>
4356<property name="orient">wxHORIZONTAL</property>
4357<property name="permission">none</property>
4358<object class="sizeritem" expanded="1">
4359<property name="border">5</property>
4360<property name="flag">wxEXPAND</property>
4361<property name="proportion">1</property>
4362<object class="spacer" expanded="1">
4363<property name="height">0</property>
4364<property name="permission">protected</property>
4365<property name="width">0</property>
4366</object>
4367</object>
4368<object class="sizeritem" expanded="1">
4369<property name="border">5</property>
4370<property name="flag">wxALL</property>
4371<property name="proportion">0</property>
4372<object class="wxButton" expanded="1">
4373<property name="bg"></property>
4374<property name="context_help"></property>
4375<property name="default">0</property>
4376<property name="enabled">1</property>
4377<property name="fg"></property>
4378<property name="font"></property>
4379<property name="hidden">0</property>
4380<property name="id">wxID_BUTTONEDIT</property>
4381<property name="label">&Edit...</property>
4382<property name="maximum_size"></property>
4383<property name="minimum_size">85,25</property>
4384<property name="name">m_buttonEdit</property>
4385<property name="permission">protected</property>
4386<property name="pos"></property>
4387<property name="size"></property>
4388<property name="style"></property>
4389<property name="subclass"></property>
4390<property name="tooltip"></property>
4391<property name="window_extra_style"></property>
4392<property name="window_name"></property>
4393<property name="window_style"></property>
4394<event name="OnButtonClick">OnButtonClick</event>
4395<event name="OnChar"></event>
4396<event name="OnEnterWindow"></event>
4397<event name="OnEraseBackground"></event>
4398<event name="OnKeyDown"></event>
4399<event name="OnKeyUp"></event>
4400<event name="OnKillFocus"></event>
4401<event name="OnLeaveWindow"></event>
4402<event name="OnLeftClick"></event>
4403<event name="OnLeftDown"></event>
4404<event name="OnLeftUp"></event>
4405<event name="OnMiddleClick"></event>
4406<event name="OnMiddleDown"></event>
4407<event name="OnMiddleUp"></event>
4408<event name="OnMotion"></event>
4409<event name="OnMouseEvents"></event>
4410<event name="OnMouseWheel"></event>
4411<event name="OnPaint"></event>
4412<event name="OnRightClick"></event>
4413<event name="OnRightDown"></event>
4414<event name="OnRightUp"></event>
4415<event name="OnSetFocus"></event>
4416<event name="OnSize"></event>
4417<event name="OnUpdateUI"></event>
4418</object>
4419</object>
4420<object class="sizeritem" expanded="1">
4421<property name="border">5</property>
4422<property name="flag">wxALL</property>
4423<property name="proportion">0</property>
4424<object class="wxButton" expanded="1">
4425<property name="bg"></property>
4426<property name="context_help"></property>
4427<property name="default">0</property>
4428<property name="enabled">1</property>
4429<property name="fg"></property>
4430<property name="font"></property>
4431<property name="hidden">0</property>
4432<property name="id">wxID_BUTTONNEW</property>
4433<property name="label">&New Address...</property>
4434<property name="maximum_size"></property>
4435<property name="minimum_size">110,25</property>
4436<property name="name">m_buttonNew</property>
4437<property name="permission">protected</property>
4438<property name="pos"></property>
4439<property name="size"></property>
4440<property name="style"></property>
4441<property name="subclass"></property>
4442<property name="tooltip"></property>
4443<property name="window_extra_style"></property>
4444<property name="window_name"></property>
4445<property name="window_style"></property>
4446<event name="OnButtonClick">OnButtonNew</event>
4447<event name="OnChar"></event>
4448<event name="OnEnterWindow"></event>
4449<event name="OnEraseBackground"></event>
4450<event name="OnKeyDown"></event>
4451<event name="OnKeyUp"></event>
4452<event name="OnKillFocus"></event>
4453<event name="OnLeaveWindow"></event>
4454<event name="OnLeftClick"></event>
4455<event name="OnLeftDown"></event>
4456<event name="OnLeftUp"></event>
4457<event name="OnMiddleClick"></event>
4458<event name="OnMiddleDown"></event>
4459<event name="OnMiddleUp"></event>
4460<event name="OnMotion"></event>
4461<event name="OnMouseEvents"></event>
4462<event name="OnMouseWheel"></event>
4463<event name="OnPaint"></event>
4464<event name="OnRightClick"></event>
4465<event name="OnRightDown"></event>
4466<event name="OnRightUp"></event>
4467<event name="OnSetFocus"></event>
4468<event name="OnSize"></event>
4469<event name="OnUpdateUI"></event>
4470</object>
4471</object>
4472<object class="sizeritem" expanded="1">
4473<property name="border">5</property>
4474<property name="flag">wxALL</property>
4475<property name="proportion">0</property>
4476<object class="wxButton" expanded="1">
4477<property name="bg"></property>
4478<property name="context_help"></property>

```

```

4479<property name="default">0</property>
4480<property name="enabled">1</property>
4481<property name="fg"></property>
4482<property name="font"></property>
4483<property name="hidden">0</property>
4484<property name="id">wxID_BUTTONDELETE</property>
4485<property name="label">&Delete</property>
4486<property name="maximum_size"></property>
4487<property name="minimum_size">85,25</property>
4488<property name="name">m_buttonDelete</property>
4489<property name="permission">protected</property>
4490<property name="pos"></property>
4491<property name="size"></property>
4492<property name="style"></property>
4493<property name="subclass"></property>
4494<property name="tooltip"></property>
4495<property name="window_extra_style"></property>
4496<property name="window_name"></property>
4497<property name="window_style"></property>
4498<event name="OnButtonClick">OnButtonDelete</event>
4499<event name="OnChar"></event>
4500<event name="OnEnterWindow"></event>
4501<event name="OnEraseBackground"></event>
4502<event name="OnKeyDown"></event>
4503<event name="OnKeyUp"></event>
4504<event name="OnKillFocus"></event>
4505<event name="OnLeaveWindow"></event>
4506<event name="OnLeftDClick"></event>
4507<event name="OnLeftDown"></event>
4508<event name="OnLeftUp"></event>
4509<event name="OnMiddleClick"></event>
4510<event name="OnMiddleDown"></event>
4511<event name="OnMiddleUp"></event>
4512<event name="OnMotion"></event>
4513<event name="OnMouseEvents"></event>
4514<event name="OnMouseWheel"></event>
4515<event name="OnPaint"></event>
4516<event name="OnRightDClick"></event>
4517<event name="OnRightDown"></event>
4518<event name="OnRightUp"></event>
4519<event name="OnSetFocus"></event>
4520<event name="OnSize"></event>
4521<event name="OnUpdateUI"></event>
4522</object>
4523</object>
4524<object class="sizeritem" expanded="1">
4525<property name="border">5</property>
4526<property name="flag">wxALL</property>
4527<property name="proportion">0</property>
4528<object class="wxButton" expanded="1">
4529<property name="bg"></property>
4530<property name="context_help"></property>
4531<property name="default">0</property>
4532<property name="enabled">1</property>
4533<property name="fg"></property>
4534<property name="font"></property>
4535<property name="hidden">0</property>
4536<property name="id">wxID_OK</property>
4537<property name="label">OK</property>
4538<property name="maximum_size"></property>
4539<property name="minimum_size">85,25</property>
4540<property name="name">m_buttonOK</property>
4541<property name="permission">protected</property>
4542<property name="pos"></property>
4543<property name="size">1,-1</property>
4544<property name="style"></property>
4545<property name="subclass"></property>
4546<property name="tooltip"></property>
4547<property name="window_extra_style"></property>
4548<property name="window_name"></property>
4549<property name="window_style"></property>
4550<event name="OnButtonClick">OnButtonOK</event>
4551<event name="OnChar"></event>
4552<event name="OnEnterWindow"></event>
4553<event name="OnEraseBackground"></event>
4554<event name="OnKeyDown"></event>
4555<event name="OnKeyUp"></event>
4556<event name="OnKillFocus"></event>
4557<event name="OnLeaveWindow"></event>
4558<event name="OnLeftDClick"></event>
4559<event name="OnLeftDown"></event>
4560<event name="OnLeftUp"></event>
4561<event name="OnMiddleClick"></event>
4562<event name="OnMiddleDown"></event>
4563<event name="OnMiddleUp"></event>
4564<event name="OnMotion"></event>
4565<event name="OnMouseEvents"></event>
4566<event name="OnMouseWheel"></event>
4567<event name="OnPaint"></event>
4568<event name="OnRightDClick"></event>
4569<event name="OnRightDown"></event>
4570<event name="OnRightUp"></event>
4571<event name="OnSetFocus"></event>
4572<event name="OnSize"></event>
4573<event name="OnUpdateUI"></event>
4574</object>
4575</object>
4576<object class="sizeritem" expanded="1">
4577<property name="border">5</property>
4578<property name="flag">wxALL</property>
4579<property name="proportion">0</property>
4580<object class="wxButton" expanded="1">
4581<property name="bg"></property>
4582<property name="context_help"></property>
4583<property name="default">0</property>
4584<property name="enabled">1</property>
4585<property name="fg"></property>
4586<property name="font"></property>
4587<property name="hidden">0</property>
4588<property name="id">wxID_CANCEL</property>
4589<property name="label">Cancel</property>
4590<property name="maximum_size"></property>
4591<property name="minimum_size">85,25</property>
4592<property name="name">m_buttonCancel</property>
4593<property name="permission">public</property>
4594<property name="pos"></property>
4595<property name="size">1,-1</property>
4596<property name="style"></property>
4597<property name="subclass"></property>
4598<property name="tooltip"></property>
4599<property name="window_extra_style"></property>
4600<property name="window_name"></property>
4601<property name="window_style"></property>
4602<event name="OnButtonClick">OnButtonCancel</event>
4603<event name="OnChar"></event>
4604<event name="OnEnterWindow"></event>
4605<event name="OnEraseBackground"></event>
4606<event name="OnKeyDown"></event>
4607<event name="OnKeyUp"></event>
4608<event name="OnKillFocus"></event>
4609<event name="OnLeaveWindow"></event>
4610<event name="OnLeftDClick"></event>
4611<event name="OnLeftDown"></event>
4612<event name="OnLeftUp"></event>
4613<event name="OnMiddleDClick"></event>
4614<event name="OnMiddleDown"></event>
4615<event name="OnMiddleUp"></event>
4616<event name="OnMotion"></event>
4617<event name="OnMouseEvents"></event>
4618<event name="OnMouseWheel"></event>
4619<event name="OnPaint"></event>
4620<event name="OnRightDClick"></event>
4621<event name="OnRightDown"></event>
4622<event name="OnRightUp"></event>
4623<event name="OnSetFocus"></event>
4624<event name="OnSize"></event>
4625<event name="OnUpdateUI"></event>
4626</object>
4627</object>
4628</object>
4629</object>
4630</object>
4631</object>
4632<object class="Dialog" expanded="0">
4633<property name="bg"></property>
4634<property name="center">0</property>
4635<property name="context_help"></property>
4636<property name="enabled">1</property>
4637<property name="extra_style"></property>
4638<property name="fg"></property>
4639<property name="font"></property>
4640<property name="hidden">0</property>
4641<property name="id">wxID_ANY</property>
4642<property name="maximum_size"></property>
4643<property name="minimum_size"></property>
4644<property name="name">ProductsDialogBase</property>
4645<property name="pos"></property>
4646<property name="size">708,535</property>
4647<property name="style">wxDEFAULT_DIALOG_STYLE|wxRESIZE_BORDER</property>
4648<property name="subclass"></property>
4649<property name="title">MarketPlace</property>
4650<property name="tooltip"></property>
4651<property name="window_extra_style"></property>
4652<property name="window_name"></property>
4653<property name="window_style"></property>
4654<event name="OnActivate"></event>
4655<event name="OnActivateApp"></event>
4656<event name="OnChar"></event>
4657<event name="OnClose"></event>
4658<event name="OnEnterWindow"></event>
4659<event name="OnEraseBackground"></event>
4660<event name="OnHibernate"></event>
4661<event name="OnIconize"></event>
4662<event name="OnIdle"></event>
4663<event name="OnInitDialog"></event>
4664<event name="OnKeyDown"></event>
4665<event name="OnKeyUp"></event>
4666<event name="OnKillFocus"></event>
4667<event name="OnLeaveWindow"></event>
4668<event name="OnLeftDClick"></event>
4669<event name="OnLeftDown"></event>
4670<event name="OnLeftUp"></event>
4671<event name="OnMiddleDClick"></event>
4672<event name="OnMiddleDown"></event>
4673<event name="OnMiddleUp"></event>
4674<event name="OnMotion"></event>
4675<event name="OnMouseEvents"></event>
4676<event name="OnMouseWheel"></event>
4677<event name="OnPaint"></event>
4678<event name="OnRightDClick"></event>
4679<event name="OnRightDown"></event>
4680<event name="OnRightUp"></event>
4681<event name="OnSetFocus"></event>
4682<event name="OnSize"></event>
4683<event name="OnUpdateUI"></event>

```

```

4684<object class="wxBoxSizer" expanded="1">
4685<property name="minimum_size"></property>
4686<property name="name">sbSizer22</property>
4687<property name="orient">wxVERTICAL</property>
4688<property name="permission">none</property>
4689<object class="sizeritem" expanded="1">
4690<property name="border">5</property>
4691<property name="flag">wxEXPAND|wxTOP|wxBOTTOM|wxRIGHT</property>
4692<property name="proportion">0</property>
4693<object class="wxBoxSizer" expanded="1">
4694<property name="minimum_size"></property>
4695<property name="name">sbSizer23</property>
4696<property name="orient">wxHORIZONTAL</property>
4697<property name="permission">none</property>
4698<object class="sizeritem" expanded="1">
4699<property name="border">5</property>
4700<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
4701<property name="proportion">0</property>
4702<object class="wxComboBox" expanded="1">
4703<property name="bg"></property>
4704<property name="choices">&quot;(Any Category)&quot;</property>
4705<property name="context_help"></property>
4706<property name="enabled">1</property>
4707<property name="fg"></property>
4708<property name="font"></property>
4709<property name="hidden">0</property>
4710<property name="id">wxID_ANY</property>
4711<property name="maximum_size"></property>
4712<property name="minimum_size"></property>
4713<property name="name">m_comboboxCategory</property>
4714<property name="permission">protected</property>
4715<property name="pos"></property>
4716<property name="size">150,-1</property>
4717<property name="style"></property>
4718<property name="subclass"></property>
4719<property name="tooltip"></property>
4720<property name="value">(Any Category)</property>
4721<property name="window_extra_style"></property>
4722<property name="window_name"></property>
4723<property name="window_style"></property>
4724<event name="OnChar"></event>
4725<event name="OnCombobox">OnCombobox</event>
4726<event name="OnEnterWindow"></event>
4727<event name="OnEraseBackground"></event>
4728<event name="OnKeyDown"></event>
4729<event name="OnKeyUp"></event>
4730<event name="OnKillFocus"></event>
4731<event name="OnLeaveWindow"></event>
4732<event name="OnLeftClick"></event>
4733<event name="OnLeftDown"></event>
4734<event name="OnLeftUp"></event>
4735<event name="OnMiddleClick"></event>
4736<event name="OnMiddleDown"></event>
4737<event name="OnMiddleUp"></event>
4738<event name="OnMotion"></event>
4739<event name="OnMouseEvents"></event>
4740<event name="OnMouseWheel"></event>
4741<event name="OnPaint"></event>
4742<event name="OnRightClick"></event>
4743<event name="OnRightDown"></event>
4744<event name="OnRightUp"></event>
4745<event name="OnSetFocus"></event>
4746<event name="OnSize"></event>
4747<event name="OnText"></event>
4748<event name="OnTextEnter"></event>
4749<event name="OnUpdateUI"></event>
4750</object>
4751</object>
4752<object class="sizeritem" expanded="1">
4753<property name="border">5</property>
4754<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
4755<property name="proportion">1</property>
4756<object class="wxTextCtrl" expanded="1">
4757<property name="bg"></property>
4758<property name="context_help"></property>
4759<property name="enabled">1</property>
4760<property name="fg"></property>
4761<property name="font"></property>
4762<property name="hidden">0</property>
4763<property name="id">wxID_ANY</property>
4764<property name="maximum_size"></property>
4765<property name="maxlength">0</property>
4766<property name="minimum_size"></property>
4767<property name="name">m_textCtrlSearch</property>
4768<property name="permission">protected</property>
4769<property name="pos"></property>
4770<property name="size"></property>
4771<property name="style"></property>
4772<property name="subclass"></property>
4773<property name="tooltip"></property>
4774<property name="value"></property>
4775<property name="window_extra_style"></property>
4776<property name="window_name"></property>
4777<property name="window_style"></property>
4778<event name="OnChar"></event>
4779<event name="OnEnterWindow"></event>
4780<event name="OnEraseBackground"></event>
4781<event name="OnKeyDown">OnKeyDown</event>
4782<event name="OnKeyUp"></event>
4783<event name="OnKillFocus"></event>
4784<event name="OnLeaveWindow"></event>
4785<event name="OnLeftClick"></event>
4786<event name="OnLeftDown"></event>

```

```

4787<event name="OnLeftUp"></event>
4788<event name="OnMiddleClick"></event>
4789<event name="OnMiddleDown"></event>
4790<event name="OnMiddleUp"></event>
4791<event name="OnMotion"></event>
4792<event name="OnMouseEvents"></event>
4793<event name="OnMouseWheel"></event>
4794<event name="OnPaint"></event>
4795<event name="OnRightClick"></event>
4796<event name="OnRightDown"></event>
4797<event name="OnRightUp"></event>
4798<event name="OnSetFocus"></event>
4799<event name="OnSize"></event>
4800<event name="OnText"></event>
4801<event name="OnTextEnter"></event>
4802<event name="OnTextMaxLen"></event>
4803<event name="OnTextURL"></event>
4804<event name="OnUpdateUI"></event>
4805</object>
4806</object>
4807<object class="sizeritem" expanded="1">
4808<property name="border">5</property>
4809<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
4810<property name="proportion">0</property>
4811<object class="wxButton" expanded="1">
4812<property name="bg"></property>
4813<property name="context_help"></property>
4814<property name="default">0</property>
4815<property name="enabled">1</property>
4816<property name="fg"></property>
4817<property name="font"></property>
4818<property name="hidden">0</property>
4819<property name="id">wxID_ANY</property>
4820<property name="label">&amp;Search</property>
4821<property name="maximum_size"></property>
4822<property name="minimum_size"></property>
4823<property name="name">m_buttonSearch</property>
4824<property name="permission">protected</property>
4825<property name="pos"></property>
4826<property name="size"></property>
4827<property name="style"></property>
4828<property name="subclass"></property>
4829<property name="tooltip"></property>
4830<property name="window_extra_style"></property>
4831<property name="window_name"></property>
4832<property name="window_style"></property>
4833<event name="OnButtonClick">OnButtonSearch</event>
4834<event name="OnChar"></event>
4835<event name="OnEnterWindow"></event>
4836<event name="OnEraseBackground"></event>
4837<event name="OnKeyDown"></event>
4838<event name="OnKeyUp"></event>
4839<event name="OnKillFocus"></event>
4840<event name="OnLeaveWindow"></event>
4841<event name="OnLeftClick"></event>
4842<event name="OnLeftDown"></event>
4843<event name="OnLeftUp"></event>
4844<event name="OnMiddleClick"></event>
4845<event name="OnMiddleDown"></event>
4846<event name="OnMiddleUp"></event>
4847<event name="OnMotion"></event>
4848<event name="OnMouseEvents"></event>
4849<event name="OnMouseWheel"></event>
4850<event name="OnPaint"></event>
4851<event name="OnRightClick"></event>
4852<event name="OnRightDown"></event>
4853<event name="OnRightUp"></event>
4854<event name="OnSetFocus"></event>
4855<event name="OnSize"></event>
4856<event name="OnUpdateUI"></event>
4857</object>
4858</object>
4859</object>
4860</object>
4861<object class="sizeritem" expanded="1">
4862<property name="border">5</property>
4863<property name="flag">wxALL|wxEXPAND</property>
4864<property name="proportion">1</property>
4865<object class="wxListCtrl" expanded="1">
4866<property name="bg"></property>
4867<property name="context_help"></property>
4868<property name="enabled">1</property>
4869<property name="fg"></property>
4870<property name="font"></property>
4871<property name="hidden">0</property>
4872<property name="id">wxID_ANY</property>
4873<property name="maximum_size"></property>
4874<property name="minimum_size"></property>
4875<property name="name">m_listCtrl</property>
4876<property name="permission">protected</property>
4877<property name="pos"></property>
4878<property name="size"></property>
4879<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
4880<property name="subclass"></property>
4881<property name="tooltip"></property>
4882<property name="window_extra_style"></property>
4883<property name="window_name"></property>
4884<property name="window_style"></property>
4885<event name="OnChar"></event>
4886<event name="OnEnterWindow"></event>
4887<event name="OnEraseBackground"></event>
4888<event name="OnKeyDown"></event>
4889<event name="OnKeyUp"></event>

```



```

4890<event name="OnKillFocus"></event>
4891<event name="OnLeaveWindow"></event>
4892<event name="OnLeftDClick"></event>
4893<event name="OnLeftDown"></event>
4894<event name="OnLeftUp"></event>
4895<event name="OnListBeginDrag"></event>
4896<event name="OnListBeginLabelEdit"></event>
4897<event name="OnListBeginRDrag"></event>
4898<event name="OnListCacheHint"></event>
4899<event name="OnListColBeginDrag"></event>
4900<event name="OnListColClick"></event>
4901<event name="OnListColDragging"></event>
4902<event name="OnListColEndDrag"></event>
4903<event name="OnListColRightClick"></event>
4904<event name="OnListDeleteAllItems"></event>
4905<event name="OnListDeleteItem"></event>
4906<event name="OnListEndLabelEdit"></event>
4907<event name="OnListInsertItem"></event>
4908<event name="OnListItemActivated"><OnListItemActivated</event>
4909<event name="OnListItemDeselected"></event>
4910<event name="OnListItemFocused"></event>
4911<event name="OnListItemMiddleClick"></event>
4912<event name="OnListItemRightClick"></event>
4913<event name="OnListItemSelected"></event>
4914<event name="OnListKeyDown"></event>
4915<event name="OnMiddleDClick"></event>
4916<event name="OnMiddleDown"></event>
4917<event name="OnMiddleUp"></event>
4918<event name="OnMotion"></event>
4919<event name="OnMouseEvents"></event>
4920<event name="OnMouseWheel"></event>
4921<event name="OnPaint"></event>
4922<event name="OnRightDClick"></event>
4923<event name="OnRightDown"></event>
4924<event name="OnRightUp"></event>
4925<event name="OnSetFocus"></event>
4926<event name="OnSize"></event>
4927<event name="OnUpdateUI"></event>
4928</object>
4929</object>
4930</object>
4931</object>
4932<object class="Frame" expanded="0">
4933<property name="bg">wxSYS_COLOUR_MENU</property>
4934<property name="center"></property>
4935<property name="context_help"></property>
4936<property name="enabled">1</property>
4937<property name="extra_style"></property>
4938<property name="fg"></property>
4939<property name="font"></property>
4940<property name="hidden">0</property>
4941<property name="id">wxID_ANY</property>
4942<property name="maximum_size"></property>
4943<property name="minimum_size"></property>
4944<property name="name">xCeditProductDialogBase</property>
4945<property name="pos"></property>
4946<property name="size">660,640</property>
4947<property
    name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
4948<property name="subclass"></property>
4949<property name="title">Edit Product</property>
4950<property name="tooltip"></property>
4951<property name="window_extra_style"></property>
4952<property name="window_name">wxTAB</property>
4953<property name="window_style">wxTAB_TRAVERSAL</property>
4954<property name="xrc_skip_size">1</property>
4955<event name="OnActivate"></event>
4956<event name="OnActivateApp"></event>
4957<event name="OnChar"></event>
4958<event name="OnClose"></event>
4959<event name="OnEnterWindow"></event>
4960<event name="OnEraseBackground"></event>
4961<event name="OnHibernate"></event>
4962<event name="OnIconize"></event>
4963<event name="OnIdle"></event>
4964<event name="OnKeyDown"></event>
4965<event name="OnKeyUp"></event>
4966<event name="OnKillFocus"></event>
4967<event name="OnLeaveWindow"></event>
4968<event name="OnLeftDClick"></event>
4969<event name="OnLeftDown"></event>
4970<event name="OnLeftUp"></event>
4971<event name="OnMiddleDClick"></event>
4972<event name="OnMiddleDown"></event>
4973<event name="OnMiddleUp"></event>
4974<event name="OnMotion"></event>
4975<event name="OnMouseEvents"></event>
4976<event name="OnMouseWheel"></event>
4977<event name="OnPaint"></event>
4978<event name="OnRightDClick"></event>
4979<event name="OnRightDown"></event>
4980<event name="OnRightUp"></event>
4981<event name="OnSetFocus"></event>
4982<event name="OnSize"></event>
4983<event name="OnUpdateUI"></event>
4984<object class="wxBoxSizer" expanded="1">
4985<property name="minimum_size"></property>
4986<property name="name">bSizer20</property>
4987<property name="orient">wxVERTICAL</property>
4988<property name="permission">none</property>
4989<object class="sizeritem" expanded="1">
4990<property name="border">5</property>
4991<property name="flag">wxEXPAND|wxALL</property>
4992<property name="proportion">1</property>
4993<object class="wxScrolledWindow" expanded="1">
4994<property name="bg">wxSYS_COLOUR_WINDOW</property>
4995<property name="context_help"></property>
4996<property name="enabled">1</property>
4997<property name="fg"></property>
4998<property name="font"></property>
4999<property name="hidden">0</property>
5000<property name="id">wxID_ANY</property>
5001<property name="maximum_size"></property>
5002<property name="minimum_size"></property>
5003<property name="name">m_scrolledWindow</property>
5004<property name="permission">protected</property>
5005<property name="pos"></property>
5006<property name="scroll_rate_x">5</property>
5007<property name="scroll_rate_y">5</property>
5008<property name="size"></property>
5009<property name="subclass"></property>
5010<property name="tooltip"></property>
5011<property name="window_extra_style"></property>
5012<property name="window_name"></property>
5013<property
    name="window_style">wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL</property>
5014<event name="OnChar"></event>
5015<event name="OnEnterWindow"></event>
5016<event name="OnEraseBackground"></event>
5017<event name="OnKeyDown"></event>
5018<event name="OnKeyUp"></event>
5019<event name="OnKillFocus"></event>
5020<event name="OnLeaveWindow"></event>
5021<event name="OnLeftDClick"></event>
5022<event name="OnLeftDown"></event>
5023<event name="OnLeftUp"></event>
5024<event name="OnMiddleDClick"></event>
5025<event name="OnMiddleDown"></event>
5026<event name="OnMiddleUp"></event>
5027<event name="OnMotion"></event>
5028<event name="OnMouseEvents"></event>
5029<event name="OnMouseWheel"></event>
5030<event name="OnPaint"></event>
5031<event name="OnRightDClick"></event>
5032<event name="OnRightDown"></event>
5033<event name="OnRightUp"></event>
5034<event name="OnSetFocus"></event>
5035<event name="OnSize"></event>
5036<event name="OnUpdateUI"></event>
5037<object class="wxBoxSizer" expanded="1">
5038<property name="minimum_size"></property>
5039<property name="name">bSizer21</property>
5040<property name="orient">wxVERTICAL</property>
5041<property name="permission">none</property>
5042<object class="sizeritem" expanded="1">
5043<property name="border">5</property>
5044<property name="flag">wxEXPAND|wxTOP|wxRIGHT|wxLEFT</property>
5045<property name="proportion">0</property>
5046<object class="wxFlexGridSizer" expanded="1">
5047<property name="cols">2</property>
5048<property name="flexible_direction">wxBOTH</property>
5049<property name="growablecols">1</property>
5050<property name="growablerows"></property>
5051<property name="hgap">0</property>
5052<property name="minimum_size"></property>
5053<property name="name">fgSizer8</property>
5054<property
    name="non_flexible_grow_mode">wxFLEX_GROWMODE_SPECIFIED</property>
5055<property name="permission">none</property>
5056<property name="rows">0</property>
5057<property name="vgap">0</property>
5058<object class="sizeritem" expanded="1">
5059<property name="border">5</property>
5060<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxL
    EFT</property>
5061<property name="proportion">0</property>
5062<object class="wxStaticText" expanded="1">
5063<property name="bg"></property>
5064<property name="context_help"></property>
5065<property name="enabled">1</property>
5066<property name="fg"></property>
5067<property name="font"></property>
5068<property name="hidden">0</property>
5069<property name="id">wxID_ANY</property>
5070<property name="label">Category</property>
5071<property name="maximum_size"></property>
5072<property name="minimum_size"></property>
5073<property name="name">m_staticText106</property>
5074<property name="permission">protected</property>
5075<property name="pos"></property>
5076<property name="size"></property>
5077<property name="style">wxALIGN_RIGHT</property>
5078<property name="subclass"></property>
5079<property name="tooltip"></property>
5080<property name="window_extra_style"></property>
5081<property name="window_name"></property>
5082<property name="window_style"></property>
5083<property name="wrap">1</property>
5084<event name="OnChar"></event>
5085<event name="OnEnterWindow"></event>
5086<event name="OnEraseBackground"></event>
5087<event name="OnKeyDown"></event>
5088<event name="OnKeyUp"></event>
5089<event name="OnKillFocus"></event>
5090<event name="OnLeaveWindow"></event>

```

```

5091<event name="OnLeftClick"></event>
5092<event name="OnLeftDown"></event>
5093<event name="OnLeftUp"></event>
5094<event name="OnMiddleClick"></event>
5095<event name="OnMiddleDown"></event>
5096<event name="OnMiddleUp"></event>
5097<event name="OnMotion"></event>
5098<event name="OnMouseEvents"></event>
5099<event name="OnMouseWheel"></event>
5100<event name="OnPaint"></event>
5101<event name="OnRightClick"></event>
5102<event name="OnRightDown"></event>
5103<event name="OnRightUp"></event>
5104<event name="OnSetFocus"></event>
5105<event name="OnSize"></event>
5106<event name="OnUpdateUI"></event>
5107</object>
5108</object>
5109<object class="sizeritem" expanded="1">
5110<property name="border">5</property>
5111<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
5112<property name="proportion">0</property>
5113<object class="wxComboBox" expanded="1">
5114<property name="bg"></property>
5115<property name="choices"></property>
5116<property name="context_help"></property>
5117<property name="enabled">1</property>
5118<property name="fg"></property>
5119<property name="font"></property>
5120<property name="hidden">0</property>
5121<property name="id">wxID_ANY</property>
5122<property name="maximum_size"></property>
5123<property name="minimum_size">180,-1</property>
5124<property name="name">m_comboBoxCategory</property>
5125<property name="permission">protected</property>
5126<property name="pos"></property>
5127<property name="size"></property>
5128<property name="style"></property>
5129<property name="subclass"></property>
5130<property name="tooltip"></property>
5131<property name="value"></property>
5132<property name="window_extra_style"></property>
5133<property name="window_name"></property>
5134<property name="window_style"></property>
5135<event name="OnChar"></event>
5136<event name="OnComboBox"></event>
5137<event name="OnEnterWindow"></event>
5138<event name="OnEraseBackground"></event>
5139<event name="OnKeyDown"></event>
5140<event name="OnKeyUp"></event>
5141<event name="OnKillFocus"></event>
5142<event name="OnLeaveWindow"></event>
5143<event name="OnLeftClick"></event>
5144<event name="OnLeftDown"></event>
5145<event name="OnLeftUp"></event>
5146<event name="OnMiddleClick"></event>
5147<event name="OnMiddleDown"></event>
5148<event name="OnMiddleUp"></event>
5149<event name="OnMotion"></event>
5150<event name="OnMouseEvents"></event>
5151<event name="OnMouseWheel"></event>
5152<event name="OnPaint"></event>
5153<event name="OnRightClick"></event>
5154<event name="OnRightDown"></event>
5155<event name="OnRightUp"></event>
5156<event name="OnSetFocus"></event>
5157<event name="OnSize"></event>
5158<event name="OnText"></event>
5159<event name="OnTextEnter"></event>
5160<event name="OnUpdateUI"></event>
5161</object>
5162</object>
5163<object class="sizeritem" expanded="1">
5164<property name="border">5</property>
5165<property
name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxL
EFT</property>
5166<property name="proportion">0</property>
5167<object class="wxStaticText" expanded="1">
5168<property name="bg"></property>
5169<property name="context_help"></property>
5170<property name="enabled">1</property>
5171<property name="fg"></property>
5172<property name="font"></property>
5173<property name="hidden">0</property>
5174<property name="id">wxID_ANY</property>
5175<property name="label">Title</property>
5176<property name="maximum_size"></property>
5177<property name="minimum_size"></property>
5178<property name="name">m_staticText108</property>
5179<property name="permission">protected</property>
5180<property name="pos"></property>
5181<property name="size"></property>
5182<property name="style">wxALIGN_RIGHT</property>
5183<property name="subclass"></property>
5184<property name="tooltip"></property>
5185<property name="window_extra_style"></property>
5186<property name="window_name"></property>
5187<property name="window_style"></property>
5188<property name="wrap">1</property>
5189<event name="OnChar"></event>
5190<event name="OnEnterWindow"></event>
5191<event name="OnEraseBackground"></event>
5192<event name="OnKeyDown"></event>
5193<event name="OnKeyUp"></event>
5194<event name="OnKillFocus"></event>
5195<event name="OnLeaveWindow"></event>
5196<event name="OnLeftClick"></event>
5197<event name="OnLeftDown"></event>
5198<event name="OnLeftUp"></event>
5199<event name="OnMiddleClick"></event>
5200<event name="OnMiddleDown"></event>
5201<event name="OnMiddleUp"></event>
5202<event name="OnMotion"></event>
5203<event name="OnMouseEvents"></event>
5204<event name="OnMouseWheel"></event>
5205<event name="OnPaint"></event>
5206<event name="OnRightClick"></event>
5207<event name="OnRightDown"></event>
5208<event name="OnRightUp"></event>
5209<event name="OnSetFocus"></event>
5210<event name="OnSize"></event>
5211<event name="OnUpdateUI"></event>
5212</object>
5213</object>
5214<object class="sizeritem" expanded="1">
5215<property name="border">5</property>
5216<property
name="flag">wxALL|wxALIGN_CENTER_VERTICAL|wxEXPAND</property>
5217<property name="proportion">1</property>
5218<object class="wxTextCtrl" expanded="1">
5219<property name="bg"></property>
5220<property name="context_help"></property>
5221<property name="enabled">1</property>
5222<property name="fg"></property>
5223<property name="font"></property>
5224<property name="hidden">0</property>
5225<property name="id">wxID_ANY</property>
5226<property name="maximum_size"></property>
5227<property name="maxlength">0</property>
5228<property name="minimum_size"></property>
5229<property name="name">m_textCtrlTitle</property>
5230<property name="permission">protected</property>
5231<property name="pos"></property>
5232<property name="size"></property>
5233<property name="style"></property>
5234<property name="subclass"></property>
5235<property name="tooltip"></property>
5236<property name="value"></property>
5237<property name="window_extra_style"></property>
5238<property name="window_name"></property>
5239<property name="window_style"></property>
5240<event name="OnChar"></event>
5241<event name="OnEnterWindow"></event>
5242<event name="OnEraseBackground"></event>
5243<event name="OnKeyDown">OnKeyDown</event>
5244<event name="OnKeyUp"></event>
5245<event name="OnKillFocus"></event>
5246<event name="OnLeaveWindow"></event>
5247<event name="OnLeftClick"></event>
5248<event name="OnLeftDown"></event>
5249<event name="OnLeftUp"></event>
5250<event name="OnMiddleClick"></event>
5251<event name="OnMiddleDown"></event>
5252<event name="OnMiddleUp"></event>
5253<event name="OnMotion"></event>
5254<event name="OnMouseEvents"></event>
5255<event name="OnMouseWheel"></event>
5256<event name="OnPaint"></event>
5257<event name="OnRightClick"></event>
5258<event name="OnRightDown"></event>
5259<event name="OnRightUp"></event>
5260<event name="OnSetFocus"></event>
5261<event name="OnSize"></event>
5262<event name="OnText"></event>
5263<event name="OnTextEnter"></event>
5264<event name="OnTextMaxLen"></event>
5265<event name="OnTextURL"></event>
5266<event name="OnUpdateUI"></event>
5267</object>
5268</object>
5269<object class="sizeritem" expanded="1">
5270<property name="border">5</property>
5271<property
name="flag">wxALIGN_CENTER_VERTICAL|wxALIGN_RIGHT|wxTOP|wxBOTTOM|wxL
EFT</property>
5272<property name="proportion">0</property>
5273<object class="wxStaticText" expanded="1">
5274<property name="bg"></property>
5275<property name="context_help"></property>
5276<property name="enabled">1</property>
5277<property name="fg"></property>
5278<property name="font"></property>
5279<property name="hidden">0</property>
5280<property name="id">wxID_ANY</property>
5281<property name="label">Price</property>
5282<property name="maximum_size"></property>
5283<property name="minimum_size"></property>
5284<property name="name">m_staticText107</property>
5285<property name="permission">protected</property>
5286<property name="pos"></property>
5287<property name="size"></property>
5288<property name="style">wxALIGN_RIGHT</property>
5289<property name="subclass"></property>
5290<property name="tooltip"></property>
5291<property name="window_extra_style"></property>

```

```

5292<property name="window_name"></property>
5293<property name="window_style"></property>
5294<property name="wrap">1</property>
5295<event name="OnChar"></event>
5296<event name="OnEnterWindow"></event>
5297<event name="OnEraseBackground"></event>
5298<event name="OnKeyDown"></event>
5299<event name="OnKeyUp"></event>
5300<event name="OnKillFocus"></event>
5301<event name="OnLeaveWindow"></event>
5302<event name="OnLeftDClick"></event>
5303<event name="OnLeftDown"></event>
5304<event name="OnLeftUp"></event>
5305<event name="OnMiddleDClick"></event>
5306<event name="OnMiddleDown"></event>
5307<event name="OnMiddleUp"></event>
5308<event name="OnMotion"></event>
5309<event name="OnMouseEvents"></event>
5310<event name="OnMouseWheel"></event>
5311<event name="OnPaint"></event>
5312<event name="OnRightDClick"></event>
5313<event name="OnRightDown"></event>
5314<event name="OnRightUp"></event>
5315<event name="OnSetFocus"></event>
5316<event name="OnSize"></event>
5317<event name="OnUpdateUI"></event>
5318</object>
5319</object>
5320<object class="sizeritem" expanded="1">
5321<property name="border">5</property>
5322<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
5323<property name="proportion">8</property>
5324<object class="wxTextCtrl" expanded="1">
5325<property name="bg"></property>
5326<property name="context_help"></property>
5327<property name="enabled">1</property>
5328<property name="fg"></property>
5329<property name="font"></property>
5330<property name="hidden">0</property>
5331<property name="id">wxID_ANY</property>
5332<property name="maximum_size"></property>
5333<property name="maxlength">0</property>
5334<property name="minimum_size">105,-1</property>
5335<property name="name">_m_textCtrlPrice</property>
5336<property name="permission">protected</property>
5337<property name="pos"></property>
5338<property name="size"></property>
5339<property name="style"></property>
5340<property name="subclass"></property>
5341<property name="tooltip"></property>
5342<property name="value"></property>
5343<property name="window_extra_style"></property>
5344<property name="window_name"></property>
5345<property name="window_style"></property>
5346<event name="OnChar"></event>
5347<event name="OnEnterWindow"></event>
5348<event name="OnEraseBackground"></event>
5349<event name="OnKeyDown">OnKeyDown</event>
5350<event name="OnKeyUp"></event>
5351<event name="OnKillFocus"></event>
5352<event name="OnLeaveWindow"></event>
5353<event name="OnLeftDClick"></event>
5354<event name="OnLeftDown"></event>
5355<event name="OnLeftUp"></event>
5356<event name="OnMiddleDClick"></event>
5357<event name="OnMiddleDown"></event>
5358<event name="OnMiddleUp"></event>
5359<event name="OnMotion"></event>
5360<event name="OnMouseEvents"></event>
5361<event name="OnMouseWheel"></event>
5362<event name="OnPaint"></event>
5363<event name="OnRightDClick"></event>
5364<event name="OnRightDown"></event>
5365<event name="OnRightUp"></event>
5366<event name="OnSetFocus"></event>
5367<event name="OnSize"></event>
5368<event name="OnText"></event>
5369<event name="OnTextEnter"></event>
5370<event name="OnTextMaxLen"></event>
5371<event name="OnTextURL"></event>
5372<event name="OnUpdateUI"></event>
5373</object>
5374</object>
5375</object>
5376</object>
5377<object class="sizeritem" expanded="1">
5378<property name="border">5</property>
5379<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
5380<property name="proportion">0</property>
5381<object class="wxStaticText" expanded="1">
5382<property name="bg"></property>
5383<property name="context_help"></property>
5384<property name="enabled">1</property>
5385<property name="fg"></property>
5386<property name="font"></property>
5387<property name="hidden">0</property>
5388<property name="id">wxID_ANY</property>
5389<property name="label">Page 1: Description</property>
5390<property name="maximum_size"></property>
5391<property name="minimum_size"></property>
5392<property name="name">_m_staticText22</property>
5393<property name="permission">protected</property>
5394<property name="pos"></property>
5395<property name="size"></property>
5396<property name="style"></property>
5397<property name="subclass"></property>
5398<property name="tooltip"></property>
5399<property name="window_extra_style"></property>
5400<property name="window_name"></property>
5401<property name="window_style"></property>
5402<property name="wrap">1</property>
5403<event name="OnChar"></event>
5404<event name="OnEnterWindow"></event>
5405<event name="OnEraseBackground"></event>
5406<event name="OnKeyDown"></event>
5407<event name="OnKeyUp"></event>
5408<event name="OnKillFocus"></event>
5409<event name="OnLeaveWindow"></event>
5410<event name="OnLeftDClick"></event>
5411<event name="OnLeftDown"></event>
5412<event name="OnLeftUp"></event>
5413<event name="OnMiddleDClick"></event>
5414<event name="OnMiddleDown"></event>
5415<event name="OnMiddleUp"></event>
5416<event name="OnMotion"></event>
5417<event name="OnMouseEvents"></event>
5418<event name="OnMouseWheel"></event>
5419<event name="OnPaint"></event>
5420<event name="OnRightDClick"></event>
5421<event name="OnRightDown"></event>
5422<event name="OnRightUp"></event>
5423<event name="OnSetFocus"></event>
5424<event name="OnSize"></event>
5425<event name="OnUpdateUI"></event>
5426</object>
5427</object>
5428<object class="sizeritem" expanded="1">
5429<property name="border">5</property>
5430<property name="flag">wxALL|wxEXPAND</property>
5431<property name="proportion">0</property>
5432<object class="wxTextCtrl" expanded="1">
5433<property name="bg"></property>
5434<property name="context_help"></property>
5435<property name="enabled">1</property>
5436<property name="fg"></property>
5437<property name="font"></property>
5438<property name="hidden">0</property>
5439<property name="id">wxID_ANY</property>
5440<property name="maximum_size"></property>
5441<property name="maxlength">0</property>
5442<property name="minimum_size">-1,170</property>
5443<property name="name">_m_textCtrlDescription</property>
5444<property name="permission">protected</property>
5445<property name="pos"></property>
5446<property name="size"></property>
5447<property name="style">wxTE_MULTILINE</property>
5448<property name="subclass"></property>
5449<property name="tooltip"></property>
5450<property name="value"></property>
5451<property name="window_extra_style"></property>
5452<property name="window_name"></property>
5453<property name="window_style"></property>
5454<event name="OnChar"></event>
5455<event name="OnEnterWindow"></event>
5456<event name="OnEraseBackground"></event>
5457<event name="OnKeyDown">OnKeyDown</event>
5458<event name="OnKeyUp"></event>
5459<event name="OnKillFocus"></event>
5460<event name="OnLeaveWindow"></event>
5461<event name="OnLeftDClick"></event>
5462<event name="OnLeftDown"></event>
5463<event name="OnLeftUp"></event>
5464<event name="OnMiddleDClick"></event>
5465<event name="OnMiddleDown"></event>
5466<event name="OnMiddleUp"></event>
5467<event name="OnMotion"></event>
5468<event name="OnMouseEvents"></event>
5469<event name="OnMouseWheel"></event>
5470<event name="OnPaint"></event>
5471<event name="OnRightDClick"></event>
5472<event name="OnRightDown"></event>
5473<event name="OnRightUp"></event>
5474<event name="OnSetFocus"></event>
5475<event name="OnSize"></event>
5476<event name="OnText"></event>
5477<event name="OnTextEnter"></event>
5478<event name="OnTextMaxLen"></event>
5479<event name="OnTextURL"></event>
5480<event name="OnUpdateUI"></event>
5481</object>
5482</object>
5483<object class="sizeritem" expanded="1">
5484<property name="border">5</property>
5485<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
5486<property name="proportion">0</property>
5487<object class="wxStaticText" expanded="1">
5488<property name="bg"></property>
5489<property name="context_help"></property>
5490<property name="enabled">1</property>
5491<property name="fg"></property>
5492<property name="font"></property>
5493<property name="hidden">0</property>
5494<property name="id">wxID_ANY</property>
5495<property name="label">Page 2: Order Form</property>
5496<property name="maximum_size"></property>
5497<property name="minimum_size"></property>

```



```

5498<property name="name">m_staticText23</property>
5499<property name="permission">protected</property>
5500<property name="pos"></property>
5501<property name="size"></property>
5502<property name="style"></property>
5503<property name="subclass"></property>
5504<property name="tooltip"></property>
5505<property name="window_extra_style"></property>
5506<property name="window_name"></property>
5507<property name="window_style"></property>
5508<property name="wrap">1</property>
5509<event name="OnChar"></event>
5510<event name="OnEnterWindow"></event>
5511<event name="OnEraseBackground"></event>
5512<event name="OnKeyDown"></event>
5513<event name="OnKeyUp"></event>
5514<event name="OnKillFocus"></event>
5515<event name="OnLeaveWindow"></event>
5516<event name="OnLeftDClick"></event>
5517<event name="OnLeftDown"></event>
5518<event name="OnLeftUp"></event>
5519<event name="OnMiddleDClick"></event>
5520<event name="OnMiddleDown"></event>
5521<event name="OnMiddleUp"></event>
5522<event name="OnMotion"></event>
5523<event name="OnMouseEvents"></event>
5524<event name="OnMouseWheel"></event>
5525<event name="OnPaint"></event>
5526<event name="OnRightDClick"></event>
5527<event name="OnRightDown"></event>
5528<event name="OnRightUp"></event>
5529<event name="OnSetFocus"></event>
5530<event name="OnSize"></event>
5531<event name="OnUpdateUI"></event>
5532</object>
5533</object>
5534<object class="sizeritem" expanded="1">
5535<property name="border">5</property>
5536<property name="flag">wxEXPAND|wxALL</property>
5537<property name="proportion">0</property>
5538<object class="wxTextCtrl" expanded="1">
5539<property name="bg"></property>
5540<property name="context_help"></property>
5541<property name="enabled">1</property>
5542<property name="fg"></property>
5543<property name="font"></property>
5544<property name="hidden">0</property>
5545<property name="id">wxID_ANY</property>
5546<property name="maximum_size"></property>
5547<property name="maxlength">0</property>
5548<property name="minimum_size">1,120</property>
5549<property name="name">m_textCtrlInstructions</property>
5550<property name="permission">protected</property>
5551<property name="pos"></property>
5552<property name="size"></property>
5553<property name="style">wxTE_MULTILINE</property>
5554<property name="subclass"></property>
5555<property name="tooltip"></property>
5556<property name="value"></property>
5557<property name="window_extra_style"></property>
5558<property name="window_name"></property>
5559<property name="window_style"></property>
5560<event name="OnChar"></event>
5561<event name="OnEnterWindow"></event>
5562<event name="OnEraseBackground"></event>
5563<event name="OnKeyDown">OnKeyDown</event>
5564<event name="OnKeyUp"></event>
5565<event name="OnKillFocus"></event>
5566<event name="OnLeaveWindow"></event>
5567<event name="OnLeftDClick"></event>
5568<event name="OnLeftDown"></event>
5569<event name="OnLeftUp"></event>
5570<event name="OnMiddleDClick"></event>
5571<event name="OnMiddleDown"></event>
5572<event name="OnMiddleUp"></event>
5573<event name="OnMotion"></event>
5574<event name="OnMouseEvents"></event>
5575<event name="OnMouseWheel"></event>
5576<event name="OnPaint"></event>
5577<event name="OnRightDClick"></event>
5578<event name="OnRightDown"></event>
5579<event name="OnRightUp"></event>
5580<event name="OnSetFocus"></event>
5581<event name="OnSize"></event>
5582<event name="OnText"></event>
5583<event name="OnTextEnter"></event>
5584<event name="OnTextMaxLen"></event>
5585<event name="OnTextURL"></event>
5586<event name="OnUpdateUI"></event>
5587</object>
5588</object>
5589<object class="sizeritem" expanded="1">
5590<property name="border">5</property>
5591<property name="flag">wxEXPAND</property>
5592<property name="proportion">0</property>
5593<object class="wxFlexGridSizer" expanded="0">
5594<property name="cols">3</property>
5595<property name="flexible_direction">wxBOTH</property>
5596<property name="growablecols">1</property>
5597<property name="growablerows"></property>
5598<property name="hgap">0</property>
5599<property name="minimum_size"></property>
5600<property name="name">fgSizer5</property>
5601<property
      name="non_flexible_grow_mode">wxFLEX_GROWMODE_SPECIFIED</property>
5602<property name="permission">public</property>
5603<property name="rows">0</property>
5604<property name="vgap">0</property>
5605<object class="sizeritem" expanded="1">
5606<property name="border">5</property>
5607<property
      name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT</property>
5608<property name="proportion">0</property>
5609<object class="wxStaticText" expanded="1">
5610<property name="bg"></property>
5611<property name="context_help"></property>
5612<property name="enabled">1</property>
5613<property name="fg"></property>
5614<property name="font"></property>
5615<property name="hidden">0</property>
5616<property name="id">wxID_ANY</property>
5617<property name="label">Label</property>
5618<property name="maximum_size"></property>
5619<property name="minimum_size"></property>
5620<property name="name">m_staticText24</property>
5621<property name="permission">protected</property>
5622<property name="pos"></property>
5623<property name="size"></property>
5624<property name="style"></property>
5625<property name="subclass"></property>
5626<property name="tooltip"></property>
5627<property name="window_extra_style"></property>
5628<property name="window_name"></property>
5629<property name="window_style"></property>
5630<property name="wrap">1</property>
5631<event name="OnChar"></event>
5632<event name="OnEnterWindow"></event>
5633<event name="OnEraseBackground"></event>
5634<event name="OnKeyDown"></event>
5635<event name="OnKeyUp"></event>
5636<event name="OnKillFocus"></event>
5637<event name="OnLeaveWindow"></event>
5638<event name="OnLeftDClick"></event>
5639<event name="OnLeftDown"></event>
5640<event name="OnLeftUp"></event>
5641<event name="OnMiddleDClick"></event>
5642<event name="OnMiddleDown"></event>
5643<event name="OnMiddleUp"></event>
5644<event name="OnMotion"></event>
5645<event name="OnMouseEvents"></event>
5646<event name="OnMouseWheel"></event>
5647<event name="OnPaint"></event>
5648<event name="OnRightDClick"></event>
5649<event name="OnRightDown"></event>
5650<event name="OnRightUp"></event>
5651<event name="OnSetFocus"></event>
5652<event name="OnSize"></event>
5653<event name="OnUpdateUI"></event>
5654</object>
5655</object>
5656<object class="sizeritem" expanded="1">
5657<property name="border">5</property>
5658<property
      name="flag">wxALIGN_CENTER_VERTICAL|wxTOP|wxRIGHT|wxLEFT</property>
5659<property name="proportion">0</property>
5660<object class="wxStaticText" expanded="1">
5661<property name="bg"></property>
5662<property name="context_help"></property>
5663<property name="enabled">1</property>
5664<property name="fg"></property>
5665<property name="font"></property>
5666<property name="hidden">0</property>
5667<property name="id">wxID_ANY</property>
5668<property name="label">Comma separated list of choices, or leave
      blank for text field</property>
5669<property name="maximum_size"></property>
5670<property name="minimum_size"></property>
5671<property name="name">m_staticText25</property>
5672<property name="permission">protected</property>
5673<property name="pos"></property>
5674<property name="size"></property>
5675<property name="style"></property>
5676<property name="subclass"></property>
5677<property name="tooltip"></property>
5678<property name="window_extra_style"></property>
5679<property name="window_name"></property>
5680<property name="window_style"></property>
5681<property name="wrap">1</property>
5682<event name="OnChar"></event>
5683<event name="OnEnterWindow"></event>
5684<event name="OnEraseBackground"></event>
5685<event name="OnKeyDown"></event>
5686<event name="OnKeyUp"></event>
5687<event name="OnKillFocus"></event>
5688<event name="OnLeaveWindow"></event>
5689<event name="OnLeftDClick"></event>
5690<event name="OnLeftDown"></event>
5691<event name="OnLeftUp"></event>
5692<event name="OnMiddleDClick"></event>
5693<event name="OnMiddleDown"></event>
5694<event name="OnMiddleUp"></event>
5695<event name="OnMotion"></event>
5696<event name="OnMouseEvents"></event>
5697<event name="OnMouseWheel"></event>
5698<event name="OnPaint"></event>
5699<event name="OnRightDClick"></event>

```

```

5700<event name="OnRightDown"></event>
5701<event name="OnRightUp"></event>
5702<event name="OnSetFocus"></event>
5703<event name="OnSize"></event>
5704<event name="OnUpdateUI"></event>
5705</object>
5706</object>
5707<object class="sizeritem" expanded="1">
5708<property name="border">5</property>
5709<property name="flag">wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
5710<property name="proportion">0</property>
5711<object class="wxTextCtrl" expanded="1">
5712<property name="height">0</property>
5713<property name="permission">protected</property>
5714<property name="width">0</property>
5715</object>
5716</object>
5717<object class="sizeritem" expanded="1">
5718<property name="border">5</property>
5719<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
5720<property name="proportion">0</property>
5721<object class="wxTextCtrl" expanded="1">
5722<property name="bg"></property>
5723<property name="context_help"></property>
5724<property name="enabled">1</property>
5725<property name="fg"></property>
5726<property name="font"></property>
5727<property name="hidden">0</property>
5728<property name="id">wxID_ANY</property>
5729<property name="maximum_size"></property>
5730<property name="maxlength">0</property>
5731<property name="minimum_size">150,-1</property>
5732<property name="name">m_textCtrlLabel0</property>
5733<property name="permission">protected</property>
5734<property name="pos"></property>
5735<property name="size"></property>
5736<property name="style"></property>
5737<property name="subclass"></property>
5738<property name="tooltip"></property>
5739<property name="value"></property>
5740<property name="window_extra_style"></property>
5741<property name="window_name"></property>
5742<property name="window_style"></property>
5743<event name="OnChar"></event>
5744<event name="OnEnterWindow"></event>
5745<event name="OnEraseBackground"></event>
5746<event name="OnKeyDown">OnKeyDown</event>
5747<event name="OnKeyUp"></event>
5748<event name="OnKillFocus"></event>
5749<event name="OnLeaveWindow"></event>
5750<event name="OnLeftDClick"></event>
5751<event name="OnLeftDown"></event>
5752<event name="OnLeftUp"></event>
5753<event name="OnMiddleDClick"></event>
5754<event name="OnMiddleDown"></event>
5755<event name="OnMiddleUp"></event>
5756<event name="OnMotion"></event>
5757<event name="OnMouseEvents"></event>
5758<event name="OnMouseWheel"></event>
5759<event name="OnPaint"></event>
5760<event name="OnRightDClick"></event>
5761<event name="OnRightDown"></event>
5762<event name="OnRightUp"></event>
5763<event name="OnSetFocus"></event>
5764<event name="OnSize"></event>
5765<event name="OnText"></event>
5766<event name="OnTextEnter"></event>
5767<event name="OnTextMaxLen"></event>
5768<event name="OnTextURL"></event>
5769<event name="OnUpdateUI"></event>
5770</object>
5771</object>
5772<object class="sizeritem" expanded="1">
5773<property name="border">5</property>
5774<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
5775<property name="proportion">0</property>
5776<object class="wxTextCtrl" expanded="1">
5777<property name="bg"></property>
5778<property name="context_help"></property>
5779<property name="enabled">1</property>
5780<property name="fg"></property>
5781<property name="font"></property>
5782<property name="hidden">0</property>
5783<property name="id">wxID_ANY</property>
5784<property name="maximum_size">1,-1</property>
5785<property name="maxlength">0</property>
5786<property name="minimum_size">1,-1</property>
5787<property name="name">m_textCtrlField0</property>
5788<property name="permission">protected</property>
5789<property name="pos"></property>
5790<property name="size">1,-1</property>
5791<property name="style"></property>
5792<property name="subclass"></property>
5793<property name="tooltip"></property>
5794<property name="value"></property>
5795<property name="window_extra_style"></property>
5796<property name="window_name"></property>
5797<property name="window_style"></property>
5798<event name="OnChar"></event>
5799<event name="OnEnterWindow"></event>
5800<event name="OnEraseBackground"></event>
5801<event name="OnKeyDown">OnKeyDown</event>
5802<event name="OnKeyUp"></event>
5803<event name="OnKillFocus"></event>
5804<event name="OnLeaveWindow"></event>
5805<event name="OnLeftDClick"></event>
5806<event name="OnLeftDown"></event>
5807<event name="OnLeftUp"></event>
5808<event name="OnMiddleDClick"></event>
5809<event name="OnMiddleDown"></event>
5810<event name="OnMiddleUp"></event>
5811<event name="OnMotion"></event>
5812<event name="OnMouseEvents"></event>
5813<event name="OnMouseWheel"></event>
5814<event name="OnPaint"></event>
5815<event name="OnRightDClick"></event>
5816<event name="OnRightDown"></event>
5817<event name="OnRightUp"></event>
5818<event name="OnSetFocus"></event>
5819<event name="OnSize"></event>
5820<event name="OnText"></event>
5821<event name="OnTextEnter"></event>
5822<event name="OnTextMaxLen"></event>
5823<event name="OnTextURL"></event>
5824<event name="OnUpdateUI"></event>
5825</object>
5826</object>
5827<object class="sizeritem" expanded="1">
5828<property name="border">5</property>
5829<property name="flag">wxRIGHT|wxLEFT|wxALIGN_CENTER_VERTICAL</property>
5830<property name="proportion">0</property>
5831<object class="wxButton" expanded="1">
5832<property name="bg"></property>
5833<property name="context_help"></property>
5834<property name="default">0</property>
5835<property name="enabled">1</property>
5836<property name="fg"></property>
5837<property name="font"></property>
5838<property name="hidden">0</property>
5839<property name="id">wxID_DEL0</property>
5840<property name="label">Delete</property>
5841<property name="maximum_size"></property>
5842<property name="minimum_size">-1,-1</property>
5843<property name="name">m_buttonDel0</property>
5844<property name="permission">protected</property>
5845<property name="pos"></property>
5846<property name="size">60,20</property>
5847<property name="style"></property>
5848<property name="subclass"></property>
5849<property name="tooltip"></property>
5850<property name="window_extra_style"></property>
5851<property name="window_name"></property>
5852<property name="window_style"></property>
5853<event name="OnButtonClick">OnButtonDel0</event>
5854<event name="OnChar"></event>
5855<event name="OnEnterWindow"></event>
5856<event name="OnEraseBackground"></event>
5857<event name="OnKeyDown"></event>
5858<event name="OnKeyUp"></event>
5859<event name="OnKillFocus"></event>
5860<event name="OnLeaveWindow"></event>
5861<event name="OnLeftDClick"></event>
5862<event name="OnLeftDown"></event>
5863<event name="OnLeftUp"></event>
5864<event name="OnMiddleDClick"></event>
5865<event name="OnMiddleDown"></event>
5866<event name="OnMiddleUp"></event>
5867<event name="OnMotion"></event>
5868<event name="OnMouseEvents"></event>
5869<event name="OnMouseWheel"></event>
5870<event name="OnPaint"></event>
5871<event name="OnRightDClick"></event>
5872<event name="OnRightDown"></event>
5873<event name="OnRightUp"></event>
5874<event name="OnSetFocus"></event>
5875<event name="OnSize"></event>
5876<event name="OnUpdateUI"></event>
5877</object>
5878</object>
5879<object class="sizeritem" expanded="1">
5880<property name="border">5</property>
5881<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
5882<property name="proportion">0</property>
5883<object class="wxTextCtrl" expanded="1">
5884<property name="bg"></property>
5885<property name="context_help"></property>
5886<property name="enabled">1</property>
5887<property name="fg"></property>
5888<property name="font"></property>
5889<property name="hidden">0</property>
5890<property name="id">wxID_ANY</property>
5891<property name="maximum_size"></property>
5892<property name="maxlength">0</property>
5893<property name="minimum_size">150,-1</property>
5894<property name="name">m_textCtrlLabel1</property>
5895<property name="permission">protected</property>
5896<property name="pos"></property>
5897<property name="size"></property>
5898<property name="style"></property>
5899<property name="subclass"></property>
5900<property name="tooltip"></property>
5901<property name="value"></property>
5902<property name="window_extra_style"></property>
5903<property name="window_name"></property>

```

```

5904<property name="window_style"></property>
5905<event name="OnChar"></event>
5906<event name="OnEnterWindow"></event>
5907<event name="OnEraseBackground"></event>
5908<event name="OnKeyDown"><OnKeyDown/></event>
5909<event name="OnKeyUp"></event>
5910<event name="OnKillFocus"></event>
5911<event name="OnLeaveWindow"></event>
5912<event name="OnLeftDClick"></event>
5913<event name="OnLeftDown"></event>
5914<event name="OnLeftUp"></event>
5915<event name="OnMiddleClick"></event>
5916<event name="OnMiddleDown"></event>
5917<event name="OnMiddleUp"></event>
5918<event name="OnMotion"></event>
5919<event name="OnMouseEvents"></event>
5920<event name="OnMouseWheel"></event>
5921<event name="OnPaint"></event>
5922<event name="OnRightDClick"></event>
5923<event name="OnRightDown"></event>
5924<event name="OnRightUp"></event>
5925<event name="OnSetFocus"></event>
5926<event name="OnSize"></event>
5927<event name="OnText"></event>
5928<event name="OnTextEnter"></event>
5929<event name="OnTextMaxLen"></event>
5930<event name="OnTextURL"></event>
5931<event name="OnUpdateUI"></event>
5932</object>
5933</object>
5934<object class="sizeritem" expanded="1">
5935<property name="border">5</property>
5936<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
5937<property name="proportion">0</property>
5938<object class="wxTextCtrl" expanded="1">
5939<property name="bg"></property>
5940<property name="context_help"></property>
5941<property name="enabled">1</property>
5942<property name="fg"></property>
5943<property name="font"></property>
5944<property name="hidden">0</property>
5945<property name="id">wxID_ANY</property>
5946<property name="maximum_size">1,-1</property>
5947<property name="maxlength">0</property>
5948<property name="minimum_size">1,-1</property>
5949<property name="name">m_textCtrlField1</property>
5950<property name="permission">protected</property>
5951<property name="pos"></property>
5952<property name="size">1,-1</property>
5953<property name="style"></property>
5954<property name="subclass"></property>
5955<property name="tooltip"></property>
5956<property name="value"></property>
5957<property name="window_extra_style"></property>
5958<property name="window_name"></property>
5959<property name="window_style"></property>
5960<event name="OnChar"></event>
5961<event name="OnEnterWindow"></event>
5962<event name="OnEraseBackground"></event>
5963<event name="OnKeyDown"><OnKeyDown/></event>
5964<event name="OnKeyUp"></event>
5965<event name="OnKillFocus"></event>
5966<event name="OnLeaveWindow"></event>
5967<event name="OnLeftDClick"></event>
5968<event name="OnLeftDown"></event>
5969<event name="OnLeftUp"></event>
5970<event name="OnMiddleClick"></event>
5971<event name="OnMiddleDown"></event>
5972<event name="OnMiddleUp"></event>
5973<event name="OnMotion"></event>
5974<event name="OnMouseEvents"></event>
5975<event name="OnMouseWheel"></event>
5976<event name="OnPaint"></event>
5977<event name="OnRightDClick"></event>
5978<event name="OnRightDown"></event>
5979<event name="OnRightUp"></event>
5980<event name="OnSetFocus"></event>
5981<event name="OnSize"></event>
5982<event name="OnText"></event>
5983<event name="OnTextEnter"></event>
5984<event name="OnTextMaxLen"></event>
5985<event name="OnTextURL"></event>
5986<event name="OnUpdateUI"></event>
5987</object>
5988</object>
5989<object class="sizeritem" expanded="1">
5990<property name="border">5</property>
5991<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
5992<property name="proportion">0</property>
5993<object class="wxButton" expanded="1">
5994<property name="bg"></property>
5995<property name="context_help"></property>
5996<property name="default">0</property>
5997<property name="enabled">1</property>
5998<property name="fg"></property>
5999<property name="font"></property>
6000<property name="hidden">0</property>
6001<property name="id">wxID_DEL1</property>
6002<property name="label">Delete</property>
6003<property name="maximum_size"></property>
6004<property name="minimum_size">-1,-1</property>
6005<property name="name">m_buttonDel1</property>
6006<property name="permission">protected</property>
6007<property name="pos"></property>
6008<property name="size">60,20</property>
6009<property name="style"></property>
6010<property name="subclass"></property>
6011<property name="tooltip"></property>
6012<property name="window_extra_style"></property>
6013<property name="window_name"></property>
6014<property name="window_style"></property>
6015<event name="OnButtonClick"><OnButtonDel1/></event>
6016<event name="OnChar"></event>
6017<event name="OnEnterWindow"></event>
6018<event name="OnEraseBackground"></event>
6019<event name="OnKeyDown"></event>
6020<event name="OnKeyUp"></event>
6021<event name="OnKillFocus"></event>
6022<event name="OnLeaveWindow"></event>
6023<event name="OnLeftDClick"></event>
6024<event name="OnLeftDown"></event>
6025<event name="OnLeftUp"></event>
6026<event name="OnMiddleClick"></event>
6027<event name="OnMiddleDown"></event>
6028<event name="OnMiddleUp"></event>
6029<event name="OnMotion"></event>
6030<event name="OnMouseEvents"></event>
6031<event name="OnMouseWheel"></event>
6032<event name="OnPaint"></event>
6033<event name="OnRightDClick"></event>
6034<event name="OnRightDown"></event>
6035<event name="OnRightUp"></event>
6036<event name="OnSetFocus"></event>
6037<event name="OnSize"></event>
6038<event name="OnUpdateUI"></event>
6039</object>
6040</object>
6041<object class="sizeritem" expanded="1">
6042<property name="border">5</property>
6043<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
6044<property name="proportion">0</property>
6045<object class="wxTextCtrl" expanded="1">
6046<property name="bg"></property>
6047<property name="context_help"></property>
6048<property name="enabled">1</property>
6049<property name="fg"></property>
6050<property name="font"></property>
6051<property name="hidden">0</property>
6052<property name="id">wxID_ANY</property>
6053<property name="maximum_size"></property>
6054<property name="maxlength">0</property>
6055<property name="minimum_size">150,-1</property>
6056<property name="name">m_textCtrlLabel2</property>
6057<property name="permission">protected</property>
6058<property name="pos"></property>
6059<property name="size"></property>
6060<property name="style"></property>
6061<property name="subclass"></property>
6062<property name="tooltip"></property>
6063<property name="value"></property>
6064<property name="window_extra_style"></property>
6065<property name="window_name"></property>
6066<property name="window_style"></property>
6067<event name="OnChar"></event>
6068<event name="OnEnterWindow"></event>
6069<event name="OnEraseBackground"></event>
6070<event name="OnKeyDown"><OnKeyDown/></event>
6071<event name="OnKeyUp"></event>
6072<event name="OnKillFocus"></event>
6073<event name="OnLeaveWindow"></event>
6074<event name="OnLeftDClick"></event>
6075<event name="OnLeftDown"></event>
6076<event name="OnLeftUp"></event>
6077<event name="OnMiddleClick"></event>
6078<event name="OnMiddleDown"></event>
6079<event name="OnMiddleUp"></event>
6080<event name="OnMotion"></event>
6081<event name="OnMouseEvents"></event>
6082<event name="OnMouseWheel"></event>
6083<event name="OnPaint"></event>
6084<event name="OnRightDClick"></event>
6085<event name="OnRightDown"></event>
6086<event name="OnRightUp"></event>
6087<event name="OnSetFocus"></event>
6088<event name="OnSize"></event>
6089<event name="OnText"></event>
6090<event name="OnTextEnter"></event>
6091<event name="OnTextMaxLen"></event>
6092<event name="OnTextURL"></event>
6093<event name="OnUpdateUI"></event>
6094</object>
6095</object>
6096<object class="sizeritem" expanded="1">
6097<property name="border">5</property>
6098<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
6099<property name="proportion">0</property>
6100<object class="wxTextCtrl" expanded="1">
6101<property name="bg"></property>
6102<property name="context_help"></property>
6103<property name="enabled">1</property>
6104<property name="fg"></property>
6105<property name="font"></property>
6106<property name="hidden">0</property>

```

```

6107<property name="id">wxID_ANY</property>
6108<property name="maximum_size"><1,-1</property>
6109<property name="maxlength">0</property>
6110<property name="minimum_size"><1,-1</property>
6111<property name="name">m_textCtrlField2</property>
6112<property name="permission">protected</property>
6113<property name="pos"><0</property>
6114<property name="size"><1,-1</property>
6115<property name="style"><0</property>
6116<property name="subclass"><0</property>
6117<property name="tooltip"><0</property>
6118<property name="value"><0</property>
6119<property name="window_extra_style"><0</property>
6120<property name="window_name"><0</property>
6121<property name="window_style"><0</property>
6122<event name="OnChar"><0</event>
6123<event name="OnEnterWindow"><0</event>
6124<event name="OnEraseBackground"><0</event>
6125<event name="OnKeyDown">OnKeyDown</event>
6126<event name="OnKeyUp"><0</event>
6127<event name="OnKillFocus"><0</event>
6128<event name="OnLeaveWindow"><0</event>
6129<event name="OnLeftDClick"><0</event>
6130<event name="OnLeftDown"><0</event>
6131<event name="OnLeftUp"><0</event>
6132<event name="OnMiddleDClick"><0</event>
6133<event name="OnMiddleDown"><0</event>
6134<event name="OnMiddleUp"><0</event>
6135<event name="OnMotion"><0</event>
6136<event name="OnMouseEvents"><0</event>
6137<event name="OnMouseWheel"><0</event>
6138<event name="OnPaint"><0</event>
6139<event name="OnRightDClick"><0</event>
6140<event name="OnRightDown"><0</event>
6141<event name="OnRightUp"><0</event>
6142<event name="OnSetFocus"><0</event>
6143<event name="OnSize"><0</event>
6144<event name="OnText"><0</event>
6145<event name="OnTextEnter"><0</event>
6146<event name="OnTextMaxLen"><0</event>
6147<event name="OnTextURL"><0</event>
6148<event name="OnUpdateUI"><0</event>
6149</object>
6150</object>
6151<object class="sizeritem" expanded="1">
6152<property name="border">5</property>
6153<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
6154<property name="proportion">0</property>
6155<object class="wxButton" expanded="1">
6156<property name="bg"><0</property>
6157<property name="context_help"><0</property>
6158<property name="default">0</property>
6159<property name="enabled">1</property>
6160<property name="fg"><0</property>
6161<property name="font"><0</property>
6162<property name="hidden">0</property>
6163<property name="id">wxID_DEL2</property>
6164<property name="label">Delete</property>
6165<property name="maximum_size"><0</property>
6166<property name="minimum_size"><1,-1</property>
6167<property name="name">m_buttonDel2</property>
6168<property name="permission">protected</property>
6169<property name="pos"><0</property>
6170<property name="size"><60,20</property>
6171<property name="style"><0</property>
6172<property name="subclass"><0</property>
6173<property name="tooltip"><0</property>
6174<property name="window_extra_style"><0</property>
6175<property name="window_name"><0</property>
6176<property name="window_style"><0</property>
6177<event name="OnButtonClick">OnButtonDel2</event>
6178<event name="OnChar"><0</event>
6179<event name="OnEnterWindow"><0</event>
6180<event name="OnEraseBackground"><0</event>
6181<event name="OnKeyDown"><0</event>
6182<event name="OnKeyUp"><0</event>
6183<event name="OnKillFocus"><0</event>
6184<event name="OnLeaveWindow"><0</event>
6185<event name="OnLeftDClick"><0</event>
6186<event name="OnLeftDown"><0</event>
6187<event name="OnLeftUp"><0</event>
6188<event name="OnMiddleDClick"><0</event>
6189<event name="OnMiddleDown"><0</event>
6190<event name="OnMiddleUp"><0</event>
6191<event name="OnMotion"><0</event>
6192<event name="OnMouseEvents"><0</event>
6193<event name="OnMouseWheel"><0</event>
6194<event name="OnPaint"><0</event>
6195<event name="OnRightDClick"><0</event>
6196<event name="OnRightDown"><0</event>
6197<event name="OnRightUp"><0</event>
6198<event name="OnSetFocus"><0</event>
6199<event name="OnSize"><0</event>
6200<event name="OnUpdateUI"><0</event>
6201</object>
6202</object>
6203<object class="sizeritem" expanded="1">
6204<property name="border">5</property>
6205<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
6206<property name="proportion">0</property>
6207<object class="wxTextCtrl" expanded="1">
6208<property name="bg"><0</property>
6209<property name="context_help"><0</property>
6210<property name="enabled">1</property>
6211<property name="fg"><0</property>
6212<property name="font"><0</property>
6213<property name="hidden">0</property>
6214<property name="id">wxID_ANY</property>
6215<property name="maximum_size"><0</property>
6216<property name="maxlength">0</property>
6217<property name="minimum_size"><150,-1</property>
6218<property name="name">m_textCtrlLabel3</property>
6219<property name="permission">protected</property>
6220<property name="pos"><0</property>
6221<property name="size"><0</property>
6222<property name="style"><0</property>
6223<property name="subclass"><0</property>
6224<property name="tooltip"><0</property>
6225<property name="value"><0</property>
6226<property name="window_extra_style"><0</property>
6227<property name="window_name"><0</property>
6228<property name="window_style"><0</property>
6229<event name="OnChar"><0</event>
6230<event name="OnEnterWindow"><0</event>
6231<event name="OnEraseBackground"><0</event>
6232<event name="OnKeyDown">OnKeyDown</event>
6233<event name="OnKeyUp"><0</event>
6234<event name="OnKillFocus"><0</event>
6235<event name="OnLeaveWindow"><0</event>
6236<event name="OnLeftDClick"><0</event>
6237<event name="OnLeftDown"><0</event>
6238<event name="OnLeftUp"><0</event>
6239<event name="OnMiddleDClick"><0</event>
6240<event name="OnMiddleDown"><0</event>
6241<event name="OnMiddleUp"><0</event>
6242<event name="OnMotion"><0</event>
6243<event name="OnMouseEvents"><0</event>
6244<event name="OnMouseWheel"><0</event>
6245<event name="OnPaint"><0</event>
6246<event name="OnRightDClick"><0</event>
6247<event name="OnRightDown"><0</event>
6248<event name="OnRightUp"><0</event>
6249<event name="OnSetFocus"><0</event>
6250<event name="OnSize"><0</event>
6251<event name="OnText"><0</event>
6252<event name="OnTextEnter"><0</event>
6253<event name="OnTextMaxLen"><0</event>
6254<event name="OnTextURL"><0</event>
6255<event name="OnUpdateUI"><0</event>
6256</object>
6257</object>
6258<object class="sizeritem" expanded="1">
6259<property name="border">5</property>
6260<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
6261<property name="proportion">0</property>
6262<object class="wxTextCtrl" expanded="1">
6263<property name="bg"><0</property>
6264<property name="context_help"><0</property>
6265<property name="enabled">1</property>
6266<property name="fg"><0</property>
6267<property name="font"><0</property>
6268<property name="hidden">0</property>
6269<property name="id">wxID_ANY</property>
6270<property name="maximum_size"><1,-1</property>
6271<property name="maxlength">0</property>
6272<property name="minimum_size"><1,-1</property>
6273<property name="name">m_textCtrlField3</property>
6274<property name="permission">protected</property>
6275<property name="pos"><0</property>
6276<property name="size"><1,-1</property>
6277<property name="style"><0</property>
6278<property name="subclass"><0</property>
6279<property name="tooltip"><0</property>
6280<property name="value"><0</property>
6281<property name="window_extra_style"><0</property>
6282<property name="window_name"><0</property>
6283<property name="window_style"><0</property>
6284<event name="OnChar"><0</event>
6285<event name="OnEnterWindow"><0</event>
6286<event name="OnEraseBackground"><0</event>
6287<event name="OnKeyDown">OnKeyDown</event>
6288<event name="OnKeyUp"><0</event>
6289<event name="OnKillFocus"><0</event>
6290<event name="OnLeaveWindow"><0</event>
6291<event name="OnLeftDClick"><0</event>
6292<event name="OnLeftDown"><0</event>
6293<event name="OnLeftUp"><0</event>
6294<event name="OnMiddleDClick"><0</event>
6295<event name="OnMiddleDown"><0</event>
6296<event name="OnMiddleUp"><0</event>
6297<event name="OnMotion"><0</event>
6298<event name="OnMouseEvents"><0</event>
6299<event name="OnMouseWheel"><0</event>
6300<event name="OnPaint"><0</event>
6301<event name="OnRightDClick"><0</event>
6302<event name="OnRightDown"><0</event>
6303<event name="OnRightUp"><0</event>
6304<event name="OnSetFocus"><0</event>
6305<event name="OnSize"><0</event>
6306<event name="OnText"><0</event>
6307<event name="OnTextEnter"><0</event>
6308<event name="OnTextMaxLen"><0</event>
6309<event name="OnTextURL"><0</event>
6310<event name="OnUpdateUI"><0</event>

```

```

6311</object>
6312</object>
6313<object class="sizeritem" expanded="1">
6314<property name="border">5</property>
6315<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
6316<property name="proportion">0</property>
6317<object class="wxButton" expanded="1">
6318<property name="bg"></property>
6319<property name="context_help"></property>
6320<property name="default">0</property>
6321<property name="enabled">1</property>
6322<property name="fg"></property>
6323<property name="font"></property>
6324<property name="hidden">0</property>
6325<property name="id">wxID_DEL3</property>
6326<property name="label">Delete</property>
6327<property name="maximum_size"></property>
6328<property name="minimum_size">-1,-1</property>
6329<property name="name">m_buttonDel3</property>
6330<property name="permission">protected</property>
6331<property name="pos"></property>
6332<property name="size">60,20</property>
6333<property name="style"></property>
6334<property name="subclass"></property>
6335<property name="tooltip"></property>
6336<property name="window_extra_style"></property>
6337<property name="window_name"></property>
6338<property name="window_style"></property>
6339<event name="OnButtonClick">OnButtonDel3</event>
6340<event name="OnChar"></event>
6341<event name="OnEnterWindow"></event>
6342<event name="OnEraseBackground"></event>
6343<event name="OnKeyDown"></event>
6344<event name="OnKeyUp"></event>
6345<event name="OnKillFocus"></event>
6346<event name="OnLeaveWindow"></event>
6347<event name="OnLeftDClick"></event>
6348<event name="OnLeftDown"></event>
6349<event name="OnLeftUp"></event>
6350<event name="OnMiddleDClick"></event>
6351<event name="OnMiddleDown"></event>
6352<event name="OnMiddleUp"></event>
6353<event name="OnMotion"></event>
6354<event name="OnMouseEvents"></event>
6355<event name="OnMouseWheel"></event>
6356<event name="OnPaint"></event>
6357<event name="OnRightDClick"></event>
6358<event name="OnRightDown"></event>
6359<event name="OnRightUp"></event>
6360<event name="OnSetFocus"></event>
6361<event name="OnSize"></event>
6362<event name="OnUpdateUI"></event>
6363</object>
6364</object>
6365<object class="sizeritem" expanded="1">
6366<property name="border">5</property>
6367<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
6368<property name="proportion">0</property>
6369<object class="wxTextCtrl" expanded="1">
6370<property name="bg"></property>
6371<property name="context_help"></property>
6372<property name="enabled">1</property>
6373<property name="fg"></property>
6374<property name="font"></property>
6375<property name="hidden">0</property>
6376<property name="id">wxID_ANY</property>
6377<property name="maximum_size"></property>
6378<property name="maxlength">0</property>
6379<property name="minimum_size">150,-1</property>
6380<property name="name">m_textCtrlLabel4</property>
6381<property name="permission">protected</property>
6382<property name="pos"></property>
6383<property name="size"></property>
6384<property name="style"></property>
6385<property name="subclass"></property>
6386<property name="tooltip"></property>
6387<property name="value"></property>
6388<property name="window_extra_style"></property>
6389<property name="window_name"></property>
6390<property name="window_style"></property>
6391<event name="OnChar"></event>
6392<event name="OnEnterWindow"></event>
6393<event name="OnEraseBackground"></event>
6394<event name="OnKeyDown">OnKeyDown</event>
6395<event name="OnKeyUp"></event>
6396<event name="OnKillFocus"></event>
6397<event name="OnLeaveWindow"></event>
6398<event name="OnLeftDClick"></event>
6399<event name="OnLeftDown"></event>
6400<event name="OnLeftUp"></event>
6401<event name="OnMiddleDClick"></event>
6402<event name="OnMiddleDown"></event>
6403<event name="OnMiddleUp"></event>
6404<event name="OnMotion"></event>
6405<event name="OnMouseEvents"></event>
6406<event name="OnMouseWheel"></event>
6407<event name="OnPaint"></event>
6408<event name="OnRightDClick"></event>
6409<event name="OnRightDown"></event>
6410<event name="OnRightUp"></event>
6411<event name="OnSetFocus"></event>
6412<event name="OnSize"></event>
6413<event name="OnText"></event>
6414<event name="OnTextEnter"></event>
6415<event name="OnTextMaxLen"></event>
6416<event name="OnTextURL"></event>
6417<event name="OnUpdateUI"></event>
6418</object>
6419</object>
6420<object class="sizeritem" expanded="1">
6421<property name="border">5</property>
6422<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
6423<property name="proportion">0</property>
6424<object class="wxTextCtrl" expanded="1">
6425<property name="bg"></property>
6426<property name="context_help"></property>
6427<property name="enabled">1</property>
6428<property name="fg"></property>
6429<property name="font"></property>
6430<property name="hidden">0</property>
6431<property name="id">wxID_ANY</property>
6432<property name="maximum_size">-1,-1</property>
6433<property name="maxlength">0</property>
6434<property name="minimum_size">-1,-1</property>
6435<property name="name">m_textCtrlField4</property>
6436<property name="permission">protected</property>
6437<property name="pos"></property>
6438<property name="size">1,-1</property>
6439<property name="style"></property>
6440<property name="subclass"></property>
6441<property name="tooltip"></property>
6442<property name="value"></property>
6443<property name="window_extra_style"></property>
6444<property name="window_name"></property>
6445<property name="window_style"></property>
6446<event name="OnChar"></event>
6447<event name="OnEnterWindow"></event>
6448<event name="OnEraseBackground"></event>
6449<event name="OnKeyDown">OnKeyDown</event>
6450<event name="OnKeyUp"></event>
6451<event name="OnKillFocus"></event>
6452<event name="OnLeaveWindow"></event>
6453<event name="OnLeftDClick"></event>
6454<event name="OnLeftDown"></event>
6455<event name="OnLeftUp"></event>
6456<event name="OnMiddleDClick"></event>
6457<event name="OnMiddleDown"></event>
6458<event name="OnMiddleUp"></event>
6459<event name="OnMotion"></event>
6460<event name="OnMouseEvents"></event>
6461<event name="OnMouseWheel"></event>
6462<event name="OnPaint"></event>
6463<event name="OnRightDClick"></event>
6464<event name="OnRightDown"></event>
6465<event name="OnRightUp"></event>
6466<event name="OnSetFocus"></event>
6467<event name="OnSize"></event>
6468<event name="OnText"></event>
6469<event name="OnTextEnter"></event>
6470<event name="OnTextMaxLen"></event>
6471<event name="OnTextURL"></event>
6472<event name="OnUpdateUI"></event>
6473</object>
6474</object>
6475<object class="sizeritem" expanded="1">
6476<property name="border">5</property>
6477<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
6478<property name="proportion">0</property>
6479<object class="wxButton" expanded="1">
6480<property name="bg"></property>
6481<property name="context_help"></property>
6482<property name="default">0</property>
6483<property name="enabled">1</property>
6484<property name="fg"></property>
6485<property name="font"></property>
6486<property name="hidden">0</property>
6487<property name="id">wxID_DEL4</property>
6488<property name="label">Delete</property>
6489<property name="maximum_size"></property>
6490<property name="minimum_size">-1,-1</property>
6491<property name="name">m_buttonDel4</property>
6492<property name="permission">protected</property>
6493<property name="pos"></property>
6494<property name="size">60,20</property>
6495<property name="style"></property>
6496<property name="subclass"></property>
6497<property name="tooltip"></property>
6498<property name="window_extra_style"></property>
6499<property name="window_name"></property>
6500<property name="window_style"></property>
6501<event name="OnButtonClick">OnButtonDel4</event>
6502<event name="OnChar"></event>
6503<event name="OnEnterWindow"></event>
6504<event name="OnEraseBackground"></event>
6505<event name="OnKeyDown"></event>
6506<event name="OnKeyUp"></event>
6507<event name="OnKillFocus"></event>
6508<event name="OnLeaveWindow"></event>
6509<event name="OnLeftDClick"></event>
6510<event name="OnLeftDown"></event>
6511<event name="OnLeftUp"></event>
6512<event name="OnMiddleDClick"></event>
6513<event name="OnMiddleDown"></event>

```



```

6514<event name="OnMiddleUp"></event>
6515<event name="OnMotion"></event>
6516<event name="OnMouseEvents"></event>
6517<event name="OnMouseWheel"></event>
6518<event name="OnPaint"></event>
6519<event name="OnRightDClick"></event>
6520<event name="OnRightDown"></event>
6521<event name="OnRightUp"></event>
6522<event name="OnSetFocus"></event>
6523<event name="OnSize"></event>
6524<event name="OnUpdateUI"></event>
6525</object>
6526</object>
6527<object class="sizeritem" expanded="1">
6528<property name="border">5</property>
6529<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
6530<property name="proportion">0</property>
6531<object class="wxTextCtrl" expanded="1">
6532<property name="bg"></property>
6533<property name="context_help"></property>
6534<property name="enabled">1</property>
6535<property name="fg"></property>
6536<property name="font"></property>
6537<property name="hidden">0</property>
6538<property name="id">wxID_ANY</property>
6539<property name="maximum_size"></property>
6540<property name="maxlength">0</property>
6541<property name="minimum_size">150,-1</property>
6542<property name="name">m_textCtrlLabels</property>
6543<property name="permission">protected</property>
6544<property name="pos"></property>
6545<property name="size"></property>
6546<property name="style"></property>
6547<property name="subclass"></property>
6548<property name="tooltip"></property>
6549<property name="value"></property>
6550<property name="window_extra_style"></property>
6551<property name="window_name"></property>
6552<property name="window_style"></property>
6553<event name="OnChar"></event>
6554<event name="OnEnterWindow"></event>
6555<event name="OnEraseBackground"></event>
6556<event name="OnKeyDown">OnKeyDown</event>
6557<event name="OnKeyUp"></event>
6558<event name="OnKillFocus"></event>
6559<event name="OnLeaveWindow"></event>
6560<event name="OnLeftDClick"></event>
6561<event name="OnLeftDown"></event>
6562<event name="OnLeftUp"></event>
6563<event name="OnMiddleDClick"></event>
6564<event name="OnMiddleDown"></event>
6565<event name="OnMiddleUp"></event>
6566<event name="OnMotion"></event>
6567<event name="OnMouseEvents"></event>
6568<event name="OnMouseWheel"></event>
6569<event name="OnPaint"></event>
6570<event name="OnRightDClick"></event>
6571<event name="OnRightDown"></event>
6572<event name="OnRightUp"></event>
6573<event name="OnSetFocus"></event>
6574<event name="OnSize"></event>
6575<event name="OnText"></event>
6576<event name="OnTextEnter"></event>
6577<event name="OnTextMaxLen"></event>
6578<event name="OnTextURL"></event>
6579<event name="OnUpdateUI"></event>
6580</object>
6581</object>
6582<object class="sizeritem" expanded="1">
6583<property name="border">5</property>
6584<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
6585<property name="proportion">0</property>
6586<object class="wxTextCtrl" expanded="1">
6587<property name="bg"></property>
6588<property name="context_help"></property>
6589<property name="enabled">1</property>
6590<property name="fg"></property>
6591<property name="font"></property>
6592<property name="hidden">0</property>
6593<property name="id">wxID_ANY</property>
6594<property name="maximum_size">1,-1</property>
6595<property name="maxlength">0</property>
6596<property name="minimum_size">1,-1</property>
6597<property name="name">m_textCtrlField5</property>
6598<property name="permission">protected</property>
6599<property name="pos"></property>
6600<property name="size">1,-1</property>
6601<property name="style"></property>
6602<property name="subclass"></property>
6603<property name="tooltip"></property>
6604<property name="value"></property>
6605<property name="window_extra_style"></property>
6606<property name="window_name"></property>
6607<property name="window_style"></property>
6608<event name="OnChar"></event>
6609<event name="OnEnterWindow"></event>
6610<event name="OnEraseBackground"></event>
6611<event name="OnKeyDown">OnKeyDown</event>
6612<event name="OnKeyUp"></event>
6613<event name="OnKillFocus"></event>
6614<event name="OnLeaveWindow"></event>
6615<event name="OnLeftDClick"></event>
6616<event name="OnLeftDown"></event>
6617<event name="OnLeftUp"></event>
6618<event name="OnMiddleDClick"></event>
6619<event name="OnMiddleDown"></event>
6620<event name="OnMiddleUp"></event>
6621<event name="OnMotion"></event>
6622<event name="OnMouseEvents"></event>
6623<event name="OnMouseWheel"></event>
6624<event name="OnPaint"></event>
6625<event name="OnRightDClick"></event>
6626<event name="OnRightDown"></event>
6627<event name="OnRightUp"></event>
6628<event name="OnSetFocus"></event>
6629<event name="OnSize"></event>
6630<event name="OnText"></event>
6631<event name="OnTextEnter"></event>
6632<event name="OnTextMaxLen"></event>
6633<event name="OnTextURL"></event>
6634<event name="OnUpdateUI"></event>
6635</object>
6636</object>
6637<object class="sizeritem" expanded="1">
6638<property name="border">5</property>
6639<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
6640<property name="proportion">0</property>
6641<object class="wxButton" expanded="1">
6642<property name="bg"></property>
6643<property name="context_help"></property>
6644<property name="default">0</property>
6645<property name="enabled">1</property>
6646<property name="fg"></property>
6647<property name="font"></property>
6648<property name="hidden">0</property>
6649<property name="id">wxID_DE15</property>
6650<property name="label">Delete</property>
6651<property name="maximum_size"></property>
6652<property name="minimum_size">-1,-1</property>
6653<property name="name">buttonDe15</property>
6654<property name="permission">protected</property>
6655<property name="pos"></property>
6656<property name="size">60,20</property>
6657<property name="style"></property>
6658<property name="subclass"></property>
6659<property name="tooltip"></property>
6660<property name="window_extra_style"></property>
6661<property name="window_name"></property>
6662<property name="window_style"></property>
6663<event name="OnButtonClick">OnButtonDe15</event>
6664<event name="OnChar"></event>
6665<event name="OnEnterWindow"></event>
6666<event name="OnEraseBackground"></event>
6667<event name="OnKeyDown"></event>
6668<event name="OnKeyUp"></event>
6669<event name="OnKillFocus"></event>
6670<event name="OnLeaveWindow"></event>
6671<event name="OnLeftDClick"></event>
6672<event name="OnLeftDown"></event>
6673<event name="OnLeftUp"></event>
6674<event name="OnMiddleDClick"></event>
6675<event name="OnMiddleDown"></event>
6676<event name="OnMiddleUp"></event>
6677<event name="OnMotion"></event>
6678<event name="OnMouseEvents"></event>
6679<event name="OnMouseWheel"></event>
6680<event name="OnPaint"></event>
6681<event name="OnRightDClick"></event>
6682<event name="OnRightDown"></event>
6683<event name="OnRightUp"></event>
6684<event name="OnSetFocus"></event>
6685<event name="OnSize"></event>
6686<event name="OnUpdateUI"></event>
6687</object>
6688</object>
6689<object class="sizeritem" expanded="1">
6690<property name="border">5</property>
6691<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
6692<property name="proportion">0</property>
6693<object class="wxTextCtrl" expanded="1">
6694<property name="bg"></property>
6695<property name="context_help"></property>
6696<property name="enabled">1</property>
6697<property name="fg"></property>
6698<property name="font"></property>
6699<property name="hidden">0</property>
6700<property name="id">wxID_ANY</property>
6701<property name="maximum_size"></property>
6702<property name="maxlength">0</property>
6703<property name="minimum_size">150,-1</property>
6704<property name="name">m_textCtrlLabel6</property>
6705<property name="permission">protected</property>
6706<property name="pos"></property>
6707<property name="size"></property>
6708<property name="style"></property>
6709<property name="subclass"></property>
6710<property name="tooltip"></property>
6711<property name="value"></property>
6712<property name="window_extra_style"></property>
6713<property name="window_name"></property>
6714<property name="window_style"></property>
6715<event name="OnChar"></event>
6716<event name="OnEnterWindow"></event>
6717<event name="OnEraseBackground"></event>

```

```

6718<event name="OnKeyDown">OnKeyDown</event>
6719<event name="OnKeyUp"></event>
6720<event name="OnKillFocus"></event>
6721<event name="OnLeaveWindow"></event>
6722<event name="OnLeftDClick"></event>
6723<event name="OnLeftDown"></event>
6724<event name="OnLeftUp"></event>
6725<event name="OnMiddleDClick"></event>
6726<event name="OnMiddleDown"></event>
6727<event name="OnMiddleUp"></event>
6728<event name="OnMotion"></event>
6729<event name="OnMouseEvents"></event>
6730<event name="OnMouseWheel"></event>
6731<event name="OnPaint"></event>
6732<event name="OnRightDClick"></event>
6733<event name="OnRightDown"></event>
6734<event name="OnRightUp"></event>
6735<event name="OnSetFocus"></event>
6736<event name="OnSize"></event>
6737<event name="OnText"></event>
6738<event name="OnTextEnter"></event>
6739<event name="OnTextMaxLen"></event>
6740<event name="OnTextURL"></event>
6741<event name="OnUpdateUI"></event>
6742</object>
6743</object>
6744<object class="sizeritem" expanded="1">
6745<property name="border">5</property>
6746<property
name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
6747<property name="proportion">0</property>
6748<object class="wxTextCtrl" expanded="1">
6749<property name="bg"></property>
6750<property name="context_help"></property>
6751<property name="enabled">1</property>
6752<property name="fg"></property>
6753<property name="font"></property>
6754<property name="hidden">0</property>
6755<property name="id">wxID_ANY</property>
6756<property name="maximum_size">1,1</property>
6757<property name="maxlength">0</property>
6758<property name="minimum_size">1,1</property>
6759<property name="name">m_textCtrlField6</property>
6760<property name="permission">protected</property>
6761<property name="pos"></property>
6762<property name="size">1,1</property>
6763<property name="style"></property>
6764<property name="subclass"></property>
6765<property name="tooltip"></property>
6766<property name="value"></property>
6767<property name="window_extra_style"></property>
6768<property name="window_name"></property>
6769<property name="window_style"></property>
6770<event name="OnChar"></event>
6771<event name="OnEnterWindow"></event>
6772<event name="OnEraseBackground"></event>
6773<event name="OnKeyDown">OnKeyDown</event>
6774<event name="OnKeyUp"></event>
6775<event name="OnKillFocus"></event>
6776<event name="OnLeaveWindow"></event>
6777<event name="OnLeftDClick"></event>
6778<event name="OnLeftDown"></event>
6779<event name="OnLeftUp"></event>
6780<event name="OnMiddleDClick"></event>
6781<event name="OnMiddleDown"></event>
6782<event name="OnMiddleUp"></event>
6783<event name="OnMotion"></event>
6784<event name="OnMouseEvents"></event>
6785<event name="OnMouseWheel"></event>
6786<event name="OnPaint"></event>
6787<event name="OnRightDClick"></event>
6788<event name="OnRightDown"></event>
6789<event name="OnRightUp"></event>
6790<event name="OnSetFocus"></event>
6791<event name="OnSize"></event>
6792<event name="OnText"></event>
6793<event name="OnTextEnter"></event>
6794<event name="OnTextMaxLen"></event>
6795<event name="OnTextURL"></event>
6796<event name="OnUpdateUI"></event>
6797</object>
6798</object>
6799<object class="sizeritem" expanded="1">
6800<property name="border">5</property>
6801<property
name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
6802<property name="proportion">0</property>
6803<object class="wxButton" expanded="1">
6804<property name="bg"></property>
6805<property name="context_help"></property>
6806<property name="default">0</property>
6807<property name="enabled">1</property>
6808<property name="fg"></property>
6809<property name="font"></property>
6810<property name="hidden">0</property>
6811<property name="id">wxID_DELETE</property>
6812<property name="label">Delete</property>
6813<property name="maximum_size"></property>
6814<property name="minimum_size">1,1</property>
6815<property name="name">m_buttonDel6</property>
6816<property name="permission">protected</property>
6817<property name="pos"></property>
6818<property name="size">60,20</property>
6819<property name="style"></property>
6820<property name="subclass"></property>
6821<property name="tooltip"></property>
6822<property name="window_extra_style"></property>
6823<property name="window_name"></property>
6824<property name="window_style"></property>
6825<event name="OnButtonDClick">OnButtonDel6</event>
6826<event name="OnChar"></event>
6827<event name="OnEnterWindow"></event>
6828<event name="OnEraseBackground"></event>
6829<event name="OnKeyDown"></event>
6830<event name="OnKeyUp"></event>
6831<event name="OnKillFocus"></event>
6832<event name="OnLeaveWindow"></event>
6833<event name="OnLeftDClick"></event>
6834<event name="OnLeftDown"></event>
6835<event name="OnLeftUp"></event>
6836<event name="OnMiddleDClick"></event>
6837<event name="OnMiddleDown"></event>
6838<event name="OnMiddleUp"></event>
6839<event name="OnMotion"></event>
6840<event name="OnMouseEvents"></event>
6841<event name="OnMouseWheel"></event>
6842<event name="OnPaint"></event>
6843<event name="OnRightDClick"></event>
6844<event name="OnRightDown"></event>
6845<event name="OnRightUp"></event>
6846<event name="OnSetFocus"></event>
6847<event name="OnSize"></event>
6848<event name="OnUpdateUI"></event>
6849</object>
6850</object>
6851<object class="sizeritem" expanded="1">
6852<property name="border">5</property>
6853<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
6854<property name="proportion">0</property>
6855<object class="wxTextCtrl" expanded="1">
6856<property name="bg"></property>
6857<property name="context_help"></property>
6858<property name="enabled">1</property>
6859<property name="fg"></property>
6860<property name="font"></property>
6861<property name="hidden">0</property>
6862<property name="id">wxID_ANY</property>
6863<property name="maximum_size"></property>
6864<property name="maxlength">0</property>
6865<property name="minimum_size">150,1</property>
6866<property name="name">m_textCtrlLabel7</property>
6867<property name="permission">protected</property>
6868<property name="pos"></property>
6869<property name="size"></property>
6870<property name="style"></property>
6871<property name="subclass"></property>
6872<property name="tooltip"></property>
6873<property name="value"></property>
6874<property name="window_extra_style"></property>
6875<property name="window_name"></property>
6876<property name="window_style"></property>
6877<event name="OnChar"></event>
6878<event name="OnEnterWindow"></event>
6879<event name="OnEraseBackground"></event>
6880<event name="OnKeyDown">OnKeyDown</event>
6881<event name="OnKeyUp"></event>
6882<event name="OnKillFocus"></event>
6883<event name="OnLeaveWindow"></event>
6884<event name="OnLeftDClick"></event>
6885<event name="OnLeftDown"></event>
6886<event name="OnLeftUp"></event>
6887<event name="OnMiddleDClick"></event>
6888<event name="OnMiddleDown"></event>
6889<event name="OnMiddleUp"></event>
6890<event name="OnMotion"></event>
6891<event name="OnMouseEvents"></event>
6892<event name="OnMouseWheel"></event>
6893<event name="OnPaint"></event>
6894<event name="OnRightDClick"></event>
6895<event name="OnRightDown"></event>
6896<event name="OnRightUp"></event>
6897<event name="OnSetFocus"></event>
6898<event name="OnSize"></event>
6899<event name="OnText"></event>
6900<event name="OnTextEnter"></event>
6901<event name="OnTextMaxLen"></event>
6902<event name="OnTextURL"></event>
6903<event name="OnUpdateUI"></event>
6904</object>
6905</object>
6906<object class="sizeritem" expanded="1">
6907<property name="border">5</property>
6908<property
name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
6909<property name="proportion">0</property>
6910<object class="wxTextCtrl" expanded="1">
6911<property name="bg"></property>
6912<property name="context_help"></property>
6913<property name="enabled">1</property>
6914<property name="fg"></property>
6915<property name="font"></property>
6916<property name="hidden">0</property>
6917<property name="id">wxID_ANY</property>
6918<property name="maximum_size">1,1</property>
6919<property name="maxlength">0</property>
6920<property name="minimum_size">1,1</property>

```

```

6921<property name="name">m_textCtrlField7</property>
6922<property name="permission">protected</property>
6923<property name="pos">1</property>
6924<property name="size">1,-1</property>
6925<property name="style"></property>
6926<property name="subclass"></property>
6927<property name="tooltip"></property>
6928<property name="value"></property>
6929<property name="window_extra_style"></property>
6930<property name="window_name"></property>
6931<property name="window_style"></property>
6932<event name="OnChar"></event>
6933<event name="OnEnterWindow"></event>
6934<event name="OnEraseBackground"></event>
6935<event name="OnKeyDown">OnKeyDown</event>
6936<event name="OnKeyUp"></event>
6937<event name="OnKillFocus"></event>
6938<event name="OnLeaveWindow"></event>
6939<event name="OnLeftDClick"></event>
6940<event name="OnLeftDown"></event>
6941<event name="OnLeftUp"></event>
6942<event name="OnMiddleDClick"></event>
6943<event name="OnMiddleDown"></event>
6944<event name="OnMiddleUp"></event>
6945<event name="OnMotion"></event>
6946<event name="OnMouseEvents"></event>
6947<event name="OnMouseWheel"></event>
6948<event name="OnPaint"></event>
6949<event name="OnRightDClick"></event>
6950<event name="OnRightDown"></event>
6951<event name="OnRightUp"></event>
6952<event name="OnSetFocus"></event>
6953<event name="OnSize"></event>
6954<event name="OnText"></event>
6955<event name="OnTextEnter"></event>
6956<event name="OnTextMaxLen"></event>
6957<event name="OnTextURL"></event>
6958<event name="OnUpdateUI"></event>
6959</object>
6960</object>
6961<object class="sizeritem" expanded="1">
6962<property name="border">5</property>
6963<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
6964<property name="proportion">0</property>
6965<object class="wxButton" expanded="1">
6966<property name="bg"></property>
6967<property name="context_help"></property>
6968<property name="default">0</property>
6969<property name="enabled">1</property>
6970<property name="fg"></property>
6971<property name="font"></property>
6972<property name="hidden">0</property>
6973<property name="id">wxID_DEL</property>
6974<property name="label">Delete</property>
6975<property name="maximum_size"></property>
6976<property name="minimum_size">1,-1</property>
6977<property name="name">m_buttonDel7</property>
6978<property name="permission">protected</property>
6979<property name="pos"></property>
6980<property name="size">60,20</property>
6981<property name="style"></property>
6982<property name="subclass"></property>
6983<property name="tooltip"></property>
6984<property name="window_extra_style"></property>
6985<property name="window_name"></property>
6986<property name="window_style"></property>
6987<event name="OnButtonDClick">OnButtonDel7</event>
6988<event name="OnChar"></event>
6989<event name="OnEnterWindow"></event>
6990<event name="OnEraseBackground"></event>
6991<event name="OnKeyDown"></event>
6992<event name="OnKeyUp"></event>
6993<event name="OnKillFocus"></event>
6994<event name="OnLeaveWindow"></event>
6995<event name="OnLeftDClick"></event>
6996<event name="OnLeftDown"></event>
6997<event name="OnLeftUp"></event>
6998<event name="OnMiddleDClick"></event>
6999<event name="OnMiddleDown"></event>
7000<event name="OnMiddleUp"></event>
7001<event name="OnMotion"></event>
7002<event name="OnMouseEvents"></event>
7003<event name="OnMouseWheel"></event>
7004<event name="OnPaint"></event>
7005<event name="OnRightDClick"></event>
7006<event name="OnRightDown"></event>
7007<event name="OnRightUp"></event>
7008<event name="OnSetFocus"></event>
7009<event name="OnSize"></event>
7010<event name="OnUpdateUI"></event>
7011</object>
7012</object>
7013<object class="sizeritem" expanded="1">
7014<property name="border">5</property>
7015<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7016<property name="proportion">0</property>
7017<object class="wxTextCtrl" expanded="1">
7018<property name="bg"></property>
7019<property name="context_help"></property>
7020<property name="enabled">1</property>
7021<property name="fg"></property>
7022<property name="font"></property>
7023<property name="hidden">0</property>
7024<property name="id">wxID_ANY</property>
7025<property name="maximum_size"></property>
7026<property name="maxlength">0</property>
7027<property name="minimum_size">150,-1</property>
7028<property name="name">m_textCtrlLabel8</property>
7029<property name="permission">protected</property>
7030<property name="pos"></property>
7031<property name="size"></property>
7032<property name="style"></property>
7033<property name="subclass"></property>
7034<property name="tooltip"></property>
7035<property name="value"></property>
7036<property name="window_extra_style"></property>
7037<property name="window_name"></property>
7038<property name="window_style"></property>
7039<event name="OnChar"></event>
7040<event name="OnEnterWindow"></event>
7041<event name="OnEraseBackground"></event>
7042<event name="OnKeyDown">OnKeyDown</event>
7043<event name="OnKeyUp"></event>
7044<event name="OnKillFocus"></event>
7045<event name="OnLeaveWindow"></event>
7046<event name="OnLeftDClick"></event>
7047<event name="OnLeftDown"></event>
7048<event name="OnLeftUp"></event>
7049<event name="OnMiddleDClick"></event>
7050<event name="OnMiddleDown"></event>
7051<event name="OnMiddleUp"></event>
7052<event name="OnMotion"></event>
7053<event name="OnMouseEvents"></event>
7054<event name="OnMouseWheel"></event>
7055<event name="OnPaint"></event>
7056<event name="OnRightDClick"></event>
7057<event name="OnRightDown"></event>
7058<event name="OnRightUp"></event>
7059<event name="OnSetFocus"></event>
7060<event name="OnSize"></event>
7061<event name="OnText"></event>
7062<event name="OnTextEnter"></event>
7063<event name="OnTextMaxLen"></event>
7064<event name="OnTextURL"></event>
7065<event name="OnUpdateUI"></event>
7066</object>
7067</object>
7068<object class="sizeritem" expanded="1">
7069<property name="border">5</property>
7070<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
7071<property name="proportion">0</property>
7072<object class="wxTextCtrl" expanded="1">
7073<property name="bg"></property>
7074<property name="context_help"></property>
7075<property name="enabled">1</property>
7076<property name="fg"></property>
7077<property name="font"></property>
7078<property name="hidden">0</property>
7079<property name="id">wxID_ANY</property>
7080<property name="maximum_size">1,-1</property>
7081<property name="maxlength">0</property>
7082<property name="minimum_size">1,-1</property>
7083<property name="name">m_textCtrlField08</property>
7084<property name="permission">protected</property>
7085<property name="pos"></property>
7086<property name="size">1,-1</property>
7087<property name="style"></property>
7088<property name="subclass"></property>
7089<property name="tooltip"></property>
7090<property name="value"></property>
7091<property name="window_extra_style"></property>
7092<property name="window_name"></property>
7093<property name="window_style"></property>
7094<event name="OnChar"></event>
7095<event name="OnEnterWindow"></event>
7096<event name="OnEraseBackground"></event>
7097<event name="OnKeyDown">OnKeyDown</event>
7098<event name="OnKeyUp"></event>
7099<event name="OnKillFocus"></event>
7100<event name="OnLeaveWindow"></event>
7101<event name="OnLeftDClick"></event>
7102<event name="OnLeftDown"></event>
7103<event name="OnLeftUp"></event>
7104<event name="OnMiddleDClick"></event>
7105<event name="OnMiddleDown"></event>
7106<event name="OnMiddleUp"></event>
7107<event name="OnMotion"></event>
7108<event name="OnMouseEvents"></event>
7109<event name="OnMouseWheel"></event>
7110<event name="OnPaint"></event>
7111<event name="OnRightDClick"></event>
7112<event name="OnRightDown"></event>
7113<event name="OnRightUp"></event>
7114<event name="OnSetFocus"></event>
7115<event name="OnSize"></event>
7116<event name="OnText"></event>
7117<event name="OnTextEnter"></event>
7118<event name="OnTextMaxLen"></event>
7119<event name="OnTextURL"></event>
7120<event name="OnUpdateUI"></event>
7121</object>
7122</object>
7123<object class="sizeritem" expanded="1">
7124<property name="border">5</property>

```



```

7125<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
7126<property name="proportion">0</property>
7127<object class="wxButton" expanded="1">
7128<property name="bg"></property>
7129<property name="context_help"></property>
7130<property name="default">0</property>
7131<property name="enabled">1</property>
7132<property name="fg"></property>
7133<property name="font"></property>
7134<property name="hidden">0</property>
7135<property name="id">wxID_DEL8</property>
7136<property name="label">Delete</property>
7137<property name="maximum_size"></property>
7138<property name="minimum_size">-1,-1</property>
7139<property name="name">m_buttonDel8</property>
7140<property name="permission">protected</property>
7141<property name="pos"></property>
7142<property name="size">60,20</property>
7143<property name="style"></property>
7144<property name="subclass"></property>
7145<property name="tooltip"></property>
7146<property name="window_extra_style"></property>
7147<property name="window_name"></property>
7148<property name="window_style"></property>
7149<event name="OnButtonClick">OnButtonDel8</event>
7150<event name="OnChar"></event>
7151<event name="OnEnterWindow"></event>
7152<event name="OnEraseBackground"></event>
7153<event name="OnKeyDown"></event>
7154<event name="OnKeyUp"></event>
7155<event name="OnKillFocus"></event>
7156<event name="OnLeaveWindow"></event>
7157<event name="OnLeftDClick"></event>
7158<event name="OnLeftDown"></event>
7159<event name="OnLeftUp"></event>
7160<event name="OnMiddleDClick"></event>
7161<event name="OnMiddleDown"></event>
7162<event name="OnMiddleUp"></event>
7163<event name="OnMotion"></event>
7164<event name="OnMouseEvents"></event>
7165<event name="OnMouseWheel"></event>
7166<event name="OnPaint"></event>
7167<event name="OnRightDClick"></event>
7168<event name="OnRightDown"></event>
7169<event name="OnRightUp"></event>
7170<event name="OnSetFocus"></event>
7171<event name="OnSize"></event>
7172<event name="OnUpdateUI"></event>
7173</object>
7174</object>
7175<object class="sizeritem" expanded="1">
7176<property name="border"></property>
7177<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7178<property name="proportion">0</property>
7179<object class="wxTextCtrl" expanded="1">
7180<property name="bg"></property>
7181<property name="context_help"></property>
7182<property name="enabled">1</property>
7183<property name="fg"></property>
7184<property name="font"></property>
7185<property name="hidden">0</property>
7186<property name="id">wxID_ANY</property>
7187<property name="maximum_size"></property>
7188<property name="maxlength">0</property>
7189<property name="minimum_size">150,-1</property>
7190<property name="name">m_textCtrlLabel9</property>
7191<property name="permission">protected</property>
7192<property name="pos"></property>
7193<property name="size"></property>
7194<property name="style"></property>
7195<property name="subclass"></property>
7196<property name="tooltip"></property>
7197<property name="value"></property>
7198<property name="window_extra_style"></property>
7199<property name="window_name"></property>
7200<property name="window_style"></property>
7201<event name="OnChar"></event>
7202<event name="OnEnterWindow"></event>
7203<event name="OnEraseBackground"></event>
7204<event name="OnKeyDown">OnKeyDown</event>
7205<event name="OnKeyUp"></event>
7206<event name="OnKillFocus"></event>
7207<event name="OnLeaveWindow"></event>
7208<event name="OnLeftDClick"></event>
7209<event name="OnLeftDown"></event>
7210<event name="OnLeftUp"></event>
7211<event name="OnMiddleDClick"></event>
7212<event name="OnMiddleDown"></event>
7213<event name="OnMiddleUp"></event>
7214<event name="OnMotion"></event>
7215<event name="OnMouseEvents"></event>
7216<event name="OnMouseWheel"></event>
7217<event name="OnPaint"></event>
7218<event name="OnRightDClick"></event>
7219<event name="OnRightDown"></event>
7220<event name="OnRightUp"></event>
7221<event name="OnSetFocus"></event>
7222<event name="OnSize"></event>
7223<event name="OnText"></event>
7224<event name="OnTextEnter"></event>
7225<event name="OnTextMaxLen"></event>
7226<event name="OnTextURL"></event>
7227<event name="OnUpdateUI"></event>
7228</object>
7229</object>
7230<object class="sizeritem" expanded="1">
7231<property name="border">5</property>
7232<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
7233<property name="proportion">0</property>
7234<object class="wxTextCtrl" expanded="1">
7235<property name="bg"></property>
7236<property name="context_help"></property>
7237<property name="enabled">1</property>
7238<property name="fg"></property>
7239<property name="font"></property>
7240<property name="hidden">0</property>
7241<property name="id">wxID_ANY</property>
7242<property name="maximum_size">-1,-1</property>
7243<property name="maxlength">0</property>
7244<property name="minimum_size">-1,-1</property>
7245<property name="name">m_textCtrlField9</property>
7246<property name="permission">protected</property>
7247<property name="pos"></property>
7248<property name="size">-1,-1</property>
7249<property name="style"></property>
7250<property name="subclass"></property>
7251<property name="tooltip"></property>
7252<property name="value"></property>
7253<property name="window_extra_style"></property>
7254<property name="window_name"></property>
7255<property name="window_style"></property>
7256<event name="OnChar"></event>
7257<event name="OnEnterWindow"></event>
7258<event name="OnEraseBackground"></event>
7259<event name="OnKeyDown">OnKeyDown</event>
7260<event name="OnKeyUp"></event>
7261<event name="OnKillFocus"></event>
7262<event name="OnLeaveWindow"></event>
7263<event name="OnLeftDClick"></event>
7264<event name="OnLeftDown"></event>
7265<event name="OnLeftUp"></event>
7266<event name="OnMiddleDClick"></event>
7267<event name="OnMiddleDown"></event>
7268<event name="OnMiddleUp"></event>
7269<event name="OnMotion"></event>
7270<event name="OnMouseEvents"></event>
7271<event name="OnMouseWheel"></event>
7272<event name="OnPaint"></event>
7273<event name="OnRightDClick"></event>
7274<event name="OnRightDown"></event>
7275<event name="OnRightUp"></event>
7276<event name="OnSetFocus"></event>
7277<event name="OnSize"></event>
7278<event name="OnText"></event>
7279<event name="OnTextEnter"></event>
7280<event name="OnTextMaxLen"></event>
7281<event name="OnTextURL"></event>
7282<event name="OnUpdateUI"></event>
7283</object>
7284</object>
7285<object class="sizeritem" expanded="1">
7286<property name="border">5</property>
7287<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
7288<property name="proportion">0</property>
7289<object class="wxButton" expanded="1">
7290<property name="bg"></property>
7291<property name="context_help"></property>
7292<property name="default">0</property>
7293<property name="enabled">1</property>
7294<property name="fg"></property>
7295<property name="font"></property>
7296<property name="hidden">0</property>
7297<property name="id">wxID_DEL9</property>
7298<property name="label">Delete</property>
7299<property name="maximum_size"></property>
7300<property name="minimum_size">-1,-1</property>
7301<property name="name">m_buttonDel9</property>
7302<property name="permission">protected</property>
7303<property name="pos"></property>
7304<property name="size">60,20</property>
7305<property name="style"></property>
7306<property name="subclass"></property>
7307<property name="tooltip"></property>
7308<property name="window_extra_style"></property>
7309<property name="window_name"></property>
7310<property name="window_style"></property>
7311<event name="OnButtonClick">OnButtonDel9</event>
7312<event name="OnChar"></event>
7313<event name="OnEnterWindow"></event>
7314<event name="OnEraseBackground"></event>
7315<event name="OnKeyDown"></event>
7316<event name="OnKeyUp"></event>
7317<event name="OnKillFocus"></event>
7318<event name="OnLeaveWindow"></event>
7319<event name="OnLeftDClick"></event>
7320<event name="OnLeftDown"></event>
7321<event name="OnLeftUp"></event>
7322<event name="OnMiddleDClick"></event>
7323<event name="OnMiddleDown"></event>
7324<event name="OnMiddleUp"></event>
7325<event name="OnMotion"></event>
7326<event name="OnMouseEvents"></event>
7327<event name="OnMouseWheel"></event>

```

```

7328<event name="OnPaint"></event>
7329<event name="OnRightDClick"></event>
7330<event name="OnRightDown"></event>
7331<event name="OnRightUp"></event>
7332<event name="OnSetFocus"></event>
7333<event name="OnSize"></event>
7334<event name="OnUpdateUI"></event>
7335</object>
7336</object>
7337<object class="sizeritem" expanded="1">
7338<property name="border">5</property>
7339<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7340<property name="proportion">0</property>
7341<object class="wxTextCtrl" expanded="1">
7342<property name="bg"></property>
7343<property name="context_help"></property>
7344<property name="enabled">1</property>
7345<property name="fg"></property>
7346<property name="font"></property>
7347<property name="hidden">0</property>
7348<property name="id">wxID_ANY</property>
7349<property name="maximum_size"></property>
7350<property name="maxlength">0</property>
7351<property name="minimum_size">150,-1</property>
7352<property name="name">m_textCtrlLabel10</property>
7353<property name="permission">protected</property>
7354<property name="pos"></property>
7355<property name="size"></property>
7356<property name="style"></property>
7357<property name="subclass"></property>
7358<property name="tooltip"></property>
7359<property name="value"></property>
7360<property name="window_extra_style"></property>
7361<property name="window_name"></property>
7362<property name="window_style"></property>
7363<event name="OnChar"></event>
7364<event name="OnEnterWindow"></event>
7365<event name="OnEraseBackground"></event>
7366<event name="OnKeyDown">OnKeyDown</event>
7367<event name="OnKeyUp"></event>
7368<event name="OnKillFocus"></event>
7369<event name="OnLeaveWindow"></event>
7370<event name="OnLeftDClick"></event>
7371<event name="OnLeftDown"></event>
7372<event name="OnLeftUp"></event>
7373<event name="OnMiddleDClick"></event>
7374<event name="OnMiddleDown"></event>
7375<event name="OnMiddleUp"></event>
7376<event name="OnMotion"></event>
7377<event name="OnMouseEvents"></event>
7378<event name="OnMouseWheel"></event>
7379<event name="OnPaint"></event>
7380<event name="OnRightDClick"></event>
7381<event name="OnRightDown"></event>
7382<event name="OnRightUp"></event>
7383<event name="OnSetFocus"></event>
7384<event name="OnSize"></event>
7385<event name="OnText"></event>
7386<event name="OnTextEnter"></event>
7387<event name="OnTextMaxLen"></event>
7388<event name="OnTextURL"></event>
7389<event name="OnUpdateUI"></event>
7390</object>
7391</object>
7392<object class="sizeritem" expanded="1">
7393<property name="border">5</property>
7394<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
7395<property name="proportion">0</property>
7396<object class="wxTextCtrl" expanded="1">
7397<property name="bg"></property>
7398<property name="context_help"></property>
7399<property name="enabled">1</property>
7400<property name="fg"></property>
7401<property name="font"></property>
7402<property name="hidden">0</property>
7403<property name="id">wxID_ANY</property>
7404<property name="maximum_size">-1,-1</property>
7405<property name="maxlength">0</property>
7406<property name="minimum_size">-1,-1</property>
7407<property name="name">m_textCtrlField10</property>
7408<property name="permission">protected</property>
7409<property name="pos"></property>
7410<property name="size">-1,-1</property>
7411<property name="style"></property>
7412<property name="subclass"></property>
7413<property name="tooltip"></property>
7414<property name="value"></property>
7415<property name="window_extra_style"></property>
7416<property name="window_name"></property>
7417<property name="window_style"></property>
7418<event name="OnChar"></event>
7419<event name="OnEnterWindow"></event>
7420<event name="OnEraseBackground"></event>
7421<event name="OnKeyDown">OnKeyDown</event>
7422<event name="OnKeyUp"></event>
7423<event name="OnKillFocus"></event>
7424<event name="OnLeaveWindow"></event>
7425<event name="OnLeftDClick"></event>
7426<event name="OnLeftDown"></event>
7427<event name="OnLeftUp"></event>
7428<event name="OnMiddleDClick"></event>
7429<event name="OnMiddleDown"></event>
7430<event name="OnMiddleUp"></event>
7431<event name="OnMotion"></event>
7432<event name="OnMouseEvents"></event>
7433<event name="OnMouseWheel"></event>
7434<event name="OnPaint"></event>
7435<event name="OnRightDClick"></event>
7436<event name="OnRightDown"></event>
7437<event name="OnRightUp"></event>
7438<event name="OnSetFocus"></event>
7439<event name="OnSize"></event>
7440<event name="OnText"></event>
7441<event name="OnTextEnter"></event>
7442<event name="OnTextMaxLen"></event>
7443<event name="OnTextURL"></event>
7444<event name="OnUpdateUI"></event>
7445</object>
7446</object>
7447<object class="sizeritem" expanded="1">
7448<property name="border">5</property>
7449<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
7450<property name="proportion">0</property>
7451<object class="wxButton" expanded="1">
7452<property name="bg"></property>
7453<property name="context_help"></property>
7454<property name="default">0</property>
7455<property name="enabled">1</property>
7456<property name="fg"></property>
7457<property name="font"></property>
7458<property name="hidden">0</property>
7459<property name="id">wxID_DEL10</property>
7460<property name="label">Delete</property>
7461<property name="maximum_size">-1,-1</property>
7462<property name="minimum_size">-1,-1</property>
7463<property name="name">m_buttonDel10</property>
7464<property name="permission">protected</property>
7465<property name="pos"></property>
7466<property name="size">60,20</property>
7467<property name="style"></property>
7468<property name="subclass"></property>
7469<property name="tooltip"></property>
7470<property name="window_extra_style"></property>
7471<property name="window_name"></property>
7472<property name="window_style"></property>
7473<event name="OnButtonClick">OnButtonDel10</event>
7474<event name="OnChar"></event>
7475<event name="OnEnterWindow"></event>
7476<event name="OnEraseBackground"></event>
7477<event name="OnKeyDown"></event>
7478<event name="OnKeyUp"></event>
7479<event name="OnKillFocus"></event>
7480<event name="OnLeaveWindow"></event>
7481<event name="OnLeftDClick"></event>
7482<event name="OnLeftDown"></event>
7483<event name="OnLeftUp"></event>
7484<event name="OnMiddleDClick"></event>
7485<event name="OnMiddleDown"></event>
7486<event name="OnMiddleUp"></event>
7487<event name="OnMotion"></event>
7488<event name="OnMouseEvents"></event>
7489<event name="OnMouseWheel"></event>
7490<event name="OnPaint"></event>
7491<event name="OnRightDClick"></event>
7492<event name="OnRightDown"></event>
7493<event name="OnRightUp"></event>
7494<event name="OnSetFocus"></event>
7495<event name="OnSize"></event>
7496<event name="OnUpdateUI"></event>
7497</object>
7498</object>
7499<object class="sizeritem" expanded="1">
7500<property name="border">5</property>
7501<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7502<property name="proportion">0</property>
7503<object class="wxTextCtrl" expanded="1">
7504<property name="bg"></property>
7505<property name="context_help"></property>
7506<property name="enabled">1</property>
7507<property name="fg"></property>
7508<property name="font"></property>
7509<property name="hidden">0</property>
7510<property name="id">wxID_ANY</property>
7511<property name="maximum_size"></property>
7512<property name="maxlength">0</property>
7513<property name="minimum_size">150,-1</property>
7514<property name="name">m_textCtrlLabel11</property>
7515<property name="permission">protected</property>
7516<property name="pos"></property>
7517<property name="size"></property>
7518<property name="style"></property>
7519<property name="subclass"></property>
7520<property name="tooltip"></property>
7521<property name="value"></property>
7522<property name="window_extra_style"></property>
7523<property name="window_name"></property>
7524<property name="window_style"></property>
7525<event name="OnChar"></event>
7526<event name="OnEnterWindow"></event>
7527<event name="OnEraseBackground"></event>
7528<event name="OnKeyDown">OnKeyDown</event>
7529<event name="OnKeyUp"></event>
7530<event name="OnKillFocus"></event>
7531<event name="OnLeaveWindow"></event>

```

```

7532<event name="OnLeftDClick"></event>
7533<event name="OnLeftDown"></event>
7534<event name="OnLeftUp"></event>
7535<event name="OnMiddleDClick"></event>
7536<event name="OnMiddleDown"></event>
7537<event name="OnMiddleUp"></event>
7538<event name="OnMotion"></event>
7539<event name="OnMouseEvents"></event>
7540<event name="OnMouseWheel"></event>
7541<event name="OnPaint"></event>
7542<event name="OnRightDClick"></event>
7543<event name="OnRightDown"></event>
7544<event name="OnRightUp"></event>
7545<event name="OnSetFocus"></event>
7546<event name="OnSize"></event>
7547<event name="OnText"></event>
7548<event name="OnTextEnter"></event>
7549<event name="OnTextMaxLen"></event>
7550<event name="OnTextURL"></event>
7551<event name="OnUpdateUI"></event>
7552</object>
7553</object>
7554<object class="sizeritem" expanded="1">
7555<property name="border">5</property>
7556<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
7557<property name="proportion">0</property>
7558<object class="wxTextCtrl" expanded="1">
7559<property name="bg"></property>
7560<property name="context_help"></property>
7561<property name="enabled">1</property>
7562<property name="fg"></property>
7563<property name="font"></property>
7564<property name="hidden">0</property>
7565<property name="id">wxID_ANY</property>
7566<property name="maximum_size">1,-1</property>
7567<property name="maxlength">0</property>
7568<property name="minimum_size">1,-1</property>
7569<property name="name">m_textCtrlField11</property>
7570<property name="permission">protected</property>
7571<property name="pos"></property>
7572<property name="size">-1,-1</property>
7573<property name="style"></property>
7574<property name="subclass"></property>
7575<property name="tooltip"></property>
7576<property name="value"></property>
7577<property name="window_extra_style"></property>
7578<property name="window_name"></property>
7579<property name="window_style"></property>
7580<event name="OnChar"></event>
7581<event name="OnEnterWindow"></event>
7582<event name="OnEraseBackground"></event>
7583<event name="OnKeyDown"></event>
7584<event name="OnKeyUp"></event>
7585<event name="OnKillFocus"></event>
7586<event name="OnLeaveWindow"></event>
7587<event name="OnLeftDClick"></event>
7588<event name="OnLeftDown"></event>
7589<event name="OnLeftUp"></event>
7590<event name="OnMiddleDClick"></event>
7591<event name="OnMiddleDown"></event>
7592<event name="OnMiddleUp"></event>
7593<event name="OnMotion"></event>
7594<event name="OnMouseEvents"></event>
7595<event name="OnMouseWheel"></event>
7596<event name="OnPaint"></event>
7597<event name="OnRightDClick"></event>
7598<event name="OnRightDown"></event>
7599<event name="OnRightUp"></event>
7600<event name="OnSetFocus"></event>
7601<event name="OnSize"></event>
7602<event name="OnText"></event>
7603<event name="OnTextEnter"></event>
7604<event name="OnTextMaxLen"></event>
7605<event name="OnTextURL"></event>
7606<event name="OnUpdateUI"></event>
7607</object>
7608</object>
7609<object class="sizeritem" expanded="1">
7610<property name="border">5</property>
7611<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
7612<property name="proportion">0</property>
7613<object class="wxButton" expanded="1">
7614<property name="bg"></property>
7615<property name="context_help"></property>
7616<property name="default">0</property>
7617<property name="enabled">1</property>
7618<property name="fg"></property>
7619<property name="font"></property>
7620<property name="hidden">0</property>
7621<property name="id">wxID_DEL11</property>
7622<property name="label"><Delete></property>
7623<property name="maximum_size"></property>
7624<property name="minimum_size">1,-1</property>
7625<property name="name">m_buttonDel11</property>
7626<property name="permission">protected</property>
7627<property name="pos"></property>
7628<property name="size">60,20</property>
7629<property name="style"></property>
7630<property name="subclass"></property>
7631<property name="tooltip"></property>
7632<property name="window_extra_style"></property>
7633<property name="window_name"></property>
7634<property name="window_style"></property>
7635<event name="OnButtonClick">OnButtonDel11</event>
7636<event name="OnChar"></event>
7637<event name="OnEnterWindow"></event>
7638<event name="OnEraseBackground"></event>
7639<event name="OnKeyDown"></event>
7640<event name="OnKeyUp"></event>
7641<event name="OnKillFocus"></event>
7642<event name="OnLeaveWindow"></event>
7643<event name="OnLeftDClick"></event>
7644<event name="OnLeftDown"></event>
7645<event name="OnLeftUp"></event>
7646<event name="OnMiddleDClick"></event>
7647<event name="OnMiddleDown"></event>
7648<event name="OnMiddleUp"></event>
7649<event name="OnMotion"></event>
7650<event name="OnMouseEvents"></event>
7651<event name="OnMouseWheel"></event>
7652<event name="OnPaint"></event>
7653<event name="OnRightDClick"></event>
7654<event name="OnRightDown"></event>
7655<event name="OnRightUp"></event>
7656<event name="OnSetFocus"></event>
7657<event name="OnSize"></event>
7658<event name="OnUpdateUI"></event>
7659</object>
7660</object>
7661<object class="sizeritem" expanded="1">
7662<property name="border">5</property>
7663<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7664<property name="proportion">0</property>
7665<object class="wxTextCtrl" expanded="1">
7666<property name="bg"></property>
7667<property name="context_help"></property>
7668<property name="enabled">1</property>
7669<property name="fg"></property>
7670<property name="font"></property>
7671<property name="hidden">0</property>
7672<property name="id">wxID_ANY</property>
7673<property name="maximum_size"></property>
7674<property name="maxlength">0</property>
7675<property name="minimum_size">150,-1</property>
7676<property name="name">m_textCtrlLabel12</property>
7677<property name="permission">protected</property>
7678<property name="pos"></property>
7679<property name="size"></property>
7680<property name="style"></property>
7681<property name="subclass"></property>
7682<property name="tooltip"></property>
7683<property name="value"></property>
7684<property name="window_extra_style"></property>
7685<property name="window_name"></property>
7686<property name="window_style"></property>
7687<event name="OnChar"></event>
7688<event name="OnEnterWindow"></event>
7689<event name="OnEraseBackground"></event>
7690<event name="OnKeyDown">OnKeyDown</event>
7691<event name="OnKeyUp"></event>
7692<event name="OnKillFocus"></event>
7693<event name="OnLeaveWindow"></event>
7694<event name="OnLeftDClick"></event>
7695<event name="OnLeftDown"></event>
7696<event name="OnLeftUp"></event>
7697<event name="OnMiddleDClick"></event>
7698<event name="OnMiddleDown"></event>
7699<event name="OnMiddleUp"></event>
7700<event name="OnMotion"></event>
7701<event name="OnMouseEvents"></event>
7702<event name="OnMouseWheel"></event>
7703<event name="OnPaint"></event>
7704<event name="OnRightDClick"></event>
7705<event name="OnRightDown"></event>
7706<event name="OnRightUp"></event>
7707<event name="OnSetFocus"></event>
7708<event name="OnSize"></event>
7709<event name="OnText"></event>
7710<event name="OnTextEnter"></event>
7711<event name="OnTextMaxLen"></event>
7712<event name="OnTextURL"></event>
7713<event name="OnUpdateUI"></event>
7714</object>
7715</object>
7716<object class="sizeritem" expanded="1">
7717<property name="border">5</property>
7718<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
7719<property name="proportion">0</property>
7720<object class="wxTextCtrl" expanded="1">
7721<property name="bg"></property>
7722<property name="context_help"></property>
7723<property name="enabled">0</property>
7724<property name="fg"></property>
7725<property name="font"></property>
7726<property name="hidden">0</property>
7727<property name="id">wxID_ANY</property>
7728<property name="maximum_size">-1,-1</property>
7729<property name="maxlength">0</property>
7730<property name="minimum_size">-1,-1</property>
7731<property name="name">m_textCtrlField12</property>
7732<property name="permission">protected</property>
7733<property name="pos"></property>
7734<property name="size">-1,-1</property>

```

```

7735<property name="style"></property>
7736<property name="subclass"></property>
7737<property name="tooltip"></property>
7738<property name="value"></property>
7739<property name="window_extra_style"></property>
7740<property name="window_name"></property>
7741<property name="window_style"></property>
7742<event name="OnChar"></event>
7743<event name="OnEnterWindow"></event>
7744<event name="OnEraseBackground"></event>
7745<event name="OnKeyDown"><OnKeyDown/></event>
7746<event name="OnKeyUp"></event>
7747<event name="OnKillFocus"></event>
7748<event name="OnLeaveWindow"></event>
7749<event name="OnLeftDClick"></event>
7750<event name="OnLeftDown"></event>
7751<event name="OnLeftUp"></event>
7752<event name="OnMiddleDClick"></event>
7753<event name="OnMiddleDown"></event>
7754<event name="OnMiddleUp"></event>
7755<event name="OnMotion"></event>
7756<event name="OnMouseEvents"></event>
7757<event name="OnMouseWheel"></event>
7758<event name="OnPaint"></event>
7759<event name="OnRightDClick"></event>
7760<event name="OnRightDown"></event>
7761<event name="OnRightUp"></event>
7762<event name="OnSetFocus"></event>
7763<event name="OnSize"></event>
7764<event name="OnText"></event>
7765<event name="OnTextEnter"></event>
7766<event name="OnTextMaxLen"></event>
7767<event name="OnTextURL"></event>
7768<event name="OnUpdateUI"></event>
7769</object>
7770</object>
7771<object class="sizeritem" expanded="1">
7772<property name="border">5</property>
7773<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
7774<property name="proportion">0</property>
7775<object class="wxButton" expanded="1">
7776<property name="bg"></property>
7777<property name="context_help"></property>
7778<property name="default">0</property>
7779<property name="enabled">1</property>
7780<property name="fg"></property>
7781<property name="font"></property>
7782<property name="hidden">0</property>
7783<property name="id">wxID_DEL12</property>
7784<property name="label">Delete</property>
7785<property name="maximum_size"></property>
7786<property name="minimum_size">1,1</property>
7787<property name="name">m_buttonDel12</property>
7788<property name="permission">protected</property>
7789<property name="pos"></property>
7790<property name="size">60,20</property>
7791<property name="style"></property>
7792<property name="subclass"></property>
7793<property name="tooltip"></property>
7794<property name="window_extra_style"></property>
7795<property name="window_name"></property>
7796<property name="window_style"></property>
7797<event name="OnButtonClick">OnButtonDel12</event>
7798<event name="OnChar"></event>
7799<event name="OnEnterWindow"></event>
7800<event name="OnEraseBackground"></event>
7801<event name="OnKeyDown"></event>
7802<event name="OnKeyUp"></event>
7803<event name="OnKillFocus"></event>
7804<event name="OnLeaveWindow"></event>
7805<event name="OnLeftDClick"></event>
7806<event name="OnLeftDown"></event>
7807<event name="OnLeftUp"></event>
7808<event name="OnMiddleDClick"></event>
7809<event name="OnMiddleDown"></event>
7810<event name="OnMiddleUp"></event>
7811<event name="OnMotion"></event>
7812<event name="OnMouseEvents"></event>
7813<event name="OnMouseWheel"></event>
7814<event name="OnPaint"></event>
7815<event name="OnRightDClick"></event>
7816<event name="OnRightDown"></event>
7817<event name="OnRightUp"></event>
7818<event name="OnSetFocus"></event>
7819<event name="OnSize"></event>
7820<event name="OnUpdateUI"></event>
7821</object>
7822</object>
7823<object class="sizeritem" expanded="1">
7824<property name="border">5</property>
7825<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7826<property name="proportion">0</property>
7827<object class="wxTextCtrl" expanded="1">
7828<property name="bg"></property>
7829<property name="context_help"></property>
7830<property name="enabled">1</property>
7831<property name="fg"></property>
7832<property name="font"></property>
7833<property name="hidden">0</property>
7834<property name="id">wxID_ANY</property>
7835<property name="maximum_size"></property>
7836<property name="maxlength">0</property>
7837<property name="minimum_size">150,-1</property>
7838<property name="name">m_textCtrlLabel13</property>
7839<property name="permission">protected</property>
7840<property name="pos"></property>
7841<property name="size"></property>
7842<property name="style"></property>
7843<property name="subclass"></property>
7844<property name="tooltip"></property>
7845<property name="value"></property>
7846<property name="window_extra_style"></property>
7847<property name="window_name"></property>
7848<property name="window_style"></property>
7849<event name="OnChar"></event>
7850<event name="OnEnterWindow"></event>
7851<event name="OnEraseBackground"></event>
7852<event name="OnKeyDown"><OnKeyDown/></event>
7853<event name="OnKeyUp"></event>
7854<event name="OnKillFocus"></event>
7855<event name="OnLeaveWindow"></event>
7856<event name="OnLeftDClick"></event>
7857<event name="OnLeftDown"></event>
7858<event name="OnLeftUp"></event>
7859<event name="OnMiddleDClick"></event>
7860<event name="OnMiddleDown"></event>
7861<event name="OnMiddleUp"></event>
7862<event name="OnMotion"></event>
7863<event name="OnMouseEvents"></event>
7864<event name="OnMouseWheel"></event>
7865<event name="OnPaint"></event>
7866<event name="OnRightDClick"></event>
7867<event name="OnRightDown"></event>
7868<event name="OnRightUp"></event>
7869<event name="OnSetFocus"></event>
7870<event name="OnSize"></event>
7871<event name="OnText"></event>
7872<event name="OnTextEnter"></event>
7873<event name="OnTextMaxLen"></event>
7874<event name="OnTextURL"></event>
7875<event name="OnUpdateUI"></event>
7876</object>
7877</object>
7878<object class="sizeritem" expanded="1">
7879<property name="border">5</property>
7880<property
    name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
7881<property name="proportion">0</property>
7882<object class="wxTextCtrl" expanded="1">
7883<property name="bg"></property>
7884<property name="context_help"></property>
7885<property name="enabled">1</property>
7886<property name="fg"></property>
7887<property name="font"></property>
7888<property name="hidden">0</property>
7889<property name="id">wxID_ANY</property>
7890<property name="maximum_size">1,1</property>
7891<property name="maxlength">0</property>
7892<property name="minimum_size">1,1</property>
7893<property name="name">m_textCtrlField13</property>
7894<property name="permission">protected</property>
7895<property name="pos"></property>
7896<property name="size">1,1</property>
7897<property name="style"></property>
7898<property name="subclass"></property>
7899<property name="tooltip"></property>
7900<property name="value"></property>
7901<property name="window_extra_style"></property>
7902<property name="window_name"></property>
7903<property name="window_style"></property>
7904<event name="OnChar"></event>
7905<event name="OnEnterWindow"></event>
7906<event name="OnEraseBackground"></event>
7907<event name="OnKeyDown"><OnKeyDown/></event>
7908<event name="OnKeyUp"></event>
7909<event name="OnKillFocus"></event>
7910<event name="OnLeaveWindow"></event>
7911<event name="OnLeftDClick"></event>
7912<event name="OnLeftDown"></event>
7913<event name="OnLeftUp"></event>
7914<event name="OnMiddleDClick"></event>
7915<event name="OnMiddleDown"></event>
7916<event name="OnMiddleUp"></event>
7917<event name="OnMotion"></event>
7918<event name="OnMouseEvents"></event>
7919<event name="OnMouseWheel"></event>
7920<event name="OnPaint"></event>
7921<event name="OnRightDClick"></event>
7922<event name="OnRightDown"></event>
7923<event name="OnRightUp"></event>
7924<event name="OnSetFocus"></event>
7925<event name="OnSize"></event>
7926<event name="OnText"></event>
7927<event name="OnTextEnter"></event>
7928<event name="OnTextMaxLen"></event>
7929<event name="OnTextURL"></event>
7930<event name="OnUpdateUI"></event>
7931</object>
7932</object>
7933<object class="sizeritem" expanded="1">
7934<property name="border">5</property>
7935<property
    name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
7936<property name="proportion">0</property>
7937<object class="wxButton" expanded="1">

```

```

7938<property name="bg"></property>
7939<property name="context_help"></property>
7940<property name="default">0</property>
7941<property name="enabled">1</property>
7942<property name="fg"></property>
7943<property name="font"></property>
7944<property name="hidden">0</property>
7945<property name="id">wxID_DEL13</property>
7946<property name="label">Delete</property>
7947<property name="maximum_size"></property>
7948<property name="minimum_size">-1,-1</property>
7949<property name="name">m_buttonDel13</property>
7950<property name="permission">protected</property>
7951<property name="pos"></property>
7952<property name="size">60,20</property>
7953<property name="style"></property>
7954<property name="subclass"></property>
7955<property name="tooltip"></property>
7956<property name="window_extra_style"></property>
7957<property name="window_name"></property>
7958<property name="window_style"></property>
7959<event name="OnButtonClick">OnButtonDel13</event>
7960<event name="OnChar"></event>
7961<event name="OnEnterWindow"></event>
7962<event name="OnEraseBackground"></event>
7963<event name="OnKeyDown"></event>
7964<event name="OnKeyUp"></event>
7965<event name="OnKillFocus"></event>
7966<event name="OnLeaveWindow"></event>
7967<event name="OnLeftClick"></event>
7968<event name="OnLeftDown"></event>
7969<event name="OnLeftUp"></event>
7970<event name="OnMiddleClick"></event>
7971<event name="OnMiddleDown"></event>
7972<event name="OnMiddleUp"></event>
7973<event name="OnMotion"></event>
7974<event name="OnMouseEvents"></event>
7975<event name="OnMouseWheel"></event>
7976<event name="OnPaint"></event>
7977<event name="OnRightClick"></event>
7978<event name="OnRightDown"></event>
7979<event name="OnRightUp"></event>
7980<event name="OnSetFocus"></event>
7981<event name="OnSize"></event>
7982<event name="OnUpdateUI"></event>
7983</object>
7984</object>
7985<object class="sizeritem" expanded="1">
7986<property name="border">5</property>
7987<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
7988<property name="proportion">0</property>
7989<object class="wxTextCtrl" expanded="1">
7990<property name="bg"></property>
7991<property name="context_help"></property>
7992<property name="enabled">1</property>
7993<property name="fg"></property>
7994<property name="font"></property>
7995<property name="hidden">0</property>
7996<property name="id">wxID_ANY</property>
7997<property name="maximum_size"></property>
7998<property name="max_length">0</property>
7999<property name="minimum_size">150,-1</property>
8000<property name="name">m_textCtrlLabel14</property>
8001<property name="permission">protected</property>
8002<property name="pos"></property>
8003<property name="size"></property>
8004<property name="style"></property>
8005<property name="subclass"></property>
8006<property name="tooltip"></property>
8007<property name="value"></property>
8008<property name="window_extra_style"></property>
8009<property name="window_name"></property>
8010<property name="window_style"></property>
8011<event name="OnChar"></event>
8012<event name="OnEnterWindow"></event>
8013<event name="OnEraseBackground"></event>
8014<event name="OnKeyDown">OnKeyDown</event>
8015<event name="OnKeyUp"></event>
8016<event name="OnKillFocus"></event>
8017<event name="OnLeaveWindow"></event>
8018<event name="OnLeftClick"></event>
8019<event name="OnLeftDown"></event>
8020<event name="OnLeftUp"></event>
8021<event name="OnMiddleClick"></event>
8022<event name="OnMiddleDown"></event>
8023<event name="OnMiddleUp"></event>
8024<event name="OnMotion"></event>
8025<event name="OnMouseEvents"></event>
8026<event name="OnMouseWheel"></event>
8027<event name="OnPaint"></event>
8028<event name="OnRightClick"></event>
8029<event name="OnRightDown"></event>
8030<event name="OnRightUp"></event>
8031<event name="OnSetFocus"></event>
8032<event name="OnSize"></event>
8033<event name="OnText"></event>
8034<event name="OnTextEnter"></event>
8035<event name="OnTextMaxLen"></event>
8036<event name="OnTextURL"></event>
8037<event name="OnUpdateUI"></event>
8038</object>
8039</object>
8040<object class="sizeritem" expanded="1">
8041<property name="border">5</property>
8042<property
      name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
8043<property name="proportion">0</property>
8044<object class="wxTextCtrl" expanded="1">
8045<property name="bg"></property>
8046<property name="context_help"></property>
8047<property name="enabled">1</property>
8048<property name="fg"></property>
8049<property name="font"></property>
8050<property name="hidden">0</property>
8051<property name="id">wxID_ANY</property>
8052<property name="maximum_size">-1,-1</property>
8053<property name="max_length">0</property>
8054<property name="minimum_size">-1,-1</property>
8055<property name="name">m_textCtrlField14</property>
8056<property name="permission">protected</property>
8057<property name="pos"></property>
8058<property name="size">-1,-1</property>
8059<property name="style"></property>
8060<property name="subclass"></property>
8061<property name="tooltip"></property>
8062<property name="value"></property>
8063<property name="window_extra_style"></property>
8064<property name="window_name"></property>
8065<property name="window_style"></property>
8066<event name="OnChar"></event>
8067<event name="OnEnterWindow"></event>
8068<event name="OnEraseBackground"></event>
8069<event name="OnKeyDown">OnKeyDown</event>
8070<event name="OnKeyUp"></event>
8071<event name="OnKillFocus"></event>
8072<event name="OnLeaveWindow"></event>
8073<event name="OnLeftClick"></event>
8074<event name="OnLeftDown"></event>
8075<event name="OnLeftUp"></event>
8076<event name="OnMiddleClick"></event>
8077<event name="OnMiddleDown"></event>
8078<event name="OnMiddleUp"></event>
8079<event name="OnMotion"></event>
8080<event name="OnMouseEvents"></event>
8081<event name="OnMouseWheel"></event>
8082<event name="OnPaint"></event>
8083<event name="OnRightClick"></event>
8084<event name="OnRightDown"></event>
8085<event name="OnRightUp"></event>
8086<event name="OnSetFocus"></event>
8087<event name="OnSize"></event>
8088<event name="OnText"></event>
8089<event name="OnTextEnter"></event>
8090<event name="OnTextMaxLen"></event>
8091<event name="OnTextURL"></event>
8092<event name="OnUpdateUI"></event>
8093</object>
8094</object>
8095<object class="sizeritem" expanded="1">
8096<property name="border">5</property>
8097<property
      name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
8098<property name="proportion">0</property>
8099<object class="wxButton" expanded="1">
8100<property name="bg"></property>
8101<property name="context_help"></property>
8102<property name="default">0</property>
8103<property name="enabled">1</property>
8104<property name="fg"></property>
8105<property name="font"></property>
8106<property name="hidden">0</property>
8107<property name="id">wxID_DEL14</property>
8108<property name="label">Delete</property>
8109<property name="maximum_size"></property>
8110<property name="minimum_size">-1,-1</property>
8111<property name="name">m_buttonDel14</property>
8112<property name="permission">protected</property>
8113<property name="pos"></property>
8114<property name="size">60,20</property>
8115<property name="style"></property>
8116<property name="subclass"></property>
8117<property name="tooltip"></property>
8118<property name="window_extra_style"></property>
8119<property name="window_name"></property>
8120<property name="window_style"></property>
8121<event name="OnButtonClick">OnButtonDel14</event>
8122<event name="OnChar"></event>
8123<event name="OnEnterWindow"></event>
8124<event name="OnEraseBackground"></event>
8125<event name="OnKeyDown"></event>
8126<event name="OnKeyUp"></event>
8127<event name="OnKillFocus"></event>
8128<event name="OnLeaveWindow"></event>
8129<event name="OnLeftClick"></event>
8130<event name="OnLeftDown"></event>
8131<event name="OnLeftUp"></event>
8132<event name="OnMiddleClick"></event>
8133<event name="OnMiddleDown"></event>
8134<event name="OnMiddleUp"></event>
8135<event name="OnMotion"></event>
8136<event name="OnMouseEvents"></event>
8137<event name="OnMouseWheel"></event>
8138<event name="OnPaint"></event>
8139<event name="OnRightClick"></event>
8140<event name="OnRightDown"></event>
8141<event name="OnRightUp"></event>

```

```

8142<event name="OnSetFocus"></event>
8143<event name="OnSize"></event>
8144<event name="OnUpdateUI"></event>
8145</object>
8146</object>
8147<object class="sizeritem" expanded="1">
8148<property name="border">5</property>
8149<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
8150<property name="proportion">0</property>
8151<object class="wxTextCtrl" expanded="1">
8152<property name="bg"></property>
8153<property name="context_help"></property>
8154<property name="enabled">1</property>
8155<property name="fg"></property>
8156<property name="font"></property>
8157<property name="hidden">0</property>
8158<property name="id">wxID_ANY</property>
8159<property name="maximum_size"></property>
8160<property name="maxlength">0</property>
8161<property name="minimum_size">150,-1</property>
8162<property name="name">m_textCtrlLabel15</property>
8163<property name="permission">protected</property>
8164<property name="pos"></property>
8165<property name="size"></property>
8166<property name="style"></property>
8167<property name="subclass"></property>
8168<property name="tooltip"></property>
8169<property name="value"></property>
8170<property name="window_extra_style"></property>
8171<property name="window_name"></property>
8172<property name="window_style"></property>
8173<event name="OnChar"></event>
8174<event name="OnEnterWindow"></event>
8175<event name="OnEraseBackground"></event>
8176<event name="OnKeyDown">OnKeyDown</event>
8177<event name="OnKeyUp"></event>
8178<event name="OnKillFocus"></event>
8179<event name="OnLeaveWindow"></event>
8180<event name="OnLeftDClick"></event>
8181<event name="OnLeftDown"></event>
8182<event name="OnLeftUp"></event>
8183<event name="OnMiddleDClick"></event>
8184<event name="OnMiddleDown"></event>
8185<event name="OnMiddleUp"></event>
8186<event name="OnMotion"></event>
8187<event name="OnMouseEvents"></event>
8188<event name="OnMouseWheel"></event>
8189<event name="OnPaint"></event>
8190<event name="OnRightDClick"></event>
8191<event name="OnRightDown"></event>
8192<event name="OnRightUp"></event>
8193<event name="OnSetFocus"></event>
8194<event name="OnSize"></event>
8195<event name="OnText"></event>
8196<event name="OnTextEnter"></event>
8197<event name="OnTextMaxLen"></event>
8198<event name="OnTextURL"></event>
8199<event name="OnUpdateUI"></event>
8200</object>
8201</object>
8202<object class="sizeritem" expanded="1">
8203<property name="border">5</property>
8204<property
name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
8205<property name="proportion">0</property>
8206<object class="wxTextCtrl" expanded="1">
8207<property name="bg"></property>
8208<property name="context_help"></property>
8209<property name="enabled">1</property>
8210<property name="fg"></property>
8211<property name="font"></property>
8212<property name="hidden">0</property>
8213<property name="id">wxID_ANY</property>
8214<property name="maximum_size">1,-1</property>
8215<property name="maxlength">0</property>
8216<property name="minimum_size">1,-1</property>
8217<property name="name">m_textCtrlField15</property>
8218<property name="permission">protected</property>
8219<property name="pos"></property>
8220<property name="size">1,-1</property>
8221<property name="style"></property>
8222<property name="subclass"></property>
8223<property name="tooltip"></property>
8224<property name="value"></property>
8225<property name="window_extra_style"></property>
8226<property name="window_name"></property>
8227<property name="window_style"></property>
8228<event name="OnChar"></event>
8229<event name="OnEnterWindow"></event>
8230<event name="OnEraseBackground"></event>
8231<event name="OnKeyDown">OnKeyDown</event>
8232<event name="OnKeyUp"></event>
8233<event name="OnKillFocus"></event>
8234<event name="OnLeaveWindow"></event>
8235<event name="OnLeftDClick"></event>
8236<event name="OnLeftDown"></event>
8237<event name="OnLeftUp"></event>
8238<event name="OnMiddleDClick"></event>
8239<event name="OnMiddleDown"></event>
8240<event name="OnMiddleUp"></event>
8241<event name="OnMotion"></event>
8242<event name="OnMouseEvents"></event>
8243<event name="OnMouseWheel"></event>
8244<event name="OnPaint"></event>
8245<event name="OnRightDClick"></event>
8246<event name="OnRightDown"></event>
8247<event name="OnRightUp"></event>
8248<event name="OnSetFocus"></event>
8249<event name="OnSize"></event>
8250<event name="OnText"></event>
8251<event name="OnTextEnter"></event>
8252<event name="OnTextMaxLen"></event>
8253<event name="OnTextURL"></event>
8254<event name="OnUpdateUI"></event>
8255</object>
8256</object>
8257<object class="sizeritem" expanded="1">
8258<property name="border">5</property>
8259<property
name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
8260<property name="proportion">0</property>
8261<object class="wxButton" expanded="1">
8262<property name="bg"></property>
8263<property name="context_help"></property>
8264<property name="default">0</property>
8265<property name="enabled">1</property>
8266<property name="fg"></property>
8267<property name="font"></property>
8268<property name="hidden">0</property>
8269<property name="id">wxID_DEL15</property>
8270<property name="label">Delete</property>
8271<property name="maximum_size"></property>
8272<property name="minimum_size">-1,-1</property>
8273<property name="name">m_buttonDel15</property>
8274<property name="permission">protected</property>
8275<property name="pos"></property>
8276<property name="size">60,20</property>
8277<property name="style"></property>
8278<property name="subclass"></property>
8279<property name="tooltip"></property>
8280<property name="window_extra_style"></property>
8281<property name="window_name"></property>
8282<property name="window_style"></property>
8283<event name="OnButtonDClick">OnButtonDel15</event>
8284<event name="OnChar"></event>
8285<event name="OnEnterWindow"></event>
8286<event name="OnEraseBackground"></event>
8287<event name="OnKeyDown"></event>
8288<event name="OnKeyUp"></event>
8289<event name="OnKillFocus"></event>
8290<event name="OnLeaveWindow"></event>
8291<event name="OnLeftDClick"></event>
8292<event name="OnLeftDown"></event>
8293<event name="OnLeftUp"></event>
8294<event name="OnMiddleDClick"></event>
8295<event name="OnMiddleDown"></event>
8296<event name="OnMiddleUp"></event>
8297<event name="OnMotion"></event>
8298<event name="OnMouseEvents"></event>
8299<event name="OnMouseWheel"></event>
8300<event name="OnPaint"></event>
8301<event name="OnRightDClick"></event>
8302<event name="OnRightDown"></event>
8303<event name="OnRightUp"></event>
8304<event name="OnSetFocus"></event>
8305<event name="OnSize"></event>
8306<event name="OnUpdateUI"></event>
8307</object>
8308</object>
8309<object class="sizeritem" expanded="1">
8310<property name="border">5</property>
8311<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
8312<property name="proportion">0</property>
8313<object class="wxTextCtrl" expanded="1">
8314<property name="bg"></property>
8315<property name="context_help"></property>
8316<property name="enabled">1</property>
8317<property name="fg"></property>
8318<property name="font"></property>
8319<property name="hidden">0</property>
8320<property name="id">wxID_ANY</property>
8321<property name="maximum_size"></property>
8322<property name="maxlength">0</property>
8323<property name="minimum_size">150,-1</property>
8324<property name="name">m_textCtrlLabel16</property>
8325<property name="permission">protected</property>
8326<property name="pos"></property>
8327<property name="size"></property>
8328<property name="style"></property>
8329<property name="subclass"></property>
8330<property name="tooltip"></property>
8331<property name="value"></property>
8332<property name="window_extra_style"></property>
8333<property name="window_name"></property>
8334<property name="window_style"></property>
8335<event name="OnChar"></event>
8336<event name="OnEnterWindow"></event>
8337<event name="OnEraseBackground"></event>
8338<event name="OnKeyDown">OnKeyDown</event>
8339<event name="OnKeyUp"></event>
8340<event name="OnKillFocus"></event>
8341<event name="OnLeaveWindow"></event>
8342<event name="OnLeftDClick"></event>
8343<event name="OnLeftDown"></event>
8344<event name="OnLeftUp"></event>
8345<event name="OnMiddleDClick"></event>

```



```

8346<event name="OnMiddleDown"></event>
8347<event name="OnMiddleUp"></event>
8348<event name="OnMotion"></event>
8349<event name="OnMouseEvents"></event>
8350<event name="OnMouseWheel"></event>
8351<event name="OnPaint"></event>
8352<event name="OnRightDClick"></event>
8353<event name="OnRightDown"></event>
8354<event name="OnRightUp"></event>
8355<event name="OnSetFocus"></event>
8356<event name="OnSize"></event>
8357<event name="OnText"></event>
8358<event name="OnTextEnter"></event>
8359<event name="OnTextMaxLen"></event>
8360<event name="OnTextURL"></event>
8361<event name="OnUpdateUI"></event>
8362</object>
8363</object>
8364<object class="sizeritem" expanded="1">
8365<property name="border">5</property>
8366<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
8367<property name="proportion">0</property>
8368<object class="wxTextCtrl" expanded="1">
8369<property name="bg"></property>
8370<property name="context_help"></property>
8371<property name="enabled">1</property>
8372<property name="fg"></property>
8373<property name="font"></property>
8374<property name="hidden">0</property>
8375<property name="id">wxID_ANY</property>
8376<property name="maximum_size">1,1</property>
8377<property name="maxlength">0</property>
8378<property name="minimum_size">1,1</property>
8379<property name="name">m_textCtrlField16</property>
8380<property name="permission">protected</property>
8381<property name="pos"></property>
8382<property name="size">1,1</property>
8383<property name="style"></property>
8384<property name="subclass"></property>
8385<property name="tooltip"></property>
8386<property name="value"></property>
8387<property name="window_extra_style"></property>
8388<property name="window_name"></property>
8389<property name="window_style"></property>
8390<event name="OnChar"></event>
8391<event name="OnEnterWindow"></event>
8392<event name="OnEraseBackground"></event>
8393<event name="OnKeyDown"></event>
8394<event name="OnKeyUp"></event>
8395<event name="OnKillFocus"></event>
8396<event name="OnLeaveWindow"></event>
8397<event name="OnLeftDClick"></event>
8398<event name="OnLeftDown"></event>
8399<event name="OnLeftUp"></event>
8400<event name="OnMiddleDClick"></event>
8401<event name="OnMiddleDown"></event>
8402<event name="OnMiddleUp"></event>
8403<event name="OnMotion"></event>
8404<event name="OnMouseEvents"></event>
8405<event name="OnMouseWheel"></event>
8406<event name="OnPaint"></event>
8407<event name="OnRightDClick"></event>
8408<event name="OnRightDown"></event>
8409<event name="OnRightUp"></event>
8410<event name="OnSetFocus"></event>
8411<event name="OnSize"></event>
8412<event name="OnText"></event>
8413<event name="OnTextEnter"></event>
8414<event name="OnTextMaxLen"></event>
8415<event name="OnTextURL"></event>
8416<event name="OnUpdateUI"></event>
8417</object>
8418</object>
8419<object class="sizeritem" expanded="1">
8420<property name="border">5</property>
8421<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
8422<property name="proportion">0</property>
8423<object class="wxButton" expanded="1">
8424<property name="bg"></property>
8425<property name="context_help"></property>
8426<property name="default">0</property>
8427<property name="enabled">1</property>
8428<property name="fg"></property>
8429<property name="font"></property>
8430<property name="hidden">0</property>
8431<property name="id">wxID_DEL16</property>
8432<property name="label">Delete</property>
8433<property name="maximum_size"></property>
8434<property name="minimum_size">1,1</property>
8435<property name="name">m_buttonDel16</property>
8436<property name="permission">protected</property>
8437<property name="pos"></property>
8438<property name="size">60,20</property>
8439<property name="style"></property>
8440<property name="subclass"></property>
8441<property name="tooltip"></property>
8442<property name="window_extra_style"></property>
8443<property name="window_name"></property>
8444<property name="window_style"></property>
8445<event name="OnButtonClick">OnButtonDel16</event>
8446<event name="OnChar"></event>
8447<event name="OnEnterWindow"></event>
8448<event name="OnEraseBackground"></event>
8449<event name="OnKeyDown"></event>
8450<event name="OnKeyUp"></event>
8451<event name="OnKillFocus"></event>
8452<event name="OnLeaveWindow"></event>
8453<event name="OnLeftDClick"></event>
8454<event name="OnLeftDown"></event>
8455<event name="OnLeftUp"></event>
8456<event name="OnMiddleDClick"></event>
8457<event name="OnMiddleDown"></event>
8458<event name="OnMiddleUp"></event>
8459<event name="OnMotion"></event>
8460<event name="OnMouseEvents"></event>
8461<event name="OnMouseWheel"></event>
8462<event name="OnPaint"></event>
8463<event name="OnRightDClick"></event>
8464<event name="OnRightDown"></event>
8465<event name="OnRightUp"></event>
8466<event name="OnSetFocus"></event>
8467<event name="OnSize"></event>
8468<event name="OnUpdateUI"></event>
8469</object>
8470</object>
8471<object class="sizeritem" expanded="1">
8472<property name="border">5</property>
8473<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
8474<property name="proportion">0</property>
8475<object class="wxTextCtrl" expanded="1">
8476<property name="bg"></property>
8477<property name="context_help"></property>
8478<property name="enabled">1</property>
8479<property name="fg"></property>
8480<property name="font"></property>
8481<property name="hidden">0</property>
8482<property name="id">wxID_ANY</property>
8483<property name="maximum_size"></property>
8484<property name="maxlength">0</property>
8485<property name="minimum_size">150,1</property>
8486<property name="name">m_textCtrlLabel17</property>
8487<property name="permission">protected</property>
8488<property name="pos"></property>
8489<property name="size"></property>
8490<property name="style"></property>
8491<property name="subclass"></property>
8492<property name="tooltip"></property>
8493<property name="value"></property>
8494<property name="window_extra_style"></property>
8495<property name="window_name"></property>
8496<property name="window_style"></property>
8497<event name="OnChar"></event>
8498<event name="OnEnterWindow"></event>
8499<event name="OnEraseBackground"></event>
8500<event name="OnKeyDown">OnKeyDown</event>
8501<event name="OnKeyUp"></event>
8502<event name="OnKillFocus"></event>
8503<event name="OnLeaveWindow"></event>
8504<event name="OnLeftDClick"></event>
8505<event name="OnLeftDown"></event>
8506<event name="OnLeftUp"></event>
8507<event name="OnMiddleDClick"></event>
8508<event name="OnMiddleDown"></event>
8509<event name="OnMiddleUp"></event>
8510<event name="OnMotion"></event>
8511<event name="OnMouseEvents"></event>
8512<event name="OnMouseWheel"></event>
8513<event name="OnPaint"></event>
8514<event name="OnRightDClick"></event>
8515<event name="OnRightDown"></event>
8516<event name="OnRightUp"></event>
8517<event name="OnSetFocus"></event>
8518<event name="OnSize"></event>
8519<event name="OnText"></event>
8520<event name="OnTextEnter"></event>
8521<event name="OnTextMaxLen"></event>
8522<event name="OnTextURL"></event>
8523<event name="OnUpdateUI"></event>
8524</object>
8525</object>
8526<object class="sizeritem" expanded="1">
8527<property name="border">5</property>
8528<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
8529<property name="proportion">0</property>
8530<object class="wxTextCtrl" expanded="1">
8531<property name="bg"></property>
8532<property name="context_help"></property>
8533<property name="enabled">1</property>
8534<property name="fg"></property>
8535<property name="font"></property>
8536<property name="hidden">0</property>
8537<property name="id">wxID_ANY</property>
8538<property name="maximum_size">1,1</property>
8539<property name="maxlength">0</property>
8540<property name="minimum_size">1,1</property>
8541<property name="name">m_textCtrlField17</property>
8542<property name="permission">protected</property>
8543<property name="pos"></property>
8544<property name="size">1,1</property>
8545<property name="style"></property>
8546<property name="subclass"></property>
8547<property name="tooltip"></property>
8548<property name="value"></property>

```

```

8549<property name="window_extra_style"></property>
8550<property name="window_name"></property>
8551<property name="window_style"></property>
8552<event name="OnChar"></event>
8553<event name="OnEnterWindow"></event>
8554<event name="OnEraseBackground"></event>
8555<event name="OnKeyDown"><OnKeyDown/></event>
8556<event name="OnKeyUp"></event>
8557<event name="OnKillFocus"></event>
8558<event name="OnLeaveWindow"></event>
8559<event name="OnLeftDClick"></event>
8560<event name="OnLeftDown"></event>
8561<event name="OnLeftUp"></event>
8562<event name="OnMiddleClick"></event>
8563<event name="OnMiddleDown"></event>
8564<event name="OnMiddleUp"></event>
8565<event name="OnMotion"></event>
8566<event name="OnMouseEvents"></event>
8567<event name="OnMouseWheel"></event>
8568<event name="OnPaint"></event>
8569<event name="OnRightDClick"></event>
8570<event name="OnRightDown"></event>
8571<event name="OnRightUp"></event>
8572<event name="OnSetFocus"></event>
8573<event name="OnSize"></event>
8574<event name="OnText"></event>
8575<event name="OnTextEnter"></event>
8576<event name="OnTextMaxLen"></event>
8577<event name="OnTextURL"></event>
8578<event name="OnUpdateUI"></event>
8579</object>
8580</object>
8581<object class="sizeritem" expanded="1">
8582<property name="border">5</property>
8583<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
8584<property name="proportion">0</property>
8585<object class="wxButton" expanded="1">
8586<property name="bg"></property>
8587<property name="context_help"></property>
8588<property name="default">0</property>
8589<property name="enabled">1</property>
8590<property name="fg"></property>
8591<property name="font"></property>
8592<property name="hidden">0</property>
8593<property name="id">wxID_DEL17</property>
8594<property name="label">Delete</property>
8595<property name="maximum_size"></property>
8596<property name="minimum_size">1, -1</property>
8597<property name="name">m_buttonDel17</property>
8598<property name="permission">protected</property>
8599<property name="pos"></property>
8600<property name="size">60, 20</property>
8601<property name="style"></property>
8602<property name="subclass"></property>
8603<property name="tooltip"></property>
8604<property name="window_extra_style"></property>
8605<property name="window_name"></property>
8606<property name="window_style"></property>
8607<event name="OnButtonClick">OnButtonDel17</event>
8608<event name="OnChar"></event>
8609<event name="OnEnterWindow"></event>
8610<event name="OnEraseBackground"></event>
8611<event name="OnKeyDown"></event>
8612<event name="OnKeyUp"></event>
8613<event name="OnKillFocus"></event>
8614<event name="OnLeaveWindow"></event>
8615<event name="OnLeftDClick"></event>
8616<event name="OnLeftDown"></event>
8617<event name="OnLeftUp"></event>
8618<event name="OnMiddleClick"></event>
8619<event name="OnMiddleDown"></event>
8620<event name="OnMiddleUp"></event>
8621<event name="OnMotion"></event>
8622<event name="OnMouseEvents"></event>
8623<event name="OnMouseWheel"></event>
8624<event name="OnPaint"></event>
8625<event name="OnRightDClick"></event>
8626<event name="OnRightDown"></event>
8627<event name="OnRightUp"></event>
8628<event name="OnSetFocus"></event>
8629<event name="OnSize"></event>
8630<event name="OnUpdateUI"></event>
8631</object>
8632</object>
8633<object class="sizeritem" expanded="1">
8634<property name="border">5</property>
8635<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
8636<property name="proportion">0</property>
8637<object class="wxTextCtrl" expanded="1">
8638<property name="bg"></property>
8639<property name="context_help"></property>
8640<property name="enabled">1</property>
8641<property name="fg"></property>
8642<property name="font"></property>
8643<property name="hidden">0</property>
8644<property name="id">wxID_ANY</property>
8645<property name="maximum_size"></property>
8646<property name="maxlength">0</property>
8647<property name="minimum_size">150, -1</property>
8648<property name="name">m_textCtrlLabel18</property>
8649<property name="permission">protected</property>
8650<property name="pos"></property>
8651<property name="size"></property>
8652<property name="style"></property>
8653<property name="subclass"></property>
8654<property name="tooltip"></property>
8655<property name="value"></property>
8656<property name="window_extra_style"></property>
8657<property name="window_name"></property>
8658<property name="window_style"></property>
8659<event name="OnChar"></event>
8660<event name="OnEnterWindow"></event>
8661<event name="OnEraseBackground"></event>
8662<event name="OnKeyDown"><OnKeyDown/></event>
8663<event name="OnKeyUp"></event>
8664<event name="OnKillFocus"></event>
8665<event name="OnLeaveWindow"></event>
8666<event name="OnLeftDClick"></event>
8667<event name="OnLeftDown"></event>
8668<event name="OnLeftUp"></event>
8669<event name="OnMiddleDClick"></event>
8670<event name="OnMiddleDown"></event>
8671<event name="OnMiddleUp"></event>
8672<event name="OnMotion"></event>
8673<event name="OnMouseEvents"></event>
8674<event name="OnMouseWheel"></event>
8675<event name="OnPaint"></event>
8676<event name="OnRightDClick"></event>
8677<event name="OnRightDown"></event>
8678<event name="OnRightUp"></event>
8679<event name="OnSetFocus"></event>
8680<event name="OnSize"></event>
8681<event name="OnText"></event>
8682<event name="OnTextEnter"></event>
8683<event name="OnTextMaxLen"></event>
8684<event name="OnTextURL"></event>
8685<event name="OnUpdateUI"></event>
8686</object>
8687</object>
8688<object class="sizeritem" expanded="1">
8689<property name="border">5</property>
8690<property
  name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
8691<property name="proportion">0</property>
8692<object class="wxTextCtrl" expanded="1">
8693<property name="bg"></property>
8694<property name="context_help"></property>
8695<property name="enabled">1</property>
8696<property name="fg"></property>
8697<property name="font"></property>
8698<property name="hidden">0</property>
8699<property name="id">wxID_ANY</property>
8700<property name="maximum_size">1, -1</property>
8701<property name="maxlength">0</property>
8702<property name="minimum_size">1, -1</property>
8703<property name="name">m_textCtrlField18</property>
8704<property name="permission">protected</property>
8705<property name="pos"></property>
8706<property name="size">1, -1</property>
8707<property name="style"></property>
8708<property name="subclass"></property>
8709<property name="tooltip"></property>
8710<property name="value"></property>
8711<property name="window_extra_style"></property>
8712<property name="window_name"></property>
8713<property name="window_style"></property>
8714<event name="OnChar"></event>
8715<event name="OnEnterWindow"></event>
8716<event name="OnEraseBackground"></event>
8717<event name="OnKeyDown"><OnKeyDown/></event>
8718<event name="OnKeyUp"></event>
8719<event name="OnKillFocus"></event>
8720<event name="OnLeaveWindow"></event>
8721<event name="OnLeftDClick"></event>
8722<event name="OnLeftDown"></event>
8723<event name="OnLeftUp"></event>
8724<event name="OnMiddleDClick"></event>
8725<event name="OnMiddleDown"></event>
8726<event name="OnMiddleUp"></event>
8727<event name="OnMotion"></event>
8728<event name="OnMouseEvents"></event>
8729<event name="OnMouseWheel"></event>
8730<event name="OnPaint"></event>
8731<event name="OnRightDClick"></event>
8732<event name="OnRightDown"></event>
8733<event name="OnRightUp"></event>
8734<event name="OnSetFocus"></event>
8735<event name="OnSize"></event>
8736<event name="OnText"></event>
8737<event name="OnTextEnter"></event>
8738<event name="OnTextMaxLen"></event>
8739<event name="OnTextURL"></event>
8740<event name="OnUpdateUI"></event>
8741</object>
8742</object>
8743<object class="sizeritem" expanded="1">
8744<property name="border">5</property>
8745<property
  name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
8746<property name="proportion">0</property>
8747<object class="wxButton" expanded="1">
8748<property name="bg"></property>
8749<property name="context_help"></property>
8750<property name="default">0</property>
8751<property name="enabled">1</property>

```



```

8752<property name="fg"></property>
8753<property name="font"></property>
8754<property name="hidden">0</property>
8755<property name="id">wxID_DEL18</property>
8756<property name="label">Delete</property>
8757<property name="maximum_size"></property>
8758<property name="minimum_size">-1,-1</property>
8759<property name="name">m_buttonDel18</property>
8760<property name="permission">protected</property>
8761<property name="pos"></property>
8762<property name="size">60,20</property>
8763<property name="style"></property>
8764<property name="subclass"></property>
8765<property name="tooltip"></property>
8766<property name="window_extra_style"></property>
8767<property name="window_name"></property>
8768<property name="window_style"></property>
8769<event name="OnButtonClick">OnButtonDel18</event>
8770<event name="OnChar"></event>
8771<event name="OnEnterWindow"></event>
8772<event name="OnEraseBackground"></event>
8773<event name="OnKeyDown"></event>
8774<event name="OnKeyUp"></event>
8775<event name="OnKillFocus"></event>
8776<event name="OnLeaveWindow"></event>
8777<event name="OnLeftClick"></event>
8778<event name="OnLeftDown"></event>
8779<event name="OnLeftUp"></event>
8780<event name="OnMiddleClick"></event>
8781<event name="OnMiddleDown"></event>
8782<event name="OnMiddleUp"></event>
8783<event name="OnMotion"></event>
8784<event name="OnMouseEvents"></event>
8785<event name="OnMouseWheel"></event>
8786<event name="OnPaint"></event>
8787<event name="OnRightClick"></event>
8788<event name="OnRightDown"></event>
8789<event name="OnRightUp"></event>
8790<event name="OnSetFocus"></event>
8791<event name="OnSize"></event>
8792<event name="OnUpdateUI"></event>
8793</object>
8794</object>
8795<object class="sizeritem" expanded="1">
8796<property name="border">5</property>
8797<property name="flag">wxALL|wxALIGN_CENTER_VERTICAL</property>
8798<property name="proportion">0</property>
8799<object class="wxTextCtrl" expanded="1">
8800<property name="bg"></property>
8801<property name="context_help"></property>
8802<property name="enabled">1</property>
8803<property name="fg"></property>
8804<property name="font"></property>
8805<property name="hidden">0</property>
8806<property name="id">wxID_ANY</property>
8807<property name="maximum_size"></property>
8808<property name="maxlength">0</property>
8809<property name="minimum_size">150,-1</property>
8810<property name="name">m_textCtrlLabel19</property>
8811<property name="permission">protected</property>
8812<property name="pos"></property>
8813<property name="size"></property>
8814<property name="style"></property>
8815<property name="subclass"></property>
8816<property name="tooltip"></property>
8817<property name="value"></property>
8818<property name="window_extra_style"></property>
8819<property name="window_name"></property>
8820<property name="window_style"></property>
8821<event name="OnChar"></event>
8822<event name="OnEnterWindow"></event>
8823<event name="OnEraseBackground"></event>
8824<event name="OnKeyDown">OnKeyDown</event>
8825<event name="OnKeyUp"></event>
8826<event name="OnKillFocus"></event>
8827<event name="OnLeaveWindow"></event>
8828<event name="OnLeftClick"></event>
8829<event name="OnLeftDown"></event>
8830<event name="OnLeftUp"></event>
8831<event name="OnMiddleClick"></event>
8832<event name="OnMiddleDown"></event>
8833<event name="OnMiddleUp"></event>
8834<event name="OnMotion"></event>
8835<event name="OnMouseEvents"></event>
8836<event name="OnMouseWheel"></event>
8837<event name="OnPaint"></event>
8838<event name="OnRightClick"></event>
8839<event name="OnRightDown"></event>
8840<event name="OnRightUp"></event>
8841<event name="OnSetFocus"></event>
8842<event name="OnSize"></event>
8843<event name="OnText"></event>
8844<event name="OnTextEnter"></event>
8845<event name="OnTextMaxLen"></event>
8846<event name="OnTextURL"></event>
8847<event name="OnUpdateUI"></event>
8848</object>
8849</object>
8850<object class="sizeritem" expanded="1">
8851<property name="border">5</property>
8852<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_VERTICAL</property>
8853<property name="proportion">0</property>
8854<object class="wxTextCtrl" expanded="1">
8855<property name="bg"></property>
8856<property name="context_help"></property>
8857<property name="enabled">1</property>
8858<property name="fg"></property>
8859<property name="font"></property>
8860<property name="hidden">0</property>
8861<property name="id">wxID_ANY</property>
8862<property name="maximum_size">-1,-1</property>
8863<property name="maxlength">0</property>
8864<property name="minimum_size">-1,-1</property>
8865<property name="name">m_textCtrlField19</property>
8866<property name="permission">protected</property>
8867<property name="pos"></property>
8868<property name="size">-1,-1</property>
8869<property name="style"></property>
8870<property name="subclass"></property>
8871<property name="tooltip"></property>
8872<property name="value"></property>
8873<property name="window_extra_style"></property>
8874<property name="window_name"></property>
8875<property name="window_style"></property>
8876<event name="OnChar"></event>
8877<event name="OnEnterWindow"></event>
8878<event name="OnEraseBackground"></event>
8879<event name="OnKeyDown">OnKeyDown</event>
8880<event name="OnKeyUp"></event>
8881<event name="OnKillFocus"></event>
8882<event name="OnLeaveWindow"></event>
8883<event name="OnLeftClick"></event>
8884<event name="OnLeftDown"></event>
8885<event name="OnLeftUp"></event>
8886<event name="OnMiddleClick"></event>
8887<event name="OnMiddleDown"></event>
8888<event name="OnMiddleUp"></event>
8889<event name="OnMotion"></event>
8890<event name="OnMouseEvents"></event>
8891<event name="OnMouseWheel"></event>
8892<event name="OnPaint"></event>
8893<event name="OnRightClick"></event>
8894<event name="OnRightDown"></event>
8895<event name="OnRightUp"></event>
8896<event name="OnSetFocus"></event>
8897<event name="OnSize"></event>
8898<event name="OnText"></event>
8899<event name="OnTextEnter"></event>
8900<event name="OnTextMaxLen"></event>
8901<event name="OnTextURL"></event>
8902<event name="OnUpdateUI"></event>
8903</object>
8904</object>
8905<object class="sizeritem" expanded="1">
8906<property name="border">5</property>
8907<property name="flag">wxALIGN_CENTER_VERTICAL|wxRIGHT|wxLEFT</property>
8908<property name="proportion">0</property>
8909<object class="wxButton" expanded="1">
8910<property name="bg"></property>
8911<property name="context_help"></property>
8912<property name="default">0</property>
8913<property name="enabled">1</property>
8914<property name="fg"></property>
8915<property name="font"></property>
8916<property name="hidden">0</property>
8917<property name="id">wxID_DEL19</property>
8918<property name="label">Delete</property>
8919<property name="maximum_size"></property>
8920<property name="minimum_size">-1,-1</property>
8921<property name="name">m_buttonDel19</property>
8922<property name="permission">protected</property>
8923<property name="pos"></property>
8924<property name="size">60,20</property>
8925<property name="style"></property>
8926<property name="subclass"></property>
8927<property name="tooltip"></property>
8928<property name="window_extra_style"></property>
8929<property name="window_name"></property>
8930<property name="window_style"></property>
8931<event name="OnButtonClick">OnButtonDel19</event>
8932<event name="OnChar"></event>
8933<event name="OnEnterWindow"></event>
8934<event name="OnEraseBackground"></event>
8935<event name="OnKeyDown"></event>
8936<event name="OnKeyUp"></event>
8937<event name="OnKillFocus"></event>
8938<event name="OnLeaveWindow"></event>
8939<event name="OnLeftClick"></event>
8940<event name="OnLeftDown"></event>
8941<event name="OnLeftUp"></event>
8942<event name="OnMiddleClick"></event>
8943<event name="OnMiddleDown"></event>
8944<event name="OnMiddleUp"></event>
8945<event name="OnMotion"></event>
8946<event name="OnMouseEvents"></event>
8947<event name="OnMouseWheel"></event>
8948<event name="OnPaint"></event>
8949<event name="OnRightClick"></event>
8950<event name="OnRightDown"></event>
8951<event name="OnRightUp"></event>
8952<event name="OnSetFocus"></event>
8953<event name="OnSize"></event>
8954<event name="OnUpdateUI"></event>
8955</object>

```

```

8956</object>
8957</object>
8958</object>
8959<object class="sizeritem" expanded="1">
8960<property name="border">5</property>
8961<property name="flag">wxALIGN_CENTER_HORIZONTAL</property>
8962<property name="proportion">0</property>
8963<object class="wxBoxSizer" expanded="1">
8964<property name="minimum_size"></property>
8965<property name="name">bSizer25</property>
8966<property name="orient">wxHORIZONTAL</property>
8967<property name="permission">none</property>
8968<object class="sizeritem" expanded="1">
8969<property name="border">5</property>
8970<property name="flag">wxALL</property>
8971<property name="proportion">0</property>
8972<object class="wxButton" expanded="1">
8973<property name="bg"></property>
8974<property name="context_help"></property>
8975<property name="default">0</property>
8976<property name="enabled">1</property>
8977<property name="fg"></property>
8978<property name="font"></property>
8979<property name="hidden">0</property>
8980<property name="id">wxID_ANY</property>
8981<property name="label">&Add Field</property>
8982<property name="maximum_size"></property>
8983<property name="minimum_size"></property>
8984<property name="name">m_buttonAddField</property>
8985<property name="permission">protected</property>
8986<property name="pos"></property>
8987<property name="size"></property>
8988<property name="style"></property>
8989<property name="subclass"></property>
8990<property name="tooltip"></property>
8991<property name="window_extra_style"></property>
8992<property name="window_name"></property>
8993<property name="window_style"></property>
8994<event name="OnButtonClick">OnButtonAddField</event>
8995<event name="OnChar"></event>
8996<event name="OnEnterWindow"></event>
8997<event name="OnEraseBackground"></event>
8998<event name="OnKeyDown"></event>
8999<event name="OnKeyUp"></event>
9000<event name="OnKillFocus"></event>
9001<event name="OnLeaveWindow"></event>
9002<event name="OnLeftDClick"></event>
9003<event name="OnLeftDown"></event>
9004<event name="OnLeftUp"></event>
9005<event name="OnMiddleDClick"></event>
9006<event name="OnMiddleDown"></event>
9007<event name="OnMiddleUp"></event>
9008<event name="OnMotion"></event>
9009<event name="OnMouseEvents"></event>
9010<event name="OnMouseWheel"></event>
9011<event name="OnPaint"></event>
9012<event name="OnRightDClick"></event>
9013<event name="OnRightDown"></event>
9014<event name="OnRightUp"></event>
9015<event name="OnSetFocus"></event>
9016<event name="OnSize"></event>
9017<event name="OnUpdateUI"></event>
9018</object>
9019</object>
9020</object>
9021</object>
9022</object>
9023</object>
9024</object>
9025<object class="sizeritem" expanded="1">
9026<property name="border">5</property>
9027<property name="flag">wxALIGN_RIGHT</property>
9028<property name="proportion">0</property>
9029<object class="wxBoxSizer" expanded="1">
9030<property name="minimum_size"></property>
9031<property name="name">bSizer26</property>
9032<property name="orient">wxHORIZONTAL</property>
9033<property name="permission">none</property>
9034<object class="sizeritem" expanded="1">
9035<property name="border">5</property>
9036<property name="flag">wxALL</property>
9037<property name="proportion">0</property>
9038<object class="wxButton" expanded="1">
9039<property name="bg"></property>
9040<property name="context_help"></property>
9041<property name="default">0</property>
9042<property name="enabled">1</property>
9043<property name="fg"></property>
9044<property name="font"></property>
9045<property name="hidden">0</property>
9046<property name="id">wxID_BUTTONSEND</property>
9047<property name="label">&Send</property>
9048<property name="maximum_size"></property>
9049<property name="minimum_size">85,25</property>
9050<property name="name">m_buttonOK</property>
9051<property name="permission">protected</property>
9052<property name="pos"></property>
9053<property name="size"></property>
9054<property name="style"></property>
9055<property name="subclass"></property>
9056<property name="tooltip"></property>
9057<property name="window_extra_style"></property>
9058<property name="window_name"></property>
9059<property name="window_style"></property>
9060<event name="OnButtonClick">OnButtonSend</event>
9061<event name="OnChar"></event>
9062<event name="OnEnterWindow"></event>
9063<event name="OnEraseBackground"></event>
9064<event name="OnKeyDown"></event>
9065<event name="OnKeyUp"></event>
9066<event name="OnKillFocus"></event>
9067<event name="OnLeaveWindow"></event>
9068<event name="OnLeftDClick"></event>
9069<event name="OnLeftDown"></event>
9070<event name="OnLeftUp"></event>
9071<event name="OnMiddleDClick"></event>
9072<event name="OnMiddleDown"></event>
9073<event name="OnMiddleUp"></event>
9074<event name="OnMotion"></event>
9075<event name="OnMouseEvents"></event>
9076<event name="OnMouseWheel"></event>
9077<event name="OnPaint"></event>
9078<event name="OnRightDClick"></event>
9079<event name="OnRightDown"></event>
9080<event name="OnRightUp"></event>
9081<event name="OnSetFocus"></event>
9082<event name="OnSize"></event>
9083<event name="OnUpdateUI"></event>
9084</object>
9085</object>
9086<object class="sizeritem" expanded="1">
9087<property name="border">5</property>
9088<property name="flag">wxALL</property>
9089<property name="proportion">0</property>
9090<object class="wxButton" expanded="1">
9091<property name="bg"></property>
9092<property name="context_help"></property>
9093<property name="default">0</property>
9094<property name="enabled">1</property>
9095<property name="fg"></property>
9096<property name="font"></property>
9097<property name="hidden">0</property>
9098<property name="id">wxID_BUTTONPREVIEW</property>
9099<property name="label">&Preview</property>
9100<property name="maximum_size"></property>
9101<property name="minimum_size">85,25</property>
9102<property name="name">m_buttonPreview</property>
9103<property name="permission">protected</property>
9104<property name="pos"></property>
9105<property name="size"></property>
9106<property name="style"></property>
9107<property name="subclass"></property>
9108<property name="tooltip"></property>
9109<property name="window_extra_style"></property>
9110<property name="window_name"></property>
9111<property name="window_style"></property>
9112<event name="OnButtonClick">OnButtonPreview</event>
9113<event name="OnChar"></event>
9114<event name="OnEnterWindow"></event>
9115<event name="OnEraseBackground"></event>
9116<event name="OnKeyDown"></event>
9117<event name="OnKeyUp"></event>
9118<event name="OnKillFocus"></event>
9119<event name="OnLeaveWindow"></event>
9120<event name="OnLeftDClick"></event>
9121<event name="OnLeftDown"></event>
9122<event name="OnLeftUp"></event>
9123<event name="OnMiddleDClick"></event>
9124<event name="OnMiddleDown"></event>
9125<event name="OnMiddleUp"></event>
9126<event name="OnMotion"></event>
9127<event name="OnMouseEvents"></event>
9128<event name="OnMouseWheel"></event>
9129<event name="OnPaint"></event>
9130<event name="OnRightDClick"></event>
9131<event name="OnRightDown"></event>
9132<event name="OnRightUp"></event>
9133<event name="OnSetFocus"></event>
9134<event name="OnSize"></event>
9135<event name="OnUpdateUI"></event>
9136</object>
9137</object>
9138<object class="sizeritem" expanded="1">
9139<property name="border">5</property>
9140<property name="flag">wxALL</property>
9141<property name="proportion">0</property>
9142<object class="wxButton" expanded="1">
9143<property name="bg"></property>
9144<property name="context_help"></property>
9145<property name="default">0</property>
9146<property name="enabled">1</property>
9147<property name="fg"></property>
9148<property name="font"></property>
9149<property name="hidden">0</property>
9150<property name="id">wxID_CANCEL</property>
9151<property name="label">&Cancel</property>
9152<property name="maximum_size"></property>
9153<property name="minimum_size">85,25</property>
9154<property name="name">m_buttonCancel</property>
9155<property name="permission">protected</property>
9156<property name="pos"></property>
9157<property name="size"></property>
9158<property name="style"></property>
9159<property name="subclass"></property>
9160<property name="tooltip"></property>
9161<property name="window_extra_style"></property>

```

```

9162<property name="window_name"></property>
9163<property name="window_style"></property>
9164<event name="OnButtonClick">OnButtonCancel</event>
9165<event name="OnChar"></event>
9166<event name="OnEnterWindow"></event>
9167<event name="OnEraseBackground"></event>
9168<event name="OnKeyDown"></event>
9169<event name="OnKeyUp"></event>
9170<event name="OnKillFocus"></event>
9171<event name="OnLeaveWindow"></event>
9172<event name="OnLeftDClick"></event>
9173<event name="OnLeftDown"></event>
9174<event name="OnLeftUp"></event>
9175<event name="OnMiddleDClick"></event>
9176<event name="OnMiddleDown"></event>
9177<event name="OnMiddleUp"></event>
9178<event name="OnMotion"></event>
9179<event name="OnMouseEvents"></event>
9180<event name="OnMouseWheel"></event>
9181<event name="OnPaint"></event>
9182<event name="OnRightDClick"></event>
9183<event name="OnRightDown"></event>
9184<event name="OnRightUp"></event>
9185<event name="OnSetFocus"></event>
9186<event name="OnSize"></event>
9187<event name="OnUpdateUI"></event>
9188</object>
9189</object>
9190</object>
9191</object>
9192</object>
9193</object>
9194<object class="Frame" expanded="0">
9195<property name="bg">wxSYS_COLOUR_MENU</property>
9196<property name="center"></property>
9197<property name="context_help"></property>
9198<property name="enabled">1</property>
9199<property name="extra_style"></property>
9200<property name="fg"></property>
9201<property name="font"></property>
9202<property name="hidden">0</property>
9203<property name="id">wxID_ANY</property>
9204<property name="maximum_size"></property>
9205<property name="minimum_size"></property>
9206<property name="name">CViewProductDialogBase</property>
9207<property name="pos"></property>
9208<property name="size">630,520</property>
9209<property
  name="style">wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
9210<property name="subclass"></property>
9211<property name="title">Order Form</property>
9212<property name="tooltip"></property>
9213<property name="window_extra_style"></property>
9214<property name="window_name"></property>
9215<property name="window_style">wxTAB_TRAVERSAL</property>
9216<property name="xrc_skip_size">1</property>
9217<event name="OnActivate"></event>
9218<event name="OnActivateApp"></event>
9219<event name="OnChar"></event>
9220<event name="OnClose"></event>
9221<event name="OnEnterWindow"></event>
9222<event name="OnEraseBackground"></event>
9223<event name="OnHibernate"></event>
9224<event name="OnIconize"></event>
9225<event name="OnIdle"></event>
9226<event name="OnKeyDown"></event>
9227<event name="OnKeyUp"></event>
9228<event name="OnKillFocus"></event>
9229<event name="OnLeaveWindow"></event>
9230<event name="OnLeftDClick"></event>
9231<event name="OnLeftDown"></event>
9232<event name="OnLeftUp"></event>
9233<event name="OnMiddleDClick"></event>
9234<event name="OnMiddleDown"></event>
9235<event name="OnMiddleUp"></event>
9236<event name="OnMotion"></event>
9237<event name="OnMouseEvents"></event>
9238<event name="OnMouseWheel"></event>
9239<event name="OnPaint"></event>
9240<event name="OnRightDClick"></event>
9241<event name="OnRightDown"></event>
9242<event name="OnRightUp"></event>
9243<event name="OnSetFocus"></event>
9244<event name="OnSize"></event>
9245<event name="OnUpdateUI"></event>
9246<object class="wxBoxSizer" expanded="1">
9247<property name="minimum_size"></property>
9248<property name="name">bSizer20</property>
9249<property name="orient">wxVERTICAL</property>
9250<property name="permission">none</property>
9251<object class="sizeritem" expanded="1">
9252<property name="border">5</property>
9253<property name="flag">wxEXPAND</property>
9254<property name="proportion">1</property>
9255<object class="wxBoxSizer" expanded="1">
9256<property name="minimum_size"></property>
9257<property name="name">bSizer116</property>
9258<property name="orient">wxHORIZONTAL</property>
9259<property name="permission">none</property>
9260<object class="sizeritem" expanded="1">
9261<property name="border">5</property>
9262<property name="flag">wxALL|wxEXPAND</property>
9263<property name="proportion">1</property>
9264<object class="wxHtmlWindow" expanded="1">
9265<property name="bg"></property>
9266<property name="context_help"></property>
9267<property name="enabled">1</property>
9268<property name="fg"></property>
9269<property name="font"></property>
9270<property name="hidden">1</property>
9271<property name="id">wxID_ANY</property>
9272<property name="maximum_size"></property>
9273<property name="minimum_size"></property>
9274<property name="name">m_htmlWinReviews</property>
9275<property name="permission">protected</property>
9276<property name="pos"></property>
9277<property name="size"></property>
9278<property name="style">wxHW_SCROLLBAR_AUTO</property>
9279<property name="subclass"></property>
9280<property name="tooltip"></property>
9281<property name="window_extra_style"></property>
9282<property name="window_name"></property>
9283<property name="window_style"></property>
9284<event name="OnChar"></event>
9285<event name="OnEnterWindow"></event>
9286<event name="OnEraseBackground"></event>
9287<event name="OnHtmlCellClicked"></event>
9288<event name="OnHtmlCellHover"></event>
9289<event name="OnHtmlLinkClicked"></event>
9290<event name="OnKeyDown"></event>
9291<event name="OnKeyUp"></event>
9292<event name="OnKillFocus"></event>
9293<event name="OnLeaveWindow"></event>
9294<event name="OnLeftDClick"></event>
9295<event name="OnLeftDown"></event>
9296<event name="OnLeftUp"></event>
9297<event name="OnMiddleDClick"></event>
9298<event name="OnMiddleDown"></event>
9299<event name="OnMiddleUp"></event>
9300<event name="OnMotion"></event>
9301<event name="OnMouseEvents"></event>
9302<event name="OnMouseWheel"></event>
9303<event name="OnPaint"></event>
9304<event name="OnRightDClick"></event>
9305<event name="OnRightDown"></event>
9306<event name="OnRightUp"></event>
9307<event name="OnSetFocus"></event>
9308<event name="OnSize"></event>
9309<event name="OnUpdateUI"></event>
9310</object>
9311</object>
9312<object class="sizeritem" expanded="1">
9313<property name="border">5</property>
9314<property name="flag">wxEXPAND|wxALL</property>
9315<property name="proportion">1</property>
9316<object class="wxScrolledWindow" expanded="1">
9317<property name="bg">wxSYS_COLOUR_WINDOW</property>
9318<property name="context_help"></property>
9319<property name="enabled">1</property>
9320<property name="fg"></property>
9321<property name="font"></property>
9322<property name="hidden">0</property>
9323<property name="id">wxID_ANY</property>
9324<property name="maximum_size"></property>
9325<property name="minimum_size"></property>
9326<property name="name">m_scrolledWindow</property>
9327<property name="permission">protected</property>
9328<property name="pos"></property>
9329<property name="scroll_rate_x">5</property>
9330<property name="scroll_rate_y">5</property>
9331<property name="size"></property>
9332<property name="subclass"></property>
9333<property name="tooltip"></property>
9334<property name="window_extra_style"></property>
9335<property name="window_name"></property>
9336<property
  name="window_style">wxHSCROLL|wxTAB_TRAVERSAL|wxVSCROLL</property>
9337<event name="OnChar"></event>
9338<event name="OnEnterWindow"></event>
9339<event name="OnEraseBackground"></event>
9340<event name="OnKeyDown"></event>
9341<event name="OnKeyUp"></event>
9342<event name="OnKillFocus"></event>
9343<event name="OnLeaveWindow"></event>
9344<event name="OnLeftDClick"></event>
9345<event name="OnLeftDown"></event>
9346<event name="OnLeftUp"></event>
9347<event name="OnMiddleDClick"></event>
9348<event name="OnMiddleDown"></event>
9349<event name="OnMiddleUp"></event>
9350<event name="OnMotion"></event>
9351<event name="OnMouseEvents"></event>
9352<event name="OnMouseWheel"></event>
9353<event name="OnPaint"></event>
9354<event name="OnRightDClick"></event>
9355<event name="OnRightDown"></event>
9356<event name="OnRightUp"></event>
9357<event name="OnSetFocus"></event>
9358<event name="OnSize"></event>
9359<event name="OnUpdateUI"></event>
9360<object class="wxBoxSizer" expanded="1">
9361<property name="minimum_size"></property>
9362<property name="name">bSizer21</property>
9363<property name="orient">wxVERTICAL</property>
9364<property name="permission">none</property>
9365<object class="sizeritem" expanded="1">

```

```

9366<property name="border">5</property>
9367<property name="flag">wxEXPAND</property>
9368<property name="proportion">0</property>
9369<object class="wxRichTextCtrl" expanded="1">
9370<property name="bg"></property>
9371<property name="context_help"></property>
9372<property name="enabled">1</property>
9373<property name="fg"></property>
9374<property name="font"></property>
9375<property name="hidden">0</property>
9376<property name="id">wxID_ANY</property>
9377<property name="maximum_size"></property>
9378<property name="minimum_size">-1,-1</property>
9379<property name="name">m_richTextHeading</property>
9380<property name="permission">protected</property>
9381<property name="pos"></property>
9382<property name="size">-1,50</property>
9383<property name="style">wxTE_READONLY</property>
9384<property name="subclass"></property>
9385<property name="tooltip"></property>
9386<property name="window_extra_style"></property>
9387<property name="window_name"></property>
9388<property name="window_style">wxNO_BORDER</property>
9389<event name="OnChar"></event>
9390<event name="OnEnterWindow"></event>
9391<event name="OnEraseBackground"></event>
9392<event name="OnKeyDown"></event>
9393<event name="OnKeyUp"></event>
9394<event name="OnKillFocus"></event>
9395<event name="OnLeaveWindow"></event>
9396<event name="OnLeftClick"></event>
9397<event name="OnLeftDown"></event>
9398<event name="OnLeftUp"></event>
9399<event name="OnMiddleClick"></event>
9400<event name="OnMiddleDown"></event>
9401<event name="OnMiddleUp"></event>
9402<event name="OnMotion"></event>
9403<event name="OnMouseEvents"></event>
9404<event name="OnMouseWheel"></event>
9405<event name="OnPaint"></event>
9406<event name="OnRichTextCharacter"></event>
9407<event name="OnRichTextContentDeleted"></event>
9408<event name="OnRichTextContentInserted"></event>
9409<event name="OnRichTextDelete"></event>
9410<event name="OnRichTextReturn"></event>
9411<event name="OnRichTextStyleChanged"></event>
9412<event name="OnRichTextStyleSheetChanged"></event>
9413<event name="OnRichTextStyleSheetReplaced"></event>
9414<event name="OnRichTextStyleSheetReplacing"></event>
9415<event name="OnRightClick"></event>
9416<event name="OnRightDown"></event>
9417<event name="OnRightUp"></event>
9418<event name="OnSetFocus"></event>
9419<event name="OnSize"></event>
9420<event name="OnText"></event>
9421<event name="OnTextEnter"></event>
9422<event name="OnTextMaxLen"></event>
9423<event name="OnTextURL"></event>
9424<event name="OnUpdateUI"></event>
9425</object>
9426</object>
9427<object class="sizeritem" expanded="1">
9428<property name="border">5</property>
9429<property name="flag">wxALL|wxEXPAND</property>
9430<property name="proportion">0</property>
9431<object class="wxStaticText" expanded="1">
9432<property name="bg"></property>
9433<property name="context_help"></property>
9434<property name="enabled">1</property>
9435<property name="fg"></property>
9436<property name="font"></property>
9437<property name="hidden">0</property>
9438<property name="id">wxID_ANY</property>
9439<property name="label">Order Form instructions here&#x0A;multiple
Lines&#x0A;1&#x0A;2&#x0A;3&#x0A;4&#x0A;5&#x0A;6</property>
9440<property name="maximum_size"></property>
9441<property name="minimum_size"></property>
9442<property name="name">m_staticTextInstructions</property>
9443<property name="permission">protected</property>
9444<property name="pos"></property>
9445<property name="size"></property>
9446<property name="style"></property>
9447<property name="subclass"></property>
9448<property name="tooltip"></property>
9449<property name="window_extra_style"></property>
9450<property name="window_name"></property>
9451<property name="window_style"></property>
9452<property name="wrap">1</property>
9453<event name="OnChar"></event>
9454<event name="OnEnterWindow"></event>
9455<event name="OnEraseBackground"></event>
9456<event name="OnKeyDown"></event>
9457<event name="OnKeyUp"></event>
9458<event name="OnKillFocus"></event>
9459<event name="OnLeaveWindow"></event>
9460<event name="OnLeftClick"></event>
9461<event name="OnLeftDown"></event>
9462<event name="OnLeftUp"></event>
9463<event name="OnMiddleClick"></event>
9464<event name="OnMiddleDown"></event>
9465<event name="OnMiddleUp"></event>
9466<event name="OnMotion"></event>
9467<event name="OnMouseEvents"></event>
9468<event name="OnMouseWheel"></event>
9469<event name="OnPaint"></event>
9470<event name="OnRightClick"></event>
9471<event name="OnRightDown"></event>
9472<event name="OnRightUp"></event>
9473<event name="OnSetFocus"></event>
9474<event name="OnSize"></event>
9475<event name="OnUpdateUI"></event>
9476</object>
9477</object>
9478<object class="sizeritem" expanded="1">
9479<property name="border">5</property>
9480<property name="flag">wxALIGN_CENTER_HORIZONTAL</property>
9481<property name="proportion">0</property>
9482<object class="wxBoxSizer" expanded="1">
9483<property name="minimum_size"></property>
9484<property name="name">bSizer25</property>
9485<property name="orient">wxHORIZONTAL</property>
9486<property name="permission">none</property>
9487<object class="sizeritem" expanded="1">
9488<property name="border">5</property>
9489<property name="flag">wxALL</property>
9490<property name="proportion">0</property>
9491<object class="wxButton" expanded="1">
9492<property name="bg"></property>
9493<property name="context_help"></property>
9494<property name="default">0</property>
9495<property name="enabled">1</property>
9496<property name="fg"></property>
9497<property name="font"></property>
9498<property name="hidden">0</property>
9499<property name="id">wxID_BUTTONSAMPLE</property>
9500<property name="label">Submit</property>
9501<property name="maximum_size"></property>
9502<property name="minimum_size">-1,-1</property>
9503<property name="name">m_buttonSubmitForm</property>
9504<property name="permission">protected</property>
9505<property name="pos"></property>
9506<property name="size"></property>
9507<property name="style"></property>
9508<property name="subclass"></property>
9509<property name="tooltip"></property>
9510<property name="window_extra_style"></property>
9511<property name="window_name"></property>
9512<property name="window_style"></property>
9513<event name="OnButtonClick">OnButtonSubmitForm</event>
9514<event name="OnChar"></event>
9515<event name="OnEnterWindow"></event>
9516<event name="OnEraseBackground"></event>
9517<event name="OnKeyDown"></event>
9518<event name="OnKeyUp"></event>
9519<event name="OnKillFocus"></event>
9520<event name="OnLeaveWindow"></event>
9521<event name="OnLeftClick"></event>
9522<event name="OnLeftDown"></event>
9523<event name="OnLeftUp"></event>
9524<event name="OnMiddleClick"></event>
9525<event name="OnMiddleDown"></event>
9526<event name="OnMiddleUp"></event>
9527<event name="OnMotion"></event>
9528<event name="OnMouseEvents"></event>
9529<event name="OnMouseWheel"></event>
9530<event name="OnPaint"></event>
9531<event name="OnRightClick"></event>
9532<event name="OnRightDown"></event>
9533<event name="OnRightUp"></event>
9534<event name="OnSetFocus"></event>
9535<event name="OnSize"></event>
9536<event name="OnUpdateUI"></event>
9537</object>
9538</object>
9539<object class="sizeritem" expanded="1">
9540<property name="border">5</property>
9541<property name="flag">wxALL</property>
9542<property name="proportion">0</property>
9543<object class="wxButton" expanded="1">
9544<property name="bg"></property>
9545<property name="context_help"></property>
9546<property name="default">1</property>
9547<property name="enabled">1</property>
9548<property name="fg"></property>
9549<property name="font"></property>
9550<property name="hidden">0</property>
9551<property name="id">wxID_CANCEL2</property>
9552<property name="label">Cancel</property>
9553<property name="maximum_size"></property>
9554<property name="minimum_size"></property>
9555<property name="name">m_buttonCancelForm</property>
9556<property name="permission">protected</property>
9557<property name="pos"></property>
9558<property name="size"></property>
9559<property name="style"></property>
9560<property name="subclass"></property>
9561<property name="tooltip"></property>
9562<property name="window_extra_style"></property>
9563<property name="window_name"></property>
9564<property name="window_style"></property>
9565<event name="OnButtonClick">OnButtonCancelForm</event>
9566<event name="OnChar"></event>
9567<event name="OnEnterWindow"></event>
9568<event name="OnEraseBackground"></event>
9569<event name="OnKeyDown"></event>
9570<event name="OnKeyUp"></event>

```

```

9571<event name="OnKillFocus"></event>
9572<event name="OnLeaveWindow"></event>
9573<event name="OnLeftDClick"></event>
9574<event name="OnLeftDown"></event>
9575<event name="OnLeftUp"></event>
9576<event name="OnMiddleDClick"></event>
9577<event name="OnMiddleDown"></event>
9578<event name="OnMiddleUp"></event>
9579<event name="OnMotion"></event>
9580<event name="OnMouseEvents"></event>
9581<event name="OnMouseWheel"></event>
9582<event name="OnPaint"></event>
9583<event name="OnRightDClick"></event>
9584<event name="OnRightDown"></event>
9585<event name="OnRightUp"></event>
9586<event name="OnSetFocus"></event>
9587<event name="OnSize"></event>
9588<event name="OnUpdateUI"></event>
9589</object>
9590</object>
9591</object>
9592</object>
9593</object>
9594</object>
9595</object>
9596</object>
9597</object>
9598<object class="sizeritem" expanded="1">
9599<property name="border">5</property>
9600<property name="flag">wxALIGN_RIGHT</property>
9601<property name="proportion">0</property>
9602<object class="wxBoxSizer" expanded="1">
9603<property name="minimum_size">0</property>
9604<property name="name">Sizer26</property>
9605<property name="orient">wxHORIZONTAL</property>
9606<property name="permission">none</property>
9607<object class="sizeritem" expanded="1">
9608<property name="border">5</property>
9609<property name="flag">wxALL</property>
9610<property name="proportion">0</property>
9611<object class="wxButton" expanded="1">
9612<property name="bg"></property>
9613<property name="context_help"></property>
9614<property name="default">0</property>
9615<property name="enabled">0</property>
9616<property name="fg"></property>
9617<property name="font">0</property>
9618<property name="hidden">0</property>
9619<property name="id">wxID_BUTTONBACK</property>
9620<property name="label">&lt; &amp;Back </property>
9621<property name="maximum_size">0</property>
9622<property name="minimum_size">85,25</property>
9623<property name="name">m_buttonBack</property>
9624<property name="permission">protected</property>
9625<property name="pos"></property>
9626<property name="size"></property>
9627<property name="style"></property>
9628<property name="subclass"></property>
9629<property name="tooltip"></property>
9630<property name="window_extra_style"></property>
9631<property name="window_name"></property>
9632<property name="window_style"></property>
9633<event name="OnButtonClick">OnButtonBack</event>
9634<event name="OnChar"></event>
9635<event name="OnEnterWindow"></event>
9636<event name="OnEraseBackground"></event>
9637<event name="OnKeyDown"></event>
9638<event name="OnKeyUp"></event>
9639<event name="OnKillFocus"></event>
9640<event name="OnLeaveWindow"></event>
9641<event name="OnLeftDClick"></event>
9642<event name="OnLeftDown"></event>
9643<event name="OnLeftUp"></event>
9644<event name="OnMiddleDClick"></event>
9645<event name="OnMiddleDown"></event>
9646<event name="OnMiddleUp"></event>
9647<event name="OnMotion"></event>
9648<event name="OnMouseEvents"></event>
9649<event name="OnMouseWheel"></event>
9650<event name="OnPaint"></event>
9651<event name="OnRightDClick"></event>
9652<event name="OnRightDown"></event>
9653<event name="OnRightUp"></event>
9654<event name="OnSetFocus"></event>
9655<event name="OnSize"></event>
9656<event name="OnUpdateUI"></event>
9657</object>
9658</object>
9659<object class="sizeritem" expanded="1">
9660<property name="border">5</property>
9661<property name="flag">wxALL</property>
9662<property name="proportion">0</property>
9663<object class="wxButton" expanded="1">
9664<property name="bg"></property>
9665<property name="context_help"></property>
9666<property name="default">0</property>
9667<property name="enabled">1</property>
9668<property name="fg"></property>
9669<property name="font"></property>
9670<property name="hidden">0</property>
9671<property name="id">wxID_BUTTONNEXT</property>
9672<property name="label">&lt; &amp;Next &gt;</property>
9673<property name="maximum_size"></property>
9674<property name="minimum_size">85,25</property>
9675<property name="name">m_buttonNext</property>
9676<property name="permission">protected</property>
9677<property name="pos"></property>
9678<property name="size"></property>
9679<property name="style"></property>
9680<property name="subclass"></property>
9681<property name="tooltip"></property>
9682<property name="window_extra_style"></property>
9683<property name="window_name"></property>
9684<property name="window_style"></property>
9685<event name="OnButtonClick">OnButtonNext</event>
9686<event name="OnChar"></event>
9687<event name="OnEnterWindow"></event>
9688<event name="OnEraseBackground"></event>
9689<event name="OnKeyDown"></event>
9690<event name="OnKeyUp"></event>
9691<event name="OnKillFocus"></event>
9692<event name="OnLeaveWindow"></event>
9693<event name="OnLeftDClick"></event>
9694<event name="OnLeftDown"></event>
9695<event name="OnLeftUp"></event>
9696<event name="OnMiddleDClick"></event>
9697<event name="OnMiddleDown"></event>
9698<event name="OnMiddleUp"></event>
9699<event name="OnMotion"></event>
9700<event name="OnMouseEvents"></event>
9701<event name="OnMouseWheel"></event>
9702<event name="OnPaint"></event>
9703<event name="OnRightDClick"></event>
9704<event name="OnRightDown"></event>
9705<event name="OnRightUp"></event>
9706<event name="OnSetFocus"></event>
9707<event name="OnSize"></event>
9708<event name="OnUpdateUI"></event>
9709</object>
9710</object>
9711<object class="sizeritem" expanded="1">
9712<property name="border">5</property>
9713<property name="flag">wxALL</property>
9714<property name="proportion">0</property>
9715<object class="wxButton" expanded="1">
9716<property name="bg"></property>
9717<property name="context_help"></property>
9718<property name="default">0</property>
9719<property name="enabled">1</property>
9720<property name="fg"></property>
9721<property name="font"></property>
9722<property name="hidden">0</property>
9723<property name="id">wxID_CANCEL</property>
9724<property name="label">Cancel</property>
9725<property name="maximum_size"></property>
9726<property name="minimum_size">85,25</property>
9727<property name="name">m_buttonCancel</property>
9728<property name="permission">protected</property>
9729<property name="pos"></property>
9730<property name="size"></property>
9731<property name="style"></property>
9732<property name="subclass"></property>
9733<property name="tooltip"></property>
9734<property name="window_extra_style"></property>
9735<property name="window_name"></property>
9736<property name="window_style"></property>
9737<event name="OnButtonClick">OnButtonCancel</event>
9738<event name="OnChar"></event>
9739<event name="OnEnterWindow"></event>
9740<event name="OnEraseBackground"></event>
9741<event name="OnKeyDown"></event>
9742<event name="OnKeyUp"></event>
9743<event name="OnKillFocus"></event>
9744<event name="OnLeaveWindow"></event>
9745<event name="OnLeftDClick"></event>
9746<event name="OnLeftDown"></event>
9747<event name="OnLeftUp"></event>
9748<event name="OnMiddleDClick"></event>
9749<event name="OnMiddleDown"></event>
9750<event name="OnMiddleUp"></event>
9751<event name="OnMotion"></event>
9752<event name="OnMouseEvents"></event>
9753<event name="OnMouseWheel"></event>
9754<event name="OnPaint"></event>
9755<event name="OnRightDClick"></event>
9756<event name="OnRightDown"></event>
9757<event name="OnRightUp"></event>
9758<event name="OnSetFocus"></event>
9759<event name="OnSize"></event>
9760<event name="OnUpdateUI"></event>
9761</object>
9762</object>
9763</object>
9764</object>
9765</object>
9766</object>
9767<object class="Frame" expanded="0">
9768<property name="bg">wxSYS_COLOUR_MENU</property>
9769<property name="center"></property>
9770<property name="context_help"></property>
9771<property name="enabled">1</property>
9772<property name="extra_style"></property>
9773<property name="fg"></property>
9774<property name="font"></property>
9775<property name="hidden">0</property>
9776<property name="id">wxID_ANY</property>

```



```

9777<property name="maximum_size"></property>
9778<property name="minimum_size"></property>
9779<property name="name"><wCViewOrderDialogBase</property>
9780<property name="pos"></property>
9781<property name="size"><630,520</property>
9782<property
  name="style"><wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
9783<property name="subclass"></property>
9784<property name="title"><View Order</property>
9785<property name="tooltip"></property>
9786<property name="window_extra_style"></property>
9787<property name="window_name"></property>
9788<property name="window_style"><wxTAB_TRAVERSAL</property>
9789<property name="xrc_skip_size"><1</property>
9790<event name="OnActivate"></event>
9791<event name="OnActivateApp"></event>
9792<event name="OnChar"></event>
9793<event name="OnClose"></event>
9794<event name="OnEnterWindow"></event>
9795<event name="OnEraseBackground"></event>
9796<event name="OnHibernate"></event>
9797<event name="OnIconize"></event>
9798<event name="OnIdle"></event>
9799<event name="OnKeyDown"></event>
9800<event name="OnKeyUp"></event>
9801<event name="OnKillFocus"></event>
9802<event name="OnLeaveWindow"></event>
9803<event name="OnLeftDClick"></event>
9804<event name="OnLeftDown"></event>
9805<event name="OnLeftUp"></event>
9806<event name="OnMiddleClick"></event>
9807<event name="OnMiddleDown"></event>
9808<event name="OnMiddleUp"></event>
9809<event name="OnMotion"></event>
9810<event name="OnMouseEvents"></event>
9811<event name="OnMouseWheel"></event>
9812<event name="OnPaint"></event>
9813<event name="OnRightDClick"></event>
9814<event name="OnRightDown"></event>
9815<event name="OnRightUp"></event>
9816<event name="OnSetFocus"></event>
9817<event name="OnSize"></event>
9818<event name="OnUpdateUI"></event>
9819<object class="wxBoxSizer" expanded="1">
9820<property name="minimum_size"></property>
9821<property name="name"><bSizer20</property>
9822<property name="orient"><wxVERTICAL</property>
9823<property name="permission"><none</property>
9824<object class="sizeritem" expanded="1">
9825<property name="border"><5</property>
9826<property name="flag"><wxEXPAND</property>
9827<property name="proportion"><1</property>
9828<object class="wxBoxSizer" expanded="1">
9829<property name="minimum_size"></property>
9830<property name="name"><bSizer116</property>
9831<property name="orient"><wxHORIZONTAL</property>
9832<property name="permission"><none</property>
9833<object class="sizeritem" expanded="1">
9834<property name="border"><5</property>
9835<property name="flag"><wxALL|wxEXPAND</property>
9836<property name="proportion"><1</property>
9837<object class="wxHtmlWindow" expanded="1">
9838<property name="bg"></property>
9839<property name="context_help"></property>
9840<property name="enabled"><1</property>
9841<property name="fg"></property>
9842<property name="font"></property>
9843<property name="hidden"><0</property>
9844<property name="id"><wxID_ANY</property>
9845<property name="maximum_size"></property>
9846<property name="minimum_size"></property>
9847<property name="name"><m_htmlWin</property>
9848<property name="permission"><protected</property>
9849<property name="pos"></property>
9850<property name="size"></property>
9851<property name="style"><wxHW_SCROLLBAR_AUTO</property>
9852<property name="subclass"></property>
9853<property name="tooltip"></property>
9854<property name="window_extra_style"></property>
9855<property name="window_name"></property>
9856<property name="window_style"></property>
9857<event name="OnChar"></event>
9858<event name="OnEnterWindow"></event>
9859<event name="OnEraseBackground"></event>
9860<event name="OnHtmlCellClicked"></event>
9861<event name="OnHtmlCellHover"></event>
9862<event name="OnHtmlLinkClicked"></event>
9863<event name="OnKeyDown"></event>
9864<event name="OnKeyUp"></event>
9865<event name="OnKillFocus"></event>
9866<event name="OnLeaveWindow"></event>
9867<event name="OnLeftDClick"></event>
9868<event name="OnLeftDown"></event>
9869<event name="OnLeftUp"></event>
9870<event name="OnMiddleClick"></event>
9871<event name="OnMiddleDown"></event>
9872<event name="OnMiddleUp"></event>
9873<event name="OnMotion"></event>
9874<event name="OnMouseEvents"></event>
9875<event name="OnMouseWheel"></event>
9876<event name="OnPaint"></event>
9877<event name="OnRightDClick"></event>
9878<event name="OnRightDown"></event>
9879<event name="OnRightUp"></event>
9880<event name="OnSetFocus"></event>
9881<event name="OnSize"></event>
9882<event name="OnUpdateUI"></event>
9883</object>
9884</object>
9885</object>
9886</object>
9887<object class="sizeritem" expanded="1">
9888<property name="border"><5</property>
9889<property name="flag"><wxALIGN_RIGHT</property>
9890<property name="proportion"><0</property>
9891<object class="wxBoxSizer" expanded="1">
9892<property name="minimum_size"></property>
9893<property name="name"><bSizer26</property>
9894<property name="orient"><wxHORIZONTAL</property>
9895<property name="permission"><none</property>
9896<object class="sizeritem" expanded="1">
9897<property name="border"><5</property>
9898<property name="flag"><wxALL</property>
9899<property name="proportion"><0</property>
9900<object class="wxButton" expanded="1">
9901<property name="bg"></property>
9902<property name="context_help"></property>
9903<property name="default"><0</property>
9904<property name="enabled"><1</property>
9905<property name="fg"></property>
9906<property name="font"></property>
9907<property name="hidden"><0</property>
9908<property name="id"><wxID_OK</property>
9909<property name="label"><OK</property>
9910<property name="maximum_size"></property>
9911<property name="minimum_size"><85,25</property>
9912<property name="name"><m_buttonOK</property>
9913<property name="permission"><protected</property>
9914<property name="pos"></property>
9915<property name="size"></property>
9916<property name="style"></property>
9917<property name="subclass"></property>
9918<property name="tooltip"></property>
9919<property name="window_extra_style"></property>
9920<property name="window_name"></property>
9921<property name="window_style"></property>
9922<event name="OnButtonClick"><OnButtonOK</event>
9923<event name="OnChar"></event>
9924<event name="OnEnterWindow"></event>
9925<event name="OnEraseBackground"></event>
9926<event name="OnKeyDown"></event>
9927<event name="OnKeyUp"></event>
9928<event name="OnKillFocus"></event>
9929<event name="OnLeaveWindow"></event>
9930<event name="OnLeftDClick"></event>
9931<event name="OnLeftDown"></event>
9932<event name="OnLeftUp"></event>
9933<event name="OnMiddleClick"></event>
9934<event name="OnMiddleDown"></event>
9935<event name="OnMiddleUp"></event>
9936<event name="OnMotion"></event>
9937<event name="OnMouseEvents"></event>
9938<event name="OnMouseWheel"></event>
9939<event name="OnPaint"></event>
9940<event name="OnRightDClick"></event>
9941<event name="OnRightDown"></event>
9942<event name="OnRightUp"></event>
9943<event name="OnSetFocus"></event>
9944<event name="OnSize"></event>
9945<event name="OnUpdateUI"></event>
9946</object>
9947</object>
9948</object>
9949</object>
9950</object>
9951</object>
9952<object class="Frame" expanded="0">
9953<property name="bg"><wxSYS_COLOUR_MENU</property>
9954<property name="center"></property>
9955<property name="context_help"></property>
9956<property name="enabled"><1</property>
9957<property name="extra_style"></property>
9958<property name="fg"></property>
9959<property name="font"></property>
9960<property name="hidden"><0</property>
9961<property name="id"><wxID_ANY</property>
9962<property name="maximum_size"></property>
9963<property name="minimum_size"></property>
9964<property name="name"><EditReviewDialogBase</property>
9965<property name="pos"></property>
9966<property name="size"><630,440</property>
9967<property
  name="style"><wxDEFAULT_FRAME_STYLE|wxRESIZE_BORDER</property>
9968<property name="subclass"></property>
9969<property name="title"><Enter Review</property>
9970<property name="tooltip"></property>
9971<property name="window_extra_style"></property>
9972<property name="window_name"></property>
9973<property name="window_style"><wxTAB_TRAVERSAL</property>
9974<property name="xrc_skip_size"><1</property>
9975<event name="OnActivate"></event>
9976<event name="OnActivateApp"></event>
9977<event name="OnChar"></event>
9978<event name="OnClose"></event>
9979<event name="OnEnterWindow"></event>
9980<event name="OnEraseBackground"></event>

```

```

9981<event name="OnHibernate"></event>
9982<event name="OnIconize"></event>
9983<event name="OnIdle"></event>
9984<event name="OnKeyDown"></event>
9985<event name="OnKeyUp"></event>
9986<event name="OnKillFocus"></event>
9987<event name="OnLeaveWindow"></event>
9988<event name="OnLeftDClick"></event>
9989<event name="OnLeftDown"></event>
9990<event name="OnLeftUp"></event>
9991<event name="OnMiddleClick"></event>
9992<event name="OnMiddleDown"></event>
9993<event name="OnMiddleUp"></event>
9994<event name="OnMotion"></event>
9995<event name="OnMouseEvents"></event>
9996<event name="OnMouseWheel"></event>
9997<event name="OnPaint"></event>
9998<event name="OnRightDClick"></event>
9999<event name="OnRightDown"></event>
1000<event name="OnRightUp"></event>
1000<event name="OnSetFocus"></event>
1000<event name="OnSize"></event>
1000<event name="OnUpdateUI"></event>
1000<object class="wxBoxSizer" expanded="1">
1000<property name="minimum_size"></property>
1000<property name="name">wSizer12</property>
1000<property name="orient">wxVERTICAL</property>
1000<property name="permission">none</property>
1000<object class="sizeritem" expanded="1">
1001<property name="border">5</property>
1001<property name="flag"></property>
1001<property name="proportion">0</property>
1001<object class="spacer" expanded="1">
1001<property name="height">3</property>
1001<property name="permission">protected</property>
1001<property name="width">0</property>
1001</object>
1001</object>
1001<object class="sizeritem" expanded="1">
1002<property name="border">5</property>
1002<property name="flag">wxALL|wxEXPAND</property>
1002<property name="proportion">0</property>
1002<object class="wxStaticText" expanded="1">
1002<property name="bg"></property>
1002<property name="context_help"></property>
1002<property name="enabled">1</property>
1002<property name="fg"></property>
1002<property name="font"></property>
1002<property name="hidden"></property>
1003<property name="id">wxID_ANY</property>
1003<property name="label"></property>
1003<property name="maximum_size"></property>
1003<property name="minimum_size"></property>
1003<property name="name">m_staticTextSeller</property>
1003<property name="permission">protected</property>
1003<property name="pos"></property>
1003<property name="size"></property>
1003<property name="style"></property>
1003<property name="subclass"></property>
1004<property name="tooltip"></property>
1004<property name="window_extra_style"></property>
1004<property name="window_name"></property>
1004<property name="window_style"></property>
1004<property name="wrap">1</property>
1004<event name="OnChar"></event>
1004<event name="OnEnterWindow"></event>
1004<event name="OnEraseBackground"></event>
1004<event name="OnKeyDown"></event>
1004<event name="OnKeyUp"></event>
1005<event name="OnKillFocus"></event>
1005<event name="OnLeaveWindow"></event>
1005<event name="OnLeftDClick"></event>
1005<event name="OnLeftDown"></event>
1005<event name="OnLeftUp"></event>
1005<event name="OnMiddleClick"></event>
1005<event name="OnMiddleDown"></event>
1005<event name="OnMiddleUp"></event>
1005<event name="OnMotion"></event>
1005<event name="OnMouseEvents"></event>
1006<event name="OnMouseWheel"></event>
1006<event name="OnPaint"></event>
1006<event name="OnRightDClick"></event>
1006<event name="OnRightDown"></event>
1006<event name="OnRightUp"></event>
1006<event name="OnSetFocus"></event>
1006<event name="OnSize"></event>
1006<event name="OnUpdateUI"></event>
1006</object>
1006</object>
1007<object class="sizeritem" expanded="1">
1007<property name="border">5</property>
1007<property name="flag"></property>
1007<property name="proportion">0</property>
1007<object class="spacer" expanded="1">
1007<property name="height">3</property>
1007<property name="permission">protected</property>
1007<property name="width">0</property>
1007</object>
1007</object>
1008<object class="sizeritem" expanded="1">
1008<property name="border">5</property>
1008<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
1008<property name="proportion">0</property>

```

```

1008<object class="wxStaticText" expanded="1">
1008<property name="bg"></property>
1008<property name="context_help"></property>
1008<property name="enabled">1</property>
1008<property name="fg"></property>
1008<property name="font"></property>
1009<property name="hidden">0</property>
1009<property name="id">wxID_ANY</property>
1009<property name="label">Rating</property>
1009<property name="maximum_size"></property>
1009<property name="minimum_size"></property>
1009<property name="name">m_staticText110</property>
1009<property name="permission">protected</property>
1009<property name="pos"></property>
1009<property name="size"></property>
1009<property name="style"></property>
1010<property name="subclass"></property>
1010<property name="tooltip"></property>
1010<property name="window_extra_style"></property>
1010<property name="window_name"></property>
1010<property name="window_style"></property>
1010<property name="wrap">1</property>
1010<event name="OnChar"></event>
1010<event name="OnEnterWindow"></event>
1010<event name="OnEraseBackground"></event>
1010<event name="OnKeyDown"></event>
1011<event name="OnKillFocus"></event>
1011<event name="OnLeaveWindow"></event>
1011<event name="OnLeftDClick"></event>
1011<event name="OnLeftDown"></event>
1011<event name="OnLeftUp"></event>
1011<event name="OnMiddleClick"></event>
1011<event name="OnMiddleDown"></event>
1011<event name="OnMiddleUp"></event>
1011<event name="OnMotion"></event>
1012<event name="OnMouseEvents"></event>
1012<event name="OnMouseWheel"></event>
1012<event name="OnPaint"></event>
1012<event name="OnRightDClick"></event>
1012<event name="OnRightDown"></event>
1012<event name="OnRightUp"></event>
1012<event name="OnSetFocus"></event>
1012<event name="OnSize"></event>
1012<event name="OnUpdateUI"></event>
1012</object>
1013</object>
1013<object class="sizeritem" expanded="1">
1013<property name="border">5</property>
1013<property name="flag">wxALL</property>
1013<property name="proportion">0</property>
1013<object class="wxChoice" expanded="1">
1013<property name="bg"></property>
1013<property name="choices">1 star&quot; 2 stars&quot;
&quot; 3 stars&quot; &quot; 4 stars&quot; &quot; 5 stars&quot;
</property>
1013<property name="context_help"></property>
1013<property name="enabled">1</property>
1014<property name="fg"></property>
1014<property name="font"></property>
1014<property name="hidden">0</property>
1014<property name="id">wxID_ANY</property>
1014<property name="maximum_size"></property>
1014<property name="minimum_size"></property>
1014<property name="name">m_choiceStars</property>
1014<property name="permission">protected</property>
1014<property name="pos"></property>
1014<property name="selection">0</property>
1015<property name="size"></property>
1015<property name="subclass"></property>
1015<property name="tooltip"></property>
1015<property name="window_extra_style"></property>
1015<property name="window_name"></property>
1015<property name="window_style"></property>
1015<event name="OnChar"></event>
1015<event name="OnEnterWindow"></event>
1015<event name="OnEraseBackground"></event>
1016<event name="OnKeyDown"></event>
1016<event name="OnKeyUp"></event>
1016<event name="OnKillFocus"></event>
1016<event name="OnLeaveWindow"></event>
1016<event name="OnLeftDClick"></event>
1016<event name="OnLeftDown"></event>
1016<event name="OnLeftUp"></event>
1016<event name="OnMiddleClick"></event>
1016<event name="OnMiddleDown"></event>
1016<event name="OnMiddleUp"></event>
1017<event name="OnMotion"></event>
1017<event name="OnMouseEvents"></event>
1017<event name="OnMouseWheel"></event>
1017<event name="OnPaint"></event>
1017<event name="OnRightDClick"></event>
1017<event name="OnRightDown"></event>
1017<event name="OnRightUp"></event>
1017<event name="OnSetFocus"></event>
1017<event name="OnSize"></event>
1017<event name="OnUpdateUI"></event>
1018</object>
1018</object>
1018<object class="sizeritem" expanded="1">
1018<property name="border">5</property>
1018<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>

```

```

1018<property name="proportion">0</property>
1018<object class="wxStaticText" expanded="1">
1018<property name="bg"></property>
1018<property name="context_help"></property>
1018<property name="enabled">1</property>
1019<property name="fg"></property>
1019<property name="font"></property>
1019<property name="hidden">0</property>
1019<property name="id">wxID_ANY</property>
1019<property name="label">Review</property>
1019<property name="maximum_size"></property>
1019<property name="minimum_size"></property>
1019<property name="name">m_staticText43</property>
1019<property name="permission">protected</property>
1019<property name="pos"></property>
1020<property name="size"></property>
1020<property name="style"></property>
1020<property name="subclass"></property>
1020<property name="tooltip"></property>
1020<property name="window_extra_style"></property>
1020<property name="window_name"></property>
1020<property name="window_style"></property>
1020<property name="wrap">1</property>
1020<event name="OnChar"></event>
1020<event name="OnEnterWindow"></event>
1021<event name="OnEraseBackground"></event>
1021<event name="OnKeyDown"></event>
1021<event name="OnKeyUp"></event>
1021<event name="OnKillFocus"></event>
1021<event name="OnLeaveWindow"></event>
1021<event name="OnLeftDClick"></event>
1021<event name="OnLeftDown"></event>
1021<event name="OnLeftUp"></event>
1021<event name="OnMiddleDClick"></event>
1021<event name="OnMiddleDown"></event>
1022<event name="OnMiddleUp"></event>
1022<event name="OnMotion"></event>
1022<event name="OnMouseEvents"></event>
1022<event name="OnMouseWheel"></event>
1022<event name="OnPaint"></event>
1022<event name="OnRightDClick"></event>
1022<event name="OnRightDown"></event>
1022<event name="OnRightUp"></event>
1022<event name="OnSetFocus"></event>
1022<event name="OnSize"></event>
1023<event name="OnUpdateUI"></event>
1023</object>
1023</object>
1023<object class="sizeritem" expanded="1">
1023<property name="border">5</property>
1023<property name="flag">wxALL|wxEXPAND</property>
1023<property name="proportion">1</property>
1023<object class="wxTextCtrl" expanded="1">
1023<property name="bg"></property>
1023<property name="context_help"></property>
1024<property name="enabled">1</property>
1024<property name="fg"></property>
1024<property name="font"></property>
1024<property name="hidden">0</property>
1024<property name="id">wxID_ANY</property>
1024<property name="maximum_size"></property>
1024<property name="maxlength">0</property>
1024<property name="name">m_textCtrlReview</property>
1024<property name="permission">protected</property>
1025<property name="pos"></property>
1025<property name="size"></property>
1025<property name="style">wxTE_MULTILINE</property>
1025<property name="subclass"></property>
1025<property name="tooltip"></property>
1025<property name="value"></property>
1025<property name="window_extra_style"></property>
1025<property name="window_name"></property>
1025<property name="window_style"></property>
1025<event name="OnChar"></event>
1026<event name="OnEnterWindow"></event>
1026<event name="OnEraseBackground"></event>
1026<event name="OnKeyDown">OnKeyDown</event>
1026<event name="OnKeyUp"></event>
1026<event name="OnKillFocus"></event>
1026<event name="OnLeaveWindow"></event>
1026<event name="OnLeftDClick"></event>
1026<event name="OnLeftDown"></event>
1026<event name="OnLeftUp"></event>
1026<event name="OnMiddleDClick"></event>
1027<event name="OnMiddleDown"></event>
1027<event name="OnMiddleUp"></event>
1027<event name="OnMotion"></event>
1027<event name="OnMouseEvents"></event>
1027<event name="OnMouseWheel"></event>
1027<event name="OnPaint"></event>
1027<event name="OnRightDClick"></event>
1027<event name="OnRightDown"></event>
1027<event name="OnRightUp"></event>
1027<event name="OnSetFocus"></event>
1028<event name="OnSize"></event>
1028<event name="OnText"></event>
1028<event name="OnTextEnter"></event>
1028<event name="OnTextMaxLen"></event>
1028<event name="OnTextURL"></event>
1028<event name="OnUpdateUI"></event>
1028</object>
1028</object>

```

```

1028<object class="sizeritem" expanded="1">
1028<property name="border">5</property>
1029<property name="flag">wxALIGN_RIGHT</property>
1029<property name="proportion">0</property>
1029<object class="wxBoxSizer" expanded="1">
1029<property name="minimum_size"></property>
1029<property name="name">mSizer113</property>
1029<property name="orient">wxHORIZONTAL</property>
1029<property name="permission">none</property>
1029<object class="sizeritem" expanded="1">
1029<property name="border">5</property>
1029<property name="flag">wxALL</property>
1030<property name="proportion">0</property>
1030<object class="wxButton" expanded="1">
1030<property name="bg"></property>
1030<property name="context_help"></property>
1030<property name="default">0</property>
1030<property name="enabled">1</property>
1030<property name="fg"></property>
1030<property name="font"></property>
1030<property name="hidden">0</property>
1030<property name="id">wxID_SUBMIT</property>
1031<property name="label">Submit</property>
1031<property name="maximum_size"></property>
1031<property name="minimum_size">85,25</property>
1031<property name="name">m_buttonSubmit</property>
1031<property name="permission">protected</property>
1031<property name="pos"></property>
1031<property name="size"></property>
1031<property name="style"></property>
1031<property name="subclass"></property>
1031<property name="tooltip"></property>
1032<property name="window_extra_style"></property>
1032<property name="window_name"></property>
1032<property name="window_style"></property>
1032<event name="OnButtonDClick">OnButtonSubmit</event>
1032<event name="OnChar"></event>
1032<event name="OnEnterWindow"></event>
1032<event name="OnEraseBackground"></event>
1032<event name="OnKeyDown"></event>
1032<event name="OnKeyUp"></event>
1032<event name="OnKillFocus"></event>
1033<event name="OnLeaveWindow"></event>
1033<event name="OnLeftDClick"></event>
1033<event name="OnLeftDown"></event>
1033<event name="OnLeftUp"></event>
1033<event name="OnMiddleDClick"></event>
1033<event name="OnMiddleDown"></event>
1033<event name="OnMiddleUp"></event>
1033<event name="OnMotion"></event>
1033<event name="OnMouseEvents"></event>
1033<event name="OnMouseWheel"></event>
1034<event name="OnPaint"></event>
1034<event name="OnRightDClick"></event>
1034<event name="OnRightDown"></event>
1034<event name="OnRightUp"></event>
1034<event name="OnSetFocus"></event>
1034<event name="OnSize"></event>
1034<event name="OnUpdateUI"></event>
1034</object>
1034</object>
1034<object class="sizeritem" expanded="1">
1035<property name="border">5</property>
1035<property name="flag">wxALL</property>
1035<property name="proportion">0</property>
1035<object class="wxButton" expanded="1">
1035<property name="bg"></property>
1035<property name="context_help"></property>
1035<property name="default">0</property>
1035<property name="enabled">1</property>
1035<property name="fg"></property>
1035<property name="font"></property>
1036<property name="hidden">0</property>
1036<property name="id">wxID_CANCEL</property>
1036<property name="label">Cancel</property>
1036<property name="maximum_size"></property>
1036<property name="minimum_size">85,25</property>
1036<property name="name">m_buttonCancel</property>
1036<property name="permission">protected</property>
1036<property name="pos"></property>
1036<property name="size"></property>
1036<property name="style"></property>
1037<property name="subclass"></property>
1037<property name="tooltip"></property>
1037<property name="window_extra_style"></property>
1037<property name="window_name"></property>
1037<property name="window_style"></property>
1037<event name="OnButtonDClick">OnButtonCancel</event>
1037<event name="OnChar"></event>
1037<event name="OnEnterWindow"></event>
1037<event name="OnEraseBackground"></event>
1037<event name="OnKeyDown"></event>
1038<event name="OnKeyUp"></event>
1038<event name="OnKillFocus"></event>
1038<event name="OnLeaveWindow"></event>
1038<event name="OnLeftDClick"></event>
1038<event name="OnLeftDown"></event>
1038<event name="OnLeftUp"></event>
1038<event name="OnMiddleDClick"></event>
1038<event name="OnMiddleDown"></event>
1038<event name="OnMiddleUp"></event>
1038<event name="OnMotion"></event>
1039<event name="OnMouseEvents"></event>

```



```

1039<event name="OnMouseWheel"></event>
1039<event name="OnPaint"></event>
1039<event name="OnRightDClick"></event>
1039<event name="OnRightDown"></event>
1039<event name="OnRightUp"></event>
1039<event name="OnSetFocus"></event>
1039<event name="OnSize"></event>
1039<event name="OnUpdateUI"></event>
1039</object>
1040</object>
1040</object>
1040</object>
1040</object>
1040</object>
1040<object class="Frame" expanded="0">
1040<property name="bg">wxSYS_COLOUR_BTNFACE</property>
1040<property name="center"></property>
1040<property name="context_help"></property>
1040<property name="enabled">1</property>
1041<property name="extra_style"></property>
1041<property name="fg"></property>
1041<property name="font"></property>
1041<property name="hidden">0</property>
1041<property name="id">wxID_ANY</property>
1041<property name="maximum_size"></property>
1041<property name="minimum_size"></property>
1041<property name="name">PokerLobbyDialogBase</property>
1041<property name="pos"></property>
1041<property name="size">586, 457</property>
1042<property name="style">wxDEFAULT_FRAME_STYLE</property>
1042<property name="subclass"></property>
1042<property name="title">Poker Lobby</property>
1042<property name="tooltip"></property>
1042<property name="window_extra_style"></property>
1042<property name="window_name"></property>
1042<property name="window_style">wxTAB_TRAVERSAL</property>
1042<property name="xrc_skip_size">1</property>
1042<event name="OnActivate"></event>
1042<event name="OnActivateApp"></event>
1043<event name="OnChar"></event>
1043<event name="OnClose"></event>
1043<event name="OnEnterWindow"></event>
1043<event name="OnEraseBackground"></event>
1043<event name="OnHibernate"></event>
1043<event name="OnIconize"></event>
1043<event name="OnIdle"></event>
1043<event name="OnKeyDown"></event>
1043<event name="OnKeyUp"></event>
1043<event name="OnKillFocus"></event>
1044<event name="OnLeaveWindow"></event>
1044<event name="OnLeftDClick"></event>
1044<event name="OnLeftDown"></event>
1044<event name="OnLeftUp"></event>
1044<event name="OnMiddleDClick"></event>
1044<event name="OnMiddleDown"></event>
1044<event name="OnMiddleUp"></event>
1044<event name="OnMotion"></event>
1044<event name="OnMouseEvents"></event>
1044<event name="OnMouseWheel"></event>
1045<event name="OnPaint"></event>
1045<event name="OnRightDClick"></event>
1045<event name="OnRightDown"></event>
1045<event name="OnRightUp"></event>
1045<event name="OnSetFocus"></event>
1045<event name="OnSize"></event>
1045<event name="OnUpdateUI"></event>
1045<object class="wxBoxSizer" expanded="1">
1045<property name="minimum_size"></property>
1045<property name="name">Sizer156</property>
1046<property name="orient">wxHORIZONTAL</property>
1046<property name="permission">none</property>
1046<object class="sizeritem" expanded="1">
1046<property name="border">5</property>
1046<property name="flag">wxEXPAND|wxTOP|wxBOTTOM|wxLEFT</property>
1046<property name="proportion">0</property>
1046<object class="wxTreeCtrl" expanded="1">
1046<property name="bg"></property>
1046<property name="context_help"></property>
1046<property name="enabled">1</property>
1047<property name="fg"></property>
1047<property name="font"></property>
1047<property name="hidden">0</property>
1047<property name="id">wxID_ANY</property>
1047<property name="maximum_size">130, -1</property>
1047<property name="name">m_treeCtrl</property>
1047<property name="permission">protected</property>
1047<property name="pos"></property>
1047<property name="size"></property>
1048<property
name="style">wxTR_HAS_BUTTONS|wxTR_HIDE_ROOT|wxTR_LINES_AT_ROOT</pro
perty>
1048<property name="subclass"></property>
1048<property name="tooltip"></property>
1048<property name="window_extra_style"></property>
1048<property name="window_name"></property>
1048<property name="window_style"></property>
1048<event name="OnChar"></event>
1048<event name="OnEnterWindow"></event>
1048<event name="OnEraseBackground"></event>
1048<event name="OnKeyDown"></event>
1049<event name="OnKeyUp"></event>
1049<event name="OnKillFocus"></event>
1049<event name="OnLeaveWindow"></event>
1049<event name="OnLeftDClick"></event>
1049<event name="OnLeftDown"></event>
1049<event name="OnLeftUp"></event>
1049<event name="OnMiddleDClick"></event>
1049<event name="OnMiddleDown"></event>
1049<event name="OnMiddleUp"></event>
1049<event name="OnMotion"></event>
1050<event name="OnMouseEvents"></event>
1050<event name="OnMouseWheel"></event>
1050<event name="OnPaint"></event>
1050<event name="OnRightDClick"></event>
1050<event name="OnRightDown"></event>
1050<event name="OnRightUp"></event>
1050<event name="OnSetFocus"></event>
1050<event name="OnSize"></event>
1050<event name="OnTreeBeginDrag"></event>
1050<event name="OnTreeBeginLabelEdit"></event>
1051<event name="OnTreeBeginRDrag"></event>
1051<event name="OnTreeDeleteItem"></event>
1051<event name="OnTreeEndDrag"></event>
1051<event name="OnTreeEndLabelEdit"></event>
1051<event name="OnTreeGetInfo"></event>
1051<event name="OnTreeItemActivated"></event>
1051<event name="OnTreeItemCollapsed"></event>
1051<event name="OnTreeItemCollapsing"></event>
1051<event name="OnTreeItemExpanded"></event>
1051<event name="OnTreeItemExpanding"></event>
1052<event name="OnTreeItemGetTooltip"></event>
1052<event name="OnTreeItemMenu"></event>
1052<event name="OnTreeItemMiddleClick"></event>
1052<event name="OnTreeItemRightClick"></event>
1052<event name="OnTreeKeyDown"></event>
1052<event name="OnTreeSelChanged">OnTreeSelChanged</event>
1052<event name="OnTreeSelChanging"></event>
1052<event name="OnTreeSetInfo"></event>
1052<event name="OnTreeStateImageClick"></event>
1052<event name="OnUpdateUI"></event>
1053</object>
1053</object>
1053<object class="sizeritem" expanded="1">
1053<property name="border">5</property>
1053<property name="flag">wxEXPAND</property>
1053<property name="proportion">1</property>
1053<object class="wxBoxSizer" expanded="1">
1053<property name="minimum_size"></property>
1053<property name="name">Sizer172</property>
1053<property name="orient">wxVERTICAL</property>
1054<property name="permission">none</property>
1054<object class="sizeritem" expanded="1">
1054<property name="border">5</property>
1054<property name="flag">wxEXPAND|wxALL</property>
1054<property name="proportion">1</property>
1054<object class="wxListCtrl" expanded="1">
1054<property name="bg"></property>
1054<property name="context_help"></property>
1054<property name="enabled">1</property>
1054<property name="fg"></property>
1055<property name="font"></property>
1055<property name="hidden">0</property>
1055<property name="id">wxID_ANY</property>
1055<property name="maximum_size"></property>
1055<property name="minimum_size"></property>
1055<property name="name">m_listCtrl</property>
1055<property name="permission">protected</property>
1055<property name="pos"></property>
1055<property name="size"></property>
1055<property name="style">wxLC_NO_SORT_HEADER|wxLC_REPORT</property>
1056<property name="subclass"></property>
1056<property name="tooltip"></property>
1056<property name="window_extra_style"></property>
1056<property name="window_name"></property>
1056<property name="window_style"></property>
1056<event name="OnChar"></event>
1056<event name="OnEnterWindow"></event>
1056<event name="OnEraseBackground"></event>
1056<event name="OnKeyDown"></event>
1056<event name="OnKeyUp"></event>
1057<event name="OnKillFocus"></event>
1057<event name="OnLeaveWindow"></event>
1057<event name="OnLeftDClick"></event>
1057<event name="OnLeftDown"></event>
1057<event name="OnLeftUp"></event>
1057<event name="OnListBeginDrag"></event>
1057<event name="OnListBeginLabelEdit"></event>
1057<event name="OnListBeginRDrag"></event>
1057<event name="OnListCacheHint"></event>
1057<event name="OnListColBeginDrag"></event>
1058<event name="OnListColClick"></event>
1058<event name="OnListColDragging"></event>
1058<event name="OnListColEndDrag"></event>
1058<event name="OnListColRightClick"></event>
1058<event name="OnListDeleteAllItems"></event>
1058<event name="OnListDeleteItem"></event>
1058<event name="OnListEndLabelEdit"></event>
1058<event name="OnListInsertItem"></event>
1058<event name="OnListItemActivated">OnListItemActivated</event>
1058<event name="OnListItemDeselected"></event>
1059<event name="OnListItemFocused"></event>
1059<event name="OnListItemMiddleClick"></event>
1059<event name="OnListItemRightClick"></event>
1059<event name="OnListItemSelected">OnListItemSelected</event>
1059<event name="OnListKeyDown"></event>

```

```

1059<event name="OnMiddleDClick"></event>
1059<event name="OnMiddleDown"></event>
1059<event name="OnMiddleUp"></event>
1059<event name="OnMotion"></event>
1059<event name="OnMouseEvents"></event>
1060<event name="OnMouseWheel"></event>
1060<event name="OnPaint"></event>
1060<event name="OnRightDClick"></event>
1060<event name="OnRightDown"></event>
1060<event name="OnRightUp"></event>
1060<event name="OnSetFocus"></event>
1060<event name="OnSize"></event>
1060<event name="OnUpdateUI"></event>
1060</object>
1060</object>
1061<object class="sizeritem" expanded="1">
1061<property name="border">5</property>
1061<property name="flag">wxALL</property>
1061<property name="proportion">0</property>
1061<object class="wxButton" expanded="1">
1061<property name="bg"></property>
1061<property name="context_help"></property>
1061<property name="default">0</property>
1061<property name="enabled">1</property>
1061<property name="fg"></property>
1062<property name="font"></property>
1062<property name="hidden">0</property>
1062<property name="id">wxID_OPENNEWTABLE</property>
1062<property name="label">&Open New Table</property>
1062<property name="maximum_size"></property>
1062<property name="minimum_size"></property>
1062<property name="name">m_buttonNewTable</property>
1062<property name="permission">protected</property>
1062<property name="pos"></property>
1062<property name="size"></property>
1063<property name="style"></property>
1063<property name="subclass"></property>
1063<property name="tooltip"></property>
1063<property name="window_extra_style"></property>
1063<property name="window_name"></property>
1063<property name="window_style"></property>
1063<event name="OnButtonClick">OnButtonNewTable</event>
1063<event name="OnChar"></event>
1063<event name="OnEnterWindow"></event>
1063<event name="OnEraseBackground"></event>
1064<event name="OnKeyDown"></event>
1064<event name="OnKeyUp"></event>
1064<event name="OnKillFocus"></event>
1064<event name="OnLeaveWindow"></event>
1064<event name="OnLeftDClick"></event>
1064<event name="OnLeftDown"></event>
1064<event name="OnLeftUp"></event>
1064<event name="OnMiddleDClick"></event>
1064<event name="OnMiddleDown"></event>
1064<event name="OnMiddleUp"></event>
1065<event name="OnMotion"></event>
1065<event name="OnMouseEvents"></event>
1065<event name="OnMouseWheel"></event>
1065<event name="OnPaint"></event>
1065<event name="OnRightDClick"></event>
1065<event name="OnRightDown"></event>
1065<event name="OnRightUp"></event>
1065<event name="OnSetFocus"></event>
1065<event name="OnSize"></event>
1065<event name="OnUpdateUI"></event>
1066</object>
1066</object>
1066</object>
1066</object>
1066</object>
1066</object>
1066<object class="Frame" expanded="0">
1066<property name="bg"></property>
1066<property name="center"></property>
1066<property name="context_help"></property>
1067<property name="enabled">1</property>
1067<property name="extra_style"></property>
1067<property name="fg"></property>
1067<property name="font"></property>
1067<property name="hidden">0</property>
1067<property name="id">wxID_ANY</property>
1067<property name="maximum_size"></property>
1067<property name="minimum_size"></property>
1067<property name="name">CPokerDialogBase</property>
1067<property name="pos"></property>
1068<property name="size">806, 550</property>
1068<property
  name="style">wxDEFAULT_FRAME_STYLE|wxFRAME_NO_TASKBAR</property>
1068<property name="subclass"></property>
1068<property name="title">Poker</property>
1068<property name="tooltip"></property>
1068<property name="window_extra_style"></property>
1068<property name="window_name"></property>
1068<property
  name="window_style">wxFULL_PAINT|wxTAB_TRAVERSAL</property>
1068<property name="xrc_skip_size">1</property>
1068<event name="OnActivate"></event>
1069<event name="OnActivateApp"></event>
1069<event name="OnChar"></event>
1069<event name="OnClose">OnClose</event>
1069<event name="OnEnterWindow"></event>
1069<event name="OnEraseBackground"></event>
1069<event name="OnHibernate"></event>
1069<event name="OnIconize"></event>
1069<event name="OnIdle"></event>
1069<event name="OnKeyDown"></event>
1069<event name="OnKeyUp"></event>
1070<event name="OnKillFocus"></event>
1070<event name="OnLeaveWindow"></event>
1070<event name="OnLeftDClick"></event>
1070<event name="OnLeftDown"></event>
1070<event name="OnLeftUp"></event>
1070<event name="OnMiddleDClick"></event>
1070<event name="OnMiddleDown"></event>
1070<event name="OnMiddleUp"></event>
1070<event name="OnMotion"></event>
1070<event name="OnMouseEvents">OnMouseEvents</event>
1071<event name="OnMouseWheel"></event>
1071<event name="OnPaint">OnPaint</event>
1071<event name="OnRightDClick"></event>
1071<event name="OnRightDown"></event>
1071<event name="OnRightUp"></event>
1071<event name="OnSetFocus"></event>
1071<event name="OnSize">OnSize</event>
1071<event name="OnUpdateUI"></event>
1071<object class="wxBoxSizer" expanded="1">
1071<property name="minimum_size"></property>
1072<property name="name">bSizer14</property>
1072<property name="orient">VERTICAL</property>
1072<property name="permission">none</property>
1072<object class="sizeritem" expanded="1">
1072<property name="border">5</property>
1072<property name="flag">wxALL</property>
1072<property name="proportion">0</property>
1072<object class="wxCheckBox" expanded="1">
1072<property name="bg"></property>
1072<property name="checked">0</property>
1073<property name="context_help"></property>
1073<property name="enabled">1</property>
1073<property name="fg"></property>
1073<property name="font"></property>
1073<property name="hidden">0</property>
1073<property name="id">wxID_ANY</property>
1073<property name="label">Deal Me Out</property>
1073<property name="maximum_size"></property>
1073<property name="minimum_size"></property>
1073<property name="name">m_checkSitOut</property>
1074<property name="permission">public</property>
1074<property name="pos"></property>
1074<property name="size"></property>
1074<property name="style"></property>
1074<property name="subclass"></property>
1074<property name="tooltip"></property>
1074<property name="window_extra_style"></property>
1074<property name="window_name"></property>
1074<property name="window_style"></property>
1074<event name="OnChar"></event>
1075<event name="OnCheckBox">OnCheckSitOut</event>
1075<event name="OnEnterWindow"></event>
1075<event name="OnEraseBackground"></event>
1075<event name="OnKeyDown"></event>
1075<event name="OnKeyUp"></event>
1075<event name="OnKillFocus"></event>
1075<event name="OnLeaveWindow"></event>
1075<event name="OnLeftDClick"></event>
1075<event name="OnLeftDown"></event>
1075<event name="OnLeftUp"></event>
1076<event name="OnMiddleDClick"></event>
1076<event name="OnMiddleDown"></event>
1076<event name="OnMiddleUp"></event>
1076<event name="OnMotion"></event>
1076<event name="OnMouseEvents"></event>
1076<event name="OnMouseWheel"></event>
1076<event name="OnPaint"></event>
1076<event name="OnRightDClick"></event>
1076<event name="OnRightDown"></event>
1076<event name="OnRightUp"></event>
1077<event name="OnSetFocus"></event>
1077<event name="OnSize"></event>
1077<event name="OnUpdateUI"></event>
1077</object>
1077</object>
1077<object class="sizeritem" expanded="1">
1077<property name="border">5</property>
1077<property name="flag">wxALL</property>
1077<property name="proportion">0</property>
1077<object class="wxButton" expanded="1">
1078<property name="bg"></property>
1078<property name="context_help"></property>
1078<property name="default">0</property>
1078<property name="enabled">1</property>
1078<property name="fg"></property>
1078<property name="font"></property>
1078<property name="hidden">0</property>
1078<property name="id">wxID_DEALHAND</property>
1078<property name="label">&Deal Hand</property>
1078<property name="maximum_size"></property>
1079<property name="minimum_size"></property>
1079<property name="name">m_buttonDealHand</property>
1079<property name="permission">protected</property>
1079<property name="pos"></property>
1079<property name="size">150, 25</property>
1079<property name="style"></property>
1079<property name="subclass"></property>
1079<property name="tooltip"></property>

```

```

1079<property name="window_extra_style"></property>  

1080<property name="window_name"></property>  

1081<property name="window_style"></property>  

1082<event name="OnButtonClick">OnButtonDealHand</event>  

1083<event name="OnChar"></event>  

1084<event name="OnEnterWindow"></event>  

1085<event name="OnEraseBackground"></event>  

1086<event name="OnKeyDown"></event>  

1087<event name="OnKeyUp"></event>  

1088<event name="OnKillFocus"></event>  

1089<event name="OnLeaveWindow"></event>  

1090<event name="OnLeftDClick"></event>  

1091<event name="OnLeftDown"></event>  

1092<event name="OnLeftUp"></event>  

1093<event name="OnMiddleDClick"></event>  

1094<event name="OnMiddleDown"></event>  

1095<event name="OnMiddleUp"></event>  

1096<event name="OnMotion"></event>  

1097<event name="OnMouseEvents"></event>  

1098<event name="OnMouseMoveWheel"></event>  

1099<event name="OnPaint"></event>  

1100<event name="OnRightDClick"></event>  

1101<event name="OnRightDown"></event>  

1102<event name="OnRightUp"></event>  

1103<event name="OnSetFocus"></event>  

1104<event name="OnSize"></event>  

1105<event name="OnUpdateUI"></event>  

1106</object>  

1107</object>  

1108<object class="SizerItem" expanded="1">  

1109<property name="border">5</property>  

1110<property name="flag">wxALL</property>  

1111<property name="proportion">0</property>  

1112<object class="wxBUTTON" expanded="1">  

1113<property name="bg"></property>  

1114<property name="context_help"></property>  

1115<property name="default">0</property>  

1116<property name="enabled">1</property>  

1117<property name="fg"></property>  

1118<property name="font"></property>  

1119<property name="hidden">0</property>  

1120<property name="id">wxID_FOLD</property>  

1121<property name="label">&amp; Fold</property>  

1122<property name="maximum_size"></property>  

1123<property name="minimum_size"></property>  

1124<property name="name">m_buttonFold</property>  

1125<property name="permission">-protected</property>  

1126<property name="pos"></property>  

1127<property name="size">80,25</property>  

1128<property name="style"></property>  

1129<property name="subclass"></property>  

1130<property name="tooltip"></property>  

1131</object>  

1132<property name="window_extra_style"></property>  

1133<property name="window_name"></property>  

1134<property name="window_style"></property>  

1135<event name="OnButtonClick">OnButtonFold</event>  

1136<event name="OnChar"></event>  

1137<event name="OnEnterWindow"></event>  

1138<event name="OnEraseBackground"></event>  

1139<event name="OnKeyDown"></event>  

1140<event name="OnKeyUp"></event>  

1141<event name="OnKillFocus"></event>  

1142<event name="OnLeaveWindow"></event>  

1143<event name="OnLeftDClick"></event>  

1144<event name="OnLeftDown"></event>  

1145<event name="OnLeftUp"></event>  

1146<event name="OnMiddleDClick"></event>  

1147<event name="OnMiddleDown"></event>  

1148<event name="OnMiddleUp"></event>  

1149<event name="OnMotion"></event>  

1150<event name="OnMouseEvents"></event>  

1151<event name="OnMouseMoveWheel"></event>  

1152<event name="OnPaint"></event>  

1153<event name="OnRightDClick"></event>  

1154<event name="OnRightDown"></event>  

1155<event name="OnRightUp"></event>  

1156<event name="OnSetFocus"></event>  

1157<event name="OnSize"></event>  

1158<event name="OnUpdateUI"></event>  

1159</object>  

1160</object>  

1161<object class="SizerItem" expanded="1">  

1162<property name="border">5</property>  

1163<property name="flag">wxALL</property>  

1164<property name="proportion">0</property>  

1165<object class="wxBUTTON" expanded="1">  

1166<property name="bg"></property>  

1167<property name="context_help"></property>  

1168<property name="default">0</property>  

1169<property name="enabled">1</property>  

1170<property name="fg"></property>  

1171<property name="font"></property>  

1172<property name="hidden">0</property>  

1173<property name="id">wxID_CALL</property>  

1174<property name="label">&amp; Call</property>  

1175<property name="maximum_size"></property>  

1176<property name="minimum_size"></property>  

1177<property name="name">m_buttonCall</property>  

1178<property name="permission">-protected</property>  

1179<property name="pos"></property>  

1180<property name="size">80,25</property>  

1181<property name="style"></property>  

1182<property name="subclass"></property>  


```

```

1008<property name="subclass"></property>
1108<property name="permission">public</property>
1109<property name="pos"></property>
1110<property name="size">100,-1</property>
1111<property name="style"></property>
1112<property name="subclass"></property>
1113<property name="tooltip"></property>
1114<property name="window_extra_style"></property>
1115<property name="window_name"></property>
1116<property name="window_style"></property>
1117<event names="OnChar"></event>
1118<event names="OnEnterWindow"></event>
1119<event names="OnEraseBackground"></event>
1120<event names="OnKeyDown"></event>
1121<event names="OnKeyUp"></event>
1122<event names="OnKillFocus"></event>
1123<event names="OnLeaveWindow"></event>
1124<event names="OnLeftDClick"></event>
1125<event names="OnLeftDown"></event>
1126<event names="OnLeftUp"></event>
1127<event names="OnMiddleBClick"></event>
1128<event names="OnMiddleDown"></event>
1129<event names="OnMiddleUp"></event>
1130<event names="OnMotion"></event>
1131<event names="OnMouseEvents"></event>
1132<event names="OnMouseWheel"></event>
1133<event names="OnPaint"></event>
1134<event names="OnRightDClick"></event>
1135<event names="OnRightDown"></event>
1136<event names="OnRightUp"></event>
1137<event names="OnSetFocus"></event>
1138<event names="OnSize"></event>
1139<event names="OnUpdateUI"></event>
1140</object>
1141</object>
1142<object class="sizeritem" expanded="1">
1143<property name="border">5</property>
1144<property name="flag">wxALL</property>
1145<property name="proportion">0</property>
1146<object class="wxTextCtrl" expanded="1">
1147<property name="bg"></property>
1148<property name="context_help"></property>
1149<property name="enabled">1</property>
1150<property name="fg"></property>
1151<property name="font"></property>
1152<property name="hidden">0</property>
1153<property name="id">wxID_DITCHPLAYER</property>
1154<property name="maximum_size"></property>
1155<property name="maxlength">0</property>
1156<property name="minimum_size"></property>
1157<property name="name">m_textDitchPlayer</property>
1158<property name="permission">protected</property>
1159<property name="pos"></property>
1160<property name="size">45,-1</property>
1161<property name="style">wxE_PROCESS_ENTER</property>
1162<property name="subclass"></property>
1163<property name="tooltip"></property>
1164<property name="value"></property>
1165<property name="window_extra_style"></property>
1166<property name="window_name"></property>
1167<property name="window_style"></property>
1168<event names="OnChar"></event>
1169<event names="OnEnterWindow"></event>
1170<event names="OnEraseBackground"></event>
1171<event names="OnKeyDown"></event>
1172<event names="OnKeyUp"></event>
1173<event names="OnKillFocus"></event>
1174<event names="OnLeaveWindow"></event>
1175<event names="OnLeftDClick"></event>
1176<event names="OnLeftDown"></event>
1177<event names="OnLeftUp"></event>
1178<event names="OnMiddleBClick"></event>
1179<event names="OnMiddleDown"></event>
1180<event names="OnMiddleUp"></event>
1181<event names="OnMotion"></event>
1182<event names="OnMouseEvents"></event>
1183<event names="OnMouseWheel"></event>
1184<event names="OnPaint"></event>
1185<event names="OnRightDClick"></event>
1186<event names="OnRightDown"></event>
1187<event names="OnRightUp"></event>
1188<event names="OnSetFocus"></event>
1189<event names="OnSize"></event>
1190<event names="OnText"></event>
1191<event names="OnTextEnter">OnDitchPlayer</event>
1192<event names="OnTextMaxLen"></event>
1193<event names="OnTextURL"></event>
1194<event names="OnUpdateUI"></event>
1195</object>
1196</object>
1197<object class="sizeritem" expanded="1">
1198<property name="border">5</property>
1199<property name="flag">wxALL</property>
1200<property name="proportion">0</property>
1201<object class="wxCheckBox" expanded="1">
1202<property name="bg"></property>
1203<property name="checked">0</property>
1204<property name="context_help"></property>
1205<property name="enabled">1</property>
1206<property name="fg"></property>
1207<property name="font"></property>
1208<property name="hidden">0</property>
1209<property name="id">wxID_ANY</property>
1210<property name="label">FOLD</property>
1211<property name="maximum_size"></property>
1212<property name="minimum_size"></property>
1213<property name="name">m_checkPreFold</property>

```

[illegible]

```

1141<property name="window_extra_style"></property>
1141<property name="window_name"></property>
1141<property name="window_style"></property>
1141<event name="OnActivate"></event>
1142<event name="OnActivateApp"></event>
1142<event name="OnChar"></event>
1142<event name="OnClose"><OnClose</event>
1142<event name="OnEnterWindow"></event>
1142<event name="OnEraseBackground"></event>
1142<event name="OnHibernate"></event>
1142<event name="OnIconize"></event>
1142<event name="OnIdle"></event>
1142<event name="OnInitDialog"></event>
1142<event name="OnKeyDown"></event>
1143<event name="OnKeyUp"></event>
1143<event name="OnKillFocus"></event>
1143<event name="OnLeaveWindow"></event>
1143<event name="OnLeftDClick"></event>
1143<event name="OnLeftDown"></event>
1143<event name="OnLeftUp"></event>
1143<event name="OnMiddleDClick"></event>
1143<event name="OnMiddleDown"></event>
1143<event name="OnMiddleUp"></event>
1143<event name="OnMotion"></event>
1144<event name="OnMouseEvents"></event>
1144<event name="OnMouseWheel"></event>
1144<event name="OnPaint"></event>
1144<event name="OnRightDClick"></event>
1144<event name="OnRightDown"></event>
1144<event name="OnRightUp"></event>
1144<event name="OnSetFocus"></event>
1144<event name="OnSize"></event>
1144<event name="OnUpdateUI"></event>
1144<object class="wxBoxSizer" expanded="1">
1145<property name="minimum_size"></property>
1145<property name="name">bSizer79</property>
1145<property name="orient">wxVERTICAL</property>
1145<property name="permission">none</property>
1145<object class="sizeritem" expanded="1">
1145<property name="border">10</property>
1145<property name="flag">wxEXPAND|wxALL</property>
1145<property name="proportion">1</property>
1145<object class="wxBoxSizer" expanded="1">
1145<property name="minimum_size"></property>
1146<property name="name">bSizer81</property>
1146<property name="orient">wxVERTICAL</property>
1146<property name="permission">none</property>
1146<object class="sizeritem" expanded="1">
1146<property name="border">5</property>
1146<property name="flag">wxEXPAND</property>
1146<property name="proportion">1</property>
1146<object class="spacer" expanded="1">
1146<property name="height">0</property>
1146<property name="permission">protected</property>
1147<property name="width">0</property>
1147</object>
1147</object>
1147<object class="sizeritem" expanded="1">
1147<property name="border">5</property>
1147<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
1147<property name="proportion">0</property>
1147<object class="wxStaticText" expanded="1">
1147<property name="bg"></property>
1147<property name="context_help"></property>
1148<property name="enabled">1</property>
1148<property name="fg"></property>
1148<property name="font"></property>
1148<property name="hidden">0</property>
1148<property name="id">wxID_ANY</property>
1148<property name="label"></property>
1148<property name="maximum_size"></property>
1148<property name="minimum_size"></property>
1148<property name="name">m_staticTextMessage1</property>
1148<property name="permission">protected</property>
1149<property name="pos"></property>
1149<property name="size"></property>
1149<property name="style"></property>
1149<property name="subclass"></property>
1149<property name="tooltip"></property>
1149<property name="window_extra_style"></property>
1149<property name="window_name"></property>
1149<property name="window_style"></property>
1149<property name="wrap">1</property>
1149<event name="OnChar"></event>
1150<event name="OnEnterWindow"></event>
1150<event name="OnEraseBackground"></event>
1150<event name="OnKeyDown"></event>
1150<event name="OnKeyUp"></event>
1150<event name="OnKillFocus"></event>
1150<event name="OnLeaveWindow"></event>
1150<event name="OnLeftDClick"></event>
1150<event name="OnLeftDown"></event>
1150<event name="OnLeftUp"></event>
1150<event name="OnMotion"></event>
1151<event name="OnMouseEvents"></event>
1151<event name="OnMouseWheel"></event>
1151<event name="OnPaint"></event>
1151<event name="OnRightDClick"></event>
1151<event name="OnRightDown"></event>
1151<event name="OnRightUp"></event>

```

```

1151<event name="OnSetFocus"></event>
1152<event name="OnSize"></event>
1152<event name="OnUpdateUI"></event>
1152</object>
1152</object>
1152<object class="sizeritem" expanded="1">
1152<property name="border">5</property>
1152<property name="flag">wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL</property>
1152<property name="proportion">0</property>
1152<object class="wxTextCtrl" expanded="1">
1152<property name="bg"></property>
1153<property name="context_help"></property>
1153<property name="enabled">1</property>
1153<property name="fg"></property>
1153<property name="font"></property>
1153<property name="hidden">0</property>
1153<property name="id">wxID_TEXTCTRL</property>
1153<property name="maximum_size"></property>
1153<property name="maxlength">0</property>
1153<property name="minimum_size"></property>
1153<property name="name">m_textCtrl1</property>
1154<property name="permission">protected</property>
1154<property name="pos"></property>
1154<property name="size"></property>
1154<property name="style">wxTE_PROCESS_ENTER</property>
1154<property name="subclass"></property>
1154<property name="tooltip"></property>
1154<property name="value"></property>
1154<property name="window_extra_style"></property>
1154<property name="window_name"></property>
1154<property name="window_style"></property>
1155<event name="OnChar"></event>
1155<event name="OnEnterWindow"></event>
1155<event name="OnEraseBackground"></event>
1155<event name="OnKeyDown"><OnKeyDown</event>
1155<event name="OnKeyUp"></event>
1155<event name="OnKillFocus"></event>
1155<event name="OnLeaveWindow"></event>
1155<event name="OnLeftDClick"></event>
1155<event name="OnLeftDown"></event>
1155<event name="OnLeftUp"></event>
1156<event name="OnMiddleDClick"></event>
1156<event name="OnMiddleDown"></event>
1156<event name="OnMiddleUp"></event>
1156<event name="OnMotion"></event>
1156<event name="OnMouseEvents"></event>
1156<event name="OnMouseWheel"></event>
1156<event name="OnPaint"></event>
1156<event name="OnRightDClick"></event>
1156<event name="OnRightDown"></event>
1156<event name="OnRightUp"></event>
1157<event name="OnSetFocus"></event>
1157<event name="OnSize"></event>
1157<event name="OnText"></event>
1157<event name="OnTextEnter"></event>
1157<event name="OnTextMaxLen"></event>
1157<event name="OnTextURL"></event>
1157<event name="OnUpdateUI"></event>
1157</object>
1157</object>
1157<object class="sizeritem" expanded="1">
1158<property name="border">5</property>
1158<property name="flag">wxTOP|wxRIGHT|wxLEFT</property>
1158<property name="proportion">0</property>
1158<object class="wxStaticText" expanded="1">
1158<property name="bg"></property>
1158<property name="context_help"></property>
1158<property name="enabled">1</property>
1158<property name="fg"></property>
1158<property name="font"></property>
1158<property name="hidden">1</property>
1159<property name="id">wxID_ANY</property>
1159<property name="label"></property>
1159<property name="maximum_size"></property>
1159<property name="minimum_size"></property>
1159<property name="name">m_staticTextMessage2</property>
1159<property name="permission">protected</property>
1159<property name="pos"></property>
1159<property name="size"></property>
1159<property name="style"></property>
1159<property name="subclass"></property>
1160<property name="tooltip"></property>
1160<property name="window_extra_style"></property>
1160<property name="window_name"></property>
1160<property name="window_style"></property>
1160<property name="wrap">1</property>
1160<event name="OnChar"></event>
1160<event name="OnEnterWindow"></event>
1160<event name="OnEraseBackground"></event>
1160<event name="OnKeyDown"></event>
1160<event name="OnKeyUp"></event>
1161<event name="OnKillFocus"></event>
1161<event name="OnLeaveWindow"></event>
1161<event name="OnLeftDClick"></event>
1161<event name="OnLeftDown"></event>
1161<event name="OnLeftUp"></event>
1161<event name="OnMiddleDClick"></event>
1161<event name="OnMiddleDown"></event>
1161<event name="OnMiddleUp"></event>
1161<event name="OnMotion"></event>
1161<event name="OnMouseEvents"></event>
1162<event name="OnMouseWheel"></event>

```



```

1612<event name="OnPaint"></event>
1612<event name="OnRightClick"></event>
1612<event name="OnRightDown"></event>
1612<event name="OnRightUp"></event>
1612<event name="OnSetFocus"></event>
1612<event name="OnSize"></event>
1612<event name="OnUpdateUI"></event>
1612</object>
1612</object>
1613<object class="sizeritem" expanded="1">
1613<property name="border">5</property>
1613<property
    names="flag">wxALL|wxEXPAND|wxALIGN_CENTER_HORIZONTAL</property>
1613<property name="proportion">0</property>
1613<object class="wxTextCtrl" expanded="1">
1613<property name="bg"></property>
1613<property name="context_help"></property>
1613<property name="enabled">1</property>
1613<property name="fg"></property>
1613<property name="font"></property>
1613<property name="hidden">1</property>
1614<property name="id">wxID_TEXTCTRL</property>
1614<property name="maximum_size"></property>
1614<property name="maxlength">0</property>
1614<property name="minimum_size"></property>
1614<property name="name">m_textCtrl2</property>
1614<property name="permission">protected</property>
1614<property name="pos"></property>
1614<property name="size"></property>
1614<property name="style">wxTE_PROCESS_ENTER</property>
1615<property name="subclass"></property>
1615<property name="tooltip"></property>
1615<property name="value"></property>
1615<property name="window_extra_style"></property>
1615<property name="window_name"></property>
1615<property name="window_style"></property>
1615<event name="OnChar"></event>
1615<event name="OnEnterWindow"></event>
1615<event name="OnEraseBackground"></event>
1615<event name="OnKeyDown"></event>
1616<event name="OnKeyUp"></event>
1616<event name="OnKillFocus"></event>
1616<event name="OnLeaveWindow"></event>
1616<event name="OnLeftClick"></event>
1616<event name="OnLeftDown"></event>
1616<event name="OnLeftUp"></event>
1616<event name="OnMiddleClick"></event>
1616<event name="OnMiddleDown"></event>
1616<event name="OnMiddleUp"></event>
1616<event name="OnMotion"></event>
1617<event name="OnMouseEvents"></event>
1617<event name="OnMouseWheel"></event>
1617<event name="OnPaint"></event>
1617<event name="OnRightClick"></event>
1617<event name="OnRightDown"></event>
1617<event name="OnRightUp"></event>
1617<event name="OnSetFocus"></event>
1617<event name="OnSize"></event>
1617<event name="OnUpdateUI"></event>
1617</object>
1617</object>
1617<object class="sizeritem" expanded="1">
1617<property name="border">5</property>
1617<property names="flag">wxALL</property>
1617<property name="proportion">0</property>
1617<object class="wxButton" expanded="1">
1617<property name="bg"></property>
1617<property name="context_help"></property>
1617<property name="default">0</property>
1617<property name="enabled">1</property>
1617<property name="fg"></property>
1617<property name="font"></property>
1617<property name="hidden">0</property>
1618<property name="id">wxID_CANCEL</property>
1618<property name="label">Cancel</property>
1618<property name="maximum_size"></property>
1618<property name="minimum_size">85,25</property>
1618<property name="name">m_buttonCancel</property>
1618<property name="permission">protected</property>
1618<property name="pos"></property>
1618<property name="size"></property>
1618<property name="style"></property>
1619<property name="subclass"></property>
1619<property name="tooltip"></property>
1619<property name="window_extra_style"></property>
1619<property name="window_name"></property>
1619<event name="OnButtonClick">OnButtonCancel</event>
1619<event name="OnChar"></event>
1619<event name="OnEnterWindow"></event>
1619<event name="OnEraseBackground"></event>
1619<event name="OnKeyDown"></event>
1619<event name="OnKeyUp"></event>
1620<event name="OnKillFocus"></event>
1620<event name="OnLeaveWindow"></event>
1620<event name="OnLeftClick"></event>
1620<event name="OnLeftDown"></event>
1620<event name="OnLeftUp"></event>
1620<event name="OnMiddleClick"></event>
1620<event name="OnMiddleDown"></event>
1620<event name="OnMiddleUp"></event>
1620<event name="OnMotion"></event>
1621<event name="OnMouseEvents"></event>
1621<event name="OnMouseWheel"></event>
1621<event name="OnPaint"></event>
1621<event name="OnRightClick"></event>
1621<event name="OnRightDown"></event>
1621<event name="OnRightUp"></event>
1621<event name="OnSetFocus"></event>
1621<event name="OnSize"></event>
1621<event name="OnUpdateUI"></event>
1621</object>
1621</object>
1621<object class="sizeritem" expanded="1">
1621<property name="border">5</property>
1621<property names="flag">wxALL</property>
1621<property name="proportion">0</property>
1622<object class="wxButton" expanded="1">
1622<property name="bg"></property>
1622<property name="context_help"></property>

```

11826

SUMMARY

Opening `uiproject.fbp` in `wxFormBuilder` today is like archaeological excavation—you can see exactly how Satoshi laid out every pixel of the original Bitcoin experience. The choices made here influenced every wallet that followed: the address display, the send dialog, the transaction list. This XML file is where cryptocurrency user experience began.

util.cpp

374 lines CPP

THE TOOLBOX

Behind every elegant cryptographic system lies a collection of mundane but essential utilities: random number generation, time handling, file operations, string formatting, and debugging. This file is Bitcoin's toolbox—the unsexy infrastructure that makes everything else work.

The most critical function here is random number generation. Your private key's security depends entirely on unpredictable randomness. If an attacker can predict or bias the random number generator, they can steal your coins. Satoshi used OpenSSL's `RAND_bytes()`, which draws entropy from operating system sources (keyboard timing, disk activity, network packets). This code adds additional entropy sources to strengthen the randomness pool.

Also here: `FormatMoney()` and `ParseMoney()`—the functions that convert between human-readable "1.50" BTC and machine-friendly 150000000 satoshis. These handle edge cases like leading zeros and decimal precision that would otherwise cause subtle bugs. Getting money formatting wrong could mean losing funds to rounding errors.

The time synchronization code deserves special attention. Bitcoin blocks include timestamps, and nodes must agree roughly on what time it is. But trusting a single clock source would be a centralization risk. Satoshi's solution: take the median of peer clocks, with bounds to prevent manipulation. This elegant approach—"never go to sea with two chronometers; take one or three"—appears in the time adjustment code below.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5 #include "headers.h"
6
7
8
9 bool fDebug = false;
10
11
12
13
14 // Init openssl library multithreading support
15 static HANDLE* lock_cs;
16
17 void win32_locking_callback(int mode, int type, const char* file, int line)
18 {
19     if (mode & CRYPTO_LOCK)
20         WaitForSingleObject(lock_cs[type], INFINITE);
21     else
22         ReleaseMutex(lock_cs[type]);
23 }
24
25 // Init
26 class CInit

```

`CInit`—the static initialization class. C++ guarantees that global objects are constructed before `main()` runs. Satoshi uses this pattern to initialize OpenSSL's thread safety and seed the random number generator at startup. The `instance_of_cinit` object at line 52 triggers this constructor automatically when Bitcoin launches. This is a common C++ idiom for initialization that must happen exactly once, before any other code runs.

```

27 {
28     public:
29     CInit()
30     {
31         // Init openssl library multithreading support
32         lock_cs = (HANDLE*)OPENSSL_malloc(CRYPTO_num_locks() * sizeof(HANDLE));
33         for (int i = 0; i < CRYPTO_num_locks(); i++)
34             lock_cs[i] = CreateMutex(NULL, FALSE, NULL);
35         CRYPTO_set_locking_callback(win32_locking_callback);
36
37         // Seed random number generator with screen scrape and other hardware sources
38         RAND_screen();

```

`RAND_screen()`—Windows-specific entropy harvesting. This OpenSSL function captures screen pixels, mouse position, and other visual data as randomness sources. The idea: screen content is unpredictable and attacker-unobservable. On non-Windows systems, OpenSSL uses `/dev/urandom` instead. Satoshi's paranoia about entropy was justified—weak RNG caused the 2013 Android Bitcoin wallet vulnerability that drained users' funds.

```

39
40 // Seed random number generator with perfmon data
41 RandAddSeed(true);
42 }
43 ~CInit()
44 {
45     // Shutdown openssl library multithreading support
46     CRYPTO_set_locking_callback(NULL);
47     for (int i = 0; i < CRYPTO_num_locks(); i++)
48         CloseHandle(lock_cs[i]);
49     OPENSSL_free(lock_cs);
50 }

```

```

51 }
52 instance_of_cinit;
53
54
55
56
57 void RandAddSeed(bool fPerfmon)

```

RandAddSeed() —Satoshi's defense-in-depth for randomness. Even though OpenSSL seeds from system entropy, this function adds more: CPU performance counters (nanosecond-precision timing) and Windows performance monitor data (250KB of system metrics). The `fPerfmon` parameter controls whether to harvest the expensive perfmon data—only done every 5 minutes or at startup. Why so paranoid? Private key generation calls this. If an attacker could predict the RNG output, they could reconstruct private keys and steal all coins. The 2010 PlayStation 3 ECDSA key leak happened because Sony reused a 'random' value—Satoshi clearly understood this risk.

```

58 {
59 // Seed with CPU performance counter
60     LARGE_INTEGER PerformanceCount;
61     QueryPerformanceCounter(&PerformanceCount);
62     RAND_add(&PerformanceCount, sizeof(PerformanceCount), 1.5);

```

RAND_add() —feeds entropy with a quality estimate. The third parameter (1.5) tells OpenSSL 'this data contributes about 1.5 bytes of entropy.' Conservative estimates are safer—overestimating entropy quality weakens the RNG. The performance counter has high precision but limited unpredictability, hence the modest estimate.

```

63 memset(&PerformanceCount, 0, sizeof(PerformanceCount));
64
65 static int64 nLastPerfmon;
66 if (fPerfmon || GetTime() > nLastPerfmon + 5 * 60)
67 {
68     nLastPerfmon = GetTime();
69
70 // Seed with the entire set of perfmon data
71 unsigned char pdata[250000];
72 memset(pdata, 0, sizeof(pdata));
73 unsigned long nSize = sizeof(pdata);
74 long ret = RegQueryValueEx(HKEY_PERFORMANCE_DATA, "Global", NULL, NULL, pdata, &nSize);

```

Windows Performance Data—an unusual but clever entropy source. `HKEY_PERFORMANCE_DATA` is a special registry key that returns live system metrics: CPU usage, memory stats, disk activity, network counters. This data is then SHA256-hashed to extract randomness. The 250KB buffer accommodates the substantial data returned. Satoshi adds entropy proportional to data size: `min(nSize/500.0, sizeof(hash))` conservatively estimates 1 byte of entropy per 500 bytes of performance data.

```

75 RegCloseKey(HKEY_PERFORMANCE_DATA);
76 if (ret == ERROR_SUCCESS)
77 {
78     uint256 hash;
79     SHA256(pdata, nSize, (unsigned char*)&hash);
80     RAND_add(&hash, sizeof(hash), min(nSize/500.0, (double)sizeof(hash)));
81     hash = 0;
82     memset(pdata, 0, nSize);
83     printf("RandAddSeed() got %d bytes of performance data\n", nSize);
84 }
85 }
86 }
87
88

```

```

89
90
91
92
93
94
95
96
97 // Safer snprintf

```

Satoshi's original comment explains this function: 'Safer snprintf'. The standard C `snprintf()` has platform-dependent behavior regarding null termination and return values. This wrapper guarantees: (1) output is always null-terminated even when truncated, (2) return value is actual characters written. Buffer overflows have caused countless security vulnerabilities; this defensive coding style appears throughout Bitcoin.

```

98 // - prints up to limit-1 characters
99 // - output string is always null terminated even if limit reached
100 // - return value is the number of characters actually printed
101 int my_snprintf(char* buffer, size_t limit, const char* format, ...)
102 {
103     if (limit == 0)
104         return 0;
105     va_list arg_ptr;
106     va_start(arg_ptr, format);
107     int ret = _vsnprintf(buffer, limit, format, arg_ptr);
108     va_end(arg_ptr);
109     if (ret < 0 || ret >= limit)
110     {
111         ret = limit - 1;
112         buffer[limit-1] = 0;
113     }
114     return ret;
115 }
116
117
118 string strprintf(const char* format, ...)

```

`strprintf()` —dynamic string formatting. Unlike fixed-buffer `snprintf()`, this allocates exactly the memory needed. The `loop` macro (a Satoshi-ism for `for(;;)`) doubles buffer size until the output fits. The 50KB initial buffer handles typical cases without allocation. This function is called throughout Bitcoin for debug logging and error messages.

```

119 {
120     char buffer[50000];
121     char* p = buffer;
122     int limit = sizeof(buffer);
123     int ret;
124     loop
125     {
126         va_list arg_ptr;
127         va_start(arg_ptr, format);
128         ret = _vsnprintf(p, limit, format, arg_ptr);
129         va_end(arg_ptr);
130         if (ret >= 0 && ret < limit)
131             break;
132         if (p != buffer)
133             delete p;
134         limit *= 2;
135         p = new char[limit];
136         if (p == NULL)
137             throw std::bad_alloc();
138     }
139     #ifdef _MSC_VER
140     // msvc optimisation

```

```

141 if (p == buffer)
142     return string(p, p+ret);
143 #endif
144     string str(p, p+ret);
145 if (p != buffer)
146     delete p;
147 return str;
148 }
149
150
151 bool error(const char* format, ...)

```

`error()`—centralized error reporting. Always returns false, allowing patterns like `return error("something failed")`. The error is logged to console. In production code, you'd see these messages in Bitcoin's debug.log file—invaluable for diagnosing issues in the field.

```

152 {
153     char buffer[50000];
154     int limit = sizeof(buffer);
155     va_list arg_ptr;
156     va_start(arg_ptr, format);
157     int ret = _vsnprintf(buffer, limit, format, arg_ptr);
158     va_end(arg_ptr);
159     if (ret < 0 || ret >= limit)
160     {
161         ret = limit - 1;
162         buffer[limit-1] = 0;
163     }
164     printf("ERROR: %s\n", buffer);
165     return false;
166 }
167
168
169 void PrintException(std::exception* pex, const char* pszThread)
170 {
171     char pszModule[260];
172     pszModule[0] = '\0';
173     GetModuleFileName(NULL, pszModule, sizeof(pszModule));
174     _strlwr(pszModule);
175     char pszMessage[1000];
176     if (pex)
177         snprintf(pszMessage, sizeof(pszMessage),
178             "EXCEPTION: %s \n%s \n%s in %s \n", typeid(*pex).name(), pex->what(), pszModule,
179             pszThread);
180     else
181         snprintf(pszMessage, sizeof(pszMessage),
182             "UNKNOWN EXCEPTION \n%s in %s \n", pszModule, pszThread);
183     printf("\n\n*****\n%s", pszMessage);
184     if (wxTheApp)
185         wxMessageBox(pszMessage, "Error", wxOK | wxICON_ERROR);
186     throw;
187     //DebugBreak();
188 }
189
190 void ParseString(const string& str, char c, vector<string>& v)
191 {
192     unsigned int i1 = 0;
193     unsigned int i2;
194     do
195     {
196         i2 = str.find(c, i1);
197         v.push_back(str.substr(i1, i2-i1));
198         i1 = i2+1;
199     }
200     while (i2 != str.npos);
201 }
202
203

```

```
204 string FormatMoney(int64 n, bool fPlus)
```

`FormatMoney()` —converts internal satoshi representation to human-readable format. Bitcoin stores all amounts as integers (satoshis) to avoid floating-point rounding errors. This function divides by `CENT` (1,000,000 satoshis), formats with two decimal places, and inserts thousand separators. The `fPlus` parameter adds '+' prefix for positive values, useful in transaction displays showing debits vs credits. Getting this wrong could cause users to send wrong amounts—the code carefully handles negative values and edge cases.

```
205 {
206     n /= CENT;
```

Division by `CENT`, not `COIN`. Bitcoin has two constants: `COIN` (100,000,000 satoshis = 1 BTC) and `CENT` (1,000,000 satoshis = 0.01 BTC). The division here means `FormatMoney()` displays with 2 decimal places (cents), not 8 (satoshis). This was a v0.1 design choice—early Bitcoin showed values like '50.00 BTC' rather than '50.00000000 BTC'. Modern Bitcoin software often shows more precision.

```
207     string str = sprintf("%I64d.%02I64d", (n > 0 ? n : -n)/100, (n > 0 ? n : -n)%100);
208     for (int i = 6; i < str.size(); i += 4)
209     if (isdigit(str[str.size() - i - 1]))
210         str.insert(str.size() - i, 1, ',');
211     if (n < 0)
212         str.insert((unsigned int)0, 1, '-');
213     else if (fPlus && n > 0)
214         str.insert((unsigned int)0, 1, '+');
215     return str;
216 }
217
218 bool ParseMoney(const char* pszIn, int64& nRet)
```

`ParseMoney()` —the inverse of `FormatMoney()`. Converts user-entered strings like '1,234.56' to satoshis. Handles thousand separators, leading/trailing whitespace, and validates format. The check `strWhole.size() > 17` prevents overflow—17 digits exceeds the 21 million BTC cap. Returns false for malformed input rather than throwing exceptions, following C convention.

```
219 {
220     string strWhole;
221     int64 nCents = 0;
222     const char* p = pszIn;
223     while (isspace(*p))
224         p++;
225     for (; *p; p++)
226     {
227         if (*p == ',' && p > pszIn && isdigit(p[-1]) && isdigit(p[1]) && isdigit(p[2]) && isdigit(p[3]) &&
            !isdigit(p[4]))
228             continue;
229         if (*p == '.')
230         {
231             p++;
232             if (!isdigit(p[0]) || !isdigit(p[1]))
233                 return false;
234             nCents = atoi64(p);
235             if (nCents < 0 || nCents > 99)
236                 return false;
237             p += 2;
238             break;
239         }
240         if (isspace(*p))
241             break;
242         if (!isdigit(*p))
243             return false;
```

```

244         strWhole.insert(strWhole.end(), *p);
245     }
246     for (; *p; p++)
247         if (!isspace(*p))
248             return false;
249     if (strWhole.size() > 17)
250         return false;
251     int64 nWhole = atoi64(strWhole);
252     int64 nValue = nWhole * 100 + nCents;
253     if (nValue / 100 != nWhole)

```

Overflow check: `nValue / CENT != nWhole`. After multiplying, division should recover the original. If not, multiplication overflowed. This defensive check prevents accepting astronomically large invalid amounts that wrapped around due to integer overflow.

```

254     return false;
255     nValue *= CENT;
256     nRet = nValue;
257     return true;
258 }
259
260
261
262
263
264
265
266
267
268
269 bool FileExists(const char* psz)
270 {
271     #ifdef WIN32
272     return GetFileAttributes(psz) != -1;
273     #else
274     return access(psz, 0) != -1;
275     #endif
276 }
277
278 int GetFileSize(FILE* file)
279 {
280     int nSavePos = ftell(file);
281     int nFileSize = -1;
282     if (fseek(file, 0, SEEK_END) == 0)
283         nFileSize = ftell(file);
284     fseek(file, nSavePos, SEEK_SET);
285     return nFileSize;
286 }
287
288
289
290
291
292
293
294
295 uint64 GetRand(uint64 nMax)

```

`GetRand()` —generates cryptographically secure random numbers. This is the function called when generating private keys, selecting peers, and other security-critical operations. The `do-while` loop implements rejection sampling: since `RAND_bytes()` produces uniform random bits, but we want uniform distribution over `[0, nMax)`, we must reject values that would bias the result. The calculation `nRange = (_UI64_MAX / nMax) * nMax` finds the largest multiple of `nMax` that fits in 64 bits. Values `>= nRange` would favor lower results if taken mod `nMax`, so they're rejected and we try again.

```

296 {
297     if (nMax == 0)
298         return 0;
299
300     // The range of the random source must be a multiple of the modulus
301     // to give every possible output value an equal possibility
302     uint64 nRange = (_UI64_MAX / nMax) * nMax;
303     uint64 nRand = 0;
304     do
305         RAND_bytes((unsigned char*)&nRand, sizeof(nRand));
306     while (nRand >= nRange);
307     return (nRand % nMax);
308 }
309
310
311
312
313
314
315
316
317
318
319 //
```

The chronometers quote—Satoshi's comment reveals his thinking on distributed time. The saying 'never go to sea with two chronometers; take one or three' means: with one clock, you trust it; with two disagreeing clocks, you don't know which is right; with three, you take the majority. Bitcoin implements this philosophy: rather than trusting any single time source, it takes the median of connected peers' clocks (adjusted for network latency). This prevents a single malicious peer from manipulating timestamps, which could affect block validity checking.

```

320 // "Never go to sea with two chronometers; take one or three."
321 // Our three chronometers are:
322 // - System clock
323 // - Median of other server's clocks
324 // - NTP servers
325 //
326 // note: NTP isn't implemented yet, so until then we just use the median
327 // of other nodes clocks to correct ours.
328 //
329
330 int64 GetTime()
331 {
332     return time(NULL);
333 }
334
335 static int64 nTimeOffset = 0;
336
337 int64 GetAdjustedTime()
```

`GetAdjustedTime()` —returns current time adjusted by network consensus. `nTimeOffset` is the difference between local clock and peer median. If your clock is 5 minutes fast, `nTimeOffset` is -300 seconds. Blocks are rejected if their timestamp is too far in the future—this adjustment prevents your node from rejecting valid blocks due to local clock drift.

```

338 {
339     return GetTime() + nTimeOffset;
340 }
341
342 void AddTimeData(unsigned int ip, int64 nTime)
    
```

`AddTimeData()` —collects time samples from peers. When connecting to a peer, nodes exchange timestamps. This function tracks offsets (`peer_time - local_time`), deduplicates by IP, and computes the median. The logic waits for 5+ samples and only updates on odd counts (ensuring a true median exists). If median offset exceeds 5 minutes, the code notes 'go to NTP servers'—though NTP wasn't actually implemented in v0.1. The 5-minute threshold prevents attackers from gradually skewing network time.

```

343 {
344     int64 nOffsetSample = nTime - GetTime();
345
346     // Ignore duplicates
347     static set<unsigned int> setKnown;
348     if (!setKnown.insert(ip).second)
349         return;
350
351     // Add data
352     static vector<int64> vTimeOffsets;
353     if (vTimeOffsets.empty())
354         vTimeOffsets.push_back(0);
355     vTimeOffsets.push_back(nOffsetSample);
356     printf("Added time data, samples %d, ip %08x, offset %+I64d (%+I64d minutes)\n", vTimeOffsets.size(),
357           ip, vTimeOffsets.back(), vTimeOffsets.back()/60);
358     if (vTimeOffsets.size() >= 5 && vTimeOffsets.size() % 2 == 1)
359     {
360         sort(vTimeOffsets.begin(), vTimeOffsets.end());
361         int64 nMedian = vTimeOffsets[vTimeOffsets.size()/2];
362         nTimeOffset = nMedian;
363     }
364     if ((nMedian > 0 ? nMedian : -nMedian) > 5 * 60)
    
```

Safety limit: offsets beyond ± 5 minutes trigger a TODO comment about NTP. Satoshi recognized the vulnerability: if an attacker controlled many peer connections, they could manipulate your time offset. The 5-minute cap limits damage. Later Bitcoin versions added stricter checks and warnings when clock offset is large.

```

363 {
364     // Only let other nodes change our clock so far before we
365     // go to the NTP servers
366     /// todo: Get time from NTP servers, then set a flag
367     /// to make sure it doesn't get changed again
368 }
369 foreach(int64 n, vTimeOffsets)
370     printf("%+I64d ", n);
371 printf("| nTimeOffset = %+I64d (%+I64d minutes)\n", nTimeOffset, nTimeOffset/60);
372 }
373 }
374
    
```

SUMMARY

These mundane-seeming functions are critical infrastructure. The random number generator secures every private key ever generated. The money formatting functions prevent users from accidentally sending wrong amounts. The time synchronization enables consensus without a central time authority.

Satoshi's paranoid approach to randomness proved prescient. In 2013, a weak RNG in Android's Java `SecureRandom` class allowed attackers to derive private keys from signed transactions, draining wallets. In 2010, Sony's PlayStation 3 signing key was compromised because they used a static 'random' value in ECDSA signatures. Bitcoin's defense-in-depth—multiple entropy sources, conservative estimates, rejection sampling—has prevented similar disasters.

The time synchronization elegantly solves a distributed systems problem: agreeing on time without trusting any party. By taking medians with bounds, Bitcoin achieves rough consensus sufficient for block timestamps without introducing centralization through NTP servers or trusted time sources.

util.h

400 lines CPP

THE SWISS ARMY KNIFE

Declares utility functions, macros, and global variables used throughout Bitcoin. Debug logging, thread synchronization, and configuration settings are defined here.

```

1 // Copyright (c) 2009 Satoshi Nakamoto
2 // Distributed under the MIT/X11 software license, see the accompanying
3 // file license.txt or http://www.opensource.org/licenses/mit-license.php.
4
5
6 #if defined(_MSC_VER) || defined(__BORLANDC__)
7 typedef __int64 int64;
8 typedef unsigned __int64 uint64;
9 #else
10 typedef long long int64;
11 typedef unsigned long long uint64;
12 #endif
13 #if defined(_MSC_VER) && _MSC_VER < 1300
14 #define for if (false) ; else for
15 #endif
16
17 #ifndef _MSC_VER
18 #define __forceinline inline
19 #endif
20
21 #define foreach BOOST_FOREACH

```

foreach macro - convenient iteration over collections

```

22 #define loop for (;;)

```

loop macro - infinite loop shorthand

```

23 #define BEGIN(a) ((char*)&(a))
24 #define END(a) ((char*)&((a))[1])
25 #define UBEGIN(a) ((unsigned char*)&(a))
26 #define UEND(a) ((unsigned char*)&((a))[1])
27 #define ARRAYLEN(array) (sizeof(array)/sizeof(array[0]))
28
29 #ifdef _WINDOWS
30 #define printf OutputDebugStringF
31 #endif
32
33 #ifdef snprintf
34 #undef snprintf
35 #endif
36 #define snprintf my_snprintf
37
38 #ifndef PRIu64
39 #if defined(_MSC_VER) || defined(__BORLANDC__) || defined(__MSVCRT__)
40 #define PRIu64 "I64u"
41 #define PRIu64 "I64u"
42 #define PRIx64 "I64x"
43 #else
44 #define PRIu64 "lld"
45 #define PRIu64 "llu"
46 #define PRIx64 "llx"
47 #endif
48 #endif
49
50 // This is needed because the foreach macro can't get over the comma in pair<t1, t2>
51 #define PAIRTYPE(t1, t2) pair<t1, t2>
52
53 // Used to bypass the rule against non-const reference to temporary
54 // where it makes sense with wrappers such as CFlatData or CTxDB
55 template<typename T>
56 inline T& REF(const T& val)
57 {
58     return (T&)val;
59 }
60
61

```

```

62
63
64
65
66
67
68
69 extern bool fDebug;
70
71 void RandAddSeed(bool fPerfmon=false);
72 int my_snprintf(char* buffer, size_t limit, const char* format, ...);
73 string sprintf(const char* format, ...);
74 bool error(const char* format, ...);
75 void PrintException(std::exception* pex, const char* pszThread);
76 void ParseString(const string& str, char c, vector<string>& v);
77 string FormatMoney(int64 n, bool fPlus=false);
78 bool ParseMoney(const char* pszIn, int64& nRet);
79 bool FileExists(const char* psz);
80 int GetFileSize(FILE* file);
81 uint64 GetRand(uint64 nMax);
82 int64 GetTime();
83 int64 GetAdjustedTime();
84 void AddTimeData(unsigned int ip, int64 nTime);
85
86
87
88
89
90
91
92
93
94
95
96
97 // Wrapper to automatically initialize critical section
98 // Could use wxCriticalSection for portability, but it doesn't support TryEnterCriticalSection
99 class CCriticalSection

```

CCriticalSection - thread-safe locking primitive

```

100 {
101 protected:
102     CRITICAL_SECTION cs;
103 public:
104     char* pszFile;
105     int nLine;
106     explicit CCriticalSection() { InitializeCriticalSection(&cs); }
107     ~CCriticalSection() { DeleteCriticalSection(&cs); }
108     void Enter() { EnterCriticalSection(&cs); }
109     void Leave() { LeaveCriticalSection(&cs); }
110     bool TryEnter() { return TryEnterCriticalSection(&cs); }
111     CRITICAL_SECTION* operator&() { return &cs; }
112 };
113
114 // Automatically leave critical section when leaving block, needed for exception safety
115 class CCriticalBlock
116 {
117 protected:
118     CRITICAL_SECTION* pcs;
119 public:
120     CCriticalBlock(CRITICAL_SECTION& csIn) { pcs = &csIn; EnterCriticalSection(pcs); }
121     CCriticalBlock(CCriticalSection& csIn) { pcs = &csIn; EnterCriticalSection(pcs); }
122     ~CCriticalBlock() { LeaveCriticalSection(pcs); }
123 };
124
125 // WARNING: This will catch continue and break!
126 // break is caught with an assertion, but there's no way to detect continue.
127 // I'd rather be careful than suffer the other more error prone syntax.

```

```

128 // The compiler will optimise away all this loop junk.
129 #define CRITICAL_BLOCK(cs)\

```

CRITICAL_BLOCK - protects code from concurrent access

```

130 for (bool fcriticalblockonce=true; fcriticalblockonce; assert(("break caught by CRITICAL_BLOCK!",
!fcriticalblockonce)), fcriticalblockonce=false) \
131 for (CCriticalBlock criticalblock(cs); fcriticalblockonce && (cs.pszFile=__FILE__, cs.nLine=__LINE__,
true); fcriticalblockonce=false, cs.pszFile=NULL, cs.nLine=0)
132
133 class CTryCriticalSection
134 {
135 protected:
136     CRITICAL_SECTION* pcs;
137 public:
138     CTryCriticalSection(CRITICAL_SECTION& csIn) { pcs = (TryEnterCriticalSection(&csIn) ? &csIn : NULL); }
139     CTryCriticalSection(CCRITICAL_SECTION& csIn) { pcs = (TryEnterCriticalSection(&csIn) ? &csIn : NULL); }
140     ~CTryCriticalSection() { if (pcs) LeaveCriticalSection(pcs); }
141     bool Entered() { return pcs != NULL; }
142 };
143
144 #define TRY_CRITICAL_BLOCK(cs)\
145 for (bool fcriticalblockonce=true; fcriticalblockonce; assert(("break caught by TRY_CRITICAL_BLOCK!",
!fcriticalblockonce)), fcriticalblockonce=false) \
146 for (CTryCriticalSection criticalblock(cs); fcriticalblockonce && (fcriticalblockonce =
criticalblock.Entered()) && (cs.pszFile=__FILE__, cs.nLine=__LINE__, true); fcriticalblockonce=false,
cs.pszFile=NULL, cs.nLine=0)
147
148
149
150
151
152
153
154
155
156
157
158
159 inline string i64tostr(int64 n)
160 {
161     return sprintf("%"PRI64, n);
162 }
163
164 inline string itostr(int n)
165 {
166     return sprintf("%d", n);
167 }
168
169 inline int64 atoi64(const char* psz)
170 {
171     #ifdef _MSC_VER
172     return _atoi64(psz);
173     #else
174     return strtoll(psz, NULL, 10);
175     #endif
176 }
177
178 inline int64 atoi64(const string& str)
179 {
180     #ifdef _MSC_VER
181     return _atoi64(str.c_str());
182     #else
183     return strtoll(str.c_str(), NULL, 10);
184     #endif
185 }
186
187 inline int atoi(const string& str)
188 {

```

```

189 return atoi(str.c_str());
190 }
191
192 inline int roundint(double d)
193 {
194     return (int)(d > 0 ? d + 0.5 : d - 0.5);
195 }
196
197 template<typename T>
198 string HexStr(const T itbegin, const T itend, bool fSpaces=true)
199 {
200     const unsigned char* pbegin = (const unsigned char*)&itbegin[0];
201     const unsigned char* pend = pbegin + (itend - itbegin) * sizeof(itbegin[0]);
202     string str;
203     for (const unsigned char* p = pbegin; p != pend; p++)
204         str += sprintf((fSpaces && p != pend-1 ? "%02x " : "%02x"), *p);
205     return str;
206 }
207
208 template<typename T>
209 string HexNumStr(const T itbegin, const T itend, bool f0x=true)
210 {
211     const unsigned char* pbegin = (const unsigned char*)&itbegin[0];
212     const unsigned char* pend = pbegin + (itend - itbegin) * sizeof(itbegin[0]);
213     string str = (f0x ? "0x" : "");
214     for (const unsigned char* p = pend-1; p >= pbegin; p--)
215         str += sprintf("%02X", *p);
216     return str;
217 }
218
219 template<typename T>
220 void PrintHex(const T pbegin, const T pend, const char* pszFormat="%s", bool fSpaces=true)
221 {
222     printf(pszFormat, HexStr(pbegin, pend, fSpaces).c_str());
223 }
224
225
226
227
228
229
230
231
232 inline int OutputDebugStringF(const char* pszFormat, ...)
233 {
234     #ifdef __WXDEBUG__
235         // log file
236         FILE* fileout = fopen("debug.log", "a");
237         if (fileout)
238         {
239             va_list arg_ptr;
240             va_start(arg_ptr, pszFormat);
241             vfprintf(fileout, pszFormat, arg_ptr);
242             va_end(arg_ptr);
243             fclose(fileout);
244         }
245
246         // accumulate a line at a time
247         static CCriticalSection cs_OutputDebugStringF;
248         CRITICAL_BLOCK(cs_OutputDebugStringF)
249         {
250             static char pszBuffer[50000];
251             static char* pend;
252             if (pend == NULL)
253                 pend = pszBuffer;
254             va_list arg_ptr;
255             va_start(arg_ptr, pszFormat);
256             int limit = END(pszBuffer) - pend - 2;
257             int ret = _vsprintf(pend, limit, pszFormat, arg_ptr);
258             va_end(arg_ptr);
259             if (ret < 0 || ret >= limit)
260             {

```

```

261         pend = END(pszBuffer) - 2;
262 *pend++ = '\n';
263 }
264 else
265     pend += ret;
266 *pend = '\0';
267 char* p1 = pszBuffer;
268 char* p2;
269 while (p2 = strchr(p1, '\n'))
270 {
271     p2++;
272     char c = *p2;
273     *p2 = '\0';
274     OutputDebugString(p1);
275     *p2 = c;
276     p1 = p2;
277 }
278 if (p1 != pszBuffer)
279     memmove(pszBuffer, p1, pend - p1 + 1);
280     pend -= (p1 - pszBuffer);
281 return ret;
282 }
283 #endif
284
285 if (!wxTheApp)
286 {
287     // print to console
288     va_list arg_ptr;
289     va_start(arg_ptr, pszFormat);
290     vprintf(pszFormat, arg_ptr);
291     va_end(arg_ptr);
292 }
293 return 0;
294 }
295
296
297
298
299
300
301
302
303
304 inline void heapchk()
305 {
306     if (_heapchk() != _HEAPOK)
307         DebugBreak();
308 }
309
310 // Randomize the stack to help protect against buffer overrun exploits
311 #define IMPLEMENT_RANDOMIZE_STACK(ThreadFn)\
312 {\
313     static char nLoops;\
314     if (nLoops <= 0)\
315         nLoops = GetRand(50) + 1;\
316     if (nLoops-- > 1)\
317     {\
318         ThreadFn;\
319     }\
320 }\
321 }
322
323 #define CATCH_PRINT_EXCEPTION(pszFn)\
324 catch (std::exception& e) {\
325     PrintException(&e, (pszFn));\
326 } catch (...) {\
327     PrintException(NULL, (pszFn));\
328 }\
329
330
331
332

```



```

333
334
335
336
337
338 template<typename T1>
339 inline uint256 Hash(const T1 pbegin, const T1 pend)
340 {
341     uint256 hash1;
342     SHA256((unsigned char*)&pbegin[0], (pend - pbegin) * sizeof(pbegin[0]), (unsigned char*)&hash1);
343     uint256 hash2;
344     SHA256((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
345     return hash2;
346 }
347
348 template<typename T1, typename T2>
349 inline uint256 Hash(const T1 p1begin, const T1 p1end,
350 const T2 p2begin, const T2 p2end)
351 {
352     uint256 hash1;
353     SHA256_CTX ctx;
354     SHA256_Init(&ctx);
355     SHA256_Update(&ctx, (unsigned char*)&p1begin[0], (p1end - p1begin) * sizeof(p1begin[0]));
356     SHA256_Update(&ctx, (unsigned char*)&p2begin[0], (p2end - p2begin) * sizeof(p2begin[0]));
357     SHA256_Final((unsigned char*)&hash1, &ctx);
358     uint256 hash2;
359     SHA256((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
360     return hash2;
361 }
362
363 template<typename T1, typename T2, typename T3>
364 inline uint256 Hash(const T1 p1begin, const T1 p1end,
365 const T2 p2begin, const T2 p2end,
366 const T3 p3begin, const T3 p3end)
367 {
368     uint256 hash1;
369     SHA256_CTX ctx;
370     SHA256_Init(&ctx);
371     SHA256_Update(&ctx, (unsigned char*)&p1begin[0], (p1end - p1begin) * sizeof(p1begin[0]));
372     SHA256_Update(&ctx, (unsigned char*)&p2begin[0], (p2end - p2begin) * sizeof(p2begin[0]));
373     SHA256_Update(&ctx, (unsigned char*)&p3begin[0], (p3end - p3begin) * sizeof(p3begin[0]));
374     SHA256_Final((unsigned char*)&hash1, &ctx);
375     uint256 hash2;
376     SHA256((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
377     return hash2;
378 }
379
380 template<typename T>
381 uint256 SerializeHash(const T& obj, int nType=SER_GETHASH, int nVersion=VERSION)
382 {
383     // Most of the time is spent allocating and deallocating CDataStream's
384     // buffer. If this ever needs to be optimized further, make a CStaticStream
385     // class with its buffer on the stack.
386     CDataStream ss(nType, nVersion);
387     ss.reserve(10000);
388     ss << obj;
389     return Hash(ss.begin(), ss.end());
390 }
391
392 inline uint160 Hash160(const vector<unsigned char>& vch)
393 {
394     uint256 hash1;
395     SHA256(&vch[0], vch.size(), (unsigned char*)&hash1);
396     uint160 hash2;
397     RIPEMD160((unsigned char*)&hash1, sizeof(hash1), (unsigned char*)&hash2);
398     return hash2;
399 }
400

```

SUMMARY

The CRITICAL_BLOCK macro for thread locking and the printf-style debug logging are patterns you'll see used in every other file.

WHAT CAME AFTER

You've now studied Bitcoin at its origin—the pristine v0.01 code that launched a monetary revolution. But Bitcoin didn't stop evolving on January 9, 2009. Over seventeen years, the codebase has grown from ~15,000 lines to over 200,000, with hundreds of contributors refining Satoshi's vision. Here's a chronological guide to what came next, so you know which versions to study for specific improvements.

THE SATOSHI ERA (2009–2011)

Satoshi personally maintained Bitcoin for its first two years, releasing incremental improvements while communicating with early adopters on forums and mailing lists.

v0.1.0	Jan 9, 2009	First public release (this book). Windows only. IRC-based peer discovery.
v0.1.5	Feb 4, 2009	Bug fixes. First transaction between Satoshi and Hal Finney (Jan 12).
v0.2.0	Dec 16, 2009	Linux support. Command-line daemon mode. Numerous optimizations.
v0.3.0	Jul 6, 2010	Mac OS X support. JSON-RPC API begins. Safer database writes.
v0.3.1	Jul 15, 2010	1 MB block size limit added by Satoshi as anti-spam measure. Originally temporary, became contentious years later.
v0.3.6	Aug 15, 2010	Testnet introduced for development testing without real value.
v0.3.9	Aug 16, 2010	OP_RETURN disabled after discovery of a critical script bug allowing anyone to spend any coins. Satoshi's emergency soft fork.
v0.3.10	Aug 17, 2010	Value overflow fix. Block 74638 exploited integer overflow to create 184 billion BTC. Chain reorganized to remove it.
v0.3.21	Apr 27, 2011	Satoshi's final release. He disappeared shortly after, leaving Bitcoin to the community.

Study these versions if: You want to trace Satoshi's thinking as he responded to real-world usage. The JSON-RPC API (v0.3.0+) enabled the first exchanges and services. The testnet (v0.3.6) is still used today.

EARLY COMMUNITY DEVELOPMENT (2011–2013)

After Satoshi's departure, Gavin Andresen became lead maintainer. The project moved to GitHub, adopted a more formal release process, and began professionalizing.

v0.4.0	Sep 23, 2011	Encrypted wallets. AES-256-CBC encryption protects private keys.
v0.5.0	Nov 21, 2011	New Qt GUI. Better address book. "Bitcoin-Qt" naming begins.
v0.6.0	Mar 30, 2012	Bloom filters (BIP 37) for lightweight SPV clients.
v0.7.0	Sep 17, 2012	Reduced memory usage. Better fee handling. Prepare for BIP 16 (P2SH).
v0.8.0	Feb 18, 2013	LevelDB replaces Berkeley DB for UTXO set. 10x sync speed improvement.
v0.8.1	Mar 18, 2013	Chain split fix. BerkeleyDB lock limit caused v0.7 nodes to reject valid blocks that v0.8 accepted. 6-hour fork resolved by emergency downgrade.

Study these versions if: You're interested in wallet encryption (v0.4), SPV/light clients (v0.6), or the database architecture (v0.8). The March 2013 chain split is a critical lesson in consensus-critical code changes—even "non-consensus" changes can break consensus.

MATURATION & REBRANDING (2014–2016)

The project rebranded to "Bitcoin Core" to distinguish it from the broader Bitcoin ecosystem. Development became more rigorous, with extensive review processes and a focus on security.

v0.9.0	Mar 19, 2014	Rebranded to "Bitcoin Core". OP_RETURN outputs allowed (40-byte data limit). Payment protocol (BIP 70).
v0.10.0	Feb 16, 2015	Headers-first sync. Download block headers before blocks for faster IBD.
v0.11.0	Jul 12, 2015	Block pruning. libsecp256k1 replaces OpenSSL for ECDSA—7x faster. OP_RETURN limit raised to 80 bytes.
v0.12.0	Feb 23, 2016	Opt-in RBF (Replace-By-Fee). Memory pool limiting. Faster

signature validation.

v0.13.0	Aug 23, 2016	SegWit preparation (BIPs 141, 143, 144). Compact blocks (BIP 152).
----------------	--------------	---

Study these versions if: You want to understand modern sync strategies (v0.10), the libsecp256k1 library (v0.11), or prepare for SegWit. The shift from OpenSSL to libsecp256k1 is a masterclass in security-focused refactoring.

SEGWIT & SCALING DEBATES (2017–2019)

Segregated Witness (SegWit) was the most significant protocol upgrade since the beginning. It fixed transaction malleability, enabled the Lightning Network, and increased effective block capacity—all through a soft fork.

v0.14.0	Mar 8, 2017	Performance optimizations. Manual pruning. Improved fee estimation.
v0.15.0	Sep 14, 2017	Better fee estimation. Multi-wallet support. Script caching.
v0.16.0	Feb 26, 2018	Full SegWit wallet support. Native bech32 addresses (bc1...).
v0.17.0	Oct 3, 2018	Partial Spend Avoidance. Branch and Bound coin selection. Scantxoutset RPC.
v0.18.0	May 2, 2019	Output descriptors. Hardware wallet support via HWI.
v0.19.0	Nov 24, 2019	BIP 158 block filters (Neutrino). CPFP carve-out for Lightning.

Study these versions if: You want to understand SegWit implementation (v0.13–v0.16), coin selection algorithms (v0.17), or output descriptors (v0.18). The descriptor wallet model fundamentally changed how Bitcoin Core manages keys.

MODERN BITCOIN CORE (2020–PRESENT)

Recent development has focused on Taproot (the largest upgrade since SegWit), privacy improvements, and hardening against sophisticated attacks.

v0.20.0	Jun 3, 2020	ASMap for BGP attack resistance. Remove BIP 70 payment protocol.
v0.21.0	Jan 14, 2021	Descriptor wallets default. Tor V3 support. Signet test network.

v23.0	Apr 25, 2022	Taproot multisig (MuSig). CJDNS network support. Tracepoints for debugging.
v24.0	Nov 25, 2022	Full RBF option (mempoolfullrbf). Watch-only descriptor wallets.
v25.0	May 26, 2023	Miniscript for complex spending conditions. Improved coin selection.
v26.0	Dec 6, 2023	V2 transport protocol (BIP 324). Encrypted P2P connections.
v27.0	Apr 17, 2024	libbitcoinkernel progress. Improved assumeUTXO. Fee estimation improvements.
v28.0	Oct 2, 2024	Testnet4. Package relay groundwork (1P1C). Full TRUC/V3 transaction support.

Study these versions if: You want to understand Schnorr signatures and Taproot (v22), Miniscript (v25), or encrypted P2P (v26). The libbitcoinkernel project (ongoing) is modularizing the consensus code for use by other implementations.

NOTABLE BUGS & INCIDENTS

Bitcoin's history includes several critical bugs that tested the network's resilience. Studying these incidents reveals how consensus systems fail and recover:

Aug 2010	OP_RETURN Bug	<i>Introduced by: Satoshi (v0.1)</i> — A flaw in script evaluation let anyone spend anyone's coins using OP_RETURN. Present since day one but undiscovered. Satoshi disabled multiple opcodes via emergency soft fork within hours of disclosure.
Aug 2010	Value Overflow	<i>Introduced by: Satoshi (v0.1)</i> — Integer overflow created 184 billion BTC in block 74638. Missing overflow check in transaction validation. Community coordinated to orphan the block within 5 hours.
Mar 2013	BDB Lock Crisis	<i>Introduced by: Pieter Wuille (v0.8)</i> — LevelDB migration inadvertently changed consensus behavior. BerkeleyDB's lock limits caused v0.7 nodes to reject blocks v0.8 accepted. 6-hour chain split resolved by emergency downgrade.
Jul 2015	BIP 66 Fork	<i>Introduced by: miners (SPV mining)</i> — Miners signaling BIP 66 (strict DER) weren't actually validating blocks. Brief fork when an invalid block was extended. Exposed risks of mining on headers without full validation.
Sep 2018	CVE-2018-17144	<i>Introduced by: Core developers (v0.14)</i> — Optimization to skip duplicate input checking inadvertently allowed double-spends within a single block. Silently patched in v0.16.3. Most critical vulnerability since 2010.

Key lessons: (1) Even Satoshi's original code contained critical bugs—peer review is essential. (2) Consensus code changes require extreme caution—even database backends can affect consensus. (3) Quick community response is crucial. (4) Silent patching of critical bugs is sometimes necessary to prevent exploitation before nodes upgrade.

KEY ARCHITECTURAL CHANGES

Looking at the big picture, these are the most significant architectural departures from the v0.01 code you just studied:

COMPONENT	v0.01	MODERN BITCOIN CORE
Cryptography	OpenSSL (ECDSA)	libsecp256k1, Schnorr signatures
Database	Berkeley DB	LevelDB (UTXO), SQLite (wallet)
Peer Discovery	IRC channels	DNS seeds, addr gossip
Script	Basic opcodes	P2SH, SegWit, Tapscript
Addresses	1... (P2PKH only)	3... (P2SH), bc1q... (SegWit), bc1p... (Taproot)
Wallet	Loose key collection	HD wallets, descriptors
GUI	wxWidgets	Qt Framework
Build System	Makefiles (hand-written)	CMake (as of v28)
Code Size	~15,000 lines	~200,000+ lines

RECOMMENDED STUDY PATH

Having mastered v0.01, here's a suggested progression based on your interests:

- **Consensus Rules:** v0.3.10 (overflow fix) → v0.8.1 (DB fork) → v22.0 (Taproot)
- **Cryptography:** v0.11.0 (libsecp256k1) → v22.0 (Schnorr/Taproot)
- **Wallet Development:** v0.4.0 (encryption) → v0.18.0 (descriptors) → v0.21.0 (descriptor default)
- **Networking:** v0.2.0 (daemon) → v0.6.0 (Bloom) → v26.0 (encrypted P2P)

- **Script/Smart Contracts:** v0.6.0 (P2SH) → v0.13.0 (SegWit) → v22.0 (Tapscript) → v25.0 (Miniscript)
- **Lightning Prerequisites:** v0.13.0 (SegWit) → v0.19.0 (CPFP carve-out) → v24.0 (full RBF)
- **Security & Bugs:** v0.3.9 (OP_RETURN) → v0.3.10 (overflow) → v0.8.1 (BDB) → v0.16.3 (CVE-2018-17144)

RESOURCES FOR CONTINUED STUDY

- **Bitcoin Core GitHub:** github.com/bitcoin/bitcoin — Full commit history since 2009
- **Bitcoin Improvement Proposals:** github.com/bitcoin/bips — All protocol specifications
- **Bitcoin Optech:** bitcoinops.org — Weekly technical newsletter since 2018
- **Bitcoin Stack Exchange:** bitcoin.stackexchange.com — Q&A for technical questions
- **Chaincode Labs Seminars:** chaincode.com — Free courses on Bitcoin/Lightning development
- **Learning Bitcoin from the Command Line:** github.com/BlockchainCommons — Hands-on tutorials

The code you've studied in this book is where it all began. Every feature, every optimization, every security fix in modern Bitcoin Core traces back to these 15,000 lines. Satoshi wrote: "I'm sure that in 20 years there will either be very large transaction volume or no volume." Seventeen years in, the code keeps evolving—and now you know where to look.

INDEX

A

AcceptBlock, 119, 165
AcceptTransaction, 119, 165, 315
Adam Back, 19, 119
addr, 75, 87, 103, 119, 203, 207, 225, 315, 363
AddToBlockIndex, 119, 165

B

b-money, 19, 119, 309
base58.h, 55, 99, 189, 193
Berkeley DB, 515, 75, 87, 189, 243
Bit Gold, 19, 119
BitcoinMiner, 119, 165, 207
blk0001.dat, 165
blkindex.dat, 87, 315
block, 63, 75, 87, 119, 165, 207, 225, 265, 277, 299, 309, 315, 419, 497, 507
blockchain, 75, 87, 119, 165, 207, 277, 315, 363

C

CAddress, 75, 87, 103, 119, 165, 203, 207, 225, 315, 363
CBigNum, 39, 55, 63, 119, 165, 247, 265
CBlock, 39, 119, 165
CBlockIndex, 75, 119, 165, 315, 363
CDataStream, 75, 87, 119, 165, 207, 225, 247, 277, 315, 363, 507
CheckBlock, 119, 165
CheckTransaction, 119, 165
CInv, 119, 207, 225, 315
CKey, 39, 111, 119, 165, 197, 247
CNode, 119, 165, 207, 225, 315
coinbase, 55, 87, 119, 165, 315
ConnectBlock, 119, 165
consensus, 15, 75, 119, 165, 265, 277, 309, 419, 497
COutPoint, 75, 87, 119, 165, 315
CreateTransaction, 119, 165, 315
CScript, 119, 165, 247, 265, 277, 315
CTransaction, 39, 75, 87, 119, 165, 247, 265
CTxIn, 119, 165, 247, 315
CTxOut, 119, 165, 247, 315
cypherpunk, 19

D

David Chaum, 19
db.cpp, 75, 189, 193
db.h, 87, 99, 189, 193
DecodeBase58, 55
difficulty, 19, 63, 87, 119, 165, 419
DigiCash, 19, 119
digital signature, 19, 29, 111, 119, 165, 197, 247, 265
discrete logarithm, 29, 111
double-spend, 15, 119

E

ECDSA, 25, 29, 63, 99, 111, 247, 497
elliptic curve, 29, 63, 111
EncodeBase58, 55
EvalScript, 247, 265

M

main.cpp, 15, 119, 165, 189, 193
main.h, 99, 165, 189, 193
 mempool, 87, 119
 merkle tree, 19, 29, 75, 119, 165
 message, 75, 111, 119, 207, 225, 315, 373, 437

N

net.cpp, 15, 189, 193, 207
net.h, 99, 189, 193, 225
 Nick Szabo, 19
 node, 15, 87, 103, 109, 111, 119, 165, 207, 225, 265, 315, 497
 nonce, 111, 119, 165, 309

O

OP_CHECKSIG, 119, 247, 265, 315
OP_DUP, 247, 265, 315
OP_EQUALVERIFY, 247, 265, 315
OP_HASH160, 247, 265, 315
 OpenSSL, 29, 63, 99, 111, 119, 189, 193, 243, 373, 437, 497
 orphan block, 119

P

peer-to-peer, 15, 103, 119, 165, 203, 207, 225
 private key, 25, 29, 55, 87, 111, 119, 265, 309, 419, 497
ProcessBlock, 119
 proof-of-work, 15, 19, 29, 63, 119, 165, 299, 309, 419, 437
 public key, 25, 29, 55, 87, 111, 165, 247, 265, 309, 315

R

RIPEMD-160, 19, 29, 55, 99, 247, 265, 309, 419, 507
 RPOW, 19

S

satoshi, 15, 515, 55, 63, 75, 87, 99, 103, 109, 111, 117, 119, 165, 189, 193, 197, 203, 207, 225, 243, 247, 265, 277, 309, 315, 363, 373, 407, 419, 437, 497, 507
 Satoshi Nakamoto, 15, 55, 63, 75, 87, 99, 103, 109, 111, 117, 119, 165, 189, 193, 197, 203, 207, 225, 243, 247, 265, 277, 315, 363, 373, 407, 419, 437, 497, 507
script.cpp, 15, 189, 193, 247, 265
script.h, 99, 189, 193, 265
scriptPubKey, 55, 119, 165, 247, 265, 315
scriptSig, 119, 165, 247
 secp256k1, 29, 111
SendMoney, 119, 165, 315
serialize.h, 63, 99, 111, 189, 193, 225, 277, 419
 SHA-256, 19, 25, 29, 55, 119, 247, 265, 299, 309, 419, 497, 507
Sign, 55, 63, 111, 119, 165, 247, 265, 315
 socket, 103, 109, 207, 225
Solver, 247

T

ThreadBitcoinMiner, 207, 225, 315
 timestamp, 87, 119, 197, 225, 497
 transaction, 63, 75, 87, 119, 165, 207, 225, 247, 265, 277, 299, 309, 315, 363, 373, 407, 419, 437, 497

U

ui.cpp, 189, 193, 315, 437
uint160, 55, 75, 87, 119, 165, 247, 265, 315, 419, 507
uint256, 39, 55, 63, 75, 87, 99, 111, 119, 165, 189, 193, 197, 203, 207, 225, 247, 265, 315, 363, 419, 497, 507
util.cpp, 189, 193, 497
util.h, 99, 189, 193, 507
 UTXO, 87, 119, 165, 265

V

verack, 225
Verify, 63, 111, 119, 165, 197, 225, 247, 265, 299, 315
 version, 515, 55, 63, 75, 87, 119, 165, 207, 225, 243, 247, 277, 299, 309, 315, 373, 407, 419, 437, 507

COLOPHON

About this edition. *Bitcoin v0.01 Alpha — Annotated Edition* presents the first public Bitcoin release (January 2009) as a single volume: original C++ source and headers with inline commentary, introductory primers, and reference material. The line-by-line annotations are designed for continuous reading alongside the code, not as a bare source listing.

Source material. Program text is reproduced from Satoshi Nakamoto's v0.01 Alpha tree. Source code is copyright © 2009 Satoshi Nakamoto and licensed under the MIT/X11 License unless noted in individual files. Commentary, selection, and book design are additional material to that release.

Typography. Body and annotations are set in **Inter**. Code listings, headings, and UI labels use **Basis Grotesque Mono Pro** (with a monospaced fallback stack for print).

Production. This volume was generated from a structured HTML manuscript: syntax highlighting via Prism, pagination via Paged.js, and PDF output via headless Chromium (Puppeteer). The toolchain is Node.js-based and open source.

Intended print specification. Trim 7" × 10" (executive), black text, perfect-bound paperback; body stock typically 60# uncoated white or equivalent; cover per printer profile (e.g. gloss or matte laminate). Exact stock and binding may vary by print vendor.

Credits. Annotations are credited on the title page. Publisher mark: **PirateHash**.

Errata and updates. For corrections or to rebuild this edition from source, use the project that produced this file (version-controlled content, scripts, and styles).